PROGRAMMING THE

LEAR SIEGLER ADM-31



Custom firmware development for the ‘not so dumb’ terminal

WRITTEN BY

MICKEY W. LAWLESS

Contents

Preface **i**
How this book was built . . . i
What you need . . . ii
A word of caution . . . ii

I Getting started **1**

1 The ADM-31 is a computer **2**
The blocks . . . 2
What the firmware does . . . 3
The idea of a plug-in . . . 4
What a program can ignore . . . 4
How to use this book on a real machine . . . 4
Summary . . . 5

2 Bytes, hexadecimal and the way a 6800 counts **6**
Bits and bytes . . . 6
Hexadecimal . . . 6
Counting past the end . . . 7
The registers of the 6800 . . . 8
Characters . . . 8
Two shortcuts used throughout . . . 9
Summary . . . 9

3 The assembler and the listing **10**
Source files . . . 10
The directives . . . 11
Direct and extended addressing, and a quirk of the dialect . . . 11
Reading a listing . . . 12
Where the listings in this book come from . . . 13
A first look at the assembly process . . . 13
Summary . . . 14

4 A tour of the 6800 instruction set **15**
Addressing modes . . . 15
Loading, storing and moving . . . 16
Arithmetic . . . 16
Logic and shifts . . . 18

Branches and jumps . . . 18
Flag instructions . . . 19
The vectors . . . 19
The instructions used in the first programs . . . 20
Worked example: counting down . . . 20
Summary . . . 20

5 The plug-in frame: how the firmware finds and starts your program 21
What the firmware looks for . . . 21
The three ways a plug-in program starts . . . 22
The frame, line by line . . . 22
The program itself . . . 24
How to get back . . . 25
A summary of the frame's contract . . . 25
Running the frame in the emulator . . . 25
Summary . . . 25

II The machine 27

6 The memory map 28
The whole map . . . 28
Working RAM, $0000-$03FF . . . 29
Screen memory, $8000-$8FFF . . . 30
The synchronizing strobe, $7000 . . . 30
The PIA, $7800-$7803 . . . 30
The switches and the serial interfaces . . . 31
The CRT controller, $7F00-$7F0E . . . 31
Rules of the road . . . 33
Summary . . . 33

7 Your first program: HELLO 34
The plan . . . 34
The listing . . . 34
What the bytes look like . . . 36
Running it . . . 36
The same thing with a loop . . . 36
Experiments . . . 38
Summary . . . 38

8 The screen: cells, rows and a drawing library 40
The geometry . . . 40
CALCADR: row and column to address . . . 40
CLRSCR: blanking the screen . . . 42
HLINE: a run of cells . . . 42
PUTAT and PUTSTR: characters and text . . . 42
Program T02FRAME: a border and a caption . . . 43
Program T03CHARS: the whole character set . . . 45
Hardware scrolling, and what a plug-in must do about it . . . 45
Two display pages . . . 47

Summary 47

9 The keyboard **49**
The scan protocol 49
The modifier keys 50
From scan code to character: the keyboard table 50
Program T05SCAN: the explorer 51
Program T06TYPE: a typewriter 52
Reading single keys: how the games do it 54
Summary 54

10 The bell, and what a PIA bit really is **56**
How the firmware rings the bell 56
A different way to pulse: EOR 57
The four bit operations 57
Program T07BELL 58
Timing a tone 58
Summary 59

11 Time: cycles, delay loops and game speed **60**
Cycles 60
A delay loop 61
Program T08SWEEP: a marker you can time 61
How the firmware keeps time 62
Choosing a game speed 63
Timing the drawing itself 64
Summary 64

12 The serial ports and the switches **65**
The interfaces 65
How the firmware initialises them 66
Receiving: the interrupt service routine 67
Transmitting 69
What the plug-ins do about the interfaces 69
Talking to the host from a plug-in: the LINECFG technique 69
The switches 70
Summary 70

III Techniques, and a whole game **72**

13 Subroutines, the stack and a small library **73**
Call and return 73
BSR, JMP and a tail call 78
Passing arguments 78
The library 79
Writing a good subroutine 80
Summary 80

14 Strings and text **82**
How to end a string . . . 82
Counting characters: STRLEN . . . 83
Centring text . . . 83
A table of strings . . . 83
Program T16STRING: copy, convert, compare . . . 85
Other string work . . . 88
Summary . . . 88

15 Numbers: arithmetic, decimal and BCD **89**
Multi-byte arithmetic . . . 89
Multiplication by shift and add . . . 89
Division by shift and subtract . . . 90
Printing a number: the stack reverses it . . . 91
Another way: subtract the powers of ten . . . 92
Packed BCD and DAA . . . 93
Comparing numbers . . . 94
Summary . . . 94

16 Tables and the paddle **96**
Data is just bytes . . . 96
The index register as a pointer . . . 96
A hazard: the index register is shared . . . 97
Moving the paddle and clamping it . . . 98
Drawing five cells . . . 99
Pacing the loop . . . 99
What the program does . . . 100
Summary . . . 100
Exercises . . . 100
Answers to selected exercises . . . 101

17 Sprites: images that move **102**
Design before code . . . 102
The image as data . . . 102
Drawing: two loops and a pointer . . . 103
Erasing is drawing the background . . . 104
The main loop . . . 104
What can go wrong . . . 105
The test . . . 105
Many sprites . . . 105
Summary . . . 105
Exercises . . . 106

18 A bouncing ball: collision by reading the screen **107**
The play box . . . 107
Position, step and the three questions . . . 108
Erase, move, draw . . . 110
Counting ticks . . . 110
How the test proves it . . . 111
Limits of the method . . . 111

Summary . . . 112
Exercises . . . 112

19 Random numbers **113**
The idea . . . 113
The routine . . . 113
Using the output . . . 114
Seeding . . . 116
Testing a random number generator . . . 116
The counters, in passing . . . 116
Summary . . . 117
Exercises . . . 117

20 PONG: the design **118**
The screen . . . 118
The state . . . 119
One tick . . . 120
The computer's player . . . 121
Keys . . . 122
The match . . . 122
Hardware use . . . 122
Why this design . . . 123
Exercises . . . 123

21 PONG: the code, routine by routine **124**
The frame and the entry points . . . 124
RESET and the match loop . . . 126
SERVE . . . 129
BALLMOV: moving the ball and reading the screen . . . 131
PADHIT: the angle, the speed, the computer's mistake . . . 136
POLLHUM and MOVPAD: your bat . . . 138
CPUAI: the computer's bat . . . 141
PAUSE and WAITSP . . . 142
The screen helpers . . . 143
BEEP and the text . . . 147
What to notice . . . 147
Exercises . . . 147

22 PONG: testing it, and what is still unproven **149**
What has been checked . . . 149
What has *not* been checked . . . 150
Running it on the real machine . . . 151
Troubleshooting . . . 151
Knobs . . . 152
What the tests would not catch . . . 152
Exercises . . . 153

23 Borrowing the firmware's routines **154**
Why borrow, and why not . . . 154
Three routines . . . 154
T14FWCALL . . . 156
The two sets are not the same . . . 157
One ROM, both sets: a thunk . . . 157
Rules for borrowing . . . 159
Exercises . . . 159

24 Reading other people's ROMs **160**
A method . . . 160
The frames compared . . . 161
Rubout, closely . . . 162
Shuttle Attack . . . 166
Cosmic Intruders and Space Invaders . . . 167
What to take from the working programs . . . 168
Exercises . . . 168

25 Hooks: sharing the machine with the firmware **169**
The plug-in flag . . . 169
The five times the firmware looks at socket A . . . 169
The summary table . . . 172
Three behaviours . . . 172
Cooperating with the scheduler . . . 174
Rules for a well-behaved plug-in . . . 174
Exercises . . . 174

IV The firmware **175**

26 Power-on: what the firmware does **176**
Reset . . . 176
The RAM test . . . 177
The screen RAM test . . . 178
The ROM test . . . 179
The plug-in probe . . . 180
Initialising the hardware . . . 180
The configuration table . . . 180
The screen and the verdict . . . 181
Entering the scheduler . . . 182
What a plug-in can learn from this . . . 182
Exercises . . . 182

27 The scheduler and the interrupt **184**
The data . . . 184
Setting up . . . 184
The loop . . . 185
Blocking and waking . . . 186
Building one: T21TASKS . . . 187
The interrupt . . . 190

What this means for a plug-in . . . 192
Exercises . . . 192

28 From key to screen: the keyboard task and the escape sequences 193
The road a keystroke takes . . . 193
Task 0: finding a key . . . 194
Translating: KBDTAB and the three modifiers . . . 194
Delivering the character, and auto-repeat . . . 197
Task 1: KEYHAND . . . 198
The shared handler: LEFF6 . . . 200
ESCDISP and the tables . . . 202
What it means for a program . . . 204
Exercises . . . 204

29 The display routines: cursor, scroll, clear and edit 206
The state . . . 206
Row and column to address . . . 207
Cursor addressing: ESC = . . . 208
Moving the cursor . . . 209
Clearing . . . 212
Protected fields . . . 214
Inserting and deleting lines . . . 214
Two pages, and the saved state . . . 218
What a plug-in can take from this . . . 220
Exercises . . . 220

30 Talking to the host: the transmit path, the replies, and the printer 222
A note on the emulator . . . 222
The queue and the routine that fills it . . . 222
Task 7: letting the line go quiet . . . 224
Replies: task 4 . . . 226
Keystrokes to the host, observed . . . 227
The receive side, run . . . 228
The printer port . . . 228
What a plug-in can take from this . . . 228
Exercises . . . 229

31 Writing your own firmware: FW31 230
What it does, and what it leaves out . . . 230
Where it lives . . . 231
Reset . . . 231
The main loop . . . 232
The character machine . . . 232
Control codes . . . 233
Escape sequences . . . 236
The vector table . . . 238
Testing it against the real thing . . . 238
What this proves, and what it does not . . . 239
Going further . . . 239
Exercises . . . 239

V Appendices **241**

A The 6800 opcode table **242**

B The address ledger **245**
Zero-page variables of the 129827 . . . 248
Socket names . . . 248

C ESC and control-code tables **249**

D Scan codes and the keyboard table **252**

E PONG: the complete listing **254**

F The firmware listing: 129867 **279**

G Two plug-ins by other programmers **343**
Rubout . . . 343
Shuttle Attack . . . 356

H The two firmware sets compared **373**
Fixed points . . . 373
Plug-in hooks . . . 373
Routines and tables . . . 374
Zero page . . . 374
What the experiments show . . . 374

I How each program was verified **375**
The matrix . . . 375
What was measured, and what was not . . . 376
The corrections to older notes . . . 377
The emulator . . . 377
Troubleshooting a first run on hardware . . . 377

J Glossary **378**

K The assembler: acpx.c **380**
Getting the source . . . 380
Compiling it . . . 380
Using it for this book’s programs . . . 380
The source . . . 381

L The emulator: adm31emu.c **405**
Getting the source . . . 405
What it emulates . . . 405
Using it . . . 405
What was checked for this book . . . 406
The source . . . 406

Preface

The Lear Siegler ADM-31 is a "dumb terminal" only in the sense that it never became a personal computer. Open the case and you find a complete small computer: a Motorola 6800 microprocessor, 1 KB of working RAM, a few kilobytes of ROM, 4 KB of video RAM that the processor shares with the display circuits, a peripheral interface adapter, two serial ports and a keyboard that the processor scans one key at a time. The firmware that ships in the ROMs turns that computer into a terminal. Nothing stops you from replacing it, extending it, or running something else entirely.

This book teaches you to do that. It starts with what a byte is and ends with a complete game of PONG, a guided tour of the factory firmware, and a minimal firmware of your own. Along the way every program in the book is real: it was assembled, the assembler's listing is printed here as it came out, and the program was run.

How this book was built

Three rules governed the writing.

1. Only verified facts about the machine. Every memory location, port bit and routine address that this book relies on was checked against one of three sources. They are marked in the address ledger (Appendix B) and in the text with these tags:

- **[FW]** the address or behaviour appears in the code of the Lear Siegler firmware (set 129867, and where noted set 129827), read directly from the ROM images.
- **[PGM]** the address is used by a program known to run: the plug-in ROMs *Rubout*, *Shuttle Attack*, *Space Invaders*, *Cosmic Intruders* and *Outbreak*, all of which run on a real ADM-31, and the author's PONG. Rubout, Shuttle Attack, Space Invaders and Cosmic Intruders were written by others; Outbreak, a variant of Rubout, is the author's own.
- **[EMU]** the behaviour was observed when a program ran in the emulator used for this book. An emulator is not the machine. Anything that is only [EMU] is said to be only [EMU].

Where a fact is none of these (for example the processor clock frequency, which this book never measured), the text says so plainly.

2. Only real listings. Every program listing is the output of an assembler listing in the format of the AcPx/acp6800 assembler: address, object bytes, line number, source line, followed by a symbol table and a cross-reference table. The listings of the programs in Parts II to IV, and of the complete firmware in Appendix F, are the output of the author's assembler itself: its C source is printed in Appendix K, and every program in this book was run through it. (The firmware listing was also checked by assembling the factory firmware and comparing every one of the 6,144 bytes with the ROM images.) The two listings in Appendix G are of programs written by

others (Rubout and Shuttle Attack); the author disassembled them, and the assembler printed the listings.

3. Easy first, hard later. Chapters 1 to 5 need nothing but patience. Chapters 6 to 12 introduce the hardware one device at a time with programs of a few dozen lines. Chapters 13 to 19 build techniques (strings, numbers, tables, sprites, collision, random numbers). Chapters 20 to 25 take a whole game apart. Chapters 26 to 31 read the firmware itself and then write some.

What you need

An ADM-31 with a plug-in ROM socket (socket A, addresses $E000-$E7FF) and a way to put a program in it, or an emulator. A cross-assembler: this book uses the AcPx/acp6800 dialect, described in Chapter 3. A text editor. That is all.

A word of caution

The programs in this book were checked in an emulator, and the plug-in ROMs the book draws on (Chapter 24) have all been verified to run on a real ADM-31. PONG itself, the game the book builds, has been run on a real ADM-31 by the author and works there. The tutorial programs and FW31 (Chapter 31) have been checked only in the emulator. Appendix I lists, program by program, what has been verified how.

Part I

Getting started

Chapter 1

The ADM-31 is a computer

If you have only ever used an ADM-31 as a terminal, the first surprise is how little of it is terminal-specific hardware. There is no character generator program and no "terminal chip" that understands escape sequences. Pressing a key does not send a code down a wire until the processor has *decided* to send one. Printing a character does not happen until the processor has *stored* it in memory. Everything that makes the box a terminal is software, and that software lives in ROM.

This chapter tours the machine at the level of blocks and buses. Later chapters give addresses and bit meanings; here the aim is a picture you can hold in your head.

The blocks

```
E800                 57  ; Power-on / reset entry.  The IRQ vector is $ED83;
E800                 58  ; SWI, NMI and RESET all point here (see the vector
E800                 59  ; table at $FFF8).
E800  0F             60  RESET           SEI
E801  B7 7F 0E       61                  STAA    VTACE
E804  B7 7F 0A       62                  STAA    VTACA
E807  CE 7F 00       63                  LDX     #$7F00
E80A  86 65          64                  LDAA    #$65
E80C  A7 00          65                  STAA    $00,X
E80E  86 44          66                  LDAA    #$44
E810  A7 01          67                  STAA    $01,X
E812  86 5D          68                  LDAA    #$5D
E814  A7 02          69                  STAA    $02,X
E816  B6 7E 00       70                  LDAA    SWBNK1
E819  84 02          71                  ANDA    #$02
```

Skip the listing above on a first reading. It is the first instruction the processor executes after power-on, and it is shown here only so that you can see, early, what a *real* listing looks like; Chapter 3 explains every column. What matters now is the idea: when the power comes on, the processor fetches a two-byte address from the top of memory, jumps there, and begins executing the firmware.

The ADM-31's circuitry, as far as a program can see it, is these blocks:

- **The processor**: a Motorola MC6800 (the opcodes used by the factory firmware and by every program in this book are the 6800's; Chapter 4 lists them). It has an 8-bit data bus and a 16-bit address bus, so it can address 65,536 locations, numbered $0000 to

$FFFF. (Hexadecimal numbers are written with a leading dollar sign throughout this book; Chapter 2 explains them.)

- **Working RAM**: 1,024 bytes, at addresses $0000-$03FF [FW]. The factory firmware sets the stack pointer to $03FF, the very top of this RAM, and fills it with test patterns at power-on to check that it works [FW].
- **Screen memory**: 4,096 bytes at $8000-$8FFF [FW]. This is ordinary read/write memory that the display circuits also read, over and over, to produce the picture. One byte holds one character cell. The visible screen is 80 columns by 24 rows, 1,920 cells, starting at $8000; the cell for row *r*, column *c* is at $8000 + 80*r + c [PGM]. The remaining memory is the second display page and spare room (Chapter 8).
- **Program ROM**: in the firmware set 129867, three 2 KB ROMs fill $E800-$FFFF [FW] (6,144 bytes in all); this is the terminal firmware. A fourth socket, **socket A**, decodes to $E000-$E7FF [PGM]. That socket is for *plug-in ROMs*, and it is where every program in this book lives.
- **A parallel interface** (a PIA) at $7800-$7803 [FW]. It is used to scan the keyboard (the processor writes a key number, the hardware reports whether that key is down), to read the SHIFT and CTRL keys, to switch the display page, to drive the alpha-lock bit, and to pulse the bell [FW][PGM].
- **Two serial interfaces** (ACIAs): the host port at $7A00-$7A01 and the printer port at $7C00-$7C01 [FW]. A set of switches, read at $7900, $7D00, $7E00 and $7E80, selects speed and format [FW].
- **The CRT controller**, a block of registers at $7F00-$7F0E that the firmware loads at power-on to set up the display timing, and which holds the cursor position and the first-displayed-line number that gives hardware scrolling [FW]. This book calls it the VTAC.
- **A synchronizing strobe** at $7000: every access to screen memory in the factory firmware, and in every working plug-in ROM examined for this book, is immediately preceded by a write to this address [FW][PGM]. It is not a RAM location. The usual reading is that the strobe synchronizes the processor with the display circuits; this book has not measured what happens without it, and simply obeys the rule everywhere (Chapter 6).

A rough map of the address space therefore looks like this (the full table is Chapter 6):

Range	Contents
$0000-$03FF	working RAM, stack at the top
$7000	CRT synchronizing strobe
$7800-$7803	PIA: keyboard, bell, display page
$7900-$7E80	switches, ACIAs, personality PROM window
$7F00-$7F0E	CRT controller registers
$8000-$8FFF	screen memory
$E000-$E7FF	socket A (plug-in ROM)
$E800-$FFFF	firmware ROMs B, C, D

What the firmware does

When the ADM-31 starts, the firmware, in order [FW]:

1. masks interrupts, loads the CRT controller registers, and sets the stack pointer to $03FF;
2. tests the 1 KB RAM and the screen memory with a pattern ($AA, then its complement), clearing each location as it goes;

3. adds up (exclusive-ORs) all 6,144 bytes of firmware; the last byte of the image is a pad that makes the result come out to zero (this book's own check on the ROM images agrees: the XOR of all 6,144 bytes is $00);
4. **looks at socket A** for a plug-in (Chapter 5 describes the test precisely);
5. sets up the PIA and both ACIAs, copies a table of initial values into RAM, clears the screen, and prints "Test Complete" (or "MEMORY ERROR" if any test failed);
6. enters a small *scheduler* (Chapter 27) that runs the keyboard scanner, the serial receive and transmit code and the escape-sequence interpreter as cooperating tasks.

After step 5 the terminal behaves as a terminal. Everything a host sends is run through the escape-sequence interpreter and drawn into screen memory; everything you type is converted to a character code and queued for transmission.

The idea of a plug-in

Because the firmware looks at socket A during startup, you can put your own program in a ROM there. The factory firmware offers the plug-in two chances. During start-up it calls *into* the plug-in at fixed addresses so that the plug-in may install its own handlers. Later, an operator can press ESC followed by the grave accent character (`, the character with ASCII code $60) to *launch* the plug-in's main program. Chapter 5 shows the exact entry points. Programs written as plug-ins run on top of the firmware, can use its routines, and — with some care — can ignore it altogether and own the whole machine: read the keyboard directly, write to screen memory directly, never return. All the games in this book do the latter, because a game is simpler when the operating system is out of the way.

What a program can ignore

A great deal of the ADM-31's complexity is irrelevant to a plug-in:

- Serial communication. A game does not talk to a host. The only serial requirement is that the game does not leave the ACIAs generating interrupts, which it avoids by never enabling interrupts at all (Chapter 5).
- The scheduler. A game that never returns from its start routine simply never lets the scheduler run again.
- The escape-sequence interpreter. A game draws characters directly.

and a great deal of it is essential:

- The CRT synchronizing strobe. Every working program uses it; this book does too.
- The keyboard scan protocol, because there is no keyboard buffer to rely on once the scheduler has stopped.
- The first-line register and cursor registers, which a plug-in should set to known values at start-up (Chapter 8).

How to use this book on a real machine

You will need some means to turn a binary file into the contents of a ROM in socket A. This book does not prescribe one; the assembler produces a 2,048-byte binary image for $E000-$E7FF in every chapter, and anything that can burn or emulate a 2716-style 2 KB ROM can use it.

Programs begin at $E100 and share a “frame” at $E000-$E0CA described in Chapter 5, which makes them start whether the firmware calls them at boot time or launches them with ESC `.

If you have no hardware, the emulator used for this book runs the same images; but see the caution in the Preface. Where this book says “the screen shows”, it means the emulator’s screen. Where it says “on the machine”, it means the claim is supported by the factory firmware or by a ROM known to run.

Summary

- The ADM-31 is a 6800 computer with 1 KB RAM, 4 KB shared screen memory, 6 KB firmware ROM and one plug-in socket.
- Every terminal feature is firmware. A plug-in may use the firmware or ignore it.
- Addresses and bits in this book are tagged [FW], [PGM] or [EMU]; Appendix B is the full ledger.

Exercises

1. Compute, in decimal, how many bytes of screen memory the 80 x 24 visible screen occupies, and how many bytes remain in $8000-$8FFF.
2. From the table above, list the address ranges that are *not* ordinary memory (reading or writing them operates a device).
3. The firmware ROMs fill $E800-$FFFF. How many 2 KB ROMs is that? Check your answer against the preceding text.

Chapter 2

Bytes, hexadecimal and the way a 6800 counts

A computer stores everything as groups of binary digits. Before any assembly language makes sense you need three habits: reading hexadecimal, thinking of a byte as eight switches, and knowing what happens when a number goes past the end of its range. This chapter builds those habits with small worked examples. You can skip what you know; the exercises at the end are the test.

Bits and bytes

A **bit** is a binary digit, 0 or 1. A **byte** is eight bits. The 6800 moves data eight bits at a time, and every location in memory holds one byte. Bits are numbered from the right, starting at 0:

bit	7	6	5	4	3	2	1	0
weight	128	64	32	16	8	4	2	1

The byte `0100 1000` has bits 6 and 3 set, so its value is 64 + 8 = 72. The ADM-31's keyboard hardware, the PIA control registers and many other devices attach meaning to *individual* bits, so you will constantly say things like "bit 7 of $7800 is set when the key is down" or "pulse bit 3 of $7803 to ring the bell". Both statements are about one switch within a byte.

Bit 7 is called the **most significant bit** (MSB) and bit 0 the **least significant** (LSB). Eight bits can hold 256 different values, 0 to 255.

Hexadecimal

Writing eight binary digits for every byte is tedious, so programmers group them four at a time. Four bits hold sixteen values, 0 to 15, and one *hexadecimal* (hex) digit stands for one of them:

decimal	0	1	2	3	4	5	6	7
hex	0	1	2	3	4	5	6	7
binary	0000	0001	0010	0011	0100	0101	0110	0111

decimal	8	9	10	11	12	13	14	15
hex	8	9	A	B	C	D	E	F
binary	1000	1001	1010	1011	1100	1101	1110	1111

A byte is therefore two hex digits. The byte `0100 1000` is `4` then `8`: $48. In this book (and in the AcPx assembler used for it) a hexadecimal number is written with a leading dollar sign: `$48`, `$7000`, `$E100`. A number with no prefix is decimal. The character `$48` is also the ASCII code for the letter H, a coincidence you will use in Chapter 7.

To convert hex to decimal, multiply each digit by its power of 16 and add: $7F = 7 x 16 + 15 = 127. A 16-bit value such as $E100 has four hex digits, with weights 4096, 256, 16 and 1: 14 x 4096 + 1 x 256 = 57,600. To convert decimal to hex, divide by 16 repeatedly and read the remainders from the bottom up. Practise on a few addresses you will meet: the top of RAM $03FF is 1023, the screen at $8000 is 32,768, and $8000 + 1,920 = $8780.

A 16-bit address occupies two bytes. The **high byte** is the left-hand pair of digits and the **low byte** the right-hand pair; address $E100 has high byte $E1 and low byte $00. The 6800 stores a 16-bit value in memory **high byte first**: the byte at the lower address is the high byte. You will see this when you look at a listing: `JMP $E03C` assembles to `7E E0 3C`, the opcode followed by high byte $E0 and low byte $3C.

Counting past the end

A byte cannot hold 256. Add 1 to $FF and the result is $00, with the extra “carry” bit set in a special register (see below). Likewise, subtract 1 from $00 and you get $FF with a “borrow”. Arithmetic on the 6800 is *modular*: eight-bit results wrap around. This is how loops count down to zero, and it is also the commonest source of bugs.

Unsigned and signed

The same byte can be read two ways. As an **unsigned** number it is 0 to 255. As a **signed** number in *two’s complement* it is -128 to +127, with bit 7 acting as the sign: if bit 7 is set the value is negative. To negate a number, invert all its bits and add 1. For example, +5 is `0000 0101`; invert to `1111 1010` and add 1 for `1111 1011` = $FB = -5. Check: $05 + $FB = $100, which in eight bits is $00.

byte	unsigned	signed
$00	0	0
$01	1	+1
$7F	127	+127
$80	128	-128
$FF	255	-1

The hardware adds both kinds of numbers with the same circuit; only *your interpretation* differs, and the processor gives you separate conditional branches for each interpretation (Chapter 4). The tutorial programs use signed values as direction steps: a ball that moves up has a row step of $FF, which is -1, and adding it to the row makes the row smaller.

The registers of the 6800

The processor's own storage is a small set of registers. A program works by moving data between memory and these registers and operating on them:

register	size	purpose
A	8 bits	accumulator: most arithmetic and logic happens here
B	8 bits	second accumulator
X	16 bits	index register: holds a memory address
SP	16 bits	stack pointer: address of the next free byte on the stack
PC	16 bits	program counter: address of the next instruction
CC	8 bits	condition codes: status flags

There is no combined 16-bit accumulator: A and B are separate, and 16-bit work is done in X or with two 8-bit steps (`ABA` adds B to A, for example).

The condition codes

The CC register holds six flags that record what the last instruction did. Branch instructions test them:

flag	meaning
C (carry)	the last add produced a carry out of bit 7, or the last subtract needed a borrow; also the bit shifted out by a shift or rotate
V (overflow)	the signed result did not fit in 8 bits
Z (zero)	the result was zero
N (negative)	bit 7 of the result is set
I (interrupt mask)	set: interrupts are ignored
H (half carry)	carry out of bit 3, used by the decimal adjust instruction

Two flags matter most in early chapters: Z (did the loop counter reach zero?) and N (is bit 7 set?). The latter is how this book tests the "key down" bit of the keyboard port: loading the port into A sets N from bit 7, and `BMI` ("branch if minus") branches when it is set.

A hazard to remember. Every `LDAA`, `LDAB`, `STAA`, `STAB`, `INX`, `DEX` and most other instructions *change* the flags. If you compare two things and then load a register before you branch, you branch on the load and not on the comparison. Several bugs found while writing this book's programs were exactly that.

Characters

The screen shows the character whose code is stored in a cell. The ADM-31 uses ordinary seven-bit ASCII for the printable range: $20 is a blank, $30-$39 are the digits "0"-"9", $41-$5A the capital letters, $61-$7A the lowercase letters. The firmware clears the screen with the code $20 [FW]; the working game ROMs examined here clear with $00 [PGM], and the display shows an empty cell for both. This book's programs clear with $00, as the games do, and use $20 wherever a program compares a cell with "blank".

The firmware also uses bit 7 of a stored character: the display-attribute mask at zero-page location $19 is ORed into characters the firmware stores [FW]. This book's plug-ins store plain seven-bit characters and never set bit 7.

Two shortcuts used throughout

Shifts as multiplication. Shifting a number left one bit multiplies it by 2; shifting right divides it by 2 (discarding the remainder). The screen needs row x 80, and 80 = 64 + 16: so row x 80 = (row x 64) + (row x 16), and each of those is a series of left shifts. Chapter 8 builds the routine.

Masks. To keep only the low four bits of a byte, AND it with $0F. To set bit 3, OR with $08. To flip bit 3, EOR with $08. To test bit 6, AND with $40 and look at the zero flag. These four operations cover nearly all the bit-twiddling in this book.

Summary

- A byte is eight bits, written as two hex digits, with $ in front.
- 16-bit values are stored high byte first.
- Eight-bit arithmetic wraps; two's complement lets one adder serve signed and unsigned numbers.
- A, B, X, SP, PC and CC are the registers; Z, N, C, V are the flags you will use first.
- Loads and stores change the flags.

Exercises

1. Convert $A7 to binary and to decimal (unsigned and signed).
2. What is $7F + 1 as unsigned? As signed? Which flag distinguishes "wrapped" in each reading?
3. Write the 16-bit address of screen row 5, column 10 in hex (cell = $8000 + 80 x row + column).
4. What mask clears bit 7 of a byte? Which instruction applies it?
5. Why does `JMP $E03C` appear as `7E E0 3C` in a listing and not `7E 3C E0`?

Answers. (1) 1010 0111; 167; -89. (2) $80; 128 unsigned, -128 signed — the V flag marks the signed overflow, and C is *not* set for the signed case here. (3) 5 x 80 + 10 = 410 = $19A, so $819A. (4) $7F; ANDA #$7F. (5) The 6800 stores the high byte first.

Chapter 3

The assembler and the listing

A processor runs *machine code*, bytes such as `86 48`. People write *assembly language*, text such as `LDAA #$48`, and an **assembler** translates one into the other. The assembler also prints a **listing** that shows, side by side, each source line and the bytes it produced. The listing is the programmer's best instrument: with it you can check by eye what the machine will actually run, find out where a routine landed, and see which lines refer to which symbols.

This book uses the AcPx/acp6800 dialect throughout. This chapter explains the source format, the directives, the quirks you must respect and, most important, how to read a listing. Every listing printed in later chapters has exactly the layout described here.

Source files

An assembly source file is plain text, one statement per line. A statement has up to four fields, separated by spaces:

```
label           mnemonic operand        ; comment
```

- The **label** (optional) starts in column 1 and names the address that the line occupies. Labels in this book are upper case, with letters, digits and underscores, and the listing's symbol table keeps the first eight characters.
- The **mnemonic** is the instruction (`LDAA`) or an assembler *directive* (`ORG`).
- The **operand** says what the instruction works on.
- The **comment** starts with a semicolon and runs to the end of the line. A line whose first character is a semicolon is a comment line.

The book's layout puts labels at column 1, the mnemonic at column 17 and the operand at column 25. That is purely a matter of style; the assembler cares only that a label starts in column 1 and anything that is not a label does not.

Numbers and characters in operands

written	meaning
`$48`	hexadecimal 48
`72`	decimal 72
`#$48`	the *value* $48 (immediate), not the contents of address $48
`0,X`	the address X + 0 (indexed)

written	meaning
`LABEL`	the address (or value) of a label
`CRTRAM+1920`	a label plus a constant

Two quirks matter in practice.

No operator precedence. An expression such as `CRTRAM+SCRW*SCRH` is *not* evaluated the way a calculator would; the AcPx assembler has no multiply and no precedence rules. Compute such constants by hand and write them as a number, with a comment saying where the number came from (for example `$8780  ; CRTRAM + 80*24`). A single `+` or `-` between a label and a constant, as in `CRTRAM+1920`, is safe and used throughout.

Character literals. Avoid writing a quoted character as an operand (for example `#'H'`); write the hexadecimal code instead and say the character in the comment. Quoted *strings* are fine inside `FCB` and `FCC` lines, as described next.

The directives

Directives are instructions to the assembler, not to the processor. Those used in this book:

directive	what it does
`ORG addr`	set the address at which the following bytes will sit
`label EQU value`	give a name to a number (produces no bytes)
`FCB b,b,...`	place one byte per operand; a quoted string places its characters
`FDB w,w,...`	place one 16-bit word per operand, high byte first
`FCC "text"`	place the characters of the string
`END`	end of source

(The author's own sources also use `RMB`, which reserves bytes without initialising them; this book's programs keep their variables as `EQU`s at fixed RAM addresses instead.)

`ORG` is the one to understand. A plug-in ROM image covers $E000-$E7FF. The binary the assembler writes runs from the lowest to the highest address the source defines, bytes in gaps inside that range are $00, and (with the padding question answered 2048) a shorter image is topped up with $FF to exactly 2,048 bytes; the programs in this book all define their $E000-$E7FF window completely. When a program says `ORG $E100`, the first instruction after it goes at address $E100, which is 256 bytes into the image. A later `ORG $E03C` jumps *backwards* to fill in a section that was skipped; that is how the frame at the top of the ROM, with its small fixed pieces at $E000, $E002, $E03C, $E07D and so on, is built (Chapter 5).

Direct and extended addressing, and a quirk of the dialect

The 6800 can name a memory location in two sizes. The **direct** form uses a one-byte address and reaches only $0000-$00FF; the **extended** form uses two bytes and reaches everything. The assembler picks the shorter form when it *already knows* the address is below $0100. When it meets a name it has not seen yet (a forward reference), it assumes the long form. Some instructions have no direct form at all (`CLR`, `INC`, `DEC`, `ASL`, `ROL`, `TST` on memory); they are always extended.

This matters in one place in this book. The plug-in frame needs `CLR $0000`, which can only be extended: `7F 00 00`. The author's convention, adopted here, is to keep such cases visible. Where the assembled form has to differ from what the instruction would normally pick (for instance a three-byte `LDAA $0016` inside firmware), a line is emitted as three raw bytes with the intended instruction in the comment. You will see this in the firmware listing in Appendix F: `FCB $B6,$00,$4D ; LDAA $004D` is the extended encoding of an instruction that the assembler would otherwise turn into the shorter `96 4D`. The listing remains byte-exact either way, and that exactness is the whole point of the exercise.

Reading a listing

Here is a short piece of a real listing, from the first program in the book (Chapter 7):

```
E100  0F              48  START           SEI               ; no interrupts
E101  8E 03 FF        50                  LDS     #$03FF
E104  CE 83 8E        52                  LDX     #CRTRAM+910
E107  86 48           53                  LDAA    #$48      ; 'H'
E109  B7 70 00        54                  STAA    HALT      ; sync with the CRT
E10C  01              55                  NOP               ; settle
E10D  A7 00           56                  STAA    0,X       ; put 'H' on screen
E10F  08              57                  INX               ; next cell
E110  86 45           58                  LDAA    #$45      ; 'E'
E112  B7 70 00        59                  STAA    HALT
E115  01              60                  NOP
E116  A7 00           61                  STAA    0,X
```

Each line has four parts:

- **ADDR**: the address at which the line's bytes are placed. Comment lines and `EQU` lines show the current address (or 0000 before the first `ORG`).
- **CODE**: the bytes the assembler produced, in hex, up to four on a line. `86 48` is `LDAA #$48`: `86` is the opcode for "load A, immediate" and `48` is the value. `B7 70 00` is `STAA HALT`: `B7` is "store A, extended" followed by the high and low bytes of $7000. An `EQU` line shows the value it defines instead, as `=7000`.
- **LINE**: a line number. For historical reasons the listing's numbers are one more than the line's position in the source file: the first source line is shown as line 2. This book keeps that convention so that its listings match the author's own.
- **SOURCE STATEMENT**: the source line as written. The listing field is narrow; a source line longer than 53 characters is cut off and the last character is replaced by `>`. That is why long comments in this book's programs sit on their own lines, above the instruction they describe.

Notice that addresses advance by the number of bytes: `LDS #$03FF` at $E101 takes three bytes, so the next instruction is at $E104. You can add them up yourself, and you should, early on: if the next ADDR is not the previous plus its byte count, something (an `ORG`, usually) intervened.

The symbol table and cross-reference

After the code, the listing prints two tables. The **symbol table** lists every name and its value, in the order the names were defined, five to a row:

```
SYMBOL TABLE
  HALT     7000  PIA1     7800  PIA1B    7802  PIA1CRB  7803  CRTRAM   8000
  CRTC6    7F06  CURCOL   7F0C  CH_PAD   0023  CH_WALL  003D  CH_NET   003A
```

```
  CH_BALL  004F  WINSC    0007  DLYSTART 1900  HPAD     0100  CPAD     0102
  BROW     0104  BCOL     0105  BHDIR    0106  BVDIR    0107  BVRATE   0108
  BVCNT    0109  CURADR   010A  OLDADR   010C  OLDCOL   010E  SCOREH   010F
  SCOREC   0110  TICK     0111  AIMOFF   0112  POINT    0113  LOSER    0114
  DLYCNT   0115  SEED     0117  TRIES    0119  ROWARG   011A  COLARG   011B
```

Names are cut to eight characters here (`KEYENTRY` fits exactly, longer names would be truncated). The symbol table is where you read off where each routine ended up: `RESET E100`, `GAME E14E`.

The **cross-reference table** lists the same symbols alphabetically, and for each says its type (`EQU` for a defined constant, `LAB` for a label), its value, the line on which it was defined (`DEF`), how many times it is used (`CNT`) and the lines that use it:

```
CROSS-REFERENCE TABLE
SYMBOL          TYP  VAL   DEF     CNT  REFERENCES
--------------  ---  ----  -----  ----  -----------------------------------
ADRPTR          EQU  011C  L116     25  L1089 L1092 L1094 L1096 L1097 L1099
                                       L1100 L1102 L1103 L1104 L1106 L1108
                                       L1112 L1114 L1115 L1116 L1120 L1122
                                       L1126 L1127 L1130 L1132 L1133 L1138
                                       L1140
AIMOFF          EQU  0112  L101      3  L236  L680  L810
BALLCOL         LAB  E2AE  L542      2  L430  L472
BALLMOV         LAB  E212  L417      1  L275
BALLROW         LAB  E2BB  L557      1  L432
BCOL            EQU  0105  L80       9  L358  L386  L422  L436  L476  L548
                                       L551  L591  L655
```

A reference count of 0, like a variable that is defined but never used, is worth a look: it is either a leftover or a bug. The last lines of a listing give the error count; a clean assembly ends with `TOTAL ERRORS: 0` and `STATUS: ASSEMBLY SUCCESSFUL - NO ERRORS`.

Where the listings in this book come from

Every listing in this book is assembler output in the layout above. The sources in Parts II to IV were assembled with the author's AcPx assembler (its source is in Appendix K; the emulator used to try the programs is in Appendix L); Appendix F is the firmware, regenerated from the ROM images and verified byte for byte, and Appendix G is the disassembly listings of two plug-in games written by others (Rubout and Shuttle Attack), as printed by the assembler. For every program the binary was also cross-checked against a second, independent assembler written for this book: the two agree byte for byte.

One thing about source *macros*: the tutorial sources are assembled with a small pre-processor that inserts the plug-in frame and a few library routines into the source before assembly. The pre-processor is a convenience of the book's build; the assembler never sees it. The listings show the *expanded* source, which is exactly what the assembler assembled. When a listing in a later chapter shows a standard routine (say `CALCADR`), you will find that routine's explanation in the chapter that introduces it, and it is the same text in every program that uses it.

A first look at the assembly process

The sequence for every program is the same:

1. write or edit the `.ASM` source;

2. assemble it: the assembler reads the source and writes the binary image (`.BIN`) and the listing (`.LST`);
3. read the listing: check that the error count is zero, that the addresses are what you intended, and that every `ORG` region is where it belongs;
4. put the binary into socket A, or load it into the emulator;
5. run, observe, and, when something is wrong, return to the listing.

Keep the habit of reading the listing even when the program works. Bugs in assembly programs are very often visible in the listing before they are visible on the screen: a branch that goes to the wrong label, a constant that is not what you meant, a forward reference that came out as a two-byte address, or a code region that has run into data.

Summary

- Source has label, mnemonic, operand and comment fields; labels start in column 1.
- Hex is written with `$`; `#` means “the value itself”.
- There is no operator precedence; compute constants by hand.
- `ORG` places code; the ROM image is 2,048 bytes of $E000-$E7FF filled with $00.
- A listing shows address, bytes, line number and source; a symbol table and a cross-reference follow.

Exercises

1. In the T01HELLO listing above, the instruction at $E104 is three bytes long. At what address does the following instruction start?
2. What does `CE 83 8E` mean? (Opcode `CE` is “load X, immediate”.) What screen cell does $838E correspond to? (Use $8000 + 80 x row + column.)
3. Which line of the cross-reference tells you the number of times `HALT` is used in a program? Find it in the listing of your first program once you have assembled it.
4. Why do long comments appear above, rather than beside, the code in this book’s listings?

Answers. (1) $E107. (2) Load X with $838E; $838E - $8000 = 910 = 11 x 80 + 30, so row 11, column 30. (3) The `HALT` row of the cross-reference table, column `CNT`. (4) The listing’s source field is 53 characters wide and longer lines are cut off.

Chapter 4

A tour of the 6800 instruction set

The 6800 has 197 valid opcodes (Appendix A lists every one). That sounds like a lot, but they are 72 operations used in a handful of addressing modes, and a working program needs a few dozen. This chapter introduces them by job rather than alphabetically, and gives each group a small worked example. The aim is fluency, not completeness: after this chapter you should be able to read any listing in the book and say what each line does.

Addressing modes

An instruction's *mode* says where its operand comes from.

mode	written	operand	example	bytes
inherent	`INX`	none, or a register implied by the opcode	`08`	1
immediate	`LDAA #$48`	the byte (or word) in the instruction	`86 48`	2 (3 for 16-bit)
direct	`LDAA $4D`	the byte at address $004D	`96 4D`	2
extended	`LDAA $7802`	the byte at a 16-bit address	`B6 78 02`	3
indexed	`LDAA 5,X`	the byte at X + 5 (offset 0-255)	`A6 05`	2
relative	`BNE LOOP`	a branch target within -126 to +129 bytes	`26 F9`	2

The same operation has a different opcode in each mode: `LDAA` is $86 (immediate), $96 (direct), $A6 (indexed), $B6 (extended). The assembler picks the opcode from the operand's form.

Indexed addressing deserves a moment. The index register X holds an address and the offset is *added* to it, but X itself does not change. `LDAA 0,X` loads the byte at the address in X. That is how a program walks across a string or the screen: load X with the first cell, then `STAA 0,X`, `INX`, `STAA 0,X`, and so on. The offset is an unsigned byte, 0 to 255; it cannot be negative.

Relative addressing is used only by branches. The byte after the opcode is a signed displacement added to the address *after* the branch instruction. The range is about 128 bytes backward and 127 forward; if the target is farther away, the assembler reports an error and you use a `JMP` around it (an example appears in Chapter 21).

Loading, storing and moving

instruction	does
`LDAA LDAB`	load A or B from memory or a constant
`STAA STAB`	store A or B into memory
`LDX LDS`	load X or SP (16 bits)
`STX STS`	store X or SP
`TAB TBA`	copy A to B, or B to A
`TAP TPA`	copy A to the condition codes, or back
`TSX TXS`	X = SP + 1, or SP = X - 1
`PSHA PSHB`	push A or B on the stack
`PULA PULB`	pull A or B from the stack
`INX DEX`	add 1 to X or subtract 1
`INS DES`	add 1 to SP or subtract 1
`CLRA CLRB CLR`	set A, B or a memory byte to zero

All loads and stores set N and Z from the value moved and clear V; they leave C alone.

There is **no memory-to-memory move**. To copy a byte you load it into A and store it elsewhere. Several lines of tutorial code are that pattern. And there is no instruction to load X from A: build a 16-bit value in two memory bytes and `LDX` it, or increment X by a count with `INX` in a loop. (The firmware keeps a routine for X = X + A at $EFAD, which Chapter 23 uses.)

The stack

The stack is a region of RAM that the processor itself uses for return addresses (and that you may use for temporary values). The stack pointer SP holds the address of the next *free* byte; a push stores a byte at SP and then decrements SP; a pull increments SP and then reads. The stack grows downward from the top of RAM, so you set SP once at start-up: `LDS #$03FF`. Both the factory firmware [FW] and every working plug-in [PGM] do exactly that.

A pull must match each push. Forgetting one is the classic way to crash: `RTS` pulls two bytes as the return address, so an unbalanced `PSHA` inside a subroutine makes the `RTS` return to a nonsense address.

Arithmetic

instruction	does
`ADDA ADDB`	A = A + operand
`ADCA ADCB`	A = A + operand + C (add with carry)
`ABA`	A = A + B
`SUBA SUBB`	A = A - operand
`SBCA SBCB`	A = A - operand - C (subtract with borrow)
`SBA`	A = A - B
`NEGA NEGB NEG`	negate (0 - value)
`INCA INCB INC`	add 1 (does not touch C)
`DECA DECB DEC`	subtract 1 (does not touch C)

instruction	does
`DAA`	decimal adjust A after a BCD addition
`CMPA CMPB CBA`	compare: set flags as if subtracted, discard the result
`CPX`	compare X with a 16-bit operand
`TSTA TSTB TST`	set N and Z from the value, clear V and C

Multi-byte arithmetic uses the carry: add the low bytes with `ADDA`, then the high bytes with `ADCA` so that the carry from the low addition is included. Chapter 8's address routine is a worked example:

```
E1AA                 135  ; CALCADR - X = CRTRAM + ROWARG*80 + COLARG
E1AA                 136  ; Uses 80 = 64 + 16: shift row left 4 (x16),
E1AA                 137  ; keep it, shift 2 more (x64), add the two.
E1AA  7F 01 02       138  CALCADR         CLR     ADRPTR    ; high byte = 0
E1AD  B6 01 00       139                  LDAA    ROWARG
E1B0  B7 01 03       140                  STAA    ADRPTR+1  ; value = row
E1B3  78 01 03       141                  ASL     ADRPTR+1  ; x2
E1B6  79 01 02       142                  ROL     ADRPTR
E1B9  78 01 03       143                  ASL     ADRPTR+1  ; x4
E1BC  79 01 02       144                  ROL     ADRPTR
E1BF  78 01 03       145                  ASL     ADRPTR+1  ; x8
E1C2  79 01 02       146                  ROL     ADRPTR
E1C5  78 01 03       147                  ASL     ADRPTR+1  ; x16
E1C8  79 01 02       148                  ROL     ADRPTR
E1CB  B6 01 02       149                  LDAA    ADRPTR    ; A:B = row*16
E1CE  F6 01 03       150                  LDAB    ADRPTR+1
E1D1  36             151                  PSHA              ; save row*16
E1D2  37             152                  PSHB
E1D3  78 01 03       153                  ASL     ADRPTR+1  ; x32
E1D6  79 01 02       154                  ROL     ADRPTR
E1D9  78 01 03       155                  ASL     ADRPTR+1  ; x64
E1DC  79 01 02       156                  ROL     ADRPTR
E1DF  33             157                  PULB              ; B = low(row*16)
E1E0  FB 01 03       158                  ADDB    ADRPTR+1  ; + low(row*64)
E1E3  F7 01 03       159                  STAB    ADRPTR+1
E1E6  32             160                  PULA              ; A = high(row*16)
E1E7  B9 01 02       161                  ADCA    ADRPTR    ; + high(row*64)
E1EA  B7 01 02       162                  STAA    ADRPTR    ; row*80
E1ED  B6 01 01       163                  LDAA    COLARG
E1F0  BB 01 03       164                  ADDA    ADRPTR+1  ; + column
E1F3  B7 01 03       165                  STAA    ADRPTR+1
E1F6  B6 01 02       166                  LDAA    ADRPTR
E1F9  89 00          167                  ADCA    #0
E1FB  8B 80          168                  ADDA    #$80      ; + $8000
E1FD  B7 01 02       169                  STAA    ADRPTR
E200  FE 01 02       170                  LDX     ADRPTR    ; X = address
E203  39             171                  RTS
E204                 172  ; PUTAT - write A at (ROWARG,COLARG)
E204                 173  ; save the character
```

The routine computes `row x 80 + column + $8000`, using only shifts, a push and pull and one add-with-carry. Every step is a 16-bit operation done as two 8-bit steps.

CMP and flags. `CMPA #$58` subtracts $58 from A *without storing the answer*, only setting the flags. Then `BEQ` branches if A was $58, and the other conditional branches test greater or

less. `INC` and `DEC` do not alter the carry, which is why you can run a counter in the middle of a multi-byte add without disturbing it.

A word on CPX. On the 6800, `CPX` sets Z, N and V but *not* C. After a `CPX` use `BEQ` or `BNE` (the loops that run until an address is reached), or the signed branches; do not use the unsigned branches `BHI`, `BLS`, `BCC`, `BCS` after it.

Decimal. `DAA` corrects the result of adding two packed BCD bytes (two decimal digits per byte): $19 + $01 = $1A as binary; `DAA` turns it into $20, the BCD answer. Chapter 15's scoring routine uses it.

Logic and shifts

instruction	does
`ANDA ANDB`	A = A AND operand (clears bits)
`ORAA ORAB`	A = A OR operand (sets bits)
`EORA EORB`	A = A XOR operand (flips bits)
`COMA COMB COM`	invert all bits
`BITA BITB`	AND without storing: test bits, set N and Z
`ASLA ASLB ASL`	shift left; bit 7 goes to C, bit 0 gets 0
`LSRA LSRB LSR`	shift right; bit 0 goes to C, bit 7 gets 0
`ASRA ASRB ASR`	shift right keeping bit 7 (signed halve)
`ROLA ROLB ROL`	rotate left through C
`RORA RORB ROR`	rotate right through C

Shifts and rotates together move bits *between bytes*: shift the low byte of a 16-bit value left with `ASL`, then `ROL` the high byte to catch the bit that fell out. That is exactly what the address routine above does.

`BITA #$40` is how a program tests one bit: it computes A AND $40 into the flags and leaves A unchanged. The firmware does this when it tests whether bit 6 of the keyboard port (the CTRL key) is set.

Branches and jumps

Branches test the flags set by the previous instruction. The branch is taken only if the condition holds.

instruction	taken when	use
`BRA`	always	short jump
`BEQ BNE`	Z = 1 / Z = 0	equal, not equal; zero, not zero
`BMI BPL`	N = 1 / N = 0	bit 7 set / clear
`BCS BCC`	C = 1 / C = 0	unsigned lower / unsigned higher-or-same
`BHI BLS`	C or Z: unsigned higher / lower-or-same	after `CMPA` of unsigned values
`BGT BGE BLT BLE`	signed greater / greater-equal / less / less-equal	after `CMPA` of signed values

instruction	taken when	use
BVS BVC	V = 1 / V = 0	signed overflow
BSR	always, saves return address	short subroutine call

Choose the signed or unsigned family according to how you *interpret* the numbers. Screen positions, character codes and scan codes are unsigned (0-255); direction steps ($FF = -1) are signed.

instruction	does
JMP addr	go to the address (extended or indexed)
JSR addr	push the return address, then go there
RTS	pull the return address and go there
RTI	return from interrupt (pulls CC, B, A, X, PC)
SWI	software interrupt (vector $FFFA)
WAI	wait for an interrupt
NOP	do nothing for one cycle

A **subroutine** is code that other code calls with JSR (or BSR) and which ends with RTS. The calling program continues after the JSR. Because the return address is on the stack, subroutines can call other subroutines to any depth that the stack can hold. Chapter 13 builds a small library this way.

Flag instructions

SEC and CLC set and clear C. SEI and CLI set and clear the interrupt mask. SEV and CLV set and clear V. Every program in this book begins with SEI (interrupts off) because the games poll the hardware directly, and the unit's two serial interfaces can raise interrupts that would otherwise run into the firmware's handler while the firmware's scheduler is no longer running.

The vectors

The processor's four special addresses live at the top of memory, two bytes each, high byte first. In the 129867 firmware they are, from the vector table at $FFF8 [FW]:

address	vector	in this firmware
$FFF8	IRQ (maskable interrupt)	$ED83
$FFFA	SWI	$E800
$FFFC	NMI	$E800
$FFFE	RESET	$E800

On power-up and reset the processor loads PC from $FFFE and starts executing at $E800. The IRQ vector points at the firmware's interrupt service routine (Chapter 27). SWI and NMI both simply restart the firmware.

The 129827 firmware's table is different [FW]: IRQ $FA87, SWI $E900, NMI $FA87, RESET $E900. Its reset entry is at $E900, not $E800, and its NMI shares the interrupt routine. A program that must work on both sets never hard-codes $E800: it reads the RESET vector at $FFFE.

The instructions used in the first programs

This is the set the early tutorial programs use. Learn these, and the rest of the table will follow naturally:

- `LDAA #n`, `STAA addr`, `LDAA addr`, `LDX #addr`, `INX`, `DEX`, `BNE`, `BEQ`, `BRA`, `JSR`, `RTS`, `CMPA #n`, `ADDA #n`, `PSHA`, `PULA`, `SEI`, `LDS #addr`, `CLR addr`, `ANDA #n`, `ORAA #n`, `EORA #n`, `ASL addr`, `ROL addr`.

Worked example: counting down

A loop that does something ten times:

```
        LDAB    #10
LOOP    (the work goes here)
        DECB
        BNE     LOOP
```

`DECB` sets Z when B reaches zero; `BNE` branches back while Z is 0. The body runs ten times (B = 10, 9, ..., 1). If the body also contains instructions that change the flags, that is fine: the flags that matter are the ones set by `DECB`, which is the last instruction before the branch. If you put the `DECB` earlier and then use `LDAA`, the branch tests the load: the classic hazard from Chapter 2.

Summary

- Six addressing modes; the same operation has a different opcode per mode.
- Load, store, arithmetic, logic, shift, branch, jump, flag instructions.
- The stack grows downward from $03FF; every push has a pull.
- Use signed or unsigned branches to match how you read the numbers; after `CPX` use only `BEQ`/`BNE`.
- RESET starts at the address in $FFFE; IRQ goes through $FFF8.

Exercises

1. What are the bytes of `LDAB #$FF`? (Hint: Appendix A.)
2. A loop `LOOP INX / CPX #$8780 / BNE LOOP` starts with X = $8000. How many times does `INX` execute?
3. Write the instructions to compute $80 + $90 as 16 bits, leaving the high byte in $0100 and the low byte in $0101.
4. Why does `CPX` followed by `BHI` fail on the 6800?

Answers. (1) `C6 FF`. (2) 1,920 times (to $8780, $780 = 1,920 steps). (3) `LDAA #$80 / ADDA #$90 / STAA $0101 / CLRA / ADCA #0 / STAA $0100`. (4) `CPX` does not set the carry flag, which `BHI` depends on.

Chapter 5

The plug-in frame: how the firmware finds and starts your program

Every program in this book is a plug-in for socket A, and every one begins with the same few lines at the same few addresses. This chapter explains why. It is the most important chapter in Part I, because the frame is the contract between your program and the factory firmware: get it wrong and the terminal never runs your code, or never recovers afterwards.

What the firmware looks for

During start-up, after the memory tests, the 129867 firmware does the following. This is the real code, from the firmware listing in Appendix F:

```
E8BB                 152  ; Plug-in probe: is there a ROM in socket A that
E8BB                 153  ; starts "TJ"?
E8BB  FE E0 00       154  PLUGPROB        LDX     $E000
E8BE  8C 54 4A       155                  CPX     #$544A
E8C1  26 0D          156                  BNE     NOPLUG
E8C3  B6 7D 00       157                  LDAA    SWBNK3
E8C6  2B 08          158                  BMI     NOPLUG
E8C8  7C 00 00       159                  FCB     $7C,$00,$00 ; INC $0000
E8CB  BD E0 7D       160                  JSR     $E07D
E8CE  20 04          161                  BRA     PIAINIT
E8D0                 162  ; No usable plug-in: continue the normal boot.
E8D0  86 1D          163  NOPLUG          LDAA    #$1D
E8D2  97 DB          164                  STAA    $DB
```

Read it line by line:

1. `LDX $E000` loads X with the two bytes at $E000-$E001. `CPX #$544A` compares them with $544A. The bytes $54, $4A are the ASCII letters **T** and **J**. If socket A is empty, or holds something that does not begin with “TJ”, the branch to `NOPLUG` is taken and the plug-in is ignored. The “TJ” signature is how the firmware recognises a plug-in ROM [FW].
2. `LDAA SWBNK3` reads the switch bank at $7D00 and `BMI NOPLUG` skips the plug-in if bit 7 of it is set. A switch can therefore disable the plug-in even when one is installed [FW].
3. `FCB $7C,$00,$00` is `INC $0000`, which adds one to the byte at address $0000. That byte is the **plug-in flag**: it is zero when no plug-in is active and non-zero when the firmware has decided to run one. (The assembler cannot write `INC $0000` in the direct form because

`INC` has no direct form; the extended instruction is emitted as raw bytes by this book's convention, see Chapter 3.)

4. `JSR $E07D` calls address $E07D in socket A: the **boot hook**. The firmware then continues to `PIAINIT` to set up the keyboard interface and the rest of the hardware.

In the 129827 firmware (an earlier set) the sequence is the same, except that the hook is at $E087 and the skip path is at $E9D0. The two sets are compared in Appendix H.

If you look at the signature bytes of the working plug-in ROMs they all begin with $54, $4A [PGM].

The three ways a plug-in program starts

Our frame lets the same program start in three ways.

1. ESC ` (grave). After the terminal has booted, an operator types ESC followed by the grave accent, the character with ASCII code $60. The firmware's escape-sequence interpreter has an entry for $60, and that handler is the second routine in the plug-in probe:

```
F22C                1140  ; ESC ` : if socket A holds "TJ" jump to its start
F22C                1141  ; ($E03C).
F22C  FE E0 00      1142  ESCGRAVE        LDX     $E000
F22F  8C 54 4A      1143                  CPX     #$544A
F232  26 F7         1144                  BNE     LF22B
F234  7E E0 3C      1145                  JMP     $E03C
```

It repeats the "TJ" test and then jumps to **$E03C** [FW]. In the 129827 set the jump goes to **$E03E** instead [FW]. The frame handles both with three `NOP` instructions at $E03C-$E03E followed by a `JMP` to the program. Entering at $E03C slides through all three; entering at $E03E executes one and then falls through to the `JMP`.

(A note on keys, to avoid a trap. According to the firmware's own keyboard table, the grave accent is the *shifted* character of the key that types `@` (scan code $3B), and the apostrophe is the shifted character of the key that types 7 (scan code $14); Appendix D has the full table. The firmware has an escape-sequence entry for the apostrophe too, code $27, but that entry is an unrelated command: it clears the protect flag ("protect off"). To launch a plug-in you need the grave accent, code $60, so the sequence is ESC, then SHIFT with the `@` key. Chapter 28 lists the whole escape table.)

2. The boot hook. During power-on the firmware calls $E07D (129867) or $E087 (129827). The frame's hook *declines* the call and lets the normal boot continue (see below). A plug-in that wants to take over at power-on, as some of the working games do, uses this hook for its own purposes (Chapter 25).

3. Direct entry at $E002. The frame places `JMP $E03C` at $E002 so that an environment that starts a plug-in ROM without a firmware (a ROM-only emulator that begins at the first instruction) can start the game. On a real terminal this entry is never used.

The frame, line by line

This is the frame as it appears in the first program of the book, together with the tutorial constants around it:

```
E000  54 4A           20                  FCB     $54,$4A   ; "TJ" signature
E002  7E E0 3C        23  PLUGSTART       JMP     $E03C
```

```
E03C  01              26  ENTRY           NOP
E03D  01              27                  NOP
E03E  01              28                  NOP
E03F  7E E1 00        29                  JMP     START
E07D  7F 00 00        32  BOOT67          CLR     $0000
E080  32              33                  PULA
E081  32              34                  PULA
E082  7E E8 D0        35                  JMP     $E8D0
E087  7F 00 00        37  BOOT27          CLR     $0000
E08A  32              38                  PULA
E08B  32              39                  PULA
E08C  7E E9 D0        40                  JMP     $E9D0
E094  39              42                  RTS
E0C3  39              44                  RTS
E0CA  39              46                  RTS
```

- `$E000: 54 4A` the signature.
- `$E002: JMP $E03C` the direct entry.
- `$E03C-$E03E: NOP NOP NOP`, then `JMP START`. The three no-ops let one frame serve both firmware sets.
- `$E07D: CLR $0000 / PULA / PULA / JMP $E8D0` the 129867 boot hook.
- `$E087: CLR $0000 / PULA / PULA / JMP $E9D0` the 129827 boot hook.
- `$E094`, `$E0C3`, `$E0CA`: each holds a single `RTS`.

Every ROM image is 2,048 bytes, with the unused space filled with $00. The program itself, in the tutorials, begins at `ORG $E100`.

Why the hook looks the way it does

The hook is entered by a `JSR`, so the stack holds a two-byte return address. The hook does three things:

- `CLR $0000` zeroes the plug-in flag the firmware has just incremented. With the flag at zero the firmware behaves exactly as it would with no plug-in. (A non-zero flag changes the scheduler start-up, below.)
- `PULA`, `PULA` discard the two bytes of return address, so the stack is balanced.
- `JMP $E8D0` jumps to the firmware's "no plug-in" continuation, which is the very next step of the normal boot (`NOPLUG` in the listing above). The firmware proceeds as if the plug-in were not there.

The net effect: the terminal boots normally to "Test Complete", and the plug-in sits waiting in its socket until the operator presses ESC `. This is the behaviour of the Space Invaders ROM's own hook, which this book adopted as a convention [PGM].

What the flag changes

Why bother clearing it? The scheduler start-up in the firmware looks at the flag:

```
EBF8  CE 00 85        368  LEBF8           LDX     #$0085
EBFB  86 80           369                  LDAA    #$80
EBFD  C6 08           370                  LDAB    #$08
EBFF  8D 4B           371                  BSR     LEC4C
EC01  CE 00 8D        372                  LDX     #$008D
EC04  C6 08           373                  LDAB    #$08
EC06  4F              374                  CLRA
EC07  8D 43           375                  BSR     LEC4C
```

```
EC09  CE EE 06       376                LDX     #KBSCAN
EC0C  DF 65          377                STX     $65
EC0E  CE FD 27       378                LDX     #TXDELAY
EC11  DF 73          379                STX     $73
EC13  96 00          380                LDAA    PLUGFLG
EC15  26 1B          381                BNE     LEC32
EC17  CE EE A1       382                LDX     #KEYHAND
EC1A  DF 67          383                STX     $67
EC1C  CE ED 05       384                LDX     #LED05
EC1F  DF 69          385                STX     $69
EC21  CE ED 58       386                LDX     #LED58
EC24  DF 6B          387                STX     $6B
EC26  CE F9 F5       388                LDX     #LF9F5
EC29  DF 6D          389                STX     $6D
EC2B  CE FE 03       390                LDX     #LFE03
EC2E  DF 6F          391                STX     $6F
EC30  20 03          392                BRA     LEC35
EC32  BD E0 CA       393  LEC32        JSR     $E0CA
EC35  CE 00 00       394  LEC35        LDX     #$0000
EC38  DF 9D          395                STX     CURTASK
EC3A  DF 9F          396                STX     $9F
EC3C  CE 00 00       397                LDX     #$0000
EC3F  BD EC BA       398                JSR     WAKE
EC42  CE 00 02       399                LDX     #$0002
EC45  BD EC BA       400                JSR     WAKE
EC48  0E             401                CLI
EC49  7E EC 53       402                JMP     SCHED
EC4C  A7 00          403  LEC4C         STAA    $00,X
EC4E  08             404                INX
EC4F  5A             405                DECB
EC50  2E FA          406                BGT     LEC4C
EC52  39             407                RTS
```

If `PLUGFLG` ($0000) is zero, the firmware installs its own five task routines in the vector table at $67-$6F (the keyboard handler, and so on; Chapter 27). If it is **not** zero, the firmware calls **$E0CA** instead, and the plug-in is expected to install the task table itself [FW]. A plug-in that wants to replace the firmware's tasks, as Space Invaders does, leaves the flag set and implements $E0CA. The frame's $E0CA is just an `RTS`, which would leave the task table empty, which is why the hook clears the flag first. Both outcomes are consistent: the flag is zero, so $E0CA is never called.

The program itself

The frame jumps to a label named `START`, defined in the program. Every tutorial's `START` does the same two things before anything else:

```
START   SEI                     ; no interrupts
        LDS     #$03FF          ; stack at top of RAM
```

`SEI` is there because the firmware, by the time ESC ` is pressed, has enabled interrupts (it executes `CLI` before entering the scheduler [FW]) and its ACIA and timer interrupts will continue to arrive. If a game does not mask them, an interrupt vectors through $FFF8 into the firmware's handler, which expects the scheduler's state to be intact. `LDS #$03FF` sets the stack at the top of RAM, as the firmware and all the working plug-ins do [FW][PGM].

When your program starts it **owns the machine**. The firmware is no longer running, since your program never returns to the scheduler. You must do your own keyboard polling and your own screen drawing. The cost is that the terminal does not talk to its host while the game runs. The benefit is simplicity and speed, and the games in this book rely on it.

How to get back

A program that never returns can only be left by reset. On the ADM-31 you reset by switching power off and on. If your program should hand control back to the firmware on request, use the hook technique of Chapter 25, or jump through the RESET vector (`LDX $FFFE` then `JMP 0,X`), which is `$E800` in the 129867 firmware and `$E900` in the 129827 [FW]; Chapter 25 has a program that does this. From there the whole boot happens again, including the tests, and the screen returns to the terminal's "Test Complete".

A summary of the frame's contract

address	contents	used by
$E000	`54 4A` "TJ"	the signature test at boot and at ESC `
$E002	`JMP $E03C`	ROM-only start
$E03C	`NOP NOP NOP`, `JMP START`	ESC ` (129867 at $E03C, 129827 at $E03E)
$E07D	`CLR $0000`, `PULA`, `PULA`, `JMP $E8D0`	boot hook, 129867
$E087	`CLR $0000`, `PULA`, `PULA`, `JMP $E9D0`	boot hook, 129827
$E094, $E0C3	`RTS`	129827 callbacks (defensive)
$E0CA	`RTS`	129867 task-table callback (defensive)
$E100	`START`	the program

Running the frame in the emulator

Two tests were run for every tutorial, and the results are in Appendix I. First, the ROM image was started at $E03C in a ROM-only emulator. Second, the full firmware was booted (129867 and 129827 sets), left to finish its self-test, and then ESC and the grave accent were sent to the keyboard; the plug-in's output then appeared below the firmware's "Test Complete" line. For T01HELLO both firmware sets gave the same screen.

```
Test Complete
                              HELLO
```

The "Test Complete" message stays at the top of the screen because the program draws only the cells it needs.

Summary

- The firmware recognises a plug-in by the two bytes "TJ" at $E000.
- It starts the plug-in at $E03C (129867) or $E03E (129827) when ESC ` is typed, and calls $E07D (129867) or $E087 (129827) at boot.

- The boot hook clears the plug-in flag, discards its return address and resumes the normal boot.
- A program owns the machine once started; the only way back is reset.

Exercises

1. What happens if the first two bytes of socket A are `54 4B`?
2. Why are there three `NOP`s at $E03C?
3. If you removed the two `PULA` instructions from the hook, which instruction would eventually go wrong, and how?
4. What is the number $544A in ASCII?

Answers. (1) The comparison fails and the plug-in is ignored at boot and at ESC `. (2) To serve both firmware entry addresses with one image. (3) Probably nothing visible: the stack would be two bytes deeper for the rest of the boot, and the firmware itself leaves five self-test bytes on the stack at this point [FW]. But the habit is bad: in any routine that is meant to return, an unbalanced stack makes `RTS` return to the wrong place. (4) “TJ”.

Part II

The machine

Chapter 6

The memory map

A 6800 program talks to the world in only one way: it reads and writes addresses. Some addresses are RAM, some are ROM, and some are *devices*, where a read or write changes something in the hardware. This chapter lays out the ADM-31's address space, region by region, with the evidence for each entry. Keep it open beside the later chapters; the address ledger (Appendix B) is its compact companion.

The whole map

range	what is there	evidence
$0000-$03FF	working RAM (1 KB)	[FW]
$7000	CRT synchronizing strobe	[FW][PGM]
$7800-$7803	PIA 1: keyboard, display page, bell	[FW][PGM]
$7900	switch bank 2	[FW]
$7A00-$7A01	host serial interface (ACIA): control/status, data	[FW]
$7B00	personality PROM window ($7B1E is read at boot)	[FW]
$7C00-$7C01	printer serial interface (ACIA): control/status, data	[FW]
$7D00	switch bank 3	[FW]
$7E00	switch bank 1	[FW]
$7E80	baud switch	[FW]
$7F00-$7F0E	CRT controller registers (VTAC)	[FW][PGM]
$8000-$8FFF	screen memory (two pages of 2 KB)	[FW][PGM]
$E000-$E7FF	socket A: plug-in ROM	[PGM]
$E800-$EFFF	firmware ROM, first 2 KB	[FW]
$F000-$F7FF	firmware ROM, second 2 KB	[FW]
$F800-$FFFF	firmware ROM, third 2 KB (vectors at $FFF8-$FFFF)	[FW]

Everything not listed is unused as far as this book has been able to verify. **Do not write to addresses outside this table.** Several of the devices are sensitive to writes and reads at addresses very near the ones used (the PIA, for instance, answers at four consecutive addresses). A write to an address in a gap may reach a device through partial address decoding. This book never relies on the gaps, and neither do the working ROMs.

A note on socket names

Two sources consulted for this book disagree about the *board designators* of the ROM sockets: one set of notes labels the four sockets U62 through U65 as ROMs A to D; another file (a configuration list for the firmware) names the firmware chips U-65, U-66 and U-67. This book therefore names sockets by **address range** and the firmware ROMs by their **part-number suffix**: $E800-$EFFF is ROM -67, $F000-$F7FF is ROM -66 and $F800-$FFFF is ROM -65, in the 129867 set. It uses "socket A" for the plug-in socket at $E000. Whatever your board is silk-screened, check the address decoding against that table rather than the label.

Working RAM, $0000-$03FF

The ADM-31 has 1,024 bytes of RAM and the firmware uses all of it:

- **Zero page, $0000-$00FF.** The firmware keeps its variables here, because direct addressing makes them cheaper to reach. The ones this book names are:

address	name	role
$00	PLUGFLG	plug-in flag (Chapter 5)
$11-$12	CRTBASE	base address of the page being displayed ($8000 or $8800)
$13-$14	CURADR	address of the cursor cell
$15	FIRSTLN	scroll position: the memory row shown on the bottom screen line (mirrored into CRT register 6)
$16	ROW	cursor row
$17	COL	cursor column
$18	PROTECT	protect flag
$19	ATTRMSK	mask ORed into every character the firmware stores
$1A	MONMODE	monitor mode flag
$1D	KBLOCK	keyboard lock flag
$45	ESCCHR	the escape character, $1B
$4D	BLANK	the blank character, $20
$65-$6F	task table	addresses of the scheduler's task routines (Chapter 27)
$85-$8C		task status bytes
$8D-$94		task timers
$9D	CURTASK	scheduler: current task

The names are this book's, given for convenience. All the addresses were read from firmware code [FW].

- **$0100-$010B** is the firmware's transmit queue, and **$0110-$037F** is the circular receive buffer [FW].
- **The stack** starts at $03FF [FW][PGM].

A plug-in that takes over the machine can use all of $0100-$03FE for its own variables, because the firmware's buffers are not in use while the plug-in runs. The working games do this: PONG keeps its variables at $0100-$0126 [PGM]. The tutorials use $0100-$010F for a shared set of scratch variables (Chapter 13) and a few more above $0110 for each program's own data.

A plug-in must not expect the firmware's variables to survive, and must not expect to return to a working terminal. Once your program has written over zero page or the buffers, the only supported way back is reset.

Screen memory, $8000-$8FFF

The CRT circuits read this memory continuously to draw the picture. Section "The screen" in Chapter 8 gives the arithmetic; the facts to hold now are:

- The visible 80 x 24 screen is the 1,920 bytes $8000-$877F. The cell for row *r*, column *c* is $8000 + 80*r* + *c* [PGM].
- The firmware keeps its **per-page state** in the last bytes of each page: at $87D0-$87D4 for the first page and $8FD0-$8FD4 for the second page [FW]. A program that clears *only* $8000-$877F (as PONG, Rubout and Shuttle Attack do [PGM]) leaves that state untouched.
- The second display page begins at $8800. Bit 2 of PIA 1 port B chooses which page the hardware displays [FW]: the firmware sets the base register $11 to $80 or $88 from that bit. Plug-ins clear bit 2 of $7802 at start-up so that page 1 is shown [PGM].
- At power-on the firmware tests all 4 KB: it writes $AA to every location in turn, checks it, writes the complement, checks again, and clears the location [FW].

The synchronizing strobe, $7000

$7000 is not memory. It is touched (a read or a write; the working programs write) before every access to screen memory [FW][PGM]:

```
        STAA    HALT            ; touch the strobe
        NOP                     ; let it settle
        STAA    0,X             ; now the CRT RAM access
```

The `NOP` in between appears in the firmware and in every working plug-in examined [FW][PGM]; this book copies the pattern exactly, in every routine that touches screen memory. The explanation usually given for the strobe is that it makes the processor wait for the right moment in the display cycle so that the access does not collide with the display circuits' own reads. This book did not measure that, and relies only on the rule: *strobe, NOP, then access*.

For loops that touch many cells (clearing the screen), the strobe sits *inside* the loop, once per cell. Reading a cell needs the strobe exactly as writing does.

The PIA, $7800-$7803

PIA 1 is a parallel interface with two 8-bit ports (A at $7800, B at $7802) and a control register for each ($7801, $7803). The firmware initialises it at boot [FW]:

```
E8D4                 165  ; Initialise PIA 1 (keyboard, display page, bell).
E8D4  CE 78 00       166  PIAINIT         LDX     #$7800
E8D7  6F 01          167                  CLR     $01,X
E8D9  6F 03          168                  CLR     $03,X
E8DB  86 7F          169                  LDAA    #$7F
E8DD  A7 00          170                  STAA    $00,X
E8DF  86 1D          171                  LDAA    #$1D
E8E1  A7 02          172                  STAA    $02,X
E8E3  86 3C          173                  LDAA    #$3C
E8E5  A7 01          174                  STAA    $01,X
```

```
E8E7  A7 03          175                 STAA    $03,X
E8E9  86 01          176                 LDAA    #$01
E8EB  A7 00          177                 STAA    $00,X
```

The sequence is the usual way of programming a 6820/6821-type PIA: clear the control registers, program the data-direction registers (port A direction $7F: bits 0-6 outputs, bit 7 input; port B direction $1D: bits 0, 2, 3 and 4 outputs), then write $3C to each control register to switch the ports to data mode, then set both ports to $01. Chapter 9 uses port A as the keyboard scan port and port B for the modifier keys; Chapter 10 uses a bit of the control register for the bell.

address	role	bits
$7800	port A	write: scan code (bits 0-6); read: bit 7 = selected key is down
$7801	control A	firmware uses $34 and $3C
$7802	port B	bit 7 SHIFT in; bit 6 CTRL in; bit 0 alpha-lock out; bit 2 display page
$7803	control B	bit 3 pulsed to ring the bell

The switches and the serial interfaces

address	role
$7900	switch bank 2: bits 2-4 are the printer port's word format, merged into the printer ACIA control byte
$7A00	host ACIA control/status (write $03 = master reset; the firmware initialises it with $43 then a computed byte)
$7A01	host ACIA data
$7C00	printer ACIA control/status
$7C01	printer ACIA data
$7D00	switch bank 3: bit 7 set = ignore the plug-in at boot
$7E00	switch bank 1: bits 2-4 are the host word format; bit 1 selects the CRT controller timing
$7E80	baud switch: the low four bits select a delay count from a 16-entry table

All of these were read from the firmware's initialisation [FW]. The plug-ins Rubout and Shuttle Attack write $03 to $7A00 and $7C00 early in their start-up, which is the "master reset" of the serial interface [PGM]; Chapter 12 gives the details. A game that never uses the serial ports should reset them so that they do not interrupt, and then leave interrupts masked.

The CRT controller, $7F00-$7F0E

The firmware writes a block of registers at power-on to configure the display timing:

```
E800                 57  ; Power-on / reset entry.  The IRQ vector is $ED83;
E800                 58  ; SWI, NMI and RESET all point here (see the vector
E800                 59  ; table at $FFF8).
E800  0F             60  RESET           SEI
E801  B7 7F 0E       61                  STAA    VTACE
E804  B7 7F 0A       62                  STAA    VTACA
E807  CE 7F 00       63                  LDX     #$7F00
```

```
E80A  86 65           64                LDAA    #$65
E80C  A7 00           65                STAA    $00,X
E80E  86 44           66                LDAA    #$44
E810  A7 01           67                STAA    $01,X
E812  86 5D           68                LDAA    #$5D
E814  A7 02           69                STAA    $02,X
E816  B6 7E 00        70                LDAA    SWBNK1
E819  84 02           71                ANDA    #$02
E81B  27 0A           72                BEQ     LE827
E81D  86 20           73                LDAA    #$20
E81F  A7 04           74                STAA    $04,X
E821  86 14           75                LDAA    #$14
E823  A7 05           76                STAA    $05,X
E825  20 08           77                BRA     LE82F
E827  86 40           78  LE827         LDAA    #$40
E829  A7 04           79                STAA    $04,X
E82B  86 35           80                LDAA    #$35
E82D  A7 05           81                STAA    $05,X
E82F  86 17           82  LE82F         LDAA    #$17
E831  A7 03           83                STAA    $03,X
E833  A7 06           84                STAA    $06,X
E835  97 15           85                STAA    FIRSTLN
E837  4F              86                CLRA
E838  A7 0D           87                STAA    $0D,X
```

The values are, register by register [FW]:

register	address	written value	purpose
0	$7F00	$65	timing
1	$7F01	$44	timing
2	$7F02	$5D	timing
3	$7F03	$17	timing
4	$7F04	$40, or $20 if switch bank 1 bit 1 is set	timing (alternative value set)
5	$7F05	$35, or $14 if bit 1 is set	timing
6	$7F06	$17	scroll register: memory row 23 on the bottom line = unscrolled
$0C	$7F0C	0	cursor column
$0D	$7F0D	0	cursor row
$0E	$7F0E	0	

Two honesty notes. First, the first two stores in the reset code (to $7F0E and $7F0A) store accumulator A as it happens to be after reset: the code does not load it first. Second, the code shows only that bit 1 of switch bank 1 chooses between two sets of values for registers 4 and 5; this book does not claim to know what each set does to the picture.

A plug-in does not reload the timing registers; the firmware already did. It does write two of them:

- `CRTC6` ($7F06) is set to $17 so that memory row 0 is the top screen line and row 23 the bottom, whatever the firmware had scrolled to (Chapter 8) [PGM];

- `CURCOL` ($7F0C) is set to a column outside the visible screen (the games use $E8 or $51, both beyond column 79) to park the hardware cursor out of sight [PGM].

Rules of the road

1. Strobe, then NOP, then every screen-memory access.
2. Never touch addresses outside the map above.
3. Set the stack to $03FF; mask interrupts.
4. Your RAM is $0100-$03FE; zero page belongs to the firmware.
5. Leave the page-state cells at $87D0 and $8FD0 alone if you ever want to return to the firmware without a reset.

Summary

- RAM at $0000-$03FF, screen at $8000-$8FFF, devices at $7000-$7F0E, plug-in at $E000, firmware at $E800-$FFFF.
- The screen's visible part is the 1,920 bytes from $8000; the page state is at the end of each 2 KB page.
- Write the strobe before every screen access.
- The ledger (Appendix B) lists each address with its evidence.

Exercises

1. Which addresses would a screen cell at row 3, column 40 occupy on the first page and on the second page (add $800)?
2. What would happen if a program put its variables in the zero page and then tried to return to the firmware? Why?
3. Look up "PIA 1 port B" in the ledger and list which of its bits this chapter says are inputs.

Answers. (1) $8000 + 3 x 80 + 40 = $8118 on page 1; $8918 on page 2. (2) The firmware's own variables live in zero page; overwriting them breaks the firmware's state, so a clean return is not possible without reset. (3) Bit 7 (SHIFT) and bit 6 (CTRL); bit 1 is also polled by the scheduler's idle loop [FW].

Chapter 7

Your first program: HELLO

This chapter builds the smallest useful program for the ADM-31: it writes the word HELLO in the middle of the screen. It has no loop, no subroutine and no data table: every instruction is spelled out, so that you can see exactly what each one does. Then a second version does the same job with a loop, and the chapter ends with a handful of experiments.

The plan

The screen is a block of memory. To make a letter appear, store its character code into the cell where you want it. The ASCII codes for the five letters are:

letter	H	E	L	L	O
code	$48	$45	$4C	$4C	$4F

Row 11, column 30 is near the centre of the screen. Its address is $8000 + 11 x 80 + 30 = $8000 + 910 = $838E. The five letters go into five consecutive cells, $838E to $8392.

The program must

1. mask interrupts and set the stack (the first lines of every program);
2. put $838E in X, the index register, to point at the first cell;
3. for each letter: load A with the code, touch the strobe, wait a moment, store A through X, and add one to X.

The listing

```
0000                    2  ; T01HELLO - the smallest useful plug-in ROM.
0000                    3  ; Writes one line of text into screen memory,
0000                    4  ; then loops forever.  Chapter 7.
0000                    5  ;
0000                    6  ; ---- hardware names used throughout this book ----
0000  =7000             7  HALT            EQU     $7000     ; CRT access strobe
0000                    8  ; keyboard scan / strobe
0000  =7800             9  PIA1            EQU     $7800
0000                   10  ; keyboard modifiers etc
0000  =7802            11  PIA1B           EQU     $7802
0000  =7803            12  PIA1CRB         EQU     $7803     ; bit 3 = bell
```

```
0000                    13  ; first-line register
0000  =7F06             14  CRTC6           EQU     $7F06
0000  =7F0C             15  CURCOL          EQU     $7F0C     ; cursor column
0000  =8000             16  CRTRAM          EQU     $8000     ; screen memory
0000                    17  ; ---- plug-in ROM A frame (the same one PONG uses)
0000                    18  ; ----
          ; ... 28 lines of the standard plug-in frame elided ...
E100                    47                  ORG     $E100
E100  0F                48  START           SEI               ; no interrupts
E101                    49  ; stack at top of RAM
E101  8E 03 FF          50                  LDS     #$03FF
E104                    51  ; row 11, column 30
E104  CE 83 8E          52                  LDX     #CRTRAM+910
E107  86 48             53                  LDAA    #$48      ; 'H'
E109  B7 70 00          54                  STAA    HALT      ; sync with the CRT
E10C  01                55                  NOP               ; settle
E10D  A7 00             56                  STAA    0,X       ; put 'H' on screen
E10F  08                57                  INX               ; next cell
E110  86 45             58                  LDAA    #$45      ; 'E'
E112  B7 70 00          59                  STAA    HALT
E115  01                60                  NOP
E116  A7 00             61                  STAA    0,X
E118  08                62                  INX
E119  86 4C             63                  LDAA    #$4C      ; 'L'
E11B  B7 70 00          64                  STAA    HALT
E11E  01                65                  NOP
E11F  A7 00             66                  STAA    0,X
E121  08                67                  INX
E122  B7 70 00          68                  STAA    HALT      ; second 'L'
E125  01                69                  NOP
E126  A7 00             70                  STAA    0,X
E128  08                71                  INX
E129  86 4F             72                  LDAA    #$4F      ; 'O'
E12B  B7 70 00          73                  STAA    HALT
E12E  01                74                  NOP
E12F  A7 00             75                  STAA    0,X
E131                    76  ; nothing more to do
E131  20 FE             77  FOREVER         BRA     FOREVER
```

The frame (the lines between `ORG $E000` and `ORG $E100`) was explained in Chapter 5 and is elided here. What remains:

The equates. `HALT`, `PIA1` and the rest are names for numbers. They produce no bytes (their listing lines show `=7000` and so on). Only `HALT` and `CRTRAM` are used by this program; the others are the shared hardware names of the book and appear in every listing so that the cross-reference table has a consistent form.

`ORG $E100`. The program starts 256 bytes into the ROM. Everything before it is frame.

`SEI` and `LDS #$03FF`. Mask interrupts, put the stack at the top of RAM. The listing shows their bytes: `0F` and `8E 03 FF`.

`LDX #CRTRAM+910`. Load X with the address of row 11, column 30. The assembler computes $8000 + 910 = $838E itself, so the listing shows `CE 83 8E`. Writing `CRTRAM+910` instead of `$838E` documents where the number came from. (An expression with a single `+` is safe in this assembler, as Chapter 3 explained.)

`LDAA #$48` loads A with the code of "H": the two bytes `86 48`.

`STAA HALT, NOP, STAA 0,X`. This is the pattern from Chapter 6: touch the strobe (`B7 70 00`), pause for one `NOP` (`01`), then store A at the address in X (`A7 00`: store A, indexed, offset 0). The letter appears on screen.

`INX` moves X to the next cell.

The same five lines are repeated for each letter, with `INX` between. The second "L" reuses the value already in A, so it skips the `LDAA`: the accumulator still holds $4C.

`FOREVER BRA FOREVER` is a loop of one instruction that jumps to itself. The assembled bytes are `20 FE`: opcode `20` for "branch always" and displacement `FE`, which is -2 in two's complement. A relative displacement counts from the address after the instruction; the instruction is two bytes long, so -2 goes back to its own first byte. The program has finished its job and has nowhere to return to (Chapter 5), so it idles. This is the programs' traditional stop.

What the bytes look like

Here is the program's own region of the finished ROM image, as a hex dump (each line starts with its address; the right-hand column shows the same bytes as characters where they are printable):

```
E100: 0F 8E 03 FF CE 83 8E 86 48 B7 70 00 01 A7 00 08  ........H.p.....
E110: 86 45 B7 70 00 01 A7 00 08 86 4C B7 70 00 01 A7  .E.p......L.p...
E120: 00 08 B7 70 00 01 A7 00 08 86 4F B7 70 00 01 A7  ...p......O.p...
E130: 00 20 FE                                         . .
```

The first line starts $0F $8E $03 $FF $CE $83 $8E $86 $48 (and the last two bytes of the dump, at $E131, are the `20 FE` of the final `BRA`): `SEI`, `LDS #$03FF`, `LDX #$838E`, `LDAA #$48`. Machine code is nothing more than that.

Running it

In the emulator used for this book, this program was started at $E03C and run for 20,000 instructions. The screen then showed:

```
HELLO
```

(The screen dump begins at the first non-empty row; HELLO is on row 11.)

Started from the real firmware the program behaves the same: the same ROM image was put in socket A of both firmware sets in the emulator, ESC and the grave accent were typed, and the screen showed the firmware's "Test Complete" on row 0 with HELLO on row 11, as in the figure of Chapter 5.

The same thing with a loop

HELLO took 30 instructions. A longer message would take hundreds. The idea of a loop is to write the instructions once and let the processor repeat them, each time with a different letter.

The loop needs two pointers: where the next letter is (in the program's data), and where it goes (on the screen). But the 6800 has *one* index register. The solution is to keep the two pointers in RAM and load X from one, use it, store it back, then load X from the other. Here is the program:

```
0000                    2  ; T01BLOOP - the same message, drawn by a loop.
0000                    3  ; Two pointers are needed (the source text and
0000                    4  ; the screen cell) but the 6800 has one index
```

```
0000                     5  ; register, so they live in RAM and are loaded
0000                     6  ; into X in turn.  Chapter 7.
0000                     7  ;
0000                     8  ; 2 bytes: text pointer
0000  =0110              9  SRC             EQU     $0110
0000                    10  ; 2 bytes: screen pointer
0000  =0112             11  DST             EQU     $0112
0000                    12  ; ---- hardware names used throughout this book ----
0000  =7000             13  HALT            EQU     $7000     ; CRT access strobe
0000                    14  ; keyboard scan / strobe
0000  =7800             15  PIA1            EQU     $7800
0000                    16  ; keyboard modifiers etc
0000  =7802             17  PIA1B           EQU     $7802
0000  =7803             18  PIA1CRB         EQU     $7803     ; bit 3 = bell
0000                    19  ; first-line register
0000  =7F06             20  CRTC6           EQU     $7F06
0000  =7F0C             21  CURCOL          EQU     $7F0C     ; cursor column
0000  =8000             22  CRTRAM          EQU     $8000     ; screen memory
0000                    23  ; ---- plug-in ROM A frame (the same one PONG uses)
0000                    24  ; ----
          ; ... 28 lines of the standard plug-in frame elided ...
E100                    53                  ORG     $E100
E100  0F                54  START           SEI               ; no interrupts
E101                    55  ; stack at top of RAM
E101  8E 03 FF          56                  LDS     #$03FF
E104                    57  ; first character of text
E104  CE E1 2C          58                  LDX     #MSG
E107  FF 01 10          59                  STX     SRC
E10A                    60  ; row 11, column 30
E10A  CE 83 8E          61                  LDX     #CRTRAM+910
E10D  FF 01 12          62                  STX     DST
E110                    63  ; fetch the next character
E110  FE 01 10          64  NEXT            LDX     SRC
E113  A6 00             65                  LDAA    0,X
E115                    66  ; zero ends the text
E115  27 13             67                  BEQ     DONE
E117  08                68                  INX
E118                    69  ; remember the new position
E118  FF 01 10          70                  STX     SRC
E11B                    71  ; where it goes on screen
E11B  FE 01 12          72                  LDX     DST
E11E  B7 70 00          73                  STAA    HALT      ; sync with the CRT
E121  01                74                  NOP               ; settle
E122                    75  ; put the character on screen
E122  A7 00             76                  STAA    0,X
E124  08                77                  INX
E125  FF 01 12          78                  STX     DST
E128  20 E6             79                  BRA     NEXT
E12A                    80  ; nothing more to do
E12A  20 FE             81  DONE            BRA     DONE
E12C  48 45 4C 4C       82  MSG             FCB     "HELLO, ADM-31",0
```

This is the first program in the book that has *variables*: `SRC` and `DST` are 16-bit locations in RAM ($0110 and $0112) that hold the two pointers. They are defined by `EQU`, which only gives the numbers names; the program never reserves the memory (RAM is not part of the ROM image), it simply uses addresses it knows are free.

Follow one pass of the loop:

1. `LDX SRC` loads X from the two bytes at $0110-$0111, the current text pointer.
2. `LDAA 0,X` loads the character at that pointer.
3. `BEQ DONE` leaves if the character is zero. The message ends in a zero byte, which is why the `FCB` line has the trailing `,0`. This is the **zero-terminated string**, which every text routine in the book uses.
4. `INX`, `STX SRC` advance the pointer and save it for next time.
5. `LDX DST` loads the screen pointer; the character is still in A.
6. Strobe, `NOP`, `STAA 0,X` draws it.
7. `INX`, `STX DST` advance the screen pointer; `BRA NEXT` repeats.

When the zero is reached, the program branches to `DONE`, which is another stop-at-the-end loop.

Notice how the loop *does not know how long the message is*. Replace the text and the program draws a different message. The price is the load-and-store of the two pointers on every pass, which costs time: the loop version executes 12 instructions per letter where the unrolled version needs 5. For a message it does not matter. In a game it will.

The listing shows one more subtle detail: the line `MSG FCB "HELLO, ADM-31",0` shows only the first four bytes (`48 45 4C 4C`) in the CODE column, though the assembler placed fourteen. The listing format has room for four bytes per line; the rest are in the ROM image. You can check the dump:

```
E12C: 48 45 4C 4C 4F 2C 20 41 44 4D 2D 33 31 00        HELLO, ADM-31.
```

Experiments

Try these. Each one teaches something about the machine.

1. **Move the message.** Change `CRTRAM+910` to `CRTRAM` (the top-left corner) and to `CRTRAM+1919` (the last cell on screen). Now try `CRTRAM+1920` and think about where that cell is.
2. **Remove the `NOP`s.** The firmware and every known-good plug-in have a `NOP` after the strobe; this book keeps it, and has not measured what would happen without it. If you try it on real hardware, expect nothing at all, or something odd.
3. **Print in two places.** After `DONE`, add a second message at another row. You will need to reload `SRC` and `DST`; do it with the same code, by turning the loop into a subroutine (Chapter 13).
4. **Characters above $7F.** Store $C8 instead of $48. Does the cell show "H" with an attribute, or something else? Record what you see on *your* screen: the firmware ORs a mask from location $19 into every character it stores (Chapter 6), which suggests bit 7 does something, but this book did not test what on a real display.

Summary

- A character appears when its code is stored in a screen cell, after the strobe.
- The cell address is $8000 + 80 x row + column.
- `INX` walks along a row; `BRA *` stops the program.
- Zero-terminated strings let one loop print any message.
- With one index register, keep extra pointers in RAM.

Exercises

1. What are the bytes of `STAA 0,X`?
2. How many bytes does the HELLO program occupy from `START` to the last `BRA`? (Subtract the addresses in the listing.)
3. Rewrite the loop version so that it prints the message *backwards*, starting from the last character at the right and moving left. (Hint: `DEX`.)
4. The listing shows `BEQ DONE` assembled as `27 13`. How far does the branch go? Check against the listing's addresses.

Answers. (1) `A7 00`. (2) The final `BRA FOREVER` is at $E131 and is two bytes long, so the program runs from $E100 to $E132: 51 bytes. (3) Store the *last* address of the message in `SRC`, `DEX` it instead of `INX`, and reverse the test for the end (compare with the start address; zero-termination does not help backwards). (4) $13 = 19 bytes beyond the instruction after the `BEQ`, i.e. from $E117 to $E12A.

Chapter 8

The screen: cells, rows and a drawing library

The first program wrote five letters at fixed addresses. Real programs draw at positions they *compute*: a ball at (row, column), a paddle that moves, a score in the corner. This chapter builds the five routines that every later program uses: clear the screen, draw a horizontal line, convert (row, column) to an address, draw one character, and draw a string. It then uses them in two programs, a frame around the screen and a chart of the whole character set, and finishes with how the hardware scroll works and what a plug-in must do about it.

The geometry

The visible screen has 24 rows of 80 columns. Screen memory is laid out row after row, in order, from $8000. Row 0 is $8000-$804F (80 bytes), row 1 starts at $8050, and so on. The cell for row *r*, column *c* is therefore

```
address = $8000 + 80 * row + column
```

with *r* in 0..23 and *c* in 0..79 [PGM]. The last visible cell is row 23, column 79 = $8000 + 1919 = $877F. The whole screen is 1,920 bytes.

The multiplication is the only awkward part. The 6800 has no multiply instruction, but 80 = 64 + 16, and multiplying by a power of two is a left shift. So

row x 80 = (row x 16) + (row x 64)

and each term is a few shifts. The routine below does exactly that.

CALCADR: row and column to address

The tutorial programs pass arguments in two RAM bytes, `ROWARG` ($0100) and `COLARG` ($0101), and `CALCADR` returns the address in X. (The 6800 has no fast way to pass arguments in registers when several are needed; fixed RAM locations are the simplest convention and cost little.) Here is the routine as assembled in the second tutorial:

```
E1AA                 135  ; CALCADR - X = CRTRAM + ROWARG*80 + COLARG
E1AA                 136  ; Uses 80 = 64 + 16: shift row left 4 (x16),
E1AA                 137  ; keep it, shift 2 more (x64), add the two.
E1AA  7F 01 02       138  CALCADR         CLR     ADRPTR    ; high byte = 0
E1AD  B6 01 00       139                  LDAA    ROWARG
```

```
E1B0  B7 01 03       140              STAA    ADRPTR+1  ; value = row
E1B3  78 01 03       141              ASL     ADRPTR+1  ; x2
E1B6  79 01 02       142              ROL     ADRPTR
E1B9  78 01 03       143              ASL     ADRPTR+1  ; x4
E1BC  79 01 02       144              ROL     ADRPTR
E1BF  78 01 03       145              ASL     ADRPTR+1  ; x8
E1C2  79 01 02       146              ROL     ADRPTR
E1C5  78 01 03       147              ASL     ADRPTR+1  ; x16
E1C8  79 01 02       148              ROL     ADRPTR
E1CB  B6 01 02       149              LDAA    ADRPTR    ; A:B = row*16
E1CE  F6 01 03       150              LDAB    ADRPTR+1
E1D1  36             151              PSHA              ; save row*16
E1D2  37             152              PSHB
E1D3  78 01 03       153              ASL     ADRPTR+1  ; x32
E1D6  79 01 02       154              ROL     ADRPTR
E1D9  78 01 03       155              ASL     ADRPTR+1  ; x64
E1DC  79 01 02       156              ROL     ADRPTR
E1DF  33             157              PULB              ; B = low(row*16)
E1E0  FB 01 03       158              ADDB    ADRPTR+1  ; + low(row*64)
E1E3  F7 01 03       159              STAB    ADRPTR+1
E1E6  32             160              PULA              ; A = high(row*16)
E1E7  B9 01 02       161              ADCA    ADRPTR    ; + high(row*64)
E1EA  B7 01 02       162              STAA    ADRPTR    ; row*80
E1ED  B6 01 01       163              LDAA    COLARG
E1F0  BB 01 03       164              ADDA    ADRPTR+1  ; + column
E1F3  B7 01 03       165              STAA    ADRPTR+1
E1F6  B6 01 02       166              LDAA    ADRPTR
E1F9  89 00          167              ADCA    #0
E1FB  8B 80          168              ADDA    #$80      ; + $8000
E1FD  B7 01 02       169              STAA    ADRPTR
E200  FE 01 02       170              LDX     ADRPTR    ; X = address
E203  39             171              RTS
```

Follow it with row 11, column 30 (the cell where HELLO was drawn). The 16-bit work value `ADRPTR` is two bytes, high byte first:

step	ADRPTR	note
`CLR ADRPTR`; low byte = row	$000B	11
four `ASL`/`ROL` pairs	$00B0	row x 16 = 176
`PSHA`, `PSHB`		save row x 16 on the stack
two more pairs	$02C0	row x 64 = 704
add the saved row x 16	$0370	= 880 = row x 80
add `COLARG` (30)	$038E	
add $80 to the high byte	$838E	the cell address

Each `ASL ADRPTR+1` shifts the low byte left and moves bit 7 into the carry; the following `ROL ADRPTR` rotates the carry into the bottom of the high byte. Together they shift a 16-bit value left one place. The same two-instruction pair is the key to every multi-byte shift on the 6800.

The addition uses `ADDB` for the low byte and `ADCA` for the high byte: `ADCA` includes the carry from the low-byte add, so a carry out of bit 7 ripples into the high byte. The final `ADCA #0` after adding the column carries a possible overflow out of the low byte; `ADDA #$80` then supplies the $8000 base.

The routine is deliberately unglamorous. It is not fast: a few dozen instructions each call. Later chapters call it only when a position changes and keep the resulting address when they can; PONG calls it once per ball move and remembers the address (Chapter 21). Real programs on this class of machine are shaped by the cost of this routine.

CLRSCR: blanking the screen

```
E18E                 116  ; CLRSCR - fill the 1920 visible cells with blanks
E18E  CE 80 00       117  CLRSCR          LDX     #CRTRAM   ; first cell
E191  4F             118                  CLRA              ; blank = $00
E192  B7 70 00       119  CS_LP           STAA    HALT      ; sync with the CRT
E195  01             120                  NOP               ; settle
E196  A7 00          121                  STAA    0,X       ; blank one cell
E198  08             122                  INX               ; next cell
E199                 123  ; past the last cell?
E199  8C 87 80       124                  CPX     #CRTRAM+1920
E19C  26 F4          125                  BNE     CS_LP     ; no: repeat
E19E  39             126                  RTS
```

`CLRSCR` loads X with the first cell, A with zero, and then stores A through X for every one of the 1,920 cells, with the strobe and `NOP` before each store. `CPX #CRTRAM+1920` compares X with $8780, the address one past the last visible cell. `BNE` loops until they match. `INX` does not touch the carry, and we use only the zero flag, so the `CPX` caveat of Chapter 4 does not apply.

The blank is **$00** [PGM]. The firmware clears with $20 (the blank character, whose code is kept at zero-page $4D) in most of its routines [FW], but Rubout, Shuttle Attack and PONG all clear with $00 [PGM] and the display shows an empty cell in both cases. This book follows the games. Where a program must *compare* a cell with "empty" (collision detection, Chapter 18), the rule must match what the program itself wrote: the tutorials write $00 and test for $00.

The loop is an example of why the strobe was explained so carefully in Chapter 6: there are 1,920 strobes in one clear. Each iteration costs a fixed number of cycles; the whole clear takes a few tens of thousands.

HLINE: a run of cells

```
E19F                 127  ; HLINE - A = char, B = count, X = first cell
E19F  B7 70 00       128  HLINE           STAA    HALT      ; sync with the CRT
E1A2  01             129                  NOP               ; settle
E1A3  A7 00          130                  STAA    0,X       ; write one cell
E1A5  08             131                  INX               ; next cell
E1A6  5A             132                  DECB              ; count down
E1A7  26 F6          133                  BNE     HLINE     ; more cells
E1A9  39             134                  RTS
```

`HLINE` is the simplest useful drawing routine: A = character, B = count, X = first cell. It stores A at the cell, steps X, counts B down, and repeats until B reaches zero. A count of zero would loop 256 times; the caller must never pass zero.

PUTAT and PUTSTR: characters and text

```
E204                 172  ; PUTAT - write A at (ROWARG,COLARG)
E204                 173  ; save the character
```

```
E204  B7 01 04       174  PUTAT           STAA    TMPA
E207  BD E1 AA       175                  JSR     CALCADR   ; X = cell address
E20A  B6 01 04       176                  LDAA    TMPA
E20D  B7 70 00       177                  STAA    HALT      ; sync with the CRT
E210  01             178                  NOP               ; settle
E211  A7 00          179                  STAA    0,X       ; write the cell
E213  39             180                  RTS
E214                 181  ; PUTSTR - print the zero-terminated string at X
E214                 182  ; starting at (ROWARG,COLARG); COLARG advances
E214  FF 01 06       183  PUTSTR          STX     PSRC      ; save pointer
E217  FE 01 06       184  PS_LP           LDX     PSRC
E21A  A6 00          185                  LDAA    0,X       ; next character
E21C                 186  ; zero ends the string
E21C  27 0C          187                  BEQ     PS_X
E21E  08             188                  INX
E21F  FF 01 06       189                  STX     PSRC
E222  BD E2 04       190                  JSR     PUTAT     ; draw it
E225  7C 01 01       191                  INC     COLARG    ; next column
E228  20 ED          192                  BRA     PS_LP
E22A  39             193  PS_X            RTS
```

`PUTAT` draws the character in A at (`ROWARG`, `COLARG`). It must save A (the routine `CALCADR` uses A as scratch) in `TMPA`, call `CALCADR`, reload A, and then do the strobe-and-store. `PUTSTR` draws a zero-terminated string whose address is in X, starting at (`ROWARG`, `COLARG`), moving `COLARG` along as it goes. It keeps its text pointer in the RAM word `PSRC`, because the same X register that holds the text pointer is needed for the screen address inside `PUTAT`.

`GETAT` (the library also has this) is the mirror image: it returns in A the character at (`ROWARG`, `COLARG`), with the same strobe-then-access discipline for the *read*:

```
E294                 257  ; GETAT - A = character at (ROWARG,COLARG)
E294  BD E2 13       258  GETAT           JSR     CALCADR   ; X = cell address
E297  B7 70 00       259                  STAA    HALT      ; sync with the CRT
E29A  01             260                  NOP               ; settle
E29B  A6 00          261                  LDAA    0,X       ; read the cell
E29D  39             262                  RTS
```

The read form is needed for collision detection in Chapter 18.

Program T02FRAME: a border and a caption

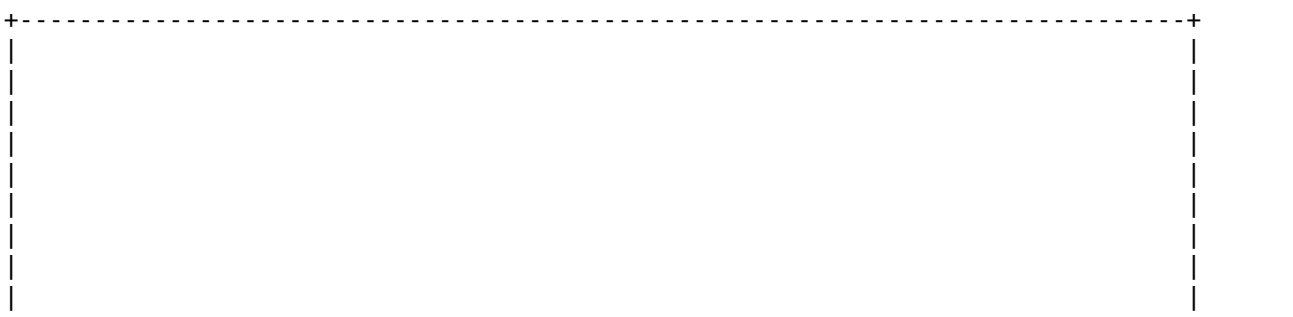
The first program to use the library draws a frame around the whole screen and a caption in the middle: dashes along rows 0 and 23, bars along columns 0 and 79, a plus in each corner. It exercises every routine above. Here is the main program part of its listing (the frame, the equates and the library routines are shown earlier or in Appendix E):

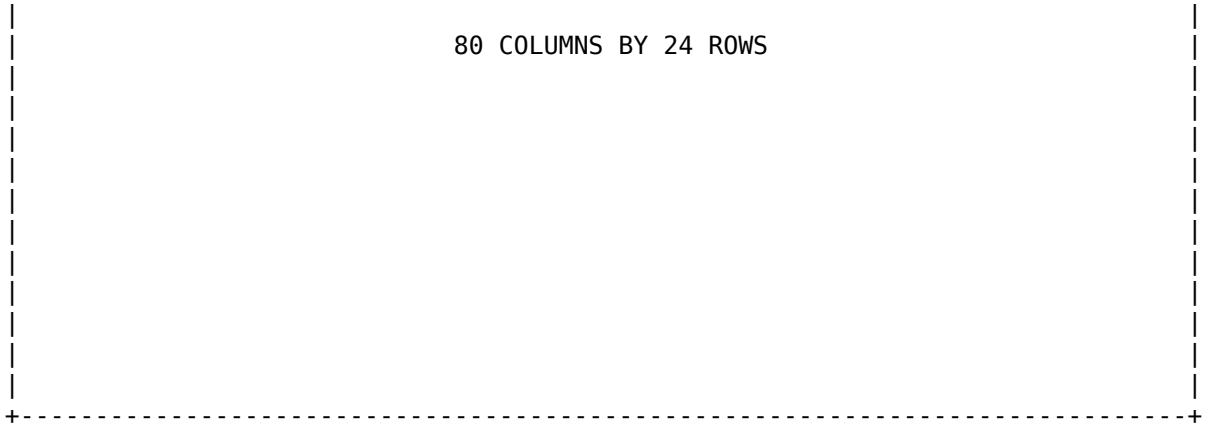
```
E100  0F              64  START           SEI
E101  8E 03 FF        65                  LDS     #$03FF
E104  BD E2 2B        66                  JSR     INITCRT
E107  BD E1 8E        67                  JSR     CLRSCR
E10A                  68  ; top row 0 and bottom row 23: 80 dashes each
E10A  CE 80 00        69                  LDX     #CRTRAM   ; row 0, col 0
E10D  86 2D           70                  LDAA    #$2D      ; dash
E10F  C6 50           71                  LDAB    #80
E111  BD E1 9F        72                  JSR     HLINE
E114  CE 87 30        73                  LDX     #CRTRAM+1840 ; row 23, col 0
E117  86 2D           74                  LDAA    #$2D
```

```
E119  C6 50           75                  LDAB    #80
E11B  BD E1 9F        76                  JSR     HLINE
E11E                  77  ; left and right edges, rows 1-22
E11E  86 01           78                  LDAA    #1
E120  B7 01 00        79                  STAA    ROWARG
E123  7F 01 01        80  EDGES           CLR     COLARG    ; column 0
E126  86 7C           81                  LDAA    #$7C      ; bar
E128  BD E2 04        82                  JSR     PUTAT
E12B  86 4F           83                  LDAA    #79       ; column 79
E12D  B7 01 01        84                  STAA    COLARG
E130  86 7C           85                  LDAA    #$7C
E132  BD E2 04        86                  JSR     PUTAT
E135  7C 01 00        87                  INC     ROWARG
E138  B6 01 00        88                  LDAA    ROWARG
E13B  81 17           89                  CMPA    #23       ; stop after row 22
E13D  26 E4           90                  BNE     EDGES
E13F                  91  ; four corners
E13F  7F 01 00        92                  CLR     ROWARG
E142  7F 01 01        93                  CLR     COLARG
E145  86 2B           94                  LDAA    #$2B      ; plus
E147  BD E2 04        95                  JSR     PUTAT
E14A  86 4F           96                  LDAA    #79
E14C  B7 01 01        97                  STAA    COLARG
E14F  86 2B           98                  LDAA    #$2B
E151  BD E2 04        99                  JSR     PUTAT
E154  86 17          100                  LDAA    #23
E156  B7 01 00       101                  STAA    ROWARG
E159  86 2B          102                  LDAA    #$2B
E15B  BD E2 04       103                  JSR     PUTAT
E15E  7F 01 01       104                  CLR     COLARG
E161  86 2B          105                  LDAA    #$2B
E163  BD E2 04       106                  JSR     PUTAT
E166                 107  ; caption in the middle
E166  86 0B          108                  LDAA    #11
E168  B7 01 00       109                  STAA    ROWARG
E16B  86 1E          110                  LDAA    #30
E16D  B7 01 01       111                  STAA    COLARG
E170  CE E1 78       112                  LDX     #CAPTION
E173  BD E2 14       113                  JSR     PUTSTR
E176  20 FE          114  FOREVER         BRA     FOREVER
```

The structure is typical: initialise, draw the horizontal edges with `HLINE`, loop over rows 1-22 drawing the side bars with `PUTAT`, draw the corners, draw the caption with `PUTSTR`, stop. The loop variable is `ROWARG` itself, which `PUTAT` does not modify. When `ROWARG` reaches 23 (`CMPA #23`), the loop ends: rows 1 to 22 have been done.

The emulator's screen after this program:





The caption is centred by hand: it is 21 characters long and starts in column 30, so it spans 30..50; the centre of the screen is between columns 39 and 40, and 30 + 10 = 40. Chapter 14 computes the starting column from the string length instead.

Program T03CHARS: the whole character set

The second program draws the 128 characters $00-$7F in a 16 x 8 chart, 16 per row, two columns apart, with hexadecimal labels, so that the code of any character can be read off as row-digit then column-digit. The emulator's screen:

```
     0 1 2 3 4 5 6 7 8 9 A B C D E F

  00
  10
  20   ! " # $ % & ' ( ) * + , - . /
  30 0 1 2 3 4 5 6 7 8 9 : ; < = > ?
  40 @ A B C D E F G H I J K L M N O
  50 P Q R S T U V W X Y Z [ \ ] ^ _
  60 ` a b c d e f g h i j k l m n o
  70 p q r s t u v w x y z { | } ~
```

Two things to notice. The row for $00-$1F shows blank cells in the emulator, because control codes have no glyph there. **What a real ADM-31 displays for those codes was not tested** and the chart is a good first experiment on real hardware. The printable codes ($20-$7E) are the ordinary ASCII set, as the emulator draws them. The chart is also a convenient test of the address routine: a mistake in the multiplication moves every character after the first row.

The program is made of the pieces you now know: `HEXDIG` to turn a nibble into a digit, `PUTAT` for each cell, a loop over the codes with the row and column worked out from the code's high and low nibbles:

```
row    = 4 + (code >> 4)        ; high nibble picks the row
column = 6 + 2 * (code & $0F)   ; low nibble picks the column
```

The right shift is four `LSRA`s; the "x 2" is an `ASLA`. The full listing is in Appendix E.

Hardware scrolling, and what a plug-in must do about it

A terminal scrolls constantly, and moving 1,840 bytes up one row for every line feed would be slow. The ADM-31 does not move the text. Its CRT controller has a register (register 6, at $7F06, mirrored in the firmware's zero-page byte $15, which this book calls `FIRSTLN`) that says

which row of memory is displayed on the bottom screen line. The memory is a ring of 24 rows. To scroll, the firmware adds one to that register (wrapping from 23 back to 0), then blanks the 80 cells of the row that has become the bottom line. Nothing else moves [FW]. Here is the heart of it, from the firmware:

```
F334  D6 55         1265  LF334           LDAB    $55
F336  C5 08         1266                  BITB    #$08
F338  26 F7         1267                  BNE     LF331
F33A  4C            1268                  INCA
F33B  81 18         1269                  CMPA    #$18
F33D  2D 01         1270                  BLT     LF340
F33F  4F            1271                  CLRA
F340  97 15         1272  LF340           STAA    FIRSTLN
F342  B7 7F 06      1273                  STAA    VTAC6
F345  BD F2 DD      1274                  JSR     LF2DD
F348  90 17         1275                  SUBA    COL
F34A  C2 00         1276                  SBCB    #$00
F34C  D7 B7         1277                  STAB    $B7
F34E  97 B8         1278                  STAA    $B8
F350  DE B7         1279                  LDX     $B7
F352  C6 50         1280                  LDAB    #$50
F354  4F            1281                  CLRA
F355  B7 70 00      1282  LF355           STAA    HALT
F358  01            1283                  NOP
F359  A7 00         1284                  STAA    $00,X
F35B  08            1285                  INX
F35C  5A            1286                  DECB
F35D  26 F6         1287                  BNE     LF355
F35F  DE 13         1288                  LDX     CURADR
F361  39            1289                  RTS
```

(`CMPA #$18 / BLT / CLRA` is the wrap: the register counts 0 to 23. `STAA FIRSTLN` and `STAA VTAC6` store the new value; the loop at `LF355` clears the new bottom row with zeros, 80 cells, strobing each.)

At power-on the register holds $17 (23): the bottom line is memory row 23, so the top line is memory row 0, and screen memory is in its natural order. After the terminal has scrolled a few times the register holds some other value and the cell at $8000 may be in the *middle* of the screen.

A plug-in therefore sets register 6 to $17 before it draws, so that $8000 is the top-left corner:

```
        LDAA    #$17
        STAA    CRTC6
```

Rubout, Shuttle Attack and PONG all do this as part of their start-up [PGM]. From the second tutorial on, every program in this book calls one small routine, `INITCRT`, which does what PONG does at the top of its start-up: set register 6 to $17, park the cursor, and clear bit 2 of port B so that page 1 is displayed:

```
E22B                194  ; INITCRT - the start-up the working games share:
E22B                195  ; memory row 0 becomes the top screen line, the
E22B                196  ; cursor is parked off screen, page 1 is shown
E22B  86 17         197  INITCRT         LDAA    #$17
E22D  B7 7F 06      198                  STAA    CRTC6     ; unscrolled
E230  43            199                  COMA              ; A = $E8
E231                200  ; cursor beyond column 79
```

```
E231  B7 7F 0C       201                 STAA    CURCOL
E234  B6 78 02       202                 LDAA    PIA1B
E237  84 FB          203                 ANDA    #$FB      ; bit 2 low: page 1
E239  B7 78 02       204                 STAA    PIA1B
E23C  39             205                 RTS
```

(The `COMA` turns the $17 left in A into $E8, the park column; PONG does it the same way. The first program, HELLO, leaves `INITCRT` out to stay as short as possible; it assumes the terminal has just booted and has not scrolled, in which case register 6 already holds $17.)

The cursor registers, `CURCOL` at $7F0C (column) and $7F0D (row) in the firmware, put a cursor on the screen at the position written. The firmware keeps them equal to its `COL` and `ROW` variables. A game does not want a cursor on the playfield, so the working games write a column beyond the right edge: $E8 (PONG) or $51 (Shuttle Attack) [PGM]. Both are well off the 80 visible columns.

Two display pages

The 4 KB of screen memory holds two pages. Bit 2 of $7802 (PIA 1 port B) chooses the page that is displayed. The firmware sets `CRTBASE` ($11-$12) to $8000 or $8800 from that bit [FW], and keeps each page's cursor state in the last bytes of the page ($87D0-$87D4 and $8FD0-$8FD4). The working plug-ins clear the bit so that page 1 is displayed and write only into page 1 [PGM]. A program that wanted double-buffering could draw into the second page and flip the bit; this book does not, because no working plug-in does and the flip has not been verified on real hardware.

Summary

- Cell address = $8000 + 80 x row + column. The visible screen is 1,920 cells.
- `CALCADR` multiplies by 80 with shifts; `PUTAT`, `PUTSTR`, `HLINE`, `CLRSCR`, `GETAT` are the drawing library.
- Every screen access is strobe, `NOP`, access.
- Scrolling is a register change plus clearing one row; a plug-in sets register 6 to $17 so that $8000 is the top line.
- Page 2 exists at $8800; this book stays on page 1.

Exercises

1. Compute by hand the address for row 20, column 5, and check it against the shifts: 20 x 16, then 20 x 64.
2. Change T02FRAME so that the border is made of # ($23) characters. Which constants change?
3. Write a routine `VLINE` that draws a vertical line of A characters, B long, starting at (`ROWARG`, `COLARG`). Hint: add 80 to X for each step.
4. What would happen on the screen if the program did not set `CRTC6` to $17 and the firmware had scrolled five lines before the plug-in started?

Answers. (1) 20 x 80 = 1,600 = $640; + 5 = $645; + $8000 = $8645. (2) The three `LDAA #$2D`, `#$7C`, `#$2B` constants (dash, bar, plus). (3) Use `CALCADR` once, then a loop that stores A and adds 80 to X with two `ADDB/ADCA` steps or with 80 `INX` calls (slower). (4) The picture would be rotated:

what the program drew in row 0 would appear five lines below the top, and the bottom rows would wrap to the top of the screen.

Chapter 9

The keyboard

The ADM-31's keyboard has no controller chip that remembers key presses. It is a switch matrix, and the processor *asks* the hardware, one key at a time, whether that key is down. Games exploit this directly: they ask only about the keys they care about, as often as they like, and never wait for a typewriter-style event. This chapter explains the protocol, shows how the firmware turns a key number into a character, and builds two programs: a scan-code explorer and a typewriter.

The scan protocol

Every key has a **scan code**, a number from $00 to $57 (88 keys). To test a key:

1. write its scan code to PIA 1 port A, address $7800 (named `PIA1` in this book's sources); only bits 0-6 are used for the code;
2. read $7800 back; **bit 7 of the value read is 1 if that key is down**.

That is all [FW][PGM]. The firmware's scanner, which tests keys continuously, is built from exactly these steps:

```
EE06                 634  ; Keyboard scan task.
EE06  C6 04          635  KBSCAN          LDAB    #$04
EE08  96 A9          636  LEE08           LDAA    $A9
EE0A  B7 78 00       637                  STAA    PIA1A
EE0D  B6 78 00       638                  LDAA    PIA1A
EE10  2B 0B          639                  BMI     LEE1D
EE12  26 02          640                  BNE     LEE16
EE14  86 58          641                  LDAA    #$58
EE16  4A             642  LEE16           DECA
EE17  97 A9          643                  STAA    $A9
EE19  5A             644                  DECB
EE1A  26 EC          645                  BNE     LEE08
EE1C  39             646                  RTS
```

(`$A9` is the scanner's current scan code; it counts *down* from $57 to $00 and wraps. `PIA1A` is `PIA1`. The routine scans four codes per call.) The test is `STAA PIA1A / LDAA PIA1A / BMI`: `BMI` is "branch if minus", which tests bit 7 of the value just loaded.

Three consequences are worth stating plainly.

- **Any key can be tested at any time**, in any order. A game's input routine is a handful of tests, not a keyboard driver.

- **Several keys can be tested in the same instant**, so a game can read "up" and "fire" together. (How many simultaneous keys the physical matrix reports correctly was not tested for this book; matrices commonly mis-read some combinations of three or more.)
- **There is no queue.** A key held for a second reads as "down" for a second. A program that wants "typed once" behaviour must keep its own *held* flag, as the typewriter program below does.

The modifier keys

SHIFT and CTRL are not part of the 88 scan codes: they are read from PIA 1 port B, address $7802 (`PIA1B`):

bit of $7802	meaning	evidence
7	SHIFT is down	[FW][EMU]
6	CTRL is down	[FW]
0	ALPHA LOCK	[FW]

The firmware reads port B immediately after it finds a key down, shifts bit 7 into the table index (see below), and tests bit 0 for alpha lock and bit 6 for CTRL. *Older notes sometimes attribute the SHIFT bit to a different position or label bit 0 differently; the firmware code above is the authority: SHIFT is bit 7, ALPHA LOCK bit 0.*

The firmware uses the modifiers like this (from the listing above):

- If **alpha lock** (bit 0) is set and the character is a lower-case letter ($61-$7A), subtract $20 to make it upper-case.
- If **CTRL** (bit 6) is set and the character is $40 or above (but not $7F), AND it with $1F: CTRL-A becomes $01, CTRL-G becomes $07 and so on.

From scan code to character: the keyboard table

The firmware keeps a 176-byte table, `KBDTAB`, in ROM at $EA1A (129867) or $EB1B (129827) [FW]. It holds two bytes for each of the 88 scan codes, **unshifted first, shifted second**. The index into the table is therefore 2 x scan, plus 1 if SHIFT is down. The firmware computes that index with a neat trick, visible in the listing: after reading port B into B, it does `ASLB`, which moves SHIFT (bit 7) into the carry, and `ROLA`, which rotates the carry into the bottom of A, where A already holds the scan code. In two instructions A becomes 2 x scan + SHIFT. Then `JSR XPLUSA` adds A to X (which holds the table address), and `LDAA 0,X` fetches the character.

The complete table is in Appendix D. A few entries you will use:

key	scan	unshifted	shifted
W	$20	`w` $77	`W` $57
S	$31	`s` $73	`S` $53
P	$28	`p` $70	`P` $50
SPACE	$4B	$20	$20
LEFT arrow	$54	$08 (BS)	$08
DOWN arrow	$52	$0A (LF)	$0A

key	scan	unshifted	shifted
UP arrow	$53	$0B (VT)	$0B
RIGHT arrow	$55	$0C (FF)	$0C
ESC	$07	$1B	$1B
RETURN	$2B	$0D	$0D
CLEAR / FIRE key	$40	$90	$91

The arrow keys generate the ASCII control codes BS, LF, VT and FF, the usual cursor-movement codes of this terminal family. Codes with bit 7 set (the table has several) are the firmware's special function codes, which the firmware handles separately; a plug-in that wants those keys tests the *scan code*, not the character.

When the firmware accepts a key it waits ten scheduler ticks before looking for the next one and four ticks for the key to be released [FW]; this is the terminal's de-bounce and its repeat delay. A game that scans in a tight loop has no such delay, and a key that is "down" is simply down.

Program T05SCAN: the explorer

The first keyboard program does nothing but display what the hardware tells it: the scan code of the first key found down, the raw value of port B, and S and C flags for SHIFT and CTRL. Here is the listing:

```
E100  0F             68  START           SEI
E101  8E 03 FF       69                  LDS     #$03FF
E104  BD E2 4F       70                  JSR     INITCRT
E107  BD E1 99       71                  JSR     CLRSCR
E10A  86 04          72                  LDAA    #4
E10C  B7 01 00       73                  STAA    ROWARG
E10F  86 14          74                  LDAA    #20
E111  B7 01 01       75                  STAA    COLARG
E114  CE E1 8E       76                  LDX     #CAPTION
E117  BD E2 14       77                  JSR     PUTSTR
E11A  7F 01 10       78  MAIN            CLR     SCAN
E11D  B6 01 10       79  SCANLP          LDAA    SCAN
E120  B7 78 00       80                  STAA    PIA1      ; select this key
E123  B6 78 00       81                  LDAA    PIA1      ; read it back
E126                 82  ; bit 7 = key is down
E126  2B 0C          83                  BMI     FOUND
E128  7C 01 10       84                  INC     SCAN
E12B  B6 01 10       85                  LDAA    SCAN
E12E  81 58          86                  CMPA    #$58      ; 88 codes
E130  26 EB          87                  BNE     SCANLP
E132                 88  ; nothing down: rescan
E132  20 E6          89                  BRA     MAIN
E134  B6 01 10       90  FOUND           LDAA    SCAN
E137  B7 01 11       91                  STAA    LASTSC
E13A                 92  ; show the scan code
E13A  86 06          93                  LDAA    #6
E13C  B7 01 00       94                  STAA    ROWARG
E13F  86 22          95                  LDAA    #34
E141  B7 01 01       96                  STAA    COLARG
E144  B6 01 10       97                  LDAA    SCAN
```

```
E147  BD E2 36        98                  JSR     PUTHEX
E14A  86 08           99                  LDAA    #8        ; show raw PIA1B
E14C  B7 01 00       100                  STAA    ROWARG
E14F  86 22          101                  LDAA    #34
E151  B7 01 01       102                  STAA    COLARG
E154  B6 78 02       103                  LDAA    PIA1B
E157  BD E2 36       104                  JSR     PUTHEX
E15A                 105  ; SHIFT flag, column 40
E15A  86 08          106                  LDAA    #8
E15C  B7 01 00       107                  STAA    ROWARG
E15F  86 28          108                  LDAA    #40
E161  B7 01 01       109                  STAA    COLARG
E164  B6 78 02       110                  LDAA    PIA1B     ; SHIFT is bit 7
E167  2A 04          111                  BPL     SHOFF
E169  86 53          112                  LDAA    #$53      ; 'S'
E16B  20 02          113                  BRA     SHPUT
E16D  86 20          114  SHOFF           LDAA    #$20      ; blank
E16F  BD E2 04       115  SHPUT           JSR     PUTAT
E172                 116  ; CTRL flag, column 42
E172  86 08          117                  LDAA    #8
E174  B7 01 00       118                  STAA    ROWARG
E177  86 2A          119                  LDAA    #42
E179  B7 01 01       120                  STAA    COLARG
E17C  B6 78 02       121                  LDAA    PIA1B
E17F  84 40          122                  ANDA    #$40      ; CTRL is bit 6
E181  27 04          123                  BEQ     CTOFF
E183  86 43          124                  LDAA    #$43      ; 'C'
E185  20 02          125                  BRA     CTPUT
E187  86 20          126  CTOFF           LDAA    #$20
E189  BD E2 04       127  CTPUT           JSR     PUTAT
E18C  20 8C          128                  BRA     MAIN
```

`MAIN` clears `SCAN` and loops through all 88 codes (`CMPA #$58` ends the pass): for each, write to `PIA1`, read back, and `BMI FOUND` if the key is down. When a pass finds nothing, `BRA MAIN` starts again. At `FOUND` the program prints the scan code with `PUTHEX` (two hex digits), then port B in the same way, then S or C if the modifier bits are set (bit 7 for SHIFT and bit 6 for CTRL), using `BPL` and `ANDA #$40`.

With the W key held in the emulator, the screen is:

```
SCAN CODE:

20

00
```

The scan code is $20 and port B reads $00: no modifiers. It is a very good first program to run on a real ADM-31: press every key and write down the code. The map in Appendix D is derived from the firmware table, so it tells you what the firmware *expects*; this program tells you what *your* keyboard sends.

Program T06TYPE: a typewriter

The second program turns key presses into characters on the screen. It needs

- an **edge-triggered** `GETKEY`: a key held down must produce one character, not thousands;
- the keyboard table, to turn scan codes into characters;

- a cursor position (row and column) that advances after each character, with RETURN starting a new line.

The listing of the key-reading routine:

```
E155                 109  ; GETKEY - returns A = character of a newly pressed
E155                 110  ; key, or 0 when nothing new.  HELD makes it edge
E155                 111  ; triggered: a held key reports once.
E155  7F 01 10       112  GETKEY          CLR     SCAN
E158  B6 01 10       113  GK_LP           LDAA    SCAN
E15B  B7 78 00       114                  STAA    PIA1      ; select scan code
E15E  B6 78 00       115                  LDAA    PIA1
E161  2B 0F          116                  BMI     GK_HIT    ; bit 7: down
E163  7C 01 10       117                  INC     SCAN
E166  B6 01 10       118                  LDAA    SCAN
E169  81 58          119                  CMPA    #$58
E16B  26 EB          120                  BNE     GK_LP
E16D  7F 01 11       121                  CLR     HELD      ; nothing down
E170  4F             122                  CLRA
E171  39             123                  RTS
E172  7D 01 11       124  GK_HIT          TST     HELD
E175  26 18          125                  BNE     GK_NONE   ; already reported
E177  86 01          126                  LDAA    #1
E179  B7 01 11       127                  STAA    HELD
E17C  B6 01 10       128                  LDAA    SCAN
E17F  48             129                  ASLA              ; scan * 2
E180  7D 78 02       130                  TST     PIA1B     ; SHIFT is bit 7
E183  2A 01          131                  BPL     GK_NS
E185  4C             132                  INCA              ; shifted entry
E186  CE E1 91       133  GK_NS           LDX     #KBDTAB
E189  BD E2 D3       134                  JSR     XPLUSA    ; X = &KBDTAB[A]
E18C  A6 00          135                  LDAA    0,X
E18E  39             136                  RTS
E18F  4F             137  GK_NONE         CLRA
E190  39             138                  RTS
```

`GETKEY` scans all 88 codes. If it finds a key down, it checks `HELD`: if the flag is already set it returns zero ("nothing new"); otherwise it sets the flag and translates the scan code. If the pass finds no key at all it clears `HELD`, so the next press is a new event. The translation is the firmware's trick: scan code x 2 (`ASLA`), plus one if `PIA1B` bit 7 is set (the `TST PIA1B` and `BPL` pair: `TST` sets the negative flag from bit 7), then `XPLUSA` to index `KBDTAB`.

One simplification to be aware of: this `GETKEY` reports the *first* key it finds in scan order, and while that key is down, other keys are not seen. A real typewriter would handle two-key rollover. For the tutorials this is enough.

The main loop calls `GETKEY` and draws the character with `PUTAT`, ignoring control codes (below $20) and DEL (`CMPA #$20 / BCS`, `CMPA #$7F / BCC`):

```
E115                  79  ; A = character or 0
E115  BD E1 55        80  MAIN            JSR     GETKEY
E118  27 FB           81                  BEQ     MAIN
E11A  81 0D           82                  CMPA    #$0D      ; RETURN
E11C  27 23           83                  BEQ     NEWLINE
E11E                  84  ; below space: ignore
E11E  81 20           85                  CMPA    #$20
E120  25 F3           86                  BCS     MAIN
E122                  87  ; DEL and above: ignore
```

```
E122  81 7F           88                CMPA    #$7F
E124  24 EF           89                BCC     MAIN
E126  36              90                PSHA
E127  B6 01 12        91                LDAA    CROW
E12A  B7 01 00        92                STAA    ROWARG
E12D  B6 01 13        93                LDAA    CCOL
E130  B7 01 01        94                STAA    COLARG
E133  32              95                PULA
E134  BD E2 AC        96                JSR     PUTAT
E137  7C 01 13        97                INC     CCOL
E13A  B6 01 13        98                LDAA    CCOL
E13D  81 50           99                CMPA    #80
E13F  26 D4          100                BNE     MAIN
E141  7F 01 13       101  NEWLINE      CLR     CCOL      ; carriage return
E144  7C 01 12       102                INC     CROW      ; line feed
E147  B6 01 12       103                LDAA    CROW
E14A  81 18          104                CMPA    #24
E14C  26 C7          105                BNE     MAIN
E14E  86 02          106                LDAA    #2        ; wrap to the top
E150  B7 01 12       107                STAA    CROW
E153  20 C0          108                BRA     MAIN
```

`BCS` and `BCC` are the *unsigned* branches, correct here because character codes are unsigned. The program keeps its cursor in `CROW`/`CCOL`, wrapping at the right edge to the next row, and at the bottom to row 2.

In the emulator the keys `Hello, ADM-31` and RETURN were injected; the screen:

```
Hello, ADM-31!
```

The table `KBDTAB` is copied into the plug-in (the library file `kbdtab.inc`) from the firmware, 176 bytes at the end of the program. A plug-in *could* point at the firmware's own copy at $EA1A, but that ties the program to one firmware set (129827's table is at $EB1B); copying the table makes the plug-in work with either. The dump of the table as assembled is in Appendix D.

Reading single keys: how the games do it

A game does not need `GETKEY`. PONG, for example, asks directly about the six keys it uses, every frame:

- UP ($53) or W ($20): paddle up
- DOWN ($52) or S ($31): paddle down
- SPACE ($4B) or CLEAR ($40): serve
- P ($28): pause

Each test is the same four instructions: load the scan code, store it to `PIA1`, load `PIA1`, branch on the sign. Chapter 16 builds a table-driven version of this idea, which handles a set of keys with one loop, and Chapter 21 shows PONG's own.

Summary

- Write a scan code ($00-$57) to $7800; read $7800; bit 7 set = key down.
- SHIFT is bit 7 and CTRL bit 6 of $7802; ALPHA LOCK is bit 0.
- The 176-byte `KBDTAB` maps (scan code, shift) to a character; the index is 2 x scan + SHIFT.
- Arrow keys give BS ($08), LF ($0A), VT ($0B), FF ($0C).

- Edge-triggered input needs a *held* flag.

Exercises

1. Which scan code does the P key have, and what are its unshifted and shifted characters?
2. Modify T05SCAN to display `A` when ALPHA LOCK (bit 0) is set. Where do you put the test?
3. Why does `GETKEY` test `HELD` *after* it finds a key and not before it scans?
4. A program wants to detect “both UP and FIRE are down”. Sketch the instructions.

Answers. (1) $28; `p` ($70) and `P` ($50). (2) After `FOUND`, read `PIA1B`, `ANDA #$01`, branch on the zero flag, and `PUTAT` the letter `A` ($41) or a blank at a free column, exactly as the CTRL flag is done. (3) Because `HELD` must be cleared when no key is down, and that is known only after the whole scan. (4) Write $53 to `PIA1`, read it back, `BPL` to “no” if bit 7 is clear; then write $40, read back, `BPL` to “no”; both down falls through.

Chapter 10

The bell, and what a PIA bit really is

The ADM-31's only sound is the bell. It has no tone generator that a program can set, no volume and no envelope; a program makes the bell sound by changing one bit in one register of the PIA. That makes the bell an ideal subject for the next lesson: *bit manipulation*. In the keyboard chapter you read bits out of a port. Here you write them, and learn how to change one bit of a register without disturbing the other seven, which is the commonest operation in hardware programming.

How the firmware rings the bell

The firmware's bell routine is five instructions:

```
F0DA                 979  ; BELL: pulse bit 3 of PIA1CRB.
F0DA  B6 78 03       980  BELL            LDAA    PIA1CRB
F0DD  84 F7          981                  ANDA    #$F7
F0DF  B7 78 03       982                  STAA    PIA1CRB
F0E2  8A 08          983                  ORAA    #$08
F0E4  B7 78 03       984                  STAA    PIA1CRB
F0E7  39             985                  RTS
```

Read it carefully. `PIA1CRB` is address $7803, PIA 1's control register B. The routine

1. loads the register into A,
2. clears bit 3 (`ANDA #$F7`: $F7 is binary 1111 0111, a mask with a zero in bit 3 and ones everywhere else),
3. stores that back to the register,
4. sets bit 3 (`ORAA #$08`: $08 is 0000 1000, a mask with a single one in bit 3),
5. stores again.

The bell is triggered by the *change* of bit 3, low then high. The two stores are made one right after the other; the second returns the bit to the state it had at the start (the firmware initialised the control register to $3C, which has bit 3 set [FW]). The routine is the handler the firmware's control-code table assigns to the BEL character ($07) [FW], and it is also called from other places where it needs to attract attention (the "MEMORY ERROR" path, for instance).

In the datasheet for a 6820-style PIA, bits 5 to 3 of a control register set the behaviour of the second control line (CB2 for register B). The firmware's value of $3C puts bits 5, 4 and 3 at 1, 1 and 1: the line is an output, held in manual mode, and bit 3 *is* the level on the line. That explains why changing bit 3 changes something in the hardware. This book did not trace the bell circuit and gives this explanation as the standard PIA behaviour; what the code proves is that pulsing

bit 3 of $7803 is the firmware's bell, and that the working game Shuttle Attack uses the same trick.

A different way to pulse: EOR

Shuttle Attack, one of the working plug-in ROMs, rings the bell like this (its own listing, as the assembler printed it):

```
E2C1                    166  BEEP
E2C1  B6 78 03          167          LDAA    $7803
E2C4  88 08             168          EORA    #$08
E2C6  B7 78 03          169          STAA    $7803
E2C9  88 08             170          EORA    #$08
E2CB  B7 78 03          171          STAA    $7803
E2CE  39                172          RTS
```

It loads the register, **exclusive-ORs** it with $08 (which flips bit 3 whatever its value), stores, flips again and stores. The second EOR undoes the first, so the register ends where it started, and the pin has gone through a complete pulse. This is neater than clearing and setting, because it does not matter what state bit 3 was in. The two techniques are equivalent when bit 3 starts at 1 (as the firmware leaves it) and different when it starts at 0: with EOR the first store *raises* the pin and the second lowers it, ending where it began.

The four bit operations

Everything you do to a hardware register is one of these:

to …	use	mask rule	example
clear bit n	`ANDA #mask`	mask has 0 in bit n, 1 elsewhere	`ANDA #$F7` clears bit 3
set bit n	`ORAA #mask`	mask has 1 in bit n, 0 elsewhere	`ORAA #$08` sets bit 3
flip bit n	`EORA #mask`	mask has 1 in bit n, 0 elsewhere	`EORA #$08` flips bit 3
test bit n	`BITA #mask`	mask has 1 in bit n	`BITA #$40`; then `BEQ`/`BNE`

The recipe for changing one bit of a hardware port is **read, modify, write**: `LDAA port`, apply one of the three, `STAA port`. That is how the plug-ins clear bit 2 of port B to select display page 1:

```
        LDAA    PIA1B           ; read the port
        ANDA    #$FB            ; clear bit 2 only
        STAA    PIA1B           ; write it back
```

$FB is 1111 1011. Every other bit keeps the value it had.

A caution. When you read an output port you read the *latch* (for the bits that are outputs) and the *pins* (for the bits that are inputs). On PIA 1 port B, bits 7 and 6 are inputs (SHIFT and CTRL) and bits 0, 2, 3 and 4 are outputs [FW]. A read-modify-write writes back the input bits' *current pin values* to the (ignored) output latch; this is harmless, because the data-direction register says those bits are inputs. But *do not* use read-modify-write on a register whose read value does not reflect what was last written (some device status registers); this book does not do so.

Program T07BELL

The listing is short enough to read at a glance: each press of B calls BEEP and increments a counter shown on screen.

```
E100  0F             65  START           SEI
E101  8E 03 FF       66                  LDS     #$03FF
E104  BD E3 18       67                  JSR     INITCRT
E107  BD E2 4D       68                  JSR     CLRSCR
E10A  7F 01 0D       69                  CLR     GKHELD
E10D  7F 01 10       70                  CLR     COUNT
E110  86 08          71                  LDAA    #8
E112  B7 01 00       72                  STAA    ROWARG
E115  86 1C          73                  LDAA    #28
E117  B7 01 01       74                  STAA    COLARG
E11A  CE E1 43       75                  LDX     #CAPTION
E11D  BD E2 C8       76                  JSR     PUTSTR
E120  BD E1 53       77  MAIN            JSR     GETKEY
E123  81 62          78                  CMPA    #$62      ; lower-case b
E125  27 04          79                  BEQ     RING
E127  81 42          80                  CMPA    #$42      ; upper-case B
E129  26 F5          81                  BNE     MAIN
E12B  BD E1 8F       82  RING            JSR     BEEP
E12E  7C 01 10       83                  INC     COUNT
E131  86 08          84                  LDAA    #8
E133  B7 01 00       85                  STAA    ROWARG
E136  86 2C          86                  LDAA    #44
E138  B7 01 01       87                  STAA    COLARG
E13B  B6 01 10       88                  LDAA    COUNT
E13E  BD E2 EA       89                  JSR     PUTHEX
E141  20 DD          90                  BRA     MAIN
```

Notice that the key test is on the *character* returned by the edge-triggered GETKEY (Chapter 9): CMPA #$62 for lower-case “b” or CMPA #$42 for upper-case. Because GETKEY is edge triggered, holding the key beeps once. The counter at COUNT is a single byte, printed with PUTHEX at row 8, column 44.

The library routine BEEP is the same as the firmware’s BELL:

```
E18F                122  ; BEEP - pulse bit 3 of PIA1 control B (the bell)
E18F  B6 78 03      123  BEEP            LDAA    PIA1CRB
E192  84 F7         124                  ANDA    #$F7      ; bit 3 low
E194  B7 78 03      125                  STAA    PIA1CRB
E197  8A 08         126                  ORAA    #$08      ; bit 3 high
E199  B7 78 03      127                  STAA    PIA1CRB
E19C  39            128                  RTS
```

This book’s emulator does not make a sound. Its test therefore checks the *effect*: after pressing B twice and SHIFT-B once, the counter on screen reads 03, and the emulator logged three transitions of bit 3 in $7803. On a real terminal you should hear three clicks or beeps. If you hear none, the instructions are probably fine and the likelier cause is the bell enable on your terminal’s rear switches, which this book has not investigated.

Timing a tone

One pulse is a click. A *train* of pulses with a delay between them is whatever the bell circuit makes of it. Shuttle Attack plays a burst of 16 pulses with a delay loop between each:

```
E3EF  36             328          PSHA
E3F0  BD E2 C1       329          JSR     BEEP
E3F3  32             330          PULA
E3F4  CE 0D 00       331          LDX     #$0D00
E3F7  09             333          DEX
E3F8  26 FD          334          BNE     RESET_128
E3FA  4A             335          DECA
E3FB  26 F2          336          BNE     RESET_120
E3FD  7E E2 AB       337          JMP     ENTRY_26F
```

`LDAA #$10` loads the pulse count (16); `PSHA`/`PULA` protect it across the `JSR BEEP` (which uses A); the inner loop `LDX #$0D00 / DEX / BNE` is a delay of 3,328 counts; `DECA`/`BNE` repeats 16 times. Whether the result sounds like a buzz or a tone depends on the bell hardware; this book has not measured the circuit and does not claim a pitch. The *method* (pulses separated by a delay) is the one the working game uses.

Summary

- The bell sounds when bit 3 of $7803 goes low and high again.
- Use AND to clear, OR to set, EOR to flip, BIT to test; read-modify-write changes one bit of a register.
- A pulse is two stores with opposite values of the bit, back to back; a tone, if any, is a train of them.

Exercises

1. Write the instruction sequence to *set* bit 4 of $7802 without changing any other bit. (Bit 4 is an output on this port; do not run it on a real machine without understanding what it drives.)
2. What mask and instruction test whether bit 6 of $7802 (CTRL) is set?
3. Why does the EOR bell work even if bit 3 starts at 0?
4. Make T07BELL beep twice per press, with a delay between.

Answers. (1) `LDAA $7802 / ORAA #$10 / STAA $7802`. (2) `LDAA PIA1B / BITA #$40`; `BNE` if set. (3) Because EOR flips whatever the bit is; two flips restore it. (4) Call `BEEP`, run a delay loop (Chapter 11), call `BEEP` again.

Chapter 11

Time: cycles, delay loops and game speed

A game has to run at a speed people can play. A computer with no timer that a program can set must make its own delays, and the only tool it has is to waste time in a loop of known length. This chapter shows how to count the time an instruction takes, how to build delay loops, how the firmware keeps time, and how the games choose their speed. It also says plainly what this book does *not* know: the processor's clock frequency was not measured, so every real-time figure here is stated as a formula with the clock as a parameter.

Cycles

Every instruction takes a fixed number of **clock cycles**. The Motorola 6800 data sheet lists them; the values for the instructions this book uses most are:

instruction	cycles	note
`NOP`	2	
`INX`, `DEX`	4	
`INCA`, `DECA`, `CLRA`, `ASLA`, `TSTA`, `TAB` …	2	register operations
`LDAA #n`	2	immediate
`LDAA addr`	3 or 4	direct 3, extended 4
`LDAA n,X`	5	indexed
`STAA addr`	4 or 5	direct 4, extended 5
`STAA n,X`	6	indexed
`ADDA #n`	2	
`CMPA #n`	2	
`BNE`, `BEQ`, `BRA` …	4	taken or not taken
`PSHA`, `PULA`	4	
`JSR addr`	9	extended
`RTS`	5	
`INC addr`, `DEC addr`	6	extended
`LDX #n`, `CPX #n`	3	
`LDX addr`	4 or 5	direct 4, extended 5

These figures come from the 6800 data sheet, not from measurements made for this book. They are standard and the games' authors used the same ones (PONG's source says "DEX/BNE = 8 clocks"), but if you build something where a cycle matters, measure it on your machine.

Two things follow at once. First, **a branch costs four cycles whether or not it is taken**: unlike later processors, the 6800 has no cheap not-taken branch. Second, **indexed and extended modes cost more than direct and immediate**, which is why firmware keeps hot variables in zero page.

A delay loop

The delay loop used by every game here is two instructions:

```
        LDX     #COUNT
WAIT    DEX                     ; 4 cycles
        BNE     WAIT            ; 4 cycles
```

`DEX` subtracts one from X and sets the zero flag when X reaches zero; `BNE` branches back while it has not. Each pass takes 4 + 4 = 8 cycles, so the loop takes about 8 x COUNT cycles, plus 3 for the `LDX`. COUNT can be anywhere from 1 to 65,535 (a count of 0 wraps and runs 65,536 times, the maximum delay: about 524,000 cycles).

With a clock of f cycles per second the delay in seconds is

```
seconds  =  8 * COUNT / f
```

If the clock is 1 MHz (this is **not verified** for the ADM-31; Chapter 6 listed what the book can and cannot say), the formula is 8 microseconds per count, so COUNT = 2,000 gives 16 ms, about one frame of a 60 Hz display. That is why the tutorial program below uses 2,000.

To delay longer than a 16-bit count allows, nest the loops:

```
        LDAB    #10
OUTER   LDX     #COUNT
INNER   DEX
        BNE     INNER
        DECB
        BNE     OUTER
```

The outer loop repeats the inner one ten times. Each of these instructions' flags is set by the one before: `DECB` sets the flag that `BNE OUTER` tests, and it is not disturbed in between because `LDX #COUNT` is *before* `DECB`.

Program T08SWEEP: a marker you can time

The program draws an asterisk in column 0 of row 12, waits, erases it, and moves one column right, wrapping at column 80, forever. The wait is the loop above with COUNT = 2,000.

```
E100  0F              67  START           SEI
E101  8E 03 FF        68                  LDS     #$03FF
E104  BD E1 D9        69                  JSR     INITCRT
E107  BD E1 47        70                  JSR     CLRSCR
E10A  CE 07 D0        71                  LDX     #2000     ; passes of DEX/BNE
E10D  FF 01 11        72                  STX     DELAY
E110  7F 01 10        73                  CLR     POS
E113  86 0C           74  SWEEP           LDAA    #12
E115  B7 01 00        75                  STAA    ROWARG
E118  B6 01 10        76                  LDAA    POS
```

```
E11B  B7 01 01        77                STAA    COLARG
E11E  86 2A           78                LDAA    #$2A      ; '*'
E120  BD E1 B2        79                JSR     PUTAT
E123  FE 01 11        80                LDX     DELAY
E126  09              81  WAIT          DEX               ; 4 cycles
E127  26 FD           82                BNE     WAIT      ; 4 cycles
E129  86 0C           83                LDAA    #12
E12B  B7 01 00        84                STAA    ROWARG
E12E  B6 01 10        85                LDAA    POS
E131  B7 01 01        86                STAA    COLARG
E134  4F              87                CLRA              ; erase
E135  BD E1 B2        88                JSR     PUTAT
E138  7C 01 10        89                INC     POS
E13B  B6 01 10        90                LDAA    POS
E13E  81 50           91                CMPA    #80
E140  26 D1           92                BNE     SWEEP
E142  7F 01 10        93                CLR     POS       ; wrap
E145  20 CC           94                BRA     SWEEP
```

(`LDX DELAY` loads the count each time, since the loop destroys X.) The structure is draw, delay, erase, advance, test for the right edge: the same shape every animation in the book has.

Here is the screen after 123,456 emulated instructions:

```
                                *
```

The emulator confirms the arithmetic: run for 100,000, 200,000 and 400,000 instructions, the marker is in columns 21, 45 and 14 (it has wrapped past 80 at the last), which gives about 4,170 instructions per column: the delay loop's 2,000 passes of two instructions, plus about 170 for drawing and erasing. The first 11,500 or so instructions go to clearing the screen.

A calibration experiment for your machine. Run T08SWEEP on the ADM-31 and time one complete sweep of the 80 columns with a stopwatch. Then the number of cycles per sweep is about 80 x (8 x 2,000 + overhead), where the overhead (drawing and erasing) is a few thousand cycles. Divide by your measured time to get the clock frequency. A sweep time of about 1.3 seconds would suggest 1 MHz. Any difference is information about your machine, and it changes nothing about the program except its speed.

How the firmware keeps time

The firmware does not use a delay loop for its own timing; it counts *ticks*. Look at the scheduler's idle section (Chapter 27 describes the scheduler in full). When every task is waiting, it runs this:

```
EC53                408  ; Cooperative scheduler: tasks wait with bit 7 of
EC53                409  ; their status set.
EC53  DE 9D         410  SCHED           LDX     CURTASK
EC55  A6 85         411  LEC55           LDAA    $85,X
EC57  2A 33         412                  BPL     LEC8C
EC59  08            413  LEC59           INX
EC5A  8C 00 08      414                  CPX     #$0008
EC5D  26 F6         415                  BNE     LEC55
EC5F  7F 00 9E      416                  FCB     $7F,$00,$9E ; CLR $009E
EC62  F6 78 02      417                  LDAB    PIA1B
EC65  C5 02         418                  BITB    #$02
EC67  26 EA         419                  BNE     SCHED
EC69  C6 34         420                  LDAB    #$34
```

```
EC6B  F7 78 01       421                 STAB    PIA1CRA
EC6E  C6 3C          422                 LDAB    #$3C
EC70  F7 78 01       423                 STAB    PIA1CRA
EC73  CE 00 00       424                 LDX     #$0000
EC76  A6 8D          425  LEC76          LDAA    $8D,X
EC78  27 0A          426                 BEQ     LEC84
EC7A  6A 8D          427                 DEC     $8D,X
EC7C  26 06          428                 BNE     LEC84
EC7E  A6 85          429                 LDAA    $85,X
EC80  84 7F          430                 ANDA    #$7F
EC82  A7 85          431                 STAA    $85,X
EC84  08             432  LEC84          INX
EC85  8C 00 08       433                 CPX     #$0008
EC88  26 EC          434                 BNE     LEC76
EC8A  20 C7          435                 BRA     SCHED
```

Read the middle of it:

- `LDAB PIA1B / BITB #$02 / BNE SCHED`: the firmware reads port B of PIA 1 and tests bit 1. If it is **set**, the loop goes back to the top and looks for runnable tasks again. Only when bit 1 reads **clear** does it go on.
- `LDAB #$34 / STAB PIA1CRA / LDAB #$3C / STAB PIA1CRA`: it writes $34 and then $3C to PIA 1's control register A, a short pulse on the PIA's CA2 control line.
- The loop at `LEC76` then goes through the eight timer bytes at $8D-$94 and decrements each non-zero one; when a timer reaches zero the corresponding task's blocked flag is cleared and it can run.

This is how the firmware's `SLEEP` works: a task puts a count in its timer and blocks. One **tick** is one pass through this idle section, and each tick is gated on bit 1 of port B going clear [FW].

What drives bit 1 is not established by the firmware code alone. A natural reading is that it is a periodic signal derived from the display (a vertical-sync or line-frequency flag), and that the pulse on the control line re-arms it. This book **did not verify** that. If it is true, then waiting for that bit is a better clock than a delay loop: it would give a steady rate that does not depend on how much work a frame does. *No working game among those examined uses it* (they all use delay loops), so everything in the games part of this book uses delay loops. If you want to experiment, run a program that counts how often bit 1 changes, with a stopwatch and a counter on screen, and see whether the rate looks like 50 or 60 per second.

Choosing a game speed

A game's speed is the length of its main delay. The rules of thumb:

1. **Pick the speed first** (how many moves per second you want), then compute the delay from the formula. Moves per second for PONG's ball: with the starting delay of 6,400 counts, one tick is 8 x 6,400 = 51,200 cycles, which, if the clock is 1 MHz, is about 51 ms: roughly 20 moves per second, and the game speeds up as the delay shrinks.
2. **Make the delay a variable**, not a constant in the loop, so that the game can speed up. PONG keeps its count in the 16-bit variable `DLYCNT` and reduces it by 200 every time the ball hits a paddle, with a floor of about 3,600 [PGM source].
3. **Poll the keyboard once per tick, at least.** PONG's main loop reads the keys, moves the computer's paddle, moves the ball, and then delays; one look at the keys per tick is enough when the tick is about 50 ms. A game with a much longer tick must poll inside the delay, or the controls feel sluggish.

4. **Measure the overhead.** The work between delays (drawing, collision tests) takes cycles too, and when the game has much to do the overhead is a visible part of the tick. Count it with the table above or measure it with the emulator's instruction count.

Timing the drawing itself

Every character drawn costs the strobe, a `NOP` and the store: `STAA HALT` is 5 cycles, `NOP` 2, `STAA 0,X` 6. But the strobe may do more than take 5 cycles; if its purpose is to synchronise the processor with the display circuits it may *stall* the processor until the right moment, and then drawing takes longer than the count says. This book did not measure it. For a game this only means that the delay loops give a *minimum* time per move, and that drawing a large amount (clearing the whole screen) can be noticeably slower than counting suggests.

Summary

- Instruction times are fixed; `DEX/BNE` takes 8 cycles per pass.
- Delay = 8 x COUNT / clock; the clock of the ADM-31 is not verified here.
- The firmware's own tick is gated on bit 1 of PIA 1 port B; no working game uses it.
- Make speed a variable; keep keyboard polling inside delays.

Exercises

1. With a 1 MHz clock, what COUNT gives a 10 ms delay? (8 us per count.)
2. How long is the largest possible single-register delay: `LDX #0` followed by the loop? (Assume 1 MHz.)
3. T08SWEEP erases the marker with `CLRA`, `JSR PUTAT`. What would it look like if you erased *after* moving to the next column instead?
4. Change COUNT in T08SWEEP so that the sweep takes about four seconds at 1 MHz. (Hint: 80 columns.)

Answers. (1) 1,250. (2) 65,536 x 8 us = 524 ms. (3) You would erase the new position, so the marker would never be visible. (4) 4 s / 80 columns = 50 ms per column, so COUNT = 50 ms / 8 us = 6,250 (the overhead per column is about 0.2 ms, negligible).

Chapter 12

The serial ports and the switches

A terminal is a device that talks to a host over a serial line, and the ADM-31 has two serial interfaces: one for the host and one for a printer. A plug-in game does not use either, but a game must leave them in a harmless state, and a plug-in that *does* want to talk to the host (a configuration program, a terminal extension) needs the whole story. This chapter is mostly reference: what the interfaces are, how the firmware sets them up, how it receives and sends characters, and what the switches do. It is also the chapter with the most explicit "this book did not verify" notes, because the serial hardware is the part of the machine that the working plug-ins use least.

The interfaces

Both interfaces are asynchronous communications adapters (ACIAs) of the Motorola 6850 type. This is how the author's own plug-in LINECFG describes them, and the control bytes the firmware writes agree with the 6850's register layout. Each has two registers at consecutive addresses:

interface	control/status	data
host	$7A00	$7A01
printer	$7C00	$7C01

Writing to the first address sets the **control register**; reading it returns the **status register**. The data register is read to receive and written to transmit. (Those are the 6850's conventions; the firmware's code follows them, as the listings below show.)

The control byte

bit	name	meaning
1-0	counter divide	00: divide by 1; 01: by 16; 10: by 64; **11: master reset**
4-2	word select	data bits, parity and stop bits (8 combinations)
6-5	transmit control	00: RTS low, transmit interrupt off; 01: RTS low, transmit interrupt **on**; 10: RTS high, transmit interrupt off; 11: RTS low, send break

bit	name	meaning
7	receive interrupt enable	1: interrupt when a character arrives

The status byte

bit	name	meaning
0	RDRF	a received character is waiting
1	TDRE	the transmitter can take another character
2	DCD	data-carrier-detect line state
3	CTS	clear-to-send line state
4	FE	framing error
5	OVRN	overrun
6	PE	parity error
7	IRQ	the interface is requesting an interrupt

The firmware uses exactly these: it tests bit 0 to receive, bit 1 to transmit, bit 2 for the carrier line, bits 4-6 (the mask $70) for errors, and bit 7 to see which interface interrupted [FW]. The word-select values are those of the 6850 data sheet: 000 is 7 data bits, even parity, 2 stop; 101 is 8 data bits, no parity, 1 stop; and so on. The author's LINECFG notes point out the consequence: there is *no* "7 bits, no parity" setting on this part.

How the firmware initialises them

The first thing the firmware does to each interface is a **master reset**: it writes $43 (whose low two bits are 11) to both control registers. It then computes the working control bytes from the switches [FW]:

```
E8D4                 165  ; Initialise PIA 1 (keyboard, display page, bell).
E8D4  CE 78 00       166  PIAINIT         LDX     #$7800
E8D7  6F 01          167                  CLR     $01,X
E8D9  6F 03          168                  CLR     $03,X
E8DB  86 7F          169                  LDAA    #$7F
E8DD  A7 00          170                  STAA    $00,X
E8DF  86 1D          171                  LDAA    #$1D
E8E1  A7 02          172                  STAA    $02,X
E8E3  86 3C          173                  LDAA    #$3C
E8E5  A7 01          174                  STAA    $01,X
E8E7  A7 03          175                  STAA    $03,X
E8E9  86 01          176                  LDAA    #$01
E8EB  A7 00          177                  STAA    $00,X
E8ED  A7 02          178                  STAA    $02,X
E8EF  86 43          179                  LDAA    #$43
E8F1  B7 7A 00       180                  STAA    ACIA
E8F4  B7 7C 00       181                  STAA    PACIA
E8F7  B6 7E 00       182                  LDAA    SWBNK1
E8FA  16             183                  TAB
E8FB  84 1C          184                  ANDA    #$1C
E8FD  8A C1          185                  ORAA    #$C1
E8FF  97 BF          186                  STAA    ACIACTL
```

```
E901  B7 7A 00       187                 STAA    ACIA
E904  53             188                 COMB
E905  C4 20          189                 ANDB    #$20
E907  58             190                 ASLB
E908  58             191                 ASLB
E909  D7 CE          192                 STAB    $CE
E90B  B6 79 00       193                 LDAA    SWBNK2
E90E  84 1C          194                 ANDA    #$1C
E910  8A 41          195                 ORAA    #$41
E912  97 C0          196                 STAA    $C0
E914  B7 7C 00       197                 STAA    PACIA
E917  86 17          198                 LDAA    #$17
E919  CE 80 00       199                 LDX     #$8000
E91C  DF 11          200                 STX     CRTBASE
E91E  DF 13          201                 STX     CURADR
E920  97 15          202                 STAA    FIRSTLN
E922  FF 87 D0       203                 STX     $87D0
```

Follow the host interface. `LDAA SWBNK1` reads switch bank 1 ($7E00); `ANDA #$1C` keeps bits 4-2, the word-select bits; `ORAA #$C1` adds the fixed bits: counter divide 01 (÷16), transmit-control 10 (RTS high, transmit interrupts off) and receive-interrupt enable. The result goes to the firmware's copy of the control byte (`ACIACTL`, zero-page $BF) *and* to the interface itself. So **the word format is set by switch bank 1 bits 4-2** [FW], directly.

The printer interface is treated the same way, with switch bank 2 ($7900): `ANDA #$1C`, then `ORAA #$41` (the same fixed bits except receive-interrupt enable), stored in $C0 and written to $7C00.

The baud rate is not set by software. The firmware writes the same counter-divide value regardless of the switches, and the author's LINECFG notes record the same finding: the line speed must come from an external clock chosen by the hardware, which software does not control. The one place the firmware uses the baud switch at $7E80 is for a *delay*: a 16-entry table of counts, indexed by the low four bits of $7E80:

```
FD75                2571  ; Delay counts (ticks) indexed by the baud switch:
FD75                2572  ; low nibble of $7E80.
FD75  AC 73 58 40   2573  BAUDTAB         FCB     $AC,$73,$58,$40,$3A,$1D
```

(The values fall from $AC to $01: the faster the line, the shorter the delay. The firmware uses the count as the number of scheduler ticks to wait before it re-enables the transmitter, in `TXDELAY`.)

Receiving: the interrupt service routine

Characters arrive on an **interrupt**. When the ACIA has a character it asserts the processor's IRQ line; the processor looks up the vector at $FFF8 (in this firmware, $ED83) and runs:

```
ED83                571  ; IRQ service: ACIA first, then the printer ACIA.
ED83  F6 7A 00      572  IRQ             LDAB    ACIA
ED86  2A 02         573                  BPL     LED8A
ED88  8D 09         574                  BSR     ACIAIRQ
ED8A  F6 7C 00      575  LED8A           LDAB    PACIA
ED8D  2A 03         576                  BPL     LED92
ED8F  BD FF AB      577                  JSR     LFFAB
ED92  3B            578  LED92           RTI
ED93  C5 01         579  ACIAIRQ         BITB    #$01
ED95  27 58         580                  BEQ     LEDEF
ED97  B6 7A 01      581                  LDAA    ACIAD
ED9A  84 7F         582                  ANDA    #$7F
```

```
ED9C  C4 70          583                ANDB    #$70
ED9E  27 09          584                BEQ     LEDA9
EDA0  C5 10          585                BITB    #$10
EDA2  27 03          586                BEQ     LEDA7
EDA4  D7 A8          587                STAB    $A8
EDA6  39             588                RTS
EDA7  8A 80          589  LEDA7         ORAA    #$80
EDA9  7D 00 A8       590  LEDA9         FCB     $7D,$00,$A8 ; TST $00A8
EDAC  27 04          591                BEQ     LEDB2
EDAE  7F 00 A8       592                FCB     $7F,$00,$A8 ; CLR $00A8
EDB1  39             593                RTS
EDB2  7D 00 A6       594  LEDB2         FCB     $7D,$00,$A6 ; TST $00A6
EDB5  2B 31          595                BMI     LEDE8
EDB7  DE A3          596                LDX     $A3
EDB9  A7 00          597                STAA    $00,X
EDBB  08             598                INX
EDBC  8C 03 80       599                CPX     #$0380
EDBF  26 03          600                BNE     LEDC4
EDC1  CE 01 10       601                LDX     #$0110
EDC4  DF A3          602  LEDC4         STX     $A3
EDC6  4D             603                TSTA
EDC7  2A 0D          604                BPL     LEDD6
EDC9  E7 00          605                STAB    $00,X
EDCB  08             606                INX
EDCC  8C 03 80       607                CPX     #$0380
EDCF  26 03          608                BNE     LEDD4
EDD1  CE 01 10       609                LDX     #$0110
EDD4  DF A3          610  LEDD4         STX     $A3
EDD6  08             611  LEDD6         INX
EDD7  8C 03 80       612                CPX     #$0380
EDDA  26 03          613                BNE     LEDDF
EDDC  CE 01 10       614                LDX     #$0110
EDDF  9C A1          615  LEDDF         CPX     $A1
EDE1  26 04          616                BNE     LEDE7
EDE3  86 80          617                LDAA    #$80
EDE5  97 A6          618                STAA    $A6
EDE7  39             619  LEDE7         RTS
EDE8  86 80          620  LEDE8         LDAA    #$80
EDEA  97 A7          621                STAA    $A7
EDEC  7E F0 DA       622                JMP     BELL
EDEF  C5 02          623  LEDEF         BITB    #$02
EDF1  27 0B          624                BEQ     LEDFE
EDF3  96 BF          625                LDAA    ACIACTL
EDF5  84 60          626                ANDA    #$60
EDF7  81 20          627                CMPA    #$20
EDF9  26 03          628                BNE     LEDFE
EDFB  7E FD 85       629                JMP     LFD85
EDFE  C5 04          630  LEDFE         BITB    #$04
EE00  27 03          631                BEQ     LEE05
EE02  B6 7A 01       632                LDAA    ACIAD
EE05  39             633  LEE05         RTS
```

`IRQ` reads the host status, tests bit 7 (`BPL` skips if not set), and calls `ACIAIRQ` if the host interrupted; then it reads the printer's status and calls the printer's service routine if that interface interrupted; finally `RTI`. `ACIAIRQ` is the receive routine: if RDRF is set it reads the data, masks it to seven bits, tests the error bits, and stores the character into a **ring buffer** in RAM, wrapping from $0380 back to $0110 [FW]:

- the write pointer is the 16-bit value at zero-page $A3, the read pointer at $A1;
- the buffer occupies $0110-$037F;
- if the write pointer catches the read pointer, the buffer is full and the firmware sets a flag ($A6) and, if more arrive, rings the bell ($A7 and `JMP BELL`).

That bell is the "buffer overflow" alarm of the terminal.

The *consumer* of the ring buffer is the scheduler task that blocks in the routine `GETRX` (Chapter 27); the escape-sequence interpreter calls `GETRX` whenever it needs the next character.

Transmitting

When a task wants to send, it places the character in the transmit queue ($0100-$010B, with pointers at $C7 and $C9 and a count at $CB [FW]) and arranges a transmit interrupt. `IRQ`'s service code for TDRE (bit 1, with control bits 6-5 equal to 01) jumps to the transmit routine. A routine at $FD27, which this book names `TXDELAY`, waits until the queue is empty and the ACIA is ready, waits the baud-dependent delay, rewrites the control byte and wakes two tasks: the processor sends characters at the pace the line can take them.

The details of the transmit path are not needed by a plug-in that owns the machine, so they are left to Chapter 30, which walks through the code.

What the plug-ins do about the interfaces

Rubout and Shuttle Attack, two of the working plug-ins, execute, early in their start-up, two writes that you should now recognise:

```
        LDAA    #$03
        STAA    $7A00           ; host ACIA: master reset
        STAA    $7C00           ; printer ACIA: master reset
```

The master reset (control byte with bits 1-0 = 11) clears the interface and, importantly, **turns off its interrupt request** [PGM]. A game running with interrupts masked (`SEI`) never sees an interrupt anyway; the reset is belt and braces, and it also stops the interface from holding a pending request across the game. PONG does the same silencing by never enabling interrupts at all and never touching the ACIAs; since it started from the firmware's own state, the receive interrupt remains *armed* but masked.

If a game wanted to run with interrupts enabled (to receive host commands during play), it would need its own interrupt routine, the IRQ vector at $FFF8 is in the firmware ROM and cannot be changed, and the vector points to the firmware's $ED83. A plug-in can only share the firmware's interrupt handling, not replace it, which is one reason the games mask interrupts.

Talking to the host from a plug-in: the LINECFG technique

The author's LINECFG plug-in is a menu that lets the operator change terminal settings (duplex, conversation or block mode, personality prefix and so on). To make the changes it does not touch the ACIAs for most of them; it *sends the real escape sequences through the firmware's own processor*. The routine is four instructions, from the author's own listing:

```
E0CB  DE 67           120           LDX     ESCVEC
E0CD  AD 00           121           JSR     0,X
E0CF  39              122           RTS
```

`ESCVEC` is zero-page $67, one entry of the scheduler's task table. With the plug-in flag clear, the firmware stores there the address of its keyboard-handler task (`KEYHAND`, Chapter 28), whose job is to take a character in A and process it as though it had been typed. So `LDX $67 / JSR 0,X` hands a character to the firmware as if the operator had pressed that key. The plug-in sends `ESC` ($1B) then, for example, `C` ($43), and the firmware does what ESC C does. The author notes that this was "confirmed by direct trace"; the code above is the whole mechanism.

For the settings that cannot be done through escape sequences (parity, data bits, stop bits), LINECFG rewrites the ACIA control byte itself:

```
E217  B6 01 23       294          LDAA    WORDSELV
E21A  48             295          ASLA
E21B  48             296          ASLA
E21C  8A C0          297          ORAA    #$C0
E21E  BA 01 2A       298          ORAA    BAUDV
E221  B7 7A 00       299          STAA    ACIA
E224  39             300          RTS
E225  81 68          301  KD_H    CMPA    #$68                    ; 'h' -> ste>
E227  26 18          302          BNE     KD_I
E229  B6 01 2A       303          LDAA    BAUDV
```

The word-select field is shifted into bits 4-2 (`ASLA`, `ASLA`), the fixed bits $C0 added (receive-interrupt enable and transmit control = 10, which, as the author notes, the stock firmware also holds constant) and the counter divide ORed in. The result is stored in the ACIA's control register. That is the register-level complement of what the firmware's initialisation does.

The switches

The switch banks are read-only registers: reading the address returns the switch settings.

address	read as	used by the firmware for
$7E00	switch bank 1	bits 4-2: host word format; bit 1: CRT timing values (Chapter 6); bits 7-6: copied to $41
$7900	switch bank 2	bits 4-2: printer word format
$7D00	switch bank 3	bit 7: ignore the plug-in at boot
$7E80	baud switch	low nibble: index of the delay table

What each switch is *labelled* on the circuit board or the rear panel is not recorded in the firmware code, and this book does not reproduce a switch chart. Use the terminal's documentation for that and treat the meanings above as the ones verified.

Summary

- Both serial interfaces are 6850-type ACIAs at $7A00 and $7C00; control/status at the base, data at base+1.
- Writing $03 resets one; the firmware resets with $43 and then writes (switches & $1C) | $C1.
- Word format comes from the switches; baud is hardware.
- Receive is interrupt driven into a ring buffer at $0110-$037F; the IRQ vector at $FFF8 points into the firmware.
- A plug-in sends data to the firmware's ESC processor through the vector at $67.

Exercises

1. What control byte does the firmware compute for the host interface if switch bank 1 reads $14? (Mask with $1C, OR with $C1.)
2. Why does the firmware test bit 7 of the status byte before it calls the receive routine?
3. Look at the baud-delay table. Which entry would the firmware use with the baud switch at 5? Which at 15?
4. A plug-in wants to ring the terminal's bell by sending BEL to the firmware's processor. What do the instructions look like using ESCSEND?

Answers. (1) $14 AND $1C = $14; OR $C1 = $D5. (2) Bit 7 is the IRQ flag: it says *this* interface interrupted, which tells the shared handler which of the two to service. (3) The sixth entry ($1D) for 5; the last ($01) for 15 (the table starts at index 0). (4) `LDAA #$07 / JSR ESCSEND`, where ESCSEND is `LDX $67 / JSR 0,X / RTS`.

Part III

Techniques, and a whole game

Chapter 13

Subroutines, the stack and a small library

The programs so far have repeated themselves: five copies of the draw-a-letter sequence, four copies of the add-and-store pattern. A **subroutine** is code written once and used from many places. This chapter explains exactly how the 6800 calls and returns, what the stack looks like while it does, how to pass information to a subroutine and get an answer back, and which of the book's routines form the small library that every later program includes.

Call and return

`JSR address` ("jump to subroutine") does two things: it **pushes the return address**, the address of the instruction after the `JSR`, on the stack, and it jumps to `address`. `RTS` ("return from subroutine") **pulls** two bytes off the stack into the program counter. The return address pushed is that of the *next instruction*, so execution resumes where it left off.

The best way to understand this is to look. T15STACK uses `TSX`, which copies the stack pointer plus one into X, to print where the stack is in the main program, inside a subroutine, and inside a subroutine called from a subroutine. It also reads back the return address that the processor pushed. The whole program:

```
0000                     2  ; T15STACK - what the stack does.  TSX copies SP+1
0000                     3  ; into X; the program prints it in the main
0000                     4  ; routine, inside two nested subroutines, and
0000                     5  ; after three PSHA instructions, and reads back
0000                     6  ; the return address a JSR left.  Chapter 13.
0000                     7  ;
0000                     8  ; ---- shared scratch variables (RAM $0100-$010F)
0000                     9  ; ----
0000  =0100             10  ROWARG          EQU     $0100     ; row argument
0000  =0101             11  COLARG          EQU     $0101     ; column argument
0000                    12  ; 2 bytes: address work
0000  =0102             13  ADRPTR          EQU     $0102
0000  =0104             14  TMPA            EQU     $0104     ; scratch byte
0000  =0105             15  TMPB            EQU     $0105     ; scratch byte
0000                    16  ; 2 bytes: string pointer
0000  =0106             17  PSRC            EQU     $0106
0000                    18  ; 2 bytes: random state
0000  =0108             19  SEED            EQU     $0108
```

```
0000  =010A           20  TMPX            EQU     $010A     ; 2 bytes: scratch
0000  =010C           21  GKSCAN          EQU     $010C     ; GETKEY: scan code
0000                  22  ; GETKEY: key held flag
0000  =010D           23  GKHELD          EQU     $010D
0000                  24  ; 2 bytes: SP+1 as captured
0000  =0110           25  SPSAVE          EQU     $0110
0000                  26  ; 2 bytes: a return address read from the stack
0000  =0112           27  RETADR          EQU     $0112
0000                  28  ; ---- hardware names used throughout this book ----
0000  =7000           29  HALT            EQU     $7000     ; CRT access strobe
0000                  30  ; keyboard scan / strobe
0000  =7800           31  PIA1            EQU     $7800
0000                  32  ; keyboard modifiers etc
0000  =7802           33  PIA1B           EQU     $7802
0000  =7803           34  PIA1CRB         EQU     $7803     ; bit 3 = bell
0000                  35  ; first-line register
0000  =7F06           36  CRTC6           EQU     $7F06
0000  =7F0C           37  CURCOL          EQU     $7F0C     ; cursor column
0000  =8000           38  CRTRAM          EQU     $8000     ; screen memory
0000                  39  ; ---- plug-in ROM A frame (the same one PONG uses)
0000                  40  ; ----
          ; ... 28 lines of the standard plug-in frame elided ...
E100                  69                  ORG     $E100
E100  0F              70  START           SEI
E101  8E 03 FF        71                  LDS     #$03FF
E104  BD E2 8E        72                  JSR     INITCRT
E107  BD E1 D8        73                  JSR     CLRSCR
E10A  30              74                  TSX               ; X = SP + 1
E10B  FF 01 10        75                  STX     SPSAVE
E10E  86 04           76                  LDAA    #4
E110  CE E1 92        77                  LDX     #LBLMAIN
E113  BD E1 75        78                  JSR     SHOW
E116  BD E1 4E        79                  JSR     LEVEL1    ; two calls deep
E119                  80  ; the return address LEVEL2 found
E119  86 08           81                  LDAA    #8
E11B  B7 01 00        82                  STAA    ROWARG
E11E  86 0A           83                  LDAA    #10
E120  B7 01 01        84                  STAA    COLARG
E123  CE E1 C9        85                  LDX     #LBLRET
E126  BD E2 53        86                  JSR     PUTSTR
E129  86 1A           87                  LDAA    #26
E12B  B7 01 01        88                  STAA    COLARG
E12E  B6 01 12        89                  LDAA    RETADR
E131  BD E2 75        90                  JSR     PUTHEX
E134  B6 01 13        91                  LDAA    RETADR+1
E137  BD E2 75        92                  JSR     PUTHEX
E13A  36              93                  PSHA              ; three pushes ...
E13B  36              94                  PSHA
E13C  36              95                  PSHA
E13D  30              96                  TSX
E13E  FF 01 10        97                  STX     SPSAVE
E141  86 07           98                  LDAA    #7
E143  CE E1 BC        99                  LDX     #LBLPSH
E146  BD E1 75       100                  JSR     SHOW
E149                 101  ; ... and three pulls
E149  32             102                  PULA
E14A  32             103                  PULA
```

```
E14B  32             104                 PULA
E14C  20 FE          105  FOREVER        BRA     FOREVER
E14E                 106  ; LEVEL1 - one call deep: the return address is on
E14E                 107  ; the stack
E14E  30             108  LEVEL1         TSX
E14F  FF 01 10       109                 STX     SPSAVE
E152  86 05          110                 LDAA    #5
E154  CE E1 9F       111                 LDX     #LBL1
E157  BD E1 75       112                 JSR     SHOW
E15A                 113  ; call one level deeper
E15A  BD E1 5E       114                 JSR     LEVEL2
E15D  39             115                 RTS
E15E  30             116  LEVEL2         TSX
E15F  FF 01 10       117                 STX     SPSAVE
E162                 118  ; the byte pushed last ...
E162  A6 00          119                 LDAA    0,X
E164  B7 01 12       120                 STAA    RETADR
E167                 121  ; ... and the one before it
E167  A6 01          122                 LDAA    1,X
E169  B7 01 13       123                 STAA    RETADR+1
E16C  86 06          124                 LDAA    #6
E16E  CE E1 AD       125                 LDX     #LBL2
E171  BD E1 75       126                 JSR     SHOW
E174  39             127                 RTS
E175                 128  ; SHOW - A = row, X = label text.  Prints the label
E175                 129  ; at column 10 and SPSAVE as four hex digits at
E175                 130  ; column 26
E175  B7 01 00       131  SHOW            STAA    ROWARG
E178  86 0A          132                 LDAA    #10
E17A  B7 01 01       133                 STAA    COLARG
E17D  BD E2 53       134                 JSR     PUTSTR
E180  86 1A          135                 LDAA    #26
E182  B7 01 01       136                 STAA    COLARG
E185  B6 01 10       137                 LDAA    SPSAVE
E188  BD E2 75       138                 JSR     PUTHEX
E18B  B6 01 11       139                 LDAA    SPSAVE+1
E18E  BD E2 75       140                 JSR     PUTHEX
E191  39             141                 RTS
E192  4D 41 49 4E    142  LBLMAIN         FCB     "MAIN PROGRAM",0
E19F  4F 4E 45 20    143  LBL1            FCB     "ONE CALL DEEP",0
E1AD  54 57 4F 20    144  LBL2            FCB     "TWO CALLS DEEP",0
E1BC  41 46 54 45    145  LBLPSH          FCB     "AFTER 3 PSHA",0
E1C9  52 45 54 55    146  LBLRET          FCB     "RETURN ADDRESS",0
E1D8                 147  ; CLRSCR - fill the 1920 visible cells with blanks
E1D8  CE 80 00       148  CLRSCR          LDX     #CRTRAM   ; first cell
E1DB  4F             149                 CLRA              ; blank = $00
E1DC  B7 70 00       150  CS_LP           STAA    HALT      ; sync with the CRT
E1DF  01             151                 NOP               ; settle
E1E0  A7 00          152                 STAA    0,X       ; blank one cell
E1E2  08             153                 INX               ; next cell
E1E3                 154  ; past the last cell?
E1E3  8C 87 80       155                 CPX     #CRTRAM+1920
E1E6  26 F4          156                 BNE     CS_LP     ; no: repeat
E1E8  39             157                 RTS
E1E9                 158  ; CALCADR - X = CRTRAM + ROWARG*80 + COLARG
E1E9                 159  ; Uses 80 = 64 + 16: shift row left 4 (x16),
E1E9                 160  ; keep it, shift 2 more (x64), add the two.
```

```
E1E9  7F 01 02       161  CALCADR         CLR     ADRPTR    ; high byte = 0
E1EC  B6 01 00       162                  LDAA    ROWARG
E1EF  B7 01 03       163                  STAA    ADRPTR+1  ; value = row
E1F2  78 01 03       164                  ASL     ADRPTR+1  ; x2
E1F5  79 01 02       165                  ROL     ADRPTR
E1F8  78 01 03       166                  ASL     ADRPTR+1  ; x4
E1FB  79 01 02       167                  ROL     ADRPTR
E1FE  78 01 03       168                  ASL     ADRPTR+1  ; x8
E201  79 01 02       169                  ROL     ADRPTR
E204  78 01 03       170                  ASL     ADRPTR+1  ; x16
E207  79 01 02       171                  ROL     ADRPTR
E20A  B6 01 02       172                  LDAA    ADRPTR    ; A:B = row*16
E20D  F6 01 03       173                  LDAB    ADRPTR+1
E210  36             174                  PSHA              ; save row*16
E211  37             175                  PSHB
E212  78 01 03       176                  ASL     ADRPTR+1  ; x32
E215  79 01 02       177                  ROL     ADRPTR
E218  78 01 03       178                  ASL     ADRPTR+1  ; x64
E21B  79 01 02       179                  ROL     ADRPTR
E21E  33             180                  PULB              ; B = low(row*16)
E21F  FB 01 03       181                  ADDB    ADRPTR+1  ; + low(row*64)
E222  F7 01 03       182                  STAB    ADRPTR+1
E225  32             183                  PULA              ; A = high(row*16)
E226  B9 01 02       184                  ADCA    ADRPTR    ; + high(row*64)
E229  B7 01 02       185                  STAA    ADRPTR    ; row*80
E22C  B6 01 01       186                  LDAA    COLARG
E22F  BB 01 03       187                  ADDA    ADRPTR+1  ; + column
E232  B7 01 03       188                  STAA    ADRPTR+1
E235  B6 01 02       189                  LDAA    ADRPTR
E238  89 00          190                  ADCA    #0
E23A  8B 80          191                  ADDA    #$80      ; + $8000
E23C  B7 01 02       192                  STAA    ADRPTR
E23F  FE 01 02       193                  LDX     ADRPTR    ; X = address
E242  39             194                  RTS
E243                 195  ; PUTAT - write A at (ROWARG,COLARG)
E243                 196  ; save the character
E243  B7 01 04       197  PUTAT           STAA    TMPA
E246  BD E1 E9       198                  JSR     CALCADR   ; X = cell address
E249  B6 01 04       199                  LDAA    TMPA
E24C  B7 70 00       200                  STAA    HALT      ; sync with the CRT
E24F  01             201                  NOP               ; settle
E250  A7 00          202                  STAA    0,X       ; write the cell
E252  39             203                  RTS
E253                 204  ; PUTSTR - print the zero-terminated string at X
E253                 205  ; starting at (ROWARG,COLARG); COLARG advances
E253  FF 01 06       206  PUTSTR          STX     PSRC      ; save pointer
E256  FE 01 06       207  PS_LP           LDX     PSRC
E259  A6 00          208                  LDAA    0,X       ; next character
E25B                 209  ; zero ends the string
E25B  27 0C          210                  BEQ     PS_X
E25D  08             211                  INX
E25E  FF 01 06       212                  STX     PSRC
E261  BD E2 43       213                  JSR     PUTAT     ; draw it
E264  7C 01 01       214                  INC     COLARG    ; next column
E267  20 ED          215                  BRA     PS_LP
E269  39             216  PS_X            RTS
E26A                 217  ; HEXDIG - low nibble of A -> ASCII '0'-'9','A'-'F'
```

```
E26A  84 0F          218  HEXDIG          ANDA    #$0F
E26C  81 0A          219                  CMPA    #$0A
E26E  2D 02          220                  BLT     HD_1      ; 0-9
E270  8B 07          221                  ADDA    #$07      ; skip ':' to '@'
E272  8B 30          222  HD_1            ADDA    #$30
E274  39             223                  RTS
E275                 224  ; PUTHEX - print A as two hex digits at
E275                 225  ; (ROWARG,COLARG); COLARG advances by 2
E275  36             226  PUTHEX          PSHA              ; keep the byte
E276  44             227                  LSRA              ; high nibble ...
E277  44             228                  LSRA
E278  44             229                  LSRA
E279  44             230                  LSRA
E27A  BD E2 6A       231                  JSR     HEXDIG
E27D  BD E2 43       232                  JSR     PUTAT     ; ... first digit
E280  7C 01 01       233                  INC     COLARG
E283  32             234                  PULA
E284  BD E2 6A       235                  JSR     HEXDIG    ; low nibble
E287  BD E2 43       236                  JSR     PUTAT
E28A  7C 01 01       237                  INC     COLARG
E28D  39             238                  RTS
E28E                 239  ; INITCRT - the start-up the working games share:
E28E                 240  ; memory row 0 becomes the top screen line, the
E28E                 241  ; cursor is parked off screen, page 1 is shown
E28E  86 17          242  INITCRT         LDAA    #$17
E290  B7 7F 06       243                  STAA    CRTC6     ; unscrolled
E293  43             244                  COMA              ; A = $E8
E294                 245  ; cursor beyond column 79
E294  B7 7F 0C       246                  STAA    CURCOL
E297  B6 78 02       247                  LDAA    PIA1B
E29A  84 FB          248                  ANDA    #$FB      ; bit 2 low: page 1
E29C  B7 78 02       249                  STAA    PIA1B
E29F  39             250                  RTS
```

The structure to notice is the nesting: `START` calls `LEVEL1`, which calls `LEVEL2`, and each returns with `RTS`. The program's output, from the emulator:

```
MAIN PROGRAM    0400
ONE CALL DEEP   03FE
TWO CALLS DEEP  03FC
AFTER 3 PSHA    03FD
RETURN ADDRESS  E15D
```

Read the numbers:

- In the main program the stack pointer is at its starting value, $03FF, so `TSX` gives $0400 (SP + 1).
- In `LEVEL1`, one `JSR` deeper, the value is $03FE: the `JSR` pushed two bytes and SP fell by two.
- In `LEVEL2`, another two: $03FC.
- After three `PSHA`s in the main program, SP is down by three, so `TSX` gives $03FD.
- The return address that `LEVEL2` finds on its stack is the address of the instruction *after* the `JSR LEVEL2` in `LEVEL1`: the listing shows `JSR LEVEL2` followed by one byte, an `RTS`, then the `LEVEL2` label, so the pushed address is one less than `LEVEL2`'s address.

Two more facts follow from the way the program reads the return address with `LDAA 0,X` and `LDAA 1,X` after the `TSX`: the byte at `0,X`, the one pushed last, is the **high** byte of the return ad-

dress, and `1,X` is the low byte. In memory the address sits high byte first at ascending addresses, the same way every 16-bit value on the 6800 does.

The stack discipline

1. SP starts at $03FF (the top of RAM) and moves *down* as things are pushed.
2. Every `JSR` and `PSHA`/`PSHB` must be matched by an `RTS` and a `PULA`/`PULB`. An unmatched push before an `RTS` makes the `RTS` pull the wrong two bytes and jump to nonsense.
3. The stack also holds **temporary values**. That is how `CALCADR` saves a partial product (`PSHA` and `PSHB`, later `PULB` and `PULA`: in reverse order, because the stack is last-in, first-out).
4. Depth is limited only by how much RAM lies below SP; there is no check. Each nested call costs two bytes. The firmware and the working plug-ins use at most a handful of levels, so a 1 KB RAM is generous; just be careful with *recursion*, which this book does not use.

BSR, JMP and a tail call

`BSR` is "branch to subroutine": the same as `JSR` but with a relative target, one byte shorter (2 bytes against 3) and limited to a nearby routine. The firmware uses it for short helpers; the book's code mostly uses `JSR`.

A tail call saves a return: if the last thing a routine does is call another routine and return, replace `JSR other / RTS` by `JMP other`. The `other` routine's `RTS` then returns directly to the original caller. The firmware's `BLITCPY` entry in one of its helpers is exactly this (`JMP BLITCPY` at the end of the routine `LE9CA`).

Passing arguments

A subroutine needs inputs and produces outputs. The book uses three ways.

In registers. A, B and X carry values in and out. Examples: `HLINE` takes the character in A, the count in B and the first cell in X. `HEXDIG` takes a value in A and returns a digit in A. Registers are the fastest way, but there are only three of them.

In fixed RAM locations. The drawing routines use `ROWARG` and `COLARG` ($0100, $0101) for the position. It is clumsy but simple, it leaves the registers free, and a routine that can be called with the position *left over from the last call* (as `PUTSTR` does by incrementing `COLARG`) benefits.

On the stack. A caller can push arguments before the `JSR` and the callee reads them with `TSX` and indexed addressing (as T15STACK reads the return address). This is how languages pass arguments but it is expensive on the 6800 (there is no direct way to address relative to SP), and no program in this book does it.

Saving registers

A routine that changes a register its caller expects to keep must save it. The 6800 has no "push all". The convention in this book is that **every library routine may change A, B and X** unless its description says otherwise, and the *caller* saves whatever it needs: `PUTAT`, for example, destroys X (it returns the cell address in it), so a loop that uses X as a pointer across a `PUTAT` call must save X in RAM. The T04 and T09 programs do that.

The alternative is for each routine to save what it changes on entry and restore it on exit. That costs time on every call whether or not the caller cared. For a language with 3 registers the "caller saves" rule is cheaper.

The library

Every program from T02 on pulls these routines in. Each is a few lines to a page; each has been assembled and tested. All are shown in this part of the book, in the chapter that first uses them.

routine	in	out	changes	chapter
`CLRSCR`		screen = $00	A, X	8
`HLINE`	A char, B count, X cell		B, X	8
`CALCADR`	`ROWARG`, `COLARG`	X = cell address	A, B, X, `ADRPTR`	8
`PUTAT`	A char, `ROWARG`, `COLARG`		A, X	8
`GETAT`	`ROWARG`, `COLARG`	A = char	A, X	8
`PUTSTR`	X string, `ROWARG`, `COLARG`	`COLARG` advanced	A, X, `PSRC`	8
`HEXDIG`	A (low nibble)	A = ASCII digit	A	8
`PUTHEX`	A byte, position	`COLARG` + 2	A, X	8
`INITCRT`		CRT registers set	A	8
`XPLUSA`	A, X	X = X + A	A, X	9
`GETKEY`		A = new character, or 0	A, B, X	9
`BEEP`		bell pulsed	A	10
`RAND`	`SEED`	A = random byte	A	19

The shared scratch variables, at $0100-$010D, are:

```
; ---- shared scratch variables (RAM $0100-$010F) ----
ROWARG          EQU     $0100           ; row argument
COLARG          EQU     $0101           ; column argument
ADRPTR          EQU     $0102           ; 2 bytes: address work
TMPA            EQU     $0104           ; scratch byte
TMPB            EQU     $0105           ; scratch byte
PSRC            EQU     $0106           ; 2 bytes: string pointer
SEED            EQU     $0108           ; 2 bytes: random state
TMPX            EQU     $010A           ; 2 bytes: scratch
GKSCAN          EQU     $010C           ; GETKEY: scan code
GKHELD          EQU     $010D           ; GETKEY: key held flag
```

A program's *own* variables live above them, starting at $0110 in the tutorials.

Dependencies

Some routines call others: `PUTAT` calls `CALCADR`; `PUTHEX` calls `HEXDIG` and `PUTAT`; `GETKEY` calls `XPLUSA` and needs the table `KBDTAB`. A program that includes `PUTHEX` must therefore include all of them; the source files in this book list the include order explicitly (`;@LIB name`), and the finished listing contains every routine that was used.

A note on XPLUSA

The 6800 has no instruction to add a register to X. The firmware has a routine for it at $EFAD, which Chapter 23 calls directly. The tutorials carry their own copy so that they run on any firmware set:

```
E2D3                236  ; XPLUSA - X = X + A (unsigned), the same job as
E2D3                237  ; the firmware routine at $EFAD
E2D3  FF 01 0A      238  XPLUSA          STX     TMPX
E2D6  BB 01 0B      239                  ADDA    TMPX+1
E2D9  B7 01 0B      240                  STAA    TMPX+1
E2DC  B6 01 0A      241                  LDAA    TMPX
E2DF  89 00         242                  ADCA    #0
E2E1  B7 01 0A      243                  STAA    TMPX
E2E4  FE 01 0A      244                  LDX     TMPX
E2E7  39            245                  RTS
```

It stores X in RAM, adds A to the low byte, carries into the high byte, and loads X back. Seven instructions where a better processor would have one.

Writing a good subroutine

Some habits that pay off in assembly:

1. **Put a comment above every routine** saying what it takes, what it returns and what it changes. The library routines in this book do, and you will see the same style in the firmware listing's comments.
2. **One entry, one exit.** Several RTSs are fine, but the reader (and the stack) is happier with one.
3. **Keep routines short.** A routine that does one job can be tested with a four-line driver.
4. **Name by what it does,** not by how: CALCADR, not SHIFTLOOP.
5. **Do not leave flags to chance** at the return point if the caller will branch on them; load a result register last, so that the caller can test A directly with BEQ, or return the flag you mean it to.

Summary

- JSR pushes the return address (high byte at the lower address, pushed last); RTS pulls it.
- SP starts at $03FF and falls two bytes per nested call.
- Pass arguments in registers, in fixed RAM, or on the stack; the book uses the first two.
- The caller saves the registers it needs; library routines may change A, B and X.
- The book's library: CLRSCR HLINE CALCADR PUTAT GETAT PUTSTR HEXDIG PUTHEX INITCRT XPLUSA GETKEY BEEP RAND.

Exercises

1. In T15STACK, what would TSX give inside LEVEL1 if START had executed one more PSHA before calling it?
2. A routine begins PSHA / PSHB and ends PULA / PULB / RTS. What is wrong?
3. Rewrite JSR LEVEL2 / RTS at the end of LEVEL1 as a tail call. What changes on the stack?
4. Write SWAP, a routine that exchanges A and B using the stack.

Answers. (1) $03FD instead of $03FE: one byte lower. (2) The pulls are in the wrong order: the last thing pushed (B) must be pulled first, so A and B come back swapped. (3) `JMP LEVEL2`: `LEVEL1` no longer holds a return address of its own while `LEVEL2` runs, so the stack is two bytes shallower and `LEVEL2`'s `TSX` shows $03FE. (4) `PSHA / PSHB / PULA / PULB / RTS`: pushing A then B and pulling into A then B swaps them.

Chapter 14

Strings and text

Text is the first thing every program needs to show and the first place assembly language feels laborious: there is no string type, no `PRINT` and no `strlen`. A string is bytes in memory, and everything else is a convention about how to find its end. This chapter covers the three conventions you will meet, builds the basic string operations (length, copy, compare, case conversion), centres text on the screen, and shows a table of strings reached through a table of pointers.

How to end a string

A routine that prints or copies a string has to know when to stop. Three conventions are common:

convention	how the end is marked	cost	used by
zero-terminated	a $00 byte after the last character	one extra byte; no length known without scanning	every program in this book
counted	a length is stored or passed separately	must keep the count with the data	the firmware's messages
bit-7 terminated	bit 7 set in the last character	no extra byte; limits characters to 7 bits	not used here

The firmware's own messages are of the second kind. "Test Complete" and "MEMORY ERROR" are both exactly 13 characters long, back to back in ROM, and the code that prints them passes the length 13 in B:

```
E9B0                 263  ; Self-test messages (13 characters each).
E9B0  54 65 73 74    264  TSTMSG          FCB     "Test Complete"
E9BD  4D 45 4D 4F    265  ERRMSG          FCB     "MEMORY ERROR "
E9CA  DF B3          266  LE9CA           STX     $B3
E9CC  C6 0D          267                  LDAB    #$0D
E9CE  DE 13          268                  LDX     CURADR
E9D0  DF B1          269                  STX     $B1
E9D2  7E F6 BB       270                  JMP     BLITCPY
```

The routine `LE9CA` loads the count `$0D` into B and falls into `BLITCPY`, which copies B bytes (Chapter 23); the caller passes the text address in X. That is a *counted* string: the length is a constant in the code, not stored with the text.

Zero-termination is the book's choice because it is the most flexible: a table of messages of different lengths needs nothing but a zero after each. The assembler writes it for you:

```
MSG     FCB     "HELLO",0
```

The zero must be the **last byte**, and no message may contain a zero byte in the middle. Zero is the code for a blank cell on this screen (Chapter 8), which you would not want in the middle of a message anyway.

Counting characters: STRLEN

To centre a string you need its length. The routine scans for the zero:

```
E148                    102  ; STRLEN - X = string, returns A = length
E148  FF 01 02          103  STRLEN          STX     ADRPTR
E14B  5F                104                  CLRB
E14C  FE 01 02          105  SL_LP           LDX     ADRPTR
E14F  A6 00             106                  LDAA    0,X
E151  27 07             107                  BEQ     SL_X
E153  08                108                  INX
E154  FF 01 02          109                  STX     ADRPTR
E157  5C                110                  INCB
E158  20 F2             111                  BRA     SL_LP
E15A  17                112  SL_X            TBA
E15B  39                113                  RTS
```

The pointer lives in the RAM word `ADRPTR` (borrowed from the address routine, since the two are never in use at once; a routine that borrows another's scratch variable must say so loudly in a comment, because an unexpected overlap is a typical hard-to-find bug). B counts characters. When the zero is found, `TBA` copies the count into A so the caller receives it in A.

Centring text

The screen is 80 columns wide. A string of length *n* is centred when it starts in column (80 - *n*) / 2:

```
        LDAA    #80
        SUBA    SLEN            ; 80 - length
        LSRA                    ; divide by 2
        STAA    COLARG
```

`LSRA` is a logical shift right: the divide by two, with the odd leftover dropped. (A string of odd length cannot be exactly centred on an even width.) With the column in `COLARG` and the row in `ROWARG`, `PUTSTR` draws it.

A table of strings

To print several messages you do not repeat the code: you keep a **table of pointers**, one 16-bit address per string, and walk the table. The table in T04TEXT is made with `FDB`:

```
TABLE   FDB     S1,S2,S3,S4,0
S1      FCB     "LEAR SIEGLER ADM-31",0
S2      FCB     "MC6800 ASSEMBLY",0
S3      FCB     "PLUG-IN ROM A",0
S4      FCB     "CENTRED BY THE PROGRAM",0
```

`FDB` places one 16-bit word per operand, high byte first (the 6800's byte order, Chapter 2). The table ends with a zero word, the equivalent of a string's terminating zero. The program's main loop:

```
E100  0F              68  START           SEI
E101  8E 03 FF        69                  LDS     #$03FF
E104  BD E2 41        70                  JSR     INITCRT
E107  BD E1 AF        71                  JSR     CLRSCR
E10A                  72  ; table of string addresses
E10A  CE E1 5C        73                  LDX     #TABLE
E10D  FF 01 12        74                  STX     TABPTR
E110  86 04           75                  LDAA    #4        ; first row
E112  B7 01 00        76                  STAA    ROWARG
E115  7F 01 10        77                  CLR     LINENO
E118  FE 01 12        78  LINELP          LDX     TABPTR
E11B                  79  ; X = address of next string
E11B  EE 00           80                  LDX     0,X
E11D                  81  ; zero address ends the table
E11D  27 27           82                  BEQ     DONE
E11F  BD E1 48        83                  JSR     STRLEN    ; A = length
E122  B7 01 11        84                  STAA    SLEN
E125  86 50           85                  LDAA    #80
E127  B0 01 11        86                  SUBA    SLEN
E12A  44              87                  LSRA              ; (80 - len) / 2
E12B  B7 01 01        88                  STAA    COLARG
E12E  FE 01 12        89                  LDX     TABPTR
E131                  90  ; reload string address
E131  EE 00           91                  LDX     0,X
E133  BD E2 2A        92                  JSR     PUTSTR
E136  7C 01 00        93                  INC     ROWARG
E139  7C 01 00        94                  INC     ROWARG    ; double spacing
E13C                  95  ; advance to next table entry
E13C  FE 01 12        96                  LDX     TABPTR
E13F  08              97                  INX
E140  08              98                  INX
E141  FF 01 12        99                  STX     TABPTR
E144  20 D2          100                  BRA     LINELP
E146  20 FE          101  DONE            BRA     DONE
```

The central idea is in `LDX TABPTR / LDX 0,X`. `TABPTR` holds the *address of a table entry*. The first `LDX` loads X with that address. The second, `LDX 0,X`, loads X with the **16 bits stored at that address**, which is the address of the string. This "pointer to a pointer" is the 6800's way of indexing an array of addresses. `BEQ DONE` tests the zero flag set by that 16-bit load: a zero address ends the table.

To advance to the next entry, X is incremented twice (`INX / INX`), since each entry is two bytes.

The program's screen:

```
                    LEAR SIEGLER ADM-31

                     MC6800 ASSEMBLY

                      PLUG-IN ROM A

                   CENTRED BY THE PROGRAM
```

Each line is centred and the rows are two apart (`INC ROWARG` twice).

The same trick is how the firmware dispatches escape sequences: its ESC table is a table of handler addresses, and the dispatcher indexes it with `LDX 0,X` and `JMP 0,X` (Chapter 28).

Program T16STRING: copy, convert, compare

Three operations are needed constantly: **copy** a string, **change case**, **compare** two strings. T16STRING performs them all and shows the results. The three routines:

```
E1B8                150  ; STRCPY - copy the string at SRCP (with its zero)
E1B8                151  ; to DSTP
E1B8  FE 01 10      152  STRCPY          LDX     SRCP
E1BB  A6 00         153                  LDAA    0,X       ; next character
E1BD  08            154                  INX
E1BE  FF 01 10      155                  STX     SRCP
E1C1  FE 01 12      156                  LDX     DSTP
E1C4  A7 00         157                  STAA    0,X
E1C6  08            158                  INX
E1C7  FF 01 12      159                  STX     DSTP
E1CA  4D            160                  TSTA              ; was it the zero?
E1CB  26 EB         161                  BNE     STRCPY
E1CD  39            162                  RTS
E1CE                163  ; UPCASE - X = string; a-z become A-Z in place
E1CE  FF 01 10      164  UPCASE          STX     SRCP
E1D1  FE 01 10      165  UC_LP           LDX     SRCP
E1D4  A6 00         166                  LDAA    0,X
E1D6  27 12         167                  BEQ     UC_X      ; zero: finished
E1D8  81 61         168                  CMPA    #$61      ; below "a"?
E1DA  25 08         169                  BCS     UC_NEXT
E1DC  81 7B         170                  CMPA    #$7B      ; above "z"?
E1DE  24 04         171                  BCC     UC_NEXT
E1E0  80 20         172                  SUBA    #$20      ; lower to upper
E1E2  A7 00         173                  STAA    0,X
E1E4  08            174  UC_NEXT         INX
E1E5  FF 01 10      175                  STX     SRCP
E1E8  20 E7         176                  BRA     UC_LP
E1EA  39            177  UC_X            RTS
E1EB                178  ; STRCMP - compare the strings at SRCP and DSTP.
E1EB                179  ; Returns Z = 1 if they are equal, Z = 0 if not.
E1EB  FE 01 10      180  STRCMP          LDX     SRCP
E1EE  A6 00         181                  LDAA    0,X
E1F0  08            182                  INX
E1F1  FF 01 10      183                  STX     SRCP
E1F4  FE 01 12      184                  LDX     DSTP
E1F7  E6 00         185                  LDAB    0,X
E1F9  08            186                  INX
E1FA  FF 01 12      187                  STX     DSTP
E1FD  11            188                  CBA               ; A - B
E1FE  26 03         189                  BNE     SC_X      ; different: Z = 0
E200  4D            190                  TSTA              ; equal; both zero?
E201                191  ; no: next characters
E201  26 E8         192                  BNE     STRCMP
E203  39            193  SC_X            RTS
```

STRCPY copies the string at the source pointer to the destination pointer, *including* the terminating zero. It keeps both pointers in RAM (`SRCP`, `DSTP`) because the 6800 has one index register, and loads each into X in turn, as in the loop program of Chapter 7. The test for the end is `TSTA`

after the store: it must come *after* the INXs because INX changes the zero flag, so the flag has to be set again from the character itself.

UPCASE converts in place. It applies the rule the firmware uses for ALPHA LOCK (Chapter 9): a character from $61 (“a”) to $7A (“z”) is made upper-case by subtracting $20. The two range tests are the standard pair:

```
        CMPA    #$61            ; below "a"?
        BCS     UC_NEXT         ; yes: leave it alone
        CMPA    #$7B            ; above "z"?
        BCC     UC_NEXT         ; yes: leave it alone
```

BCS is branch-if-carry-set, i.e. “lower than” for unsigned numbers; BCC is “higher or same”. Together they pick out the interval from $61 up to but not including $7B.

STRCMP compares the strings at SRCP and DSTP one character at a time and returns its answer **in the Z flag**: Z = 1 means they were equal. It loads one character into A and the other into B, CBA subtracts them (setting Z when equal), and BNE leaves with Z = 0 on the first difference. If the characters are equal and also zero, both strings have ended together: TSTA sets Z = 1 and the routine falls out of its loop through RTS. Returning the answer in a flag lets the caller write JSR STRCMP / BEQ SAME, which is the idiom a compiler would use.

The flags hazard, in practice

The first version of this program was wrong, and the error is instructive. The calling code was

```
        JSR     STRCMP
        LDAA    #6              ; row for the label
        LDX     #LBLCMP1
        JSR     LABEL
        JSR     VERDICT         ; BEQ on the flags from STRCMP
```

and the program printed DIFFERENT for strings that were equal. Between STRCMP and VERDICT the program loads A and X and calls LABEL, and every one of those instructions *changes the flags*. By the time VERDICT branched, the flags no longer said anything about the strings. The fix is to do everything that changes flags *before* the comparison, and to call VERDICT immediately after STRCMP:

```
E100  0F             72  START           SEI
E101  8E 03 FF       73                  LDS     #$03FF
E104  BD E3 33       74                  JSR     INITCRT
E107  BD E2 A1       75                  JSR     CLRSCR
E10A                 76  ; row 4: the original, in ROM
E10A  86 04          77                  LDAA    #4
E10C  CE E2 04       78                  LDX     #LBLORG
E10F  BD E1 99       79                  JSR     LABEL
E112  CE E2 79       80                  LDX     #ORIG
E115  BD E3 1C       81                  JSR     PUTSTR
E118                 82  ; copy it to RAM and show the copy on row 5
E118  CE E2 79       83                  LDX     #ORIG
E11B  FF 01 10       84                  STX     SRCP
E11E  CE 01 20       85                  LDX     #BUF
E121  FF 01 12       86                  STX     DSTP
E124  BD E1 B8       87                  JSR     STRCPY
E127  86 05          88                  LDAA    #5
E129  CE E2 14       89                  LDX     #LBLCPY
E12C  BD E1 99       90                  JSR     LABEL
E12F  CE 01 20       91                  LDX     #BUF
```

```
E132  BD E3 1C        92                  JSR     PUTSTR
E135                  93  ; compare the original with the copy: row 6
E135  86 06           94                  LDAA    #6
E137  CE E2 20        95                  LDX     #LBLCMP1
E13A  BD E1 99        96                  JSR     LABEL
E13D  CE E2 79        97                  LDX     #ORIG
E140  FF 01 10        98                  STX     SRCP
E143  CE 01 20        99                  LDX     #BUF
E146  FF 01 12       100                  STX     DSTP
E149  BD E1 EB       101                  JSR     STRCMP    ; flags: Z = equal
E14C                 102  ; (nothing between that changes them)
E14C  BD E1 AA       103                  JSR     VERDICT
E14F                 104  ; upper-case the copy, show it on row 7
E14F  CE 01 20       105                  LDX     #BUF
E152  BD E1 CE       106                  JSR     UPCASE
E155  86 07          107                  LDAA    #7
E157  CE E2 32       108                  LDX     #LBLUP
E15A  BD E1 99       109                  JSR     LABEL
E15D  CE 01 20       110                  LDX     #BUF
E160  BD E3 1C       111                  JSR     PUTSTR
E163                 112  ; compare original with the upper-cased copy: row 8
E163  86 08          113                  LDAA    #8
E165  CE E2 44       114                  LDX     #LBLCMP2
E168  BD E1 99       115                  JSR     LABEL
E16B  CE E2 79       116                  LDX     #ORIG
E16E  FF 01 10       117                  STX     SRCP
E171  CE 01 20       118                  LDX     #BUF
E174  FF 01 12       119                  STX     DSTP
E177  BD E1 EB       120                  JSR     STRCMP    ; flags: Z = equal
E17A                 121  ; (nothing between that changes them)
E17A  BD E1 AA       122                  JSR     VERDICT
E17D                 123  ; compare the upper-cased copy with a constant: row 9
E17D  86 09          124                  LDAA    #9
E17F  CE E2 57       125                  LDX     #LBLCMP3
E182  BD E1 99       126                  JSR     LABEL
E185  CE 01 20       127                  LDX     #BUF
E188  FF 01 10       128                  STX     SRCP
E18B  CE E2 8D       129                  LDX     #UPPER
E18E  FF 01 12       130                  STX     DSTP
E191  BD E1 EB       131                  JSR     STRCMP    ; flags: Z = equal
E194                 132  ; (nothing between that changes them)
E194  BD E1 AA       133                  JSR     VERDICT
```

Look at each of the three compare blocks in the listing. The label is printed first. Then the two pointers are set up, then `JSR STRCMP` and `JSR VERDICT`, back to back. A `JSR` itself does not alter the flags, so the pair works. This ordering rule, *consume a flag in the very next instruction, or save it*, is the same hazard Chapter 2 warned about.

The program's output:

```
ORIGINAL (ROM):              hello, ADM-31 world
COPY (RAM):                  hello, ADM-31 world
ORIGINAL VS COPY:            SAME
UPPER-CASED COPY:            HELLO, ADM-31 WORLD
ORIGINAL VS UPPER:           DIFFERENT
UPPER VS CONSTANT:           SAME
```

The "original versus copy" compare says SAME. The upper-cased copy differs from the original (case differs) and equals the constant `UPPER`. (The listing also shows how `VERDICT` ends with `JMP PUTSTR` rather than `JSR PUTSTR / RTS`: a tail call, Chapter 13.)

Other string work

Things you will want and can now build:

- **Number to text**: Chapter 15 converts numbers to digits.
- **Scrolling text**: print a window of a longer string, moving the start.
- **Search**: a `STRCMP` against each entry of a table gives a menu or command lookup.
- **Fixed-width fields**: since the screen is a grid, many programs print fields at fixed columns and never compute lengths.

Summary

- A string is bytes; the end is a convention. This book uses zero termination.
- `STRLEN` scans for the zero; centring is (80 - length) / 2 with `LSRA`.
- A table of pointers is `FDB` entries; `LDX 0,X` loads the pointed-to address.
- Copy, upper-case and compare are short loops; the test for "lower" and "higher" uses `BCS`/`BCC`.
- Consume flags in the next instruction or save them.

Exercises

1. Where would the string "MC6800 ASSEMBLY" (15 characters) start if centred? ((80 - 15) / 2 = 32.5, rounded down.)
2. Write `LOWCASE`, the inverse of `UPCASE`.
3. Why must `STRCPY`'s `TSTA` come after the `INX`s and not before?
4. Modify `STRCMP` so it is case-insensitive. (Hint: apply the `UPCASE` test to both characters.)

Answers. (1) Column 32. (2) Same loop; test for $41-$5A (`CMPA #$41 / BCS`, `CMPA #$5B / BCC`) and `ADDA #$20`. (3) `INX` sets or clears Z from the pointer, destroying the flag the loop needs; the `TSTA` re-establishes Z from the character just copied. (4) After loading A and B, convert each with a small subroutine that applies the range test and subtracts $20 where it applies; then compare.

Chapter 15

Numbers: arithmetic, decimal and BCD

The 6800 adds and subtracts bytes. It cannot multiply, divide, or print a number. Everything else, a score, a ball speed, a screen position, a countdown shown in decimal digits, is built from those two operations and the shifts. This chapter builds the tools: 16-bit arithmetic with the carry, multiplication by shift and add, division by shift and subtract, two ways to print a number in decimal, and **packed BCD**, the representation the 6800 has special hardware for and that games use for scores.

Multi-byte arithmetic

To add two 16-bit numbers held as high and low bytes, add the low bytes with `ADDA` and the high bytes with `ADCA`:

```
        LDAA    X+1             ; low byte of x
        ADDA    Y+1             ; + low byte of y: may set the carry
        STAA    Z+1
        LDAA    X               ; high byte of x
        ADCA    Y               ; + high byte of y + the carry
        STAA    Z
```

`ADDA` produces a carry out of bit 7 when the sum exceeds 255. `ADCA` adds the carry. The same pair with `SUBA` and `SBCA` subtracts: `SUBA` leaves the **borrow** in the carry flag and `SBCA` takes it into account. All the address arithmetic of Chapter 8 is built this way.

Remember that **loads and stores do not change the carry** (and `INC/DEC` do not), so you can put them between the two halves; but `CLRA` *clears* the carry (it is a store of zero that sets C = 0). That is why `CALCADR` uses `LDAA` rather than `CLRA` between its add and its add-with-carry.

After the 16-bit subtract, the carry flag holds the answer to “was x less than y?”: the final `SBCA` leaves C = 1 if the whole 16-bit subtraction needed a borrow. `BCS` then branches when x < y (unsigned). `PUTDEC`, below, uses this.

Multiplication by shift and add

You multiplied by hand at school like this: for each digit of the multiplier, from the right, add the multiplicand (shifted left one more place each time) if the digit is non-zero. In binary the only

digits are 0 and 1, so there is nothing to multiply at all, only add or skip. T17MATH implements it for 8 x 8 -> 16 bits:

```
E1D1                 167  ; MUL8 - A x B -> PROD (16 bits), by shift and add.
E1D1                 168  ; For each of the eight bits of the multiplier, low
E1D1                 169  ; bit first: if it is 1, add the multiplicand to the
E1D1                 170  ; product; then double the multiplicand.
E1D1                 171  ; multiplicand, low byte
E1D1  B7 01 17       172  MUL8            STAA    MCAND+1
E1D4  7F 01 16       173                  CLR     MCAND     ; ... high byte 0
E1D7  F7 01 15       174                  STAB    MULB
E1DA  7F 01 18       175                  CLR     PROD
E1DD  7F 01 19       176                  CLR     PROD+1
E1E0  C6 08          177                  LDAB    #8        ; eight bits
E1E2                 178  ; low bit to the carry
E1E2  74 01 15       179  MUL_LP          LSR     MULB
E1E5  24 12          180                  BCC     MUL_SKIP  ; 0: nothing to add
E1E7  B6 01 19       181                  LDAA    PROD+1
E1EA  BB 01 17       182                  ADDA    MCAND+1
E1ED  B7 01 19       183                  STAA    PROD+1
E1F0  B6 01 18       184                  LDAA    PROD
E1F3  B9 01 16       185                  ADCA    MCAND
E1F6  B7 01 18       186                  STAA    PROD
E1F9  78 01 17       187  MUL_SKIP        ASL     MCAND+1   ; multiplicand x 2
E1FC  79 01 16       188                  ROL     MCAND
E1FF  5A             189                  DECB
E200  26 E0          190                  BNE     MUL_LP
E202  39             191                  RTS
```

`LSR MULB` shifts the multiplier right, dropping its lowest bit into the carry. If the carry is 1 (`BCC MUL_SKIP` does *not* branch) the multiplicand is added to the 16-bit product. Then the multiplicand is doubled by `ASL MCAND+1 / ROL MCAND`, ready for the next bit. Eight passes, counted in B. The multiplicand must be 16 bits wide because after eight doublings the 8-bit value would overflow.

An example: 7 x 6. The multiplier 6 is `110`. Pass 1: bit 0 = 0, skip, multiplicand becomes 14. Pass 2: bit = 1, add 14 (product 14), multiplicand 28. Pass 3: bit = 1, add 28 (product 42). The remaining bits are zero. The product is 42.

When one factor is a constant, you do not need the loop: multiply by 10 as `x * 8 + x * 2`, three shifts and an add. The address routine's 80 = 64 + 16 is the same idea.

Division by shift and subtract

Long division in binary is simpler than in decimal, because each quotient digit is 0 or 1: shift the dividend's top bit into a running remainder; if the remainder is at least the divisor, subtract and write a 1 in the quotient, else write a 0. `DIV10` divides a 16-bit value by 10 this way, 16 passes:

```
E203                 192  ; DIV10 - VAL (16 bits) / 10: VAL = quotient, A =
E203                 193  ; remainder.  Long division, one bit per pass.
E203  7F 01 1C       194  DIV10           CLR     REM
E206  C6 10          195                  LDAB    #16
E208                 196  ; shift dividend left;
E208  78 01 1B       197  DV_LP           ASL     VAL+1
E20B                 198  ; its top bit goes to carry
```

```
E20B  79 01 1A       199                  ROL     VAL
E20E                 200  ; ... and into the remainder
E20E  79 01 1C       201                  ROL     REM
E211  B6 01 1C       202                  LDAA    REM
E214  81 0A          203                  CMPA    #10
E216  25 08          204                  BCS     DV_NO     ; remainder < 10
E218  80 0A          205                  SUBA    #10
E21A  B7 01 1C       206                  STAA    REM
E21D  7C 01 1B       207                  INC     VAL+1     ; quotient bit = 1
E220  5A             208  DV_NO           DECB
E221  26 E5          209                  BNE     DV_LP
E223  B6 01 1C       210                  LDAA    REM
E226  39             211                  RTS
```

The dividend `VAL` is shifted left with `ASL`/`ROL`; the bit that falls out goes, by the same `ROL`, into the remainder `REM`. The comparison `CMPA #10` / `BCS` skips the subtraction when the remainder is under 10. The quotient bits accumulate in `VAL` itself: after each shift the low bit of `VAL` is free, and `INC VAL+1` sets it to 1 when a subtraction was done. When the 16 passes finish, `VAL` holds the quotient and A the remainder.

Printing a number: the stack reverses it

Dividing a number by 10 repeatedly yields its decimal digits *from the right*, the wrong order for printing. The stack fixes that: push each digit as it is found, then pull them off in reverse. `PUTNUM`:

```
E227                 212  ; PUTNUM - print VAL in decimal, no leading zeros, at
E227                 213  ; (ROWARG,COLARG).  DIV10 gives the digits low first;
E227                 214  ; each is pushed, then they are pulled (high first)
E227                 215  ; and drawn.
E227  7F 01 1D       216  PUTNUM          CLR     NDIG
E22A  BD E2 03       217  PN_LP           JSR     DIV10
E22D  8B 30          218                  ADDA    #$30      ; ASCII digit
E22F  36             219                  PSHA
E230  7C 01 1D       220                  INC     NDIG
E233  B6 01 1A       221                  LDAA    VAL
E236                 222  ; quotient zero yet?
E236  BA 01 1B       223                  ORAA    VAL+1
E239  26 EF          224                  BNE     PN_LP
E23B  32             225  PN_OUT          PULA
E23C  BD E2 4E       226                  JSR     PUTCH
E23F  7A 01 1D       227                  DEC     NDIG
E242  26 F7          228                  BNE     PN_OUT
E244  39             229                  RTS
```

Each pass calls `DIV10`, turns the remainder into an ASCII digit with `ADDA #$30` and pushes it. The loop continues while the quotient is not zero (`LDAA VAL` / `ORAA VAL+1` sets Z only if both bytes are zero). Then `PN_OUT` pulls the digits (the last one pushed, the leftmost digit, comes first) and draws each. The count `NDIG` says how many to pull. There are no leading zeros: 7 prints as “7”, not “00007”.

The output of T17MATH:

```
7 x 6 = 42
255 x 255 = 65025
100 x 200 = 20000
13 x 0 = 0
```

```
12 x 12 = 144


12345 / 10 = 1234 R 5
```

Every line was checked against Python's own arithmetic: the product of each pair, and 12,345 / 10 = 1,234 remainder 5. The test also caught a real bug in the first version: the remainder was saved in `REM`, the same variable `DIV10` uses as its working remainder, so the second call to `DIV10` (from `PUTNUM`) overwrote the saved value and the program printed "R 1". The saved copy now has its own variable, `SAVER`. **Never keep a result in a variable that a routine you are about to call also uses.**

Another way: subtract the powers of ten

T09NUM prints a 16-bit value as exactly five digits, with leading zeros, by a different method: subtract 10,000 as many times as it will go and count; then 1,000; then 100, 10, 1. The count each time is the digit. The routine:

```
E17A                130  ; PUTDEC - print VAL as five decimal digits at
E17A                131  ; (ROWARG,COLARG).  Subtract 10000, 1000, 100, 10, 1.
E17A  CE E1 CF      132  PUTDEC          LDX     #POWERS
E17D  FF 01 18      133                  STX     PWRPTR
E180  FE 01 18      134  PD_NEXT         LDX     PWRPTR
E183                135  ; high byte of power
E183  A6 00         136                  LDAA    0,X
E185  E6 01         137                  LDAB    1,X       ; low byte
E187  B7 01 13      138                  STAA    TENP
E18A  F7 01 14      139                  STAB    TENP+1
E18D  7F 01 12      140                  CLR     DIGIT
E190  B6 01 11      141  PD_SUB          LDAA    VAL+1     ; VAL - power
E193  B0 01 14      142                  SUBA    TENP+1
E196  B7 01 05      143                  STAA    TMPB
E199  B6 01 10      144                  LDAA    VAL
E19C  B2 01 13      145                  SBCA    TENP
E19F  25 0E         146                  BCS     PD_OUT    ; borrow: stop
E1A1  B7 01 10      147                  STAA    VAL
E1A4  B6 01 05      148                  LDAA    TMPB
E1A7  B7 01 11      149                  STAA    VAL+1
E1AA  7C 01 12      150                  INC     DIGIT
E1AD  20 E1         151                  BRA     PD_SUB
E1AF  B6 01 12      152  PD_OUT          LDAA    DIGIT
E1B2  8B 30         153                  ADDA    #$30      ; ASCII digit
E1B4  BD E2 50      154                  JSR     PUTAT
E1B7  7C 01 01      155                  INC     COLARG
E1BA  FE 01 18      156                  LDX     PWRPTR    ; next power of ten
E1BD  08            157                  INX
E1BE  08            158                  INX
E1BF  FF 01 18      159                  STX     PWRPTR
E1C2  B6 01 14      160                  LDAA    TENP+1
E1C5                161  ; was that the ones digit?
E1C5  81 01         162                  CMPA    #1
E1C7  26 B7         163                  BNE     PD_NEXT
E1C9  B6 01 13      164                  LDAA    TENP
E1CC  26 B2         165                  BNE     PD_NEXT
E1CE  39            166                  RTS
```

The power-of-ten table is `FDB 10000,1000,100,10,1`. For each power the inner loop subtracts it from `VAL` with `SUBA` and `SBCA`, and `BCS PD_OUT` stops when the subtraction borrows (the result went negative: the digit is complete, and the value *before* this subtraction is kept, because the new `VAL` is stored only if there was no borrow). `PUTDEC` is shorter and simpler than `PUTNUM` but prints a fixed width. Its output:

```
65535
12345
00255
00100
00007



0133
```

The first five lines are the values 65,535, 12,345, 255, 100 and 7, each in five digits.

Compare the two routines. `PUTDEC` is easier to follow and a good choice when the width is fixed (a five-digit score box). `PUTNUM` needs the divide routine but handles any size, and can be used for any base by changing the 10. For a score that only goes up by small steps, neither is needed: see BCD.

Packed BCD and DAA

Binary-coded decimal stores each decimal digit in four bits. A byte holds two digits: $47 means forty-seven, not 71. The advantage is that *printing is free*: the high nibble and low nibble of each byte are already the digits, so `PUTHEX` (Chapter 8's hex printer) prints a BCD number as decimal. The disadvantage is that ordinary binary addition gives wrong answers: $19 + $01 = $1A in binary, which is not a BCD number.

The 6800 has an instruction to fix this: **`DAA`**, decimal adjust accumulator. Executed straight after an `ADDA` or `ADCA` of two BCD bytes, it adjusts the result back to valid BCD, using the half-carry and carry flags the addition left. $19 + $01 = $1A, then `DAA` makes it $20. The rules for using it:

1. **Only after ADDA/ADCA.** `DAA` does not work after a subtraction on the 6800.
2. **Immediately.** Anything between the add and `DAA` that changes the flags breaks it.
3. **Chain with ADCA.** For a multi-byte BCD number add the low byte with `ADDA` / `DAA`, then the next byte with `ADCA #0` / `DAA` to carry into it.

T09NUM keeps a four-digit score in two bytes and adds to it:

```
E133                    96  ; packed BCD: start at 0, add 1 ninety-nine times,
E133                    97  ; then add 34 as BCD
E133  7F 01 15          98  BCDPART         CLR     SCORE
E136  7F 01 16          99                  CLR     SCORE+1
E139  C6 63            100                  LDAB    #99
E13B  B6 01 16         101  ADDONE          LDAA    SCORE+1
E13E  8B 01            102                  ADDA    #1
E140  19               103                  DAA                ; decimal adjust
E141  B7 01 16         104                  STAA    SCORE+1
E144  B6 01 15         105                  LDAA    SCORE
E147                   106  ; carry into the high byte
E147  89 00            107                  ADCA    #0
E149  19               108                  DAA
```

```
E14A  B7 01 15       109                 STAA    SCORE
E14D  5A             110                 DECB
E14E  26 EB          111                 BNE     ADDONE
E150  B6 01 16       112                 LDAA    SCORE+1   ; add BCD 34
E153  8B 34          113                 ADDA    #$34
E155  19             114                 DAA
E156  B7 01 16       115                 STAA    SCORE+1
E159  B6 01 15       116                 LDAA    SCORE
E15C  89 00          117                 ADCA    #0
E15E  19             118                 DAA
E15F  B7 01 15       119                 STAA    SCORE
E162  86 0C          120                 LDAA    #12
E164  B7 01 00       121                 STAA    ROWARG
E167  86 0A          122                 LDAA    #10
E169  B7 01 01       123                 STAA    COLARG
E16C                 124  ; print the BCD bytes as hex digits
E16C  B6 01 15       125                 LDAA    SCORE
E16F  BD E2 82       126                 JSR     PUTHEX
E172  B6 01 16       127                 LDAA    SCORE+1
E175  BD E2 82       128                 JSR     PUTHEX
E178  20 FE          129  FOREVER        BRA     FOREVER
```

It adds 1 ninety-nine times (to reach 99) and then BCD 34 (`ADDA #$34 / DAA`). The final score is printed with `PUTHEX` on each byte. The expected value is 99 + 34 = 133, so the bottom line of the T09NUM screen above reads `0133`; the test suite checks exactly that.

In a game the score is a BCD pair. Awarding points is `LDAA SCORE+1 / ADDA #5 / DAA / STAA SCORE+1 / LDAA SCORE / ADCA #0 / DAA / STAA SCORE`. Showing it is two `PUTHEX` calls. PONG keeps its scores in plain binary because they only go to 7 (Chapter 21); a game with larger scores, such as Rubout (bricks worth 7, 5 or 3 points), would use BCD.

Comparing numbers

`CMPA` subtracts without storing and sets the flags. For unsigned values use `BCS/BCC/BHI/BLS`; for signed, `BLT/BGE/BGT/BLE`. For 16-bit comparisons, either subtract with `SUBA/SBCA` (as `PUTDEC` does) and test the carry, or use `CPX` with the caution of Chapter 4: after `CPX`, test only for equal or not equal, or use the signed branches, since the carry is not set.

Summary

- 16-bit add: `ADDA` low, `ADCA` high; subtract with `SUBA/SBCA`; the carry or borrow links the halves.
- Multiply by shift and add; divide by shift and subtract; both take a pass per bit.
- Decimal digits come out of repeated division low digit first; the stack reverses them.
- BCD keeps decimal digits in nibbles; `DAA` after `ADDA/ADCA` repairs the sum.

Exercises

1. Step through MUL8 for 5 x 3. What is PROD after each of the eight passes?
2. What does `DAA` do to $99 + $01 = $9A? What does the carry hold afterwards?
3. Change `PUTDEC` to suppress leading zeros. (Hint: a flag that says “a non-zero digit has been printed”; the ones digit is always printed.)

4. Write the instructions that add 10 points to a BCD score held in `SCORE` (high) and `SCORE+1` (low).

Answers. (1) The multiplier 3 is `11`: pass 1: add 5, PROD = 5; pass 2: add 10, PROD = 15; passes 3-8 add nothing. (2) It produces $00 with the carry set: 100 in BCD, carried into the next byte. (3) Keep a flag in RAM, clear at the start; when a digit is non-zero or the flag is set or it is the last digit, print it and set the flag; otherwise skip. (4) `LDAA SCORE+1 / ADDA #$10 / DAA / STAA SCORE+1 / LDAA SCORE / ADCA #0 / DAA / STAA SCORE`.

Chapter 16

Tables and the paddle

Until now every decision in a program has been a branch: compare, `BEQ`, `BNE`. That is fine for two or three cases. It is a poor way to say “these four keys mean these two directions”, because the program would be a ladder of compares, and adding a fifth key would mean editing the code. The alternative is to put the facts in a **table**, a block of bytes in ROM, and write one short loop that walks it. The code stays the same and the table grows. This chapter introduces tables, the index register as a pointer, and the last thing PONG needs before its design is clear: a bat that a player can move.

Data is just bytes

A ROM is a list of bytes. The assembler cannot tell code from data; the processor can only tell by what it is asked to do. Data is placed with `FCB` (a list of bytes), `FDB` (a list of 16-bit words, high byte first, the 6800’s order) and `FCC` (text). You have used `FCB "text",0` already. A table is nothing more than `FCB` lines with a label in front:

```
KEYTAB          FCB     $53,$FF,$20,$FF
                FCB     $52,$01,$31,$01
                FCB     0
```

This is T13PADDLE’s key table. It holds **pairs**: a keyboard scan code and the direction the paddle moves when that key is down. Direction is a signed byte: `$FF` is -1 (up one row, because row numbers grow downward) and `$01` is +1. A final zero ends the table. Zero is safe as an end marker because it is never a scan code the table needs: the table lists keys that exist, and scan code `$00` is not used here.

The scan codes come from Chapter 9. `$53` is the up-arrow key and `$52` the down-arrow key; `$20` and `$31` are the letters W and S, so a player can use either hand arrangement. All four are taken from the keyboard table in Appendix D, which was read from the firmware. The program does not need to know *which* keys they are, only the numbers.

The index register as a pointer

The 6800 has one 16-bit index register, X. Two things matter about it.

1. `LDX #KEYTAB` loads X with the **address** of the table (the # means “the number KEYTAB”, not “the contents at KEYTAB”). From then on X points at the first byte.

2. `LDAA 0,X` loads A from the address *X + 0*. The 8-bit number before the comma is an **offset**, 0 to 255, added to X to form the address. `LDAA 1,X` reads the byte after the one X points at. X itself is not changed.

This is **indexed addressing**. Written out, the loop that walks the table is:

```
E112                    75  ; table of (scan, direction)
E112  CE E1 76          76  POLL            LDX     #KEYTAB
E115  A6 00             77  PL_LP           LDAA    0,X
E117                    78  ; zero ends the table
E117  27 14             79                  BEQ     PL_END
E119  B7 78 00          80                  STAA    PIA1
E11C  B6 78 00          81                  LDAA    PIA1
E11F  2B 04             82                  BMI     PL_HIT    ; key is down
E121  08                83                  INX
E122  08                84                  INX
E123  20 F0             85                  BRA     PL_LP
E125  A6 01             86  PL_HIT          LDAA    1,X
E127  B7 01 11          87                  STAA    DIR
E12A  BD E1 35          88                  JSR     MOVE
```

Read it slowly:

- `LDX #KEYTAB` points X at the first pair.
- `LDAA 0,X` fetches the scan code. The load sets the Z flag, so `BEQ PL_END` tests for the terminating zero with no `CMPA` at all.
- `STAA PIA1` puts the scan code on the keyboard port (Chapter 9: bits 0 to 6 of the PIA's output select a key) and `LDAA PIA1` reads the answer back. Bit 7 of the answer is 1 when that key is down, which is the *sign bit*, so `BMI` branches on it.
- If the key is up, two `INX` instructions step X over the pair to the next one and the loop goes round again.
- If it is down, `PL_HIT` fetches the second byte of the pair, `LDAA 1,X`, saves it in `DIR` and calls `MOVE`.

Adding a fifth key, say a joystick button or a second player's keys, means adding one `FCB` line. The loop does not change. That is the point of a table.

Why two INX?

The 6800 has no "add 2 to X" instruction. `INX` adds one, so the program uses it twice. Another way is to make the table a list of single bytes in two parallel arrays, `KEYS` and `DIRS`, and index both with the same counter; that needs two index registers, which the 6800 does not have, so the pair layout is the better design here. The general rule is that a table entry should hold everything the loop needs next to each other.

A hazard: the index register is shared

X has to carry the table pointer through the whole loop, but nothing else in this program may change it while the loop is running. `MOVE` is called from inside the loop, and `MOVE` calls `PADCHAR`, which calls `PUTAT`, which uses X for its address arithmetic. After that call returns, X no longer points into the table. This is harmless here, because after `PL_HIT` the program *leaves* the loop and goes to the pacing delay; it does not return to `PL_LP`. If it did, the walk would continue from the wrong place. When you write a loop that calls a subroutine, ask which registers the

subroutine destroys. If X matters afterwards, save it in memory or on the stack first. (PSHX does not exist on the 6800; you store X into two bytes of RAM with STX and reload it with LDX.)

Moving the paddle and clamping it

MOVE adds the direction to the paddle's top row and refuses the move if the result would leave the play area:

```
E135                    93  ; MOVE - slide the paddle one row in DIR, if allowed
E135  B6 01 10          94  MOVE            LDAA    PTOP
E138  BB 01 11          95                  ADDA    DIR
E13B  81 02             96                  CMPA    #2
E13D                    97  ; above row 2: refuse
E13D  25 0F             98                  BCS     MV_X
E13F                    99  ; top row 17 is the last
E13F  81 12            100                  CMPA    #18
E141  24 0B            101                  BCC     MV_X      ; below it: refuse
E143  36               102                  PSHA
E144                   103  ; blank the old image
E144  4F               104                  CLRA
E145  BD E1 51         105                  JSR     PADCHAR
E148  32               106                  PULA
E149  B7 01 10         107                  STAA    PTOP
E14C  20 01            108                  BRA     DRAWPAD
E14E  39               109  MV_X            RTS
```

The addition ADDA DIR works for both directions without any special case, because $FF is -1 in two's complement: 5 + $FF = 4 with the carry flag set, which we ignore. The result is the **new top row**, and it must lie in rows 2 to 17, so that the five-cell paddle fits between the top border at row 1 and the bottom border at row 22.

Two compares do it:

- CMPA #2 / BCS MV_X. The compare subtracts 2 without storing the result. BCS (branch if carry set) is taken when the unsigned value in A is **below** 2. If the paddle were on row 2 and moved up, the new top row would be 1, which is less than 2, and the move is refused.
- CMPA #18 / BCC MV_X. BCC (branch if carry clear) is taken when A is **18 or more**. The largest allowed top row is 17, so 18 is refused. A paddle with its top on row 17 covers rows 17 to 21.

Both tests are **unsigned**, which is what we want: the row numbers are small positive numbers. The signed branches (BLT, BGT) would also work for such small values but they are the wrong tool for something that is just a position. A useful habit is to write the comparison as a range and say it aloud: "allowed if 2 <= top < 18". The two branches refuse everything outside it.

Only when both tests pass does the routine erase the old image and draw the new one. The order is important. The old position must be erased *before* PTOP changes, because PADCHAR computes its rows from PTOP:

```
                PSHA                    ; keep the new top row
                CLRA                    ; blank the old image
                JSR     PADCHAR
                PULA
                STAA    PTOP
                BRA     DRAWPAD         ; and paint the new one
```

PADCHAR is the second entry point of DRAWPAD. DRAWPAD is just LDAA #$23 (the character #), and the next instruction, the label PADCHAR, takes whatever is in A as the character to paint. Calling PADCHAR with A = 0 paints five blanks, which is an erase. One routine, two uses, no copying. The BRA DRAWPAD at the end is a **tail call**: instead of JSR DRAWPAD / RTS, it jumps, and DRAWPAD's own RTS returns directly to MOVE's caller. It saves a stack entry and four bytes.

The PSHA / PULA pair holds the new top row across the call to PADCHAR, which uses A. Without it the new row number would be lost. Chapter 13 (the stack) is the reason this works: the stack is a place to park a register for a moment.

Why CLRA and not LDAA #0?

Both put zero in A. CLRA is one byte, LDAA #0 is two. The difference that matters is the carry flag. CLRA clears carry; LDAA #0 leaves it alone (loads do not change C). Here nothing reads the carry afterwards, so the shorter one is correct. In Chapter 4's exercise, where the carry mattered, the longer form was needed.

Drawing five cells

PADCHAR paints a column of five characters:

- TMPB holds the character (A is about to be needed for the row arithmetic);
- TMPX counts 0 to 4;
- each pass sets ROWARG = PTOP + TMPX and COLARG = 4, loads the character into A and calls PUTAT.

PUTAT (Chapter 8) draws the character in A at (ROWARG, COLARG) through the CRT RAM with the HALT strobe in the right place. All the screen-access details are inside it; the paddle code only says *where* and *what*.

The variable names TMPB and TMPX come from the library's variable block (lib/vars.inc). They are scratch bytes: free for any routine to use *as long as it does not call anything that also uses them*. PADCHAR calls PUTAT, and PUTAT does not touch TMPB or TMPX; that was checked when the library was written. This is the same discipline as the shared REM bug in Chapter 15, applied in advance.

Pacing the loop

The last part of POLL is a delay:

```
PL_END          LDX     #400            ; pace the loop
PACE            DEX
                BNE     PACE
                BRA     POLL
```

Without it the program would scan the keyboard as fast as the processor can run, and each press would move the paddle many rows before the finger could lift. A paddle that crosses the screen in one keystroke is not playable. DEX followed by BNE is a loop of 400 passes. Each pass takes the time of the two instructions; from the data sheet's cycle counts, DEX is 4 cycles and a taken BNE is 4, so each pass is 8 cycles and the whole delay is 3,200 cycles plus the setup. This book does not turn that into seconds, because the clock rate of the processor has not been verified from the machine (Chapter 1). What matters is the *shape*: the loop count is a single number that you adjust by trial until the paddle moves at a pleasant speed. In the emulator,

which is not real time, the speed is meaningless; on the real terminal, adjust `400` until it feels right. PONG also paces itself with a counted `DEX/BNE` loop, but its count is a 16-bit variable that gets smaller as a rally goes on, so the game speeds up (Chapter 20).

What the program does

```
#
#
#
#
#
```

The test run starts with the paddle at row 9, holds the DOWN key, and stops after a fixed number of cycles. The paddle sits at rows 17 to 21, its lowest legal position, and has not gone below it: the clamp works. The automatic check in `tests.py` confirms exactly those five rows, and nothing else on the screen, contain the `#`.

To test the other end, edit `KEYTAB` in the emulator run so that the held key is UP; the paddle stops at rows 2 to 6. The two limits are the two places where clamping code is most often wrong, by one. Always test **both ends** of a range, and test the value just inside as well as the value just outside.

Summary

- A table is data with a label. Walk it with `LDX #TABLE` and indexed loads `0,X`, `1,X`.
- A zero byte (or other impossible value) ends a table; the load that fetches it sets Z, so no compare is needed.
- Offsets are 0 to 255 and are *added* to X. For tables bigger than 256 bytes, move X itself.
- `INX/DEX` step by one. The 6800 has no way to add a constant to X.
- To clamp a value into a range, use two compares with `BCS` (below) and `BCC` (at or above) and refuse the move if either fires.
- Erase before you change the variable that says where the object is; then draw.
- A routine with two entry points saves code: the first one sets a register, then falls into the second.
- A `BRA` to a routine that ends in `RTS` is a tail call.

Exercises

1. Add the keys `A` and `Z` to `KEYTAB` as a second pair of controls. (Look up their scan codes in Appendix D.) How many bytes of ROM does it cost, and how many instructions change?
2. Make the paddle move two rows per keypress by changing only the table. What else must change so that the clamp still works? (Hint: what range of top rows is now possible?)
3. Change the paddle to seven cells tall. Which constants change? Find every place that depends on the height (there are three).
4. Replace the pair layout with a table of three bytes per entry: scan code, direction, and the character to draw. Use `INX` three times. Which key now draws a different paddle?
5. `PL_HIT` calls `MOVE` and then falls into the pacing loop. What happens, and why is it a feature, if two keys are held down together?
6. Explain why the unsigned branch `BCS` is correct for `CMPA #2` here, but would be *wrong* if the paddle's coordinate could be negative.

Answers to selected exercises

1. Four bytes for the two pairs; no instruction changes. The loop is unchanged.
2. Change the `ADDA DIR` step by changing the table to `$FE` and `$02`. Starting from row 9 the paddle then visits 7, 5, 3 going up; the next step would give 1, which `CMPA #2` refuses, so it stops on row 3 and can never reach row 2. Going down it visits 11, 13, 15, 17 and stops there, because 19 is refused. The legal rows are now an odd-numbered subset, so the two ends are no longer symmetric. Either keep steps of one, or clamp to the nearest legal row instead of refusing.
3. The table is walked in order, so the first key found down (up-arrow) wins and the second is never read in that pass. The paddle moves in that direction only; holding both does not make it jitter. The fact that the order of the table is the order of priority is a property you can use deliberately.
4. A negative position stored in a byte looks like a large unsigned number, 128 to 255. `BCS` after `CMPA #2` would treat -1 (255) as “above 2” and allow it. The coordinates in this book are never negative; the direction byte is, and it is only *added*.

Chapter 17

Sprites: images that move

A **sprite** is a small picture that moves over a background. On machines with graphics hardware the hardware draws sprites. The ADM-31 is a character terminal: its "pixels" are character cells, and nothing moves unless the program writes characters. A sprite on this machine is therefore a few cells of text, and *moving* it means two operations done in the right order: **erase** the cells it occupied, then **draw** it in the new place. This chapter builds the smallest useful example, a two-row tank that slides along the bottom of the screen under the control of the LEFT and RIGHT arrow keys.

Design before code

Decide the facts first; the program is then just a transcription.

- **The image.** Two rows of three characters: `/^\` above `[#]`. Six cells, stored as six bytes in ROM.
- **The position.** One byte, `SC`, the column of the image's left edge. The rows never change: the tank lives on rows 21 and 22, which leaves the last screen row (23) empty.
- **The limits.** The tank is three cells wide. Column 0 is the left-most legal left edge; the right-most is 80 - 3 = 77. The frame of T13PADDLE had walls to bounce off; this program simply refuses a move that would leave 0..77.
- **The controls.** LEFT is scan code `$54`, RIGHT is `$55` (Chapter 9, Appendix D).

Writing the facts as constants and a table is not decoration. Every bug in a sprite program is a disagreement between two places that each "knew" the width or the row.

The image as data

```
E1AC  2F 5E 5C       147  IMAGE           FCB     $2F,$5E,$5C ; /^\
```

The six bytes are `/^\` (`$2F,$5E,$5C`) and `[#]` (`$5B,$23,$5D`). The assembler dialect in use accepts character literals poorly (Chapter 3), so the codes are written in hex, and the comment shows the picture. The image is stored **row by row**, left to right, the same order the program draws it. That ordering is the whole trick: the drawing loop only needs a pointer that moves forward through the six bytes and never has to compute where a cell is.

Drawing: two loops and a pointer

```
E14A                 101  ; DRAWSPR - draw the two image rows at column SC
E14A  CE E1 AC       102  DRAWSPR         LDX     #IMAGE
E14D  FF 01 11       103                  STX     SPR
E150  86 15          104                  LDAA    #21       ; first image row
E152  B7 01 13       105                  STAA    SROW
E155  B6 01 13       106  DS_ROW          LDAA    SROW
E158  B7 01 00       107                  STAA    ROWARG
E15B  B6 01 10       108                  LDAA    SC
E15E  B7 01 01       109                  STAA    COLARG
E161                 110  ; three cells per row
E161  C6 03          111                  LDAB    #3
E163  FE 01 11       112  DS_CELL         LDX     SPR
E166  A6 00          113                  LDAA    0,X
E168  08             114                  INX
E169  FF 01 11       115                  STX     SPR
E16C  37             116                  PSHB
E16D  BD E2 1D       117                  JSR     PUTAT
E170  33             118                  PULB
E171  7C 01 01       119                  INC     COLARG
E174  5A             120                  DECB
E175  26 EC          121                  BNE     DS_CELL
E177  7C 01 13       122                  INC     SROW
E17A  B6 01 13       123                  LDAA    SROW
E17D  81 17          124                  CMPA    #23       ; rows 21 and 22
E17F  26 D4          125                  BNE     DS_ROW
E181  39             126                  RTS
```

Read `DRAWSPR` as two nested loops. The outer loop walks the image rows; `SROW` is the screen row, starting at 21 and ending when it reaches 23. The inner loop is three cells; B counts them down.

Inside the inner loop:

```
DS_CELL         LDX     SPR             ; where in the image are we?
                LDAA    0,X             ; the next image byte
                INX
                STX     SPR             ; remember the new place
                PSHB                    ; PUTAT may change B
                JSR     PUTAT
                PULB
                INC     COLARG          ; next screen column
                DECB
                BNE     DS_CELL
```

Three things are worth studying.

The image pointer lives in memory. `SPR` is two bytes of RAM holding an address. Each trip reloads X from it, reads the byte, steps it, and stores it back. The reason is the same as in Chapter 16: `PUTAT` uses X to form the screen address, so X cannot carry the image pointer across the call. A pointer that must survive a subroutine call is kept in RAM with `STX`/`LDX`.

B is saved around the call. `CALCADR`, called by `PUTAT`, uses B for its arithmetic. The cell counter would be destroyed, so it is pushed before the call and pulled after. This is the first time the book has pushed B: `PSHB` and `PULB` work exactly like `PSHA` and `PULA`. The alternative is to keep the counter in a RAM variable and use `DEC`, which is slower to write and to run but cannot be disturbed. Either is correct; choose one and be consistent.

The column is advanced by the caller. `PUTAT` leaves `ROWARG` and `COLARG` as it found them, so the loop increments `COLARG` itself. Contrast `PUTSTR` (Chapter 14), which advances `COLARG` for you because it exists to print a whole string. The convention for each routine is written in its header comment in `lib/`; read it before relying on it.

Erasing is drawing the background

`ERASESPR` is `DRAWSPR` with a different source for the character: instead of reading the image, it uses `CLRA`, zero, in every cell. Because this program's background is blank, "draw the background" and "write zero" are the same. In a game with a patterned background (a wall, a field of dots) the erase would have to *redraw* the background cell. A common technique is to keep a copy of the background in RAM and restore from it; PONG avoids the issue by having a blank field with a few simple, known objects (Chapter 20).

Why two nearly identical routines, when one with a flag would do? Duplication is a legitimate trade for speed and clarity at this size: each routine is short and reads as a sentence. In a larger program you would write one `DRAWROWS` routine taking a pointer to either the image or a table of zeros, and call it with the pointer in X. Exercise 4 asks you to do that.

The main loop

```
E110  86 54           77  MAIN            LDAA    #$54      ; LEFT key
E112  B7 78 00        78                  STAA    PIA1
E115  B6 78 00        79                  LDAA    PIA1
E118  2A 0E           80                  BPL     NOTL
E11A  B6 01 10        81                  LDAA    SC
E11D  27 09           82                  BEQ     NOTL      ; at the left edge
E11F  BD E1 82        83                  JSR     ERASESPR
E122  7A 01 10        84                  DEC     SC
E125  BD E1 4A        85                  JSR     DRAWSPR
E128  86 55           86  NOTL            LDAA    #$55      ; RIGHT key
E12A  B7 78 00        87                  STAA    PIA1
E12D  B6 78 00        88                  LDAA    PIA1
E130  2A 10           89                  BPL     NOTR
E132  B6 01 10        90                  LDAA    SC
E135                  91  ; 80 columns - 3 wide
E135  81 4D           92                  CMPA    #77
E137  27 09           93                  BEQ     NOTR
E139  BD E1 82        94                  JSR     ERASESPR
E13C  7C 01 10        95                  INC     SC
E13F  BD E1 4A        96                  JSR     DRAWSPR
```

Each pass polls two keys. For LEFT:

1. Write the scan code `$54` to `PIA1` and read it back. If bit 7 is clear (`BPL`) the key is up, and the code skips to the right-key test.
2. If it is down, load `SC` and test for 0 (`BEQ`): at the left edge, nothing to do.
3. Otherwise `JSR ERASESPR`, `DEC SC`, `JSR DRAWSPR`. **Erase first, change the position second, draw third.** If the order were erase, draw, change, the tank would be redrawn where it already was, and the erased cells would be left blank, so the tank would vanish.

For RIGHT the check is `CMPA #77 / BEQ NOTR`, the limit calculated above. `BEQ` is enough here because `SC` only ever moves by one and starts inside the range; an == test is safe when the

variable cannot jump over the limit. If you ever let the tank move by two, use the range test from Chapter 16.

Nothing else in the program depends on the tank. That separation, *state in a variable, drawing in a routine*, is the pattern for everything that follows. The state of the program is `SC`. The screen is only a picture of the state.

What can go wrong

- **Leftover images.** If the program ever changes `SC` without erasing, the old image stays on the screen. In the test run this shows as a smeared tank. The cure is to put the erase, the change and the draw into one routine (`MOVETANK`) so that a new caller cannot forget one of them.
- **Off-by-one at the right edge.** Using 78 or 80 instead of 77 draws cells in columns 80 and beyond. Screen memory is a flat array with 80 bytes per row (Chapter 8), so there is no column 80: the byte after column 79 *is* column 0 of the next row. The tank's right-hand cells would be written at the far left of the following line. The failure is easy to recognise once you know the cause.
- **Registers clobbered by calls.** Most beginner bugs in this kind of loop are a register assumed to survive a call.

The test

The emulator test holds the RIGHT key for a long time and checks the result:

```
/^\
[#]
```

The tank is flush against the right edge, occupying columns 77 to 79 of rows 21 and 22. `tests.py` asserts exactly that: the string `/^\` begins at column 77 of row 21 and `[#]` is alone on row 22. Because the key is held for far more passes than there are columns, the test would show a tank in the wrong place if the limit were wrong by even one. (As with the paddle, a test that runs *past* the limit is the best test of the limit.)

Many sprites

The same structure extends. A table of sprite records (row, column, image pointer) and one `DRAWALL` routine that walks it is how larger games manage many objects. The key ideas stay the same: state in memory, one routine to draw, one to erase, erase before update. The number of cells you can move per second is limited by the speed of `PUTAT`, which does the HALT strobe (Chapter 8) and the address arithmetic on every write; Chapter 18 shows a program that also *reads* the screen.

Summary

- A character-cell sprite is a few bytes of image plus a position; moving it is erase, update, draw.
- Store the image in drawing order so a single forward pointer is enough.
- A pointer that has to survive a call goes in RAM (`STX/LDX`); a counter that has to survive a call goes on the stack or in RAM.

- Limits are computed from the width: right edge = 80 - width.
- PSHB/PULB exist; there is no PSHX.
- Test the limits by pushing past them.

Exercises

1. Make the tank four cells wide: /^^\ and [##]. List every constant that must change. (Answer: the image data, the inner loop count, the right limit 77 -> 76, and nothing else.)
2. Add a third row to the image so the tank has a barrel on row 20. What must change in DRAWSPR and ERASESPR?
3. Make the tank wrap round: leaving the right edge brings it back on the left. Which tests change?
4. Write a single routine PAINT that takes a pointer to six bytes in X and draws or erases depending on the pointer. Make DRAWSPR call it with #IMAGE and ERASESPR call it with #BLANKS.
5. Move the tank two columns per pass. Is BEQ still enough for the right limit? Why not?
6. Explain, in terms of the registers, why PSHB/PULB surround JSR PUTAT but nothing surrounds INC COLARG.

Chapter 18

A bouncing ball: collision by reading the screen

A ball that moves is the sprite of Chapter 17 with one cell and two velocity bytes. A ball that *bounces* needs something new: it has to find out that it has hit something. There are two ways to do that. One is to keep a model in memory of where every object is (a map, or coordinates and sizes) and compare numbers. The other, available on a machine whose screen memory can be read back, is to ask the screen: **look at the cell the ball is about to enter; if it is not blank, something is there.** The second way needs almost no code and no map, and it handles walls, paddles and bricks with the same test. T11BOUNCE uses it, and so does PONG.

This works because the CRT RAM is ordinary readable memory. Chapter 8 showed the access sequence: a strobe to `HALT`, a `NOP`, then the access. The library routine `GETAT` is the mirror of `PUTAT`:

```
; GETAT - A = character at (ROWARG,COLARG)
GETAT           JSR     CALCADR         ; X = cell address
                STAA    HALT            ; sync with the CRT
                NOP                     ; settle
                LDAA    0,X             ; read the cell
                RTS
```

It returns in A the character code in the cell at (`ROWARG`, `COLARG`). Because the final instruction is a load, the Z flag already reflects the value: **Z is set when the cell holds zero, and zero displays as a blank**. A following `TSTA` is redundant on the real hardware but makes the intent obvious to the reader; the program keeps it.

The play box

T11BOUNCE draws a rectangle of `#` characters, rows 2 to 21 and columns 5 to 74, using `HLINE` for the horizontal sides and a loop of `PUTAT` calls for the vertical sides:

```
E100  0F              74  START           SEI
E101  8E 03 FF        75                  LDS     #$03FF
E104  BD E2 9E        76                  JSR     INITCRT
E107  BD E1 F7        77                  JSR     CLRSCR
E10A                  78  ; box: rows 2-21, columns 5-74, drawn with '#'
E10A  CE 80 A5        79                  LDX     #BOXTOP
E10D  86 23           80                  LDAA    #$23
E10F  C6 46           81                  LDAB    #70
```

```
E111  BD E2 08        82                JSR     HLINE     ; top
E114  CE 86 95        83                LDX     #BOXBOT
E117  86 23           84                LDAA    #$23
E119  C6 46           85                LDAB    #70
E11B  BD E2 08        86                JSR     HLINE     ; bottom
E11E  86 03           87                LDAA    #3
E120  B7 01 00        88                STAA    ROWARG
E123  86 05           89  SIDES         LDAA    #5
E125  B7 01 01        90                STAA    COLARG
E128  86 23           91                LDAA    #$23
E12A  BD E2 6D        92                JSR     PUTAT     ; left side
E12D  86 4A           93                LDAA    #74
E12F  B7 01 01        94                STAA    COLARG
E132  86 23           95                LDAA    #$23
E134  BD E2 6D        96                JSR     PUTAT     ; right side
E137  7C 01 00        97                INC     ROWARG
E13A  B6 01 00        98                LDAA    ROWARG
E13D  81 15           99                CMPA    #21
E13F  26 E2          100                BNE     SIDES
E141                 101  ; ball starts at row 5, column 10, heading down-right
E141  86 05          102                LDAA    #5
E143  B7 01 10       103                STAA    BR
E146  86 0A          104                LDAA    #10
E148  B7 01 11       105                STAA    BC
E14B  86 01          106                LDAA    #1
E14D  B7 01 12       107                STAA    DR
E150  B7 01 13       108                STAA    DC
E153  7F 01 14       109                CLR     TICKS
E156  7F 01 15       110                CLR     TICKS+1
```

`HLINE` is the fastest way to fill a run of cells: it is given the character in A, the count in B and the first cell's *address* in X, and it strobes `HALT` and stores for each cell, with no address arithmetic at all. The two constants `BOXTOP` and `BOXBOT` are those first-cell addresses, calculated once by hand: `$8000 + row * 80 + column`. Row 2, column 5 is `$8000 + 160 + 5 = $80A5`. Row 21, column 5 is `$8000 + 1680 + 5 = $8695`. (Check the second by the other method: 21 x 80 = 1680 = `$0690`, plus 5 is `$0695`, plus `$8000` is `$8695`.) Constants you calculate by hand are a rich source of errors; the way to catch them is to look at the result. The `@@SCREEN` view below, and the model in `tests.py`, confirm them.

Position, step and the three questions

The ball's state is four bytes: its row `BR` and column `BC`, and its two **steps** `DR` (row) and `DC` (column). A step is 1 (toward larger numbers: down or right) or `$FF` (-1: up or left). Each tick the ball moves by (`DR`, `DC`), so it travels diagonally, one cell in each direction.

Before moving, `STEP` asks three questions, always in this order:

```
E165                 116  ; STEP - one tick: test, turn, erase, move, draw
E165                 117  ; vertical neighbour
E165  B6 01 10       118  STEP            LDAA    BR
E168  BB 01 12       119                  ADDA    DR
E16B  B7 01 00       120                  STAA    ROWARG
E16E  B6 01 11       121                  LDAA    BC
E171  B7 01 01       122                  STAA    COLARG
E174  BD E2 94       123                  JSR     GETAT
E177  4D             124                  TSTA
```

```
E178  27 07          125                  BEQ     VFREE
E17A                 126  ; blocked: reverse row step
E17A  B6 01 12       127                  LDAA    DR
E17D  40             128                  NEGA
E17E  B7 01 12       129                  STAA    DR
E181                 130  ; horizontal neighbour
E181  B6 01 10       131  VFREE           LDAA    BR
E184  B7 01 00       132                  STAA    ROWARG
E187  B6 01 11       133                  LDAA    BC
E18A  BB 01 13       134                  ADDA    DC
E18D  B7 01 01       135                  STAA    COLARG
E190  BD E2 94       136                  JSR     GETAT
E193  4D             137                  TSTA
E194  27 07          138                  BEQ     HFREE
E196                 139  ; blocked: reverse column step
E196  B6 01 13       140                  LDAA    DC
E199  40             141                  NEGA
E19A  B7 01 13       142                  STAA    DC
E19D  B6 01 10       143  HFREE           LDAA    BR        ; diagonal target
E1A0  BB 01 12       144                  ADDA    DR
E1A3  B7 01 00       145                  STAA    ROWARG
E1A6  B6 01 11       146                  LDAA    BC
E1A9  BB 01 13       147                  ADDA    DC
E1AC  B7 01 01       148                  STAA    COLARG
E1AF  BD E2 94       149                  JSR     GETAT
E1B2  4D             150                  TSTA
E1B3  27 0E          151                  BEQ     DFREE
E1B5                 152  ; inner corner: reverse both
E1B5  B6 01 12       153                  LDAA    DR
E1B8  40             154                  NEGA
E1B9  B7 01 12       155                  STAA    DR
E1BC  B6 01 13       156                  LDAA    DC
E1BF  40             157                  NEGA
E1C0  B7 01 13       158                  STAA    DC
```

1. **Is the cell above/below me, in the direction I am travelling vertically, blocked?** The position tested is (`BR + DR`, `BC`). If it is not blank, reverse `DR`.
2. **Is the cell beside me, in the direction I am travelling horizontally, blocked?** The position is (`BR`, `BC + DC`). If so, reverse `DC`.
3. **Is the diagonal cell blocked?** After steps 1 and 2 have updated `DR` and `DC`, test (`BR + DR`, `BC + DC`). If it is still blocked, reverse both.

Why three, and why that order?

A ball moving diagonally approaches a flat wall: it will meet either the floor (a vertical neighbour is blocked) or a side wall (a horizontal neighbour is blocked). Tests 1 and 2 catch these. They are done *separately* so that a ball sliding along a wall bounces only in the direction of the wall it touches: a ball hitting the floor reverses its row direction but keeps going the same way across.

The third test catches the awkward case: the ball is heading for the corner of an obstacle, so that neither the vertical neighbour nor the horizontal neighbour is blocked but the diagonal cell is. That happens at the *outside* corner of an object (a brick, say, or a paddle's end). Reversing both directions sends the ball back the way it came, which is the conventional result. Inside T11BOUNCE's box there are no outside corners, so the third test never fires; when the ball arrives in a corner of the room, the two neighbouring cells are both walls and tests 1 and 2

have already reversed both steps. The test is there because a bat has ends, and because it costs only a few instructions to be right.

If you leave the third test out, a ball can pass diagonally past the end of a brick that it should have bounced from. Every game built on this method has to decide what a corner means; PONG handles the corner cases by retesting (Chapter 21).

Negation

`NEGA` replaces A by its two's complement, 0 - A. For 1 it gives `$FF`; for `$FF` it gives 1. Reversing a step is therefore `LDAA DR / NEGA / STAA DR`. `NEG` also exists for memory (extended and indexed forms), and `NEGB` for B, but in the assembler used here we prefer the load, negate, store form for clarity. An alternative that avoids `NEGA` is to store the steps in a table and index it.

Erase, move, draw

The tail of `STEP` is Chapter 17's pattern:

```
DFREE           LDAA    BR              ; erase the old ball
                STAA    ROWARG
                LDAA    BC
                STAA    COLARG
                CLRA
                JSR     PUTAT
                ...                     ; BR += DR, BC += DC
                ...                     ; draw '*' ($2A) at the new place
```

The ball is a *. The next tick's collision test will see this * as "something" if it were ever in the cell being tested, but the ball is never tested against itself, since the tested cell is always one step *ahead*. The ball must not be drawn in a cell it will then read as blocked; this is one reason to **test, then move, then draw**, in that order, and never draw and then test.

Counting ticks

The main loop is:

```
MAIN            JSR     STEP
                LDX     TICKS
                INX
                STX     TICKS
                BRA     MAIN
```

A 16-bit counter in `TICKS` (two bytes), incremented with `INX`. Every program that moves things should have such a counter while it is being developed; it turns "the program ran for a while" into a number a test can use. Because `INX` is the only 16-bit increment on this processor, a counter in X, stored to memory and loaded back, is the cheapest way to count past 255.

This main loop has **no delay**. In the emulator that is fine; on a real terminal the ball would cross the box faster than the eye can follow. Chapter 20 shows the pacing PONG uses. For now the property of interest is that the bounce logic is correct, and the test checks it.

How the test proves it

The emulator is stopped after a fixed number of cycles and `tests.py` reads RAM. At the end of this run `TICKS` held 1,554 and the ball was at row 11, column 56. The test contains an independent model of the ball written in Python: the same rules, in the same order, applied to a Python function that says which cells of the box are solid. It runs the model for 1,554 ticks and compares the final row and column with the ones the 6800 program left in RAM. They agree, and so does the character on the screen. A match after 1,554 ticks is strong evidence: one wrong bounce anywhere along the path would send the two to different places.

```
################################################################################
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                 *                            #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
#                                                                              #
################################################################################
```

Following the model, the first bounces are at tick 15 (the ball reaches row 20, column 25, and the floor reverses its row step), tick 32 (the ceiling), tick 49 (the floor again), and at tick 63 the ball reaches the cell just inside the right wall (row 6, column 73) and reverses only its column step, so it leaves heading up and to the left. Three ticks later (tick 66, row 3, column 70) it meets the ceiling and turns downward. Each of these first events was computed by the model and each is checked, implicitly, by the final position.

This **model-against-implementation** test is worth copying. When a result is a number that depends on a long chain of small decisions, write the same logic a second time in a language with a debugger and compare. The two implementations will rarely share a bug, because you will not make the same mistake in both.

Limits of the method

- **It is only as good as "blank equals zero".** If any other character is in the way, even a stray letter, the ball bounces from it. For a game this is a feature.
- **The ball learns nothing about *what* it hit.** It sees only a code. A game that needs to know whether it hit a paddle or a wall compares the code in A: `CMPA #$23` and so on. PONG's ball does this: a wall, a paddle and the centre line each get a different response (Chapter 21).

- **Reading is as slow as writing.** `GETAT` strobes `HALT` and then reads. Three `GETAT` calls per tick plus one erase and one draw is five screen accesses per ball per tick. For one ball that is nothing. For forty bullets it matters.
- **The screen must show the truth.** If the program ever draws something for decoration in a cell that the game logic treats as empty, the ball will bounce off the decoration.

Summary

- The CRT RAM can be read back with the same `HALT` sequence as for writing.
- To detect a collision, read the cell the object will enter next; non-zero means blocked.
- Test vertical, then horizontal, then diagonal, reversing the step in each case that is blocked.
- `NEGA` reverses a step.
- Test, then erase, move, draw.
- Use a counter and an independent model to prove a long-running behaviour.

Exercises

1. Remove the third test (the diagonal). Make the ball start so that it heads into the exact bottom-right corner of the room. What does it do, and why is it still correct?
2. Put a single `#` in the middle of the box (using `PUTAT` once at the start). Predict where the ball bounces, then run it.
3. Change the starting velocity to `DR = 1, DC = 0`. Describe the motion. What happens to test 3?
4. Add a second ball using a table of four-byte records (row, column, row step, column step) and a loop. Which state must be saved across the `JSR GETAT` calls?
5. `GETAT` ends with `LDAA 0,X`. Why does that leave Z valid for the caller, and what would break the guarantee if someone added an instruction after it?
6. If you wanted the ball to *speed up* every 100 bounces, which bytes would you add?

Chapter 19

Random numbers

A game that always does the same thing is a puzzle, not a game. PONG needs chance in three places: where the ball starts, which way it first goes, and how well the computer's bat judges the return. The 6800 has no random-number instruction and the ADM-31 has no hardware noise source we can read, so the program computes numbers that *look* random from a rule. Such numbers are called **pseudo-random**. The rule used by PONG, by T10RAND and is a **linear-feedback shift register**, an LFSR, and this chapter shows how it works, why it is so cheap on a 6800, how to check it, and how to turn its output into the small ranges a game needs.

The idea

Take a 16-bit number, the **state**. To get the next state, shift it right one place. The bit that falls off the right-hand end decides what happens next: if it was 0, the new state is just the shifted value; if it was 1, the shifted value is also combined, with exclusive-OR, with a fixed mask called the **taps**. Do this repeatedly and the state wanders through a long sequence of values.

For a good choice of taps the state visits every non-zero 16-bit value exactly once before repeating. This sequence has 65,535 members. The taps used here are `$B400`, and the chosen starting value, the **seed**, is `$ACE1`. A short Python program (the same kind of check as in Chapter 18) confirms that starting at `$ACE1` the state first returns to `$ACE1` after exactly 65,535 steps. Zero is the exception: a zero state shifts to zero and stays there forever, which is why the routine's header says **SEED must be non-zero**.

The form of the rule just described, shifting right and XOR-ing the mask into the high byte when a 1 falls off, is called a **Galois** LFSR. It is popular on small processors because the whole update is one shift of the state and, at most, one XOR of a constant into one byte.

The routine

```
; RAND - 16-bit Galois LFSR, taps $B400.
; Returns A = low byte of the new state.
; SEED must be non-zero.
RAND            LDAA    SEED+1
                ANDA    #1              ; bit that shifts out
                STAA    TMPA
                LSR     SEED            ; shift the 16-bit state
                ROR     SEED+1
```

```
                TST     TMPA            ; was the bit 1?
                BEQ     RND_X           ; no feedback
                LDAA    SEED
                EORA    #$B4            ; apply taps
                STAA    SEED
RND_X           LDAA    SEED+1
                RTS
```

Follow it for the seed `$ACE1`:

- `LDAA SEED+1` loads the low byte, `$E1`; `ANDA #1` isolates the bit about to fall off: 1. The bit is parked in `TMPA`.
- `LSR SEED` shifts the high byte right (`$AC` becomes `$56`); its low bit, 0, goes into the carry. `ROR SEED+1` shifts the low byte right and feeds the carry into its top bit: `$E1` becomes `$70`. The state is now `$5670`. The bit that left the low byte (the 1) is gone; this is why the routine saved it in `TMPA` first.
- `TST TMPA` tests the saved bit. It was 1, so the branch to the exit is not taken.
- `LDAA SEED` fetches the high byte `$56`, `EORA #$B4` makes it `$E2`, and `STAA SEED` stores it. The state is now `$E270`.
- `LDAA SEED+1` returns the low byte, `$70`, in A.

The first output is `$70`, and the first row on T10RAND's screen is the same thing:

```
          70  38  9C  4E  27  13  89  C4
          62  B1  58  2C  16  8B  C5  62


          007D 0075 0075 008E 0074 008F 008E 007A
```

The sixteen bytes `70 38 9C 4E 27 13 89 C4 62 B1 58 2C 16 8B C5 62` are the first sixteen outputs. `tests.py` computes the same sequence in Python and demands that the 6800 produced the same bytes. (Consecutive outputs of an LFSR are related by the shift: each new low byte is mostly the old low byte shifted right one place, with the neighbouring bits of the state moving in. Taken a byte at a time, the generator is not a source of independent bytes. Read the next section before choosing which bits to use.)

Parking the bit

`TST TMPA` could have been `LDAA TMPA`; both set Z for the `BEQ`. The routine parks the outgoing bit in `TMPA` because the shifts that follow destroy the carry and the test comes after them. Another way to write it is to test the bit of the low byte with `BITA #1` before shifting and branch afterwards on a flag saved in memory; the result is the same. `BITA` (Chapter 4) is the instruction for testing a bit without changing A.

Using the output

`RAND` returns a byte, 0 to 255. Games need smaller ranges. The rules:

- **Power-of-two ranges: mask.** To get 0 to 15, `ANDA #$0F`. To get 0 or 1, `ANDA #1`. The mask keeps the low bits, which for a Galois LFSR are acceptable (the state is only 16 bits wide, and each output is a window onto it).
- **Offset ranges: mask, then add.** PONG chooses the serve row with:

```
    JSR     RAND
    ANDA    #$0F            ; 0-15
```

```
        ADDA    #4              ; 4-19
        STAA    BROW
```

which gives rows 4 to 19, the middle of the court, never touching the border rows. * **One bit as a coin toss.** PONG takes the *sign bit* of a random byte as "up or down": `STAA BVDIR`, and later code tests bit 7. A coin toss costs no arithmetic. * **Non-power-of-two ranges: avoid.** To get 0 to 9 you would have to divide (Chapter 15) or discard values 10 to 15 and try again. Design your game so that the ranges are powers of two.

A bell-shaped number

Sometimes uniform is wrong. PONG's computer opponent aims its bat at the ball, but aims with an **error**. A uniform error of -3 to +3 would make the computer miss as often when it is nearly right as when it is badly wrong. The better shape is *bell-shaped*: small errors common, large errors rare. The way to get it is to **add two small random numbers**:

```
E336  B6 01 05       655  PH_AIM          LDAA    BCOL      ; A = ball column
E339                 656  ; left half of the court?
E339  81 28          657                  CMPA    #40
E33B                 658  ; computer's hit: nothing more
E33B  24 1A          659                  BCC     PH_X
E33D                 660  ; AIMOFF = -3..+3, bell-shaped (sum of two
E33D  BD E5 BF       661                  JSR     RAND
E340                 662  ; 0-3 rolls, minus 3).  +-3 is outside the
E340  B7 01 1E       663                  STAA    TMPA
E343                 664  ; paddle: the computer misjudges ~1 in 8.
E343  84 03          665                  ANDA    #3
E345                 666  ; keep the first 0-3 roll
E345  B7 01 1F       667                  STAA    TMPB
E348                 668  ; reload the random byte
E348  B6 01 1E       669                  LDAA    TMPA
E34B                 670  ; shift right twice ...
E34B  44             671                  LSRA
E34C                 672  ; ... to bring bits 2-3 down
E34C  44             673                  LSRA
E34D  84 03          674                  ANDA    #3        ; second 0-3 roll
E34F                 675  ; add the first roll: 0-6
E34F  BB 01 1F       676                  ADDA    TMPB
E352                 677  ; centre on zero: -3..+3
E352  80 03          678                  SUBA    #3
E354                 679  ; store the aim error
E354  B7 01 12       680                  STAA    AIMOFF
E357  39             681  PH_X            RTS               ; done
E358                 682
E358                 683  ; ================================================
E358                 684  ; POLLHUM -- scan the human's keys; first one found
```

The lines are from PONG's real listing (Chapter 21 explains the rest). Take a random byte. Its low two bits give a number from 0 to 3. Shift right twice, and the next two bits give another. Add them: the sum is 0 to 6, and the sum is most often 3 (four ways in sixteen: 0+3, 1+2, 2+1, 3+0), and 0 or 6 only one way each (1 in 16 each). Subtract 3 and the aim error is -3 to +3 with the shape of a small bell. The bat of this program covers five cells, which is a reach of 2 each side of its centre; so an error of -3 or +3 sends the bat's centre too far and the ball is missed. Two ends at 1/16 each make the computer fail 2/16 = 1 time in 8. That is a game-design decision taken with arithmetic you can do in your head.

The same trick gives a die: add two and you get 2 to 12 with 7 the most common.

Seeding

The LFSR is completely deterministic. Start it at the same seed and it produces the same sequence every time. If the program sets `SEED` to `$ACE1` at the start and then just uses the numbers, every game would begin identically. PONG avoids that without any clock or noise source: **while it waits for the player to press SPACE, it calls `RAND` in a loop** and throws the answers away. The state advances every time round. The player presses the key at an unpredictable moment, so the state at that moment, and therefore all the numbers that follow, vary from game to game. The time between the player's presses is the entropy.

You will find this loop in `WAITSP` in the PONG listing (Chapter 21): `JSR RAND` is the first instruction of the loop that waits for the key. It is the cheapest random seed there is, and it is the only randomness available to a program on a machine with no clock.

The initial `LDX #$ACE1 / STX SEED` is not the source of randomness; it merely guarantees that the state is not zero.

Testing a random number generator

You cannot prove a generator is random by looking, but you can check that it is *the generator you meant* and that it is not obviously biased:

1. **Same as a model.** T10RAND's first test is that the first sixteen bytes equal Python's. This catches coding mistakes (a wrong tap, a missing `TST`).
2. **Histogram.** T10RAND then draws 1,024 more values, masks each to 0 to 7 and counts how often each value occurs in eight 16-bit counters. The counts on its last row are in hex:

   `007D 0075 0075 008E 0074 008F 008E 007A`

   which are 125, 117, 117, 142, 116, 143, 142 and 122. The expected value for a perfectly uniform generator is 1,024 / 8 = 128 each. The counts are all within about 12 of 128 and total 1,024, as they must. The sum of (count - 128)² / 128 is 8.2; for seven degrees of freedom that is an entirely ordinary value (the 5% point of the statistic is about 14), so there is no evidence of bias. The test in `tests.py` does not use the statistic: it compares the 6800's counts with the counts computed by the model, which is a stricter check, because it demands equality, not plausibility.

   A word of caution. A good histogram says the values are *spread evenly*, not that they are *unpredictable*. An LFSR fails many tests of randomness (consecutive outputs are related by the shift, as the first eight bytes show). For a game of Pong that is irrelevant.
3. **Period.** Run the generator 65,535 times in the model and check that it returns to the seed. That checks the taps, not the code.

The counters, in passing

The program that builds the histogram also shows another technique. Each counter is two bytes, and the code adds one with:

```
        INC     1,X             ; low byte
        BNE     NOCARRY
```

```
        INC     0,X             ; carry into high byte
```

`INC 1,X` raises the low byte; if it wrapped to zero (`BNE` not taken) the high byte is incremented. This is a 16-bit increment in memory that does not touch A, and it works through any pointer in X. To get at the right counter the program loads X with the table's address and adds twice the index with `XPLUSA` (which adds A to X; the 6800 has no such instruction, and the library routine is from the firmware, Chapter 23).

Summary

- A 16-bit Galois LFSR with taps `$B400` is a cheap generator: one shift, one conditional XOR.
- The seed must not be zero. The generator does not repeat for 65,535 steps.
- Mask for powers of two; add two small numbers for a bell shape; take one bit for a coin toss.
- Spin the generator while waiting for a keypress to seed it unpredictably.
- Test a generator against a model, and look at a histogram.

Exercises

1. By hand, compute the second output of the generator from the state `$E270`. (Answer: the low bit is 0, so the state becomes `$7138`, and the output is `$38`, the second byte on T10RAND's first row.)
2. Change the taps to `$B401`. What is wrong? Run the model and measure the period.
3. Write a routine `DIE` returning 1 to 6 in A. Why is it harder than `ANDA #7`? Describe two methods.
4. In PONG's serve, the ball's vertical rate is `ANDA #1 / INCA`, giving 1 or 2. What would `ANDA #3 / INCA` give, and which of the game's variables would it break? (Chapter 20 explains the rate.)
5. Use the sum of three 0-1 rolls instead of two 0-3 rolls. What is the probability of the extreme values?

Chapter 20

PONG: the design

Everything so far was practice. This chapter and the next two describe one complete program, PONG, written for plug-in socket A. It is the program you built toward: it draws a court, moves two bats, moves a ball that bounces and speeds up, keeps score, plays a bell, and lets you pause. Its code and text occupy `$E100` to `$E679`, 1,402 bytes, in a 2 KB ROM image (plus a small entry frame at the bottom of the socket), and every routine in it is one you have already met.

Before the code, the design. A program of this size cannot be written by starting at the top and typing. Decide what the *screen* looks like, what the *state* is, and what happens in *one tick*; the code then follows almost mechanically.

> **Status of this program.** PONG has been assembled, byte-checked against the delivered `PONG.BIN`, and run in the emulator many thousands of ticks (Chapter 22). **It has also been run on a real ADM-31, and it works.** Everything the program uses (the PIA keyboard scan, the HALT strobe, the CRT RAM, the bell bit, the cursor-hiding pair) is taken from programs that are known to work on real machines.

The screen

```
row  0   YOU 0               ADM-31 PONG  By: Mickey Lawless               CPU 0
row  1  ================================================================================
rows 2-21        :     (centre line, column 40)
         #                                                              #
         (left bat, column 4)                          (right bat, column 75)
row 22  ================================================================================
row 23   message line (prompts, PAUSED, win/lose)
```

```
    YOU 0               ADM-31 PONG  By: Mickey Lawless               CPU 0
================================================================================
                                        :
                                        :
                                        :
                                        :
                                        :
                                        :
                                        :
    #                                   :                                   #
    #                                   :                                   #
    #                                   :                                   #
    #                                   :                                   #
```

```
    #                                        :                                        #
                                             :
                                             :
                                             :
                                             :
                                             :
                                             :
                                             :
                                             :
================================================================================
                SPACE=SERVE  UP/DOWN OR W/S=MOVE  P=PAUSE
```

The court is rows 2 to 21 (20 rows). Rows 1 and 22 are walls made of = (`$3D`). The centre line is a column of : (`$3A`) in column 40. Each bat is five cells of # (`$23`): your bat is in column 4 and the computer's in column 75. The ball is an O (`$4F`). Row 0 holds the captions and the two scores, and row 23 is a message line. A handful of character codes do all the work, and **each means something to the program**: the ball learns what it has touched by reading the code of the cell it is about to enter (Chapter 18).

The scores are single digits, drawn at columns 8 and 74 of row 0. The title text, including the author's credit, is at column 24 of row 0.

The state

Everything the program remembers is in RAM at `$0100-$0126`. The names are the source's own (Appendix E has the full listing):

Address	Name	Meaning
$0100	HPAD	your bat: byte 0 = top row, byte 1 = column
$0102	CPAD	the computer's bat, same layout
$0104	BROW	ball row
$0105	BCOL	ball column
$0106	BHDIR	bit 7 set = ball moving right
$0107	BVDIR	bit 7 set = ball moving down
$0108	BVRATE	the ball takes a vertical step every N ticks; 0 = flat
$0109	BVCNT	ticks left before the next vertical step
$010A	CURADR	2 bytes: CRT address of the ball's cell
$010C	OLDADR	2 bytes: address of the ball's previous cell
$010E	OLDCOL	the ball's previous column
$010F	SCOREH	your score, 0 to 7
$0110	SCOREC	the computer's score, 0 to 7
$0111	TICK	tick counter (its low bit paces the computer)
$0112	AIMOFF	the computer's aiming error, -3 to +3
$0113	POINT	0 = rally on, 1 = you scored, 2 = computer scored
$0114	LOSER	$80 = next serve goes right (toward the computer), $7F = left
$0115	DLYCNT	2 bytes: the length of one tick, in delay-loop passes
$0117	SEED	2 bytes: the random generator's state
$0119	TRIES	limit on re-testing the ball's cell
$011A-$0126		argument and scratch bytes for the screen routines

Two conventions are worth noticing. **Direction is a sign bit.** `BHDIR` and `BVDIR` are tested with `TST` followed by `BMI`/`BPL`, and reversed with `COM`, which complements every bit and so flips bit 7 (`$80` becomes `$7F` and back). A direction flag therefore costs two instructions to read and one to reverse. **A bat is a record**: two bytes, a row and a column, so one routine (`MOVPAD`, `DRAWPAD`) serves both bats if it is given a pointer in X to the record.

One tick

The main loop does the same four things over and over:

1. **Poll the keys** and move your bat.
2. **Computer's move**: decide whether to move its bat.
3. **Ball**: move it, test what it hit, react.
4. **Delay**: waste time so the game runs at a playable speed.

then it checks `POINT`; if somebody scored it plays the bell, updates the score, and either ends the match or goes back to wait for the next serve. This order is a design decision. Moving the bats *before* the ball means a bat that has just moved is already in place when the ball tests the cell it is about to enter.

The ball

The ball moves **one column every tick**, always. Horizontal speed is therefore fixed by the length of a tick; only the vertical speed varies. A ball that moves one cell horizontally per tick cannot jump over a bat, so the collision test (read the cell, look for `#`) cannot miss it. A faster horizontal speed would need a more careful test.

The vertical speed is `BVRATE`, a **tick divider**. A rate of 1 means a row step on every tick (a 45-degree path, steep); 2 means a row step on every second tick (a shallow path, two columns per row); 0 means no row step (a flat path). Each tick `BALLROW` decrements `BVCNT`; when it reaches zero the ball steps a row and `BVCNT` is reloaded from `BVRATE`. Three angles per direction need no multiplication and no fractions.

Each tick, the ball's new cell is **read** before the ball is drawn. What the program finds there decides what happens:

Code in the cell	Meaning	Reaction
0	blank	the move stands
`:`	the centre line	the move stands (the ball passes over it)
`#`	a bat	reverse horizontal direction, choose the angle, step back out, test again
`=`	a wall	reverse vertical direction, step back out, test again
anything else		treated as free

After the cell is accepted the program checks the *column*: if the ball is in column 1 or lower, it got past your bat and the computer scores; if column 78 or higher, you score. Nothing else is needed to decide a point.

The old cell is blanked afterwards, but **only if it still holds the ball** (and with `:` rather than a blank if it was in column 40, so the centre line is restored). That check handles the awkward case where the ball bounced and is now back in a cell it has just drawn.

The angle of return

A ball that always came back at the same angle would be dull. PONG lets the player aim by **where on the bat the ball lands**. The bat has five cells, numbered 0 (top) to 4 (bottom) by subtracting the bat's top row from the ball's row:

Cell hit	Return
0	up, steep (rate 1)
1	up, shallow (rate 2)
2	flat (rate 0)
3	down, shallow (rate 2)
4	down, steep (rate 1)

The same code handles both bats. It chooses which bat's record to use by the ball's column: the left half of the court (column below 40) is your bat's.

Speed

At the start of each serve the tick delay `DLYCNT` is 6,400 passes of the two-instruction loop `DEX`/`BNE`. Each pass is 8 clock cycles by the data sheet (4 + 4), so a tick's delay is 51,200 cycles, plus the work of the tick itself. **Every bat hit (either bat) shortens the delay by 200 passes.** The rule is not "down to a floor" but a test on the delay's high byte: if it is already below `$0E`, no further reduction. Starting from 6,400 (`$1900`), the sequence is 6,400, 6,200, … 3,600, 3,400, and at 3,400 (`$0D48`) the high byte is `$0D`, below `$0E`, so it stops: after **15 hits** the tick is 3,400 passes, 27,200 cycles. (The source's comment calls the floor "about 3,600"; the arithmetic gives 3,400. The program does what the code says, and the emulator test confirms the delay is always a multiple of 200 between 3,400 and 6,400.) A new serve resets the delay to 6,400.

How long is a tick in seconds? That depends on the clock, and this book does not know the clock. If it were 1 MHz, a start tick would be about 51 ms and a floor tick about 27 ms (plus the program's own work), but **that 1 MHz is an assumption, not a measurement**. The speeds were chosen by the author on the machine by trial; if the game feels too fast or too slow on your terminal, change `DLYSTART` and the `SUBA #200`.

The computer's player

The computer moves its bat **at most once every other tick**, which makes it slower than you (you can move every tick). On each of its ticks it chooses a *target row*:

- if the ball is moving toward the computer (`BHDIR` bit 7 set), the target is the ball's row plus `AIMOFF`;
- otherwise the target is row 11, roughly the middle of the court, so the bat drifts home.

The bat's *centre* (top row + 2) is compared with the target and the bat moves one row toward it, or stays if they are equal. The comparison is **signed** (`BGT`), because the target can be negative-looking near the top (ball row 2, error -3 gives -1).

`AIMOFF` is the computer's *mistake*. It is chosen **each time you hit the ball** (not when the computer hits it), by adding two random 0-3 numbers and subtracting 3 (Chapter 19), so it is -3 to +3 and bell-shaped. The computer therefore aims at a point that is a few rows off, consistently, for that whole return. A bat covers its centre plus or minus 2, so an error of plus or minus 3 puts the ball outside the bat: the computer misses about one return in eight, and otherwise returns the ball with some accuracy. The player can win.

Keys

Keys	Scan codes	Action
UP arrow, W, LEFT arrow	$53, $20, $54	your bat up one row
DOWN arrow, S, RIGHT arrow	$52, $31, $55	your bat down one row
SPACE or CLEAR	$4B, $40	serve; also start a new match
P	$28	pause (P again resumes)

The scan codes are all in the keyboard table of Appendix D. The key table has six entries, in the pair format of Chapter 16; where two keys are held at once, the first in the table wins. The source's own comment notes that the LEFT and RIGHT arrows ($54, $55) are Rubout's keys. The program reads the key matrix directly (Chapter 9) rather than waiting for a character, so a key is "down" for exactly as long as it is held.

The match

- The match is **first to 7**.
- A new match begins with a clear screen, both scores at 0 and the first serve to the computer.
- Each serve places the ball in column 40 at a random row from 4 to 19, aimed toward the player who just lost the point (so the loser always receives), at a random vertical direction and a random rate of 1 or 2.
- The program then waits for SPACE or CLEAR. While it waits it keeps calling the random generator, and that is where the randomness of the serve comes from (Chapter 19).
- When a score reaches 7, a message ("YOU WIN! SPACE = NEW GAME" or "CPU WINS SPACE = NEW GAME") appears on row 23, and the next SPACE starts a new match.
- Each point, and each bat hit, sounds the bell.

Hardware use

Everything PONG touches on the machine is listed here, with how each use is justified in the book's verification scheme (Chapter 1). Nothing else is used during play: no serial port, no interrupt, no firmware routine.

Use	Address	Justification
keyboard scan out / state in	`$7800` (bit 7 = key down)	read by Rubout, Shuttle Attack and the firmware
bit 2 of PIA1B cleared at start	`$7802`	copied from Rubout's reset

Use	Address	Justification
bell bit pulsed	`$7803` bit 3	Rubout's `BEEP`
HALT strobe before screen access	`$7000`	every working program
cursor hidden	`$7F06` = `$17`, `$7F0C` = `$E8`	Rubout's reset sequence
screen memory	`$8000`-`$877F`	every working program

The program never enables interrupts (it starts with `SEI`) and, once started, never calls the firmware. It has the boot hooks for both firmware sets (Chapter 25), and in the emulator it starts correctly from the plug-in-only boot and from both real firmware sets with ESC ` (Chapter 22).

Why this design

It is worth saying why the program looks the way it does, because every choice is a trade-off:

- **No back buffer.** The screen *is* the state: the program asks it where things are. That makes the code short and the collision rules automatic, and it costs a screen read per tick.
- **No arithmetic on positions beyond adding and comparing.** There is no multiplication in the main loop. The only address arithmetic (`CALCADR`) is done when a cell is drawn.
- **One tick, one cell.** The ball cannot pass through anything.
- **All parameters are numbers in one place.** `DLYSTART`, `WINSC`, and the character codes are `EQU` constants at the top of the source.
- **Table-driven keys** (Chapter 16) and **sign-bit direction flags** keep the code branch-free where possible.

The next chapter takes the real assembler listing and goes through it routine by routine.

Exercises

1. Make the game first to 11. Which constant changes? (`WINSC`.) What else limits the score, and what goes wrong if you ignore it? (The score is a single digit on the screen: it must stay within 0-9.)
2. Add a third angle to each side by having `BVRATE` take the value 3. Which two places in the program would have to know?
3. The ball's horizontal speed is fixed. Describe, without writing code, how you would make the ball move two columns per tick at the end of a long rally, and what new collision problem appears.
4. Change the computer so that it moves every tick. What is the effect on the game? How would you give the player a handicap instead?
5. Why is `AIMOFF` re-chosen when *you* hit the ball and not when the computer does?

Chapter 21

PONG: the code, routine by routine

This chapter walks through PONG's real assembler listing. Every excerpt below is copied, unedited, from `PONG.LST`, the listing produced by the assembler from `PONG.ASM`; the bytes in the second column are the bytes in `PONG.BIN`. (Appendix E prints the whole listing.) Read the chapter with the design of Chapter 20 beside you: the design says *what*, the listing says *how*.

A note on the layout of the listing. The assembler shows 53 characters of each source line and cuts longer lines. Where a line's comment would not fit beside the instruction, the listing in this book shows the comment on its own line **above** the instruction it belongs to. The code is the same; only the display differs. Addresses on a comment-only line are those of the instruction that follows.

The frame and the entry points

The first bytes of the ROM are the plug-in frame of Chapter 5. They are the part you copy unchanged into every plug-in:

```
E002                130  KEYENTRY
E002                131  ; $E002: same entry as ROM C's jump, so an
E002  7E E0 3C      132                  JMP     $E03C
E005                133  ; emulator that starts a plugin at $E000 (ADM31EMU
E005                134  ; --boot plugin) runs the
E005                135  ; game.  We SEI and poll the keys ourselves, so
E005                136  ; nothing else needs KEY_ENTRY.
E005                137
E005                138  ; ESC-` (grave, $60) entry sled.  The 129867 firmware
E005                139  ; JMPs to $E03C, the 129827 firmware
E005                140  ; JMPs to $E03E; NOPs let either one fall through to
E005                141  ; the same JMP (the same
E005                142  ; trick Shuttle Attack uses at $E03C).
E005                143  ; 129867 firmware enters here
E03C                144                  ORG     $E03C
E03C                145  ; $E03C: slide into the JMP
E03C  01            146                  NOP
E03D                147  ; $E03D: slide into the JMP
E03D  01            148                  NOP
E03E                149  ; $E03E: 129827 firmware enters here
E03E  01            150                  NOP
E03F                151  ; $E03F: start the game
E03F  7E E1 00      152                  JMP     RESET
```

```
E042                153
E042                154  ; Boot hooks.  At power-up both firmwares see "TJ",
E042                155  ; INC the plugin flag at
E042                156  ; $0000 and JSR into the plugin ($E07D on 129867,
E042                157  ; $E087 on 129827).  With the
E042                158  ; flag left set the firmware skips its own task-table
E042                159  ; setup and expects the
E042                160  ; plugin to supply one, so a plugin that does not
E042                161  ; must clear the flag and send
E042                162  ; the firmware down its normal (no-plugin) boot path.
E042                163  ; Space Invaders and
E042                164  ; Cosmic Intruder do exactly this: CLR $0000, drop
E042                165  ; the return address, and
E042                166  ; JMP to the firmware's "no plugin" label ($E8D0 /
E042                167  ; $E9D0).
E042                168  ; 129867 firmware boot hook
E07D                169                  ORG     $E07D
E07D                170  ; clear the plugin flag: stock firmware runs
E07D  7F 00 00      171  PLUGCB2          CLR     $0000
E080                172  ; drop the JSR return address (high byte)
E080  32            173                  PULA
E081                174  ; drop the JSR return address (low byte)
E081  32            175                  PULA
E082                176  ; 129867 "no plugin" boot path
E082  7E E8 D0      177                  JMP     $E8D0
E085                178  ; 129827 firmware boot hook
E087                179                  ORG     $E087
E087                180  ; clear the plugin flag: stock firmware runs
E087  7F 00 00      181  STUB87          CLR     $0000
E08A                182  ; drop the JSR return address (high byte)
E08A  32            183                  PULA
E08B                184  ; drop the JSR return address (low byte)
E08B  32            185                  PULA
E08C                186  ; 129827 "no plugin" boot path
E08C  7E E9 D0      187                  JMP     $E9D0
E08F                188
E08F                189  ; With the flag cleared the firmware never calls the
E08F                190  ; remaining callbacks; they
E08F                191  ; stay as RTS in case something does.
E08F                192  ; 129827 firmware ESC callback
E094                193                  ORG     $E094
E094                194  ; stub: return at once
E094  39            195  STUB94          RTS
E095                196  ; 129827 firmware ESC callback
E0C3                197                  ORG     $E0C3
E0C3                198  ; stub: return at once
E0C3  39            199  STUBC3          RTS
E0C4                200  ; 129867 firmware task-table callback
E0CA                201                  ORG     $E0CA
E0CA                202  ; stub: return at once
E0CA  39            203  PLUGCB3         RTS
E0CB                204
E0CB                205  ; ================================================
E0CB                206  ; RESET -- one-time init (same hardware setup as
E0CB                207  ; Rubout's RESET / RESET_011)
E0CB                208  ; ================================================
E0CB                209  ; main program starts here
```

```
E100                 210                  ORG     $E100
```

- `$E000`: the two bytes `54 4A` are "TJ". The firmware looks for them to decide that a plug-in is present.
- `$E002`: `JMP $E03C`. A firmware that starts a plug-in jumps here (and some emulators start at `$E000`).
- `$E03C-$E03E`: three `NOP`s and a `JMP RESET`. ESC `` ` `` (the grave accent) makes the **129867** firmware jump to `$E03C` and the **129827** firmware jump to `$E03E`. The `NOP`s are a "sled": enter at either address and the processor slides on to the same `JMP`. (Chapter 25 shows how the firmware chooses these entries; Appendix H lists the differences between the two firmware sets.)
- `$E07D` and `$E087`: boot hooks, one for each firmware. At power-up both firmwares see "TJ", increment a flag at `$0000` and call into the plug-in. A plug-in that supplies no task table of its own has to clear the flag, throw away the return address pushed by the call (two `PULA`s) and jump back to the firmware's normal boot, at `$E8D0` (129867) or `$E9D0` (129827). The source's comment says Space Invaders and Cosmic Intruders do exactly this; Chapter 25 compares them.
- `$E094`, `$E0C3`, `$E0CA`: `RTS` stubs, in case the firmware calls any of the other hooks.

The frame, `$E000-$E0CA`, is mostly empty space between the hooks; the game proper starts at `$E100`.

RESET and the match loop

```
E100                 211  RESET
E100                 212  ; own the machine, we poll the keys
E100  0F             213                  SEI
E101                 214  ; stack pointer to the top of RAM
E101  8E 03 FF       215                  LDS     #$03FF
E104                 216  ; first half of the cursor-hide pair
E104  86 17          217                  LDAA    #$17
E106                 218  ; write it to the CRT controller
E106  B7 7F 06       219                  STAA    CRTC6
E109                 220  ; complement it: A becomes $E8
E109  43             221                  COMA
E10A                 222  ; $E8 = cursor parked off screen
E10A  B7 7F 0C       223                  STAA    CURCOL
E10D  B6 78 02       224                  LDAA    PIA1B     ; read PIA1 port B
E110                 225  ; clear bit 2, as Rubout does
E110  84 FB          226                  ANDA    #$FB
E112                 227  ; write PIA1 port B back
E112  B7 78 02       228                  STAA    PIA1B
E115                 229  ; non-zero seed for the random LFSR
E115  CE AC E1       230                  LDX     #$ACE1
E118                 231  ; store the 16-bit seed
E118  FF 01 17       232                  STX     SEED
E11B                 233  ; tick counter starts at 0
E11B  7F 01 11       234                  CLR     TICK
E11E                 235  ; computer's aim error starts at 0
E11E  7F 01 12       236                  CLR     AIMOFF
E121                 237  NEWMATCH
E121                 238  ; blank the whole screen
E121  BD E4 99       239                  JSR     CLRSCR
E124  7F 01 0F       240                  CLR     SCOREH    ; your score = 0
```

```
E127                241  ; computer's score = 0
E127  7F 01 10      242                  CLR     SCOREC
E12A                243  ; $80 = serve toward the computer
E12A  86 80         244                  LDAA    #$80
E12C                245  ; first serve goes toward the computer
E12C  B7 01 14      246                  STAA    LOSER
E12F                247  ; draw court, scores, paddles
E12F  BD E4 ED      248                  JSR     DRAWFLD
E132                249  SERVEPT
E132                250  ; clear the message row
E132  BD E4 AA      251                  JSR     MSGCLR
E135  86 17         252                  LDAA    #23       ; message row 23
E137                253  ; row argument for PUTSTR
E137  B7 01 1A      254                  STAA    ROWARG
E13A  86 0F         255                  LDAA    #15       ; message column 15
E13C                256  ; column argument for PUTSTR
E13C  B7 01 1B      257                  STAA    COLARG
E13F                258  ; X points at the serve prompt
E13F  CE E6 14      259                  LDX     #SSERVE
E142  BD E4 77      260                  JSR     PUTSTR    ; print the prompt
E145                261  ; wait for SPACE/CLEAR (spins the RNG)
E145  BD E4 3B      262                  JSR     WAITSP
E148  BD E4 AA      263                  JSR     MSGCLR    ; erase the prompt
E14B                264  ; place the ball, pick its direction
E14B  BD E1 BB      265                  JSR     SERVE
E14E                266
E14E                267  ; ---- main game loop: one tick = poll, computer,
E14E                268  ; ball, delay ---------------
E14E                269  GAME
E14E                270  ; read the keys, move your paddle
E14E  BD E3 58      271                  JSR     POLLHUM
E151                272  ; computer moves its paddle
E151  BD E3 DD      273                  JSR     CPUAI
E154                274  ; move the ball a cell, test collisions
E154  BD E2 12      275                  JSR     BALLMOV
E157                276  ; X = current tick delay count
E157  FE 01 15      277                  LDX     DLYCNT
E15A                278  ; count the delay down ...
E15A  09            279  GDLY            DEX
E15B                280  ; ... until X reaches 0
E15B  26 FD         281                  BNE     GDLY
E15D  7C 01 11      282                  INC     TICK      ; count this tick
E160                283  ; has anyone scored?
E160  7D 01 13      284                  TST     POINT
E163                285  ; no: run the next tick
E163  27 E9         286                  BEQ     GAME
E165                287
E165                288  ; ---- somebody scored ------------------------------
E165                289  ; -------------------------
E165                290  ; sound the bell for the point
E165  BD E5 DE      291                  JSR     BEEP
E168                292  ; A = 1 you scored, 2 computer scored
E168  B6 01 13      293                  LDAA    POINT
E16B  81 01         294                  CMPA    #1        ; was it you?
E16D                295  ; no: the computer scored
E16D  26 0A         296                  BNE     GP_CPU
E16F                297  ; add a point for you
```

```
E16F  7C 01 0F       298                  INC     SCOREH
E172                 299  ; $80 = next serve goes to the computer
E172  86 80          300                  LDAA    #$80
E174                 301  ; computer lost the point: serve to it
E174  B7 01 14       302                  STAA    LOSER
E177                 303  ; skip the computer's branch
E177  20 08          304                  BRA     GP_SHOW
E179                 305  ; add a point for the computer
E179  7C 01 10       306  GP_CPU          INC     SCOREC
E17C                 307  ; $7F = next serve goes to you
E17C  86 7F          308                  LDAA    #$7F
E17E                 309  ; you lost the point: serve to you
E17E  B7 01 14       310                  STAA    LOSER
E181                 311  ; redraw both scores
E181  BD E4 B3       312  GP_SHOW         JSR     SHOWSC
E184  B6 01 0F       313                  LDAA    SCOREH    ; A = your score
E187                 314  ; reached the winning score?
E187  81 07          315                  CMPA    #WINSC
E189                 316  ; no: check the computer
E189  26 05          317                  BNE     GP_N1
E18B                 318  ; X points at the "YOU WIN" text
E18B  CE E6 45       319                  LDX     #SWIN
E18E  20 0F          320                  BRA     ENDGAME   ; go show it
E190                 321  ; A = computer's score
E190  B6 01 10       322  GP_N1           LDAA    SCOREC
E193                 323  ; reached the winning score?
E193  81 07          324                  CMPA    #WINSC
E195  26 05          325                  BNE     GP_NEXT   ; no: play on
E197                 326  ; X points at the "CPU WINS" text
E197  CE E6 60       327                  LDX     #SLOSE
E19A  20 03          328                  BRA     ENDGAME   ; go show it
E19C                 329  ; serve the next point
E19C  7E E1 32       330  GP_NEXT         JMP     SERVEPT
E19F                 331
E19F                 332  ENDGAME
E19F                 333  ; save the text pointer (MSGCLR uses X)
E19F  FF 01 22       334                  STX     KPTR
E1A2                 335  ; clear the message row
E1A2  BD E4 AA       336                  JSR     MSGCLR
E1A5  86 17          337                  LDAA    #23       ; message row 23
E1A7                 338  ; row argument for PUTSTR
E1A7  B7 01 1A       339                  STAA    ROWARG
E1AA  86 16          340                  LDAA    #22       ; message column 22
E1AC                 341  ; column argument for PUTSTR
E1AC  B7 01 1B       342                  STAA    COLARG
E1AF                 343  ; restore the text pointer
E1AF  FE 01 22       344                  LDX     KPTR
E1B2                 345  ; print the win/lose message
E1B2  BD E4 77       346                  JSR     PUTSTR
E1B5                 347  ; wait for SPACE or CLEAR
E1B5  BD E4 3B       348                  JSR     WAITSP
E1B8  7E E1 21       349                  JMP     NEWMATCH  ; start a new match
E1BB                 350
```

- `SEI` forbids interrupts: PONG polls the keyboard and wants no surprises. `LDS #$03FF` puts the stack at the top of RAM, as every program in this book does.

- **Hiding the cursor.** Registers `$7F06` and `$7F0C` of the display controller position the cursor. PONG writes `$17` to one and `$E8` to the other, which parks the cursor off the visible screen. The `COMA` is a small economy: `$E8` is the complement of `$17`, so the program does not need to load a second constant. This pair is the one Rubout uses.
- `LDAA PIA1B / ANDA #$FB / STAA PIA1B` clears bit 2 of the PIA's port B. The source says it does this "as Rubout does"; it is the display-page bit (Appendix B), and the book has only the working programs' word that clearing it is right.
- `LDX #$ACE1 / STX SEED` loads the non-zero random seed (Chapter 19). `CLR TICK` and `CLR AIMOFF` initialise two variables that are used before they are set.

`NEWMATCH` clears the screen, zeros both scores, sets `LOSER` to `$80` (the first serve goes to the computer), and `DRAWFLD` draws the court. `SERVEPT` is the top of the loop that runs once per point: it clears the message row, prints the prompt `SPACE=SERVE ...` at row 23, column 15, and calls `WAITSP`, which does not return until SPACE or CLEAR is pressed. Then it blanks the prompt and calls `SERVE`.

The game loop

`GAME` is the loop that runs once per tick:

- `JSR POLLHUM` reads the keys and moves your bat.
- `JSR CPUAI` moves the computer's bat.
- `JSR BALLMOV` moves the ball and tests what it hit.
- `LDX DLYCNT / DEX / BNE` burns the tick's time.
- `INC TICK` counts the tick, and `TST POINT / BEQ GAME` loops until someone has scored.

When `POINT` becomes non-zero the code that follows plays the bell (`JSR BEEP`), adds one to the right score and sets `LOSER` so the next serve goes to the player who lost the point. `SHOWSC` redraws the scores; then the code compares each score to `WINSC` (7). If someone has won, it points X at the right message and goes to `ENDGAME`; otherwise `GP_NEXT` returns to `SERVEPT`.

Two small details. `GP_NEXT JMP SERVEPT` is a jump, not a branch, because `SERVEPT` is more than 127 bytes away; the assembler would refuse a `BRA`. And `ENDGAME` begins with `STX KPTR`: the message pointer is saved in memory because the very next call, `MSGCLR`, uses X, as Chapter 16 warned.

SERVE

```
E1BB                351  ; ================================================
E1BB                352  ; SERVE -- ball to the centre line at a random row,
E1BB                353  ; random angle
E1BB                354  ; ================================================
E1BB                355  SERVE
E1BB  86 28         356                  LDAA    #40       ; centre column
E1BD                357  ; ball starts on the centre line
E1BD  B7 01 05      358                  STAA    BCOL
E1C0  BD E5 BF      359                  JSR     RAND      ; random byte in A
E1C3  84 0F         360                  ANDA    #$0F      ; keep 0-15
E1C5  8B 04         361                  ADDA    #4        ; shift to 4-19
E1C7  B7 01 04      362                  STAA    BROW      ; rows 4-19
E1CA                363  ; A = serve direction ($80 or $7F)
E1CA  B6 01 14      364                  LDAA    LOSER
E1CD                365  ; ball heads toward the last loser
E1CD  B7 01 06      366                  STAA    BHDIR
```

```
E1D0  BD E5 BF       367                  JSR     RAND      ; random byte in A
E1D3                 368  ; bit 7 = random up / down
E1D3  B7 01 07       369                  STAA    BVDIR
E1D6  BD E5 BF       370                  JSR     RAND      ; random byte in A
E1D9                 371  ; keep one bit: 0 or 1
E1D9  84 01          372                  ANDA    #1
E1DB  4C             373                  INCA              ; now 1 or 2
E1DC                 374  ; 1 = steep, 2 = shallow
E1DC  B7 01 08       375                  STAA    BVRATE
E1DF                 376  ; first step countdown = rate
E1DF  B7 01 09       377                  STAA    BVCNT
E1E2                 378  ; starting tick delay
E1E2  CE 19 00       379                  LDX     #DLYSTART
E1E5                 380  ; set the delay count
E1E5  FF 01 15       381                  STX     DLYCNT
E1E8                 382  ; no point scored yet
E1E8  7F 01 13       383                  CLR     POINT
E1EB  B6 01 04       384                  LDAA    BROW      ; A = ball row
E1EE  B7 01 1A       385                  STAA    ROWARG    ; row argument
E1F1  B6 01 05       386                  LDAA    BCOL      ; A = ball column
E1F4  B7 01 1B       387                  STAA    COLARG    ; column argument
E1F7                 388  ; old column = start column
E1F7  B7 01 0E       389                  STAA    OLDCOL
E1FA                 390  ; X = CRTRAM address of the ball cell
E1FA  BD E5 65       391                  JSR     CALCADR
E1FD                 392  ; remember the current ball address
E1FD  FF 01 0A       393                  STX     CURADR
E200                 394  ; old address = current address
E200  FF 01 0C       395                  STX     OLDADR
E203                 396  ; A = ball character
E203  86 4F          397                  LDAA    #CH_BALL
E205                 398  ; sync to the CRT before writing
E205  B7 70 00       399                  STAA    HALT
E208                 400  ; let the strobe settle
E208  01             401                  NOP
E209  A7 00          402                  STAA    0,X       ; draw the ball
E20B                 403  ; ~0.5 s breather so the ball can be seen
E20B  CE 00 00       404                  LDX     #0
E20E                 405  ; burn time: 65536 passes ...
E20E  09             406  SV_DLY          DEX
E20F                 407  ; ... as the breather
E20F  26 FD          408                  BNE     SV_DLY
E211                 409  ; ball is in play: return
E211  39             410                  RTS
E212                 411
```

SERVE puts the ball in column 40, picks a random row from 4 to 19 (a mask and an offset, Chapter 19), and sets the direction from LOSER. The vertical direction is a whole random byte stored in BVDIR: only bit 7 matters. The vertical rate is a random 1 or 2 (mask to one bit, then INCA); BVCNT is loaded with the same number so the first row step comes after BVRATE ticks. The tick delay returns to DLYSTART (6,400) and POINT is cleared.

The ball is then drawn: CALCADR converts row and column to a CRT address in X, which is saved in both CURADR and OLDADR; the character $4F is stored behind the usual HALT strobe. The last four lines are a deliberate pause: LDX #0 followed by DEX/BNE runs the loop 65,536 times (a zero loaded into X wraps to $FFFF on the first DEX). The source comment calls this "~0.5 s"; that

figure assumes a 1 MHz clock that is not verified. It is 65,536 x 8 = 524,288 clock cycles, a pause long enough for the player to see where the ball is before it moves.

BALLMOV: moving the ball and reading the screen

This is the heart of the program, and it is the Chapter 18 idea made robust enough for a real game.

```
E212                412  ; ================================================
E212                413  ; BALLMOV -- Rubout's BALLMOV: advance column,
E212                414  ; advance row, read back the
E212                415  ; destination cell, branch on what is there.
E212                416  ; ================================================
E212                417  BALLMOV
E212                418  ; X = the ball's current cell address
E212  FE 01 0A      419                  LDX     CURADR
E215                420  ; keep it as the previous address
E215  FF 01 0C      421                  STX     OLDADR
E218  B6 01 05      422                  LDAA    BCOL      ; A = ball column
E21B                423  ; keep it as the previous column
E21B  B7 01 0E      424                  STAA    OLDCOL
E21E                425  ; re-test limit: 6 tries
E21E  86 06         426                  LDAA    #6
E220                427  ; store the re-test counter
E220  B7 01 19      428                  STAA    TRIES
E223                429  ; step the column by its direction
E223  BD E2 AE      430                  JSR     BALLCOL
E226                431  ; step the row (when due)
E226  BD E2 BB      432                  JSR     BALLROW
E229                433  BM_EVAL
E229  B6 01 04      434                  LDAA    BROW      ; A = ball row
E22C  B7 01 1A      435                  STAA    ROWARG    ; row argument
E22F  B6 01 05      436                  LDAA    BCOL      ; A = ball column
E232  B7 01 1B      437                  STAA    COLARG    ; column argument
E235                438  ; X = CRTRAM address of that cell
E235  BD E5 65      439                  JSR     CALCADR
E238                440  ; remember it as the current cell
E238  FF 01 0A      441                  STX     CURADR
E23B                442  ; sync to the CRT, then READ the cell
E23B  B7 70 00      443                  STAA    HALT
E23E                444  ; let the strobe settle
E23E  01            445                  NOP
E23F                446  ; B = character already in that cell
E23F  E6 00         447                  LDAB    0,X
E241                448  ; count down the re-test limiter
E241  7A 01 19      449                  DEC     TRIES
E244                450  ; safety valve: stop re-testing
E244  27 1F         451                  BEQ     BM_FREE
E246                452  ; is the cell blank?
E246  5D            453                  TSTB
E247  27 1C         454                  BEQ     BM_FREE   ; blank
E249  C1 3A         455                  CMPB    #CH_NET   ; centre line?
E24B                456  ; centre line is passable
E24B  27 18         457                  BEQ     BM_FREE
E24D  C1 23         458                  CMPB    #CH_PAD   ; paddle?
E24F  27 0C         459                  BEQ     BM_PAD    ; paddle
```

```
E251  C1 3D          460                  CMPB    #CH_WALL  ; wall?
E253                 461  ; anything else: treat as free
E253  26 10          462                  BNE     BM_FREE
E255                 463  ; wall: flip vertical direction ...
E255  73 01 07       464                  COM     BVDIR
E258                 465  ; ... step back out, re-test the cell
E258  BD E2 C8       466                  JSR     BR_STEP
E25B  20 CC          467                  BRA     BM_EVAL   ; re-test the cell
E25D                 468  BM_PAD
E25D                 469  ; flip horizontal, pick the angle
E25D  BD E2 D5       470                  JSR     PADHIT
E260                 471  ; step back out of the paddle column
E260  BD E2 AE       472                  JSR     BALLCOL
E263  20 C4          473                  BRA     BM_EVAL   ; re-test the cell
E265                 474
E265                 475  BM_FREE
E265  B6 01 05       476                  LDAA    BCOL      ; A = ball column
E268  81 02          477                  CMPA    #2        ; column below 2?
E26A                 478  ; got past your paddle
E26A  25 34          479                  BCS     BM_HMISS
E26C                 480  ; column 78 or more?
E26C  81 4E          481                  CMPA    #78
E26E                 482  ; got past the computer's paddle
E26E  24 37          483                  BCC     BM_CMISS
E270                 484  ; X = the new ball cell
E270  FE 01 0A       485                  LDX     CURADR
E273                 486  ; A = ball character
E273  86 4F          487                  LDAA    #CH_BALL
E275                 488  ; sync to the CRT before writing
E275  B7 70 00       489                  STAA    HALT
E278                 490  ; let the strobe settle
E278  01             491                  NOP
E279  A7 00          492                  STAA    0,X       ; draw the ball
E27B                 493  BM_ERASE
E27B                 494  ; X = the previous ball cell
E27B  FE 01 0C       495                  LDX     OLDADR
E27E                 496  ; bounced straight back into the same cell?
E27E  BC 01 0A       497                  CPX     CURADR
E281                 498  ; then the ball we just drew is the one there
E281  27 1C          499                  BEQ     BM_DONE
E283                 500  ; sync to the CRT before reading
E283  B7 70 00       501                  STAA    HALT
E286                 502  ; let the strobe settle
E286  01             503                  NOP
E287                 504  ; A = what is in the old cell now
E287  A6 00          505                  LDAA    0,X
E289                 506  ; only blank it if it is still the ball
E289  81 4F          507                  CMPA    #CH_BALL
E28B                 508  ; not the ball: leave it alone
E28B  26 12          509                  BNE     BM_DONE
E28D                 510  ; A = column of the old cell
E28D  B6 01 0E       511                  LDAA    OLDCOL
E290                 512  ; was it the centre column?
E290  81 28          513                  CMPA    #40
E292                 514  ; yes: put the net back
E292  27 03          515                  BEQ     BM_NET
E294  4F             516                  CLRA              ; no: use a blank
```

```
E295  20 02          517                  BRA     BM_WR     ; go write it
E297                 518  ; restore the centre line
E297  86 3A          519  BM_NET          LDAA    #CH_NET
E299                 520  ; sync to the CRT before writing
E299  B7 70 00       521  BM_WR           STAA    HALT
E29C                 522  ; let the strobe settle
E29C  01             523                  NOP
E29D                 524  ; write the blank or net character
E29D  A7 00          525                  STAA    0,X
E29F  39             526  BM_DONE         RTS               ; return to GAME
E2A0                 527  BM_HMISS
E2A0                 528  ; point code 2: computer scores
E2A0  86 02          529                  LDAA    #2
E2A2  B7 01 13       530                  STAA    POINT     ; computer scores
E2A5                 531  ; go erase the old ball
E2A5  20 D4          532                  BRA     BM_ERASE
E2A7                 533  BM_CMISS
E2A7                 534  ; point code 1: you score
E2A7  86 01          535                  LDAA    #1
E2A9  B7 01 13       536                  STAA    POINT     ; you score
E2AC                 537  ; go erase the old ball
E2AC  20 CD          538                  BRA     BM_ERASE
E2AE                 539
```

Read it in four stages.

1. Remember where we were. The current address and column are copied to `OLDADR` and `OLDCOL`. `TRIES` is set to 6; you will see why in a moment.

2. Move. `BALLCOL` always steps the column by one. `BALLROW` steps the row when its tick divider says so. Both are described below.

3. Look at the destination (`BM_EVAL`). The new row and column are converted to an address with `CALCADR` and the address saved in `CURADR`. The cell is **read**: `STAA HALT` (the value in A does not matter, only the write), a `NOP`, and `LDAB 0,X` loads the character into B. B is used because A is still free for other jobs. Then:

- `DEC TRIES / BEQ BM_FREE`: the **safety valve**. Each pass through `BM_EVAL` uses up one try, and when the six are spent the cell is accepted whatever it holds. A bounce re-tests the cell it has just stepped back into, and in a corner that could in principle loop. Six re-tests is far more than any bounce needs; the limit guarantees the game cannot hang inside `BALLMOV`.
- `TSTB / BEQ BM_FREE`: a blank cell. Accept.
- `CMPB #CH_NET / BEQ BM_FREE`: the centre line is passable.
- `CMPB #CH_PAD / BEQ BM_PAD`: a bat.
- `CMPB #CH_WALL / BNE BM_FREE`: anything that is not a wall is accepted; this is where the ball's own character is accepted, as explained next.
- A **wall**: `COM BVDIR` flips the vertical direction, `JSR BR_STEP` moves the ball one row back the way it came, and `BRA BM_EVAL` tests the new cell.
- A **bat** (`BM_PAD`): `JSR PADHIT` flips the horizontal direction and chooses the new angle; `JSR BALLCOL` steps the column back out of the bat; `BRA BM_EVAL` tests again.

Why does the re-test not bounce forever off its own previous position? After a bounce the ball is stepped back into the cell it occupied on the previous tick, and **that cell still holds the ball's `0`**, because the old image is only erased at the very end. The test sees an `0`, which is none of blank, centre line, bat or wall, so the branch "anything else: treat as free" accepts it. The odd-looking

`BNE BM_FREE` on the wall test is therefore not sloppiness. It is what makes the re-test terminate. (And the safety valve covers the odd cases.)

4. Accept the cell (`BM_FREE`).

```
E265                475  BM_FREE
E265  B6 01 05      476                  LDAA    BCOL      ; A = ball column
E268  81 02         477                  CMPA    #2        ; column below 2?
E26A                478  ; got past your paddle
E26A  25 34         479                  BCS     BM_HMISS
E26C                480  ; column 78 or more?
E26C  81 4E         481                  CMPA    #78
E26E                482  ; got past the computer's paddle
E26E  24 37         483                  BCC     BM_CMISS
E270                484  ; X = the new ball cell
E270  FE 01 0A      485                  LDX     CURADR
E273                486  ; A = ball character
E273  86 4F         487                  LDAA    #CH_BALL
E275                488  ; sync to the CRT before writing
E275  B7 70 00      489                  STAA    HALT
E278                490  ; let the strobe settle
E278  01            491                  NOP
E279  A7 00         492                  STAA    0,X       ; draw the ball
E27B                493  BM_ERASE
E27B                494  ; X = the previous ball cell
E27B  FE 01 0C      495                  LDX     OLDADR
E27E                496  ; bounced straight back into the same cell?
E27E  BC 01 0A      497                  CPX     CURADR
E281                498  ; then the ball we just drew is the one there
E281  27 1C         499                  BEQ     BM_DONE
E283                500  ; sync to the CRT before reading
E283  B7 70 00      501                  STAA    HALT
E286                502  ; let the strobe settle
E286  01            503                  NOP
E287                504  ; A = what is in the old cell now
E287  A6 00         505                  LDAA    0,X
E289                506  ; only blank it if it is still the ball
E289  81 4F         507                  CMPA    #CH_BALL
E28B                508  ; not the ball: leave it alone
E28B  26 12         509                  BNE     BM_DONE
E28D                510  ; A = column of the old cell
E28D  B6 01 0E      511                  LDAA    OLDCOL
E290                512  ; was it the centre column?
E290  81 28         513                  CMPA    #40
E292                514  ; yes: put the net back
E292  27 03         515                  BEQ     BM_NET
E294  4F            516                  CLRA              ; no: use a blank
E295  20 02         517                  BRA     BM_WR     ; go write it
E297                518  ; restore the centre line
E297  86 3A         519  BM_NET          LDAA    #CH_NET
E299                520  ; sync to the CRT before writing
E299  B7 70 00      521  BM_WR           STAA    HALT
E29C                522  ; let the strobe settle
E29C  01            523                  NOP
E29D                524  ; write the blank or net character
E29D  A7 00         525                  STAA    0,X
E29F  39            526  BM_DONE         RTS               ; return to GAME
E2A0                527  BM_HMISS
```

```
E2A0                528  ; point code 2: computer scores
E2A0  86 02         529                  LDAA    #2
E2A2  B7 01 13      530                  STAA    POINT     ; computer scores
E2A5                531  ; go erase the old ball
E2A5  20 D4         532                  BRA     BM_ERASE
E2A7                533  BM_CMISS
E2A7                534  ; point code 1: you score
E2A7  86 01         535                  LDAA    #1
E2A9  B7 01 13      536                  STAA    POINT     ; you score
E2AC                537  ; go erase the old ball
E2AC  20 CD         538                  BRA     BM_ERASE
E2AE                539
```

The column decides the point: below 2 and the ball has gone past your bat (`BCS BM_HMISS`, which sets `POINT` to 2: the computer scores); 78 or more and it has gone past the computer's (`BM_CMISS`, `POINT = 1`). Both still erase the old ball, so no stray `0` is left behind. Otherwise the ball is drawn at `CURADR`.

Then the old image is removed (`BM_ERASE`):

- If the old and the new address are the same (`CPX CURADR / BEQ BM_DONE`), nothing more to do: the ball bounced straight back to where it was.
- Otherwise the old cell is read, and it is blanked **only if it still holds the ball**. This guards against blanking something else that has since been drawn there (a bat, say, if the bat moved onto the cell).
- If the old cell was in column 40, the centre line `:` is written back instead of a blank.

The whole accept-and-erase path, `BM_FREE` to the `RTS` at `BM_DONE`, is 59 bytes (`$E265-$E29F`).

Stepping the ball

```
E2AE                540  ; BALLCOL -- Rubout's BALLCOL: step column by the
E2AE                541  ; sign flag.
E2AE                542  BALLCOL
E2AE                543  ; test the horizontal direction flag
E2AE  7D 01 06      544                  TST     BHDIR
E2B1                545  ; bit 7 set: moving right
E2B1  2B 04         546                  BMI     BC_R
E2B3                547  ; moving left: column - 1
E2B3  7A 01 05      548                  DEC     BCOL
E2B6  39            549                  RTS               ; done
E2B7                550  ; moving right: column + 1
E2B7  7C 01 05      551  BC_R            INC     BCOL
E2BA  39            552                  RTS               ; done
E2BB                553
E2BB                554  ; BALLROW -- Rubout's BALLROW, with a tick divider so
E2BB                555  ; shallow shots move
E2BB                556  ; one row every second tick.
E2BB                557  BALLROW
E2BB                558  ; A = vertical rate (0 = flat)
E2BB  B6 01 08      559                  LDAA    BVRATE
E2BE  27 14         560                  BEQ     BR_X      ; flat shot
E2C0                561  ; count down to the next row step
E2C0  7A 01 09      562                  DEC     BVCNT
E2C3                563  ; not due yet: leave the row alone
E2C3  26 0F         564                  BNE     BR_X
E2C5                565  ; reload the countdown from the rate
```

```
E2C5  B7 01 09       566                  STAA    BVCNT
E2C8                 567  ; test the vertical direction flag
E2C8  7D 01 07       568  BR_STEP         TST     BVDIR
E2CB                 569  ; bit 7 set: moving down
E2CB  2B 04          570                  BMI     BR_D
E2CD                 571  ; moving up: row - 1
E2CD  7A 01 04       572                  DEC     BROW
E2D0  39             573                  RTS             ; done
E2D1                 574  ; moving down: row + 1
E2D1  7C 01 04       575  BR_D            INC     BROW
E2D4  39             576  BR_X            RTS             ; done
E2D5                 577
```

`BALLCOL`: test the horizontal flag; minus means right, so increment the column; otherwise decrement. `BALLROW`: a rate of 0 returns at once (flat path). Otherwise `DEC BVCNT`; if it is not yet zero, return; if it is zero, reload it from the rate (the value is still in A from the `LDAA BVRATE`) and fall into `BR_STEP`, which tests the vertical flag and increments or decrements the row. `BR_STEP` is also called directly from the wall code, which is why it is a label of its own, and it ends in the `RTS` shared with the flat-path exit (`BR_X`).

PADHIT: the angle, the speed, the computer's mistake

```
E2D5                 578  ; PADHIT -- the ball has entered a paddle cell.
E2D5                 579  ; Reverse horizontal
E2D5                 580  ; direction, choose the angle from where on the
E2D5                 581  ; paddle it landed, beep,
E2D5                 582  ; speed the rally up, and (when you hit it) re-roll
E2D5                 583  ; the computer's aim.
E2D5                 584  ; cell 0 up steep | 1 up shallow | 2 flat | 3 down
E2D5                 585  ; shallow | 4 down steep
E2D5                 586  PADHIT
E2D5                 587  ; reverse the horizontal direction
E2D5  73 01 06       588                  COM     BHDIR
E2D8                 589  ; assume it is your paddle
E2D8  CE 01 00       590                  LDX     #HPAD
E2DB  B6 01 05       591                  LDAA    BCOL      ; A = ball column
E2DE                 592  ; left half of the court?
E2DE  81 28          593                  CMPA    #40
E2E0                 594  ; yes: keep your paddle record
E2E0  25 03          595                  BCS     PH_REC
E2E2                 596  ; no: use the computer's paddle
E2E2  CE 01 02       597                  LDX     #CPAD
E2E5  F6 01 04       598  PH_REC          LDAB    BROW      ; B = ball row
E2E8                 599  ; B = 0..4, offset from paddle top
E2E8  E0 00          600                  SUBB    0,X
E2EA  26 07          601                  BNE     PH_1      ; not cell 0
E2EC  7F 01 07       602                  CLR     BVDIR     ; cell 0: head up
E2EF                 603  ; steep: row step every tick
E2EF  86 01          604                  LDAA    #1
E2F1  20 23          605                  BRA     PH_SET    ; apply it
E2F3  5A             606  PH_1            DECB              ; test for cell 1
E2F4  26 07          607                  BNE     PH_2      ; not cell 1
E2F6  7F 01 07       608                  CLR     BVDIR     ; cell 1: head up
E2F9                 609  ; shallow: row step every other tick
E2F9  86 02          610                  LDAA    #2
```

```
E2FB  20 19          611                  BRA     PH_SET    ; apply it
E2FD  5A             612  PH_2            DECB               ; test for cell 2
E2FE  26 03          613                  BNE     PH_3      ; not cell 2
E300                 614  ; cell 2: rate 0 = flat return
E300  4F             615                  CLRA
E301  20 13          616                  BRA     PH_SET    ; apply it
E303  5A             617  PH_3            DECB               ; test for cell 3
E304                 618  ; not cell 3, so it is cell 4
E304  26 09          619                  BNE     PH_4
E306  86 80          620                  LDAA    #$80      ; cell 3: head down
E308                 621  ; set the down direction
E308  B7 01 07       622                  STAA    BVDIR
E30B                 623  ; shallow: row step every other tick
E30B  86 02          624                  LDAA    #2
E30D  20 07          625                  BRA     PH_SET    ; apply it
E30F  86 80          626  PH_4            LDAA    #$80      ; cell 4: head down
E311                 627  ; set the down direction
E311  B7 01 07       628                  STAA    BVDIR
E314                 629  ; steep: row step every tick
E314  86 01          630                  LDAA    #1
E316                 631  ; store the vertical rate
E316  B7 01 08       632  PH_SET          STAA    BVRATE
E319                 633  ; first step countdown = rate
E319  B7 01 09       634                  STAA    BVCNT
E31C                 635  ; sound the bell for the hit
E31C  BD E5 DE       636                  JSR     BEEP
E31F                 637  ; speed up: -200 per hit, floor ~3600
E31F  B6 01 15       638                  LDAA    DLYCNT
E322                 639  ; delay high byte already low enough?
E322  81 0E          640                  CMPA    #$0E
E324                 641  ; yes: stop speeding up
E324  25 10          642                  BCS     PH_AIM
E326                 643  ; A = delay low byte
E326  B6 01 16       644                  LDAA    DLYCNT+1
E329                 645  ; shorten the delay by 200
E329  80 C8          646                  SUBA    #200
E32B                 647  ; store the low byte
E32B  B7 01 16       648                  STAA    DLYCNT+1
E32E                 649  ; A = delay high byte
E32E  B6 01 15       650                  LDAA    DLYCNT
E331                 651  ; carry the borrow into it
E331  82 00          652                  SBCA    #0
E333                 653  ; store the high byte
E333  B7 01 15       654                  STAA    DLYCNT
E336  B6 01 05       655  PH_AIM          LDAA    BCOL      ; A = ball column
E339                 656  ; left half of the court?
E339  81 28          657                  CMPA    #40
E33B                 658  ; computer's hit: nothing more
E33B  24 1A          659                  BCC     PH_X
E33D                 660  ; AIMOFF = -3..+3, bell-shaped (sum of two
E33D  BD E5 BF       661                  JSR     RAND
E340                 662  ; 0-3 rolls, minus 3).  +-3 is outside the
E340  B7 01 1E       663                  STAA    TMPA
E343                 664  ; paddle: the computer misjudges ~1 in 8.
E343  84 03          665                  ANDA    #3
E345                 666  ; keep the first 0-3 roll
E345  B7 01 1F       667                  STAA    TMPB
```

```
E348                668  ; reload the random byte
E348  B6 01 1E      669                  LDAA    TMPA
E34B                670  ; shift right twice ...
E34B  44            671                  LSRA
E34C                672  ; ... to bring bits 2-3 down
E34C  44            673                  LSRA
E34D  84 03         674                  ANDA    #3        ; second 0-3 roll
E34F                675  ; add the first roll: 0-6
E34F  BB 01 1F      676                  ADDA    TMPB
E352                677  ; centre on zero: -3..+3
E352  80 03         678                  SUBA    #3
E354                679  ; store the aim error
E354  B7 01 12      680                  STAA    AIMOFF
E357  39            681  PH_X            RTS               ; done
E358                682
```

- `COM BHDIR`: the ball turns round.
- **Whose bat?** X is loaded with `HPAD`, your bat's record, and replaced with `CPAD` if the ball's column is 40 or more. (`CMPA #40 / BCS` keeps your bat when the column is below 40.)
- **Which cell?** `LDAB BROW / SUBB 0,X` leaves 0 to 4 in B: the ball's row minus the bat's top row. A chain of `BNE`/`DECB` tests B for 0, then 1, 2, 3, and by elimination 4. Each case sets `BVDIR` and loads A with the new rate; they all meet at `PH_SET`, which stores the rate in `BVRATE` and `BVCNT`. Cell 2 loads zero, a flat return; the other four cases are the table of Chapter 20. `CLR BVDIR` sets "up" (bit 7 clear) and `LDAA #$80 / STAA BVDIR` sets "down".

  The chain is a *computed switch* on 6800 terms: a series of "is it zero, if not subtract one". A table of five addresses would take more bytes of code and data and is no clearer.
- `JSR BEEP` sounds the bell on every bat hit.
- **Speed up.** If `DLYCNT`'s high byte is below `$0E`, skip. Otherwise subtract 200 from the 16-bit value: `SUBA #200` on the low byte, `SBCA #0` on the high byte to take the borrow (Chapter 15). The result after 15 hits is 3,400 and stops there (Chapter 20).
- **The computer's aim.** If the ball's column is 40 or more it was the *computer's* hit and nothing more happens. If it was yours, a new `AIMOFF` is computed, as explained in Chapter 19. Notice that this routine uses `TMPA` and `TMPB` for the two rolls; `RAND` itself uses `TMPA` internally, but it finishes before the program stores its result there, so the sequence is safe.

POLLHUM and MOVPAD: your bat

```
E358                683  ; ================================================
E358                684  ; POLLHUM -- scan the human's keys; first one found
E358                685  ; moves the paddle 1 row
E358                686  ; ================================================
E358                687  POLLHUM
E358  86 28         688                  LDAA    #$28      ; P
E35A                689  ; scan code out to PIA1
E35A  B7 78 00      690                  STAA    PIA1
E35D                691  ; read the key state back
E35D  B6 78 00      692                  LDAA    PIA1
E360                693  ; bit 7 clear: P is not down
E360  2A 03         694                  BPL     PK_NP
E362  BD E4 0D      695                  JSR     PAUSE     ; P is down: pause
```

```
E365                    696  ; X -> table of (scan code, direction)
E365  CE E3 81          697  PK_NP           LDX     #KEYTAB
E368                    698  ; A = scan code of this entry
E368  A6 00             699  PK_LP           LDAA    0,X
E36A                    700  ; zero ends the table
E36A  27 14             701                  BEQ     PK_END
E36C                    702  ; scan code out to PIA1
E36C  B7 78 00          703                  STAA    PIA1
E36F                    704  ; read the key state back
E36F  B6 78 00          705                  LDAA    PIA1
E372                    706  ; bit 7 set: key is down
E372  2B 04             707                  BMI     PK_HIT
E374                    708  ; skip the scan code ...
E374  08                709                  INX
E375                    710  ; ... and the direction: next entry
E375  08                711                  INX
E376  20 F0             712                  BRA     PK_LP     ; try the next key
E378                    713  ; A = direction for the pressed key
E378  A6 01             714  PK_HIT          LDAA    1,X
E37A                    715  ; X -> your paddle record
E37A  CE 01 00          716                  LDX     #HPAD
E37D                    717  ; slide the paddle one row
E37D  BD E3 8E          718                  JSR     MOVPAD
E380  39                719  PK_END          RTS               ; return to GAME
E381                    720
E381                    721  ; scan code, direction.   $53 up   $52 down  (cursor
E381                    722  ; keys, ^K / ^J in KBDTAB)
E381                    723  ; $20 W    $31 S     $54 left $55 right (Rubout's
E381                    724  ; keys)
E381                    725  ; up, W, left keys -> $FF (up)
E381  53 FF 20 FF       726  KEYTAB          FCB     $53,$FF,$20,$FF,$54,$FF
E387                    727  ; down, S, right keys -> $01 (down)
E387  52 01 31 01       728                  FCB     $52,$01,$31,$01,$55,$01
E38D                    729  ; zero ends the table
E38D  00                730                  FCB     0
E38E                    731
E38E                    732  ; MOVPAD -- X = paddle record, A = $FF (up) or $01
E38E                    733  ; (down).  Slides the
E38E                    734  ; paddle one row: writes the new leading end, blanks
E38E                    735  ; the old trailing end.
E38E                    736  ; Top row is limited to 2..17 (court rows 2-21).
E38E                    737  MOVPAD
E38E                    738  ; remember the direction
E38E  B7 01 26          739                  STAA    MPDIR
E391                    740  ; B = paddle top row
E391  E6 00             741                  LDAB    0,X
E393                    742  ; B = proposed new top row
E393  FB 01 26          743                  ADDB    MPDIR
E396                    744  ; above court row 2?
E396  C1 02             745                  CMPB    #2
E398                    746  ; yes: refuse the move
E398  25 42             747                  BCS     MP_X
E39A                    748  ; beyond the lowest top row (17)?
E39A  C1 12             749                  CMPB    #18
E39C                    750  ; yes: refuse the move
E39C  24 3E             751                  BCC     MP_X
E39E                    752  ; store the new top row
```

```
E39E  E7 00          753                 STAB    0,X
E3A0  F7 01 1F       754                 STAB    TMPB      ; keep a copy
E3A3  A6 01          755                 LDAA    1,X       ; A = paddle column
E3A5  B7 01 1B       756                 STAA    COLARG    ; column argument
E3A8                 757  ; moving up or down?
E3A8  7D 01 26       758                 TST     MPDIR
E3AB  2B 18          759                 BMI     MP_UP     ; negative: up
E3AD                 760  ; down: draw top+4, blank top-1
E3AD  B6 01 1F       761                 LDAA    TMPB
E3B0                 762  ; bottom cell = top + 4
E3B0  8B 04          763                 ADDA    #4
E3B2  B7 01 1A       764                 STAA    ROWARG    ; row argument
E3B5  86 23          765                 LDAA    #CH_PAD   ; paddle character
E3B7                 766  ; draw the new bottom cell
E3B7  BD E4 67       767                 JSR     PUTAT
E3BA  B6 01 1F       768                 LDAA    TMPB      ; A = new top row
E3BD                 769  ; the row just above it
E3BD  4A             770                 DECA
E3BE  B7 01 1A       771                 STAA    ROWARG    ; row argument
E3C1  4F             772                 CLRA              ; blank character
E3C2                 773  ; blank the old top cell, then return
E3C2  7E E4 67       774                 JMP     PUTAT
E3C5                 775  ; up: draw top, blank top+5
E3C5  B6 01 1F       776  MP_UP           LDAA    TMPB
E3C8                 777  ; row argument = new top row
E3C8  B7 01 1A       778                 STAA    ROWARG
E3CB  86 23          779                 LDAA    #CH_PAD   ; paddle character
E3CD                 780  ; draw the new top cell
E3CD  BD E4 67       781                 JSR     PUTAT
E3D0  B6 01 1F       782                 LDAA    TMPB      ; A = new top row
E3D3                 783  ; the row just below the paddle
E3D3  8B 05          784                 ADDA    #5
E3D5  B7 01 1A       785                 STAA    ROWARG    ; row argument
E3D8  4F             786                 CLRA              ; blank character
E3D9                 787  ; blank the old bottom cell, return
E3D9  7E E4 67       788                 JMP     PUTAT
E3DC  39             789  MP_X           RTS               ; done
E3DD                 790
```

`POLLHUM` begins by testing P. If P is down it calls `PAUSE`. Then it walks `KEYTAB` exactly as in Chapter 16: pairs of (scan code, direction), terminated by zero. The table has six entries, so the keys UP, W and LEFT map to `$FF`, and DOWN, S and RIGHT map to `$01`. When a key is down, `PK_HIT` loads the direction, points X at `HPAD` and calls `MOVPAD`.

`MOVPAD` is the refined version of T13PADDLE's `MOVE`. It takes the bat's record in X and the direction in A, so the human's bat and the computer's go through the same code. It stores the direction in `MPDIR`, computes the proposed top row in B, and applies the same two compares (`CMPB #2 / BCS`, `CMPB #18 / BCC`) to keep the top between 2 and 17. If the move is accepted it stores the new top row, then redraws **only the two cells that changed**:

- moving down: draw a # at the new bottom cell (top + 4) and blank the cell above the new top (top - 1);
- moving up: draw a # at the new top and blank the cell below the new bottom (top + 5).

That is two `PUTAT` calls instead of the ten that T13PADDLE's erase-all-then-draw-all needs, and it avoids a flicker in which the whole bat vanishes for an instant. The second call in each branch is a `JMP PUTAT`, a tail call, so `PUTAT`'s `RTS` returns directly to `MOVPAD`'s caller.

CPUAI: the computer's bat

```
E3DD                 791  ; ================================================
E3DD                 792  ; CPUAI -- the computer player.  Moves every other
E3DD                 793  ; tick.  Aims its paddle
E3DD                 794  ; centre at (ball row + AIMOFF) while the ball is
E3DD                 795  ; coming at it, otherwise
E3DD                 796  ; drifts back to row 11.
E3DD                 797  ; ================================================
E3DD                 798  CPUAI
E3DD  B6 01 11       799                  LDAA    TICK      ; A = tick counter
E3E0  84 01          800                  ANDA    #1        ; keep the low bit
E3E2                 801  ; odd tick: skip (half speed)
E3E2  26 28          802                  BNE     CA_X
E3E4                 803  ; default target: row 11
E3E4  86 0B          804                  LDAA    #11
E3E6                 805  ; which way is the ball going?
E3E6  7D 01 06       806                  TST     BHDIR
E3E9  2A 06          807                  BPL     CA_TGT    ; ball going away
E3EB                 808  ; coming at us: A = ball row
E3EB  B6 01 04       809                  LDAA    BROW
E3EE  BB 01 12       810                  ADDA    AIMOFF    ; add the aim error
E3F1                 811  ; store the target row
E3F1  B7 01 1E       812  CA_TGT          STAA    TMPA
E3F4                 813  ; A = paddle top row
E3F4  B6 01 02       814                  LDAA    CPAD
E3F7                 815  ; centre row of the paddle
E3F7  8B 02          816                  ADDA    #2
E3F9                 817  ; compare paddle centre to target
E3F9  B1 01 1E       818                  CMPA    TMPA
E3FC                 819  ; already there: no move
E3FC  27 0E          820                  BEQ     CA_X
E3FE                 821  ; signed: target can be -1 near the top
E3FE  2E 04          822                  BGT     CA_UP
E400  86 01          823                  LDAA    #1        ; direction: down
E402  20 02          824                  BRA     CA_MV     ; go move
E404  86 FF          825  CA_UP           LDAA    #$FF      ; direction: up
E406                 826  ; X -> computer's paddle record
E406  CE 01 02       827  CA_MV           LDX     #CPAD
E409                 828  ; slide the paddle one row
E409  BD E3 8E       829                  JSR     MOVPAD
E40C  39             830  CA_X            RTS               ; done
E40D                 831
```

The computer moves only when the low bit of `TICK` is zero, so on every other tick. The default target row is 11. If the ball is heading toward the computer (`TST BHDIR` / `BPL CA_TGT` skips the next two instructions when bit 7 is clear, i.e. the ball is going away), the target becomes the ball's row plus `AIMOFF`. The target is parked in `TMPA`. The bat's centre, its top row plus 2, is compared to the target: equal means no move; greater (`BGT`, signed) means the bat is below the target and must go up (`$FF`); otherwise down (`$01`). Then the same `MOVPAD` moves it. The computer's bat is therefore always one row away from where it thinks it should be, moving at half your speed: a simple opponent with a human flaw built in.

PAUSE and WAITSP

```
E40D                832  ; ================================================
E40D                833  ; PAUSE / WAITSP -- keyboard waits
E40D                834  ; ================================================
E40D                835  PAUSE
E40D                836  ; wait for P to be released
E40D  BD E4 30      837                  JSR     PZREL
E410  86 17         838                  LDAA    #23       ; message row 23
E412  B7 01 1A      839                  STAA    ROWARG    ; row argument
E415  86 25         840                  LDAA    #37       ; message column 37
E417  B7 01 1B      841                  STAA    COLARG    ; column argument
E41A                842  ; X points at the PAUSED text
E41A  CE E6 3E      843                  LDX     #SPAUSE
E41D  BD E4 77      844                  JSR     PUTSTR    ; print it
E420  86 28         845  PZW             LDAA    #$28      ; scan code for P
E422                846  ; scan code out to PIA1
E422  B7 78 00      847                  STAA    PIA1
E425                848  ; read the key state back
E425  B6 78 00      849                  LDAA    PIA1
E428                850  ; wait for P to be pressed again
E428  2A F6         851                  BPL     PZW
E42A  BD E4 30      852                  JSR     PZREL     ; and released
E42D                853  ; erase the message, then return
E42D  7E E4 AA      854                  JMP     MSGCLR
E430  86 28         855  PZREL           LDAA    #$28      ; scan code for P
E432                856  ; scan code out to PIA1
E432  B7 78 00      857                  STAA    PIA1
E435                858  ; read the key state back
E435  B6 78 00      859                  LDAA    PIA1
E438                860  ; still down: keep waiting
E438  2B F6         861                  BMI     PZREL
E43A  39            862                  RTS               ; P is released
E43B                863
E43B                864  ; WAITSP -- wait for SPACE ($4B) or CLEAR ($40, the
E43B                865  ; other start key Rubout
E43B                866  ; uses; ADM31EMU.EXE also reports its space bar as
E43B                867  ; $40), spinning the random
E43B                868  ; generator while we wait, which is what seeds each
E43B                869  ; serve.  Then wait for
E43B                870  ; both to be released.
E43B                871  WAITSP
E43B                872  ; spin the generator (seed source)
E43B  BD E5 BF      873                  JSR     RAND
E43E                874  ; scan code for SPACE
E43E  86 4B         875                  LDAA    #$4B
E440                876  ; scan code out to PIA1
E440  B7 78 00      877                  STAA    PIA1
E443                878  ; read the key state back
E443  B6 78 00      879                  LDAA    PIA1
E446                880  ; SPACE is down: wait for release
E446  2B 0A         881                  BMI     WSREL
E448                882  ; scan code for CLEAR
E448  86 40         883                  LDAA    #$40
E44A                884  ; scan code out to PIA1
E44A  B7 78 00      885                  STAA    PIA1
E44D                886  ; read the key state back
```

```
E44D  B6 78 00       887                 LDAA    PIA1
E450                 888  ; neither is down: keep waiting
E450  2A E9          889                 BPL     WAITSP
E452                 890  ; release wait: scan code for SPACE
E452  86 4B          891  WSREL          LDAA    #$4B
E454                 892  ; scan code out to PIA1
E454  B7 78 00       893                 STAA    PIA1
E457                 894  ; read the key state back
E457  B6 78 00       895                 LDAA    PIA1
E45A                 896  ; still down: keep waiting
E45A  2B F6          897                 BMI     WSREL
E45C                 898  ; scan code for CLEAR
E45C  86 40          899                 LDAA    #$40
E45E                 900  ; scan code out to PIA1
E45E  B7 78 00       901                 STAA    PIA1
E461                 902  ; read the key state back
E461  B6 78 00       903                 LDAA    PIA1
E464                 904  ; still down: keep waiting
E464  2B EC          905                 BMI     WSREL
E466                 906  ; both keys are released
E466  39             907                 RTS
E467                 908
```

`PAUSE` waits for P to be **released** (`PZREL`), prints PAUSED at row 23, column 37, waits for P to be pressed again, waits for it to be released, and then erases the message with `JMP MSGCLR`. Waiting for release before looking for the next press is what keeps one keystroke from being counted twice: the program acts on *edges*, not levels, whenever one press must mean one event (see Chapter 9).

`WAITSP` waits for SPACE (`$4B`) or CLEAR (`$40`). Its first instruction is `JSR RAND`: **the random generator runs once per pass of the waiting loop**, which is the seed source of Chapter 19. After either key is seen, it waits until both are released, so a held key does not serve twice.

The screen helpers

```
E467                 909  ; ================================================
E467                 910  ; SCREEN HELPERS
E467                 911  ; ================================================
E467                 912  ; PUTAT -- write A at (ROWARG,COLARG)
E467                 913  PUTAT
E467                 914  ; save the character
E467  B7 01 1E       915                 STAA    TMPA
E46A                 916  ; X = cell address for (ROWARG,COLARG)
E46A  BD E5 65       917                 JSR     CALCADR
E46D                 918  ; restore the character
E46D  B6 01 1E       919                 LDAA    TMPA
E470                 920  ; sync to the CRT before writing
E470  B7 70 00       921                 STAA    HALT
E473                 922  ; let the strobe settle
E473  01             923                 NOP
E474                 924  ; write the character to the screen
E474  A7 00          925                 STAA    0,X
E476  39             926                 RTS             ; done
E477                 927
E477                 928  ; PUTSTR -- zero-terminated string at X, written from
E477                 929  ; (ROWARG,COLARG)
```

```
E477                 930  PUTSTR
E477                 931  ; save the string pointer
E477  FF 01 24       932                  STX     PSRC
E47A                 933  ; X = string pointer
E47A  FE 01 24       934  PS_LP           LDX     PSRC
E47D                 935  ; A = next character
E47D  A6 00          936                  LDAA    0,X
E47F                 937  ; zero ends the string
E47F  27 0C          938                  BEQ     PS_X
E481                 939  ; advance the pointer
E481  08             940                  INX
E482  FF 01 24       941                  STX     PSRC      ; save the pointer
E485                 942  ; draw the character
E485  BD E4 67       943                  JSR     PUTAT
E488  7C 01 1B       944                  INC     COLARG    ; next column
E48B  20 ED          945                  BRA     PS_LP     ; repeat
E48D  39             946  PS_X            RTS               ; done
E48E                 947
E48E                 948  ; HLINE -- A = char, B = count, X = first CRTRAM
E48E                 949  ; address
E48E                 950  HLINE
E48E                 951  ; sync to the CRT before writing
E48E  B7 70 00       952                  STAA    HALT
E491                 953  ; let the strobe settle
E491  01             954                  NOP
E492  A7 00          955                  STAA    0,X       ; write one cell
E494  08             956                  INX               ; next cell
E495  5A             957                  DECB              ; count down
E496                 958  ; more cells: repeat
E496  26 F6          959                  BNE     HLINE
E498  39             960                  RTS               ; done
E499                 961
E499                 962  CLRSCR
E499                 963  ; X = first screen cell
E499  CE 80 00       964                  LDX     #CRTRAM
E49C  4F             965                  CLRA              ; A = 0 (blank)
E49D                 966  ; sync to the CRT before writing
E49D  B7 70 00       967  CS_LP           STAA    HALT
E4A0                 968  ; let the strobe settle
E4A0  01             969                  NOP
E4A1  A7 00          970                  STAA    0,X       ; blank one cell
E4A3  08             971                  INX               ; next cell
E4A4                 972  ; past the last cell?
E4A4  8C 87 80       973                  CPX     #CRTRAM+1920
E4A7  26 F4          974                  BNE     CS_LP     ; no: repeat
E4A9  39             975                  RTS               ; done
E4AA                 976
E4AA                 977  MSGCLR
E4AA  CE 87 30       978                  LDX     #CRTRAM+1840 ; row 23
E4AD  C6 50          979                  LDAB    #80       ; 80 cells in a row
E4AF  4F             980                  CLRA              ; blank character
E4B0                 981  ; blank the row, then return
E4B0  7E E4 8E       982                  JMP     HLINE
E4B3                 983
E4B3                 984  ; SHOWSC -- redraw both score digits
E4B3                 985  SHOWSC
E4B3  7F 01 1A       986                  CLR     ROWARG    ; row 0
```

```
E4B6  86 08          987                 LDAA    #8        ; your score column
E4B8  B7 01 1B       988                 STAA    COLARG    ; column argument
E4BB  B6 01 0F       989                 LDAA    SCOREH    ; A = your score
E4BE                 990  ; convert the digit to ASCII
E4BE  8B 30          991                 ADDA    #$30
E4C0  BD E4 67       992                 JSR     PUTAT     ; draw it
E4C3                 993  ; computer's score column
E4C3  86 4A          994                 LDAA    #74
E4C5  B7 01 1B       995                 STAA    COLARG    ; column argument
E4C8                 996  ; A = computer's score
E4C8  B6 01 10       997                 LDAA    SCOREC
E4CB                 998  ; convert the digit to ASCII
E4CB  8B 30          999                 ADDA    #$30
E4CD                1000  ; draw it, then return
E4CD  7E E4 67      1001                 JMP     PUTAT
E4D0                1002
E4D0                1003  ; DRAWPAD -- X = paddle record; draw all 5 cells
E4D0                1004  DRAWPAD
E4D0  A6 01         1005                 LDAA    1,X       ; A = paddle column
E4D2  B7 01 1B      1006                 STAA    COLARG    ; column argument
E4D5                1007  ; A = paddle top row
E4D5  A6 00         1008                 LDAA    0,X
E4D7  B7 01 1A      1009                 STAA    ROWARG    ; row argument
E4DA  86 05         1010                 LDAA    #5        ; five cells tall
E4DC  B7 01 1F      1011                 STAA    TMPB      ; cell counter
E4DF  86 23         1012  DP_LP          LDAA    #CH_PAD   ; paddle character
E4E1  BD E4 67      1013                 JSR     PUTAT     ; draw one cell
E4E4  7C 01 1A      1014                 INC     ROWARG    ; next row
E4E7  7A 01 1F      1015                 DEC     TMPB      ; count down
E4EA                1016  ; more cells: repeat
E4EA  26 F3         1017                 BNE     DP_LP
E4EC  39            1018                 RTS               ; done
E4ED                1019
```

The helpers are the ones you built in Chapters 8 and 14: `PUTAT`, `PUTSTR`, `HLINE`. Three are new here:

- `CLRSCR` writes zero to all 1,920 cells, `$8000` to `$877F`, with `CPX #CRTRAM+1920` as the end test. (No other cell is touched: the CRT RAM above `$877F` is not the visible screen.)
- `MSGCLR` clears row 23 by loading X with `CRTRAM+1840` (23 x 80), B with 80 and A with zero, and **jumping into `HLINE`**: a blank line is a horizontal line of blanks.
- `SHOWSC` converts each score to an ASCII digit (`ADDA #$30`) and draws it at row 0, column 8 for you and column 74 for the computer. It ends with `JMP PUTAT`.

```
E4ED                1020  ; DRAWFLD -- walls, centre line, captions, scores,
E4ED                1021  ; paddles
E4ED                1022  DRAWFLD
E4ED  86 3D         1023                 LDAA    #CH_WALL  ; wall character
E4EF  CE 80 50      1024                 LDX     #CRTRAM+80 ; row 1: top wall
E4F2  C6 50         1025                 LDAB    #80       ; 80 cells
E4F4  BD E4 8E      1026                 JSR     HLINE     ; draw the top wall
E4F7  86 3D         1027                 LDAA    #CH_WALL  ; wall character
E4F9                1028  ; row 22: bottom wall
E4F9  CE 86 E0      1029                 LDX     #CRTRAM+1760
E4FC  C6 50         1030                 LDAB    #80       ; 80 cells
E4FE                1031  ; draw the bottom wall
E4FE  BD E4 8E      1032                 JSR     HLINE
```

```
E501  86 02         1033                  LDAA    #2        ; first court row
E503  B7 01 1A      1034                  STAA    ROWARG    ; row argument
E506  86 28         1035                  LDAA    #40       ; centre column
E508  B7 01 1B      1036                  STAA    COLARG    ; column argument
E50B  86 3A         1037  DF_NET          LDAA    #CH_NET   ; net character
E50D  BD E4 67      1038                  JSR     PUTAT     ; draw one net cell
E510  7C 01 1A      1039                  INC     ROWARG    ; next row
E513  B6 01 1A      1040                  LDAA    ROWARG    ; A = row
E516                1041  ; reached the bottom wall?
E516  81 16         1042                  CMPA    #22
E518                1043  ; no: continue the net
E518  26 F1         1044                  BNE     DF_NET
E51A  7F 01 1A      1045                  CLR     ROWARG    ; row 0
E51D  86 04         1046                  LDAA    #4        ; "YOU" column
E51F  B7 01 1B      1047                  STAA    COLARG    ; column argument
E522  CE E5 EC      1048                  LDX     #SYOU     ; X points at "YOU"
E525  BD E4 77      1049                  JSR     PUTSTR    ; draw it
E528  7F 01 1A      1050                  CLR     ROWARG    ; row 0
E52B  86 18         1051                  LDAA    #24       ; title column
E52D  B7 01 1B      1052                  STAA    COLARG    ; column argument
E530                1053  ; X points at the title text
E530  CE E5 F4      1054                  LDX     #STITLE
E533  BD E4 77      1055                  JSR     PUTSTR    ; draw it
E536  7F 01 1A      1056                  CLR     ROWARG    ; row 0
E539  86 46         1057                  LDAA    #70       ; "CPU" column
E53B  B7 01 1B      1058                  STAA    COLARG    ; column argument
E53E  CE E5 F0      1059                  LDX     #SCPU     ; X points at "CPU"
E541  BD E4 77      1060                  JSR     PUTSTR    ; draw it
E544                1061  ; draw both score digits
E544  BD E4 B3      1062                  JSR     SHOWSC
E547                1063  ; starting paddle top row
E547  86 09         1064                  LDAA    #9
E549                1065  ; your paddle top row
E549  B7 01 00      1066                  STAA    HPAD
E54C                1067  ; computer's paddle top row
E54C  B7 01 02      1068                  STAA    CPAD
E54F                1069  ; your paddle column
E54F  86 04         1070                  LDAA    #4
E551                1071  ; store it in your paddle record
E551  B7 01 01      1072                  STAA    HPAD+1
E554                1073  ; computer's paddle column
E554  86 4B         1074                  LDAA    #75
E556                1075  ; store it in the computer's record
E556  B7 01 03      1076                  STAA    CPAD+1
E559                1077  ; X -> your paddle record
E559  CE 01 00      1078                  LDX     #HPAD
E55C  BD E4 D0      1079                  JSR     DRAWPAD   ; draw it
E55F                1080  ; X -> computer's paddle record
E55F  CE 01 02      1081                  LDX     #CPAD
E562                1082  ; draw it, then return
E562  7E E4 D0      1083                  JMP     DRAWPAD
E565                1084
```

`DRAWFLD` draws the two walls with `HLINE` (X = `CRTRAM+80` for row 1 and `CRTRAM+1760` for row 22), the centre line with a loop of `PUTAT`, the three captions with `PUTSTR`, and the scores; then it sets both bats' top row to 9 and their columns to 4 and 75 and draws each with `DRAWPAD`.

DRAWPAD is now a general routine: the bat's record in X gives the column and the top row, and it draws five cells with PUTAT.

CALCADR, the row x 80 + column routine of Chapter 8, is reproduced in the listing after DRAWFLD. It is 90 bytes ($E565-$E5BE), and it is called every time anything is drawn.

BEEP and the text

```
E5DE                1168  ; BEEP -- Rubout's BEEP: pulse PIA1 CRB bit 3
E5DE                1169  BEEP
E5DE                1170  ; A = PIA1 control register B
E5DE  B6 78 03      1171                  LDAA    PIA1CRB
E5E1                1172  ; clear bit 3: bell line low
E5E1  84 F7         1173                  ANDA    #$F7
E5E3  B7 78 03      1174                  STAA    PIA1CRB   ; write it
E5E6                1175  ; set bit 3: bell line high
E5E6  8A 08         1176                  ORAA    #$08
E5E8  B7 78 03      1177                  STAA    PIA1CRB   ; write it
E5EB  39            1178                  RTS               ; done
E5EC                1179
```

BEEP reads the PIA's control register B ($7803), clears bit 3, writes it, sets bit 3 and writes it again. The line goes low and then high: one pulse, which is what sounds the bell (Chapter 10). The routine is Rubout's.

The text is a series of zero-terminated strings built with FCC: "YOU", "CPU", the title with the author's name, the prompts, and the win and lose messages. They are all plain ASCII; PONG uses no special graphics characters, so it works with the standard character generator.

The listing ends with ORG $E7FF and FCB $FF, which makes the image exactly 2,048 bytes by putting one byte at the very end of the ROM. The assembler writes everything from $E000 to $E7FF, the unused space between routines as zeros.

What to notice

Reading the whole program, a few themes recur:

- **State in variables, picture on the screen**, drawn and erased incrementally.
- **The screen is read** to learn what was hit.
- **Direction as a sign bit**, tested with TST and flipped with COM.
- **Records addressed through X**: one routine for two bats.
- **Tail calls** (JMP PUTAT, BRA DRAWPAD) to save bytes.
- **Safety valves.** TRIES guarantees the ball logic terminates.
- **Every number is a constant at the top** (WINSC, DLYSTART, the character codes) or a table.

Exercises

1. Trace by hand the ball moving left along row 12 from column 6 with BVRATE = 0, with your bat's top at row 10. Give the value of BCOL, the cell contents tested, and the branch taken for each tick until the bat is hit, and then for the three ticks after.
2. In BM_EVAL, change CMPB #CH_WALL / BNE BM_FREE to BNE BM_PAD. What happens, and why?

3. Why does `ENDGAME` save X in `KPTR` before calling `MSGCLR`?
4. How many bytes would you save by making `PADHIT` use a table of five (rate, direction) pairs? How many would you spend on the table and loop?
5. `MOVPAD` redraws two cells. Which of the two writes could be dropped if the bat were only ever drawn, never erased, and what would the screen look like after a few moves?

Chapter 22

PONG: testing it, and what is still unproven

A program is not finished when it assembles. It is finished when you know what it does, and *how you know*. This chapter lists what has been checked about PONG, by what method, and what has not been checked. It then gives a short procedure for running it on a real ADM-31, a table of likely faults and their causes, and the list of numbers you can change to tune the game.

What has been checked

Claim	How it was checked
The assembler output is the delivered ROM	`PONG.BIN` from this book's pipeline has the same MD5 as the delivered file: `04dc28290ac9edd938f9382e00991dc0`.
The listing is a real acp6800-format listing	The listing in Appendix E is the output of the author's assembler (Appendix K) run on the source, and its binary equals `PONG.BIN`.
The program starts and draws the court	The emulator's screen at the first prompt is shown in Chapter 20.
Both bats stop at their limits	The emulator holds UP (or DOWN) repeatedly: the bat sits on rows 2-6 (or 17-21).
Direction flags, tick divider, angles, bounce and scoring work	The ball position, scores and delay are sampled from RAM along a deterministic run of twelve million instructions and checked against rules below.
It runs in the author's emulator (Appendix L)	The emulator's CPU, memory and keyboard code, run with the stock firmware images, in both the `plugin` and `game` boot modes: the court and serve prompt appear, SPACE serves, the ball moves, and the bats reach rows 2 and 17.
It runs on a real ADM-31	The author put `PONG.BIN` in socket A of a real terminal and played it: it starts, draws the court, and plays. This is the strongest check in the table.

Claim	How it was checked
It starts from both firmware sets	Boot each real firmware ROM set in the emulator, send ESC ` (a grave accent) and see the title screen: done for 129867 and 129827.

The rules for the long run are simple facts the design guarantees at every instant:

- each bat's top row is between 2 and 17;
- each score is at most 7;
- the tick delay is a multiple of 200 between 3,400 and 6,400;
- the ball's row is on the court or the wall, and its column on the screen;
- over the run, the score *changes* (the program is actually playing points).

Every one of these checks has to hold at every sample, and they do. The run used a "human" who never moves the bat and presses SPACE at regular intervals: the computer wins most points, which is what an idle player deserves: at the last sample the score was 6 to 1 in the computer's favour. (The computer does miss now and then, as designed.) The test is deterministic: the emulator has no clock, so the same run gives the same numbers every time.

How slow is a tick?

Counting instructions in the emulator, one tick at the starting delay is about 12,900 instructions: 12,800 for the delay loop (6,400 passes of `DEX` and `BNE`) plus about 100 for everything else. That is the *instruction* count. In clock cycles, the data sheet gives `DEX` 4 and a taken `BNE` 4, so the delay is 51,200 cycles and the rest of the tick adds a few hundred. At the hardware's real clock the tick lasts 51,200+ divided by the clock frequency. As the clock has not been verified, the book does not claim a speed. (If it is 1 MHz, as is common for a 6800, a tick is a little over 51 ms.)

What has *not* been checked

Being honest about what is unproven is part of the job. PONG itself has run on a real ADM-31 and works; what follows is what the book has not *measured*, even though the game runs.

1. **Speed.** Every timing in the game is a count of loop passes. The book did not time the game on the machine, so it states no speed in seconds.
2. **The bell.** The routine pulses the bell bit exactly as Rubout's does. How long the bell sounds is a property of the terminal, and the book did not measure it.
3. **Key behaviour.** The emulator presses a key for a fixed number of instructions and then releases it. A real key bounces, and a person holds it for tens of milliseconds. The game is tolerant (a bat move is one row per tick, whatever the key does).
4. **The cursor.** The cursor-hiding pair is copied from Rubout. The emulator does not show a cursor, so it can say nothing about it.
5. **The comment on the delay.** The "about 0.5 s" in the source depends on the clock, which the book did not verify.
6. **Which firmware sets.** The book's emulator launches PONG from both firmware sets, running the real firmware code. The real-terminal run was done on the author's own terminal; the book does not record a run on each of the two sets. The emulator's model of the hardware is the author's reconstruction; it is not the machine.

Running it on the real machine

1. Put `PONG.BIN` (2,048 bytes) into an EPROM, or any equivalent 2 KB device that fits your plug-in socket, and fit it in socket A (the book's convention; the sockets' designators are discussed in Appendix B, because two sources label them differently).
2. Power on. The terminal should come up as usual: the plug-in's hook sends the firmware down its normal boot path, so with the switches in their usual positions nothing should look different.
3. Press **ESC** and then **` (the grave accent)**. On this keyboard the grave accent is the SHIFTed `@` key (scan code `$3B`); the apostrophe, which is a different command, is SHIFT-7 (scan code `$14`). Chapter 5 explains. The title line should appear:

   ```
   YOU 0                ADM-31 PONG  By: Mickey Lawless                CPU 0
   ```

   with the court below and `SPACE=SERVE ...` on the bottom line.
4. Press SPACE. A ball (`O`) should appear on the centre line and, after a short pause, start to move.
5. Press UP or W; your bat (the `#` column at the left) should move one row per tick.
6. Let a point be scored. You should hear the bell and see the score change.
7. Press P to pause; P again to continue.

If anything differs, the table below is the place to start.

Troubleshooting

Symptom	Likely cause	What to look at
Nothing happens after ESC `	Wrong key (the apostrophe, not the grave accent) or the ROM is not seen	Chapter 5: `ESC '` (code `$27`) is a different command. (The `$7D00` bit-7 switch is looked at only by the boot probe; the ESC ` handler does not test it.) Check the ROM's signature, the bytes `54 4A` at `$E000`.
Terminal does not boot at all with the ROM fitted	The boot hook	The bytes at `$E07D` (129867) or `$E087` (129827), and the jump to the firmware's no-plug-in boot: `$E8D0` or `$E9D0`.
Court appears, ball never moves	Waiting in `WAITSP`	The scan code of SPACE (`$4B`) and CLEAR (`$40`); press either.
Garbage on the screen	The CRT RAM is accessed without the HALT strobe, or the character generator differs	`PUTAT` and `HLINE` (the `STAA HALT` / `NOP` before each access).
Bat sticks or moves two rows	Keyboard bounce or too short a tick	Increase `DLYSTART`, or add a release wait.
Game too fast / too slow	The clock is not what was assumed	`DLYSTART` (6,400) and the `SUBA #200` in `PADHIT`.

Symptom	Likely cause	What to look at
Ball goes through a bat	The ball is at the bat's column when the bat is not there	Intended: a miss. If the bat is clearly in the way, the bat's column constants (4 and 75).
Score never changes	`POINT` is not set	`BM_HMISS` and `BM_CMISS` and the column limits 2 and 78.
No sound	The bell bit	`BEEP`; the PIA's control register B at `$7803`. Chapter 10 tests the same routine in isolation (T07BELL).
Cursor visible	The cursor-hiding pair	The two stores at `RESET` to `$7F06` and `$7F0C`.

Knobs

All of these are single numbers, easy to find and change, and all of them are matters of taste rather than correctness:

What	Where	Now
Starting tick delay	`DLYSTART (EQU 6400)`	6,400 passes
Speed-up per bat hit	`SUBA #200` in `PADHIT`	200 passes
When speed-up stops	`CMPA #$0E` in `PADHIT`	high byte `$0E`
Points to win	`WINSC`	7
Computer's speed	`ANDA #1` in `CPUAI`	every second tick
Computer's mistakes	`ANDA #3` and `SUBA #3` in `PADHIT`	error -3 to +3
Bat size	`5` in `DRAWPAD` and the limits 18 and +4/+5 in `MOVPAD`	5 cells
Serve pause	`LDX #0` in `SERVE`	65,536 passes

Change one at a time, rebuild, and run the emulator test again. A test that passes before the change and after it tells you the change was safe; a test that fails tells you which rule you broke.

What the tests would not catch

Be aware of what a clean test run does *not* say. It says nothing about whether the game is fun. It cannot detect a flaw in the model of the hardware shared by the emulator and the program (if both are wrong in the same way, they agree). And it cannot tell you whether the real keyboard sends what the emulator thinks it sends. The tests prove that the program does what its source says it does. That the source is right for the terminal on your desk is proved only by running it there. Chapter 1 set out the three levels of evidence used in this book (seen in the firmware, used by a working program, emulator only); PONG's hardware accesses are of the second kind, and its behaviour is of the third.

Exercises

1. Add a test, in the style of `tests.py`, that holds DOWN and checks `HPAD` is 17 and the screen has `#` in rows 17 to 21 and nowhere else in column 4. (Everything you need is in `pongtest.py`.)
2. Suppose the real clock is half the speed you assumed. What happens to the game, and which one number would you change to compensate?
3. List three observations on the real machine that would show the HALT strobe is being misused, and for each say which routine you would inspect first.
4. Why does the test for the tick delay check “a multiple of 200”, and not just “between 3,400 and 6,400”?

Chapter 23

Borrowing the firmware's routines

Your program owns the machine, but the firmware is still sitting in ROM at `$E800-$FFFF` with a toolbox of routines in it. Some of them are tiny and general: copy a block with the screen strobe built in, add a register to the index register, ring the bell. A program may call them. This chapter shows how, what it costs, and the one serious trap: **the routines are at different addresses in the two firmware sets**.

Why borrow, and why not

Reasons to borrow:

- **Size.** A 2 KB ROM fills quickly.
- **Proven code.** The firmware ran on thousands of terminals. Its `BLITCPY` has been copying bytes to screens since before you were born.
- **Learning.** Reading a routine and then calling it is the quickest way to understand the machine.

Reasons not to:

- **Coupling.** If you call `$F6BB`, your program works only on a firmware that has `BLITCPY` at `$F6BB`. A different firmware revision moves it, or removes it. Your own routine has no such dependence.
- **Hidden state.** The firmware keeps its variables in the same RAM as yours. A routine that uses `$B1-$B4` will overwrite whatever you put there.
- **Conventions.** Firmware routines take their arguments in whatever form the firmware's author found convenient, often in zero-page variables.

PONG borrows nothing. The tutorial programs of this book keep their own copies of `XPLUSA` and `PUTAT` and so on. This chapter's programs are the exception, to show the technique.

The book's rule for facts applies: addresses below are marked [FW], seen in the firmware ROM images, and the programs were run in the emulator on the real ROM images of both firmware sets.

Three routines

XPLUSA: X = X + A

```
EFAD                836  ; XPLUSA: X = X + A.
```

```
EFAD  DF 9F          837  XPLUSA          STX     $9F
EFAF  9B A0          838                  ADDA    $A0
EFB1  97 A0          839                  STAA    $A0
EFB3  96 9F          840                  LDAA    $9F
EFB5  89 00          841                  ADCA    #$00
EFB7  97 9F          842                  STAA    $9F
EFB9  DE 9F          843                  LDX     $9F
EFBB  39             844                  RTS
```

The 6800 cannot add a register to X. This routine stores X into two scratch bytes at `$9F-$A0`, adds A to the low byte, adds the carry to the high byte (the same `ADDA/ADCA` pair as Chapter 15), and reloads X. The firmware uses it to index into tables: the keyboard scanner and the escape-sequence dispatcher both call it (Chapter 28). The book's `lib/xplusa.inc` is the same routine with the book's own scratch bytes.

BELL: pulse the bell line

```
F0DA                 979  ; BELL: pulse bit 3 of PIA1CRB.
F0DA  B6 78 03       980  BELL            LDAA    PIA1CRB
F0DD  84 F7          981                  ANDA    #$F7
F0DF  B7 78 03       982                  STAA    PIA1CRB
F0E2  8A 08          983                  ORAA    #$08
F0E4  B7 78 03       984                  STAA    PIA1CRB
F0E7  39             985                  RTS
```

Exactly the five lines of Chapter 10: read the PIA's control register B, clear bit 3, write it, set bit 3, write it. No arguments, no results except A and the flags.

BLITCPY: copy B bytes

```
F6BB                1731  ; BLITCPY: copy B bytes from ($B3) to ($B1) with
F6BB                1732  ; HALT strobe.
F6BB  DE B3         1733  BLITCPY         LDX     $B3
F6BD  B7 70 00      1734                  STAA    HALT
F6C0  01            1735                  NOP
F6C1  A6 00         1736                  LDAA    $00,X
F6C3  08            1737                  INX
F6C4  DF B3         1738                  STX     $B3
F6C6  DE B1         1739                  LDX     $B1
F6C8  B7 70 00      1740                  STAA    HALT
F6CB  01            1741                  NOP
F6CC  A7 00         1742                  STAA    $00,X
F6CE  08            1743                  INX
F6CF  DF B1         1744                  STX     $B1
F6D1  5A            1745                  DECB
F6D2  26 E7         1746                  BNE     BLITCPY
F6D4  39            1747                  RTS
```

`BLITCPY` copies B bytes from the address in `$B3-$B4` to the address in `$B1-$B2`, and in each trip round the loop it does the screen strobe (`STAA HALT`, `NOP`) before both the read and the write. That makes it safe to copy to *or from* the screen. Both pointers are left advanced past the last byte copied, so a second call continues where the first stopped.

Three things to know before using it:

- **B is the count, and B = 0 means 256.** The loop decrements B and then tests it, so a count of zero decrements to $FF and copies 256 bytes. Never call it with B = 0 unless you mean 256.
- **A and X are destroyed**, and B ends at 0.
- The pointers are *direct page* variables at fixed addresses, $B1-$B4. They are shared with the firmware's own use (Chapter 29 shows the firmware using BLITCPY for its page switch and its line insert, and the firmware's message routine at $E961 uses it too). Because your program never returns to the firmware after starting, nobody else will be using them at the same time, but you must set both pointers every time you call it.

T14FWCALL

The tutorial program copies a 26-byte message to row 10 with BLITCPY, picks the fourth letter of a table with XPLUSA, and rings the bell:

```
E100  0F             73  START           SEI
E101  8E 03 FF       74                  LDS     #$03FF
E104  BD E1 50       75                  JSR     INITCRT
E107                 76  ; copy the 26-byte message to row 10, column 20
E107  CE E1 2E       77                  LDX     #MSG
E10A  DF B3          78                  STX     BLITSRC
E10C  CE 83 34       79                  LDX     #$8334    ; 8000 + 10*80 + 20
E10F  DF B1          80                  STX     BLITDST
E111  C6 1A          81                  LDAB    #26
E113  BD F6 BB       82                  JSR     BLITCPY
E116                 83  ; pick letter number 3 from a table with XPLUSA
E116  CE E1 48       84                  LDX     #LETTERS
E119  86 03          85                  LDAA    #3
E11B  BD EF AD       86                  JSR     XPLUSA    ; X = LETTERS + 3
E11E  A6 00          87                  LDAA    0,X       ; 'D'
E120  CE 83 CA       88                  LDX     #$83CA    ; row 12, column 10
E123  B7 70 00       89                  STAA    HALT
E126  01             90                  NOP
E127  A7 00          91                  STAA    0,X
E129  BD F0 DA       92                  JSR     BELL      ; ring the bell
E12C  20 FE          93  FOREVER         BRA     FOREVER
```

The destination, $8334, is $8000 + 10 x 80 + 20, row 10 column 20, calculated by hand, as in Chapter 18. Call it as JSR BLITCPY after loading the two pointers with LDX #... and STX. Then XPLUSA with X pointing at the table and A = 3 gives X = LETTERS + 3, which holds D. The test starts the real firmware, sends ESC `, and checks the screen:

```
Test Complete




                COPIED BY FIRMWARE BLITCPY

        D
```

(A row 10 text, and a `D` at row 12, column 10. The screen shows the 129867 run.) Notice that this program is written with the **addresses of the 129867 firmware**. On the other firmware it would call whatever happens to be at `$F6BB`.

The two sets are not the same

The 129827 firmware contains the same three routines, but not at the same places and not with the same scratch variables. Searching the 129827 ROM images for the instruction sequences of the 129867 routines finds them:

Routine	129867	129827
`XPLUSA`	`$EFAD`, scratch `$9F-$A0`	`$EE9C`, scratch `$CD-$CE`
`BELL`	`$F0DA`	`$FB8B`
`BLITCPY`	`$F6BB`, source `$B3`, destination `$B1`	`$F412`, source `$7F`, destination `$7D`

(Each routine is the same code, with only the addresses and the zero-page bytes changed. In the 129827 `BLITCPY` begins at `$F412`; the `DE 7F` that starts it is the `LDX $7F` of its source pointer.) T14FW827 is T14FWCALL with the six addresses changed:

```
0000                    27  ; firmware 129827: BLITCPY destination
0000  =007D             28  BLITDST         EQU     $7D
0000                    29  ; firmware 129827: BLITCPY source
0000  =007F             30  BLITSRC         EQU     $7F
0000  =F412             31  BLITCPY         EQU     $F412
0000  =EE9C             32  XPLUSA          EQU     $EE9C
```

and runs on the 129827 set. Both programs print the same screen.

Notice that **the zero-page variable moved too**. Zero page is the firmware's private notebook; its layout changes from one firmware to the next as freely as the code does. This is the strongest reason not to build on firmware internals unless you have to.

One ROM, both sets: a thunk

If you want one ROM that works on either, the program must find out which firmware it is running on and use the right addresses. T14AUTO does this. It tells the sets apart by the first byte of the code at `$F0DA`: in the 129867 it is `$B6` (the first byte of `BELL`, `LDAA PIA1CRB`), in the 129827 it is not. Then it must *call* an address chosen at run time. The 6800 has `JSR` with an indexed operand, but X is needed for `XPLUSA`'s own argument, so that is no use here. The solution is a **thunk**: three bytes of RAM holding a `JMP` instruction, built at run time.

```
E100  0F             73  START           SEI
E101  8E 03 FF       74                  LDS     #$03FF
E104  BD E2 25       75                  JSR     INITCRT
E107  BD E1 93       76                  JSR     CLRSCR
E10A  CE E1 7D       77                  LDX     #T867     ; assume 129867 ...
E10D  B6 F0 DA       78                  LDAA    $F0DA
E110                 79  ; BELL starts with LDAA PIA1CRB
E110  81 B6          80                  CMPA    #$B6
E112  27 03          81                  BEQ     KNOWN
E114                 82  ; ... otherwise 129827
E114  CE E1 88       83                  LDX     #T827
E117  FF 01 46       84  KNOWN           STX     FWPTR
```

```
E11A                    85  ; build the thunks: opcode $7E (JMP extended) +
E11A                    86  ; address
E11A  86 7E             87                  LDAA    #$7E
E11C  B7 01 40          88                  STAA    THUNKX
E11F  B7 01 43          89                  STAA    THUNKB
E122  FE 01 46          90                  LDX     FWPTR
E125  EE 00             91                  LDX     0,X       ; address of XPLUSA
E127  FF 01 41          92                  STX     THUNKX+1
E12A  FE 01 46          93                  LDX     FWPTR
E12D  EE 02             94                  LDX     2,X       ; address of BELL
E12F  FF 01 44          95                  STX     THUNKB+1
E132                    96  ; print the name of the set
E132  86 04             97                  LDAA    #4
E134  B7 01 00          98                  STAA    ROWARG
E137  86 0A             99                  LDAA    #10
E139  B7 01 01         100                  STAA    COLARG
E13C  CE E1 6B         101                  LDX     #LABEL
E13F  BD E2 0E         102                  JSR     PUTSTR
E142  FE 01 46         103                  LDX     FWPTR
E145                   104  ; the name is 4 bytes in
E145  86 04            105                  LDAA    #4
E147                   106  ; X = FWPTR + 4 (firmware XPLUSA)
E147  BD 01 40         107                  JSR     THUNKX
E14A  BD E2 0E         108                  JSR     PUTSTR
E14D                   109  ; use the firmware's XPLUSA: letter number 5
E14D  CE E1 75         110                  LDX     #LETTERS
E150  86 05            111                  LDAA    #5
E152  BD 01 40         112                  JSR     THUNKX    ; X = LETTERS + 5
E155  A6 00            113                  LDAA    0,X
E157  36               114                  PSHA
E158  86 06            115                  LDAA    #6
E15A  B7 01 00         116                  STAA    ROWARG
E15D  86 0A            117                  LDAA    #10
E15F  B7 01 01         118                  STAA    COLARG
E162  32               119                  PULA
E163  BD E1 FE         120                  JSR     PUTAT
E166  BD 01 43         121                  JSR     THUNKB    ; ring the bell
```

`$7E` is the opcode of `JMP` (extended). The program stores `$7E` at `THUNKX`, loads X from the table entry that holds the address of the right `XPLUSA`, and stores it at `THUNKX+1`: the three bytes `7E hh ll` are now a `JMP` to the firmware routine. `JSR THUNKX` then calls it; the routine's `RTS` returns to the instruction after the `JSR`, because the thunk does not touch the stack. RAM is executable on this machine, which is what makes the trick possible. The firmware itself uses a related idea: its task table at `$65`-`$6F` is a table of addresses in RAM (Chapter 27).

The output:

```
FIRMWARE 129867

F
```

(The screen above is the run from the 129867 set; the 129827 run prints `129827`.) The test runs the same ROM on both sets and checks the name, and that `XPLUSA` really added: the letter at row 6 is `F`, letter number 5 of `ABCDEFGH`.

The cost of portability is a few dozen bytes and the risk that a third firmware revision, unknown to the table, is mistaken for one of the two. A robust program would also compare more than one byte, and would refuse to run if neither set matched.

Rules for borrowing

1. Say in a comment which firmware the address is for, and where you found it.
2. Prefer routines with few arguments and no zero-page variables.
3. If you need a zero-page variable, set it every time.
4. If your program might run on either set, test which one and look the addresses up in a table.
5. Never *depend* on a firmware routine's side effects beyond what you have read in its code.

Exercises

1. Write a program that uses `BLITCPY` to copy row 5 of the screen to row 15. What values go in `$B3` and `$B1`, and what is B? (A row is 80 bytes; B can hold 80.)
2. Change T14AUTO to detect the set by comparing the *two* bytes at `$F0DA` and `$FB8B`, and to print `UNKNOWN` if neither pair matches. Test it by running it from the plug-in-only boot (where there is no firmware).
3. What does T14FWCALL do on the 129827 set? (In the emulator it does not print its message: it jumps into the middle of unrelated firmware code, wanders into the zero page and stops at an illegal opcode. The bytes at `$F6BB` in that ROM begin `F7 B2 86 01`, which is not `BLITCPY`.) What does this tell you about running a program on firmware you have not checked?
4. The firmware's `BELL` leaves the bell bit set. What does that mean for the next call?

Chapter 24

Reading other people's ROMs

Everything in this book that says "[PGM]" comes from reading a program that works on a real terminal. This chapter shows how that reading is done, using the five working plug-in ROMs this book draws on: **Rubout**, **Shuttle Attack**, **Space Invaders**, **Cosmic Intruders** and **Outbreak**. You will see what the programs have in common, where they differ, and what a programmer's habits look like in code. All five are verified to run on a real ADM-31. Four of them were written by others; the fifth, Outbreak (a variant of Rubout), is the author's own. He disassembled and studied them. The skill is the most useful one in the book: the manuals are gone, but the ROMs are not.

The first two have been disassembled and reassembled byte for byte with the same assembler this book uses (the disassembly listings made for this book, `RUBOUT.LST` and `SHUTTLEATTACK.LST`; Appendix G prints them). The other three are known here from their 2,048-byte images only, and the facts below about them are limited to what a scan of the bytes shows.

A method

Given a ROM image and nothing else, in this order:

1. **Check the signature.** Bytes 0 and 1 are `54 4A`, "TJ"? If not, the firmware will never run it.
2. **Look at the strings.** Printable runs of text tell you what the program is and often how it is used ("cursor left & right keys control the paddle").
3. **Look at the entry points.** Whatever is at `$E002`, at `$E03C`-`$E03F`, at the two hook addresses `$E07D` and `$E087`.
4. **Trace the code.** Start at the entry points and follow every `JMP`, `JSR` and branch. What you reach is code; what you do not reach is data, or code you have not yet found. (The disassembly tool used for this book is a few dozen lines; it marks bytes it reached as code and everything else as data.)
5. **List the hardware addresses.** Scan the traced code for operands at `$7000` and above, and for direct-page addresses. The list is the program's whole relationship with the machine.
6. **Name things.** Give each routine a descriptive name and each variable a purpose, one at a time, writing down the evidence.
7. **Rebuild it.** If the reassembled listing is byte-identical to the original, every instruction you named is certain to be what the ROM contains. If it is not, you have misread something.

Step 7 is what makes the listings of this book trustworthy: the bytes are the evidence, and the assembler is the judge.

The frames compared

Reading the bytes at the frame addresses of the five images:

	Rubout	Shuttle Attack	Space Invaders	Cosmic Intruders	Outbreak
`$E000`	`54 4A`	`54 4A`	`54 4A`	`54 4A`	`54 4A`
`$E002`	`7D 00 F4`: code	`FF`: unused	`36 BD EC C1`: `PSHA JSR $ECC1`	`B7 70 00 01`: `STAA $7000 NOP`	as Rubout
`$E03C`	`0F CE 8F FF`: `SEI, LDX #$8FFF`	`01 01 01 7E`: three `NOP`s and a `JMP`	`7E E0 88`: `JMP $E088`	`7E E0 88`: `JMP $E088`	as Rubout
`$E07D`	`20 B7`: `BRA` (lands on code that clears `$0000` and returns)	`7F 00 00 39`: `CLR $0000, RTS`	`B6 7D 00 7F 00 00 32 32 7E E8 D0`	`7F DF FF 7F 00 00 32 32 7E ..`	as Rubout
`$E087`	`96 FE 4D 2A`: game code, not a hook	`20 F4`: `BRA` back	`D0`: operand of the `JMP`	`D0 ..`	as Rubout

The details differ, and each difference is a lesson.

- **Rubout and Outbreak** are the oldest in style: they use the `$E002` entry as an ordinary subroutine of the game (the first routine of the ball logic starts there), and they begin their reset code *at* `$E03C` with `SEI`. That is fine for the 129867 firmware, which jumps to `$E03C` on ESC `` ` ``. It would not be fine for the 129827 firmware, which jumps to `$E03E`: the byte there is the middle of the next instruction, the operand of the `LDX`. PONG's three-`NOP` sled exists to avoid this. (The two images differ in 95 of their 2,048 bytes; the strings, including "Rubout Version 1.0" and the credit "Software created 1981 by David Menard", are the same in both.)
- **Shuttle Attack** uses the sled itself (`NOP NOP NOP JMP RESET`) and leaves `$E002` as padding. Its hook at `$E07D` is a single `CLR $0000` and `RTS`; the 129827 hook at `$E087` is a `BRA` that jumps back to that one.
- **Space Invaders and Cosmic Intruders** keep the full "decline the boot" hook: they clear the flag, discard the return address (`PULA` twice) and jump to the firmware's no-plug-in boot at `$E8D0`. PONG's boot hooks copy that. The two games also use `$E002` as a **callback** (the firmware's keyboard handler can call it); PONG does not need it, because it reads the keys itself.

None of this was designed together. Each author found what worked and shipped it. When you write a plug-in that must run from either firmware (as PONG must), you adopt the *union* of the conventions: the "TJ" signature, the sled, a hook for each firmware, and stubs where the firmware might call.

Rubout, closely

Rubout is a game of the Breakout family: a paddle of - characters moves along the bottom, nine balls, bricks worth 7, 5 and 3 points drawn as =, % and #. The 6800 code is 2,048 bytes with no wasted room. The real listing shows its programmer's habits.

Start-up

```
E03C                    98  RESET
E03C  0F                99          SEI
E03D  CE 8F FF         100          LDX     #$8FFF
E040                   101  RESET_004
E040  6F 00            102          CLR     $00,X
E042  09               103          DEX
E043  8C 7F FF         104          CPX     #$7FFF
E046  26 F8            105          BNE     RESET_004
E048  8D 03            106          BSR     RESET_011
E04A  7E E7 F2         107          JMP     BALLCOL_03C
E04D                   108  RESET_011
E04D  86 17            109          LDAA    #$17
E04F  B7 7F 06         110          STAA    $7F06
E052  43               111          COMA
E053  B7 7F 0C         112          STAA    $7F0C
E056  B6 78 02         113          LDAA    $7802
E059  84 FB            114          ANDA    #$FB
E05B  B7 78 02         115          STAA    $7802
E05E  CE 00 00         116          LDX     #$0000
E061  86 80            117          LDAA    #$80
E063  A7 11            118          STAA    $11,X
E065  A7 F4            119          STAA    $F4,X
E067  A7 F5            120          STAA    $F5,X
E069  86 09            121          LDAA    #$09
E06B  A7 FE            122          STAA    $FE,X
E06D  6F 17            123          CLR     $17,X
E06F  6F FD            124          CLR     $FD,X
E071  86 08            125          LDAA    #$08
E073  A7 16            126          STAA    $16,X
E075  CE 86 E0         127          LDX     #$86E0
E078  DF F6            128          STX     $F6
E07A  CE 82 80         129          LDX     #$8280
E07D  20 B7            130          BRA     DELAY2_008
E07F  39               131          RTS
E080                   132  RESET_044
E080  86 08            133          LDAA    #$08
E082  97 16            134          STAA    $16
E084  7A 00 FE         135          FCB     $7A,$00,$FE
E087  96 FE            136          LDAA    $FE
E089  4D               137          TSTA
E08A  2A 03            138          BPL     RESET_053
E08C  7E E2 6F         139          JMP     BALLCHK_069
E08F                   140  RESET_053
E08F  8A 30            141          ORAA    #$30
E091  B7 87 7A         142          STAA    $877A
E094                   143  RESET_058
E094  86 4B            144          LDAA    #$4B
E096  B7 78 00         145          STAA    $7800
```

```
E099  B6 78 00       146          LDAA    $7800
E09C  2A 05          147          BPL     RESET_067
E09E  8D 0D          148          BSR     RESET_071
E0A0  8D 0B          149          BSR     RESET_071
E0A2  39             150          RTS
E0A3                 151  RESET_067
E0A3  8D 08          152          BSR     RESET_071
E0A5  8D 11          153          BSR     RESET_07C
E0A7  8D 04          154          BSR     RESET_071
E0A9  8D 3F          155          BSR     RESET_0AE
E0AB  20 E7          156          BRA     RESET_058
E0AD                 157  RESET_071
E0AD  DF FA          158          STX     $FA
E0AF  CE 13 00       159          LDX     #$1300
E0B2                 160  RESET_076
E0B2  09             161          DEX
E0B3  26 FD          162          BNE     RESET_076
E0B5  DE FA          163          LDX     $FA
E0B7  39             164          RTS
E0B8                 165  RESET_07C
E0B8  86 03          166          LDAA    #$03
E0BA  97 F8          167          STAA    $F8
E0BC                 168  RESET_080
E0BC  86 55          169          LDAA    #$55
E0BE  B7 78 00       170          STAA    $7800
E0C1  B6 78 00       171          LDAA    $7800
E0C4  2A 1E          172          BPL     RESET_0A8
E0C6  DE F6          173          LDX     $F6
E0C8  8C 87 2B       174          CPX     #$872B
E0CB  27 17          175          BEQ     RESET_0A8
E0CD  BD E0 25       176          JSR     DELAY1
E0D0  B7 70 00       177          STAA    $7000
E0D3  01             178          NOP
E0D4  6F 00          179          CLR     $00,X
E0D6  08             180          INX
E0D7  C6 2D          181          LDAB    #$2D
E0D9  BD E0 25       182          JSR     DELAY1
E0DC  B7 70 00       183          STAA    $7000
E0DF  01             184          NOP
E0E0  E7 04          185          STAB    $04,X
E0E2  DF F6          186          STX     $F6
E0E4                 187  RESET_0A8
E0E4  7A 00 F8       188          FCB     $7A,$00,$F8
E0E7  26 D3          189          BNE     RESET_080
E0E9  39             190          RTS
E0EA                 191  RESET_0AE
E0EA  86 03          192          LDAA    #$03
E0EC  97 F8          193          STAA    $F8
E0EE                 194  RESET_0B2
E0EE  86 54          195          LDAA    #$54
E0F0  B7 78 00       196          STAA    $7800
E0F3  B6 78 00       197          LDAA    $7800
E0F6  2A 1E          198          BPL     RESET_0DA
E0F8  DE F6          199          LDX     $F6
E0FA  8C 86 E0       200          CPX     #$86E0
E0FD  27 17          201          BEQ     RESET_0DA
E0FF  BD E0 25       202          JSR     DELAY1
```

```
E102  B7 70 00       203          STAA    $7000
E105  01             204          NOP
E106  6F 04          205          CLR     $04,X
E108  09             206          DEX
E109  C6 2D          207          LDAB    #$2D
E10B  BD E0 25       208          JSR     DELAY1
E10E  B7 70 00       209          STAA    $7000
E111  01             210          NOP
E112  E7 00          211          STAB    $00,X
E114  DF F6          212          STX     $F6
E116                 213  RESET_0DA
E116  7A 00 F8       214          FCB     $7A,$00,$F8
E119  26 D3          215          BNE     RESET_0B2
E11B  39             216          RTS
```

- `SEI`; then `LDX #$8FFF` and a loop that clears every byte from `$8FFF` down to `$8000` (`CLR 0,X`, `DEX`, `CPX #$7FFF`). It clears **all 4 KB** of CRT RAM, not just the 1,920 visible bytes. A program that only clears the screen leaves whatever was in the rest.
- `RESET_011` is the hardware set-up: `$17` to `$7F06`, `COMA`, the complement `$E8` to `$7F0C` (the cursor-hiding pair that PONG copies), and clearing bit 2 of `$7802`. It also sets variables in the direct page (`$11`, `$F4`, `$F5` to `$80`, the lives counter `$FE` to 9, `$16` to 8).
- At the very end of the code, at `$E7F2`, `LDAA #3` followed by `STAA $7A00` and `STAA $7C00`: the value 3 written to the control register of each serial port is the 6850's *master reset* [FW]. Rubout does that once at the start of the game; PONG does not touch the ACIAs.

Keys and the paddle

The key-scanning routines (`RESET_058` and following) write a scan code to `$7800` and read it back, exactly as in Chapter 9:

- `$4B` (SPACE) pitches a ball;
- `$54` and `$55` (the cursor LEFT and RIGHT keys) move the paddle one cell, left and right.

The paddle is a row of four `-` characters (`$2D`; the start-up code writes `2D 2D 2D 2D` at `$86E0`, row 22) that slides along the bottom of the screen. Each move draws one new end and blanks the other, the same "write one cell, blank one cell" idea as PONG's `MOVPAD` (Chapter 21), and each cell access is preceded by a delay loop and then the `$7000` strobe, a pattern we return to.

The ball

Rubout's ball logic is the model for PONG's. Here is its core:

```
E194                 278  BALLMOV
E194  BD E7 B6       279          JSR     BALLCOL
E197                 280  BALLMOV_003
E197  BD E0 02       281          JSR     BALLROW
E19A  BD E4 20       282          JSR     CALCADR
E19D  DF F0          283          STX     $F0
E19F  B7 70 00       284          STAA    $7000
E1A2  01             285          NOP
E1A3  E6 00          286          LDAB    $00,X
E1A5  5D             287          TSTB
E1A6  27 2C          288          BEQ     BALLMOV_040
E1A8  C1 2D          289          CMPB    #$2D
E1AA  26 25          290          BNE     BALLMOV_03D
E1AC  73 00 F4 7D    291          FCB     $73,$00,$F4,$7D,$00,$F5
```

```
E1B2  2B 10          292          BMI     BALLMOV_030
E1B4  BD E0 2E       293          JSR     DELAY2
E1B7  B7 70 00       294          STAA    $7000
E1BA  01             295          NOP
E1BB  6D 02          296          TST     $02,X
E1BD  27 03          297          BEQ     BALLMOV_02E
E1BF                 298  BALLMOV_02B
E1BF  73 00 F5       299          FCB     $73,$00,$F5
E1C2                 300  BALLMOV_02E
E1C2  20 D3          301          BRA     BALLMOV_003
E1C4                 302  BALLMOV_030
E1C4  BD E0 2E       303          JSR     DELAY2
E1C7  B7 70 00       304          STAA    $7000
E1CA  01             305          NOP
E1CB  6D 02          306          TST     $02,X
E1CD  26 F0          307          BNE     BALLMOV_02B
E1CF  20 C6          308          BRA     BALLMOV_003
E1D1                 309  BALLMOV_03D
E1D1  BD E2 06       310          JSR     BALLCHK
E1D4                 311  BALLMOV_040
E1D4  86 6F          312          LDAA    #$6F
E1D6  BD E0 2E       313          JSR     DELAY2
E1D9  B7 70 00       314          STAA    $7000
E1DC  01             315          NOP
E1DD  A7 00          316          STAA    $00,X
E1DF  DE F2          317          LDX     $F2
E1E1  86 6F          318          LDAA    #$6F
E1E3  BD E0 2E       319          JSR     DELAY2
E1E6  B7 70 00       320          STAA    $7000
E1E9  01             321          NOP
E1EA  A1 00          322          CMPA    $00,X
E1EC  26 09          323          BNE     BALLMOV_063
E1EE  BD E0 2E       324          JSR     DELAY2
E1F1  B7 70 00       325          STAA    $7000
E1F4  01             326          NOP
E1F5  6F 00          327          CLR     $00,X
E1F7                 328  BALLMOV_063
E1F7  BD E0 AD       329          JSR     RESET_071
E1FA  BD E0 B8       330          JSR     RESET_07C
E1FD  BD E0 AD       331          JSR     RESET_071
E200  BD E0 EA       332          JSR     RESET_0AE
E203  7E E3 26       333          JMP     BALLCHK_120
```

Compare it with Chapter 21:

- `BALLCOL` and `BALLROW` advance the ball's column and row (`$17` and `$16`; `$F5` and `$F4` hold the directions, `$F0`/`$F2` the current and previous screen addresses).
- `CALCADR` converts them to an address; the screen is **read** (`STAA $7000`, `NOP`, `LDAB 0,X`).
- The character in B is tested: `TSTB / BEQ`: blank; `CMPB #$2D`: a paddle `-`; anything else: a brick (`BALLCHK`). This is exactly the structure of `BM_EVAL`.
- A ball that hits a brick falls into `BALLCHK`, which adds to the score (the code adds to digits stored on the screen at `$8734` and below, as decimal arithmetic on ASCII characters with a carry of ten), beeps, counts the bricks down in `$FF`, and rebuilds the wall (`INITWAL`) when the count reaches ten.

Why is Rubout's version more complex than PONG's? Because it does more: bricks, scores, balls left, new walls. The *core idea*, **read the screen to see what you hit**, is the same, and PONG's author took it from here (the PONG source says so in its comments).

Two oddities worth knowing

The scoreboard lives on the screen. Rubout keeps its score as ASCII digits *in screen memory* at `$8730`-`$8734`, and adds to them directly with `ABA`-style arithmetic and a compare against `$39`, the code of `9`. There is no score variable in RAM. It is compact and it is fragile: anything that overwrites those cells changes the score. PONG keeps its score in RAM and draws it.

Most screen accesses are preceded by a delay. Before each strobe and store in the ball and paddle code there is a `JSR DELAY2` or `DELAY1`. `DELAY2` counts A down through a full cycle of 256 values (it stops when A is back to the value it started with); `DELAY1` is a variant that starts from the complement. Because the delay comes immediately before the strobe, the calls also slow the game. Whether the delays are needed for the hardware, or are the author's way of pacing the game, cannot be told from the listing. PONG does not use them and is untested on the real machine (Chapter 22); if PONG's drawing is unreliable on the hardware, this is the first thing to compare.

It calls the firmware

At `BALLCHK_069` Rubout sets the stack, puts the address of its message text `GAME OVER HIT CLEAR TO RESET` into `$B3`, a screen address into `$B1`, a count into B and does `JSR $F6BB`: it calls the firmware's `BLITCPY` (Chapter 23) to put the message on the screen. The address `$F6BB` is `BLITCPY` in the 129867 firmware. In the 129827 firmware it is not, as Chapter 23 shows, so Rubout's game-over message is a case of a game that depends on one firmware's internals. Whether Rubout runs correctly on a 129827 terminal is not something the listing alone can settle.

Shuttle Attack

Shuttle Attack is a Space-Invaders-style game ("SHUTTLE ATTACK V.2", "COPYRIGHT (C) 1982", "3 SHIPS LEFT", "PRESS FUNCTION TO START"). About two-thirds of its bytes are reached by control-flow tracing; the rest are tables of sprite shapes, text and an object table. Its start-up:

```
E2CF                 173  RESET
E2CF  CE 80 00       174          LDX     #$8000
E2D2                 175  RESET_003
E2D2  7D 70 00       176          TST     $7000
E2D5  01             177          NOP
E2D6  6F 00          178          CLR     $00,X
E2D8  08             179          INX
E2D9  8C 87 80       180          CPX     #$8780
E2DC  26 F4          181          BNE     RESET_003
E2DE  86 03          182          LDAA    #$03
E2E0  B7 7A 00       183          STAA    $7A00
E2E3  B7 7C 00       184          STAA    $7C00
E2E6  86 17          185          LDAA    #$17
E2E8  B7 7F 06       186          STAA    $7F06
E2EB  86 51          187          LDAA    #$51
E2ED  B7 7F 0C       188          STAA    $7F0C
```

```
E2F0  B6 78 02       189          LDAA    $7802
E2F3  84 FB          190          ANDA    #$FB
E2F5  B7 78 02       191          STAA    $7802
E2F8  86 98          192          LDAA    #$98
E2FA  B7 87 30       193          STAA    $8730
E2FD  8E 00 3F       194          LDS     #$003F
E300  CE E7 B7       195          LDX     #$E7B7
E303  DF F2          196          STX     $F2
E305  C6 12          197          LDAB    #$12
E307  CE 83 3D       198          LDX     #$833D
E30A  8D 10          199          BSR     RESET_04D
E30C  C6 12          200          LDAB    #$12
E30E  CE 83 DD       201          LDX     #$83DD
E311  8D 09          202          BSR     RESET_04D
E313  C6 25          203          LDAB    #$25
E315  CE 84 74       204          LDX     #$8474
E318  8D 02          205          BSR     RESET_04D
E31A  20 16          206          BRA     RESET_063
```

- The screen is cleared with `TST $7000` as the strobe instead of the `STAA HALT` used elsewhere. The strobe is an *access* to the address `$7000`; a read serves as the strobe as well as a write does [PGM]. This book's tutorials use the store, which every program uses.
- `STAA $7A00` and `STAA $7C00` with `$03`: the serial master reset again, at the start this time.
- The cursor pair is `$17` to `$7F06` and `$51` to `$7F0C`: the same register, a different value from Rubout's `$E8`. Both numbers are larger than 79, so both evidently park the cursor beyond the right edge; both programs work.
- `LDS #$003F`: the stack pointer is set to `$003F`, in the direct page, rather than `$03FF`. The stack and the game's direct-page variables share the same 64 bytes. It works because the programmer counted the depth of calls, and it is not a habit to copy: a stack that grows into the variables corrupts them without any warning.
- `RESET_04D` is a small block-copy loop: source pointer in `$F2`, destination in X (saved in `$F0`), count in B, strobe before each store. It is the same job as the firmware's `BLITCPY`, written out again. Shuttle Attack calls no firmware routines.

Cosmic Intruders and Space Invaders

These two are known here from their images only. A scan of their traced code finds these hardware addresses:

ROM	Hardware addresses used (from `$7000` up)
Space Invaders	`$7000` (strobe), `$7D00` (switch bank), screen cells
Cosmic Intruders	`$7000`, `$7800` (keyboard), `$7E00` (a switch bank), `$7F06`/`$7F0C` (cursor)

and in addition both call into the firmware: Space Invaders `$EF31` (the ESC dispatcher), `$ECBA`/`$ECC1` (`WAKE`/`BLOCKME` in the scheduler), `$ECF0` (inside `GETRX`), `$EFF6`, `$F6BB` and `$F740` (`BLITCPY`, `CLRSCR`); Cosmic Intruders `$EFAD` (`XPLUSA`), `$F0DA` (`BELL`), `$F6BB`, `$F740`. Space Invaders' traced code touches no keyboard address at all, so it must get its keys through the firmware, and the callback at `$E002` is the likely route; Cosmic Intruders' traced code touches `$7800` as well as calling the firmware. They appear to be **firmware-cooperative** programs, letting the firmware scan keys and run the scheduler and using the callbacks of Chapter 25.

Rubout, Shuttle Attack and PONG are **machine-owning** programs: they poll the hardware themselves and do not give control back.

The firmware addresses that these two call (`$ECBA`, `$ECC1`, `$EF31`, `$EFF6`, `$F740`) are all 129867 addresses; none is checked against the 129827 set.

What to take from the working programs

- **Copy conventions, not code.** The frame, the strobe, the PIA key read, the bell pulse and the cursor pair are common to all of them and are safe to copy.
- **Differences are options, not errors.** The cursor value (`$E8` or `$51`), the stack position, `TST` or `STAA` as the strobe: each works. When two working programs differ, either choice is [PGM].
- **Dependence on the firmware is a risk.** Rubout calls `$F6BB`; so do Space Invaders and Cosmic Intruders. A machine-owning program that calls nothing in the firmware, like PONG and Shuttle Attack, has one fewer thing to break.
- **Read what the original programmers did to make it fit.** Two kilobytes is not much. Rubout's screen scoreboard and Shuttle Attack's stack in zero page are space-saving tricks, and the price is fragility.

Exercises

1. Run `tools/rdis.py` or any disassembler on the five images and list the first instruction executed after ESC ` on the 129867 firmware for each. (Answers: Rubout `SEI`; Shuttle Attack, `NOP`; the others, `JMP $E088`.)
2. Find in `RUBOUT.LST` every place where `$F6BB` is called. How many bytes does each message take?
3. Write down in your own words the algorithm of Rubout's `BALLCHK` when the ball hits a brick. Which parts are "scoring" and which are "movement"?
4. Which of the five programs would you expect to be affected if the scheduler of the 129867 firmware were replaced? Explain.

Chapter 25

Hooks: sharing the machine with the firmware

A plug-in is not only a program that runs. It is a guest in a machine whose firmware looks into socket A at several moments, each time asking “is there a plug-in, and does it want anything?”. This chapter lists every such moment in the two firmware sets, says what the firmware does when the answer is yes, and shows the three ways a plug-in can behave: **decline** (what PONG does), **return** (hand the machine back), and **own** (never return). A last section shows the one safe way to get back to the firmware from your own program.

All the addresses here are [FW]: found in the firmware images by following the code, and checked by running the real ROM sets in the emulator.

The plug-in flag

At address `$0000` of RAM is a byte the firmware calls `PLUGFLG`. It is the firmware’s note to itself: “a plug-in is present and willing”. It is zero after RAM is cleared at power-on. The firmware sets it (by adding one) when it finds a plug-in, and it consults the flag at four later points. Everything in this chapter is the consequence of those four.

The five times the firmware looks at socket A

1. The probe, at power-on

```
E8BB                152  ; Plug-in probe: is there a ROM in socket A that
E8BB                153  ; starts "TJ"?
E8BB  FE E0 00      154  PLUGPROB        LDX     $E000
E8BE  8C 54 4A      155                  CPX     #$544A
E8C1  26 0D         156                  BNE     NOPLUG
E8C3  B6 7D 00      157                  LDAA    SWBNK3
E8C6  2B 08         158                  BMI     NOPLUG
E8C8  7C 00 00      159                  FCB     $7C,$00,$00 ; INC $0000
E8CB  BD E0 7D      160                  JSR     $E07D
E8CE  20 04         161                  BRA     PIAINIT
E8D0                162  ; No usable plug-in: continue the normal boot.
E8D0  86 1D         163  NOPLUG          LDAA    #$1D
E8D2  97 DB         164                  STAA    $DB
```

During boot, after the RAM and ROM checks, the firmware examines `$E000-$E001`. If they hold "TJ" (`$544A`) **and** the switch bank at `$7D00` has bit 7 clear, the firmware does `INC $0000` and calls `$E07D`. In the 129827 firmware the same probe calls `$E087`. Whatever the plug-in does at that address happens *during boot*, with the stack in use and the hardware half-initialised.

2. The configuration table, a little later

```
E94F  96 00          219  LE94F           LDAA    PLUGFLG
E951  26 05          220                  BNE     LE958
E953  CE E9 F8       221                  LDX     #INITTAB
E956  20 03          222                  BRA     LE95B
E958  CE E5 30       223  LE958           LDX     #$E530
E95B  DF 43          224  LE95B           STX     $43
E95D  DF B3          225                  STX     $B3
E95F  C6 1E          226                  LDAB    #$1E
E961  BD F6 BB       227                  JSR     BLITCPY
E964  B6 00 4D       228                  FCB     $B6,$00,$4D ; LDAA $004D
E967  F6 00 55       229                  FCB     $F6,$00,$55 ; LDAB $0055
E96A  C5 40          230                  BITB    #$40
E96C  26 02          231                  BNE     LE970
E96E  86 20          232                  LDAA    #$20
E970  36             233  LE970           PSHA
E971  BD F6 5E       234                  JSR     LF65E
E974  BD F7 90       235                  JSR     LF790
E977  B6 00 52       236                  FCB     $B6,$00,$52 ; LDAA $0052
E97A  BD F5 56       237                  JSR     LF556
E97D  BD F6 5E       238                  JSR     LF65E
E980  BD F7 90       239                  JSR     LF790
E983  B6 00 52       240                  FCB     $B6,$00,$52 ; LDAA $0052
E986  BD F5 56       241                  JSR     LF556
E989  32             242                  PULA
E98A  30             243                  TSX
E98B  A6 01          244                  LDAA    $01,X
E98D  AA 03          245                  ORAA    $03,X
E98F  AA 00          246                  ORAA    $00,X
E991  27 42          247                  BEQ     LE9D5
E993  CE E9 BD       248                  LDX     #ERRMSG
E996  8D 32          249                  BSR     LE9CA
E998  BD F3 13       250                  JSR     LF313
E99B  30             251                  TSX
E99C  C6 05          252                  LDAB    #$05
E99E  BD F1 D0       253                  JSR     LF1D0
E9A1  32             254                  PULA
E9A2  32             255                  PULA
E9A3  32             256                  PULA
E9A4  32             257                  PULA
E9A5  32             258                  PULA
E9A6  BD EF 8B       259  LE9A6           JSR     LEF8B
E9A9  7D 00 AE       260                  FCB     $7D,$00,$AE ; TST $00AE
E9AC  2A F8          261                  BPL     LE9A6
E9AE  20 2A          262                  BRA     LE9DA
E9B0                 263  ; Self-test messages (13 characters each).
E9B0  54 65 73 74    264  TSTMSG          FCB     "Test Complete"
E9BD  4D 45 4D 4F    265  ERRMSG          FCB     "MEMORY ERROR "
```

The firmware needs a table of 30 starting values for the terminal's settings. They are copied (by `BLITCPY`, Chapter 23) to `$0045-$0062`. The source of the table is chosen in three ways:

- if the two bytes at `$7B1E` are `$30 $46` (the ASCII characters `0F`), from `$7B00`: a table in a ROM in the separate option socket;
- otherwise, if `PLUGFLG` is zero, from the firmware's own table `INITTAB` at `$E9F8`;
- otherwise, from `$E530`: **a table that the plug-in must supply**.

So a plug-in that leaves the flag set and has no valid table at `$E530` gives the terminal random starting values. This is the first reason why a plug-in that does not mean to customise the terminal clears the flag in its boot hook. (`INITTAB` begins `1B 0D 00 00 ...`; its first byte lands in `ESCCHR` at `$0045`, the escape character.)

In the 129827 firmware the same choice is made with the table at `$E545`.

3. The scheduler's task table

```
EBF8  CE 00 85       368  LEBF8          LDX     #$0085
EBFB  86 80          369                 LDAA    #$80
EBFD  C6 08          370                 LDAB    #$08
EBFF  8D 4B          371                 BSR     LEC4C
EC01  CE 00 8D       372                 LDX     #$008D
EC04  C6 08          373                 LDAB    #$08
EC06  4F             374                 CLRA
EC07  8D 43          375                 BSR     LEC4C
EC09  CE EE 06       376                 LDX     #KBSCAN
EC0C  DF 65          377                 STX     $65
EC0E  CE FD 27       378                 LDX     #TXDELAY
EC11  DF 73          379                 STX     $73
EC13  96 00          380                 LDAA    PLUGFLG
EC15  26 1B          381                 BNE     LEC32
EC17  CE EE A1       382                 LDX     #KEYHAND
EC1A  DF 67          383                 STX     $67
EC1C  CE ED 05       384                 LDX     #LED05
EC1F  DF 69          385                 STX     $69
EC21  CE ED 58       386                 LDX     #LED58
EC24  DF 6B          387                 STX     $6B
EC26  CE F9 F5       388                 LDX     #LF9F5
EC29  DF 6D          389                 STX     $6D
EC2B  CE FE 03       390                 LDX     #LFE03
EC2E  DF 6F          391                 STX     $6F
EC30  20 03          392                 BRA     LEC35
EC32  BD E0 CA       393  LEC32          JSR     $E0CA
EC35  CE 00 00       394  LEC35          LDX     #$0000
EC38  DF 9D          395                 STX     CURTASK
EC3A  DF 9F          396                 STX     $9F
EC3C  CE 00 00       397                 LDX     #$0000
EC3F  BD EC BA       398                 JSR     WAKE
EC42  CE 00 02       399                 LDX     #$0002
EC45  BD EC BA       400                 JSR     WAKE
EC48  0E             401                 CLI
EC49  7E EC 53       402                 JMP     SCHED
```

The scheduler (Chapter 27) runs up to eight tasks, whose entry addresses sit in a table at `$65`-`$6F`. The firmware always installs task 0 (`KBSCAN`, the keyboard scanner) and one more at `$73`. But the other five it installs only if `PLUGFLG` is zero. If the flag is set it calls `$E0CA` instead: a routine **in the plug-in** whose job is to put the plug-in's own tasks into the table. In the 129827 firmware the corresponding call is `$E0C3`.

4. After the boot

```
F143  D6 00         1028  LF143           LDAB    PLUGFLG
F145  27 03         1029                  BEQ     LF14A
F147  BD E0 8A      1030                  JSR     $E08A
F14A  39            1031  LF14A           RTS
```

At the end of the initialisation (called from the boot code just before the scheduler starts) the firmware calls `LF143`. If the flag is set it calls `$E08A`, a plug-in routine for any last set-up. In the 129827 firmware there is a similar call to `$E094` (from `$FBF8`).

5. ESC ` (the grave accent)

```
F22C                1140  ; ESC ` : if socket A holds "TJ" jump to its start
F22C                1141  ; ($E03C).
F22C  FE E0 00      1142  ESCGRAVE        LDX     $E000
F22F  8C 54 4A      1143                  CPX     #$544A
F232  26 F7         1144                  BNE     LF22B
F234  7E E0 3C      1145                  JMP     $E03C
```

When the operator types ESC and then the grave accent, the escape dispatcher (Chapter 28) examines `$E000` again. If “TJ” is there it jumps to `$E03C` (`$E03E` in the 129827). This is the start every program in this book uses. The handler tests only the signature: it does not look at the flag, and it does not look at the switch at `$7D00` that the boot probe honours.

The summary table

What	129867	129827	If the plug-in does nothing
boot hook (JSR at boot, flag already incremented)	`$E07D`	`$E087`	undefined: whatever bytes are at that address are executed
config table, when the flag is set	`$E530`	`$E545`	random settings
task-table hook, when the flag is set	`$E0CA`	`$E0C3`	the task table stays empty and the terminal does nothing
post-boot hook, when the flag is set	`$E08A`	`$E094`	
ESC ` entry	`$E03C`	`$E03E`	nothing happens

The sizes of the damage in the right-hand column explain why the frame has stubs at every one of these addresses: an `RTS` at each costs one byte and removes the crash.

(The two-line `RTS` stubs at `$E094`, `$E0C3` and `$E0CA` in the frame are right for the *post-boot* and *task-table* hooks only if the flag is clear, which is why the boot hook clears it: with the flag clear the firmware never calls them.)

Three behaviours

Decline (PONG, and the frame of this book)

The boot hook is

```
        CLR     $0000           ; the plug-in flag
        PULA                    ; drop the JSR return address
```

```
        PULA
        JMP     $E8D0           ; the "no plug-in" boot path
```

The flag goes back to zero, so the table in `$E530` is not used and the scheduler installs its own tasks. The two `PULA`s discard the return address that the firmware's `JSR` pushed; they are needed because the hook then jumps rather than returns. `$E8D0` is the label `NOPLUG`: the firmware carries on as if no plug-in had been found, boots normally, shows `Test Complete`, and the plug-in waits for ESC `` ` ``. In the 129827 the target is `$E9D0`. PONG, Space Invaders and Cosmic Intruders all do this.

A shorter way to decline is the one Shuttle Attack uses: `CLR $0000` and `RTS`. The flag is cleared, and since the hook *returns*, the boot continues from the instruction after the `JSR`, which is the same place `NOPLUG` leads to (the only difference is that the two instructions at `NOPLUG`, which store `$1D` into the byte at `$00DB`, are skipped). Rubout arrives at the same result by branching into the middle of its own start-up code that happens to contain `CLR $0000` and `RTS`. All three work.

Return (a plug-in that gives the machine back)

A program that starts from ESC `` ` `` can return the terminal to the state it had at power-on by restarting the firmware. The reset entry is not at the same address in both sets (`$E800` in 129867, `$E900` in 129827), but **the vector at `$FFFE` always points to it**:

```
        LDX     $FFFE           ; the firmware's reset entry
        JMP     0,X
```

T19EXIT is a tiny program that shows a message, waits for a key, and does exactly this:

```
E100  0F              67  START           SEI
E101  8E 03 FF        68                  LDS     #$03FF
E104  BD E2 00        69                  JSR     INITCRT
E107  BD E1 6E        70                  JSR     CLRSCR
E10A  86 0B           71                  LDAA    #11
E10C  B7 01 00        72                  STAA    ROWARG
E10F  86 16           73                  LDAA    #22
E111  B7 01 01        74                  STAA    COLARG
E114  CE E1 46        75                  LDX     #MSG
E117  BD E1 E9        76                  JSR     PUTSTR
E11A                  77  ; wait until no key is down (ESC ` may still be held)
E11A  BD E1 29        78  REL             JSR     ANYKEY
E11D  26 FB           79                  BNE     REL
E11F                  80  ; wait for a key
E11F  BD E1 29        81  WAITK           JSR     ANYKEY
E122  27 FB           82                  BEQ     WAITK
E124                  83  ; firmware reset entry
E124  FE FF FE        84                  LDX     $FFFE
E127  6E 00           85                  JMP     0,X
```

Two details matter. The program first waits until **no key is down**: when it starts, the grave-accent key that launched it may still be held, and a program that tested for a key at once would exit at once. Then it waits for a key and jumps. The test:

```
Test Complete
```

The screen is the terminal's own `Test Complete`, the result of rebooting: the first ESC `` ` `` started the program (it drew its message, as the test verifies), a key was pressed, and the firmware restarted. The test runs on both firmware sets and passes on both. (A first version used `JMP $E800` and worked on the 129867 only: on the 129827 `$E800` is in the middle of an instruction,

and the processor stopped on an illegal opcode. Reading the vector is the cure, and reading the vector is the habit to form.)

Everything is reinitialised: the RAM test runs and clears memory, the screen is cleared, the hardware is set up again. Nothing of the program survives, which is the point.

Own (never return)

PONG, Rubout and Shuttle Attack start and never return: the processor runs their loops until the power is switched off. They do not use the firmware's scheduler and no task of the firmware runs while they play. The terminal does not talk to its host. This is the simplest design, and it is all this book needs.

Cooperating with the scheduler

The remaining possibility is a plug-in that lets the firmware keep running and adds its own tasks. The hooks at `$E0CA` and `$E08A` are the route: leave the flag set, supply a valid table at `$E530`, install your tasks at `$E0CA`, and the scheduler will run them alongside the firmware's. Space Invaders and Cosmic Intruders call routines in the scheduler (`BLOCKME`, `WAKE`) from their own code, which is the signature of a task. The exact protocol (the format of the task's status byte, what a task must preserve, what happens on its return) is the subject of Chapter 27. It has not been tested with a program of this book's making, and a program that wanted to rely on it should be written and run in the emulator first.

Rules for a well-behaved plug-in

1. Begin with "TJ", and give the machine a frame for both sets (Chapter 5).
2. Handle all five moments, even if only with `RTS` stubs.
3. If you do not want to customise the terminal, **clear the flag and decline**.
4. Never hard-code a firmware reset address; read the vector.
5. If your program owns the machine, say so at the top of the source and begin with `SEI`.
6. Test on both sets in the emulator, from the firmware boot, with ESC `, not only from the plug-in-only start.

Exercises

1. Write a plug-in whose boot hook is the Shuttle Attack one (`CLR $0000`, `RTS`) and whose program prints a message. Test it on both firmware sets. Which of the two hooks does each set call?
2. What would happen if the boot hook did the first line and the jump of the decline sequence but not the two `PULA`s? (The boot code that follows examines the results of its self-tests on the stack at fixed offsets from the stack pointer; Chapter 26 shows it. What does it find if two extra bytes are on the stack?)
3. Change T19EXIT so that it returns to the firmware only when a particular key is pressed (pick a key and look up its scan code in Appendix D) and otherwise beeps. What must be added to avoid the key bounce problem of Chapter 9?
4. What would you have to put at `$E530` for a plug-in that wants the terminal to start with a different escape character?

Part IV

The firmware

Chapter 26

Power-on: what the firmware does

From here to the end of Part IV the book reads the firmware itself. You do not need any of it to write a game, but it is the best 6,144 bytes of 6800 code that this machine has, and it explains every convention the earlier chapters took on trust. The listing is in Appendix F; this chapter follows the 129867 firmware from the instant the processor leaves reset until the scheduler starts. Every excerpt is the real listing, produced by the book's assembler from a source that reassembles to the firmware ROM images byte for byte. Where the 129827 differs, the chapter says so; Appendix H lists the differences.

Reset

The processor takes its first instruction from the address in the RESET vector at `$FFFE`: `$E800` in the 129867, `$E900` in the 129827. The routine begins:

```
E800                 57  ; Power-on / reset entry.  The IRQ vector is $ED83;
E800                 58  ; SWI, NMI and RESET all point here (see the vector
E800                 59  ; table at $FFF8).
E800  0F             60  RESET           SEI
E801  B7 7F 0E       61                  STAA    VTACE
E804  B7 7F 0A       62                  STAA    VTACA
E807  CE 7F 00       63                  LDX     #$7F00
E80A  86 65          64                  LDAA    #$65
E80C  A7 00          65                  STAA    $00,X
E80E  86 44          66                  LDAA    #$44
E810  A7 01          67                  STAA    $01,X
E812  86 5D          68                  LDAA    #$5D
E814  A7 02          69                  STAA    $02,X
E816  B6 7E 00       70                  LDAA    SWBNK1
E819  84 02          71                  ANDA    #$02
E81B  27 0A          72                  BEQ     LE827
E81D  86 20          73                  LDAA    #$20
E81F  A7 04          74                  STAA    $04,X
E821  86 14          75                  LDAA    #$14
E823  A7 05          76                  STAA    $05,X
E825  20 08          77                  BRA     LE82F
E827  86 40          78  LE827           LDAA    #$40
E829  A7 04          79                  STAA    $04,X
E82B  86 35          80                  LDAA    #$35
E82D  A7 05          81                  STAA    $05,X
```

- `SEI` masks interrupts. Nothing has been set up to handle one yet.
- The next two instructions store A, **uncleared**, into `VTACE` and `VTACA` (`$7F0E` and `$7F0A`). The 6800 does not clear its registers on reset, so what is written is whatever A held (Chapter 6 noted this). Both registers are rewritten or are unimportant by the time the screen is used.
- The CRT controller's registers 0 to 2 are loaded with `$65`, `$44` and `$5D`. Registers 4 and 5 are loaded with `$40`/`$35`, or with `$20`/`$14` if bit 1 of the switch bank at `$7E00` is set: a switch selects between two sets of display timing (Chapter 6).
- `$17` goes to register 3 and register 6 and to the zero-page byte `FIRSTLN`, which is the firmware's copy of register 6, the scroll register (Chapter 8). Registers `$0C`, `$0D` and `$0E` (cursor column, cursor row, and one more) are cleared.

By the end of this block the CRT controller is running, so the display is scanning, though nothing is yet written to it.

The RAM test

```
E82F  86 17           82  LE82F          LDAA    #$17
E831  A7 03           83                  STAA    $03,X
E833  A7 06           84                  STAA    $06,X
E835  97 15           85                  STAA    FIRSTLN
E837  4F              86                  CLRA
E838  A7 0D           87                  STAA    $0D,X
E83A  A7 0C           88                  STAA    $0C,X
E83C  A7 0E           89                  STAA    $0E,X
E83E  8E 03 FF        90                  LDS     #$03FF
E841  7F 8F 80        91                  CLR     $8F80
E844  7F 8F 81        92                  CLR     $8F81
E847  CE 03 FF        93                  LDX     #$03FF
E84A  C6 AA           94                  LDAB    #$AA
E84C  E7 00           95  LE84C          STAB    $00,X
E84E  09              96                  DEX
E84F  26 FB           97                  BNE     LE84C
E851  CE 03 FF        98                  LDX     #$03FF
E854  C6 AA           99  LE854          LDAB    #$AA
E856  E1 00          100                  CMPB    $00,X
E858  26 10          101                  BNE     LE86A
E85A  53             102                  COMB
E85B  E7 00          103                  STAB    $00,X
E85D  E1 00          104                  CMPB    $00,X
E85F  26 09          105                  BNE     LE86A
E861  6F 00          106  LE861          CLR     $00,X
E863  09             107                  DEX
E864  26 EE          108                  BNE     LE854
E866  6F 00          109                  CLR     $00,X
E868  20 0A          110                  BRA     LE874
E86A  A6 00          111  LE86A          LDAA    $00,X
E86C  B7 8F 81       112                  STAA    $8F81
E86F  F7 8F 80       113                  STAB    $8F80
E872  20 ED          114                  BRA     LE861
```

Before trusting RAM for anything, the firmware tests it. It does not yet have a stack to use, so the routine has to work with registers alone. `LDS #$03FF` points the stack pointer at the top of RAM, but nothing is pushed until the tests are done.

The test has the shape you would write yourself:

1. Fill every byte from `$03FF` down to `$0001` with `$AA` (`LDAB #$AA`, `STAB 0,X`, `DEX`, `BNE`: the loop ends when X reaches 0, so address 0 is not written in this pass).
2. Walk the memory the same way. For each byte, compare with `$AA`; if it matches, complement B (`COMB` gives `$55`), store it, compare again; then `CLR 0,X` to leave a zero. A failure at either compare jumps to `LE86A`.
3. After the loop, `CLR 0,X` clears address 0.

The two patterns `$AA` and `$55` are complementary (`10101010` and `01010101`), so between them every bit cell has held a 1 and a 0. The test checks that each bit can be set and cleared and that no two bytes alias to the same cell in a way that shows in the pattern. It is a modest test, not a complete one, and it leaves **all of RAM cleared to zero**, which is why every plug-in starts with RAM zeroed (a plug-in launched later with ESC ` sees whatever the running firmware has left there).

If a byte fails, `LE86A` records what it found in two bytes of the **screen memory**, `$8F81` (the value read) and `$8F80` (the pattern expected), because RAM cannot be trusted and the screen RAM has not yet been tested. These two bytes were cleared at the start of the routine (`CLR $8F80`, `CLR $8F81`) so a passing run leaves zero in them. After the test, `LE874` pushes both on the stack, which is now safe to use.

The screen RAM test

```
E874  B6 8F 81       115  LE874          LDAA    $8F81
E877  36             116                 PSHA
E878  B6 8F 80       117                 LDAA    $8F80
E87B  36             118                 PSHA
E87C  CE 8F FF       119                 LDX     #$8FFF
E87F  C6 AA          120                 LDAB    #$AA
E881  E7 00          121  LE881          STAB    $00,X
E883  09             122                 DEX
E884  8C 80 00       123                 CPX     #$8000
E887  26 F8          124                 BNE     LE881
E889  CE 8F FF       125                 LDX     #$8FFF
E88C  C6 AA          126  LE88C          LDAB    #$AA
E88E  E1 00          127                 CMPB    $00,X
E890  26 11          128                 BNE     LE8A3
E892  53             129                 COMB
E893  E7 00          130                 STAB    $00,X
E895  E1 00          131                 CMPB    $00,X
E897  26 0A          132                 BNE     LE8A3
E899  6F 00          133  LE899          CLR     $00,X
E89B  09             134                 DEX
E89C  8C 80 00       135                 CPX     #$8000
E89F  26 EB          136                 BNE     LE88C
E8A1  20 08          137                 BRA     LE8AB
E8A3  A6 00          138  LE8A3          LDAA    $00,X
E8A5  97 BA          139                 STAA    $BA
E8A7  D7 B9          140                 STAB    $B9
E8A9  20 EE          141                 BRA     LE899
E8AB  96 BA          142  LE8AB          LDAA    $BA
E8AD  36             143                 PSHA
E8AE  96 B9          144                 LDAA    $B9
E8B0  36             145                 PSHA
```

```
E8B1  4F             146                 CLRA
E8B2  CE E8 00       147                 LDX     #RESET
```

The same test, on $8000-$8FFF: fill with $AA, check, complement, check, clear. It tests all 4 KB, not just the visible 1,920 bytes: the screen memory also holds the second display page and, at the top, parts the firmware uses (Chapter 29). A failure here records its two bytes in the zero-page bytes $BA and $B9, and they are pushed in turn.

The ROM test

```
E8B5  A8 00          148  ROMSUM         EORA    $00,X
E8B7  08             149                 INX
E8B8  26 FB          150                 BNE     ROMSUM
E8BA  36             151                 PSHA
```

A four-line loop: A is cleared, X is set to $E800, and every byte up to $FFFF is exclusive-ORed into A. (X counts up and wraps to zero after $FFFF; BNE ends the loop on the wrap.) The result is pushed. If the ROMs are intact, the exclusive-OR of all 6,144 bytes is **zero**. The ROMs have been made so that this holds. In the 129867 set **each of the three ROMs** has an exclusive-OR of zero by itself ($E800-$EFFF, $F000-$F7FF and $F800-$FFFF give 00, 00, 00). In the 129827 set the three ROMs give $59, $78 and $21, which are not zero separately but cancel in the total ($59 xor $78 = $21).

You can run the same test yourself. T20ROMSUM is the loop above followed by a call to PUTHEX:

```
E100  0F              66  START          SEI
E101  8E 03 FF        67                 LDS     #$03FF
E104  BD E1 FE        68                 JSR     INITCRT
E107  BD E1 48        69                 JSR     CLRSCR
E10A  4F              70                 CLRA
E10B  CE E8 00        71                 LDX     #$E800
E10E  A8 00           72  SUM            EORA    0,X
E110  08              73                 INX
E111                  74  ; X wraps to 0 after $FFFF
E111  26 FB           75                 BNE     SUM
E113  36              76                 PSHA              ; keep the checksum
E114  86 04           77                 LDAA    #4
E116  B7 01 00        78                 STAA    ROWARG
E119  86 0A           79                 LDAA    #10
E11B  B7 01 01        80                 STAA    COLARG
E11E  CE E1 34        81                 LDX     #LABEL
E121  BD E1 C3        82                 JSR     PUTSTR
E124  86 04           83                 LDAA    #4
E126  B7 01 00        84                 STAA    ROWARG
E129  86 1D           85                 LDAA    #29
E12B  B7 01 01        86                 STAA    COLARG
E12E  32              87                 PULA
E12F  BD E1 E5        88                 JSR     PUTHEX    ; two hex digits
```

and prints:

```
ROM XOR CHECKSUM = 00
```

00 on both the 129867 and the 129827 firmware sets, both in the emulator (and a Python one-liner over the six ROM files gives the same). The book's emulator test checks it. This also verifies that the ROM images used for this book are whole: a corrupted bit anywhere in the 6 KB would make the result non-zero.

The plug-in probe

```
E8BB                152  ; Plug-in probe: is there a ROM in socket A that
E8BB                153  ; starts "TJ"?
E8BB  FE E0 00      154  PLUGPROB        LDX     $E000
E8BE  8C 54 4A      155                  CPX     #$544A
E8C1  26 0D         156                  BNE     NOPLUG
E8C3  B6 7D 00      157                  LDAA    SWBNK3
E8C6  2B 08         158                  BMI     NOPLUG
E8C8  7C 00 00      159                  FCB     $7C,$00,$00 ; INC $0000
E8CB  BD E0 7D      160                  JSR     $E07D
E8CE  20 04         161                  BRA     PIAINIT
E8D0                162  ; No usable plug-in: continue the normal boot.
E8D0  86 1D         163  NOPLUG          LDAA    #$1D
E8D2  97 DB         164                  STAA    $DB
```

This is the first of the five moments of Chapter 25: look for “TJ”, look at the switch, increment the flag, call the hook. The hook runs here with five bytes on the stack (the three test results and the return address): that is why a hook that returns must not disturb the stack, and a hook that jumps away must discard the return address.

Initialising the hardware

The `PIAINIT` block sets up the keyboard PIA (the direction registers, the initial output values), master-resets both ACIAs, computes their working control bytes from the switches (Chapter 12), sets the display pointers `CRTBASE` and `CURADR` to `$8000` and the first-line register, and writes the page base address and `$17` into five bytes at the top of each display page (`$87D0`-`$87D4` and `$8FD0`-`$8FD4`), the beginning of a saved state for each page (Chapter 29). Then it calls two small routines: `LED7B` sets the two pointers of the receive queue to `$0110` (the queue occupies `$0110`-`$037F`; Chapter 27), and `LFD6A` sets up the transmit queue pointers and clears the transmit counter (Chapter 30).

The values the PIA is given are the ones Chapters 9 and 10 relied on: `$7F` in the direction register of port A (bits 0-6 are outputs, the scan code; bit 7 is the input that says the key is down), `$1D` in the direction register of port B, `$3C` in both control registers, and then `$01` in each output register.

The configuration table

```
E94F  96 00         219  LE94F           LDAA    PLUGFLG
E951  26 05         220                  BNE     LE958
E953  CE E9 F8      221                  LDX     #INITTAB
E956  20 03         222                  BRA     LE95B
E958  CE E5 30      223  LE958           LDX     #$E530
E95B  DF 43         224  LE95B           STX     $43
E95D  DF B3         225                  STX     $B3
E95F  C6 1E         226                  LDAB    #$1E
E961  BD F6 BB      227                  JSR     BLITCPY
E964  B6 00 4D      228                  FCB     $B6,$00,$4D ; LDAA $004D
E967  F6 00 55      229                  FCB     $F6,$00,$55 ; LDAB $0055
E96A  C5 40         230                  BITB    #$40
E96C  26 02         231                  BNE     LE970
E96E  86 20         232                  LDAA    #$20
```

Chapter 25 explained where the 30 bytes come from. They are copied to `$0045-$0062` by `BLITCPY`. The first, `$1B`, is `ESCCHR`: the character that begins an escape sequence.

The screen and the verdict

The next stretch (from `$E96E`) calls display routines twice, once for each display page, with the blank character; these are the routines that clear and switch pages (Chapter 29). Then the firmware examines the saved test results:

```
E970  36             233  LE970           PSHA
E971  BD F6 5E       234                  JSR     LF65E
E974  BD F7 90       235                  JSR     LF790
E977  B6 00 52       236                  FCB     $B6,$00,$52 ; LDAA $0052
E97A  BD F5 56       237                  JSR     LF556
E97D  BD F6 5E       238                  JSR     LF65E
E980  BD F7 90       239                  JSR     LF790
E983  B6 00 52       240                  FCB     $B6,$00,$52 ; LDAA $0052
E986  BD F5 56       241                  JSR     LF556
E989  32             242                  PULA
E98A  30             243                  TSX
E98B  A6 01          244                  LDAA    $01,X
E98D  AA 03          245                  ORAA    $03,X
E98F  AA 00          246                  ORAA    $00,X
E991  27 42          247                  BEQ     LE9D5
E993  CE E9 BD       248                  LDX     #ERRMSG
E996  8D 32          249                  BSR     LE9CA
E998  BD F3 13       250                  JSR     LF313
E99B  30             251                  TSX
E99C  C6 05          252                  LDAB    #$05
E99E  BD F1 D0       253                  JSR     LF1D0
E9A1  32             254                  PULA
E9A2  32             255                  PULA
E9A3  32             256                  PULA
E9A4  32             257                  PULA
E9A5  32             258                  PULA
E9A6  BD EF 8B       259  LE9A6           JSR     LEF8B
E9A9  7D 00 AE       260                  FCB     $7D,$00,$AE ; TST $00AE
E9AC  2A F8          261                  BPL     LE9A6
E9AE  20 2A          262                  BRA     LE9DA
```

`PULA` takes the ROM checksum into A. `TSX` makes X point at the stack, and the three `LDAA/ORAA` instructions combine **four bytes** (the checksum and three of the saved failure bytes) into one. If the result is zero, `BEQ LE9D5` goes to the success path, which prints `Test Complete`. Otherwise it prints `MEMORY ERROR`, then the saved values in hex (through `LF313` and `LF1D0`), and waits for the operator in a loop at `LE9A6` that polls a flag at `$00AE` until a key arrives.

This is the reason the boot hook must keep the stack balanced: the test reads its results at **fixed offsets from the stack pointer**. Two extra bytes on the stack would make `TSX` point two bytes further up, and the check would read the wrong bytes (Exercise 2 of Chapter 25).

The two messages are 13 bytes each, stored back to back at `$E9B0` and `$E9BD`: `Test Complete` and `MEMORY ERROR`. The routine at `LE9CA` copies one of them to the cursor position with `BLITCPY`:

```
E9CA  DF B3          266  LE9CA           STX     $B3
E9CC  C6 0D          267                  LDAB    #$0D
E9CE  DE 13          268                  LDX     CURADR
E9D0  DF B1          269                  STX     $B1
```

```
E9D2  7E F6 BB       270                  JMP     BLITCPY
E9D5  CE E9 B0       271  LE9D5           LDX     #TSTMSG
E9D8  8D F0          272                  BSR     LE9CA
```

(it loads the source pointer from X, the count 13 into B, the destination from `CURADR`, and *jumps* into `BLITCPY`, a tail call, Chapter 13).

Entering the scheduler

```
E9DA  B6 00 56       273  LE9DA           FCB     $B6,$00,$56 ; LDAA $0056
E9DD  16             274                  TAB
E9DE  84 01          275                  ANDA    #$01
E9E0  B7 78 02       276                  STAA    PIA1B
E9E3  C5 08          277                  BITB    #$08
E9E5  27 03          278                  BEQ     LE9EA
E9E7  BD F0 B3       279                  JSR     LF0B3
E9EA  B6 7E 00       280  LE9EA           LDAA    SWBNK1
E9ED  84 C0          281                  ANDA    #$C0
E9EF  97 41          282                  STAA    $41
E9F1  BD F1 43       283                  JSR     LF143
E9F4  7E EB F8       284                  JMP     LEBF8
E9F7  03             285                  FCB     $03
```

The last few instructions of the boot:

- copy a byte of the configuration table (`$56`) into the PIA's port B, masked to its low bit (the alpha-lock output), and call `LF0B3` if another bit of that byte is set (it sets the "split" flag from the display-page bit);
- read bits 6 and 7 of the switch bank at `$7E00` into `$41`;
- call `LF143`, which is the post-boot hook of Chapter 25 (it calls `$E08A` only if the plug-in flag is set);
- `JMP LEBF8`: the scheduler's initialisation, Chapter 27.

After that the processor never returns to this code. The next time it runs is at the next reset.

What a plug-in can learn from this

- RAM is zero at the moment the scheduler starts, apart from what boot set. A program started by ESC ` finds the **firmware's** variables in zero page, not zeros. A program that needs its own variables initialises them.
- The firmware's boot leaves the stack pointer wherever the scheduler wants it. A plug-in sets its own (`LDS #$03FF`), as every program in this book does.
- The firmware **overwrites the whole of `$8000-$8FFF`** at boot with its tests. A plug-in that wants to leave something on the screen across a reset cannot.
- The boot is long (all the tests and the table copy), and it is the only place the firmware talks to the plug-in without being asked by the operator.

Exercises

1. Run T20ROMSUM, but modify it to start the checksum at `$F000` instead of `$E800`. Is the result still zero on the 129867? On the 129827? (Answers: yes on the 129867, because each of its ROMs balances alone; no on the 129827, where the result is `$59`.)

2. The RAM test starts at `$03FF` and finishes at address 0 *separately*. Why? (Look at the loop's end condition.)
3. The firmware stores the failure record of the first test in screen RAM. Why not in zero page?
4. Compute the exclusive-OR of each of the three ROM images in each set separately, with a script or by extending T20ROMSUM. What do you find, and what does it say about how the two sets were made?
5. Which instruction of the boot makes the terminal "Test Complete" screen appear, and which address holds the text it prints?

Chapter 27

The scheduler and the interrupt

The 129867 firmware is not a loop that does one thing at a time. It is a tiny **cooperative multitasking system**: up to eight *tasks*, each a piece of code that runs, then gives up the processor by saying how long it wants to wait, and a scheduler that decides who runs next. One interrupt handler gathers incoming characters from the serial port and leaves them in a queue for the tasks. This chapter reads the scheduler and the handler, and then builds the same trick in a 40-line program of our own, so that you can see that nothing about it is magic.

(The 129827 firmware has the same structure at different addresses; this chapter reads the 129867 and Appendix H lists the 129827's entry points.)

The data

All of it is in zero page, the first 256 bytes of RAM that the firmware reserves for itself:

Address	Name	Meaning
`$65-$6F, $73`	task table	the entry address of task 0 ($65), 1 ($67), 2 ($69), 3 ($6B), 4 ($6D), 5 ($6F), 6 ($71), 7 ($73)
`$75-$84`	resume table	for each task, the address at which it will resume (two bytes each)
`$85-$8C`	status	one byte per task; **bit 7 set means blocked**
`$8D-$94`	sleep	one byte per task: ticks left to sleep
`$95-$9C`	argument	one byte per task, passed to the task in A when it runs
`$9D-$9E`	`CURTASK`	the task being considered (a 16-bit value, though only the low byte counts)

Setting up

You saw the set-up code in Chapter 25. It sets all eight status bytes to `$80` (every task blocked), all sleep counters to zero, installs task 0 (`KBSCAN`) and task 7 (`TXDELAY`), installs tasks 1 to 5 unless a plug-in has the flag set, then clears `CURTASK` and **wakes** tasks 0 and 2 (clearing bit 7 of their status bytes), and enables interrupts with `CLI` before entering the loop. Tasks 3, 4, 5 and 7 are not woken: they run only when another task wakes them. Task 6 is not installed at all.

The loop

```
EC53                 408  ; Cooperative scheduler: tasks wait with bit 7 of
EC53                 409  ; their status set.
EC53  DE 9D          410  SCHED           LDX     CURTASK
EC55  A6 85          411  LEC55           LDAA    $85,X
EC57  2A 33          412                  BPL     LEC8C
EC59  08             413  LEC59           INX
EC5A  8C 00 08       414                  CPX     #$0008
EC5D  26 F6          415                  BNE     LEC55
EC5F  7F 00 9E       416                  FCB     $7F,$00,$9E ; CLR $009E
EC62  F6 78 02       417                  LDAB    PIA1B
EC65  C5 02          418                  BITB    #$02
EC67  26 EA          419                  BNE     SCHED
EC69  C6 34          420                  LDAB    #$34
EC6B  F7 78 01       421                  STAB    PIA1CRA
EC6E  C6 3C          422                  LDAB    #$3C
EC70  F7 78 01       423                  STAB    PIA1CRA
EC73  CE 00 00       424                  LDX     #$0000
EC76  A6 8D          425  LEC76           LDAA    $8D,X
EC78  27 0A          426                  BEQ     LEC84
EC7A  6A 8D          427                  DEC     $8D,X
EC7C  26 06          428                  BNE     LEC84
EC7E  A6 85          429                  LDAA    $85,X
EC80  84 7F          430                  ANDA    #$7F
EC82  A7 85          431                  STAA    $85,X
EC84  08             432  LEC84           INX
EC85  8C 00 08       433                  CPX     #$0008
EC88  26 EC          434                  BNE     LEC76
EC8A  20 C7          435                  BRA     SCHED
EC8C  DF 9D          436  LEC8C           STX     CURTASK
EC8E  DF 9F          437                  STX     $9F
EC90  16             438                  TAB
EC91  84 7C          439                  ANDA    #$7C
EC93  A7 85          440                  STAA    $85,X
EC95  A6 95          441                  LDAA    $95,X
EC97  78 00 A0       442                  FCB     $78,$00,$A0 ; ASL $00A0
EC9A  DE 9F          443                  LDX     $9F
EC9C  C5 01          444                  BITB    #$01
EC9E  27 0E          445                  BEQ     LECAE
ECA0  C5 02          446                  BITB    #$02
ECA2  27 06          447                  BEQ     LECAA
ECA4  36             448                  PSHA
ECA5  8D 1A          449                  BSR     BLOCKME
ECA7  32             450                  PULA
ECA8  DE 9F          451                  LDX     $9F
ECAA  EE 75          452  LECAA           LDX     $75,X
ECAC  20 02          453                  BRA     LECB0
ECAE  EE 65          454  LECAE           LDX     $65,X
ECB0  C4 30          455  LECB0           ANDB    #$30
ECB2  AD 00          456                  JSR     $00,X
ECB4  DE 9D          457                  LDX     CURTASK
ECB6  20 A1          458                  BRA     LEC59
ECB8  A7 95          459  LECB8           STAA    $95,X
```

Read it as a scan over the tasks:

1. `LDX CURTASK` starts at the current task, normally task 0.

2. `LDAA $85,X` fetches that task's status. `BPL LEC8C`: if bit 7 is clear the task is **ready**; go and run it. Otherwise `INX` and try the next, until X reaches 8.
3. **A task is run** at `LEC8C`. X is stored in `CURTASK`; the status byte is masked with `$7C` (bits 6-2 survive; bits 7, 1 and 0 are cleared); the per-task argument is loaded into A; and then either the task's **start address** (`LDX $65,X` with X doubled) or its **resume address** (`LDX $75,X`) is loaded, depending on the low bits of the old status. A task that was waiting in `GETRX` (bit 1 set) is **blocked again before it is called** (`BSR BLOCKME`), so it runs once with its character and must call `GETRX` again to run again. `ANDB #$30` leaves bits 5-4 of the old status in B as two flag bits for the task. Then `JSR 0,X`: **the task is an ordinary subroutine**, called with A, B and the return address on the stack. When it returns, the loop continues with the next task.
4. When the scan passes task 7 without finding more work, the **idle section** runs. `CLR $9E` resets `CURTASK` to 0. Then it reads port B of the PIA (`$7802`) and tests bit 1. If the bit is **set** the loop starts again at once (`BNE SCHED`): the scheduler is busy-waiting, scanning the tasks over and over. When the bit is **clear**, a **tick** has occurred. The scheduler writes `$34` and then `$3C` to the PIA's control register A (`PIA1CRA`, `$7801`), which pulses the CA2 line; then it walks the eight sleep counters, and for every counter that is non-zero it decrements it, and **if the counter reaches zero it clears the task's blocked bit**. Then back to the top.

The scheduler is thus simple. There are no priorities and no pre-emption. A task runs until it returns. Everything depends on tasks being short and polite.

The source of the tick, bit 1 of port B, is the one hardware signal this loop depends on. It behaves like a regular once-per-frame event, and the sleep counts are in units of it, but **this book has not verified that it is the display's vertical sync** (Chapter 11). No working game uses it.

Blocking and waking

```
ECBA                460  ; WAKE: clear bit 7 of status byte X (task X may
ECBA                461  ; run).
ECBA  A6 85         462  WAKE            LDAA    $85,X
ECBC  84 7F         463                  ANDA    #$7F
ECBE  A7 85         464                  STAA    $85,X
ECC0  39            465                  RTS
ECC1                466  ; BLOCKME: the running task blocks itself.
ECC1  DE 9D         467  BLOCKME         LDX     CURTASK
ECC3                468  ; BLOCKX: block task X.
ECC3  A6 85         469  BLOCKX          LDAA    $85,X
ECC5  8A 80         470                  ORAA    #$80
ECC7  A7 85         471                  STAA    $85,X
ECC9  6F 8D         472                  CLR     $8D,X
ECCB  39            473                  RTS
ECCC                474  ; SLEEP: block the caller for A ticks (yield).
ECCC  DE 9D         475  SLEEP           LDX     CURTASK
ECCE  E6 85         476                  LDAB    $85,X
ECD0  CA 81         477                  ORAB    #$81
ECD2  A7 8D         478                  STAA    $8D,X
ECD4  26 02         479                  BNE     LECD8
ECD6  C4 7F         480                  ANDB    #$7F
ECD8  E7 85         481  LECD8           STAB    $85,X
ECDA  32            482                  PULA
ECDB  33            483                  PULB
```

```
ECDC  DF 9F          484                 STX     $9F
ECDE  78 00 A0       485                 FCB     $78,$00,$A0 ; ASL $00A0
ECE1  DE 9F          486                 LDX     $9F
ECE3  A7 75          487                 STAA    $75,X
ECE5  E7 76          488                 STAB    $76,X
ECE7  39             489                 RTS
ECE8                 490  ; GETRX: fetch the next received character (blocks
ECE8                 491  ; if none).
ECE8  DE 9D          492  GETRX          LDX     CURTASK
ECEA  E6 85          493                 LDAB    $85,X
ECEC  CA 83          494                 ORAB    #$83
ECEE  20 E8          495                 BRA     LECD8
```

These are the four primitives a task uses:

- `WAKE`: given X = a task number, clear its blocked bit. Anyone can wake any task. `WAKE` is how task 2, when it finds work, wakes task 3, and so on.
- `BLOCKME`: set the *current* task's blocked bit and clear its sleep counter. The task will not run again until someone wakes it. (`BLOCKX` blocks task X.)
- `SLEEP`: block the caller for A ticks. Look at what it does. It sets the blocked bit and bit 0 of the status (meaning "this task has a resume address"), stores A in the sleep counter (with A = 0 the blocked bit is cleared again at once: the task merely yields and is called on the next pass), then **`PULA, PULB`**: it removes the caller's return address from the stack and files it in the task's resume slot (`STAA $75,X`, `STAB $76,X`) and **returns**. That `RTS` does not go back to the caller; it goes to the next address on the stack, which is the scheduler's own return address from the `JSR 0,X` that started the task. The task has *finished* from the scheduler's point of view, and its next invocation will start at the instruction after `JSR SLEEP`.
- `GETRX`: the same, but it sets bits 1 and 0 too (`ORAB #$83`): the task is waiting for a received character. When one arrives and the task runs again, it resumes after `JSR GETRX` with the character in A.

This is a **stackless coroutine**. Each task has no stack of its own: it borrows the machine stack for as long as it runs, and when it yields it forgets everything except one address and the few variables the firmware keeps for it in zero page. It follows that **`SLEEP` and `GETRX` may be called only from the task's top level**: if the task has `JSR`ed into a subroutine and that subroutine calls `SLEEP`, the `PULA`/`PULB` would remove the subroutine's return address, and the task's own would be left on the stack. Every one of the firmware's tasks respects this.

Building one: T21TASKS

The trick is short enough to write in a plug-in. T21TASKS has two tasks: A runs every third pass and B every fifth, each bumping a counter and printing it. The scheduler visits both, counting sleeps down, and stops after 60 passes.

```
E100  0F             80  START          SEI
E101  8E 03 FF       81                 LDS     #$03FF
E104  BD E2 E2       82                 JSR     INITCRT
E107  BD E2 17       83                 JSR     CLRSCR
E10A  7F 01 20       84                 CLR     SLEEPTB
E10D  7F 01 21       85                 CLR     SLEEPTB+1
E110  7F 01 28       86                 CLR     CNTA
E113  7F 01 29       87                 CLR     CNTB
E116  7F 01 27       88                 CLR     PASSES
```

```
E119  86 04           89                LDAA    #4
E11B  CE E1 C4        90                LDX     #LBLA
E11E  BD E1 B9        91                JSR     LABEL
E121  86 06           92                LDAA    #6
E123  CE E1 D0        93                LDX     #LBLB
E126  BD E1 B9        94                JSR     LABEL
E129  86 08           95                LDAA    #8
E12B  CE E1 DC        96                LDX     #LBLP
E12E  BD E1 B9        97                JSR     LABEL
E131                  98  ; each task starts at its
E131  CE E1 E3        99                LDX     #TASKA
E134  FF 01 22       100                STX     CONTTB    ; first instruction
E137  CE E1 FD       101                LDX     #TASKB
E13A  FF 01 24       102                STX     CONTTB+2
E13D                 103  ; ---- the scheduler: one pass visits every task ----
E13D  7F 01 26       104  SCHED          CLR     CUR
E140  CE 01 20       105  SC_LP          LDX     #SLEEPTB
E143  B6 01 26       106                LDAA    CUR
E146  BD E2 CD       107                JSR     XPLUSA    ; X = &SLEEPTB[CUR]
E149  A6 00          108                LDAA    0,X
E14B                 109  ; zero: the task is due
E14B  27 04          110                BEQ     SC_RUN
E14D                 111  ; else count its sleep down
E14D  6A 00          112                DEC     0,X
E14F  20 0E          113                BRA     SC_NEXT
E151  CE 01 22       114  SC_RUN         LDX     #CONTTB
E154  B6 01 26       115                LDAA    CUR
E157                 116  ; two bytes per entry
E157  48             117                ASLA
E158  BD E2 CD       118                JSR     XPLUSA    ; X = &CONTTB[CUR]
E15B                 119  ; X = resume address
E15B  EE 00          120                LDX     0,X
E15D                 121  ; run until it yields
E15D  AD 00          122                JSR     0,X
E15F  7C 01 26       123  SC_NEXT        INC     CUR
E162  B6 01 26       124                LDAA    CUR
E165  81 02          125                CMPA    #2
E167  26 D7          126                BNE     SC_LP
E169  7C 01 27       127                INC     PASSES
E16C  86 08          128                LDAA    #8
E16E  B7 01 00       129                STAA    ROWARG
E171  86 10          130                LDAA    #16
E173  B7 01 01       131                STAA    COLARG
E176  B6 01 27       132                LDAA    PASSES
E179  BD E2 B4       133                JSR     PUTHEX
E17C  CE 01 2C       134                LDX     #300      ; pace the passes
E17F  09             135  PACE          DEX
E180  26 FD          136                BNE     PACE
E182  B6 01 27       137                LDAA    PASSES
E185  81 3C          138                CMPA    #60
E187  26 B4          139                BNE     SCHED
E189  20 FE          140  FOREVER       BRA     FOREVER
```

`SCHED` is the loop. For each task it looks up the task's sleep counter with `XPLUSA` (indexing into the table, Chapter 16); if it is non-zero it decrements it and moves on; otherwise it loads the task's **resume address** and calls it. A task begins at its first instruction: the table is initialised with `TASKA` and `TASKB`.

```
E18B                 141  ; ---- YIELD: A = passes to sleep.  Called from a
E18B                 142  ; task:
E18B                 143  ; pops the caller's return address, files it as the
E18B                 144  ; resume address, and RETURNS TO THE SCHEDULER. ----
E18B  B7 01 2C       145  YIELD           STAA    TICKS
E18E                 146  ; return address, high
E18E  32             147                  PULA
E18F                 148  ; return address, low
E18F  33             149                  PULB
E190  B7 01 2A       150                  STAA    HI
E193  F7 01 2B       151                  STAB    LO
E196  CE 01 22       152                  LDX     #CONTTB
E199  B6 01 26       153                  LDAA    CUR
E19C  48             154                  ASLA
E19D  BD E2 CD       155                  JSR     XPLUSA    ; X = &CONTTB[CUR]
E1A0  B6 01 2A       156                  LDAA    HI
E1A3  A7 00          157                  STAA    0,X
E1A5  B6 01 2B       158                  LDAA    LO
E1A8  A7 01          159                  STAA    1,X
E1AA  CE 01 20       160                  LDX     #SLEEPTB
E1AD  B6 01 26       161                  LDAA    CUR
E1B0  BD E2 CD       162                  JSR     XPLUSA    ; X = &SLEEPTB[CUR]
E1B3  B6 01 2C       163                  LDAA    TICKS
E1B6  A7 00          164                  STAA    0,X
E1B8                 165  ; back to the scheduler
E1B8  39             166                  RTS
E1B9                 167  ; LABEL - print the string at X on row A, column 2
E1B9  B7 01 00       168  LABEL           STAA    ROWARG
E1BC  86 02          169                  LDAA    #2
E1BE  B7 01 01       170                  STAA    COLARG
E1C1  7E E2 92       171                  JMP     PUTSTR
```

YIELD is SLEEP in our words: it takes the number of passes in A, pops the return address, files it in the table for the current task, stores the sleep count, and returns to the scheduler. The tasks:

```
E1E3                 175  ; ---- the tasks ----
E1E3  7C 01 28       176  TASKA           INC     CNTA
E1E6  86 04          177                  LDAA    #4
E1E8  B7 01 00       178                  STAA    ROWARG
E1EB  86 10          179                  LDAA    #16
E1ED  B7 01 01       180                  STAA    COLARG
E1F0  B6 01 28       181                  LDAA    CNTA
E1F3  BD E2 B4       182                  JSR     PUTHEX
E1F6  86 02          183                  LDAA    #2        ; sleep two passes
E1F8  BD E1 8B       184                  JSR     YIELD
E1FB  20 E6          185                  BRA     TASKA     ; resumes here
E1FD  7C 01 29       186  TASKB           INC     CNTB
E200  86 06          187                  LDAA    #6
E202  B7 01 00       188                  STAA    ROWARG
E205  86 10          189                  LDAA    #16
E207  B7 01 01       190                  STAA    COLARG
E20A  B6 01 29       191                  LDAA    CNTB
E20D  BD E2 B4       192                  JSR     PUTHEX
E210  86 04          193                  LDAA    #4        ; sleep four passes
E212  BD E1 8B       194                  JSR     YIELD
E215  20 E6          195                  BRA     TASKB
```

(Each task ends with `LDAA #n / JSR YIELD / BRA TASK`: the `BRA` is where it resumes.) The result:

```
  TASK A RUNS   14

  TASK B RUNS   0C

  PASSES        3C
```

A ran 20 times (`$14`) and B ran 12 (`$0C`) in 60 passes, exactly what a Python model of the same rules computes: task A, sleeping two passes between runs, runs on passes 0, 3, 6, … and so 20 times in 60; task B, sleeping four, on 0, 5, 10, … and so 12 times. The test checks both numbers against the model.

Three details to take from the exercise. First, the scheduler *itself* knows nothing about what the tasks do: all it keeps is a sleep count and an address. Second, the tasks are written as **ordinary loops that yield** rather than as state machines; the 6800 stack, borrowed for the duration of the run, does the work a state variable would do. Third, the program is cooperative: if task A looped forever without yielding, task B would never run.

The interrupt

Interrupts are the only thing that can enter the firmware while a task is running (or while the scheduler idles). There is one source of interest: the two serial interfaces.

```
ED83                 571  ; IRQ service: ACIA first, then the printer ACIA.
ED83  F6 7A 00       572  IRQ             LDAB    ACIA
ED86  2A 02          573                  BPL     LED8A
ED88  8D 09          574                  BSR     ACIAIRQ
ED8A  F6 7C 00       575  LED8A           LDAB    PACIA
ED8D  2A 03          576                  BPL     LED92
ED8F  BD FF AB       577                  JSR     LFFAB
ED92  3B             578  LED92           RTI
ED93  C5 01          579  ACIAIRQ         BITB    #$01
ED95  27 58          580                  BEQ     LEDEF
ED97  B6 7A 01       581                  LDAA    ACIAD
ED9A  84 7F          582                  ANDA    #$7F
ED9C  C4 70          583                  ANDB    #$70
ED9E  27 09          584                  BEQ     LEDA9
EDA0  C5 10          585                  BITB    #$10
EDA2  27 03          586                  BEQ     LEDA7
EDA4  D7 A8          587                  STAB    $A8
EDA6  39             588                  RTS
EDA7  8A 80          589  LEDA7           ORAA    #$80
EDA9  7D 00 A8       590  LEDA9           FCB     $7D,$00,$A8 ; TST $00A8
EDAC  27 04          591                  BEQ     LEDB2
EDAE  7F 00 A8       592                  FCB     $7F,$00,$A8 ; CLR $00A8
EDB1  39             593                  RTS
EDB2  7D 00 A6       594  LEDB2           FCB     $7D,$00,$A6 ; TST $00A6
EDB5  2B 31          595                  BMI     LEDE8
EDB7  DE A3          596                  LDX     $A3
EDB9  A7 00          597                  STAA    $00,X
EDBB  08             598                  INX
EDBC  8C 03 80       599                  CPX     #$0380
EDBF  26 03          600                  BNE     LEDC4
EDC1  CE 01 10       601                  LDX     #$0110
```

```
EDC4  DF A3          602  LEDC4           STX     $A3
EDC6  4D             603                  TSTA
EDC7  2A 0D          604                  BPL     LEDD6
EDC9  E7 00          605                  STAB    $00,X
EDCB  08             606                  INX
EDCC  8C 03 80       607                  CPX     #$0380
EDCF  26 03          608                  BNE     LEDD4
EDD1  CE 01 10       609                  LDX     #$0110
EDD4  DF A3          610  LEDD4           STX     $A3
EDD6  08             611  LEDD6           INX
EDD7  8C 03 80       612                  CPX     #$0380
EDDA  26 03          613                  BNE     LEDDF
EDDC  CE 01 10       614                  LDX     #$0110
EDDF  9C A1          615  LEDDF           CPX     $A1
EDE1  26 04          616                  BNE     LEDE7
EDE3  86 80          617                  LDAA    #$80
EDE5  97 A6          618                  STAA    $A6
EDE7  39             619  LEDE7           RTS
EDE8  86 80          620  LEDE8           LDAA    #$80
EDEA  97 A7          621                  STAA    $A7
EDEC  7E F0 DA       622                  JMP     BELL
EDEF  C5 02          623  LEDEF           BITB    #$02
EDF1  27 0B          624                  BEQ     LEDFE
EDF3  96 BF          625                  LDAA    ACIACTL
EDF5  84 60          626                  ANDA    #$60
EDF7  81 20          627                  CMPA    #$20
EDF9  26 03          628                  BNE     LEDFE
EDFB  7E FD 85       629                  JMP     LFD85
EDFE  C5 04          630  LEDFE           BITB    #$04
EE00  27 03          631                  BEQ     LEE05
EE02  B6 7A 01       632                  LDAA    ACIAD
```

The IRQ vector, at `$FFF8`, points to `IRQ`. The handler reads the **host** interface's status (`LDAB ACIA`); in the 6850 the status register's bit 7 is "this interface is requesting an interrupt", so `BPL` skips ahead if the host did not interrupt. Then it does the same for the **printer** interface (`PACIA`), calling `LFFAB` if that one did. Finally `RTI`.

`ACIAIRQ`, the host's service, looks at the status in B:

- bit 0 (received data full): read the data (`LDAA ACIAD`, and `ANDA #$7F`: the firmware works in 7 bits) and check the error bits. `ANDB #$70` keeps bits 6-4 (parity, overrun, framing). If none is set the character goes on as it is. If bit 4 (framing error) is set, the status is recorded in `$A8` and the character is thrown away, and the next character is also dropped when `$A8` is set (`TST $A8`, `CLR $A8`, `RTS`). For the other two errors the character is queued with **bit 7 set**, which marks it as suspect, and a **second byte, the status with its error bits, is queued after it**.
- The character is stored in a **ring buffer**: `$0110-$037F`, with a write pointer in `$A3-$A4` and a read pointer in `$A1-$A2`. The pointer is advanced by `INX`, and when it reaches `$0380` it wraps to `$0110` (`CPX #$0380 / BNE / LDX #$0110`). When the write pointer catches up with the read pointer the buffer is full, and the flag `$A6` is set to `$80`. A character that arrives while the flag is set is dropped: the handler sets `$A7` and **rings the bell** (`JMP BELL`). The terminal tells you it is losing characters.
- bit 1 (transmitter ready): if the interface's control byte says transmit interrupts are enabled (`ACIACTL` bits 6-5 equal `01`), jump to `LFD85`, the transmit service (Chapter 30). Otherwise nothing.

- bit 2 (carrier lost, DCD): read the data register, which clears the condition.

The **consumer** of the ring buffer is task 2, `LED05`. Task 2 is always ready: on every pass of the scheduler it runs, compares the two pointers and returns at once if they are equal. If they differ it removes one character and hands it, in the argument byte, to the next stage, waking task 3, which passes it to the display or escape-sequence code. The details are the subject of Chapter 28.

What this means for a plug-in

- **Interrupts and the plug-in.** Every program in this book begins with `SEI`, so none of this interrupt activity runs while a game plays. Characters sent from the host during a game are lost: the receiver overruns, and nobody reads it.
- **A plug-in that wants the serial port** must either leave interrupts enabled and let the firmware's handler fill the ring buffer, or handle the ACIA itself, as Chapter 12 showed.
- **A plug-in that wants the firmware's tasks to keep running** must not stop the scheduler. It would need to be a task, or to be called from one. That is the cooperative route of Chapter 25, and it has not been tried here.

Exercises

1. In T21TASKS, change the sleeps to 1 and 3. Predict the counts after 60 passes with the model, then run it. (Answer: A runs on passes 0, 2, 4, ... so 30 times; B on 0, 4, 8, ... so 15.)
2. Add a third task that prints the pass counter in its own column. What must change in `SCHED`, `CONTTB` and `SLEEPTB`?
3. What happens in T21TASKS if a task calls `YIELD` from a subroutine it has called, and why? Make the mistake on purpose and describe the failure.
4. The firmware scheduler clears `CURTASK` before every idle section, so every scan starts at task 0. What is the consequence for the order in which the tasks run, and which task therefore has the highest priority?
5. The ring buffer holds `$0380 - $0110 = 624` bytes. How long does it take to fill at 9,600 baud (ten bits per character) if no task removes anything?

Chapter 28

From key to screen: the keyboard task and the escape sequences

Chapter 9 read the keyboard the way a game does: the program writes a scan code to `$7800`, looks at bit 7, and decides for itself what the key means. The firmware does the same electrical thing, but it is built for a different job. It must turn a pressed key into *a character the terminal then acts on*: print it, send it to the host, or, for the editing keys, perform a function. This chapter follows one keystroke from the matrix to the screen. On the way it reads the table that decides what each key means, the auto-repeat, and the escape-sequence dispatcher that gives the ADM-31 most of its personality.

Everything here is read from the 129867 listing, and the parts that can be run were run, in the emulator, on the real ROM images of both firmware sets. The 129827 differs in addresses only (Appendix H), and every experiment below gives the same screen on both.

The road a keystroke takes

Three tasks of the scheduler (Chapter 27) and one shared routine are involved:

```
 matrix --> task 0 KBSCAN --> task 1 KEYHAND --+--> LEFF6 (character handler)
 (PIA)      finds a key,      decides: print,  |      printable  -> display RAM
            translates it,    send, or run a   |      control     -> CTLTAB
            repeats it        function         |      ESC + char  -> ESCTAB
                                               |
 host --> ACIA --> IRQ --> ring buffer --> task 2 --> task 3 --+
```

Characters that come **from the host** arrive at the same `LEFF6` by another road: the interrupt handler puts them in a ring buffer, task 2 takes them out, and task 3 hands them to `LEFF6`. So there is one routine that decides what a character *does*, and two ways of feeding it: the keyboard and the host. When the mode bits say “local”, as they do in the emulator, a typed key is processed exactly as if the host had sent it.

The zero-page variables of the keyboard path:

Address	Name	Meaning
`$1D`	`KBLOCK`	keyboard lock: non-zero means keystrokes are discarded

Address	Name	Meaning
$41		mode bits: bit 6 = send typed codes to the host; bit 7 = also process them locally; copied from the top two bits of switch bank 1 ($7E00) at start-up
$45	ESCCHR	the escape character ($1B at start-up)
$57		bit 7 selects what the numeric-pad codes do (see below)
$A9		scan index, counted *down* from $57 to $00
$AA		the translated character being delivered
$AB		copy of PIA port B ($7802) taken when the key was found: SHIFT, CTRL, alpha lock
$AC		repeat counter (zero means “do not repeat”)
$AD		repeat rate, counted down as the key stays pressed

Task 0: finding a key

```
EE06                    634  ; Keyboard scan task.
EE06  C6 04             635  KBSCAN          LDAB    #$04
EE08  96 A9             636  LEE08           LDAA    $A9
EE0A  B7 78 00          637                  STAA    PIA1A
EE0D  B6 78 00          638                  LDAA    PIA1A
EE10  2B 0B             639                  BMI     LEE1D
EE12  26 02             640                  BNE     LEE16
EE14  86 58             641                  LDAA    #$58
EE16  4A                642  LEE16           DECA
EE17  97 A9             643                  STAA    $A9
EE19  5A                644                  DECB
EE1A  26 EC             645                  BNE     LEE08
EE1C  39                646                  RTS
```

KBSCAN is called by the scheduler every pass, and each call examines **four** scan codes (LDAB #$04, DECB, BNE). This is the same scan as Chapter 9’s, with two differences worth noting.

First, the scan index is not recomputed: it lives in $A9, and each step stores it to port A (STAA PIA1A) and reads port A back. Because bits 0-6 of port A are *outputs*, what you read back in those bits is what you wrote (Chapter 9), and bit 7 is the key input. BMI therefore means “a key is down at this index”. If not, BNE LEE16 is taken for any non-zero index; when the index is **zero**, the code loads $58 first, and then LEE16 does DECA and stores the result. The index thus runs $57, $56, ... $01, $00, $57, ...: 88 values, one per key position, the same 88 entries as the table that follows. Because the index is remembered in $A9, the next call to KBSCAN carries on from where the last one stopped: a complete sweep of the keyboard takes 88 / 4 = 22 passes of the scheduler.

Second, when a key *is* found the routine does not return: it handles the key itself, using SLEEP for its delays, so the other tasks run while the key is held.

Translating: KBDTAB and the three modifiers

```
EE1D  F6 78 02          647  LEE1D           LDAB    PIA1B
EE20  D7 AB             648                  STAB    $AB
EE22  58                649                  ASLB
EE23  49                650                  ROLA
```

```
EE24  CE EA 1A       651                LDX     #KBDTAB
EE27  BD EF AD       652                JSR     XPLUSA
EE2A  C6 0F          653                LDAB    #$0F
EE2C  D7 AD          654                STAB    $AD
EE2E  A6 00          655                LDAA    $00,X
EE30  2B 1E          656                BMI     LEE50
EE32  D6 AB          657                LDAB    $AB
EE34  C5 01          658                BITB    #$01
EE36  27 0A          659                BEQ     LEE42
EE38  81 61          660                CMPA    #$61
EE3A  2D 06          661                BLT     LEE42
EE3C  81 7A          662                CMPA    #$7A
EE3E  2E 02          663                BGT     LEE42
EE40  80 20          664                SUBA    #$20
EE42  C5 40          665  LEE42         BITB    #$40
EE44  27 0A          666                BEQ     LEE50
EE46  81 40          667                CMPA    #$40
EE48  2D 06          668                BLT     LEE50
EE4A  81 7F          669                CMPA    #$7F
EE4C  27 02          670                BEQ     LEE50
EE4E  84 1F          671                ANDA    #$1F
```

Follow the code from `LEE1D`, where A holds the port-A value (bit 7 set, index in bits 0-6):

1. `LDAB PIA1B` reads port B and `STAB $AB` keeps it for later. Bit 7 of port B is the SHIFT key, bit 6 is CTRL, and bit 0 reflects the alpha-lock latch (the firmware's own flip-flop; Chapter 11 listed the port).
2. `ASLB` pushes bit 7 of B (SHIFT) into the carry flag; `ROLA` then rotates the carry into the **bottom** of A, and the top bit of A (the key-down bit) falls off the other end. The result is **A = index × 2 + SHIFT**.
3. `LDX #KBDTAB`, `JSR XPLUSA` adds A to X. The table is therefore **two bytes per key position, unshifted first, then shifted**, and the code fetches the right one with `LDAA $00,X`.

Scan	Unshifted	Shifted	Scan	Unshifted	Shifted
$00	$D1 special	$D1 special	$2C	$84 special	$84 special
$01	$D7 special	$D7 special	$2D	$85 special	$85 special
$02	$C5 special	$C5 special	$2E	$86 special	$86 special
$03	$D2 special	$D2 special	$2F	$09 HT	$09 HT
$04	$D4 special	$F4 special	$30	$61 a	$41 A
$05	$D9 special	$F9 special	$31	$73 s	$53 S
$06	$32 2	$22 "	$32	$64 d	$44 D
$07	$1B ESC	$1B ESC	$33	$66 f	$46 F
$08	$7F DEL	$7F DEL	$34	$67 g	$47 G
$09	$9E special	$9F special	$35	$68 h	$48 H
$0A	$D0 special	$F0 special	$36	$6A j	$4A J
$0B	$B4 special	$B6 special	$37	$6B k	$4B K
$0C	$92 special	$93 special	$38	$6C l	$4C L
$0D	$31 1	$21 !	$39	$3B ;	$2B +
$0E	$71 q	$51 Q	$3A	$3A :	$2A *
$0F	$98 special	$98 special	$3B	$40 @	$60 `
$10	$33 3	$23 #	$3C	$0A LF	$0A LF
$11	$34 4	$24 $	$3D	$82 special	$82 special

Scan	Unshifted	Shifted	Scan	Unshifted	Shifted
$12	$35 5	$25 %	$3E	$83 special	$83 special
$13	$36 6	$26 &	$3F	$81 special	$81 special
$14	$37 7	$27 '	$40	$90 special	$91 special
$15	$38 8	$28 (	$41	$7A z	$5A Z
$16	$39 9	$29)	$42	$78 x	$58 X
$17	$30 0	$5F _	$43	$63 c	$43 C
$18	$2D -	$3D =	$44	$76 v	$56 V
$19	$5E ^	$7E ~	$45	$62 b	$42 B
$1A	$5C \	$7C |	$46	$6E n	$4E N
$1B	$09 HT	$C9 special	$47	$6D m	$4D M
$1C	$87 special	$87 special	$48	$2C ,	$3C <
$1D	$88 special	$88 special	$49	$2E .	$3E >
$1E	$89 special	$89 special	$4A	$2F /	$3F ?
$1F	$2D -	$2D -	$4B	$20 SP	$20 SP
$20	$77 w	$57 W	$4C	$1F US	$CA special
$21	$65 e	$45 E	$4D	$30 0	$30 0
$22	$72 r	$52 R	$4E	$2E .	$2E .
$23	$74 t	$54 T	$4F	$0D CR	$0D CR
$24	$79 y	$59 Y	$50	$9D special	$94 special
$25	$75 u	$55 U	$51	$1E RS	$1E RS
$26	$69 i	$49 I	$52	$0A LF	$0A LF
$27	$6F o	$4F O	$53	$0B VT	$0B VT
$28	$70 p	$50 P	$54	$08 BS	$08 BS
$29	$5B [	$7B {	$55	$0C FF	$0C FF
$2A	$5D]	$7D }	$56	$E3 special	$00 NUL
$2B	$0D CR	$0D CR	$57	$2C ,	$2C ,

The full table is Appendix D. Four things stand out.

- **Printable keys** have ordinary ASCII codes in both columns: scan `$30` gives `$61` (`a`) or `$41` (`A`).
- **Codes with bit 7 set** are not characters. They are *function codes*, and the remainder of this chapter explains them. The table puts them on the editing block, the keypad, and a few keys such as the alpha-lock and break keys.
- **The apostrophe is SHIFT-7**, scan `$14` shifted (`$27`). The key that carries `@` gives `` ` `` **($60) when shifted** (scan `$3B`). Chapter 22 depends on that: the firmware launches a plug-in on **ESC** `` ` `` (the grave accent), not on ESC `'`.
- The table has **88 entries**; `$EA1A + 176 = $EACA`, the first byte of the next table, which is the ESC table. There is no room for a 89th key.

After the table lookup the code applies the modifiers, in this order, **but only to character codes** (the first test, `BMI LEE50`, skips both when bit 7 is set):

- **Alpha lock** (`BITB #$01` on the saved port B): if the code is between `$61` and `$7A` (lowercase `a`-`z`), subtract `$20`. Only letters are affected, so alpha lock does not turn `1` into `!`. (This is the ADM-31's alpha lock: it capitalises letters and nothing else.)
- **CTRL** (`BITB #$40`): if the code is `$40` or more, and not `$7F`, `ANDA #$1F`. This is the standard "control" mapping: CTRL-`A` is `$01`, CTRL-`G` is `$07` (the bell), and so on. It applies to any

code in `$40-$7E`, including punctuation: CTRL-`[` would give `$1B`, though here a dedicated ESC key exists.

Correction to older notes. Where earlier notes about this terminal disagree, this book follows the code. The firmware reads port B (`$7802`) like this: **SHIFT is bit 7, CTRL is bit 6, and the alpha-lock latch is bit 0.** The `ASLB/ROLA` pair above uses bit 7 as SHIFT and `BITB #$01` tests alpha lock; there is no other reading of this code.

Finally, the keypad rule at `LEE50`. For a code in `$80-$8F`, **if bit 7 of `$57` is clear, the code is converted into `$30-$3F`** (`ANDA #$7F`, `ADDA #$30`): the keypad types digits. `$87,$88,$89` (scan `$1C,$1D,$1E`) become 7, 8, 9; `$84,$85,$86` become 4, 5, 6; `$81,$82,$83` become 1, 2, 3. Codes from `$90` up are not converted. The emulator confirms it on the real firmware: pressing scan codes `1C 1D 1E 2C 2D 2E 3F 3D 3E` in turn types **789456123** (the three keypad rows, top to bottom):

```
789456123lete
```

(The line starts at the top left because the firmware's own idle message, `Test Complete`, is on the screen and the typing overwrites its first characters. That is why the line ends in `lete`.) What sets bit 7 of `$57`, so that the keypad would send function codes instead, was not found: no instruction in the listing stores to `$57`, and the start-up table leaves it zero, so on the shipped table the pad always types digits. That is as far as this book can say.

Delivering the character, and auto-repeat

```
EE50  4D            672  LEE50          TSTA
EE51  2A 0D         673                 BPL     LEE60
EE53  81 90         674                 CMPA    #$90
EE55  2C 09         675                 BGE     LEE60
EE57  7D 00 57      676                 FCB     $7D,$00,$57 ; TST $0057
EE5A  2B 04         677                 BMI     LEE60
EE5C  84 7F         678                 ANDA    #$7F
EE5E  8B 30         679                 ADDA    #$30
EE60  97 AA         680  LEE60          STAA    $AA
EE62  86 7D         681                 LDAA    #$7D
EE64  97 AC         682                 STAA    $AC
EE66  CE 00 01      683  LEE66          LDX     #$0001
EE69  96 AA         684                 LDAA    $AA
EE6B  BD EC B8      685                 JSR     LECB8
EE6E  86 0A         686                 LDAA    #$0A
EE70  BD EC CC      687                 JSR     SLEEP
EE73  96 AC         688                 LDAA    $AC
EE75  27 1C         689                 BEQ     LEE93
EE77  86 02         690  LEE77          LDAA    #$02
EE79  BD EC CC      691                 JSR     SLEEP
EE7C  B6 78 00      692                 LDAA    PIA1A
EE7F  2A 1C         693                 BPL     LEE9D
EE81  7A 00 AC      694                 FCB     $7A,$00,$AC ; DEC $00AC
EE84  26 F1         695                 BNE     LEE77
EE86  96 AD         696                 LDAA    $AD
EE88  81 03         697                 CMPA    #$03
EE8A  2F 03         698                 BLE     LEE8F
EE8C  7A 00 AD      699                 FCB     $7A,$00,$AD ; DEC $00AD
EE8F  97 AC         700  LEE8F          STAA    $AC
EE91  20 D3         701                 BRA     LEE66
EE93  86 04         702  LEE93          LDAA    #$04
```

```
EE95  BD EC CC       703                 JSR     SLEEP
EE98  B6 78 00       704                 LDAA    PIA1A
EE9B  2B F6          705                 BMI     LEE93
```

The character is stored in `$AA` and the repeat counter `$AC` is set to `$7D`. The loop at `LEE66` then:

1. loads X with `1` and A with the character, and calls `LECB8`. `LECB8` is a "send": **`STAA $95,X`** puts A into task 1's argument slot, and it falls into the `WAKE` of Chapter 27, which makes task 1 ready;
2. sleeps 10 ticks, so that `KEYHAND` (task 1), which the scheduler reaches next in the same pass, has time to run and, if the key calls for a function, to clear `$AC`;
3. looks at `$AC`. **If it is zero**, `LEE93`: wait, four ticks at a time, until the key is released, with no repeating. **If not**, `LEE77`: sleep 2 ticks, read port A (still holding the key's own scan index, because `$A9` was not decremented), and exit through `LEE9D` as soon as bit 7 shows the key has been released; otherwise decrement `$AC` and, when it reaches zero, take the rate from `$AD` (which was set to `$0F` when the key was found), count it down by one (to a floor of 3), put it in `$AC` and branch back to `LEE66`: **send the character again**.

That gives the following repeat schedule, in ticks of the scheduler:

Character sent	Ticks since the one before
the key's first character	0
first repeat	10 + 2 × 125 = **260**
second repeat	10 + 2 × 15 = **40**
third repeat	10 + 2 × 14 = 38
… each interval two ticks shorter …	
fourteenth and every later repeat	10 + 2 × 3 = **16**

The numbers `260`, `40` and `16` follow directly from the constants in the listing (`$7D`, `$0F` and the floor of 3). **This book has not measured the length of a tick** (Chapter 11), so it cannot convert them into seconds, and the emulator's key model releases the key too soon to show any repeat at all. What the code does show is the shape: a long first delay, then a repeat that speeds up in thirteen steps, from 40 ticks to 16, and then holds.

Two further details. **Several of the handlers clear `$AC`**: `LEF42`, `LEF61`, `SENDBRK` and `LEFA5` each contain `CLR $00AC` (written out in the listing as the bytes `7F 00 AC`, because the dialect has no direct-mode `CLR`; Chapter 3). The effect is the `LEE93` branch above: the key is delivered once and does not repeat. The alpha-lock toggle and the break are examples. A function that does not clear `$AC` repeats like a letter.

Task 1: KEYHAND

```
EEA1  36             708  KEYHAND        PSHA
EEA2  BD EC C1       709                 JSR     BLOCKME
EEA5  32             710                 PULA
EEA6  16             711                 TAB
EEA7  C4 F8          712                 ANDB    #$F8
EEA9  C1 98          713                 CMPB    #$98
EEAB  27 0C          714                 BEQ     LEEB9
EEAD  D6 1D          715                 LDAB    KBLOCK
EEAF  27 08          716                 BEQ     LEEB9
EEB1  81 90          717                 CMPA    #$90
```

```
EEB3  26 03          718                 BNE     LEEB8
EEB5  7F 00 1D       719                 FCB     $7F,$00,$1D ; CLR $001D
EEB8  39             720  LEEB8          RTS
```

KEYHAND is called with the character in A. It begins by PSHA / BLOCKME / PULA: block itself (so it runs once per character) without losing A. Then:

- **TAB; ANDB #$F8; CMPB #$98; BEQ LEEB9**: the codes $98-$9F bypass the keyboard lock entirely. They are the ones that must work even when the keyboard is locked: $98 leads (through the ESC table) to LEF61, which toggles the alpha-lock latch, and $9E to SENDBRK.
- **LDAB KBLOCK; BEQ LEEB9**: if the keyboard is not locked, go on. Otherwise only $90 gets through, and only to **clear the lock** (CLR $001D), and everything else is silently discarded (LEEB8: RTS).

The emulator confirms the lock. ESC # sets KBLOCK (Appendix C: LF0AA). Then scan $30 (the a key) is lost, scan $40 (the code $90) unlocks, and scan $31 types an s:

```
sest Complete
```

and without the unlock key the s is lost too: the screen stays at Test Complete. (Whether $90 is the key labelled “unlock” on the cap is not verified here: the table calls scan $40 unshifted $90, the shifted one $91.)

Past the lock, LEEB9 sorts the code by its top bit:

- **Bit 7 clear (a character)**: LEED8. The only special case is ESC itself. If the code is $1B and SHIFT is down ($AB bit 7, tested by LDAB $AB / BPL), then A is replaced by **ESCCHR** (LDAA ESCCHR): holding SHIFT while pressing ESC sends whatever the terminal is currently using as its escape character, which is $1B unless the host has changed it. Then the mode bits decide. **$41 bit 6 clear**: process locally, LEF08. **Bit 6 set**: send the code to the host (JSR LFCF5, the transmit queue of Chapter 30), and then, **only if bit 7 is also set**, fall into LEEF0, which processes the character locally as well, so that a terminal set to half duplex echoes what it sends. In that second branch control codes get a special treatment: CR, $04 (EOT) and $03 (ETX) also wake task 7 (TXDELAY) so that the transmit queue is flushed and the line turnaround happens.
- **Bit 7 set (a function code)**: CLRB / JSR LECF0 clears the task’s two flag bits; ANDA #$7F removes the top bit; and then three cases.
  - BITA #$60 is zero for the codes $80-$9F (after the mask, $00-$1F), which go to ESCDISP.
  - The codes $A0-$FF ($20-$7F after the mask) are **escape sequences in disguise**. If the mode bits $41 AND $C0 are exactly $40 (full duplex: send, do not process) the firmware transmits ESCCHR followed by the 7-bit code (JSR LFCF5 twice, the second as a JMP) and is done: the host sees ESC plus a letter. In every other mode it does not send anything, and enters ESCDISP directly, as if the host had sent the same two characters.

That last point is the heart of the design, and it is worth saying plainly: **most of the ADM-31’s editing keys are not hardware features. Each one is an entry in an ESC table.** The key at scan $02 has the code $C5, which is ESC E; INSLINE, the ESC-table entry for E, inserts a line. The key at scan $03 is $D2, ESC R, DELLINE, which deletes one. The key at scan $04 is $D4, ESC T, CLREOL; its shifted twin $F4 is ESC t. When a host program wants to insert a line it sends ESC E, and when the operator presses the key the terminal behaves as if the host had.

The shared handler: LEFF6

```
EF08  BD EF F6       761  LEF08           JSR     LEFF6
EF0B  DE 9D          762  LEF0B           LDX     CURTASK
EF0D  E6 85          763                  LDAB    $85,X
EF0F  C5 01          764                  BITB    #$01
EF11  26 01          765                  BNE     LEF14
EF13  39             766                  RTS
EF14  DF 9F          767  LEF14           STX     $9F
EF16  78 00 A0       768                  FCB     $78,$00,$A0 ; ASL $00A0
EF19  DE 9F          769                  LDX     $9F
EF1B  EE 75          770                  LDX     $75,X
EF1D  DF AF          771                  STX     $AF
EF1F  BD EC E8       772                  JSR     GETRX
EF22  D6 41          773                  LDAB    $41
EF24  C1 C0          774                  CMPB    #$C0
EF26  26 03          775                  BNE     LEF2B
EF28  BD FC F5       776                  JSR     LFCF5
EF2B  DE AF          777  LEF2B           LDX     $AF
EF2D  AD 00          778                  JSR     $00,X
EF2F  20 DA          779                  BRA     LEF0B
```

`LEF08` is the call to `LEFF6` for the local path, followed by a loop at `LEF0B` that deserves a paragraph of its own, because it is the cleverest piece of the whole path.

An escape sequence is not one character. `ESC = row column` is three, and the handler for it needs two more characters after the one that selected it. A handler gets them by calling `GETRX`, the scheduler's "wait for the next character", which (Chapter 27) saves the caller's return address and ends the task's run. For the characters coming from the host that is all that is needed: the next character wakes task 3, which resumes in the handler. For the keyboard it is not enough, because `KEYHAND` has called `LEFF6` with `JSR`. So when the handler calls `GETRX`, the return address that `GETRX` finds on top of the stack is the one that belongs to the `JSR LEFF6`, `LEF0B`, and `GETRX`'s own `RTS` goes there. `LEF0B` examines the task's status, sees bit 0 set (`GETRX` set it), and knows that **a handler is waiting for more characters**. `LEF14` then saves the handler's resume address in `$AF`, and calls `GETRX` itself to block `KEYHAND`; and when the next character arrives, `LEF2D` is `JSR 0,X`: the **handler's continuation is called as an ordinary subroutine with the new character in A**, and the loop goes round again until the handler does not block any more. In other words, `KEYHAND` is a small trampoline that feeds keystrokes to a coroutine that was written for a different caller. That is why `ESC = % # z` typed at the keyboard works exactly as it does from the host:

```
xyst Complete




   z
```

The two typed letters `x` and `y` overwrite the first two characters of the idle message; `ESC = % #` then puts the cursor on row 5 (`%` is `$25`, minus `$20` gives 5) column 3 (`#` is `$23`), and the `z` lands there.

Now the routine that does the work for every character, from whichever source:

```
EFF6  BD EC FE       870  LEFF6           JSR     LECFE
EFF9  C5 20          871                  BITB    #$20
EFFB  26 09          872                  BNE     LF006
```

```
EFFD  91 45          873                 CMPA    ESCCHR
EFFF  26 20          874                 BNE     LF021
F001  C6 20          875                 LDAB    #$20
F003  7E EC F0       876                 JMP     LECF0
F006  5F             877  LF006          CLRB
F007  BD EC F0       878                 JSR     LECF0
F00A  81 58          879                 CMPA    #$58
F00C  27 04          880                 BEQ     LF012
F00E  81 75          881                 CMPA    #$75
F010  26 03          882                 BNE     LF015
F012  7E F0 A6       883  LF012          JMP     LF0A6
F015  85 60          884  LF015          BITA    #$60
F017  27 21          885                 BEQ     LF03A
F019  F6 00 1A       886                 FCB     $F6,$00,$1A ; LDAB $001A
F01C  26 33          887                 BNE     LF051
F01E  7E EF 31       888                 JMP     ESCDISP
F021  85 60          889  LF021          BITA    #$60
F023  27 0A          890                 BEQ     LF02F
F025  81 7F          891                 CMPA    #$7F
F027  26 11          892                 BNE     LF03A
F029  F6 00 1A       893                 FCB     $F6,$00,$1A ; LDAB $001A
F02C  26 0C          894                 BNE     LF03A
F02E  39             895                 RTS
F02F  F6 00 1A       896  LF02F          FCB     $F6,$00,$1A ; LDAB $001A
F032  26 06          897                 BNE     LF03A
F034  CE EB AA       898                 LDX     #CTLTAB
F037  7E EF 3A       899                 JMP     LEF3A
F03A  7D 00 1E       900  LF03A          FCB     $7D,$00,$1E ; TST $001E
F03D  27 05          901                 BEQ     LF044
F03F  36             902                 PSHA
F040  BD F8 27       903                 JSR     LF827
F043  32             904                 PULA
```

Read it in the order the character meets it.

- `JSR LECFE` returns in B the task's two flag bits (`ANDB #$30`). **Bit 5 set means "the previous character was ESC".**
- **Not after ESC** (`LF006` is the after-ESC case; the fall-through is the normal one): the character is compared with `ESCCHR`. If it *is* the escape character, set the flag (`LDAB #$20; JMP LECF0`) and return: the next character will be an escape-sequence code.
- **Not ESC**: `LF021`. A code with `BITA #$60` non-zero is printable (`$20`-`$7F`). DEL (`$7F`) is ignored unless `MONMODE` (`$1A`) is non-zero. Everything else goes to `LF03A`, the display routine. A code below `$20` is a control code: unless `MONMODE` is set, `LDX #CTLTAB / JMP LEF3A` dispatches it through the control table.
- **After ESC** (`LF006`): clear the flag; `X` (`$58`) and `u` (`$75`) end monitor mode on the spot (`LF0A6` clears `MONMODE`); a code below `$20` goes to the display routine; otherwise, if `MONMODE` is set, the ESC itself is displayed as well (`LF051`), else **`ESCDISP`**.

So the character handler is a three-way switch: *print*, *control table*, *ESC table*, with a fourth mode, **monitor mode** (`ESC U` sets `$1A`, `ESC X` or `ESC u` clear it), which shows control and escape codes as characters instead of acting on them. In that mode control and escape codes are displayed instead of acted on. This book has not run monitor mode, only read it.

Printing a character: LF03A

`LF03A` is the end of the line for a printable character, and it is short enough to read entire:

- `TST $1E`: if that flag is set (`ESC q` sets `$1E` to 2 and `ESC r` clears it) `JSR LF827` first shifts the rest of the line one place to the right, to make room: it is the insert-character mode.
- `ORAA ATTRMSK`: **OR the character with the attribute mask** (`$0019`): `ESC )` sets it to `$80` (`LF074`), `ESC (` clears it to zero (`LF079`). This is the firmware's whole attribute mechanism: it sets or clears the high bit of every character stored. (This book has not characterised what the display does with that bit.)
- `LDX CURADR` loads the cursor's address (`$13`-`$14`), then **`STAA HALT; NOP; STAA $00,X`**: strobe, NOP, store, the screen-access rule of Chapter 6, in the firmware's own words.
- `JMP LF272`: move the cursor one place right. `LF272` is also the control-table entry for `$0C` (FF), as the next section shows.

ESCDISP and the tables

```
EF31                 780  ; ESC dispatch: code - $10, doubled, indexes ESCTAB.
EF31  84 7F          781  ESCDISP         ANDA    #$7F
EF33  80 10          782                  SUBA    #$10
EF35  2B 0B          783                  BMI     LEF42
EF37  CE EA CA       784                  LDX     #ESCTAB
EF3A  48             785  LEF3A           ASLA
EF3B  BD EF AD       786                  JSR     XPLUSA
EF3E  EE 00          787                  LDX     $00,X
EF40  6E 00          788                  JMP     $00,X
EF42  8B 4F          789  LEF42           ADDA    #$4F
EF44  7F 00 AC       790                  FCB     $7F,$00,$AC ; CLR $00AC
EF47  7E F9 D9       791                  JMP     LF9D9
```

`ESCDISP` is eight instructions. A holds the code after the ESC (or the function code of a key):

1. `ANDA #$7F`: seven bits only.
2. `SUBA #$10`: the table starts at `$10`. A code below `$10` gives a negative result and `BMI LEF42` is taken: those are the keypad function codes `$80`-`$8F`. `LEF42` adds `$4F`, clears `$AC` (no repeat) and jumps to `LF9D9`, which stores the complemented request number in `$C5`, wakes **task 4**, and blocks tasks 0 and 2 (the keyboard scan and the receive consumer) until task 4 has dealt with the request. Chapter 30 reads the transmit side.
3. `LDX #ESCTAB`, then `ASLA`: the code is doubled, because every table entry is a two-byte address.
4. `JSR XPLUSA`: X = X + A: the address of the entry.
5. `LDX $00,X`: X = the handler's address, and `JMP $00,X`: **go there**. The handler returns with `RTS` to whoever called the character handler, and the character is finished.

An example, worked by hand and then checked in the emulator: `ESC +` clears the screen. The `+` is `$2B`. Subtract `$10`: `$1B`. Double: `$36`. Add to `ESCTAB` (`$EACA`): `$EB00`. The word at `$EB00` is `$F740`, which is `CLRSCR`. And the emulator, on both ROM sets, shows an empty screen after `ab ESC +`.

The ESC table, then, is one two-byte address for each code from `$10` to `$7F` (112 entries, 224 bytes) and the control table is one for each code from `$00` to `$1F` (32 entries, 64 bytes). A code with no function points at `ESCNOP`, a lone `RTS` at `$EBEA`. The whole tables are printed in Appendix C; this is the part of the ESC table that the next two chapters read:

Code	Char	Handler	What this book has read
`$2B`	+	`CLRSCR` (`$F740`)	clears the screen (Chapter 29)

Code	Char	Handler	What this book has read
$3D	=	CURPOS ($F5AA)	cursor addressing: row, column (Chapter 29)
$45	E	INSLINE ($F874)	insert line (Chapter 29)
$52	R	DELLINE ($F912)	delete line (Chapter 29)
$54	T	CLREOL ($F7D5)	clear to end of line (Chapter 29)
$26	&	LF060	sets PROTECT = $80 (protect on)
$27	'	LF070	clears PROTECT (protect off)
$28 / $29	( )	LF079 / LF074	clear / set the attribute mask ATTRMSK
$23 / $22	# "	LF0AA / LF0AF	lock / unlock the keyboard
$55 / $58	U X	LF0A1 / LF0A6	monitor mode on / off
$42 / $43	B C	LF07D / LF083	clear / set bit 6 of $41
$18	(none)	LEF61	toggle the alpha-lock latch (PIA1B bit 0)
$1E	(none)	SENDBRK	transmit a break
$60	`	**ESCGRAVE** ($F22C)	**launch plug-in A**

Three comments. **First: the ' row.** ESC ' is $1B $27: the keycap apostrophe. What firmware does with it is CLR $0018, turn protect **off**. It does *not* launch anything. The plug-in launch is ESC, then the grave accent ` ($60), which on this keyboard is **SHIFT of the key that carries @** (scan $3B; Appendix D). Chapters 22 and 26 use it. A reader who types ESC ' and wonders why nothing starts has pressed the wrong key.

Second, ESCGRAVE is the whole of the launch:

```
F22C                1140  ; ESC ` : if socket A holds "TJ" jump to its start
F22C                1141  ; ($E03C).
F22C  FE E0 00      1142  ESCGRAVE        LDX     $E000
F22F  8C 54 4A      1143                  CPX     #$544A
F232  26 F7         1144                  BNE     LF22B
F234  7E E0 3C      1145                  JMP     $E03C
```

LDX $E000, CPX #$544A (the two letters TJ), BNE out if they differ, and JMP $E03C, the entry point of the frame of Chapter 5, which is what every program in this book reads in the first thirty bytes. No switch is involved: **the $7D00 bit that the boot probe honours is not tested here.**

Third, the entries for codes $10-$1F are not sequences a host types: they are reached through the function codes $90-$9F that keys produce ($98 gives entry $18, LEF61; $9E gives $1E, SENDBRK). Many handlers further down the table (LF0F3, LF458 and others) were not read for this book, and their rows are marked as unread in Appendix C. The point of the table is that it is **data**: a plug-in that wants a firmware function can call the handler's address directly (Chapter 23), without sending anything.

The control table

```
EBAA                 355  ; Control-code table: one handler per code $00-$1F.
EBAA  EB EA EB EA    356  CTLTAB          FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EBB2  EB EA F9 D5    357                  FDB     ESCNOP,LF9D5,ESCNOP,BELL
EBBA  F2 8C F3 AA    358                  FDB     LF28C,LF3AA,LF27D,LF293
EBC2  F2 72 F2 A1    359                  FDB     LF272,LF2A1,ESCNOP,ESCNOP
EBCA  EB EA EB EA    360                  FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EBD2  EB EA EB EA    361                  FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EBDA  EB EA EB EA    362                  FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EBE2  EB EA EB EA    363                  FDB     ESCNOP,ESCNOP,LF259,LF29A
```

Ten of the 32 control codes have a handler; the other 22 point to ESCNOP:

Code	Name	Handler	Effect
$05	ENQ	LF9D5	queues request number 10 for task 4 (Chapter 30)
$07	BEL	BELL	pulses bit 3 of PIA1CRB (Chapter 10)
$08	BS	LF28C	cursor left; from column 0 it goes to column 79 and then up a row (LF36F)
$09	HT	LF3AA	tab (the tab logic was not traced)
$0A	LF	LF27D	cursor down (LF317); scrolls at the bottom
$0B	VT	LF293	cursor up (LF388)
$0C	FF	LF272	cursor right (LF2F0)
$0D	CR	LF2A1	cursor to column 0 (LF2E5)
$1E	RS	LF259	cursor home (LF2CF: column 0, top row)
$1F	US	LF29A	column 0 and down a row (LF313)

The names in the second column are the standard ASCII names of the codes, not labels the firmware knows. The effects are read from the code of the handlers named in the last column, which Chapter 29 takes apart; none of them was run on its own.

What it means for a program

- **A plug-in that runs with SEI and its own scan loop** (every program in this book) bypasses all of the above. It reads the matrix as Chapter 9 did, gets its own scan codes, and nothing in KBDTAB applies unless it copies the table. The table is at $EA1A in the 129867 and at $EB1B in the 129827 (Appendix H), and a program that wants *the firmware's* idea of what a key means can index it just as KBSCAN does: `index × 2 + SHIFT`.
- **A plug-in that wants the editing keys** has two options: implement them (a line insert is a few dozen instructions, and Chapter 29 shows how the firmware does it), or call the firmware's INSLINE and DELLINE directly. Both need the firmware's variables (CURADR, CRTBASE, ROW, COL) to be right first.
- **A plug-in that wants to *be* a keyboard handler** would be a task that calls GETRX, that is, a cooperative program in the sense of Chapter 25. That is possible in principle (the firmware's own tasks are the existence proof) but it is not what any working plug-in in this collection does, and this book has not built one.
- **Both firmware sets behave alike.** The five experiments in this chapter give identical screens on 129867 and 129827 (the test suite runs both). The routine addresses differ (Appendix H) but the logic does not.

Exercises

1. In KBDTAB, which key position produces $27, and which produces $60? (Answer: scan $14 shifted, the apostrophe; scan $3B shifted, the grave accent. The first is just a character; the second is the second half of the launch sequence.)
2. A key is held with the alpha lock on, and the key is [(scan $29). What character is delivered? (Answer: [, $5B. The alpha-lock rule touches $61-$7A only.)
3. CTRL-A is typed. What does the translation produce, and what does LEFF6 do with it? (Answer: `$41 AND $1F = $01`, a control code; CTLTAB entry 1 is ESCNOP: nothing happens.)

4. Work out by hand the table address and the handler for `ESC` = and check it against the listing. (Answer: `$3D - $10 = $2D`; doubled `$5A`; `$EACA + $5A = $EB24`; the word there is `$F5AA`, `CURPOS`.)
5. Why does `KBSCAN` examine only four scan codes per call, instead of all 88 at once? What would happen to the other tasks if it did them all, and what would happen to a quick keystroke if it did *one*?
6. On a real terminal, time how long a held key takes to produce its first repeat. Divide by 260 ticks to get the tick period. (This book has not measured it; if you do, the whole of Chapter 11's timing caveat can be removed.)

Chapter 29

The display routines: cursor, scroll, clear and edit

Chapter 8 built five small screen routines of our own, and explained the hardware scroll in a few paragraphs. This chapter reads the firmware's version of the same subject at full size: how it turns a row and a column into an address, how it moves the cursor, how it clears the screen and a line, how it inserts and deletes lines, how it protects fields, and how it keeps two display pages. Every routine here is something that an escape sequence or a control code runs (Chapter 28), so every claim can be tested by typing, and was.

All the experiments use the same method as Chapter 28. The real firmware ROMs boot in the emulator with no plug-in; a sequence of keystrokes is typed; the emulator dumps its RAM; and the test reads the zero-page variables and the display memory. The expected result of each experiment is computed in Python from the plain description of what the function should do, and the test compares the two. They give identical results on the 129867 and on the 129827 (the zero-page *addresses* differ, so for the 129827 the tests read only the screen memory and the CRT controller's registers, which are at the same place on both).

The state

The display code keeps its state in a few zero-page bytes of the 129867:

Address	Name	Meaning
`$11-$12`	`CRTBASE`	start of the page being displayed: `$8000` or `$8800`
`$13-$14`	`CURADR`	the cursor's address in display memory
`$15`	`FIRSTLN`	the memory row that is on the bottom screen line (`0`-`23`)
`$16`	`ROW`	cursor row, as a **memory row** (see below)
`$17`	`COL`	cursor column, `0`-`79`
`$18`	`PROTECT`	`$80` when protect mode is on
`$19`	`ATTRMSK`	OR-ed into each character stored: `$80` or `0`
`$42`	`SPLIT`	the two-page flag (bit 7) and the page it records (bit 2)
`$B1-$B4`		`BLITCPY`'s destination and source pointers
`$B7-$BA`, `$BB-$BC`		scratch pointers and counts, reused by every routine below

The cursor is held three times over: in `ROW` and `COL`, in `CURADR`, and in the CRT controller's own cursor registers (`$7F0D` for the row, `$7F0C` for the column), which the firmware writes whenever the cursor moves so that the hardware cursor follows. The three are kept in step by always changing them together.

ROW is a memory row, not a screen row. This matters, and Chapter 8 prepared the ground for it: because scrolling only changes `FIRSTLN`, a given memory row appears at a different place on the screen after each scroll. The screen row `r` (counting from 0 at the top) is memory row `(FIRSTLN + 1 + r) mod 24`. At power-up `FIRSTLN` is 23, so screen row `r` is memory row `r`. The experiments below show both cases.

Row and column to address

```
F5EA                1616  ; Row to screen address (row * 80 by shifts).
F5EA  16            1617  ROWADDR         TAB
F5EB  48            1618                  ASLA
F5EC  48            1619                  ASLA
F5ED  1B            1620                  ABA
F5EE  5F            1621                  CLRB
F5EF  48            1622                  ASLA
F5F0  59            1623                  ROLB
F5F1  48            1624                  ASLA
F5F2  59            1625                  ROLB
F5F3  48            1626                  ASLA
F5F4  59            1627                  ROLB
F5F5  48            1628                  ASLA
F5F6  59            1629                  ROLB
F5F7  39            1630                  RTS
F5F8                1631  ; Cursor address from ROW and COL.
F5F8  96 16         1632  CURADDR         LDAA    ROW
F5FA  8D EE         1633                  BSR     ROWADDR
F5FC  9B 17         1634                  ADDA    COL
F5FE  D9 11         1635                  ADCB    CRTBASE
F600  D7 13         1636                  STAB    CURADR
F602  97 14         1637                  STAA    $14
F604  DE 13         1638                  LDX     CURADR
F606  39            1639                  RTS
```

`ROWADDR` takes a memory row in A and returns row × 80 in the register pair B:A (**B is the high byte, A the low byte**). It does it without a multiplication instruction, in two steps. `TAB; ASLA; ASLA; ABA` computes 4 × row + row = **5 × row** in A. `CLRB`, then four rounds of `ASLA; ROLB`, shift the 16-bit pair B:A left four places (each `ASLA` pushes A's top bit into the carry and `ROLB` takes it into B): multiply by 16. 5 × 16 = 80. Since 5 × 23 = 115 fits in a byte before the shifts begin, the first stage never overflows.

Compare it with the `CALCADR` of Chapter 8, which reaches the same 80 × row as 16 × row + 64 × row, through a pair of RAM bytes. The firmware's way, 5 × row and then × 16 entirely in registers, is much smaller: `ROWADDR` is 14 bytes and `CURADDR` 15, together 29, against the 90 bytes of `CALCADR`. Part of the difference is that `CALCADR` takes its arguments from RAM and the firmware's routine is handed the row in A.

`CURADDR` loads `ROW`, calls `ROWADDR`, and adds the column and the page base: `ADDA COL` adds the column to the low byte (A); `ADCB CRTBASE` adds the high byte of the base to the high byte (B) *with the carry*, so the page base is added to the high half only. That is only correct because the

low byte of CRTBASE is always zero ($8000 or $8800), a fact the code relies on without checking. It stores the high byte in CURADR ($13) and the low byte in $14 and returns the address in X.

Worked example, from the scroll experiment below: ROW = 6, COL = 0. ROWADDR gives 6 × 80 = 480 = $01E0; adding $8000 gives $81E0. The emulator's RAM dump has CURADR = $81E0. The test confirms the relation CURADR = CRTBASE + 80 × ROW + COL after an arbitrary cursor move.

Cursor addressing: ESC =

```
F5AA                  1585  ; ESC = row col : cursor address.
F5AA  BD EC E8        1586  CURPOS           JSR     GETRX
F5AD  8D 33           1587                  BSR     LF5E2
F5AF  81 18           1588                  CMPA    #$18
F5B1  2D 02           1589                  BLT     LF5B5
F5B3  86 17           1590                  LDAA    #$17
F5B5  9B 15           1591  LF5B5           ADDA    FIRSTLN
F5B7  4C              1592                  INCA
F5B8  81 18           1593                  CMPA    #$18
F5BA  2D 02           1594                  BLT     LF5BE
F5BC  80 18           1595                  SUBA    #$18
F5BE  B7 7F 0D        1596  LF5BE           STAA    VTACD
F5C1  97 16           1597                  STAA    ROW
F5C3  BD EC E8        1598                  JSR     GETRX
F5C6  8D 1A           1599                  BSR     LF5E2
F5C8  81 50           1600                  CMPA    #$50
F5CA  2D 02           1601                  BLT     LF5CE
F5CC  86 4F           1602                  LDAA    #$4F
F5CE  B7 7F 0C        1603  LF5CE           STAA    VTACC
F5D1  97 17           1604                  STAA    COL
F5D3  BD F5 F8        1605                  JSR     CURADDR
F5D6  96 53           1606                  LDAA    $53
F5D8  85 02           1607                  BITA    #$02
F5DA  27 0D           1608                  BEQ     LF5E9
F5DC  BD F2 C0        1609                  JSR     LF2C0
F5DF  7E F3 FA        1610                  JMP     LF3FA
F5E2  84 7F           1611  LF5E2           ANDA    #$7F
F5E4  80 20           1612                  SUBA    #$20
F5E6  2C 01           1613                  BGE     LF5E9
F5E8  4F              1614                  CLRA
F5E9  39              1615  LF5E9           RTS
```

CURPOS is the handler for ESC = row column. The two coordinate characters are each offset by $20 (so that they are printable), and the routine reads them with GETRX (Chapter 27), one at a time. LF5E2 is the shared conversion: ANDA #$7F, SUBA #$20, and a negative result is forced to zero.

The row:

1. CMPA #$18 / BLT / LDAA #$17: a row of 24 or more is **clamped to 23**.
2. ADDA FIRSTLN; INCA: add FIRSTLN + 1, because screen row 0 is memory row FIRSTLN + 1.
3. CMPA #$18 / BLT / SUBA #$18: reduce modulo 24.
4. The result goes to VTACD (the cursor-row register) and to ROW.

The column: clamp to 79 (CMPA #$50 / BLT / LDAA #$4F), store in VTACC and COL. Then JSR CURADDR computes the address.

That gives a precise answer to what the firmware does with out-of-range coordinates: it **does not wrap, it clamps**. The test types ESC = followed by two DEL characters ($7F, which minus $20 is $5F, far outside the screen). The cursor goes to row 23, column 79, and CURADR = $877F, the last visible cell:

Quantity	Observed	Expected
ROW, COL	23, 79	23, 79
CURADR	$877F	$8000 + 23 × 80 + 79 = $877F
VTAC $7F0D, $7F0C	23, 79	the same

Moving the cursor

The control codes of Chapter 28 reach short routines, each of which changes COL or ROW and the matching VTAC register:

```
F2F0  96 17         1232  LF2F0           LDAA    COL
F2F2  81 4F         1233                  CMPA    #$4F
F2F4  27 0F         1234                  BEQ     LF305
F2F6  7F 00 B5      1235                  FCB     $7F,$00,$B5 ; CLR $00B5
F2F9  4C            1236                  INCA
F2FA  B7 7F 0C      1237                  STAA    VTACC
F2FD  97 17         1238                  STAA    COL
F2FF  DE 13         1239                  LDX     CURADR
F301  08            1240                  INX
F302  DF 13         1241                  STX     CURADR
F304  39            1242                  RTS
F305  D6 18         1243  LF305           LDAB    PROTECT
F307  2B 0A         1244                  BMI     LF313
F309  D6 53         1245                  LDAB    $53
F30B  C5 10         1246                  BITB    #$10
F30D  27 04         1247                  BEQ     LF313
F30F  7C 00 B5      1248                  FCB     $7C,$00,$B5 ; INC $00B5
F312  39            1249                  RTS
F313  8D D0         1250  LF313           BSR     LF2E5
F315  20 00         1251                  BRA     LF317
```

- **Right** (LF2F0, FF): if COL is 79, go to LF305. There, if protect mode is on or bit 4 of the option byte $53 is clear, it calls LF313, a carriage return followed by a line feed: **the cursor wraps to column 0 of the next row**. (If protect is off and the option bit is set, the cursor stays in column 79 and $B5 counts the overflow.) The shipped start-up table has bit 4 clear, so the default is to wrap. Otherwise COL is incremented, CURADR is incremented, and the register is written. The test sees the wrap: with the cursor at row 0, column 79, typing x and then y puts x at column 79 and y at row 1, column 0.
- **Left** (LF36F): decrement COL; from column 0 go to column 79 and then **up** a row.
- **Home** (LF2CF, RS): COL = 0, and ROW is set to FIRSTLN - 23 modulo 24, which is the same as FIRSTLN + 1: the top screen row. This is the same adjustment CURPOS makes for row 0.
- **Carriage return** (LF2E5): COL = 0.
- **Up** (LF388): decrement ROW, from 0 to 23. (In two-page mode, moving up onto the bottom row switches the page; that branch was read, not run.)
- **Down** (LF317): the interesting one.

Down, and the scroll

```
F317  96 16         1252  LF317           LDAA    ROW
F319  91 15         1253                  CMPA    FIRSTLN
F31B  27 0A         1254                  BEQ     LF327
F31D  81 17         1255                  CMPA    #$17
F31F  26 02         1256                  BNE     LF323
F321  86 FF         1257                  LDAA    #$FF
F323  4C            1258  LF323           INCA
F324  7E F2 DD      1259  LF324           JMP     LF2DD
F327  7D 00 42      1260  LF327           FCB     $7D,$00,$42 ; TST $0042
F32A  26 36         1261                  BNE     LF362
F32C  7D 00 18      1262                  FCB     $7D,$00,$18 ; TST $0018
F32F  27 03         1263                  BEQ     LF334
F331  7E F2 D7      1264  LF331           JMP     LF2D7
F334  D6 55         1265  LF334           LDAB    $55
F336  C5 08         1266                  BITB    #$08
F338  26 F7         1267                  BNE     LF331
F33A  4C            1268                  INCA
F33B  81 18         1269                  CMPA    #$18
F33D  2D 01         1270                  BLT     LF340
F33F  4F            1271                  CLRA
F340  97 15         1272  LF340           STAA    FIRSTLN
F342  B7 7F 06      1273                  STAA    VTAC6
F345  BD F2 DD      1274                  JSR     LF2DD
F348  90 17         1275                  SUBA    COL
F34A  C2 00         1276                  SBCB    #$00
F34C  D7 B7         1277                  STAB    $B7
F34E  97 B8         1278                  STAA    $B8
F350  DE B7         1279                  LDX     $B7
F352  C6 50         1280                  LDAB    #$50
F354  4F            1281                  CLRA
F355  B7 70 00      1282  LF355           STAA    HALT
F358  01            1283                  NOP
F359  A7 00         1284                  STAA    $00,X
F35B  08            1285                  INX
F35C  5A            1286                  DECB
F35D  26 F6         1287                  BNE     LF355
F35F  DE 13         1288                  LDX     CURADR
F361  39            1289                  RTS
F362  BD F6 5E      1290  LF362           JSR     LF65E
F365  96 15         1291                  LDAA    FIRSTLN
F367  80 17         1292                  SUBA    #$17
F369  2A 02         1293                  BPL     LF36D
F36B  8B 18         1294                  ADDA    #$18
F36D  20 B5         1295  LF36D           BRA     LF324
```

LF317 is the code that moves the cursor down one row. A holds ROW. It compares with FIRSTLN:

- **ROW is not the bottom row** (CMPA FIRSTLN not equal): A is incremented, wrapping 23 to 0 (CMPA #$17 / BNE / LDAA #$FF / INCA), and LF2DD stores it in VTACD and ROW and recomputes the address.
- **ROW is the bottom row** (LF327): this is a scroll, unless protect mode is on or bit 3 of the option byte $55 is set, in which case the cursor wraps to the top screen row without scrolling (LF331, read but not run); the two-page case (LF362) is read only as well. Otherwise INCA and a wrap at 24 give the new FIRSTLN; it is stored in FIRSTLN **and in VTAC6**, the CRT controller's scroll register (STAA VTAC6); the cursor row follows; and then the loop at

`LF355` clears the 80 cells of the new bottom row, **to zero** (`CLRA`). Nothing else moves: not one byte of text is copied.

(The test of the register: the emulator stores the last value written to every CRT register, and the value of register 6 in its dump equals `FIRSTLN` at the end of each experiment below. What the *hardware* does with register 6 is the inference of Chapter 8, not something this book observed; the emulator draws no pictures.)

Scrolling observed

Thirty lines `L01` to `L30` are typed, each followed by CR and LF, after clearing the screen. The screen has 24 lines, so the terminal must scroll seven times. With `FIRSTLN` starting at 23, the first 24 lines fill memory rows 0 to 23 (`L01` in row 0); the line feed that follows the 24th line, and every later one, is a scroll. At the end:

Variable	Value	Why
`FIRSTLN`	6	23 + 7 scrolls = 30, modulo 24
`ROW`	6	the cursor is on the bottom line
VTAC register 6	6	the firmware wrote it at each scroll
memory row 6	all zero	the new bottom line, blanked by the scroll
memory rows 0-5	`L25`-`L30`	the lines typed after the first scroll wrapped into the old top rows
memory rows 7-23	`L08`-`L24`	untouched since they were typed

The displayed screen, with row 0 at memory row `FIRSTLN + 1 = 7`, is therefore the 24 lines `L08` to `L30` and the blank bottom line; the test builds that view from the memory dump and compares it with the list computed in Python:

```
L08
L09
L10
L11
L12
L13
L14
L15
L16
L17
L18
L19
L20
L21
L22
L23
L24
L25
L26
L27
L28
L29
L30
```

(The last, blank line is the cursor line.) The oldest seven lines, `L01` to `L07`, did not move off the screen: they were *overwritten*, one row at a time, by the lines that took their place.

Clearing

```
F76A  36           1829  LF76A     PSHA
F76B  7F 00 18     1830            FCB     $7F,$00,$18 ; CLR $0018
F76E  7F 00 19     1831  LF76E     FCB     $7F,$00,$19 ; CLR $0019
F771  BD EF BC     1832            JSR     LEFBC
F774  96 42        1833            LDAA    SPLIT
F776  27 12        1834            BEQ     LF78A
F778  84 04        1835            ANDA    #$04
F77A  F6 78 02     1836            LDAB    PIA1B
F77D  C4 04        1837            ANDB    #$04
F77F  11           1838            CBA
F780  26 03        1839            BNE     LF785
F782  BD F6 5E     1840            JSR     LF65E
F785  8D 09        1841  LF785     BSR     LF790
F787  BD F6 5E     1842            JSR     LF65E
F78A  8D 04        1843  LF78A     BSR     LF790
F78C  31           1844            INS
F78D  7E F2 79     1845            JMP     LF279
F790  BD F2 CF     1846  LF790     JSR     LF2CF
F793  D6 11        1847  LF793     LDAB    CRTBASE
F795  CB 07        1848            ADDB    #$07
F797  86 7F        1849            LDAA    #$7F
F799  D7 B7        1850            STAB    $B7
F79B  97 B8        1851            STAA    $B8
F79D  96 15        1852            LDAA    FIRSTLN
F79F  BD F5 EA     1853            JSR     ROWADDR
F7A2  8B 4F        1854            ADDA    #$4F
F7A4  D9 11        1855            ADCB    CRTBASE
F7A6  D7 B9        1856            STAB    $B9
F7A8  97 BA        1857            STAA    $BA
F7AA  30           1858            TSX
F7AB  E6 02        1859            LDAB    $02,X
F7AD  DE 13        1860            LDX     CURADR
F7AF  96 18        1861  LF7AF     LDAA    PROTECT
F7B1  2A 08        1862            BPL     LF7BB
F7B3  B7 70 00     1863            STAA    HALT
F7B6  01           1864            NOP
F7B7  A5 00        1865            BITA    $00,X
F7B9  26 06        1866            BNE     LF7C1
F7BB  B7 70 00     1867  LF7BB     STAA    HALT
F7BE  01           1868            NOP
F7BF  E7 00        1869            STAB    $00,X
F7C1  9C B9        1870  LF7C1     CPX     $B9
F7C3  27 0D        1871            BEQ     LF7D2
F7C5  9C B7        1872            CPX     $B7
F7C7  27 03        1873            BEQ     LF7CC
F7C9  08           1874            INX
F7CA  20 E3        1875            BRA     LF7AF
F7CC  DE 11        1876  LF7CC     LDX     CRTBASE
F7CE  DF 13        1877            STX     CURADR
F7D0  20 DD        1878            BRA     LF7AF
F7D2  7E F5 F8     1879  LF7D2     JMP     CURADDR
```

CLRSCR (ESC +) is a short routine in front of the work. `LDAB $55; BITB #$04` tests an option bit that came from the start-up table; with the shipped table it is clear, so the routine takes the LF73B path: `BSR LF74C` chooses the fill character (a space, `$20`, or the byte at `BLANK`, `$4D`, if bit 6

of $55 is set), pushes it, and falls into LF76E: CLR ATTRMSK (cleared areas get no attribute), JSR LEFBC (which, if the running task is task 1, clears the repeat counter $AC of Chapter 28, so that a held key does not repeat the clear), a branch that only matters in two-page mode (see below), and then:

LF790: JSR LF2CF moves the cursor **home**, and LF793 does the fill. It sets up two limits:

- $B7 = the last cell of the visible page: CRTBASE + 7 for the high byte and $7F for the low byte, $877F for page one;
- $B9 = the last cell of the **bottom screen line** (the memory row FIRSTLN, 80 cells long): ROWADDR(FIRSTLN) + 79 + CRTBASE.

The loop LF7AF stores the fill character at CURADR, then compares X with $B9 (done: jump to LF7D2, which recalculates CURADR), and with $B7 (the end of the page: wrap X to CRTBASE and carry on), and otherwise increments X. So a clear fills **a circular range**, from the cursor to the end of the bottom line, going round the end of memory if the cursor is in the middle of the ring. From the home position this is the whole screen, 1,920 cells, because home is the cell right after the bottom line in ring order. The fill character is taken from the stack with TSX; LDAB $02,X, a way of reading a pushed byte without popping it.

The loop's first lines are the **protect test**:

```
LF7AF   LDAA    PROTECT         ; A = $80 if protect mode, else 0
        BPL     LF7BB           ; protect off: store unconditionally
        STAA    HALT
        NOP
        BITA    $00,X           ; Z=1 if the cell's bit 7 is clear
        BNE     LF7C1           ; cell protected: leave it alone
LF7BB   STAA    HALT
        NOP
        STAB    $00,X           ; store the fill character
```

The test BITA $00,X uses the value $80 in A as a mask for the top bit of the cell. In protect mode, **a cell whose bit 7 is set is skipped**. The next section uses it.

CLREOL (ESC T) is the same idea on one line:

```
F7D5                1880  ; Clear to end of line.
F7D5  86 20         1881  CLREOL          LDAA    #$20
F7D7  D6 55         1882                  LDAB    $55
F7D9  C5 80         1883                  BITB    #$80
F7DB  27 02         1884                  BEQ     LF7DF
F7DD  96 4D         1885                  LDAA    BLANK
F7DF  BD EF BC      1886  LF7DF           JSR     LEFBC
F7E2  D6 55         1887  LF7E2           LDAB    $55
F7E4  C5 01         1888                  BITB    #$01
F7E6  27 03         1889                  BEQ     LF7EB
F7E8  7F 00 19      1890                  FCB     $7F,$00,$19 ; CLR $0019
F7EB  9A 19         1891  LF7EB           ORAA    ATTRMSK
F7ED  C6 50         1892                  LDAB    #$50
F7EF  D0 17         1893                  SUBB    COL
F7F1  D7 BA         1894                  STAB    $BA
F7F3  DE 13         1895                  LDX     CURADR
F7F5  D6 18         1896  LF7F5           LDAB    PROTECT
F7F7  2A 08         1897                  BPL     LF801
F7F9  B7 70 00      1898                  STAA    HALT
F7FC  01            1899                  NOP
F7FD  E6 00         1900                  LDAB    $00,X
```

```
F7FF  2B 0C         1901                BMI     LF80D
F801  B7 70 00      1902  LF801         STAA    HALT
F804  01            1903                NOP
F805  A7 00         1904                STAA    $00,X
F807  08            1905                INX
F808  7A 00 BA      1906                FCB     $7A,$00,$BA ; DEC $00BA
F80B  26 E8         1907                BNE     LF7F5
F80D  DE 13         1908  LF80D         LDX     CURADR
F80F  B7 70 00      1909                STAA    HALT
F812  01            1910                NOP
F813  A6 00         1911                LDAA    $00,X
F815  95 18         1912                BITA    PROTECT
F817  27 03         1913                BEQ     LF81C
F819  BD F3 FA      1914                JSR     LF3FA
F81C  39            1915  LF81C         RTS
```

It picks the fill character, clears ATTRMSK if an option bit says to, computes the number of cells left on the line (#$50 - COL) into $BA and loops from CURADR. With protect mode on, it behaves differently from CLRSCR: at the first protected cell, BMI LF80D **ends the loop**, instead of skipping it. So CLRSCR leaves protected cells and carries on; CLREOL stops. A test shows it:

Protected fields

The attribute mechanism is simple enough to describe in one sentence: **bit 7 of a display cell marks it protected**, ESC) makes every character typed afterwards carry that bit (ATTRMSK = $80, which LF03A ORs into each stored character, Chapter 28), ESC (stops that, and ESC & turns PROTECT on so that the clear and insert routines respect the marks. ESC ' turns PROTECT off again.

The experiment types ab, then ESC), XY, ESC (, cd, goes home with ESC = and two spaces, switches protect on with ESC &, and sends ESC T:

```
XYcd
```

The RAM dump shows what is stored: the first six cells of the screen are 20 20 D8 D9 63 64. The ab was cleared to spaces ($20), X and Y are $58 and $59 **with bit 7 set** ($D8, $D9), and ESC T stopped at the X, so cd ($63, $64) survives, though it is not protected. The emulator's screen dump drops bit 7, which is why the characters show as normal letters. With ESC + instead of ESC T, the same text becomes XY: CLRSCR clears ab and cd and steps over XY.

How the *display* draws a protected character is not something this book has seen. The attribute is the cell's top bit, the firmware's routines treat it as "protected", and what the CRT does with it (a different brightness, or reverse video) is a property of the hardware that no listing here shows.

Inserting and deleting lines

ESC E and ESC R, and so also the two editing keys that send them (Chapter 28), are the largest routines in the display code. Both work **by copying whole rows**.

```
F874                1961  ; ESC E : insert line.
F874  7D 00 18      1962  INSLINE         FCB     $7D,$00,$18 ; TST $0018
F877  27 01         1963                BEQ     LF87A
F879  39            1964                RTS
F87A  7F 00 19      1965  LF87A         FCB     $7F,$00,$19 ; CLR $0019
```

```
F87D  BD F2 E5      1966                JSR     LF2E5
F880  DE 13         1967                LDX     CURADR
F882  DF BB         1968                STX     $BB
F884  96 42         1969                LDAA    SPLIT
F886  2A 3D         1970                BPL     LF8C5
F888  F6 78 02      1971                LDAB    PIA1B
F88B  84 04         1972                ANDA    #$04
F88D  C4 04         1973                ANDB    #$04
F88F  11            1974                CBA
F890  26 33         1975                BNE     LF8C5
F892  C6 87         1976                LDAB    #$87
F894  4D            1977                TSTA
F895  26 02         1978                BNE     LF899
F897  CB 08         1979                ADDB    #$08
F899  86 D0         1980  LF899          LDAA    #$D0
F89B  D7 BB         1981                STAB    $BB
F89D  97 BC         1982                STAA    $BC
F89F  DE BB         1983                LDX     $BB
F8A1  A6 04         1984                LDAA    $04,X
F8A3  36            1985                PSHA
F8A4  4A            1986                DECA
F8A5  2A 02         1987                BPL     LF8A9
F8A7  86 17         1988                LDAA    #$17
F8A9  A7 04         1989  LF8A9          STAA    $04,X
F8AB  32            1990                PULA
F8AC  BD F5 EA      1991                JSR     ROWADDR
F8AF  EB 00         1992                ADDB    $00,X
F8B1  D7 B1         1993                STAB    $B1
F8B3  97 B2         1994                STAA    $B2
F8B5  96 15         1995                LDAA    FIRSTLN
F8B7  BD F5 EA      1996                JSR     ROWADDR
F8BA  DB 11         1997                ADDB    CRTBASE
F8BC  D7 B3         1998                STAB    $B3
F8BE  97 B4         1999                STAA    $B4
F8C0  C6 50         2000                LDAB    #$50
F8C2  BD F6 BB      2001                JSR     BLITCPY
```

`INSLINE` returns at once if protect mode is on (`TST PROTECT; BEQ; RTS`). Otherwise it clears the attribute mask, moves the cursor to column 0 (`LF2E5`) and saves its address. The block from `LF892` to `LF8C2` handles two-page mode and is skipped here: it begins with `SPLIT` tested for a negative value and, with the usual setting zero, the `BPL LF8C5` branch goes around it. The work is the loop at `LF8C5`:

```
F8C5  96 15         2002  LF8C5          LDAA    FIRSTLN
F8C7  91 16         2003  LF8C7          CMPA    ROW
F8C9  27 42         2004                BEQ     LF90D
F8CB  4D            2005                TSTA
F8CC  27 24         2006                BEQ     LF8F2
F8CE  4A            2007                DECA
F8CF  97 B9         2008                STAA    $B9
F8D1  BD F5 EA      2009                JSR     ROWADDR
F8D4  DB 11         2010                ADDB    CRTBASE
F8D6  D7 B7         2011                STAB    $B7
F8D8  97 B8         2012                STAA    $B8
F8DA  DE B7         2013                LDX     $B7
F8DC  C6 50         2014                LDAB    #$50
F8DE  B7 70 00      2015  LF8DE          STAA    HALT
```

```
F8E1  01            2016                NOP
F8E2  A6 00         2017                LDAA    $00,X
F8E4  B7 70 00      2018                STAA    HALT
F8E7  01            2019                NOP
F8E8  A7 50         2020                STAA    $50,X
F8EA  08            2021                INX
F8EB  5A            2022                DECB
F8EC  26 F0         2023                BNE     LF8DE
F8EE  96 B9         2024                LDAA    $B9
F8F0  20 D5         2025                BRA     LF8C7
F8F2  DE 11         2026  LF8F2          LDX     CRTBASE
F8F4  DF B1         2027                STX     $B1
F8F6  96 B2         2028                LDAA    $B2
F8F8  8B 30         2029                ADDA    #$30
F8FA  97 B4         2030                STAA    $B4
F8FC  96 B1         2031                LDAA    $B1
F8FE  89 07         2032                ADCA    #$07
F900  97 B3         2033                STAA    $B3
F902  C6 50         2034                LDAB    #$50
F904  BD F6 BB      2035                JSR     BLITCPY
F907  86 17         2036                LDAA    #$17
F909  97 B9         2037                STAA    $B9
F90B  20 BA         2038                BRA     LF8C7
F90D  96 4D         2039  LF90D         LDAA    BLANK
F90F  7E F7 E2      2040                JMP     LF7E2
```

Read it as a loop that **walks up from the bottom row**. `A` starts at `FIRSTLN`, the bottom memory row. Each trip first compares A with `ROW`: if equal, the walk has reached the cursor's row and the loop ends at `LF90D`. Otherwise it decrements A, computes the address of row A (`ROWADDR`), and copies the 80 cells of row A to the row below it: `LDAA $00,X` then `STAA $50,X`, since `$50` is 80, the offset of the next row. When A reaches zero the next row up is memory row 23 (`LF8F2`), which is copied with `BLITCPY` into row 0, the same copy done with the wrap. At the end, `LF90D` loads `BLANK` and jumps into `CLREOL`'s fill (`LF7E2`), which **blanks the cursor's row** from column 0.

The effect is that every row from the cursor's row to the bottom moves down by one, the bottom row is lost, and the cursor's row is empty. The experiment types five lines `A1` to `A5` after a clear, puts the cursor on row 2 with `ESC = "` and a space (the `"` is `$22`, so row 2, and the space is column 0), and sends `ESC E`:

```
A1
A2

A3
A4
A5
```

`A3` to `A5` moved down one row, and row 2 is empty. The Python model of this operation is one line: `rows[r+1:] = rows[r:-1]; rows[r] = ''`. It agrees with the emulator.

`DELLINE` does the reverse:

```
F912                2041  ; ESC R : delete line.
F912  7D 00 18      2042  DELLINE         FCB     $7D,$00,$18 ; TST $0018
F915  27 01         2043                  BEQ     LF918
F917  39            2044                  RTS
F918  7F 00 19      2045  LF918           FCB     $7F,$00,$19 ; CLR $0019
F91B  BD F2 E5      2046                  JSR     LF2E5
```

```
F91E  96 16         2047                LDAA    ROW
F920  91 15         2048  LF920         CMPA    FIRSTLN
F922  27 3C         2049                BEQ     LF960
F924  81 17         2050                CMPA    #$17
F926  26 16         2051                BNE     LF93E
F928  D6 11         2052                LDAB    CRTBASE
F92A  CB 07         2053                ADDB    #$07
F92C  86 30         2054                LDAA    #$30
F92E  D7 B1         2055                STAB    $B1
F930  97 B2         2056                STAA    $B2
F932  DE 11         2057                LDX     CRTBASE
F934  DF B3         2058                STX     $B3
F936  C6 50         2059                LDAB    #$50
F938  BD F6 BB      2060                JSR     BLITCPY
F93B  4F            2061                CLRA
F93C  20 E2         2062                BRA     LF920
F93E  36            2063  LF93E         PSHA
F93F  BD F5 EA      2064                JSR     ROWADDR
F942  DB 11         2065                ADDB    CRTBASE
F944  D7 B7         2066                STAB    $B7
F946  97 B8         2067                STAA    $B8
F948  DE B7         2068                LDX     $B7
F94A  C6 50         2069                LDAB    #$50
F94C  B7 70 00      2070  LF94C         STAA    HALT
F94F  01            2071                NOP
F950  A6 50         2072                LDAA    $50,X
F952  B7 70 00      2073                STAA    HALT
F955  01            2074                NOP
F956  A7 00         2075                STAA    $00,X
F958  08            2076                INX
F959  5A            2077                DECB
F95A  26 F0         2078                BNE     LF94C
F95C  32            2079                PULA
F95D  4C            2080                INCA
F95E  20 C0         2081                BRA     LF920
F960  BD F5 EA      2082  LF960         JSR     ROWADDR
F963  DB 11         2083                ADDB    CRTBASE
F965  D7 B1         2084                STAB    $B1
F967  D7 B7         2085                STAB    $B7
F969  97 B2         2086                STAA    $B2
F96B  97 B8         2087                STAA    $B8
F96D  96 42         2088                LDAA    SPLIT
F96F  27 33         2089                BEQ     LF9A4
F971  F6 78 02      2090                LDAB    PIA1B
F974  C4 04         2091                ANDB    #$04
F976  84 04         2092                ANDA    #$04
F978  11            2093                CBA
F979  26 29         2094                BNE     LF9A4
F97B  88 04         2095                EORA    #$04
F97D  48            2096                ASLA
F97E  8B 87         2097                ADDA    #$87
F980  C6 D0         2098                LDAB    #$D0
F982  97 B3         2099                STAA    $B3
F984  D7 B4         2100                STAB    $B4
F986  DE B3         2101                LDX     $B3
F988  A6 04         2102                LDAA    $04,X
F98A  4C            2103                INCA
```

```
F98B  81 17         2104                CMPA    #$17
F98D  2F 01         2105                BLE     LF990
F98F  4F            2106                CLRA
F990  A7 04         2107  LF990         STAA    $04,X
F992  BD F5 EA      2108                JSR     ROWADDR
F995  EB 00         2109                ADDB    $00,X
F997  D7 B3         2110                STAB    $B3
F999  97 B4         2111                STAA    $B4
F99B  DE B3         2112                LDX     $B3
F99D  DF B7         2113                STX     $B7
F99F  C6 50         2114                LDAB    #$50
F9A1  BD F6 BB      2115                JSR     BLITCPY
F9A4  DE B7         2116  LF9A4         LDX     $B7
F9A6  C6 50         2117                LDAB    #$50
F9A8  96 4D         2118                LDAA    BLANK
F9AA  B7 70 00      2119  LF9AA         STAA    HALT
F9AD  01            2120                NOP
F9AE  A7 00         2121                STAA    $00,X
F9B0  08            2122                INX
F9B1  5A            2123                DECB
F9B2  26 F6         2124                BNE     LF9AA
F9B4  39            2125                RTS
```

It walks **down** from the cursor's row. For each row A it copies row A + 1 into row A (`LDAA $50,X / STAA $00,X`); at row 23 it copies row 0 into row 23 (`LF928`, with `BLITCPY`) and carries on from zero; and when A reaches `FIRSTLN` it stops copying and clears that last row (`LF960`: compute the row address; `LF9A4` fills 80 cells with `BLANK`). So every row below the cursor's moves up one, and the bottom line is blank:

```
A1
A2
A4
A5
```

Row 2 (`A3`) has gone, `A4` and `A5` moved up, and there are now three blank rows at the end instead of two.

Notice what these two routines do *not* do: they do not touch `FIRSTLN`, and so do not scroll. Their loops work in the memory rows' ring order, and the copies wrap from row 23 to row 0 (or back). That is why the same code works wherever `FIRSTLN` happens to be.

Two pages, and the saved state

Display memory is 4 KB at `$8000-$8FFF`, which is two pages of `$800` bytes, and a page holds 1,920 visible cells, `$780` bytes. The remaining `$80` bytes of each page, `$8780-$87FF` and `$8F80-$8FFF`, are not displayed. The firmware uses the top 48 of them, `$87D0-$87FF` and `$8FD0-$8FFF`, to **save the display state** of the page:

```
F65E  C6 06         1689  LF65E         LDAB    #$06
F660  D7 B6         1690  LF660         STAB    $B6
F662  BD F6 72      1691                JSR     LF672
F665  B6 78 02      1692                LDAA    PIA1B
F668  88 04         1693                EORA    #$04
F66A  B7 78 02      1694                STAA    PIA1B
F66D  8D 16         1695                BSR     LF685
F66F  DE 13         1696                LDX     CURADR
F671  39            1697  LF671         RTS
```

```
F672  96 11         1698  LF672           LDAA    CRTBASE
F674  8B 07         1699                  ADDA    #$07
F676  97 B1         1700                  STAA    $B1
F678  86 D0         1701                  LDAA    #$D0
F67A  97 B2         1702                  STAA    $B2
F67C  CE 00 11      1703                  LDX     #$0011
F67F  DF B3         1704                  STX     $B3
F681  C6 30         1705                  LDAB    #$30
F683  20 36         1706                  BRA     BLITCPY
F685  CE 00 11      1707  LF685           LDX     #$0011
F688  DF B1         1708                  STX     $B1
F68A  B6 78 02      1709                  LDAA    PIA1B
F68D  84 04         1710                  ANDA    #$04
F68F  48            1711                  ASLA
F690  8B 80         1712                  ADDA    #$80
F692  97 11         1713                  STAA    CRTBASE
F694  8B 07         1714                  ADDA    #$07
F696  97 B3         1715                  STAA    $B3
F698  86 D0         1716                  LDAA    #$D0
F69A  97 B4         1717                  STAA    $B4
F69C  7D 00 42      1718                  FCB     $7D,$00,$42 ; TST $0042
F69F  27 04         1719                  BEQ     LF6A5
F6A1  D6 B6         1720                  LDAB    $B6
F6A3  20 02         1721                  BRA     LF6A7
F6A5  C6 30         1722  LF6A5           LDAB    #$30
F6A7  8D 12         1723  LF6A7           BSR     BLITCPY
F6A9  96 15         1724                  LDAA    FIRSTLN
F6AB  B7 7F 06      1725                  STAA    VTAC6
F6AE  96 16         1726                  LDAA    ROW
F6B0  B7 7F 0D      1727                  STAA    VTACD
F6B3  96 17         1728                  LDAA    COL
F6B5  B7 7F 0C      1729                  STAA    VTACC
F6B8  7E F5 F8      1730                  JMP     CURADDR
```

LF672 saves: it sets the destination to CRTBASE + 7 for the high byte and $D0 for the low byte (that is, $87D0 for page one), the source to $0011 and the count B to $30 (48), and calls BLITCPY. So **zero page $11 to $40 is copied into the top of the display page**: CRTBASE, CURADR, FIRSTLN, ROW, COL, PROTECT, ATTRMSK, the monitor-mode and keyboard-lock flags, and the rest of the display variables up to $40. (The mode byte $41 and SPLIT at $42 are *not* in the range.)

LF65E is the page switch: it saves the current state (LF672); toggles bit 2 of port B (LDAA PIA1B; EORA #$04; STAA PIA1B), which selects the page being displayed; then calls LF685, which restores. LF685 reads bit 2 of port B back, shifts it up one place and adds $80 to get the new CRTBASE ($80 or $88), makes the source new CRTBASE + 7, $D0 and the destination $0011, and copies 48 bytes back. Then it writes FIRSTLN to VTAC6, ROW to VTACD, COL to VTACC, and recomputes CURADR. When SPLIT is non-zero the restore copies only $B6 bytes instead of 48: 6 when LF65E is the caller, 7 when LF631 is. That is $11-$16 (the base, the cursor address, FIRSTLN and ROW), or $11-$17 with COL, so in two-page mode a switch restores only the cursor and scroll state.

Two things were observed, and one was not.

- **Observed:** after the firmware has started, memory holds an initial state image in each page: $87D0 begins 80 00 80 00 17 00 00 00 and $8FD0 begins 88 00 88 00 17 00 00 00. Read as the layout above: CRTBASE = $8000, CURADR = $8000, FIRSTLN = 23, ROW = 0, COL

= 0; and for the second page the same with $8800. The test checks both images on both ROM sets, and the 129867 start-up code that writes them is in the listing at E922-E934 (STX $87D0 ... STAA $8FD4), which is the same block that sets CRTBASE, CURADR and FIRSTLN for page one.

- **Observed:** after ESC v (LF0B3), SPLIT holds $80; the routine ORs $80 into the current page bit (bit 2 of port B, which the emulator reads as zero) and does so only under three conditions that were not examined. ESC w (LF0CD) clears SPLIT; that was read, not run.
- **Not observed:** an actual page switch. The emulator's port B returns only SHIFT and CTRL when read; it does not return the output bits (alpha lock and page) that a real PIA returns. The firmware's read-modify-write of PIA1B therefore cannot round-trip in the emulator, and a page switch run there is not meaningful. This book has not seen a page switch on the real terminal; what is described above is from the listing only.

This limitation also covers the alpha-lock toggle of Chapter 28 (LEF61 does the same read-modify-write), which is why that chapter did not test it.

What a plug-in can take from this

- **The display state is a contract.** Every firmware display routine assumes FIRSTLN, ROW, COL, CURADR and the VTAC registers agree. A plug-in that writes to the screen with its own routines (all of this book's) does not need them, but one that returns to the firmware, or calls a firmware routine, must leave them consistent. The simplest way is not to touch the VTAC registers except through the firmware.
- **Chapter 8's INITCRT writes register 6 as $17.** The firmware's FIRSTLN is not changed by that, so it can disagree with the register afterwards. The plug-ins in this book that leave, such as T19EXIT, do so through the reset vector (Chapter 25), which sets the firmware's state up again from scratch.
- **The fill-with-zero scroll and the fill-with-BLANK clear** are the two different "empty" values the firmware uses. A plug-in that wants an empty screen should write the value its own drawing treats as empty ($20 in this book), and not assume that cells the firmware has cleared hold a space.
- **A call to a firmware routine is a call to a firmware-specific address** (Chapter 23). ROWADDR has no side effects at all and CURADDR only writes CURADR, so they are the easiest of these routines to call; but their addresses are listed in Appendix H for one revision only, and this book's own CALCADR has no such dependence.

Exercises

1. Work out by hand the addresses ROWADDR and CURADDR produce for row 13, column 41, page two (CRTBASE = $8800). (Answer: 13 × 80 = 1040 = $0410; plus 41 = $0439; plus $8800 = $8C39.)
2. After the 30-line scroll experiment, which memory row holds the line L12, and which screen row displays it? (Answer: memory row 11, since row r held L(r+1) before the scroll wrapped; screen row (11 - 7) mod 24 = 4.)
3. Why does ESC T stop at the first protected cell while ESC + steps over it? What would a form-filling application want in each case?
4. INSLINE walks up from the bottom and DELLINE walks down from the cursor. Why can neither walk in the other direction? (Think about what the copy would overwrite.)

5. After 48 lines `L01`-`L48`, each followed by CR and LF, on a freshly cleared screen: what are `FIRSTLN` and `ROW`, and which lines are displayed? (Answer: 25 line feeds are scrolls, the first being the one after `L24`; `FIRSTLN = (23 + 25) mod 24 = 0`, `ROW = 0`; the screen shows `L26` to `L48` and a blank bottom line.)
6. `CLRSCR`'s fill loop tests X against `$B7` and wraps to `CRTBASE`; `CLREOL`'s loop does not. Why not?

Chapter 30

Talking to the host: the transmit path, the replies, and the printer

Chapter 12 set up the serial interfaces and Chapter 27 followed a character *in*: the interrupt, the ring buffer, task 2, task 3. This chapter follows characters *out*. Everything the terminal says to its host, a typed key, the report of the cursor position, the answerback, goes through one small routine and one interrupt service, and a task that waits for the line to go quiet. This chapter reads all three and then runs them.

A note on the emulator

The emulator that has run every program in this book up to now did not model the serial interface well enough for this chapter: it never set the interface's "interrupt request" status bit, so the firmware's transmit interrupt could never fire, and it did not record what was written to the transmit register. For this chapter it was extended in three small ways:

- the ACIA status bit 7 (IRQ) now follows the 6850's rules: set when the receive interrupt is enabled and a character is waiting, or when the transmit-control bits are `01` (transmit interrupt enabled) and the transmitter is empty;
- `--txlog FILE` writes every byte stored to the data register `$7A01` into a file;
- `--rx STRING` puts characters into the receiver as if the host had sent them, after the boot.

The transmitter is modelled as always empty and always ready, and IRQs still arrive every 20,000 instructions. So the tests of this chapter prove that the *firmware's logic* behaves as described, and the byte values the host would see. They say nothing about line timing, and nothing about what a real 6850 does.

The queue and the routine that fills it

```
FCDD  D6 CC         2498  LFCDD           LDAB    $CC
FCDF  CA 01         2499                  ORAB    #$01
FCE1  D7 CC         2500                  STAB    $CC
FCE3  36            2501                  PSHA
FCE4  F6 7E 00      2502                  LDAB    SWBNK1
FCE7  C4 1C         2503                  ANDB    #$1C
FCE9  96 BF         2504                  LDAA    ACIACTL
FCEB  84 83         2505                  ANDA    #$83
```

```
FCED  1B            2506                ABA
FCEE  97 BF         2507                STAA    ACIACTL
FCF0  B7 7A 00      2508                STAA    ACIA
FCF3  32            2509                PULA
FCF4  39            2510                RTS
FCF5  36            2511  LFCF5         PSHA
FCF6  7D 00 CC      2512                FCB     $7D,$00,$CC ; TST $00CC
FCF9  26 02         2513                BNE     LFCFD
FCFB  8D E0         2514                BSR     LFCDD
FCFD  D6 CB         2515  LFCFD         LDAB    TXCOUNT
FCFF  C1 0C         2516                CMPB    #$0C
FD01  27 FA         2517                BEQ     LFCFD
FD03  DE C9         2518                LDX     $C9
FD05  32            2519                PULA
FD06  A7 00         2520                STAA    $00,X
FD08  16            2521                TAB
FD09  D8 0E         2522                EORB    $0E
FD0B  D7 0E         2523                STAB    $0E
FD0D  7C 00 CB      2524                FCB     $7C,$00,$CB ; INC $00CB
FD10  08            2525                INX
FD11  8C 01 0C      2526                CPX     #$010C
FD14  26 03         2527                BNE     LFD19
FD16  CE 01 00      2528                LDX     #$0100
FD19  DF C9         2529  LFD19         STX     $C9
FD1B  D6 BF         2530                LDAB    ACIACTL
FD1D  C4 9F         2531                ANDB    #$9F
FD1F  CA 20         2532                ORAB    #$20
FD21  D7 BF         2533                STAB    ACIACTL
FD23  F7 7A 00      2534                STAB    ACIA
FD26  39            2535                RTS
```

The firmware never writes to the transmit register directly. A routine at `LFCF5` (this book calls it the *send* routine) puts the character in A into a **twelve-byte queue** at `$0100-$010B`. The queue has a head pointer at `$C7-$C8`, a tail pointer at `$C9-$CA`, and a count at `$CB` (`TXCOUNT`). Read `LFCF5` from the top:

1. `PSHA`. Then `TST $00CC`: `$CC` is a flag that says "a transmission is under way". If it is zero, `BSR LFCDD` runs first and starts one.
2. `LFCDD` sets bit 0 of `$CC` and builds a fresh control byte for the interface: `(SWBNK1 AND $1C) + (ACIACTL AND $83)`. The word format (bits 4-2) is **re-read from the switches** on every transmission; bits 7, 1 and 0 are kept; and **bits 6-5 are cleared**, which in the 6850 means "RTS low, transmit interrupt off". The byte goes to `ACIACTL` and to the interface. So the first thing a transmission does is to *assert* the RTS line.
3. `LFCFD: LDAB TXCOUNT; CMPB #$0C; BEQ LFCFD`: **if the queue is full, wait here**. This is a loop of two instructions that does not call `SLEEP`: it holds the processor until the interrupt service has emptied a slot. A routine that sends more than twelve characters at once therefore depends on interrupts being enabled and on the line making progress; if either fails it never returns.
4. `LDX $C9`, then the character is stored at the tail (`STAA $00,X`). The character is also XORed into `$0E` (`TAB; EORB $0E; STAB $0E`): the firmware keeps a **running XOR of every character it queues** (and, Chapter 27, every character it removes from the receive buffer). This book read the arithmetic but did not trace what uses it.
5. `INC $CB`, `INX`, and a wrap from `$010C` to `$0100`; the tail is stored.

6. **Enable the transmit interrupt**: `ACIACTL` is loaded, bits 6-5 are cleared (`ANDB #$9F`) and set to `01` (`ORAB #$20`), and the byte is written to the interface.

The transmit interrupt then fires as soon as the interface can take a character, and `IRQ` reaches `LFD85` through the `LEDEF` test of Chapter 27:

```
FD85  7D 00 CB      2576  LFD85           FCB     $7D,$00,$CB ; TST $00CB
FD88  27 1A         2577                  BEQ     LFDA4
FD8A  DE C7         2578                  LDX     $C7
FD8C  A6 00         2579                  LDAA    $00,X
FD8E  84 7F         2580                  ANDA    #$7F
FD90  9A CE         2581                  ORAA    $CE
FD92  B7 7A 01      2582                  STAA    ACIAD
FD95  7A 00 CB      2583                  FCB     $7A,$00,$CB ; DEC $00CB
FD98  08            2584                  INX
FD99  8C 01 0C      2585                  CPX     #$010C
FD9C  26 03         2586                  BNE     LFDA1
FD9E  CE 01 00      2587                  LDX     #$0100
FDA1  DF C7         2588  LFDA1           STX     $C7
FDA3  39            2589                  RTS
FDA4  96 BF         2590  LFDA4           LDAA    ACIACTL
FDA6  84 9F         2591                  ANDA    #$9F
FDA8  97 BF         2592                  STAA    ACIACTL
FDAA  B7 7A 00      2593                  STAA    ACIA
FDAD  39            2594                  RTS
FDAE  AA            2595                  FCB     $AA
```

`LFD85` is the **transmit service**: if `TXCOUNT` is zero, go to `LFDA4`, which clears bits 6-5 of the control byte (the transmit interrupt off again: nothing left to send). Otherwise it takes the character at the **head** (`LDX $C7; LDAA $00,X`), masks it to seven bits (`ANDA #$7F`) and ORs in `$CE`, writes it to the data register `ACIAD`, decrements the count, advances the head with the same `$010C` wrap and returns. **One character per interrupt**: when the interface is empty again it interrupts again, and the interrupt handler returns to whatever task was running.

The eighth bit

`$CE` is set once, at start-up (`E904-E909`, the `PIAINIT` of Chapter 26):

```
E904  53            COMB            ; B = NOT SWBNK1
E905  C4 20         ANDB    #$20    ; keep what was bit 5
E907  58            ASLB
E908  58            ASLB            ; ... and move it to bit 7
E909  D7 CE         STAB    $CE
```

So `$CE` is `$80` if bit 5 of switch bank 1 is **clear**, and zero if it is set; the firmware forces bit 7 of every byte it transmits to that value. What the switch is labelled, and what bit 7 is for on the line, is not something the listing says. The emulator reads every switch as zero, so in the tests below **every transmitted byte has bit 7 set** (`a` goes out as `$E1`). That is a property of the emulator's switch values, not a behaviour of every terminal.

Task 7: letting the line go quiet

`TXDELAY` is the firmware's task 7. It is woken by whoever has just finished sending something (Chapters 27 and 28 showed the callers: the end of a reply, and a typed CR, EOT or ETX), and it does the housekeeping that must come *after* the last character has left:

```
FD27                2536  ; TXDELAY: wait for the TX queue and ACIA to empty,
```

```
FD27                2537  ; delay, then rewrite ACIA control (bits 6-5 = 10),
FD27                2538  ; wake tasks 0 and 2.
FD27  7F 00 CC      2539  TXDELAY         FCB     $7F,$00,$CC ; CLR $00CC
FD2A  96 CB         2540  LFD2A           LDAA    TXCOUNT
FD2C  27 07         2541                  BEQ     LFD35
FD2E  86 01         2542                  LDAA    #$01
FD30  BD EC CC      2543                  JSR     SLEEP
FD33  20 F5         2544                  BRA     LFD2A
FD35  B6 7A 00      2545  LFD35           LDAA    ACIA
FD38  85 02         2546                  BITA    #$02
FD3A  27 EE         2547                  BEQ     LFD2A
FD3C  96 4F         2548                  LDAA    $4F
FD3E  26 0D         2549                  BNE     LFD4D
FD40  CE FD 75      2550                  LDX     #BAUDTAB
FD43  B6 7E 80      2551                  LDAA    BAUDSW
FD46  84 0F         2552                  ANDA    #$0F
FD48  BD EF AD      2553                  JSR     XPLUSA
FD4B  A6 00         2554                  LDAA    $00,X
FD4D  BD EC CC      2555  LFD4D           JSR     SLEEP
FD50  96 BF         2556                  LDAA    ACIACTL
FD52  84 9F         2557                  ANDA    #$9F
FD54  8A 40         2558                  ORAA    #$40
FD56  97 BF         2559                  STAA    ACIACTL
FD58  B7 7A 00      2560                  STAA    ACIA
FD5B  CE 00 00      2561                  LDX     #$0000
FD5E  BD EC BA      2562                  JSR     WAKE
FD61  CE 00 02      2563                  LDX     #$0002
FD64  BD EC BA      2564                  JSR     WAKE
FD67  7E EC C1      2565                  JMP     BLOCKME
```

1. `CLR $00CC`: the "transmission under way" flag is cleared, so the next call of `LFCF5` will start a new transmission (and re-read the switches).
2. `LFD2A`: wait until `TXCOUNT` is zero, sleeping one tick at a time (`SLEEP`), and then until the interface's TDRE bit (`BITA #$02` on `ACIA`) shows that the transmit register itself has emptied. Both waits use `SLEEP`, so unlike `LFCF5`'s loop they let the other tasks run.
3. Choose a delay: if `$4F` is non-zero, use it; otherwise index the table `BAUDTAB` (Chapter 12) with the low nibble of the switch at `$7E80` (`XPLUSA`) and use the value from there. `JSR SLEEP` for that many ticks.
4. Rewrite the control byte with bits 6-5 = `10` (`ANDA #$9F; ORAA #$40`): **RTS high, transmit interrupt off**: the line is released.
5. `WAKE` tasks 0 and 2 (the keyboard scan and the receive consumer) and block itself.

Step 5 completes a picture that Chapter 28 left open. When a key asks for a reply to the host (the next section), `LF9D9` **blocks tasks 0 and 2**; the keyboard is not scanned and nothing is taken out of the receive buffer while the reply is being sent; and it is `TXDELAY` that lets them start again. The sequence is the same as for the RTS line: asserted at the start (`LFCDD`), released at the end (task 7, step 4).

`BAUDTAB` is the same as in Chapter 12: sixteen counts falling from `$AC` (172) to `$01`. The exact line speeds that the sixteen positions of the switch select are not read by any code here; it is a property of the switch wiring.

Replies: task 4

Typed keys are not the only things transmitted. The firmware also **answers questions**: where is the cursor, which page is displayed. Those replies are built by **task 4**, which LF9DA wakes. LF9DA is the common end of nine of the ESC-table handlers (LF9B5 to LF9D5 each load a small request number into A and branch to it). It stores the request number in $C5, wakes task 4 with a zero argument, and blocks tasks 0 and 2, as above.

```
F9D9  43           2144  LF9D9          COMA
F9DA  97 C5        2145  LF9DA          STAA    $C5
F9DC  BD EF BC     2146                 JSR     LEFBC
F9DF  7F 00 AE     2147                 FCB     $7F,$00,$AE ; CLR $00AE
F9E2  4F           2148                 CLRA
F9E3  CE 00 04     2149                 LDX     #$0004
F9E6  BD EC B8     2150                 JSR     LECB8
F9E9  CE 00 00     2151                 LDX     #$0000
F9EC  BD EC C3     2152                 JSR     BLOCKX
F9EF  CE 00 02     2153                 LDX     #$0002
F9F2  7E EC C3     2154                 JMP     BLOCKX
F9F5  BD FC DD     2155  LF9F5          JSR     LFCDD
F9F8  96 C5        2156                 LDAA    $C5
F9FA  2A 05        2157                 BPL     LFA01
F9FC  CE FA 77     2158                 LDX     #LFA77
F9FF  20 0F        2159                 BRA     LFA10
FA01  D6 55        2160  LFA01          LDAB    $55
FA03  C4 20        2161                 ANDB    #$20
FA05  D7 CD        2162                 STAB    $CD
FA07  CE FA 32     2163                 LDX     #JMPTAB
FA0A  48           2164                 ASLA
FA0B  BD EF AD     2165                 JSR     XPLUSA
FA0E  EE 00        2166                 LDX     $00,X
FA10  AD 00        2167  LFA10          JSR     $00,X
FA12  96 C5        2168                 LDAA    $C5
FA14  81 0A        2169                 CMPA    #$0A
FA16  27 0E        2170                 BEQ     LFA26
FA18  96 46        2171                 LDAA    $46
FA1A  27 0A        2172                 BEQ     LFA26
FA1C  BD FC F5     2173                 JSR     LFCF5
FA1F  96 47        2174                 LDAA    $47
FA21  27 03        2175                 BEQ     LFA26
FA23  BD FC F5     2176                 JSR     LFCF5
FA26  BD FB 2F     2177  LFA26          JSR     LFB2F
FA29  CE 00 07     2178                 LDX     #$0007
FA2C  BD EC BA     2179                 JSR     WAKE
FA2F  7E EC C1     2180                 JMP     BLOCKME
FA32  FA 48 FA 48  2181  JMPTAB         FCB     $FA,$48,$FA,$48,$FA,$A5
FA38  FA A5 FA AD  2182                 FCB     $FA,$A5,$FA,$AD,$FA,$AD
FA3E  FC 0B FC 05  2183                 FCB     $FC,$0B,$FC,$05,$FA,$62
FA44  FA 49 FA 82  2184                 FCB     $FA,$49,$FA,$82
```

Task 4 (LF9F5) is the dispatcher. JSR LFCDD starts a transmission; then $C5 selects the work. A **negative** request number (the keypad's function codes from Chapter 28, which LEF42 complemented) goes to LFA77, which sends the prefix byte at $4B and then the key's own number. A **positive** number indexes JMPTAB (two bytes per request, starting at 0). The routine runs; then, unless the request was 10 (ENQ), the **terminator characters** at $46 and $47 are sent if they

are non-zero (the start-up table makes `$46 = $0D`, a carriage return, and `$47 = 0`, nothing); `LFB2F` finishes; and task 7 is woken to release the line.

Request	Set by	Routine	What it sends
8	`ESC ?`	`LFA62`	**the cursor position**: screen row + `$20`, column + `$20`
9	`ESC /`	`LFA49`	**the displayed page**: `$30` plus the page bit from port B (one more if bit 6 of the option byte `$54` is clear), followed by the cursor position as above
10	`ENQ` (CTRL-E)	`LFA82`	the **answerback**: text taken from the personality PROM window at `$7B80`
2-7	`ESC 4`, `ESC 6`, `ESC 5`, `ESC 7`, `ESC S`, `ESC s`	`LFAA5`, `LFAAD`, `LFC0B`, `LFC05`	blocks of screen text: not read in detail

Two details of the cursor report. The row it sends is not `ROW`, which is a memory row (Chapter 29): `LFA62` computes `ROW - 1 - FIRSTLN` modulo 24 (`DECA; SUBA FIRSTLN; BGE; ADDA #$18`), the inverse of the adjustment `CURPOS` makes, to get the **screen** row; then it adds `$20`, like the input form of `ESC =`. And the answerback is not stored in the program at all: `LFA82` first checks that the bytes at `$7B9E` and `$7B9F` of the personality-PROM window are `$0D` and `$8A`, and if they are, sends bytes from `$7B80` up to and including the first one that has bit 7 set. The emulator has no personality PROM, so `$7B80` reads as zero and the ENQ test (typing CTRL-E) sends nothing; the *code* path was read, the *reply* was not seen.

The replies, observed

With the cursor placed by `ESC = % #` (row 5, column 3), the keystroke `ESC ?` is typed. The log of bytes written to the transmit register is `A5 A3 8D`: row 5 + `$20` = `$25`, column 3 + `$20` = `$23`, and the CR terminator `$0D`, each with bit 7 added by `$CE`. `ESC /` gives `B0 A5 A3 8D`: the page indicator `$30` with bit 7 added, then the same three bytes. Both ROM sets give the same bytes (the tests check the 129827 as well).

Typed	Transmitted	Meaning
`ESC = % #`, `ESC ?`	`A5 A3 8D`	row `$25`-`$20` = 5, column `$23`-`$20` = 3, CR
`ESC = % #`, `ESC /`	`B0 A5 A3 8D`	page `0`, then as above

Keystrokes to the host, observed

Chapter 28 described the mode bits in `$41`. Now they can be seen from outside. After boot `$41` is zero (the emulator's switches read zero), so keys are processed locally and **nothing is transmitted**; the test types `abc` and the transmit log is empty. If `ESC C` is typed first (`LF083`, setting bit 6 of `$41`), then `abc` followed by CR gives the transmit log `E1 E2 E3 8D`: each key as the character `$61`-`$63` or `$0D` with bit 7 set, and the screen stays as it was: **in this mode the terminal shows nothing**, it relies on the host to echo. The queue is empty afterwards (`TXCOUNT = 0`) and its head and tail pointers are both `$0103`: three characters went in at the tail and the same three came out at the head, and the pointers moved on three places each.

Typing `ESC C ESC D H` shows the other half of Chapter 28's claim. The log is `9B C4 C8`: the second ESC, `D` and `H` were **sent to the host** and not executed, because in this mode they are meant to come *back* from the host. Without the echo, bit 7 of `$41` never gets set.

The receive side, run

`--rx` makes the same escape sequence arrive from the "host". The text `x y ESC = % # z` is injected into the receiver after the boot. The interrupt handler stores it in the ring buffer (Chapter 27); task 2 hands each character to task 3; and `LEFF6` (Chapter 28) draws and interprets it. The screen is identical to the one typed at the keyboard in Chapter 28:

```
xyst Complete




   z
```

`xy` over the first two characters of the idle message, and `z` at row 5, column 3. This is a test of the whole path of Chapter 27 working together: IRQ, `ACIAIRQ`, the ring buffer, task 2, task 3, `GETRX` for the two coordinate characters of `ESC =`, and the cursor routine of Chapter 29.

The printer port

The second interface, at `$7C00`, is a plain output port with its own task. `LFDAF` is the entry. The ESC-table handlers for `ESC @`, `ESC L`, `ESC P` and `ESC p` (`LF215`, `LF211`, `LF20D`) each load a mode value in A, return at once if bit 7 of switch bank 2 is set (`BITB #$80`), and otherwise jump there. `LFDE2` is the start of the job: it builds the printer's control byte from switch bank 2, **bits 4-2 and a fixed `$01`** (÷16, the word format from the switch, RTS low, no interrupts), stores it in `$C0` and in `PACIA`, wakes **task 5** and blocks tasks 0 and 2. Task 5 (`LFE03`) polls the printer's status with `LDAB PACIA; ANDB #$0A; CMPB #$02`: it sends only when the transmitter is empty (bit 1 set) and the CTS line is low (bit 3 clear). It sends one character per call, driven by a small state table in `$D0` (a chain of `DECB` tests at `LFE57`) that produces the carriage return and line feed (`LFE7F` sends `$0D`, `LFE88` sends `$0A`) between lines. This is the whole of what the book read: the printer path was **not run**, since the emulator does not model the second interface at all, and it is not described further here.

What a plug-in can take from this

- **To send a character to the host from a program that owns the machine**, write it to `ACIAD` (`$7A01`) after waiting for TDRE (Chapter 12). That works on both firmware sets, and does not need any of the queue machinery above.
- **To use the firmware's send routine from a plug-in**, call `LFCF5` at `$FCF5` (129867 only; the address on the 129827 is in Appendix H). It requires interrupts enabled (`CLI`) and the firmware's scheduler running: it waits in a loop for the queue to drain, and nothing but the interrupt can drain it. A plug-in that has `SEI` at its start, as all of this book's do, would hang the first time it sent thirteen characters.
- **Remember the RTS line.** The firmware asserts RTS for the length of a transmission and lets it go in task 7. A program that bypasses all that and writes to `ACIAD` directly is

responsible for whatever its host expects of the handshake lines; Chapter 12's table of the control byte says which bits are involved.

Exercises

1. In the emulator, with `ESC C` typed first, what are the transmitted bytes for the key sequence `Hi!` (shifted `h`, `i`, shifted `1`)? (Answer: `C8 E9 A1`: `$48`, `$69`, `$21`, each with bit 7 added.)
2. What happens when `LFCF5` is called 13 times in a row with interrupts disabled? Why does that not happen when the firmware's own tasks do it?
3. `ESC =` takes row and column characters, offset by `$20`. `ESC ?` sends them back in the same offset form. Write the byte sequence that moves the cursor to row 10, column 20, and then the reply that `ESC ?` will give (with bit 7 clear, and CR as terminator). (Answer: `1B 3D 2A 34` (`ESC = * 4`); the reply `2A 34 0D`.)
4. The transmit service sends one character per interrupt. At 9,600 baud (ten bits per character) how long does a 12-character queue take to drain, and in how many interrupts? (Answer: 12 × 10 / 9,600 = 12.5 ms, in 12 interrupts.)
5. Why does the firmware block the keyboard scan and the receive consumer while a reply is being sent? What could go wrong if a key were typed in the middle of a cursor report?

Chapter 31

Writing your own firmware: FW31

Every program in this book so far has been a *guest*: a plug-in that the firmware starts, and that starts by switching the firmware's own business off. This last chapter turns the tables. It writes the host. **FW31** is a complete firmware in 2 KB, from the reset vector to a working terminal display, assembled with the same assembler and listed in the same format as everything else. It is made almost entirely from parts that earlier chapters built or read: the keyboard table of Chapter 28, the scroll of Chapter 29, `GETKEY` and `CALCADR` and `PUTAT` from Chapters 8 and 9, and the plug-in launch of Chapters 5 and 25.

Be clear about what this is and what it is not. FW31 is a *teaching firmware*. It is not a replacement for the 6 KB of the real one, and it **has only run in the emulator**. The emulator's models of the PIA and the CRT controller are permissive: they accept what the real chips might refuse, or want in a different order. A program that passes here is a program that follows the real firmware's own sequence of writes (the start-up code is copied from the listing) and that gives the same results as the real firmware's code (the tests below compare them byte for byte). It has not run on a terminal.

What it does, and what it leaves out

FW31:

- sets up the CRT controller, the keyboard PIA and its own variables, clears the screen and starts;
- reads the keyboard with the polled `GETKEY` of Chapter 9, translating scan codes with the firmware's own `KBDTAB`;
- prints characters at the cursor, with wrap at column 79 and **hardware scroll** at the bottom line (Chapter 29);
- obeys seven control codes (BS, LF, VT, FF, CR, RS, US) and three escape sequences: `ESC +` (clear), `ESC =` row column (cursor addressing, with the same clamping) and `ESC ` ` (launch the plug-in in socket A);
- keeps the hardware cursor in step.

It leaves out the serial ports (it never touches either ACIA), interrupts (it runs with `SEI` for ever and has no scheduler), the bell, keyboard auto-repeat, CTRL and alpha-lock handling in the keyboard (the `GETKEY` of Chapter 9 does not look at them), the keypad, protected fields, line insert and delete, the two display pages, the power-on tests and everything else in Chapters 24 to 30. Each of those is a chapter that tells you how to add it.

Where it lives

The firmware occupies `$E800-$FFFF` in three 2 KB ROMs. The reset and interrupt vectors are the last eight bytes of the last ROM, `$FFF8-$FFFF` (Chapter 24), so FW31 is a single ROM placed at **`$F800-$FFFF`**: the last of the three sockets. (The emulator calls the socket "D". Chapter 24 notes that the socket designators on the real board are not settled by the sources this book has: the emulator's header names the chips `U62-U65`, but the firmware set's own configuration file lists `U-65`, `U-66` and `U-67`. FW31 does not depend on which is which, only on the address.) The source says so on a comment line, `; @BASE F800`, which the build tool reads to place the 2,048 bytes at `$F800` instead of the plug-in address `$E000`.

The ROM is 2,048 bytes and FW31 uses **820** of them, plus the 8 vector bytes:

Part	Address	Bytes
FW31's own code (reset, main loop, character machine)	`$F800-$F9B1`	434
`KBDTAB`, the keyboard table (from the real firmware)	`$F9B2-$FA61`	176
`CALCADR`	`$FA62-$FABB`	90
`PUTAT`, `PUTSTR`	`$FABC-$FAE2`	39
`XPLUSA`	`$FAE3-$FAF7`	21
`GETKEY`	`$FAF8-$FB33`	60
the vector table	`$FFF8-$FFFF`	8

Reset

```
F800  0F             47  RESET           SEI
F801  8E 03 FF       48                  LDS     #$03FF
F804                 49  ; ---- the CRT controller, as the firmware loads it
F804  CE 7F 00       50                  LDX     #$7F00
F807  86 65          51                  LDAA    #$65
F809  A7 00          52                  STAA    0,X
F80B  86 44          53                  LDAA    #$44
F80D  A7 01          54                  STAA    1,X
F80F  86 5D          55                  LDAA    #$5D
F811  A7 02          56                  STAA    2,X
F813  86 17          57                  LDAA    #$17
F815  A7 03          58                  STAA    3,X
F817  86 40          59                  LDAA    #$40
F819  A7 04          60                  STAA    4,X
F81B  86 35          61                  LDAA    #$35
F81D  A7 05          62                  STAA    5,X
F81F  86 17          63                  LDAA    #$17
F821  A7 06          64                  STAA    6,X       ; scroll register
F823  4F             65                  CLRA
F824  A7 0C          66                  STAA    $0C,X     ; cursor column
F826  A7 0D          67                  STAA    $0D,X     ; cursor row
F828  A7 0E          68                  STAA    $0E,X
F82A                 69  ; ---- the keyboard PIA, as the firmware sets it
F82A  CE 78 00       70                  LDX     #$7800
F82D  6F 01          71                  CLR     1,X
F82F  6F 03          72                  CLR     3,X
```

```
F831  86 7F           73                  LDAA    #$7F
F833  A7 00           74                  STAA    0,X
F835  86 1D           75                  LDAA    #$1D
F837  A7 02           76                  STAA    2,X
F839  86 3C           77                  LDAA    #$3C
F83B  A7 01           78                  STAA    1,X
F83D  A7 03           79                  STAA    3,X
F83F  86 01           80                  LDAA    #$01
F841  A7 00           81                  STAA    0,X
F843  A7 02           82                  STAA    2,X
F845                  83  ; ---- our own state
F845  86 17           84                  LDAA    #$17
F847  B7 01 12        85                  STAA    FIRSTLN
F84A  7F 01 13        86                  CLR     ESCST
F84D  7F 01 0D        87                  CLR     GKHELD
F850  BD F9 AD        88                  JSR     CLEAR
```

Everything in this block is the real firmware's own start-up, shortened.

- `SEI; LDS #$03FF`: interrupts masked, stack at the top of RAM, as in every program.
- **The CRT controller.** `LDX #$7F00`, then the six timing registers with the values of Chapter 6 (`$65 $44 $5D $17 $40 $35`: the firmware chooses between two sets for the last two according to a switch, and FW31 uses the default), then register 6, the scroll register, to `$17`, and the cursor registers to zero. Without this the picture would not exist; with the real firmware's values, it is the same picture.
- **The keyboard PIA**, in the order the firmware does it (listing `E8D4-E8ED`): clear both control registers, set the data-direction registers (`$7F` for port A, `$1D` for port B), write `$3C` into both control registers to select the data registers, and write `$01` to both ports.
- **Our variables.** `FIRSTLN` is `$17`, the same as after the real firmware's start-up (Chapter 29); the escape-sequence state is zero; the key-held flag is cleared. Then `JSR CLEAR`.

The memory test and the ROM checksum are not here. The real firmware spends most of its start-up time on both, and the "Test Complete" message you see after it is their result.

The main loop

```
F853                  89  ; ---- the whole terminal
F853  BD FB 08        90  MAIN            JSR     GETKEY    ; A = key, or 0
F856  27 FB           91                  BEQ     MAIN
F858  BD F8 5D        92                  JSR     CHAR
F85B  20 F6           93                  BRA     MAIN
```

A terminal with no serial port is a very short program: read a key, act on it, repeat. `GETKEY` returns zero for "nothing new" (and a character once per press), and `BEQ MAIN` keeps polling. This is the loop of T06TYPE, with a better `CHAR`.

The character machine

```
F85D                  94  ; CHAR - act on the character in A
F85D  F6 01 13        95  CHAR            LDAB    ESCST
F860  27 03           96                  BEQ     CH0
F862                  97  ; inside an escape sequence
F862  7E F9 47        98                  JMP     INESC
F865  81 1B           99  CH0             CMPA    #$1B
```

```
F867  26 03          100                 BNE     CH1
F869  7E F9 41       101                 JMP     GOTESC
F86C  81 20          102  CH1            CMPA    #$20
F86E  25 06          103                 BCS     CH2       ; below space
F870  81 7F          104                 CMPA    #$7F
F872                 105  ; DEL and keys above
F872  24 29          106                 BCC     IGNORE
F874  20 03          107                 BRA     PUTCH
F876  7E F8 9E       108  CH2            JMP     CONTROL
F879                 109  ; a printable character: store it, move right
F879  36             110  PUTCH          PSHA
F87A  B6 01 10       111                 LDAA    ROW
F87D  B7 01 00       112                 STAA    ROWARG
F880  B6 01 11       113                 LDAA    COL
F883  B7 01 01       114                 STAA    COLARG
F886  32             115                 PULA
F887  BD FA CC       116                 JSR     PUTAT
F88A  B6 01 11       117  RIGHT          LDAA    COL
F88D  81 4F          118                 CMPA    #79
F88F  27 07          119                 BEQ     WRAP
F891  4C             120                 INCA
F892  B7 01 11       121                 STAA    COL
F895  7E F9 34       122                 JMP     CURSOR
F898  7F 01 11       123  WRAP           CLR     COL       ; past column 79:
F89B                 124  ; carriage return, line feed
F89B  20 27          125                 BRA     DOWN
F89D  39             126  IGNORE         RTS
```

`CHAR` is `LEFF6` (Chapter 28) in miniature. It has the same three outcomes, in the same order: **inside an escape sequence** (`ESCST` non-zero: `INESC`), **the escape character itself** (`CMPA #$1B`: `GOTESC`), **a control code** (below space: `CONTROL`), and **a printable character**. The test for the last one is two comparisons: below `$7F`. `$7F` itself (DEL) and the function codes of `KBDTAB` above it are ignored, as the real firmware does for DEL and as it does, differently, for the function codes.

(Two of the branches are `JMP` instead of `BNE/BEQ`. The first version of this listing used branches and the assembler refused it: a 6800 relative branch reaches only 127 bytes forward or 128 back, and the escape-sequence code is too far from `CHAR`. That is the most common error in any program longer than a screenful.)

`PUTCH` puts the character on the screen. It copies `ROW` and `COL` into the `ROWARG` and `COLARG` of Chapter 8, calls `PUTAT`, and **falls into `RIGHT`**, the cursor-right routine, so that the cursor advances after every character. `RIGHT` is the firmware's `LF2F0`: if `COL` is 79, `WRAP` clears it and goes to `DOWN`, which is a carriage return and a line feed; otherwise `COL` is incremented. Either way the routine ends in `CURSOR`, which copies `COL` and `ROW` into the two hardware cursor registers. It is the pattern of Chapter 29: change the variables, then bring the hardware cursor into step.

Note that `ROW` is a **memory row**, as in the real firmware (Chapter 29). `PUTAT` and `CALCADR` therefore work without change, because they have always taken a memory row: in all the earlier programs memory row and screen row were the same because the register was `$17`. FW31 is the first program in this book in which they are not.

Control codes

```
F89E                 127  ; ---- control codes
```

```
F89E  81 08          128  CONTROL         CMPA    #$08
F8A0  27 70          129                  BEQ     LEFT
F8A2  81 0A          130                  CMPA    #$0A
F8A4  27 1E          131                  BEQ     DOWN
F8A6  81 0B          132                  CMPA    #$0B
F8A8  27 5A          133                  BEQ     UP
F8AA  81 0C          134                  CMPA    #$0C
F8AC  27 DC          135                  BEQ     RIGHT
F8AE  81 0D          136                  CMPA    #$0D
F8B0  27 09          137                  BEQ     CR
F8B2  81 1E          138                  CMPA    #$1E
F8B4  27 6F          139                  BEQ     HOME
F8B6  81 1F          140                  CMPA    #$1F
F8B8  27 07          141                  BEQ     NEWLINE
F8BA  39             142                  RTS
F8BB  7F 01 11       143  CR              CLR     COL
F8BE  7E F9 34       144                  JMP     CURSOR
F8C1  7F 01 11       145  NEWLINE         CLR     COL
F8C4  B6 01 10       146  DOWN            LDAA    ROW
F8C7  B1 01 12       147                  CMPA    FIRSTLN
F8CA                 148  ; on the bottom line
F8CA  27 0D          149                  BEQ     SCROLL
F8CC  81 17          150                  CMPA    #23
F8CE  26 02          151                  BNE     DN1
F8D0                 152  ; row 23 is followed by 0
F8D0  86 FF          153                  LDAA    #$FF
F8D2  4C             154  DN1             INCA
F8D3  B7 01 10       155                  STAA    ROW
F8D6  7E F9 34       156                  JMP     CURSOR
F8D9                 157  ; scroll: no byte moves,
F8D9  B6 01 12       158  SCROLL          LDAA    FIRSTLN
F8DC                 159  ; the bottom line is
F8DC  4C             160                  INCA
F8DD                 161  ; the next memory row
F8DD  81 18          162                  CMPA    #24
F8DF  2D 01          163                  BLT     SC1
F8E1  4F             164                  CLRA
F8E2  B7 01 12       165  SC1             STAA    FIRSTLN
F8E5  B7 7F 06       166                  STAA    CRTC6
F8E8  B7 01 10       167                  STAA    ROW
F8EB  B7 01 00       168                  STAA    ROWARG
F8EE  7F 01 01       169                  CLR     COLARG
F8F1                 170  ; X = start of that row
F8F1  BD FA 72       171                  JSR     CALCADR
F8F4  C6 50          172                  LDAB    #80
F8F6  4F             173                  CLRA
F8F7  B7 70 00       174  SC2             STAA    HALT
F8FA  01             175                  NOP
F8FB                 176  ; blank it with zeros
F8FB  A7 00          177                  STAA    0,X
F8FD  08             178                  INX
F8FE  5A             179                  DECB
F8FF  26 F6          180                  BNE     SC2
F901  7E F9 34       181                  JMP     CURSOR
```

CONTROL is a chain of compares, for the seven codes of Chapter 28's control table that do something on this firmware's display (it leaves out ENQ, which is a reply to the host, and HT, which is a tab):

Code	Routine	Action
$08 BS	LEFT	COL down by one; from column 0 go to column 79 and up a row
$0A LF	DOWN	down a row, scrolling on the bottom line
$0B VT	UP	up a row, wrapping from the top to the bottom
$0C FF	RIGHT	right, with the wrap at column 79
$0D CR	CR	COL = 0
$1E RS	HOME	column 0, top line
$1F US	NEWLINE	column 0 and down a row

DOWN and the scroll

DOWN is the firmware's LF317 and it is the heart of the program. If ROW is not the bottom line, it adds one (wrapping 23 to 0). If it **is** the bottom line, SCROLL:

1. adds one to FIRSTLN, wrapping at 24;
2. stores it in the CRT controller's register 6 (STAA CRTC6) and in ROW: the line that was the top of the ring is now the bottom;
3. computes the address of that row with CALCADR, and fills its 80 cells with zeros (CLRA / STAA HALT / NOP / STAA 0,X).

Not one byte of text is moved. The whole program fits in a few lines because the hardware does the scrolling, and the only thing the software does is to rename which memory row is the bottom. This is exactly the code of Chapter 29, with the optional paths (protect mode, the two pages, the option bits) removed.

```
F904  B6 01 10       182  UP             LDAA    ROW
F907  26 02          183                 BNE     UP1
F909  86 18          184                 LDAA    #24
F90B  4A             185  UP1            DECA
F90C  B7 01 10       186                 STAA    ROW
F90F  7E F9 34       187                 JMP     CURSOR
F912  B6 01 11       188  LEFT           LDAA    COL
F915  27 07          189                 BEQ     LF1
F917  4A             190                 DECA
F918  B7 01 11       191                 STAA    COL
F91B  7E F9 34       192                 JMP     CURSOR
F91E  86 4F          193  LF1            LDAA    #79       ; left of column 0:
F920                 194  ; end of the row above
F920  B7 01 11       195                 STAA    COL
F923  20 DF          196                 BRA     UP
F925  7F 01 11       197  HOME           CLR     COL
F928                 198  ; the top line is the
F928  B6 01 12       199                 LDAA    FIRSTLN
F92B                 200  ; row after the bottom
F92B  4C             201                 INCA
F92C  81 18          202                 CMPA    #24
F92E  2D 01          203                 BLT     HM1
F930  4F             204                 CLRA
F931  B7 01 10       205  HM1            STAA    ROW
```

```
F934                  206  ; the hardware cursor
F934  B6 01 11        207  CURSOR          LDAA    COL
F937  B7 7F 0C        208                  STAA    CURCOL
F93A  B6 01 10        209                  LDAA    ROW
F93D  B7 7F 0D        210                  STAA    CURROW
F940  39              211                  RTS
```

`UP` decrements `ROW` and wraps 0 to 23. `LEFT` goes back one column; from column 0 it sets column 79 and *then goes up*, which is why it ends in `BRA UP`. `HOME` sets column 0 and `ROW` to `FIRSTLN + 1` (mod 24), the top line, and flows into `CURSOR`.

Escape sequences

```
F941                  212  ; ---- escape sequences: ESC + and ESC = row col
F941  86 01           213  GOTESC          LDAA    #1
F943  B7 01 13        214                  STAA    ESCST
F946  39              215                  RTS
F947  C1 01           216  INESC           CMPB    #1
F949  27 40           217                  BEQ     ESC1
F94B  C1 02           218                  CMPB    #2
F94D  27 1C           219                  BEQ     ESC2
F94F                  220  ; ESCST = 3: A is the column character
F94F  84 7F           221                  ANDA    #$7F
F951  80 20           222                  SUBA    #$20
F953  2C 01           223                  BGE     EC1
F955  4F              224                  CLRA
F956  81 50           225  EC1             CMPA    #80
F958  2D 02           226                  BLT     EC2
F95A  86 4F           227                  LDAA    #79
F95C  B7 01 11        228  EC2             STAA    COL
F95F  B6 01 14        229                  LDAA    NEWROW
F962  B7 01 10        230                  STAA    ROW
F965  7F 01 13        231                  CLR     ESCST
F968  7E F9 34        232                  JMP     CURSOR
F96B                  233  ; A is the row character
F96B  84 7F           234  ESC2            ANDA    #$7F
F96D  80 20           235                  SUBA    #$20
F96F  2C 01           236                  BGE     ER1
F971  4F              237                  CLRA
F972  81 18           238  ER1             CMPA    #24
F974  2D 02           239                  BLT     ER2
F976  86 17           240                  LDAA    #23
F978                  241  ; screen row -> memory row
F978  BB 01 12        242  ER2             ADDA    FIRSTLN
F97B  4C              243                  INCA
F97C  81 18           244                  CMPA    #24
F97E  2D 02           245                  BLT     ER3
F980  80 18           246                  SUBA    #24
F982  B7 01 14        247  ER3             STAA    NEWROW
F985  86 03           248                  LDAA    #3
F987  B7 01 13        249                  STAA    ESCST
F98A  39              250                  RTS
F98B                  251  ; the code after ESC
F98B  7F 01 13        252  ESC1            CLR     ESCST
F98E  81 2B           253                  CMPA    #$2B      ; '+'
F990  27 1B           254                  BEQ     CLEAR
```

```
F992                  255  ; grave accent: launch
F992  81 60           256                  CMPA    #$60
F994                  257  ; the plug-in in socket A
F994  27 0B           258                  BEQ     LAUNCH
F996  81 3D           259                  CMPA    #$3D      ; '='
F998  27 01           260                  BEQ     ES1X
F99A  39              261                  RTS
F99B  86 02           262  ES1X            LDAA    #2
F99D  B7 01 13        263                  STAA    ESCST
F9A0  39              264                  RTS
F9A1                  265  ; is there a "TJ" ROM?
F9A1  FE E0 00        266  LAUNCH          LDX     $E000
F9A4  8C 54 4A        267                  CPX     #$544A
F9A7  26 03           268                  BNE     NOLAUNCH
F9A9  7E E0 3C        269                  JMP     $E03C     ; yes: start it
F9AC  39              270  NOLAUNCH        RTS
```

The escape machine has four states, kept in ESCST:

ESCST	Meaning	Next character
0	normal text	
1	after ESC	+: clear; =: wait for the row; `: launch; anything else: forget it
2	after ESC =	the row character
3	after the row	the column character

GOTESC sets the state to 1. ESC1 handles the character after the ESC and resets the state to 0 first, so that an unknown code leaves the machine ready for ordinary text. For ESC = the two coordinates arrive in different calls to CHAR, which is what a *non-blocking* machine has to do: the real firmware does the same job with GETRX and a coroutine (Chapter 27), but FW31 has no scheduler, so the state is an explicit variable. It is the "state machine" alternative that Chapter 27 contrasted with the stackless coroutine.

The arithmetic of the two coordinate states is the real CURPOS (Chapter 29) in a different dress: subtract $20; replace a negative result by 0; clamp the row to 23 and the column to 79; and convert the screen row to a memory row by adding FIRSTLN + 1, modulo 24 (ER2, ER3). The row is held in NEWROW until the column arrives, and the two are stored together.

CLEAR fills the 1,920 visible cells with spaces ($20, the firmware's own choice, not the zero of Chapter 8's CLRSCR) and goes HOME.

Launching a plug-in

```
F9A1                  265  ; is there a "TJ" ROM?
F9A1  FE E0 00        266  LAUNCH          LDX     $E000
F9A4  8C 54 4A        267                  CPX     #$544A
F9A7  26 03           268                  BNE     NOLAUNCH
F9A9  7E E0 3C        269                  JMP     $E03C     ; yes: start it
F9AC  39              270  NOLAUNCH        RTS
F9AD  CE 80 00        271  CLEAR           LDX     #CRTRAM   ; fill the screen
F9B0  86 20           272                  LDAA    #$20      ; with spaces,
F9B2  B7 70 00        273  CL1             STAA    HALT
F9B5  01              274                  NOP
F9B6  A7 00           275                  STAA    0,X
```

```
F9B8  08            276                 INX
F9B9  8C 87 80      277                 CPX     #CRTRAM+1920
F9BC  26 F4         278                 BNE     CL1
F9BE  7E F9 25      279                 JMP     HOME      ; and go home
```

The last handler is `LAUNCH`, `ESC` followed by the grave accent. It is the whole of the real firmware's `ESCGRAVE` (Chapter 28): compare the two bytes at `$E000` with `$544A`, the letters `TJ`, and if they match, `JMP $E03C`. FW31 has no boot-time probe, no hook calls and no `PLUGFLG`: it needs none of that, since those are the real firmware's way of letting a plug-in change the firmware's own set-up, and FW31 does not have one.

What it has is the entry point of Chapter 5, which is why every program in this book *runs under FW31 unchanged*.

The vector table

```
FFF8  F9 C1         409                 FDB     IRQRTI
FFFA  F9 C1         410                 FDB     IRQRTI
FFFC  F9 C1         411                 FDB     IRQRTI
FFFE  F8 00         412                 FDB     RESET
```

IRQ, SWI and NMI all point to `IRQRTI`, a bare `RTI`; the reset vector points to `RESET`. FW31 never enables interrupts, so the IRQ vector is never used; NMI would be, if the hardware raised it, and an `RTI` is the harmless answer.

Testing it against the real thing

A new firmware has an unusual advantage as a test subject: there is already a correct one. The test feeds the same keystrokes to FW31 and to the real firmware (both ROM sets) in the emulator and then compares, **byte for byte**: the 1,920 cells of display memory, the scroll register, and the two cursor registers. Any difference at all is a failure. The six scenarios:

Scenario	Keystrokes	What it exercises
text	`ESC +`, a line of text, CR LF, a second line	printing, CR, LF
scroll	`ESC +` and 30 lines `L01`-`L30`	`DOWN`, `SCROLL`, the ring
wrap	`ESC +` and 90 characters	`RIGHT`, `WRAP`
esc=	several `ESC =` sequences, with ordinary, out-of-range and wrap coordinates	`ESC1`-`ESC2`, clamping
ctl	text, LF LF, then BS BS, FF, VT, RS, US	`LEFT`, `RIGHT`, `UP`, `HOME`, `NEWLINE`
ctl2	text, then VT VT VT FF FF BS, RS, US US US	`UP` past the top, wraps

All six give identical memory and registers on the 129867 and on the 129827. (The control codes BS, VT, FF, RS cannot be typed through the emulator's ASCII table, which covers only the first 79 scan codes, so the test presses them by scan code: `54`, `53`, `55`, `51`.)

The scroll scenario shows the ring at work in FW31 exactly as it did in the real firmware. After the 30 lines the display (memory row `FIRSTLN + 1` at the top) begins:

```
L08
L09
L10
```

```
L11
L12
L13
```

and `FIRSTLN` has become 6, as in Chapter 29. Then the plug-ins: `T01HELLO`, `T02FRAME`, `T03CHARS` and `T04TEXT`, and `PONG`, each launched from FW31 with `ESC` and the grave accent, give the same screens as when they were run stand-alone (PONG, its title screen). That tests that FW31 leaves the machine in a state that an unmodified plug-in can start from: the stack, the PIA and the CRT controller are all initialised, and the plug-in's own `INITCRT` and `CLRSCR` do the rest.

What this proves, and what it does not

It proves the logic: given the same inputs, FW31's display and cursor registers end up identical to the real firmware's, for the supported functions, on both revisions. It proves that the book's plug-ins can start from a firmware that is not Lear Siegler's.

It does not prove that the ROM would *boot a real terminal*. The unproven parts are exactly those where the emulator is silent: whether the CRT controller wants its registers loaded in a different order or after a delay; whether the display RAM must be cleared before the first access; whether the keyboard PIA has any settling time. The real firmware's start-up does a memory test and some waiting that FW31 omits. A hardware test is the first thing a reader with a terminal and a ROM programmer should do, and it should be done with a spare machine. The tests said so in every chapter; they say so here again.

Going further

Each of these is a chapter's worth of earlier material, and is a good exercise:

- **The bell.** `BELL` in Chapter 10 is four instructions. Add a case for `$07` in `CONTROL`.
- **Clear to end of line.** `ESC T` in Chapter 29: a count of cells left on the line and a fill loop. Add it to `ESC1`.
- **CTRL and alpha lock.** Chapter 28 gives the rules; they are the first thing the real `KBSCAN` does after the table lookup. `GETKEY` has the `SHIFT` test already, a few instructions above `KBDTAB`.
- **Auto-repeat.** The schedule is in Chapter 28. It needs a timer, and FW31 has none; a calibrated delay loop (Chapter 11) will do for a first version.
- **The serial port.** A polling receive loop reads `ACIA` and `ACIAD` (Chapter 12), handing each character to `CHAR`, and a polling transmit sends each typed key. Then FW31 is a terminal in the full sense.
- **Interrupts and the scheduler.** The last step is the biggest: FW31 would grow the stackless scheduler of Chapter 27, with `SLEEP` and `GETRX`, and the receive buffer of Chapter 27. It is what the other 5 KB of the real firmware are for.

Exercises

1. Add the bell. (Answer: in `CONTROL`, `CMPA #$07`, `BEQ BELL`; `BELL` reads `PIA1CRB`, masks bit 3 to zero, writes, sets bit 3, writes. It needs `RTS`.)
2. FW31's `LEFT` from column 0 goes to column 79 of the row above. What does the real firmware do at the top-left corner of the screen? (Answer: the same code path, `LF36F` then

`LF388`, takes the cursor to row 23: the bottom line. FW31 does the same, and the test `ctl2` covers it.)

3. FW31 clears with `$20` but scrolls in zeros. Why does it matter, and where does the real firmware do the same? (Answer: it does it to match the real firmware; the difference is invisible on the screen because a zero cell is a blank, but is visible in a memory dump, and the test compares memory.)
4. Where in FW31 would you put a call to a routine that reads the serial port, so that received characters appear on the screen? What must it do with a received `$1B`? (Answer: in `MAIN`, after `GETKEY`. It passes the character to `CHAR`, exactly as for a key; `CHAR` already treats `$1B` as the start of an escape sequence.)
5. The 2 KB ROM has 1,220 unused bytes. Estimate how many of the real firmware's 6 KB could be added without a new ROM, by looking at the listing size of the routines of Chapters 28 to 30. (This one has no fixed answer.)

Part V

Appendices

Appendix A

The 6800 opcode table

Every valid opcode of the Motorola 6800, as the assembler of this book emits it. There are 197. The two columns are one table folded in half. *Mode* is the addressing mode: `inh` inherent, `imm` immediate, `dir` direct (an address below `$100`), `ext` extended (a 16-bit address), `idx` indexed (an offset from X) and `rel` relative (a branch). *Len* is the number of bytes of the instruction, opcode included.

A note on this book's dialect (Chapter 3): an extended-mode instruction whose operand is below `$100` is not accepted as extended by this assembler: it is written out with `FCB`, and the listing shows the opcode and operand bytes. The firmware listing in Appendix F has many such lines (`FCB $7F,$00,$4D ; CLR $004D`).

Op	Mnemonic	Mode	Len	Op	Mnemonic	Mode	Len
01	NOP	inh	1	8B	ADDA	imm	2
06	TAP	inh	1	8C	CPX	imm	3
07	TPA	inh	1	8D	BSR	rel	2
08	INX	inh	1	8E	LDS	imm	3
09	DEX	inh	1	90	SUBA	dir	2
0A	CLV	inh	1	91	CMPA	dir	2
0B	SEV	inh	1	92	SBCA	dir	2
0C	CLC	inh	1	94	ANDA	dir	2
0D	SEC	inh	1	95	BITA	dir	2
0E	CLI	inh	1	96	LDAA	dir	2
0F	SEI	inh	1	97	STAA	dir	2
10	SBA	inh	1	98	EORA	dir	2
11	CBA	inh	1	99	ADCA	dir	2
16	TAB	inh	1	9A	ORAA	dir	2
17	TBA	inh	1	9B	ADDA	dir	2
19	DAA	inh	1	9C	CPX	dir	2
1B	ABA	inh	1	9E	LDS	dir	2
20	BRA	rel	2	9F	STS	dir	2
22	BHI	rel	2	A0	SUBA	idx	2
23	BLS	rel	2	A1	CMPA	idx	2
24	BCC	rel	2	A2	SBCA	idx	2
25	BCS	rel	2	A4	ANDA	idx	2
26	BNE	rel	2	A5	BITA	idx	2

Op	Mnemonic	Mode	Len	Op	Mnemonic	Mode	Len
27	BEQ	rel	2	A6	LDAA	idx	2
28	BVC	rel	2	A7	STAA	idx	2
29	BVS	rel	2	A8	EORA	idx	2
2A	BPL	rel	2	A9	ADCA	idx	2
2B	BMI	rel	2	AA	ORAA	idx	2
2C	BGE	rel	2	AB	ADDA	idx	2
2D	BLT	rel	2	AC	CPX	idx	2
2E	BGT	rel	2	AD	JSR	idx	2
2F	BLE	rel	2	AE	LDS	idx	2
30	TSX	inh	1	AF	STS	idx	2
31	INS	inh	1	B0	SUBA	ext	3
32	PULA	inh	1	B1	CMPA	ext	3
33	PULB	inh	1	B2	SBCA	ext	3
34	DES	inh	1	B4	ANDA	ext	3
35	TXS	inh	1	B5	BITA	ext	3
36	PSHA	inh	1	B6	LDAA	ext	3
37	PSHB	inh	1	B7	STAA	ext	3
39	RTS	inh	1	B8	EORA	ext	3
3B	RTI	inh	1	B9	ADCA	ext	3
3E	WAI	inh	1	BA	ORAA	ext	3
3F	SWI	inh	1	BB	ADDA	ext	3
40	NEGA	inh	1	BC	CPX	ext	3
43	COMA	inh	1	BD	JSR	ext	3
44	LSRA	inh	1	BE	LDS	ext	3
46	RORA	inh	1	BF	STS	ext	3
47	ASRA	inh	1	C0	SUBB	imm	2
48	ASLA	inh	1	C1	CMPB	imm	2
49	ROLA	inh	1	C2	SBCB	imm	2
4A	DECA	inh	1	C4	ANDB	imm	2
4C	INCA	inh	1	C5	BITB	imm	2
4D	TSTA	inh	1	C6	LDAB	imm	2
4F	CLRA	inh	1	C8	EORB	imm	2
50	NEGB	inh	1	C9	ADCB	imm	2
53	COMB	inh	1	CA	ORAB	imm	2
54	LSRB	inh	1	CB	ADDB	imm	2
56	RORB	inh	1	CE	LDX	imm	3
57	ASRB	inh	1	D0	SUBB	dir	2
58	ASLB	inh	1	D1	CMPB	dir	2
59	ROLB	inh	1	D2	SBCB	dir	2
5A	DECB	inh	1	D4	ANDB	dir	2
5C	INCB	inh	1	D5	BITB	dir	2
5D	TSTB	inh	1	D6	LDAB	dir	2
5F	CLRB	inh	1	D7	STAB	dir	2
60	NEG	idx	2	D8	EORB	dir	2
63	COM	idx	2	D9	ADCB	dir	2
64	LSR	idx	2	DA	ORAB	dir	2
66	ROR	idx	2	DB	ADDB	dir	2

Op	Mnemonic	Mode	Len	Op	Mnemonic	Mode	Len
67	ASR	idx	2	DE	LDX	dir	2
68	ASL	idx	2	DF	STX	dir	2
69	ROL	idx	2	E0	SUBB	idx	2
6A	DEC	idx	2	E1	CMPB	idx	2
6C	INC	idx	2	E2	SBCB	idx	2
6D	TST	idx	2	E4	ANDB	idx	2
6E	JMP	idx	2	E5	BITB	idx	2
6F	CLR	idx	2	E6	LDAB	idx	2
70	NEG	ext	3	E7	STAB	idx	2
73	COM	ext	3	E8	EORB	idx	2
74	LSR	ext	3	E9	ADCB	idx	2
76	ROR	ext	3	EA	ORAB	idx	2
77	ASR	ext	3	EB	ADDB	idx	2
78	ASL	ext	3	EE	LDX	idx	2
79	ROL	ext	3	EF	STX	idx	2
7A	DEC	ext	3	F0	SUBB	ext	3
7C	INC	ext	3	F1	CMPB	ext	3
7D	TST	ext	3	F2	SBCB	ext	3
7E	JMP	ext	3	F4	ANDB	ext	3
7F	CLR	ext	3	F5	BITB	ext	3
80	SUBA	imm	2	F6	LDAB	ext	3
81	CMPA	imm	2	F7	STAB	ext	3
82	SBCA	imm	2	F8	EORB	ext	3
84	ANDA	imm	2	F9	ADCB	ext	3
85	BITA	imm	2	FA	ORAB	ext	3
86	LDAA	imm	2	FB	ADDB	ext	3
88	EORA	imm	2	FE	LDX	ext	3
89	ADCA	imm	2	FF	STX	ext	3
8A	ORAA	imm	2				

Appendix B

The address ledger

Every address that the tutorials and PONG use, with its meaning and the *evidence*. The evidence column names the programs and firmware whose code was traced to an instruction that uses the address. A name in that column means: a working program or the firmware contains a read, a write or a jump to this address, found by following the program's control flow (the tool that did this is described in Chapter 24). It does **not** mean that every bit of a register has been explained; the meaning column says as much as the book can stand behind.

The evidence names are *Rubout*, *Shuttle Attack*, *Space Invaders*, *Cosmic Intruders* and *Outbreak* (the plug-in ROMs described in Chapter 24); *fw 129867* and *fw 129827* (the two firmware sets); *FW867* and *FW827* (the disassemblies in Appendix F and in Chapter 23); *PONG* and *tutorials* (this book's own programs, which are [EMU] only).

Address	Name	Meaning	Seen in
$0000	PLUGFLG	plug-in-active flag: INC'd by the boot probe, tested by the keyboard/scheduler setup; hooks CLR it	Rubout, Shuttle Attack, Space Invaders, Cosmic Intruders, Outbreak, fw 129867, fw 129827; FW867 E8BB-E8CD; Space Invaders hook CLR $0000
$0011	CRTBASE	CRT base address (2 bytes)	fw 129867
$0013	CURADR	cursor address (2 bytes)	fw 129867
$0015	FIRSTLN	scroll position: memory row on the bottom screen line, mirrored to VTAC reg 6	fw 129867, fw 129827
$0016	ROW	cursor row	Rubout, Outbreak, fw 129867, fw 129827
$0017	COL	cursor column	fw 129867
$0018	PROTECT	protect flag ($80 = protect on); ESC & sets, `ESC '` clears	fw 129867, fw 129827
$0019	ATTRMSK	attribute OR-mask applied to stored characters	fw 129867
$001A	MONMODE	monitor-mode flag ($80, ESC U)	fw 129827
$001D	KBLOCK	keyboard lock flag (`ESC #` sets, `ESC "` clears)	emulator only
$0045	ESCCHR	escape character, copied from the init table ($1B)	FW867 E9F8 init table

Address	Name	Meaning	Seen in
$004D	BLANK	blank character used by the clear routines ($20)	fw 129867; FW867 E9F8 init table
$00B1	BLITDST	BLITCPY destination pointer (2 bytes)	Rubout, Space Invaders, Cosmic Intruders, Outbreak, fw 129867; Rubout, Space Invaders, Cosmic Intruders, Outbreak, fw 129867 (via BLITCPY)
$00B3	BLITSRC	BLITCPY source pointer (2 bytes)	Rubout, Space Invaders, Cosmic Intruders, Outbreak, fw 129867; Rubout, Space Invaders, Cosmic Intruders, Outbreak, fw 129867 (via BLITCPY)
$0100	SCRATCH	$0100-$010F: free RAM; the tutorials use it for variables; PONG uses $0100-$0126	PONG, tutorials; the firmware stack/queues use $0100-$03FF region (TX queue $0100-$010B)
$03FF	STACKTOP	top of RAM; the working plug-ins set the stack here (LDS #$03FF)	Rubout/Shuttle/PONG E100 area LDS #$03FF
$7000	HALT	CRT access strobe: touch (write or read) before every CRTRAM access	Rubout, Shuttle Attack, Space Invaders, Cosmic Intruders, Outbreak, fw 129867, fw 129827; all programs
$7800	PIA1	PIA 1 port A: scan code out (bits 0-6), bit 7 in = key down	Rubout, Shuttle Attack, Cosmic Intruders, Outbreak, fw 129867, fw 129827
$7801	PIA1CRA	PIA 1 control register A	fw 129867, fw 129827
$7802	PIA1B	PIA 1 port B: bit 7 SHIFT in, bit 6 CTRL in, bit 0 alpha lock (out), bit 2 display page (out)	Rubout, Shuttle Attack, Outbreak, fw 129867, fw 129827
$7803	PIA1CRB	PIA 1 control register B: bit 3 pulsed = bell	Shuttle Attack, fw 129867, fw 129827
$7900	SWBNK2	switch bank 2: bits 2-4 printer-port setting	fw 129867, fw 129827
$7A00	ACIA	host ACIA control/status (write $03 = master reset)	Rubout, Shuttle Attack, Outbreak, fw 129867, fw 129827
$7A01	ACIAD	host ACIA data register	fw 129867, fw 129827
$7B00	PERROM	personality PROM window	FW867/827 reference $7B1E
$7C00	PACIA	printer ACIA control/status	Rubout, Shuttle Attack, Outbreak, fw 129867, fw 129827
$7C01	PACIAD	printer ACIA data register	fw 129867, fw 129827
$7D00	SWBNK3	switch bank 3 (bit 7 tested by the plug-in probe: BMI skips the plug-in)	Space Invaders, fw 129867, fw 129827
$7E00	SWBNK1	switch bank 1: bits 2-4 ACIA word format, bit 1 selects the VTAC register 4/5 values	Cosmic Intruders, fw 129867, fw 129827
$7E80	BAUDSW	low nibble = baud-rate index	FW867 FD75 baud table; read via indexed access

Address	Name	Meaning	Seen in
$7F00	VTAC0	video timing/address controller registers $7F00-$7F0E (init values in the reset code)	FW867 E807-E82F LDX #$7F00 … STAA n,X
$7F06	CRTC6	VTAC register 6: scroll register (memory row on the bottom line). Plug-ins set it to $17 = unscrolled	Rubout, Shuttle Attack, Cosmic Intruders, Outbreak, fw 129867, fw 129827
$7F0A	VTACA	VTAC register A (cleared first in reset)	fw 129867, fw 129827
$7F0C	CURCOL	VTAC register C: cursor column	Rubout, Shuttle Attack, Cosmic Intruders, Outbreak, fw 129867, fw 129827
$7F0D	CURROW	VTAC register D: cursor row (firmware writes it)	fw 129867, fw 129827
$7F0E	VTACE	VTAC register E (cleared first in reset)	fw 129867, fw 129827
$8000	CRTRAM	screen memory: cell = $8000 + row*80 + col (80 x 24 = 1920 cells)	all programs (clear loops, direct stores)
$8800	CRTPG2	second display page ($8800), chosen with PIA1B bit 2	FW867 F685 ($11 = $80/$88)
$E000	PLUGSIG	“TJ” signature of a plug-in ROM A ($544A)	fw 129867, fw 129827; FW867 E8BB LDX $E000; CPX #$544A
$E002	PLUGJMP	plug-in entry used by games that share routines (Cosmic Intruders calls E002/E009/E010/E017)	Cosmic Intruders
$E03C	PLUGENT	ESC ‘ entry for 129867 (129827: $E03E)	FW867 F22C -> JMP $E03C
$E07D	PLUGHK1	129867 boot hook (JSR from E8C8)	fw 129867; FW867 E8C8 JSR $E07D
$E087	PLUGHK2	129827 boot hook	fw 129827; FW827
$E0CA	PLUGHK3	129867 task-table hook: plug-in may install its own scheduler table	fw 129867; FW867 EC32 JSR $E0CA
$E8D0	NOPLUG	stock boot continuation (what a plug-in hook returns to)	Space Invaders, Cosmic Intruders; Space Invaders, Cosmic Intruders
$EFAD	XPLUSA	X = X + A (firmware routine)	Cosmic Intruders; FW867; Cosmic Intruders
$F0DA	BELL	ring the bell (firmware routine)	Cosmic Intruders, fw 129867; FW867; Cosmic Intruders
$F6BB	BLITCPY	copy B bytes from ($B3) to ($B1) with the HALT strobe	Rubout, Space Invaders, Cosmic Intruders, Outbreak, fw 129867; Rubout, Space Invaders, Cosmic Intruders, Outbreak
$F740	CLRSCR	ESC + handler (CLRSCR): clear screen (firmware routine)	Space Invaders, Cosmic Intruders; Space Invaders, Cosmic Intruders

Zero-page variables of the 129827

The zero-page addresses in the table above are those of the 129867 firmware. The 129827 stores the same state at other addresses (for example its `BLITCPY` takes its pointers from `$7D` and `$7F`, not `$B1` and `$B3`; see Chapter 23). A program that calls firmware routines must not assume the same zero-page layout on both sets.

Socket names

Two sources disagree about the board designators of the ROM sockets. The notes that came with the emulator label the four sockets `U62` to `U65` as ROMs A to D; the firmware's configuration list (`Fware.cfg`) names the firmware chips `U-65`, `U-66` and `U-67`. This book does not resolve the disagreement. It names a socket by its address range and a firmware ROM by its part-number suffix, as Chapter 6 explains, and it uses "socket A" for the plug-in socket at `$E000`. If you fit an EPROM, check the address decoding on your own board.

Appendix C

ESC and control-code tables

The firmware's character handler (Chapter 28) has two jump tables, both read from the ROM image of the 129867 set by a program and printed here by it, so that the addresses are copies of the ROM and not transcriptions. The handler column gives the name used in this book where there is one, and the listing label (Lxxxx) otherwise. The *Meaning* column says what a chapter of this book established about the handler, by reading it, running it, or both. **"Not read" means exactly that**: the table entry exists and the book does not say what the handler does. The names of the codes as keys on the ADM-31 are not given, because the book did not verify them.

An ESC sequence is ESC ($1B) followed by one code. A few handlers read further characters (ESC = takes a row and a column; ESC D takes F or H). Codes $10 to $1F are not typed by a host: they are reached through the function codes $90 to $9F that the keyboard handler produces (Chapter 28).

The ESC table (ESCTAB, $EACA)

Every code from $10 to $7F that is not ESCNOP. Entry address = $EACA + 2 x (code - $10). "Read" means a chapter of this book traced the handler; "not read" means the address is the table's and nothing more is claimed.

Code	Char	Entry	Handler	Address	Meaning
$11		$EACC	LEFEA	$EFEA	not read
$12		$EACE	LEF4A	$EF4A	not read
$13		$EAD0	LEF4E	$EF4E	not read
$14		$EAD2	LEFC6	$EFC6	not read
$18		$EADA	LEF61	$EF61	toggle alpha lock (via key code $98)
$1D		$EAE4	LEFC6	$EFC6	not read
$1E		$EAE6	SENDBRK	$EF6D	transmit break (via key code $9E)
$1F		$EAE8	LEFA5	$EFA5	not read
$22	"	$EAEE	LF0AF	$F0AF	unlock keyboard
$23	#	$EAF0	LF0AA	$F0AA	lock keyboard
$26	&	$EAF6	LF060	$F060	protect on (PROTECT = $80)
$27	'	$EAF8	LF070	$F070	protect off (CLR $0018)

Code	Char	Entry	Handler	Address	Meaning
$28	(	$EAFA	LF079	$F079	clear attribute mask
$29	)	$EAFC	LF074	$F074	set attribute mask ($80)
$2A	*	$EAFE	LF767	$F767	not read
$2B	+	$EB00	CLRSCR	$F740	clear screen
$2C	,	$EB02	LF763	$F763	not read
$2D	-	$EB04	LF567	$F567	not read
$2E	.	$EB06	LF0F3	$F0F3	not read
$2F	/	$EB08	LF9D1	$F9D1	reply request 9: page + cursor
$30	0	$EB0A	LF501	$F501	not read
$31	1	$EB0C	LF50F	$F50F	not read
$32	2	$EB0E	LF51B	$F51B	not read
$33	3	$EB10	LF553	$F553	not read
$34	4	$EB12	LF9B5	$F9B5	reply request (Ch. 30)
$35	5	$EB14	LF9BD	$F9BD	reply request (Ch. 30)
$36	6	$EB16	LF9B9	$F9B9	reply request (Ch. 30)
$37	7	$EB18	LF9C1	$F9C1	reply request (Ch. 30)
$38	8	$EB1A	LF237	$F237	not read
$39	9	$EB1C	LF23C	$F23C	not read
$3A	:	$EB1E	LF731	$F731	not read
$3B	;	$EB20	LF73B	$F73B	not read
$3D	=	$EB24	CURPOS	$F5AA	cursor addressing: row, column
$3F	?	$EB28	LF9CD	$F9CD	reply request 8: cursor report
$40	@	$EB2A	LF215	$F215	print entry (via LFDAF)
$41	A	$EB2C	LF221	$F221	not read
$42	B	$EB2E	LF07D	$F07D	clear bit 6 of $41
$43	C	$EB30	LF083	$F083	set bit 6 of $41
$44	D	$EB32	LF08A	$F08A	bit 7 of $41, cleared or set by the next character F or H
$45	E	$EB34	INSLINE	$F874	insert line
$47	G	$EB38	LF0E8	$F0E8	not read
$49	I	$EB3C	LF458	$F458	not read
$4A	J	$EB3E	LF631	$F631	not read
$4B	K	$EB40	LF631	$F631	not read
$4C	L	$EB42	LF211	$F211	print entry (via LFDAF)
$50	P	$EB4A	LF20D	$F20D	print entry (via LFDAF)
$51	Q	$EB4C	LF824	$F824	not read
$52	R	$EB4E	DELLINE	$F912	delete line
$53	S	$EB50	LF9C5	$F9C5	reply request (Ch. 30)
$54	T	$EB52	CLREOL	$F7D5	clear to end of line
$55	U	$EB54	LF0A1	$F0A1	monitor mode on
$57	W	$EB58	LF84B	$F84B	not read
$58	X	$EB5A	LF0A6	$F0A6	monitor mode off
$59	Y	$EB5C	LF6D5	$F6D5	not read

Code	Char	Entry	Handler	Address	Meaning
$60	`	$EB6A	ESCGRAVE	$F22C	LAUNCH plug-in A (if "TJ" at $E000)
$69	i	$EB7C	LF3AA	$F3AA	not read
$6C	l	$EB82	LF221	$F221	not read
$6D	m	$EB84	LF240	$F240	not read
$6F	o	$EB88	LF14E	$F14E	not read
$70	p	$EB8A	LF211	$F211	print entry (via LFDAF)
$71	q	$EB8C	LF0D1	$F0D1	store $02 in $1E
$72	r	$EB8E	LF0D6	$F0D6	CLR $1E
$73	s	$EB90	LF9C9	$F9C9	reply request (Ch. 30)
$74	t	$EB92	LF81D	$F81D	not read
$75	u	$EB94	LF0A6	$F0A6	monitor mode off (as ESC X)
$76	v	$EB96	LF0B3	$F0B3	set SPLIT ($42) under conditions (Ch. 29)
$77	w	$EB98	LF0CD	$F0CD	CLR $0042 (SPLIT off)
$79	y	$EB9C	LF6E1	$F6E1	not read

68 of the 112 entries have a handler; the other 44 point to ESCNOP ($EBEA, a lone RTS).

The control table (CTLTAB, $EBAA)

Code	Entry	Handler	Address	Meaning
$05	$EBB4	LF9D5	$F9D5	ENQ: request 10, answerback
$07	$EBB8	BELL	$F0DA	BEL
$08	$EBBA	LF28C	$F28C	BS: cursor left
$09	$EBBC	LF3AA	$F3AA	HT: tab (not traced)
$0A	$EBBE	LF27D	$F27D	LF: down, scroll at the bottom
$0B	$EBC0	LF293	$F293	VT: up
$0C	$EBC2	LF272	$F272	FF: right
$0D	$EBC4	LF2A1	$F2A1	CR
$1E	$EBE6	LF259	$F259	RS: home
$1F	$EBE8	LF29A	$F29A	US: column 0 and down

10 of the 32 control codes have a handler; the rest point to ESCNOP.

Appendix D

Scan codes and the keyboard table

The firmware's keyboard table KBDTAB is 176 bytes at $EA1A in the 129867 set ($EB1B in the 129827): two bytes for each of the 88 scan codes, unshifted first, shifted second (Chapter 9). The table below was printed from the ROM image by a program. Each entry is the code that the firmware passes on; a value of $80 or more is a *special* code (a function key, a cursor key or a keypad key) and is handled by KEYHAND (Chapter 28), not typed as a character.

How to use it from a plug-in: read the scan code from PIA1 (Chapter 9), and either copy this table into the program (as the tutorials do) or look the scan code up here when you write a key table of your own (as Chapter 16 does for PONG). SHIFT is bit 7 and CTRL bit 6 of PIA1B.

The scan codes that PONG reads (Chapter 20): $53 and $52 (up and down), $20 and $31 (the letters w and s), $54 and $55 (left and right, which are Rubout's keys), $28 (p, pause), and $4B or $40 (SPACE or CLEAR, serve). The launch key is the shifted form of the key that types @ (scan $3B), which gives the grave accent `. The apostrophe is the shifted form of the 7 key (scan $14).

Scan	Unshifted	Shifted	Scan	Unshifted	Shifted
$00	$D1 special	$D1 special	$2C	$84 special	$84 special
$01	$D7 special	$D7 special	$2D	$85 special	$85 special
$02	$C5 special	$C5 special	$2E	$86 special	$86 special
$03	$D2 special	$D2 special	$2F	$09 HT	$09 HT
$04	$D4 special	$F4 special	$30	$61 a	$41 A
$05	$D9 special	$F9 special	$31	$73 s	$53 S
$06	$32 2	$22 "	$32	$64 d	$44 D
$07	$1B ESC	$1B ESC	$33	$66 f	$46 F
$08	$7F DEL	$7F DEL	$34	$67 g	$47 G
$09	$9E special	$9F special	$35	$68 h	$48 H
$0A	$D0 special	$F0 special	$36	$6A j	$4A J
$0B	$B4 special	$B6 special	$37	$6B k	$4B K
$0C	$92 special	$93 special	$38	$6C l	$4C L
$0D	$31 1	$21 !	$39	$3B ;	$2B +
$0E	$71 q	$51 Q	$3A	$3A :	$2A *
$0F	$98 special	$98 special	$3B	$40 @	$60 `
$10	$33 3	$23 #	$3C	$0A LF	$0A LF
$11	$34 4	$24 $	$3D	$82 special	$82 special
$12	$35 5	$25 %	$3E	$83 special	$83 special

Scan	Unshifted	Shifted	Scan	Unshifted	Shifted
$13	$36 6	$26 &	$3F	$81 special	$81 special
$14	$37 7	$27 '	$40	$90 special	$91 special
$15	$38 8	$28 (	$41	$7A z	$5A Z
$16	$39 9	$29)	$42	$78 x	$58 X
$17	$30 0	$5F _	$43	$63 c	$43 C
$18	$2D -	$3D =	$44	$76 v	$56 V
$19	$5E ^	$7E ~	$45	$62 b	$42 B
$1A	$5C \	$7C |	$46	$6E n	$4E N
$1B	$09 HT	$C9 special	$47	$6D m	$4D M
$1C	$87 special	$87 special	$48	$2C ,	$3C <
$1D	$88 special	$88 special	$49	$2E .	$3E >
$1E	$89 special	$89 special	$4A	$2F /	$3F ?
$1F	$2D -	$2D -	$4B	$20 SP	$20 SP
$20	$77 w	$57 W	$4C	$1F US	$CA special
$21	$65 e	$45 E	$4D	$30 0	$30 0
$22	$72 r	$52 R	$4E	$2E .	$2E .
$23	$74 t	$54 T	$4F	$0D CR	$0D CR
$24	$79 y	$59 Y	$50	$9D special	$94 special
$25	$75 u	$55 U	$51	$1E RS	$1E RS
$26	$69 i	$49 I	$52	$0A LF	$0A LF
$27	$6F o	$4F O	$53	$0B VT	$0B VT
$28	$70 p	$50 P	$54	$08 BS	$08 BS
$29	$5B [	$7B {	$55	$0C FF	$0C FF
$2A	$5D]	$7D }	$56	$E3 special	$00 NUL
$2B	$0D CR	$0D CR	$57	$2C ,	$2C ,

Appendix E

PONG: the complete listing

The complete assembly listing of `PONG.ASM`, as the assembler (Appendix K) prints it: the address, the object bytes, the line number, the source line (cut at 53 characters), then the symbol table and the cross-reference table. The ROM image assembled from it is 2,048 bytes with MD5 `04dc28290ac9edd938f9382e00991dc0`. PONG has been run on a real ADM-31 and works (Chapter 22).

```
0000                     2  ; ================================================
0000                     3  ; PONG
0000                     4  ; ADM-31 plugin ROM A
0000                     5  ; By: Mickey Lawless
0000                     6  ; $E000-$E7FF, 2048 B
0000                     7  ; ================================================
0000                     8  ; You (left paddle) against the computer.  First to
0000                     9  ; 7.
0000                    10  ; Keys: W/S or UP/DOWN move, SPACE/CLEAR serve, P
0000                    11  ; pause.
0000                    12  ;
0000                    13  ;  MEMORY MAP
0000                    14  ;  ------------------------------------------------
0000                    15  ;    $0100-$0126   game variables (RAM EQUs below)
0000                    16  ;    $0127-$03FE   free
0000                    17  ;    $03FF         top of stack
0000                    18  ; $7000         HALT strobe, write before touching
0000                    19  ; CRTRAM
0000                    20  ; $7800         PIA1    keyboard scan code out, bit 7
0000                    21  ; in = key down
0000                    22  ;    $7802         PIA1B
0000                    23  ;    $7803         PIA1CRB bit 3 = bell
0000                    24  ;    $7F06         CRTC6   cursor register
0000                    25  ;    $7F0C         CURCOL  cursor column register
0000                    26  ; $8000-$877F   CRTRAM  80 x 24, cell address = $8000
0000                    27  ; + row*80 + col
0000                    28  ;    $E000-$E7FF   ROM A   this program
0000                    29  ;
0000                    30  ;  ENTRY POINTS
0000                    31  ;  ------------------------------------------------
0000                    32  ;    $E000   "TJ" plugin signature
0000                    33  ; $E002   JMP $E03C       lets a plugin-only emulator
0000                    34  ; boot start the game
0000                    35  ; $E03C   NOP,NOP,NOP     ESC-` (grave) lands here
```

```
0000                    36  ; (129867) or at $E03E (129827)
0000                    37  ;    $E03F   JMP RESET       both slide into this
0000                    38  ; $E07D   PLUGCB2         boot hook, 129867: CLR
0000                    39  ; $0000, back to stock boot
0000                    40  ; $E087   STUB87          boot hook, 129827: same,
0000                    41  ; for that firmware
0000                    42  ; $E094   STUB94          RTS stub (never reached
0000                    43  ; once the flag is clear)
0000                    44  ;    $E0C3   STUBC3          RTS stub
0000                    45  ;    $E0CA   PLUGCB3         RTS stub
0000                    46  ;    $E100   RESET           game starts here
0000                    47  ;
0000                    48  ; ================================================
0000                    49
0000                    50  ; VSYNC strobe (write before CRTRAM access)
0000  =7000              51  HALT            EQU     $7000
0000                    52  ; keyboard scan / strobe (bit7 = key down)
0000  =7800              53  PIA1            EQU     $7800
0000                    54  ; PIA1 port B, keyboard control bits
0000  =7802              55  PIA1B           EQU     $7802
0000  =7803              56  PIA1CRB         EQU     $7803     ; bit 3 = bell
0000                    57  ; CRT video RAM base, 80 x 24 cells
0000  =8000              58  CRTRAM          EQU     $8000
0000                    59  ; CRT controller register (cursor row)
0000  =7F06              60  CRTC6           EQU     $7F06
0000                    61  ; cursor column register
0000  =7F0C              62  CURCOL          EQU     $7F0C
0000                    63
0000  =0023              64  CH_PAD          EQU     $23       ; '#'  paddle
0000                    65  ; '='  top / bottom wall
0000  =003D              66  CH_WALL         EQU     $3D
0000  =003A              67  CH_NET          EQU     $3A       ; ':'  centre line
0000  =004F              68  CH_BALL         EQU     $4F       ; 'O'  ball
0000                    69
0000  =0007              70  WINSC           EQU     7         ; points to win
0000                    71  ; tick delay at serve (DEX/BNE = 8 clocks)
0000  =1900              72  DLYSTART        EQU     6400
0000                    73
0000                    74  ; ---- RAM ($0100-$03FF is free; stack is at $03FF,
0000                    75  ; same as the other games) --
0000                    76  ; human paddle: +0 top row, +1 column
0000  =0100              77  HPAD            EQU     $0100
0000  =0102              78  CPAD            EQU     $0102     ; computer paddle
0000  =0104              79  BROW            EQU     $0104     ; ball row
0000  =0105              80  BCOL            EQU     $0105     ; ball column
0000                    81  ; bit7 set = moving right
0000  =0106              82  BHDIR           EQU     $0106
0000                    83  ; bit7 set = moving down
0000  =0107              84  BVDIR           EQU     $0107
0000                    85  ; vertical step every N ticks (0 = flat)
0000  =0108              86  BVRATE          EQU     $0108
0000                    87  ; ticks left before the next row step
0000  =0109              88  BVCNT           EQU     $0109
0000                    89  ; 2 bytes, ball's CRTRAM address
0000  =010A              90  CURADR          EQU     $010A
0000                    91  ; 2 bytes, previous ball address
0000  =010C              92  OLDADR          EQU     $010C
```

```
0000                    93  ; ball column before this move
0000  =010E             94  OLDCOL          EQU     $010E
0000  =010F             95  SCOREH          EQU     $010F     ; your score, 0-7
0000                    96  ; computer's score, 0-7
0000  =0110             97  SCOREC          EQU     $0110
0000                    98  ; tick counter (low bit paces the CPU)
0000  =0111             99  TICK            EQU     $0111
0000                   100  ; computer's aim error, -3..+3
0000  =0112            101  AIMOFF          EQU     $0112
0000                   102  ; 0 none, 1 human scored, 2 CPU scored
0000  =0113            103  POINT           EQU     $0113
0000                   104  ; $80 serve toward CPU, $7F toward human
0000  =0114            105  LOSER           EQU     $0114
0000                   106  ; 2 bytes, tick delay
0000  =0115            107  DLYCNT          EQU     $0115
0000                   108  ; 2 bytes, LFSR state
0000  =0117            109  SEED            EQU     $0117
0000                   110  ; bounce-retest limiter
0000  =0119            111  TRIES           EQU     $0119
0000                   112  ; row argument for CALCADR / PUTAT
0000  =011A            113  ROWARG          EQU     $011A
0000                   114  ; column argument for CALCADR / PUTAT
0000  =011B            115  COLARG          EQU     $011B
0000  =011C            116  ADRPTR          EQU     $011C     ; 2 bytes
0000  =011E            117  TMPA            EQU     $011E     ; scratch byte A
0000  =011F            118  TMPB            EQU     $011F     ; scratch byte B
0000  =0120            119  TMPX            EQU     $0120     ; 2 bytes
0000  =0122            120  KPTR            EQU     $0122     ; 2 bytes
0000  =0124            121  PSRC            EQU     $0124     ; 2 bytes
0000                   122  ; move direction, $FF up / $01 down
0000  =0126            123  MPDIR           EQU     $0126
0000                   124
0000                   125  ; ================================================
0000                   126  ; assemble from the start of ROM A
E000                   127                  ORG     $E000
E000                   128  ; "TJ" plugin signature
E000  54 4A            129                  FCB     $54,$4A
E002                   130  KEYENTRY
E002                   131  ; $E002: same entry as ROM C's jump, so an
E002  7E E0 3C         132                  JMP     $E03C
E005                   133  ; emulator that starts a plugin at $E000 (ADM31EMU
E005                   134  ; --boot plugin) runs the
E005                   135  ; game.  We SEI and poll the keys ourselves, so
E005                   136  ; nothing else needs KEY_ENTRY.
E005                   137
E005                   138  ; ESC-` (grave, $60) entry sled.  The 129867 firmware
E005                   139  ; JMPs to $E03C, the 129827 firmware
E005                   140  ; JMPs to $E03E; NOPs let either one fall through to
E005                   141  ; the same JMP (the same
E005                   142  ; trick Shuttle Attack uses at $E03C).
E005                   143  ; 129867 firmware enters here
E03C                   144                  ORG     $E03C
E03C                   145  ; $E03C: slide into the JMP
E03C  01               146                  NOP
E03D                   147  ; $E03D: slide into the JMP
E03D  01               148                  NOP
E03E                   149  ; $E03E: 129827 firmware enters here
```

```
E03E  01            150                 NOP
E03F                151  ; $E03F: start the game
E03F  7E E1 00      152                 JMP     RESET
E042                153
E042                154  ; Boot hooks.  At power-up both firmwares see "TJ",
E042                155  ; INC the plugin flag at
E042                156  ; $0000 and JSR into the plugin ($E07D on 129867,
E042                157  ; $E087 on 129827).  With the
E042                158  ; flag left set the firmware skips its own task-table
E042                159  ; setup and expects the
E042                160  ; plugin to supply one, so a plugin that does not
E042                161  ; must clear the flag and send
E042                162  ; the firmware down its normal (no-plugin) boot path.
E042                163  ; Space Invaders and
E042                164  ; Cosmic Intruder do exactly this: CLR $0000, drop
E042                165  ; the return address, and
E042                166  ; JMP to the firmware's "no plugin" label ($E8D0 /
E042                167  ; $E9D0).
E042                168  ; 129867 firmware boot hook
E07D                169                 ORG     $E07D
E07D                170  ; clear the plugin flag: stock firmware runs
E07D  7F 00 00      171  PLUGCB2         CLR     $0000
E080                172  ; drop the JSR return address (high byte)
E080  32            173                 PULA
E081                174  ; drop the JSR return address (low byte)
E081  32            175                 PULA
E082                176  ; 129867 "no plugin" boot path
E082  7E E8 D0      177                 JMP     $E8D0
E085                178  ; 129827 firmware boot hook
E087                179                 ORG     $E087
E087                180  ; clear the plugin flag: stock firmware runs
E087  7F 00 00      181  STUB87          CLR     $0000
E08A                182  ; drop the JSR return address (high byte)
E08A  32            183                 PULA
E08B                184  ; drop the JSR return address (low byte)
E08B  32            185                 PULA
E08C                186  ; 129827 "no plugin" boot path
E08C  7E E9 D0      187                 JMP     $E9D0
E08F                188
E08F                189  ; With the flag cleared the firmware never calls the
E08F                190  ; remaining callbacks; they
E08F                191  ; stay as RTS in case something does.
E08F                192  ; 129827 firmware ESC callback
E094                193                 ORG     $E094
E094                194  ; stub: return at once
E094  39            195  STUB94          RTS
E095                196  ; 129827 firmware ESC callback
E0C3                197                 ORG     $E0C3
E0C3                198  ; stub: return at once
E0C3  39            199  STUBC3          RTS
E0C4                200  ; 129867 firmware task-table callback
E0CA                201                 ORG     $E0CA
E0CA                202  ; stub: return at once
E0CA  39            203  PLUGCB3         RTS
E0CB                204
E0CB                205  ; ================================================
E0CB                206  ; RESET -- one-time init (same hardware setup as
```

```
E0CB                207  ; Rubout's RESET / RESET_011)
E0CB                208  ; ================================================
E0CB                209  ; main program starts here
E100                210                  ORG     $E100
E100                211  RESET
E100                212  ; own the machine, we poll the keys
E100  0F            213                  SEI
E101                214  ; stack pointer to the top of RAM
E101  8E 03 FF      215                  LDS     #$03FF
E104                216  ; first half of the cursor-hide pair
E104  86 17         217                  LDAA    #$17
E106                218  ; write it to the CRT controller
E106  B7 7F 06      219                  STAA    CRTC6
E109                220  ; complement it: A becomes $E8
E109  43            221                  COMA
E10A                222  ; $E8 = cursor parked off screen
E10A  B7 7F 0C      223                  STAA    CURCOL
E10D  B6 78 02      224                  LDAA    PIA1B     ; read PIA1 port B
E110                225  ; clear bit 2, as Rubout does
E110  84 FB         226                  ANDA    #$FB
E112                227  ; write PIA1 port B back
E112  B7 78 02      228                  STAA    PIA1B
E115                229  ; non-zero seed for the random LFSR
E115  CE AC E1      230                  LDX     #$ACE1
E118                231  ; store the 16-bit seed
E118  FF 01 17      232                  STX     SEED
E11B                233  ; tick counter starts at 0
E11B  7F 01 11      234                  CLR     TICK
E11E                235  ; computer's aim error starts at 0
E11E  7F 01 12      236                  CLR     AIMOFF
E121                237  NEWMATCH
E121                238  ; blank the whole screen
E121  BD E4 99      239                  JSR     CLRSCR
E124  7F 01 0F      240                  CLR     SCOREH    ; your score = 0
E127                241  ; computer's score = 0
E127  7F 01 10      242                  CLR     SCOREC
E12A                243  ; $80 = serve toward the computer
E12A  86 80         244                  LDAA    #$80
E12C                245  ; first serve goes toward the computer
E12C  B7 01 14      246                  STAA    LOSER
E12F                247  ; draw court, scores, paddles
E12F  BD E4 ED      248                  JSR     DRAWFLD
E132                249  SERVEPT
E132                250  ; clear the message row
E132  BD E4 AA      251                  JSR     MSGCLR
E135  86 17         252                  LDAA    #23       ; message row 23
E137                253  ; row argument for PUTSTR
E137  B7 01 1A      254                  STAA    ROWARG
E13A  86 0F         255                  LDAA    #15       ; message column 15
E13C                256  ; column argument for PUTSTR
E13C  B7 01 1B      257                  STAA    COLARG
E13F                258  ; X points at the serve prompt
E13F  CE E6 14      259                  LDX     #SSERVE
E142  BD E4 77      260                  JSR     PUTSTR    ; print the prompt
E145                261  ; wait for SPACE/CLEAR (spins the RNG)
E145  BD E4 3B      262                  JSR     WAITSP
E148  BD E4 AA      263                  JSR     MSGCLR    ; erase the prompt
```

```
E14B                    264  ; place the ball, pick its direction
E14B  BD E1 BB          265                  JSR     SERVE
E14E                    266
E14E                    267  ; ---- main game loop: one tick = poll, computer,
E14E                    268  ; ball, delay ---------------
E14E                    269  GAME
E14E                    270  ; read the keys, move your paddle
E14E  BD E3 58          271                  JSR     POLLHUM
E151                    272  ; computer moves its paddle
E151  BD E3 DD          273                  JSR     CPUAI
E154                    274  ; move the ball a cell, test collisions
E154  BD E2 12          275                  JSR     BALLMOV
E157                    276  ; X = current tick delay count
E157  FE 01 15          277                  LDX     DLYCNT
E15A                    278  ; count the delay down ...
E15A  09                279  GDLY            DEX
E15B                    280  ; ... until X reaches 0
E15B  26 FD             281                  BNE     GDLY
E15D  7C 01 11          282                  INC     TICK      ; count this tick
E160                    283  ; has anyone scored?
E160  7D 01 13          284                  TST     POINT
E163                    285  ; no: run the next tick
E163  27 E9             286                  BEQ     GAME
E165                    287
E165                    288  ; ---- somebody scored ------------------------------
E165                    289  ; -------------------------
E165                    290  ; sound the bell for the point
E165  BD E5 DE          291                  JSR     BEEP
E168                    292  ; A = 1 you scored, 2 computer scored
E168  B6 01 13          293                  LDAA    POINT
E16B  81 01             294                  CMPA    #1        ; was it you?
E16D                    295  ; no: the computer scored
E16D  26 0A             296                  BNE     GP_CPU
E16F                    297  ; add a point for you
E16F  7C 01 0F          298                  INC     SCOREH
E172                    299  ; $80 = next serve goes to the computer
E172  86 80             300                  LDAA    #$80
E174                    301  ; computer lost the point: serve to it
E174  B7 01 14          302                  STAA    LOSER
E177                    303  ; skip the computer's branch
E177  20 08             304                  BRA     GP_SHOW
E179                    305  ; add a point for the computer
E179  7C 01 10          306  GP_CPU          INC     SCOREC
E17C                    307  ; $7F = next serve goes to you
E17C  86 7F             308                  LDAA    #$7F
E17E                    309  ; you lost the point: serve to you
E17E  B7 01 14          310                  STAA    LOSER
E181                    311  ; redraw both scores
E181  BD E4 B3          312  GP_SHOW         JSR     SHOWSC
E184  B6 01 0F          313                  LDAA    SCOREH    ; A = your score
E187                    314  ; reached the winning score?
E187  81 07             315                  CMPA    #WINSC
E189                    316  ; no: check the computer
E189  26 05             317                  BNE     GP_N1
E18B                    318  ; X points at the "YOU WIN" text
E18B  CE E6 45          319                  LDX     #SWIN
E18E  20 0F             320                  BRA     ENDGAME   ; go show it
```

```
E190                    321  ; A = computer's score
E190  B6 01 10          322  GP_N1           LDAA    SCOREC
E193                    323  ; reached the winning score?
E193  81 07             324                  CMPA    #WINSC
E195  26 05             325                  BNE     GP_NEXT   ; no: play on
E197                    326  ; X points at the "CPU WINS" text
E197  CE E6 60          327                  LDX     #SLOSE
E19A  20 03             328                  BRA     ENDGAME   ; go show it
E19C                    329  ; serve the next point
E19C  7E E1 32          330  GP_NEXT         JMP     SERVEPT
E19F                    331
E19F                    332  ENDGAME
E19F                    333  ; save the text pointer (MSGCLR uses X)
E19F  FF 01 22          334                  STX     KPTR
E1A2                    335  ; clear the message row
E1A2  BD E4 AA          336                  JSR     MSGCLR
E1A5  86 17             337                  LDAA    #23       ; message row 23
E1A7                    338  ; row argument for PUTSTR
E1A7  B7 01 1A          339                  STAA    ROWARG
E1AA  86 16             340                  LDAA    #22       ; message column 22
E1AC                    341  ; column argument for PUTSTR
E1AC  B7 01 1B          342                  STAA    COLARG
E1AF                    343  ; restore the text pointer
E1AF  FE 01 22          344                  LDX     KPTR
E1B2                    345  ; print the win/lose message
E1B2  BD E4 77          346                  JSR     PUTSTR
E1B5                    347  ; wait for SPACE or CLEAR
E1B5  BD E4 3B          348                  JSR     WAITSP
E1B8  7E E1 21          349                  JMP     NEWMATCH  ; start a new match
E1BB                    350
E1BB                    351  ; ================================================
E1BB                    352  ; SERVE -- ball to the centre line at a random row,
E1BB                    353  ; random angle
E1BB                    354  ; ================================================
E1BB                    355  SERVE
E1BB  86 28             356                  LDAA    #40       ; centre column
E1BD                    357  ; ball starts on the centre line
E1BD  B7 01 05          358                  STAA    BCOL
E1C0  BD E5 BF          359                  JSR     RAND      ; random byte in A
E1C3  84 0F             360                  ANDA    #$0F      ; keep 0-15
E1C5  8B 04             361                  ADDA    #4        ; shift to 4-19
E1C7  B7 01 04          362                  STAA    BROW      ; rows 4-19
E1CA                    363  ; A = serve direction ($80 or $7F)
E1CA  B6 01 14          364                  LDAA    LOSER
E1CD                    365  ; ball heads toward the last loser
E1CD  B7 01 06          366                  STAA    BHDIR
E1D0  BD E5 BF          367                  JSR     RAND      ; random byte in A
E1D3                    368  ; bit 7 = random up / down
E1D3  B7 01 07          369                  STAA    BVDIR
E1D6  BD E5 BF          370                  JSR     RAND      ; random byte in A
E1D9                    371  ; keep one bit: 0 or 1
E1D9  84 01             372                  ANDA    #1
E1DB  4C                373                  INCA              ; now 1 or 2
E1DC                    374  ; 1 = steep, 2 = shallow
E1DC  B7 01 08          375                  STAA    BVRATE
E1DF                    376  ; first step countdown = rate
E1DF  B7 01 09          377                  STAA    BVCNT
```

```
E1E2                 378  ; starting tick delay
E1E2  CE 19 00       379                  LDX     #DLYSTART
E1E5                 380  ; set the delay count
E1E5  FF 01 15       381                  STX     DLYCNT
E1E8                 382  ; no point scored yet
E1E8  7F 01 13       383                  CLR     POINT
E1EB  B6 01 04       384                  LDAA    BROW      ; A = ball row
E1EE  B7 01 1A       385                  STAA    ROWARG    ; row argument
E1F1  B6 01 05       386                  LDAA    BCOL      ; A = ball column
E1F4  B7 01 1B       387                  STAA    COLARG    ; column argument
E1F7                 388  ; old column = start column
E1F7  B7 01 0E       389                  STAA    OLDCOL
E1FA                 390  ; X = CRTRAM address of the ball cell
E1FA  BD E5 65       391                  JSR     CALCADR
E1FD                 392  ; remember the current ball address
E1FD  FF 01 0A       393                  STX     CURADR
E200                 394  ; old address = current address
E200  FF 01 0C       395                  STX     OLDADR
E203                 396  ; A = ball character
E203  86 4F          397                  LDAA    #CH_BALL
E205                 398  ; sync to the CRT before writing
E205  B7 70 00       399                  STAA    HALT
E208                 400  ; let the strobe settle
E208  01             401                  NOP
E209  A7 00          402                  STAA    0,X       ; draw the ball
E20B                 403  ; ~0.5 s breather so the ball can be seen
E20B  CE 00 00       404                  LDX     #0
E20E                 405  ; burn time: 65536 passes ...
E20E  09             406  SV_DLY          DEX
E20F                 407  ; ... as the breather
E20F  26 FD          408                  BNE     SV_DLY
E211                 409  ; ball is in play: return
E211  39             410                  RTS
E212                 411
E212                 412  ; ================================================
E212                 413  ; BALLMOV -- Rubout's BALLMOV: advance column,
E212                 414  ; advance row, read back the
E212                 415  ; destination cell, branch on what is there.
E212                 416  ; ================================================
E212                 417  BALLMOV
E212                 418  ; X = the ball's current cell address
E212  FE 01 0A       419                  LDX     CURADR
E215                 420  ; keep it as the previous address
E215  FF 01 0C       421                  STX     OLDADR
E218  B6 01 05       422                  LDAA    BCOL      ; A = ball column
E21B                 423  ; keep it as the previous column
E21B  B7 01 0E       424                  STAA    OLDCOL
E21E                 425  ; re-test limit: 6 tries
E21E  86 06          426                  LDAA    #6
E220                 427  ; store the re-test counter
E220  B7 01 19       428                  STAA    TRIES
E223                 429  ; step the column by its direction
E223  BD E2 AE       430                  JSR     BALLCOL
E226                 431  ; step the row (when due)
E226  BD E2 BB       432                  JSR     BALLROW
E229                 433  BM_EVAL
E229  B6 01 04       434                  LDAA    BROW      ; A = ball row
```

```
E22C  B7 01 1A       435                  STAA    ROWARG    ; row argument
E22F  B6 01 05       436                  LDAA    BCOL      ; A = ball column
E232  B7 01 1B       437                  STAA    COLARG    ; column argument
E235                 438  ; X = CRTRAM address of that cell
E235  BD E5 65       439                  JSR     CALCADR
E238                 440  ; remember it as the current cell
E238  FF 01 0A       441                  STX     CURADR
E23B                 442  ; sync to the CRT, then READ the cell
E23B  B7 70 00       443                  STAA    HALT
E23E                 444  ; let the strobe settle
E23E  01             445                  NOP
E23F                 446  ; B = character already in that cell
E23F  E6 00          447                  LDAB    0,X
E241                 448  ; count down the re-test limiter
E241  7A 01 19       449                  DEC     TRIES
E244                 450  ; safety valve: stop re-testing
E244  27 1F          451                  BEQ     BM_FREE
E246                 452  ; is the cell blank?
E246  5D             453                  TSTB
E247  27 1C          454                  BEQ     BM_FREE   ; blank
E249  C1 3A          455                  CMPB    #CH_NET   ; centre line?
E24B                 456  ; centre line is passable
E24B  27 18          457                  BEQ     BM_FREE
E24D  C1 23          458                  CMPB    #CH_PAD   ; paddle?
E24F  27 0C          459                  BEQ     BM_PAD    ; paddle
E251  C1 3D          460                  CMPB    #CH_WALL  ; wall?
E253                 461  ; anything else: treat as free
E253  26 10          462                  BNE     BM_FREE
E255                 463  ; wall: flip vertical direction ...
E255  73 01 07       464                  COM     BVDIR
E258                 465  ; ... step back out, re-test the cell
E258  BD E2 C8       466                  JSR     BR_STEP
E25B  20 CC          467                  BRA     BM_EVAL   ; re-test the cell
E25D                 468  BM_PAD
E25D                 469  ; flip horizontal, pick the angle
E25D  BD E2 D5       470                  JSR     PADHIT
E260                 471  ; step back out of the paddle column
E260  BD E2 AE       472                  JSR     BALLCOL
E263  20 C4          473                  BRA     BM_EVAL   ; re-test the cell
E265                 474
E265                 475  BM_FREE
E265  B6 01 05       476                  LDAA    BCOL      ; A = ball column
E268  81 02          477                  CMPA    #2        ; column below 2?
E26A                 478  ; got past your paddle
E26A  25 34          479                  BCS     BM_HMISS
E26C                 480  ; column 78 or more?
E26C  81 4E          481                  CMPA    #78
E26E                 482  ; got past the computer's paddle
E26E  24 37          483                  BCC     BM_CMISS
E270                 484  ; X = the new ball cell
E270  FE 01 0A       485                  LDX     CURADR
E273                 486  ; A = ball character
E273  86 4F          487                  LDAA    #CH_BALL
E275                 488  ; sync to the CRT before writing
E275  B7 70 00       489                  STAA    HALT
E278                 490  ; let the strobe settle
E278  01             491                  NOP
```

```
E279  A7 00          492                  STAA    0,X       ; draw the ball
E27B                 493  BM_ERASE
E27B                 494  ; X = the previous ball cell
E27B  FE 01 0C       495                  LDX     OLDADR
E27E                 496  ; bounced straight back into the same cell?
E27E  BC 01 0A       497                  CPX     CURADR
E281                 498  ; then the ball we just drew is the one there
E281  27 1C          499                  BEQ     BM_DONE
E283                 500  ; sync to the CRT before reading
E283  B7 70 00       501                  STAA    HALT
E286                 502  ; let the strobe settle
E286  01             503                  NOP
E287                 504  ; A = what is in the old cell now
E287  A6 00          505                  LDAA    0,X
E289                 506  ; only blank it if it is still the ball
E289  81 4F          507                  CMPA    #CH_BALL
E28B                 508  ; not the ball: leave it alone
E28B  26 12          509                  BNE     BM_DONE
E28D                 510  ; A = column of the old cell
E28D  B6 01 0E       511                  LDAA    OLDCOL
E290                 512  ; was it the centre column?
E290  81 28          513                  CMPA    #40
E292                 514  ; yes: put the net back
E292  27 03          515                  BEQ     BM_NET
E294  4F             516                  CLRA              ; no: use a blank
E295  20 02          517                  BRA     BM_WR     ; go write it
E297                 518  ; restore the centre line
E297  86 3A          519  BM_NET          LDAA    #CH_NET
E299                 520  ; sync to the CRT before writing
E299  B7 70 00       521  BM_WR           STAA    HALT
E29C                 522  ; let the strobe settle
E29C  01             523                  NOP
E29D                 524  ; write the blank or net character
E29D  A7 00          525                  STAA    0,X
E29F  39             526  BM_DONE         RTS               ; return to GAME
E2A0                 527  BM_HMISS
E2A0                 528  ; point code 2: computer scores
E2A0  86 02          529                  LDAA    #2
E2A2  B7 01 13       530                  STAA    POINT     ; computer scores
E2A5                 531  ; go erase the old ball
E2A5  20 D4          532                  BRA     BM_ERASE
E2A7                 533  BM_CMISS
E2A7                 534  ; point code 1: you score
E2A7  86 01          535                  LDAA    #1
E2A9  B7 01 13       536                  STAA    POINT     ; you score
E2AC                 537  ; go erase the old ball
E2AC  20 CD          538                  BRA     BM_ERASE
E2AE                 539
E2AE                 540  ; BALLCOL -- Rubout's BALLCOL: step column by the
E2AE                 541  ; sign flag.
E2AE                 542  BALLCOL
E2AE                 543  ; test the horizontal direction flag
E2AE  7D 01 06       544                  TST     BHDIR
E2B1                 545  ; bit 7 set: moving right
E2B1  2B 04          546                  BMI     BC_R
E2B3                 547  ; moving left: column - 1
E2B3  7A 01 05       548                  DEC     BCOL
```

```
E2B6  39             549                 RTS             ; done
E2B7                 550  ; moving right: column + 1
E2B7  7C 01 05       551  BC_R            INC     BCOL
E2BA  39             552                 RTS             ; done
E2BB                 553
E2BB                 554  ; BALLROW -- Rubout's BALLROW, with a tick divider so
E2BB                 555  ; shallow shots move
E2BB                 556  ; one row every second tick.
E2BB                 557  BALLROW
E2BB                 558  ; A = vertical rate (0 = flat)
E2BB  B6 01 08       559                 LDAA    BVRATE
E2BE  27 14          560                 BEQ     BR_X      ; flat shot
E2C0                 561  ; count down to the next row step
E2C0  7A 01 09       562                 DEC     BVCNT
E2C3                 563  ; not due yet: leave the row alone
E2C3  26 0F          564                 BNE     BR_X
E2C5                 565  ; reload the countdown from the rate
E2C5  B7 01 09       566                 STAA    BVCNT
E2C8                 567  ; test the vertical direction flag
E2C8  7D 01 07       568  BR_STEP         TST     BVDIR
E2CB                 569  ; bit 7 set: moving down
E2CB  2B 04          570                 BMI     BR_D
E2CD                 571  ; moving up: row - 1
E2CD  7A 01 04       572                 DEC     BROW
E2D0  39             573                 RTS             ; done
E2D1                 574  ; moving down: row + 1
E2D1  7C 01 04       575  BR_D            INC     BROW
E2D4  39             576  BR_X            RTS             ; done
E2D5                 577
E2D5                 578  ; PADHIT -- the ball has entered a paddle cell.
E2D5                 579  ; Reverse horizontal
E2D5                 580  ; direction, choose the angle from where on the
E2D5                 581  ; paddle it landed, beep,
E2D5                 582  ; speed the rally up, and (when you hit it) re-roll
E2D5                 583  ; the computer's aim.
E2D5                 584  ; cell 0 up steep | 1 up shallow | 2 flat | 3 down
E2D5                 585  ; shallow | 4 down steep
E2D5                 586  PADHIT
E2D5                 587  ; reverse the horizontal direction
E2D5  73 01 06       588                 COM     BHDIR
E2D8                 589  ; assume it is your paddle
E2D8  CE 01 00       590                 LDX     #HPAD
E2DB  B6 01 05       591                 LDAA    BCOL      ; A = ball column
E2DE                 592  ; left half of the court?
E2DE  81 28          593                 CMPA    #40
E2E0                 594  ; yes: keep your paddle record
E2E0  25 03          595                 BCS     PH_REC
E2E2                 596  ; no: use the computer's paddle
E2E2  CE 01 02       597                 LDX     #CPAD
E2E5  F6 01 04       598  PH_REC          LDAB    BROW      ; B = ball row
E2E8                 599  ; B = 0..4, offset from paddle top
E2E8  E0 00          600                 SUBB    0,X
E2EA  26 07          601                 BNE     PH_1      ; not cell 0
E2EC  7F 01 07       602                 CLR     BVDIR     ; cell 0: head up
E2EF                 603  ; steep: row step every tick
E2EF  86 01          604                 LDAA    #1
E2F1  20 23          605                 BRA     PH_SET    ; apply it
```

```
E2F3  5A             606  PH_1            DECB              ; test for cell 1
E2F4  26 07          607                  BNE     PH_2      ; not cell 1
E2F6  7F 01 07       608                  CLR     BVDIR     ; cell 1: head up
E2F9                 609  ; shallow: row step every other tick
E2F9  86 02          610                  LDAA    #2
E2FB  20 19          611                  BRA     PH_SET    ; apply it
E2FD  5A             612  PH_2            DECB              ; test for cell 2
E2FE  26 03          613                  BNE     PH_3      ; not cell 2
E300                 614  ; cell 2: rate 0 = flat return
E300  4F             615                  CLRA
E301  20 13          616                  BRA     PH_SET    ; apply it
E303  5A             617  PH_3            DECB              ; test for cell 3
E304                 618  ; not cell 3, so it is cell 4
E304  26 09          619                  BNE     PH_4
E306  86 80          620                  LDAA    #$80      ; cell 3: head down
E308                 621  ; set the down direction
E308  B7 01 07       622                  STAA    BVDIR
E30B                 623  ; shallow: row step every other tick
E30B  86 02          624                  LDAA    #2
E30D  20 07          625                  BRA     PH_SET    ; apply it
E30F  86 80          626  PH_4            LDAA    #$80      ; cell 4: head down
E311                 627  ; set the down direction
E311  B7 01 07       628                  STAA    BVDIR
E314                 629  ; steep: row step every tick
E314  86 01          630                  LDAA    #1
E316                 631  ; store the vertical rate
E316  B7 01 08       632  PH_SET          STAA    BVRATE
E319                 633  ; first step countdown = rate
E319  B7 01 09       634                  STAA    BVCNT
E31C                 635  ; sound the bell for the hit
E31C  BD E5 DE       636                  JSR     BEEP
E31F                 637  ; speed up: -200 per hit, floor ~3600
E31F  B6 01 15       638                  LDAA    DLYCNT
E322                 639  ; delay high byte already low enough?
E322  81 0E          640                  CMPA    #$0E
E324                 641  ; yes: stop speeding up
E324  25 10          642                  BCS     PH_AIM
E326                 643  ; A = delay low byte
E326  B6 01 16       644                  LDAA    DLYCNT+1
E329                 645  ; shorten the delay by 200
E329  80 C8          646                  SUBA    #200
E32B                 647  ; store the low byte
E32B  B7 01 16       648                  STAA    DLYCNT+1
E32E                 649  ; A = delay high byte
E32E  B6 01 15       650                  LDAA    DLYCNT
E331                 651  ; carry the borrow into it
E331  82 00          652                  SBCA    #0
E333                 653  ; store the high byte
E333  B7 01 15       654                  STAA    DLYCNT
E336  B6 01 05       655  PH_AIM          LDAA    BCOL      ; A = ball column
E339                 656  ; left half of the court?
E339  81 28          657                  CMPA    #40
E33B                 658  ; computer's hit: nothing more
E33B  24 1A          659                  BCC     PH_X
E33D                 660  ; AIMOFF = -3..+3, bell-shaped (sum of two
E33D  BD E5 BF       661                  JSR     RAND
E340                 662  ; 0-3 rolls, minus 3).  +-3 is outside the
```

```
E340  B7 01 1E       663                 STAA    TMPA
E343                 664  ; paddle: the computer misjudges ~1 in 8.
E343  84 03          665                 ANDA    #3
E345                 666  ; keep the first 0-3 roll
E345  B7 01 1F       667                 STAA    TMPB
E348                 668  ; reload the random byte
E348  B6 01 1E       669                 LDAA    TMPA
E34B                 670  ; shift right twice ...
E34B  44             671                 LSRA
E34C                 672  ; ... to bring bits 2-3 down
E34C  44             673                 LSRA
E34D  84 03          674                 ANDA    #3        ; second 0-3 roll
E34F                 675  ; add the first roll: 0-6
E34F  BB 01 1F       676                 ADDA    TMPB
E352                 677  ; centre on zero: -3..+3
E352  80 03          678                 SUBA    #3
E354                 679  ; store the aim error
E354  B7 01 12       680                 STAA    AIMOFF
E357  39             681  PH_X           RTS               ; done
E358                 682
E358                 683  ; ================================================
E358                 684  ; POLLHUM -- scan the human's keys; first one found
E358                 685  ; moves the paddle 1 row
E358                 686  ; ================================================
E358                 687  POLLHUM
E358  86 28          688                 LDAA    #$28      ; P
E35A                 689  ; scan code out to PIA1
E35A  B7 78 00       690                 STAA    PIA1
E35D                 691  ; read the key state back
E35D  B6 78 00       692                 LDAA    PIA1
E360                 693  ; bit 7 clear: P is not down
E360  2A 03          694                 BPL     PK_NP
E362  BD E4 0D       695                 JSR     PAUSE     ; P is down: pause
E365                 696  ; X -> table of (scan code, direction)
E365  CE E3 81       697  PK_NP          LDX     #KEYTAB
E368                 698  ; A = scan code of this entry
E368  A6 00          699  PK_LP          LDAA    0,X
E36A                 700  ; zero ends the table
E36A  27 14          701                 BEQ     PK_END
E36C                 702  ; scan code out to PIA1
E36C  B7 78 00       703                 STAA    PIA1
E36F                 704  ; read the key state back
E36F  B6 78 00       705                 LDAA    PIA1
E372                 706  ; bit 7 set: key is down
E372  2B 04          707                 BMI     PK_HIT
E374                 708  ; skip the scan code ...
E374  08             709                 INX
E375                 710  ; ... and the direction: next entry
E375  08             711                 INX
E376  20 F0          712                 BRA     PK_LP     ; try the next key
E378                 713  ; A = direction for the pressed key
E378  A6 01          714  PK_HIT          LDAA    1,X
E37A                 715  ; X -> your paddle record
E37A  CE 01 00       716                 LDX     #HPAD
E37D                 717  ; slide the paddle one row
E37D  BD E3 8E       718                 JSR     MOVPAD
E380  39             719  PK_END         RTS               ; return to GAME
```

```
E381                  720
E381                  721  ; scan code, direction.   $53 up   $52 down  (cursor
E381                  722  ; keys, ^K / ^J in KBDTAB)
E381                  723  ; $20 W    $31 S     $54 left $55 right (Rubout's
E381                  724  ; keys)
E381                  725  ; up, W, left keys -> $FF (up)
E381  53 FF 20 FF     726  KEYTAB          FCB     $53,$FF,$20,$FF,$54,$FF
E387                  727  ; down, S, right keys -> $01 (down)
E387  52 01 31 01     728                  FCB     $52,$01,$31,$01,$55,$01
E38D                  729  ; zero ends the table
E38D  00              730                  FCB     0
E38E                  731
E38E                  732  ; MOVPAD -- X = paddle record, A = $FF (up) or $01
E38E                  733  ; (down).  Slides the
E38E                  734  ; paddle one row: writes the new leading end, blanks
E38E                  735  ; the old trailing end.
E38E                  736  ; Top row is limited to 2..17 (court rows 2-21).
E38E                  737  MOVPAD
E38E                  738  ; remember the direction
E38E  B7 01 26        739                  STAA    MPDIR
E391                  740  ; B = paddle top row
E391  E6 00           741                  LDAB    0,X
E393                  742  ; B = proposed new top row
E393  FB 01 26        743                  ADDB    MPDIR
E396                  744  ; above court row 2?
E396  C1 02           745                  CMPB    #2
E398                  746  ; yes: refuse the move
E398  25 42           747                  BCS     MP_X
E39A                  748  ; beyond the lowest top row (17)?
E39A  C1 12           749                  CMPB    #18
E39C                  750  ; yes: refuse the move
E39C  24 3E           751                  BCC     MP_X
E39E                  752  ; store the new top row
E39E  E7 00           753                  STAB    0,X
E3A0  F7 01 1F        754                  STAB    TMPB      ; keep a copy
E3A3  A6 01           755                  LDAA    1,X       ; A = paddle column
E3A5  B7 01 1B        756                  STAA    COLARG    ; column argument
E3A8                  757  ; moving up or down?
E3A8  7D 01 26        758                  TST     MPDIR
E3AB  2B 18           759                  BMI     MP_UP     ; negative: up
E3AD                  760  ; down: draw top+4, blank top-1
E3AD  B6 01 1F        761                  LDAA    TMPB
E3B0                  762  ; bottom cell = top + 4
E3B0  8B 04           763                  ADDA    #4
E3B2  B7 01 1A        764                  STAA    ROWARG    ; row argument
E3B5  86 23           765                  LDAA    #CH_PAD   ; paddle character
E3B7                  766  ; draw the new bottom cell
E3B7  BD E4 67        767                  JSR     PUTAT
E3BA  B6 01 1F        768                  LDAA    TMPB      ; A = new top row
E3BD                  769  ; the row just above it
E3BD  4A              770                  DECA
E3BE  B7 01 1A        771                  STAA    ROWARG    ; row argument
E3C1  4F              772                  CLRA              ; blank character
E3C2                  773  ; blank the old top cell, then return
E3C2  7E E4 67        774                  JMP     PUTAT
E3C5                  775  ; up: draw top, blank top+5
E3C5  B6 01 1F        776  MP_UP           LDAA    TMPB
```

```
E3C8                    777  ; row argument = new top row
E3C8  B7 01 1A          778                  STAA    ROWARG
E3CB  86 23             779                  LDAA    #CH_PAD   ; paddle character
E3CD                    780  ; draw the new top cell
E3CD  BD E4 67          781                  JSR     PUTAT
E3D0  B6 01 1F          782                  LDAA    TMPB      ; A = new top row
E3D3                    783  ; the row just below the paddle
E3D3  8B 05             784                  ADDA    #5
E3D5  B7 01 1A          785                  STAA    ROWARG    ; row argument
E3D8  4F                786                  CLRA              ; blank character
E3D9                    787  ; blank the old bottom cell, return
E3D9  7E E4 67          788                  JMP     PUTAT
E3DC  39                789  MP_X            RTS               ; done
E3DD                    790
E3DD                    791  ; ================================================
E3DD                    792  ; CPUAI -- the computer player.  Moves every other
E3DD                    793  ; tick.  Aims its paddle
E3DD                    794  ; centre at (ball row + AIMOFF) while the ball is
E3DD                    795  ; coming at it, otherwise
E3DD                    796  ; drifts back to row 11.
E3DD                    797  ; ================================================
E3DD                    798  CPUAI
E3DD  B6 01 11          799                  LDAA    TICK      ; A = tick counter
E3E0  84 01             800                  ANDA    #1        ; keep the low bit
E3E2                    801  ; odd tick: skip (half speed)
E3E2  26 28             802                  BNE     CA_X
E3E4                    803  ; default target: row 11
E3E4  86 0B             804                  LDAA    #11
E3E6                    805  ; which way is the ball going?
E3E6  7D 01 06          806                  TST     BHDIR
E3E9  2A 06             807                  BPL     CA_TGT    ; ball going away
E3EB                    808  ; coming at us: A = ball row
E3EB  B6 01 04          809                  LDAA    BROW
E3EE  BB 01 12          810                  ADDA    AIMOFF    ; add the aim error
E3F1                    811  ; store the target row
E3F1  B7 01 1E          812  CA_TGT          STAA    TMPA
E3F4                    813  ; A = paddle top row
E3F4  B6 01 02          814                  LDAA    CPAD
E3F7                    815  ; centre row of the paddle
E3F7  8B 02             816                  ADDA    #2
E3F9                    817  ; compare paddle centre to target
E3F9  B1 01 1E          818                  CMPA    TMPA
E3FC                    819  ; already there: no move
E3FC  27 0E             820                  BEQ     CA_X
E3FE                    821  ; signed: target can be -1 near the top
E3FE  2E 04             822                  BGT     CA_UP
E400  86 01             823                  LDAA    #1        ; direction: down
E402  20 02             824                  BRA     CA_MV     ; go move
E404  86 FF             825  CA_UP           LDAA    #$FF      ; direction: up
E406                    826  ; X -> computer's paddle record
E406  CE 01 02          827  CA_MV           LDX     #CPAD
E409                    828  ; slide the paddle one row
E409  BD E3 8E          829                  JSR     MOVPAD
E40C  39                830  CA_X            RTS               ; done
E40D                    831
E40D                    832  ; ================================================
E40D                    833  ; PAUSE / WAITSP -- keyboard waits
```

```
E40D                 834  ; ================================================
E40D                 835  PAUSE
E40D                 836  ; wait for P to be released
E40D  BD E4 30       837                  JSR     PZREL
E410  86 17          838                  LDAA    #23       ; message row 23
E412  B7 01 1A       839                  STAA    ROWARG    ; row argument
E415  86 25          840                  LDAA    #37       ; message column 37
E417  B7 01 1B       841                  STAA    COLARG    ; column argument
E41A                 842  ; X points at the PAUSED text
E41A  CE E6 3E       843                  LDX     #SPAUSE
E41D  BD E4 77       844                  JSR     PUTSTR    ; print it
E420  86 28          845  PZW             LDAA    #$28      ; scan code for P
E422                 846  ; scan code out to PIA1
E422  B7 78 00       847                  STAA    PIA1
E425                 848  ; read the key state back
E425  B6 78 00       849                  LDAA    PIA1
E428                 850  ; wait for P to be pressed again
E428  2A F6          851                  BPL     PZW
E42A  BD E4 30       852                  JSR     PZREL     ; and released
E42D                 853  ; erase the message, then return
E42D  7E E4 AA       854                  JMP     MSGCLR
E430  86 28          855  PZREL           LDAA    #$28      ; scan code for P
E432                 856  ; scan code out to PIA1
E432  B7 78 00       857                  STAA    PIA1
E435                 858  ; read the key state back
E435  B6 78 00       859                  LDAA    PIA1
E438                 860  ; still down: keep waiting
E438  2B F6          861                  BMI     PZREL
E43A  39             862                  RTS               ; P is released
E43B                 863
E43B                 864  ; WAITSP -- wait for SPACE ($4B) or CLEAR ($40, the
E43B                 865  ; other start key Rubout
E43B                 866  ; uses; ADM31EMU.EXE also reports its space bar as
E43B                 867  ; $40), spinning the random
E43B                 868  ; generator while we wait, which is what seeds each
E43B                 869  ; serve.  Then wait for
E43B                 870  ; both to be released.
E43B                 871  WAITSP
E43B                 872  ; spin the generator (seed source)
E43B  BD E5 BF       873                  JSR     RAND
E43E                 874  ; scan code for SPACE
E43E  86 4B          875                  LDAA    #$4B
E440                 876  ; scan code out to PIA1
E440  B7 78 00       877                  STAA    PIA1
E443                 878  ; read the key state back
E443  B6 78 00       879                  LDAA    PIA1
E446                 880  ; SPACE is down: wait for release
E446  2B 0A          881                  BMI     WSREL
E448                 882  ; scan code for CLEAR
E448  86 40          883                  LDAA    #$40
E44A                 884  ; scan code out to PIA1
E44A  B7 78 00       885                  STAA    PIA1
E44D                 886  ; read the key state back
E44D  B6 78 00       887                  LDAA    PIA1
E450                 888  ; neither is down: keep waiting
E450  2A E9          889                  BPL     WAITSP
E452                 890  ; release wait: scan code for SPACE
```

```
E452  86 4B          891  WSREL           LDAA    #$4B
E454                 892  ; scan code out to PIA1
E454  B7 78 00       893                  STAA    PIA1
E457                 894  ; read the key state back
E457  B6 78 00       895                  LDAA    PIA1
E45A                 896  ; still down: keep waiting
E45A  2B F6          897                  BMI     WSREL
E45C                 898  ; scan code for CLEAR
E45C  86 40          899                  LDAA    #$40
E45E                 900  ; scan code out to PIA1
E45E  B7 78 00       901                  STAA    PIA1
E461                 902  ; read the key state back
E461  B6 78 00       903                  LDAA    PIA1
E464                 904  ; still down: keep waiting
E464  2B EC          905                  BMI     WSREL
E466                 906  ; both keys are released
E466  39             907                  RTS
E467                 908
E467                 909  ; ================================================
E467                 910  ; SCREEN HELPERS
E467                 911  ; ================================================
E467                 912  ; PUTAT -- write A at (ROWARG,COLARG)
E467                 913  PUTAT
E467                 914  ; save the character
E467  B7 01 1E       915                  STAA    TMPA
E46A                 916  ; X = cell address for (ROWARG,COLARG)
E46A  BD E5 65       917                  JSR     CALCADR
E46D                 918  ; restore the character
E46D  B6 01 1E       919                  LDAA    TMPA
E470                 920  ; sync to the CRT before writing
E470  B7 70 00       921                  STAA    HALT
E473                 922  ; let the strobe settle
E473  01             923                  NOP
E474                 924  ; write the character to the screen
E474  A7 00          925                  STAA    0,X
E476  39             926                  RTS             ; done
E477                 927
E477                 928  ; PUTSTR -- zero-terminated string at X, written from
E477                 929  ; (ROWARG,COLARG)
E477                 930  PUTSTR
E477                 931  ; save the string pointer
E477  FF 01 24       932                  STX     PSRC
E47A                 933  ; X = string pointer
E47A  FE 01 24       934  PS_LP           LDX     PSRC
E47D                 935  ; A = next character
E47D  A6 00          936                  LDAA    0,X
E47F                 937  ; zero ends the string
E47F  27 0C          938                  BEQ     PS_X
E481                 939  ; advance the pointer
E481  08             940                  INX
E482  FF 01 24       941                  STX     PSRC      ; save the pointer
E485                 942  ; draw the character
E485  BD E4 67       943                  JSR     PUTAT
E488  7C 01 1B       944                  INC     COLARG    ; next column
E48B  20 ED          945                  BRA     PS_LP     ; repeat
E48D  39             946  PS_X            RTS             ; done
E48E                 947
```

```
E48E                948  ; HLINE -- A = char, B = count, X = first CRTRAM
E48E                949  ; address
E48E                950  HLINE
E48E                951  ; sync to the CRT before writing
E48E  B7 70 00      952                  STAA    HALT
E491                953  ; let the strobe settle
E491  01            954                  NOP
E492  A7 00         955                  STAA    0,X       ; write one cell
E494  08            956                  INX               ; next cell
E495  5A            957                  DECB              ; count down
E496                958  ; more cells: repeat
E496  26 F6         959                  BNE     HLINE
E498  39            960                  RTS               ; done
E499                961
E499                962  CLRSCR
E499                963  ; X = first screen cell
E499  CE 80 00      964                  LDX     #CRTRAM
E49C  4F            965                  CLRA              ; A = 0 (blank)
E49D                966  ; sync to the CRT before writing
E49D  B7 70 00      967  CS_LP           STAA    HALT
E4A0                968  ; let the strobe settle
E4A0  01            969                  NOP
E4A1  A7 00         970                  STAA    0,X       ; blank one cell
E4A3  08            971                  INX               ; next cell
E4A4                972  ; past the last cell?
E4A4  8C 87 80      973                  CPX     #CRTRAM+1920
E4A7  26 F4         974                  BNE     CS_LP     ; no: repeat
E4A9  39            975                  RTS               ; done
E4AA                976
E4AA                977  MSGCLR
E4AA  CE 87 30      978                  LDX     #CRTRAM+1840 ; row 23
E4AD  C6 50         979                  LDAB    #80       ; 80 cells in a row
E4AF  4F            980                  CLRA              ; blank character
E4B0                981  ; blank the row, then return
E4B0  7E E4 8E      982                  JMP     HLINE
E4B3                983
E4B3                984  ; SHOWSC -- redraw both score digits
E4B3                985  SHOWSC
E4B3  7F 01 1A      986                  CLR     ROWARG    ; row 0
E4B6  86 08         987                  LDAA    #8        ; your score column
E4B8  B7 01 1B      988                  STAA    COLARG    ; column argument
E4BB  B6 01 0F      989                  LDAA    SCOREH    ; A = your score
E4BE                990  ; convert the digit to ASCII
E4BE  8B 30         991                  ADDA    #$30
E4C0  BD E4 67      992                  JSR     PUTAT     ; draw it
E4C3                993  ; computer's score column
E4C3  86 4A         994                  LDAA    #74
E4C5  B7 01 1B      995                  STAA    COLARG    ; column argument
E4C8                996  ; A = computer's score
E4C8  B6 01 10      997                  LDAA    SCOREC
E4CB                998  ; convert the digit to ASCII
E4CB  8B 30         999                  ADDA    #$30
E4CD               1000  ; draw it, then return
E4CD  7E E4 67     1001                  JMP     PUTAT
E4D0               1002
E4D0               1003  ; DRAWPAD -- X = paddle record; draw all 5 cells
E4D0               1004  DRAWPAD
```

```
E4D0  A6 01         1005                  LDAA    1,X       ; A = paddle column
E4D2  B7 01 1B      1006                  STAA    COLARG    ; column argument
E4D5                1007  ; A = paddle top row
E4D5  A6 00         1008                  LDAA    0,X
E4D7  B7 01 1A      1009                  STAA    ROWARG    ; row argument
E4DA  86 05         1010                  LDAA    #5        ; five cells tall
E4DC  B7 01 1F      1011                  STAA    TMPB      ; cell counter
E4DF  86 23         1012  DP_LP           LDAA    #CH_PAD   ; paddle character
E4E1  BD E4 67      1013                  JSR     PUTAT     ; draw one cell
E4E4  7C 01 1A      1014                  INC     ROWARG    ; next row
E4E7  7A 01 1F      1015                  DEC     TMPB      ; count down
E4EA                1016  ; more cells: repeat
E4EA  26 F3         1017                  BNE     DP_LP
E4EC  39            1018                  RTS               ; done
E4ED                1019
E4ED                1020  ; DRAWFLD -- walls, centre line, captions, scores,
E4ED                1021  ; paddles
E4ED                1022  DRAWFLD
E4ED  86 3D         1023                  LDAA    #CH_WALL  ; wall character
E4EF  CE 80 50      1024                  LDX     #CRTRAM+80 ; row 1: top wall
E4F2  C6 50         1025                  LDAB    #80       ; 80 cells
E4F4  BD E4 8E      1026                  JSR     HLINE     ; draw the top wall
E4F7  86 3D         1027                  LDAA    #CH_WALL  ; wall character
E4F9                1028  ; row 22: bottom wall
E4F9  CE 86 E0      1029                  LDX     #CRTRAM+1760
E4FC  C6 50         1030                  LDAB    #80       ; 80 cells
E4FE                1031  ; draw the bottom wall
E4FE  BD E4 8E      1032                  JSR     HLINE
E501  86 02         1033                  LDAA    #2        ; first court row
E503  B7 01 1A      1034                  STAA    ROWARG    ; row argument
E506  86 28         1035                  LDAA    #40       ; centre column
E508  B7 01 1B      1036                  STAA    COLARG    ; column argument
E50B  86 3A         1037  DF_NET          LDAA    #CH_NET   ; net character
E50D  BD E4 67      1038                  JSR     PUTAT     ; draw one net cell
E510  7C 01 1A      1039                  INC     ROWARG    ; next row
E513  B6 01 1A      1040                  LDAA    ROWARG    ; A = row
E516                1041  ; reached the bottom wall?
E516  81 16         1042                  CMPA    #22
E518                1043  ; no: continue the net
E518  26 F1         1044                  BNE     DF_NET
E51A  7F 01 1A      1045                  CLR     ROWARG    ; row 0
E51D  86 04         1046                  LDAA    #4        ; "YOU" column
E51F  B7 01 1B      1047                  STAA    COLARG    ; column argument
E522  CE E5 EC      1048                  LDX     #SYOU     ; X points at "YOU"
E525  BD E4 77      1049                  JSR     PUTSTR    ; draw it
E528  7F 01 1A      1050                  CLR     ROWARG    ; row 0
E52B  86 18         1051                  LDAA    #24       ; title column
E52D  B7 01 1B      1052                  STAA    COLARG    ; column argument
E530                1053  ; X points at the title text
E530  CE E5 F4      1054                  LDX     #STITLE
E533  BD E4 77      1055                  JSR     PUTSTR    ; draw it
E536  7F 01 1A      1056                  CLR     ROWARG    ; row 0
E539  86 46         1057                  LDAA    #70       ; "CPU" column
E53B  B7 01 1B      1058                  STAA    COLARG    ; column argument
E53E  CE E5 F0      1059                  LDX     #SCPU     ; X points at "CPU"
E541  BD E4 77      1060                  JSR     PUTSTR    ; draw it
E544                1061  ; draw both score digits
```

```
E544  BD E4 B3      1062                JSR     SHOWSC
E547                1063  ; starting paddle top row
E547  86 09         1064                LDAA    #9
E549                1065  ; your paddle top row
E549  B7 01 00      1066                STAA    HPAD
E54C                1067  ; computer's paddle top row
E54C  B7 01 02      1068                STAA    CPAD
E54F                1069  ; your paddle column
E54F  86 04         1070                LDAA    #4
E551                1071  ; store it in your paddle record
E551  B7 01 01      1072                STAA    HPAD+1
E554                1073  ; computer's paddle column
E554  86 4B         1074                LDAA    #75
E556                1075  ; store it in the computer's record
E556  B7 01 03      1076                STAA    CPAD+1
E559                1077  ; X -> your paddle record
E559  CE 01 00      1078                LDX     #HPAD
E55C  BD E4 D0      1079                JSR     DRAWPAD   ; draw it
E55F                1080  ; X -> computer's paddle record
E55F  CE 01 02      1081                LDX     #CPAD
E562                1082  ; draw it, then return
E562  7E E4 D0      1083                JMP     DRAWPAD
E565                1084
E565                1085  ; CALCADR -- (ROWARG,COLARG) -> X = CRTRAM + row*80 +
E565                1086  ; col  (as ASTEROID.ASM)
E565                1087  CALCADR
E565                1088  ; work value high byte = 0
E565  7F 01 1C      1089                CLR     ADRPTR
E568  B6 01 1A      1090                LDAA    ROWARG    ; A = row
E56B                1091  ; work value low byte = row
E56B  B7 01 1D      1092                STAA    ADRPTR+1
E56E                1093  ; row x 2: shift the low byte ...
E56E  78 01 1D      1094                ASL     ADRPTR+1
E571                1095  ; ... carry into the high byte
E571  79 01 1C      1096                ROL     ADRPTR
E574  78 01 1D      1097                ASL     ADRPTR+1  ; row x 4
E577                1098  ; carry into the high byte
E577  79 01 1C      1099                ROL     ADRPTR
E57A  78 01 1D      1100                ASL     ADRPTR+1  ; row x 8
E57D                1101  ; carry into the high byte
E57D  79 01 1C      1102                ROL     ADRPTR
E580  78 01 1D      1103                ASL     ADRPTR+1  ; row x 16
E583  79 01 1C      1104                ROL     ADRPTR    ; row*16
E586                1105  ; A = high byte of row*16
E586  B6 01 1C      1106                LDAA    ADRPTR
E589                1107  ; B = low byte of row*16
E589  F6 01 1D      1108                LDAB    ADRPTR+1
E58C                1109  ; save the high byte
E58C  36            1110                PSHA
E58D  37            1111                PSHB              ; save the low byte
E58E  78 01 1D      1112                ASL     ADRPTR+1  ; row x 32
E591                1113  ; carry into the high byte
E591  79 01 1C      1114                ROL     ADRPTR
E594  78 01 1D      1115                ASL     ADRPTR+1  ; row x 64
E597  79 01 1C      1116                ROL     ADRPTR    ; row*64
E59A                1117  ; B = low byte of row*16
E59A  33            1118                PULB
```

```
E59B                1119  ; add the low byte of row*64
E59B  FB 01 1D      1120                  ADDB    ADRPTR+1
E59E                1121  ; store the low byte of row*80
E59E  F7 01 1D      1122                  STAB    ADRPTR+1
E5A1                1123  ; A = high byte of row*16
E5A1  32            1124                  PULA
E5A2                1125  ; add row*64 high byte, plus carry
E5A2  B9 01 1C      1126                  ADCA    ADRPTR
E5A5  B7 01 1C      1127                  STAA    ADRPTR    ; row*80
E5A8  B6 01 1B      1128                  LDAA    COLARG    ; A = column
E5AB                1129  ; add it to the low byte
E5AB  BB 01 1D      1130                  ADDA    ADRPTR+1
E5AE                1131  ; store the low byte
E5AE  B7 01 1D      1132                  STAA    ADRPTR+1
E5B1  B6 01 1C      1133                  LDAA    ADRPTR    ; A = high byte
E5B4                1134  ; add the carry from the low byte
E5B4  89 00         1135                  ADCA    #0
E5B6  8B 80         1136                  ADDA    #$80      ; + CRTRAM ($8000)
E5B8                1137  ; store the high byte
E5B8  B7 01 1C      1138                  STAA    ADRPTR
E5BB                1139  ; X = the finished address
E5BB  FE 01 1C      1140                  LDX     ADRPTR
E5BE                1141  ; address returned in X
E5BE  39            1142                  RTS
E5BF                1143
E5BF                1144  ; RAND -- 16-bit Galois LFSR (taps $B400), returns A
E5BF                1145  ; = low byte
E5BF                1146  RAND
E5BF                1147  ; A = low byte of the seed
E5BF  B6 01 18      1148                  LDAA    SEED+1
E5C2                1149  ; isolate the bit that shifts out
E5C2  84 01         1150                  ANDA    #1
E5C4  B7 01 1E      1151                  STAA    TMPA      ; save that bit
E5C7                1152  ; shift the seed right: high byte ...
E5C7  74 01 17      1153                  LSR     SEED
E5CA                1154  ; ... then the low byte
E5CA  76 01 18      1155                  ROR     SEED+1
E5CD                1156  ; was the shifted-out bit 1?
E5CD  7D 01 1E      1157                  TST     TMPA
E5D0  27 08         1158                  BEQ     RND_X     ; no: no feedback
E5D2  B6 01 17      1159                  LDAA    SEED      ; A = high byte
E5D5                1160  ; apply the $B400 taps
E5D5  88 B4         1161                  EORA    #$B4
E5D7                1162  ; store the high byte
E5D7  B7 01 17      1163                  STAA    SEED
E5DA                1164  ; return the low byte in A
E5DA  B6 01 18      1165  RND_X           LDAA    SEED+1
E5DD  39            1166                  RTS               ; done
E5DE                1167
E5DE                1168  ; BEEP -- Rubout's BEEP: pulse PIA1 CRB bit 3
E5DE                1169  BEEP
E5DE                1170  ; A = PIA1 control register B
E5DE  B6 78 03      1171                  LDAA    PIA1CRB
E5E1                1172  ; clear bit 3: bell line low
E5E1  84 F7         1173                  ANDA    #$F7
E5E3  B7 78 03      1174                  STAA    PIA1CRB   ; write it
E5E6                1175  ; set bit 3: bell line high
```

```
E5E6  8A 08         1176                ORAA    #$08
E5E8  B7 78 03      1177                STAA    PIA1CRB   ; write it
E5EB  39            1178                RTS               ; done
E5EC                1179
E5EC                1180  ; ================================================
E5EC                1181  ; TEXT (plain ASCII, stock character generator)
E5EC                1182  ; ================================================
E5EC                1183  ; score caption "YOU"
E5EC  59 4F 55      1184  SYOU            FCC     "YOU"
E5EF  00            1185                FCB     0         ; string terminator
E5F0                1186  ; score caption "CPU"
E5F0  43 50 55      1187  SCPU            FCC     "CPU"
E5F3  00            1188                FCB     0         ; string terminator
E5F4                1189  ; on-screen title with credit
E5F4  41 44 4D 2D   1190  STITLE          FCC     "ADM-31 PONG  By: Mickey Law>
E613  00            1191                FCB     0         ; string terminator
E614                1192  ; serve prompt
E614  53 50 41 43   1193  SSERVE          FCC     "SPACE=SERVE  UP/DOWN OR W/S>
E63D  00            1194                FCB     0         ; string terminator
E63E  50 41 55 53   1195  SPAUSE          FCC     "PAUSED"  ; pause banner
E644  00            1196                FCB     0         ; string terminator
E645                1197  ; win message
E645  59 4F 55 20   1198  SWIN            FCC     "YOU WIN!  SPACE = NEW GAME"
E65F  00            1199                FCB     0         ; string terminator
E660                1200  ; loss message
E660  43 50 55 20   1201  SLOSE           FCC     "CPU WINS  SPACE = NEW GAME"
E67A  00            1202                FCB     0         ; string terminator
E67B                1203
E67B                1204  ; pad the bank up to its last byte
E7FF                1205                ORG     $E7FF
E7FF                1206  ; end-of-bank marker
E7FF  FF            1207                FCB     $FF
E800                1208
E800                1209                END               ; end of source

SYMBOL TABLE
  HALT     7000  PIA1     7800  PIA1B    7802  PIA1CRB  7803  CRTRAM   8000
  CRTC6    7F06  CURCOL   7F0C  CH_PAD   0023  CH_WALL  003D  CH_NET   003A
  CH_BALL  004F  WINSC    0007  DLYSTART 1900  HPAD     0100  CPAD     0102
  BROW     0104  BCOL     0105  BHDIR    0106  BVDIR    0107  BVRATE   0108
  BVCNT    0109  CURADR   010A  OLDADR   010C  OLDCOL   010E  SCOREH   010F
  SCOREC   0110  TICK     0111  AIMOFF   0112  POINT    0113  LOSER    0114
  DLYCNT   0115  SEED     0117  TRIES    0119  ROWARG   011A  COLARG   011B
  ADRPTR   011C  TMPA     011E  TMPB     011F  TMPX     0120  KPTR     0122
  PSRC     0124  MPDIR    0126  KEYENTRY E002  PLUGCB2  E07D  STUB87   E087
  STUB94   E094  STUBC3   E0C3  PLUGCB3  E0CA  RESET    E100  NEWMATCH E121
  SERVEPT  E132  GAME     E14E  GDLY     E15A  GP_CPU   E179  GP_SHOW  E181
  GP_N1    E190  GP_NEXT  E19C  ENDGAME  E19F  SERVE    E1BB  SV_DLY   E20E
  BALLMOV  E212  BM_EVAL  E229  BM_PAD   E25D  BM_FREE  E265  BM_ERASE E27B
  BM_NET   E297  BM_WR    E299  BM_DONE  E29F  BM_HMISS E2A0  BM_CMISS E2A7
  BALLCOL  E2AE  BC_R     E2B7  BALLROW  E2BB  BR_STEP  E2C8  BR_D     E2D1
  BR_X     E2D4  PADHIT   E2D5  PH_REC   E2E5  PH_1     E2F3  PH_2     E2FD
  PH_3     E303  PH_4     E30F  PH_SET   E316  PH_AIM   E336  PH_X     E357
  POLLHUM  E358  PK_NP    E365  PK_LP    E368  PK_HIT   E378  PK_END   E380
  KEYTAB   E381  MOVPAD   E38E  MP_UP    E3C5  MP_X     E3DC  CPUAI    E3DD
  CA_TGT   E3F1  CA_UP    E404  CA_MV    E406  CA_X     E40C  PAUSE    E40D
  PZW      E420  PZREL    E430  WAITSP   E43B  WSREL    E452  PUTAT    E467
```

```
  PUTSTR   E477  PS_LP    E47A  PS_X     E48D  HLINE    E48E  CLRSCR   E499
  CS_LP    E49D  MSGCLR   E4AA  SHOWSC   E4B3  DRAWPAD  E4D0  DP_LP    E4DF
  DRAWFLD  E4ED  DF_NET   E50B  CALCADR  E565  RAND     E5BF  RND_X    E5DA
  BEEP     E5DE  SYOU     E5EC  SCPU     E5F0  STITLE   E5F4  SSERVE   E614
  SPAUSE   E63E  SWIN     E645  SLOSE    E660

CROSS-REFERENCE TABLE
SYMBOL          TYP  VAL   DEF     CNT  REFERENCES
--------------  ---  ----  -----  ----  ------------------------------------
ADRPTR          EQU  011C  L116     25  L1089 L1092 L1094 L1096 L1097 L1099
                                        L1100 L1102 L1103 L1104 L1106 L1108
                                        L1112 L1114 L1115 L1116 L1120 L1122
                                        L1126 L1127 L1130 L1132 L1133 L1138
                                        L1140
AIMOFF          EQU  0112  L101       3  L236  L680  L810
BALLCOL         LAB  E2AE  L542       2  L430  L472
BALLMOV         LAB  E212  L417       1  L275
BALLROW         LAB  E2BB  L557       1  L432
BCOL            EQU  0105  L80        9  L358  L386  L422  L436  L476  L548
                                        L551  L591  L655
BC_R            LAB  E2B7  L551       1  L546
BEEP            LAB  E5DE  L1169      2  L291  L636
BHDIR           EQU  0106  L82        4  L366  L544  L588  L806
BM_CMISS        LAB  E2A7  L533       1  L483
BM_DONE         LAB  E29F  L526       2  L499  L509
BM_ERASE        LAB  E27B  L493       2  L532  L538
BM_EVAL         LAB  E229  L433       2  L467  L473
BM_FREE         LAB  E265  L475       4  L451  L454  L457  L462
BM_HMISS        LAB  E2A0  L527       1  L479
BM_NET          LAB  E297  L519       1  L515
BM_PAD          LAB  E25D  L468       1  L459
BM_WR           LAB  E299  L521       1  L517
BROW            EQU  0104  L79        7  L362  L384  L434  L572  L575  L598
                                        L809
BR_D            LAB  E2D1  L575       1  L570
BR_STEP         LAB  E2C8  L568       1  L466
BR_X            LAB  E2D4  L576       2  L560  L564
BVCNT           EQU  0109  L88        4  L377  L562  L566  L634
BVDIR           EQU  0107  L84        7  L369  L464  L568  L602  L608  L622
                                        L628
BVRATE          EQU  0108  L86        3  L375  L559  L632
CALCADR         LAB  E565  L1087      3  L391  L439  L917
CA_MV           LAB  E406  L827       1  L824
CA_TGT          LAB  E3F1  L812       1  L807
CA_UP           LAB  E404  L825       1  L822
CA_X            LAB  E40C  L830       2  L802  L820
CH_BALL         EQU  004F  L68        3  L397  L487  L507
CH_NET          EQU  003A  L67        3  L455  L519  L1037
CH_PAD          EQU  0023  L64        4  L458  L765  L779  L1012
CH_WALL         EQU  003D  L66        3  L460  L1023 L1027
CLRSCR          LAB  E499  L962       1  L239
COLARG          EQU  011B  L115      15  L257  L342  L387  L437  L756  L841
                                        L944  L988  L995  L1006 L1036 L1047
                                        L1052 L1058 L1128
CPAD            EQU  0102  L78        6  L597  L814  L827  L1068 L1076 L1081
CPUAI           LAB  E3DD  L798       1  L273
CRTC6           EQU  7F06  L60        1  L219
```

```
CRTRAM          EQU  8000  L58       5  L964  L973  L978  L1024 L1029
CS_LP           LAB  E49D  L967      1  L974
CURADR          EQU  010A  L90       5  L393  L419  L441  L485  L497
CURCOL          EQU  7F0C  L62       1  L223
DF_NET          LAB  E50B  L1037     1  L1044
DLYCNT          EQU  0115  L107      7  L277  L381  L638  L644  L648  L650
                                        L654
DLYSTART        EQU  1900  L72       1  L379
DP_LP           LAB  E4DF  L1012     1  L1017
DRAWFLD         LAB  E4ED  L1022     1  L248
DRAWPAD         LAB  E4D0  L1004     2  L1079 L1083
ENDGAME         LAB  E19F  L332      2  L320  L328
GAME            LAB  E14E  L269      1  L286
GDLY            LAB  E15A  L279      1  L281
GP_CPU          LAB  E179  L306      1  L296
GP_N1           LAB  E190  L322      1  L317
GP_NEXT         LAB  E19C  L330      1  L325
GP_SHOW         LAB  E181  L312      1  L304
HALT            EQU  7000  L51       8  L399  L443  L489  L501  L521  L921
                                        L952  L967
HLINE           LAB  E48E  L950      4  L959  L982  L1026 L1032
HPAD            EQU  0100  L77       5  L590  L716  L1066 L1072 L1078
KEYENTRY        LAB  E002  L130      0
KEYTAB          LAB  E381  L726      1  L697
KPTR            EQU  0122  L120      2  L334  L344
LOSER           EQU  0114  L105      4  L246  L302  L310  L364
MOVPAD          LAB  E38E  L737      2  L718  L829
MPDIR           EQU  0126  L123      3  L739  L743  L758
MP_UP           LAB  E3C5  L776      1  L759
MP_X            LAB  E3DC  L789      2  L747  L751
MSGCLR          LAB  E4AA  L977      4  L251  L263  L336  L854
NEWMATCH        LAB  E121  L237      1  L349
OLDADR          EQU  010C  L92       3  L395  L421  L495
OLDCOL          EQU  010E  L94       3  L389  L424  L511
PADHIT          LAB  E2D5  L586      1  L470
PAUSE           LAB  E40D  L835      1  L695
PH_1            LAB  E2F3  L606      1  L601
PH_2            LAB  E2FD  L612      1  L607
PH_3            LAB  E303  L617      1  L613
PH_4            LAB  E30F  L626      1  L619
PH_AIM          LAB  E336  L655      1  L642
PH_REC          LAB  E2E5  L598      1  L595
PH_SET          LAB  E316  L632      4  L605  L611  L616  L625
PH_X            LAB  E357  L681      1  L659
PIA1            EQU  7800  L53      16  L690  L692  L703  L705  L847  L849
                                        L857  L859  L877  L879  L885  L887
                                        L893  L895  L901  L903
PIA1B           EQU  7802  L55       2  L224  L228
PIA1CRB         EQU  7803  L56       3  L1171 L1174 L1177
PK_END          LAB  E380  L719      1  L701
PK_HIT          LAB  E378  L714      1  L707
PK_LP           LAB  E368  L699      1  L712
PK_NP           LAB  E365  L697      1  L694
PLUGCB2         LAB  E07D  L171      0
PLUGCB3         LAB  E0CA  L203      0
POINT           EQU  0113  L103      5  L284  L293  L383  L530  L536
POLLHUM         LAB  E358  L687      1  L271
```

```
PSRC            EQU  0124  L121      3  L932  L934  L941
PS_LP           LAB  E47A  L934      1  L945
PS_X            LAB  E48D  L946      1  L938
PUTAT           LAB  E467  L913      9  L767  L774  L781  L788  L943  L992
                                        L1001 L1013 L1038
PUTSTR          LAB  E477  L930      6  L260  L346  L844  L1049 L1055 L1060
PZREL           LAB  E430  L855      3  L837  L852  L861
PZW             LAB  E420  L845      1  L851
RAND            LAB  E5BF  L1146     5  L359  L367  L370  L661  L873
RESET           LAB  E100  L211      1  L152
RND_X           LAB  E5DA  L1165     1  L1158
ROWARG          EQU  011A  L113     19  L254  L339  L385  L435  L764  L771
                                        L778  L785  L839  L986  L1009 L1014
                                        L1034 L1039 L1040 L1045 L1050 L1056
                                        L1090
SCOREC          EQU  0110  L97       4  L242  L306  L322  L997
SCOREH          EQU  010F  L95       4  L240  L298  L313  L989
SCPU            LAB  E5F0  L1187     1  L1059
SEED            EQU  0117  L109      7  L232  L1148 L1153 L1155 L1159 L1163
                                        L1165
SERVE           LAB  E1BB  L355      1  L265
SERVEPT         LAB  E132  L249      1  L330
SHOWSC          LAB  E4B3  L985      2  L312  L1062
SLOSE           LAB  E660  L1201     1  L327
SPAUSE          LAB  E63E  L1195     1  L843
SSERVE          LAB  E614  L1193     1  L259
STITLE          LAB  E5F4  L1190     1  L1054
STUB87          LAB  E087  L181      0
STUB94          LAB  E094  L195      0
STUBC3          LAB  E0C3  L199      0
SV_DLY          LAB  E20E  L406      1  L408
SWIN            LAB  E645  L1198     1  L319
SYOU            LAB  E5EC  L1184     1  L1048
TICK            EQU  0111  L99       3  L234  L282  L799
TMPA            EQU  011E  L117      8  L663  L669  L812  L818  L915  L919
                                        L1151 L1157
TMPB            EQU  011F  L118      9  L667  L676  L754  L761  L768  L776
                                        L782  L1011 L1015
TMPX            EQU  0120  L119      0
TRIES           EQU  0119  L111      2  L428  L449
WAITSP          LAB  E43B  L871      3  L262  L348  L889
WINSC           EQU  0007  L70       2  L315  L324
WSREL           LAB  E452  L891      3  L881  L897  L905

======================================================================
ASSEMBLY SUMMARY
======================================================================
TOTAL ERRORS: 0
STATUS: ASSEMBLY SUCCESSFUL - NO ERRORS
======================================================================
```

Appendix F

The firmware listing: 129867

The complete firmware of the 129867 set, `$E800-$FFFF`, 6,144 bytes, regenerated from the three ROM images by the disassembler of Chapter 24 and assembled again with the assembler of Appendix K. The reassembly is **byte-exact**: the output is the same 6,144 bytes as the ROM files. That is what makes this a listing and not a transcription: if a byte were wrong here, the comparison would fail.

What is *not* verified is the commentary. The labels with names (`RESET`, `KBSCAN`, `ESCDISP` and so on) were given by the author after reading the code, and the comments describe what the code does, as far as it was read. A label of the form `Lxxxx` is an address that the disassembler found to be a branch or call target and that nobody has named. The tables (`KBDTAB`, `ESCTAB`, `CTLTAB`, `JMPTAB`, `BAUDTAB`, the init tables) are printed as data bytes, and the instructions the assembler cannot emit in extended mode below `$100` are printed as `FCB` bytes with the instruction in the comment (Chapter 3).

```
0000                    2  ; FW867 - ADM-31 firmware 129867 (-67,-66,-65)
0000                    3  ; Socket order: B $E800-$EFFF, C $F000-$F7FF,
0000                    4  ; D $F800-$FFFF.
0000                    5  ; Reassembled from the ROM images; the names
0000                    6  ; are the author's.
0000                    7  ;
0000                    8  ; ---- I/O ports ----
0000  =7000             9  HALT            EQU     $7000
0000  =7800            10  PIA1A           EQU     $7800
0000  =7801            11  PIA1CRA         EQU     $7801
0000  =7802            12  PIA1B           EQU     $7802
0000  =7803            13  PIA1CRB         EQU     $7803
0000  =7900            14  SWBNK2          EQU     $7900
0000  =7A00            15  ACIA            EQU     $7A00
0000  =7A01            16  ACIAD           EQU     $7A01
0000  =7B00            17  PERROM          EQU     $7B00
0000  =7C00            18  PACIA           EQU     $7C00
0000  =7C01            19  PACIAD          EQU     $7C01
0000  =7D00            20  SWBNK3          EQU     $7D00
0000  =7E00            21  SWBNK1          EQU     $7E00
0000  =7E80            22  BAUDSW          EQU     $7E80
0000  =7F00            23  VTAC0           EQU     $7F00
0000  =7F01            24  VTAC1           EQU     $7F01
0000  =7F02            25  VTAC2           EQU     $7F02
0000  =7F03            26  VTAC3           EQU     $7F03
0000  =7F04            27  VTAC4           EQU     $7F04
```

```
0000  =7F05           28  VTAC5           EQU     $7F05
0000  =7F06           29  VTAC6           EQU     $7F06
0000  =7F07           30  VTAC7           EQU     $7F07
0000  =7F08           31  VTAC8           EQU     $7F08
0000  =7F09           32  VTAC9           EQU     $7F09
0000  =7F0A           33  VTACA           EQU     $7F0A
0000  =7F0B           34  VTACB           EQU     $7F0B
0000  =7F0C           35  VTACC           EQU     $7F0C
0000  =7F0D           36  VTACD           EQU     $7F0D
0000  =7F0E           37  VTACE           EQU     $7F0E
0000                  38  ; ---- zero-page variables ----
0000  =0000           39  PLUGFLG         EQU     $00
0000  =0011           40  CRTBASE         EQU     $11
0000  =0013           41  CURADR          EQU     $13
0000  =0015           42  FIRSTLN         EQU     $15
0000  =0016           43  ROW             EQU     $16
0000  =0017           44  COL             EQU     $17
0000  =0018           45  PROTECT         EQU     $18
0000  =0019           46  ATTRMSK         EQU     $19
0000  =001A           47  MONMODE         EQU     $1A
0000  =001D           48  KBLOCK          EQU     $1D
0000  =0042           49  SPLIT           EQU     $42
0000  =0045           50  ESCCHR          EQU     $45
0000  =004D           51  BLANK           EQU     $4D
0000  =009D           52  CURTASK         EQU     $9D
0000  =00BF           53  ACIACTL         EQU     $BF
0000  =00CB           54  TXCOUNT         EQU     $CB
0000                  55
E800                  56                  ORG     $E800
E800                  57  ; Power-on / reset entry.  The IRQ vector is $ED83;
E800                  58  ; SWI, NMI and RESET all point here (see the vector
E800                  59  ; table at $FFF8).
E800  0F              60  RESET           SEI
E801  B7 7F 0E        61                  STAA    VTACE
E804  B7 7F 0A        62                  STAA    VTACA
E807  CE 7F 00        63                  LDX     #$7F00
E80A  86 65           64                  LDAA    #$65
E80C  A7 00           65                  STAA    $00,X
E80E  86 44           66                  LDAA    #$44
E810  A7 01           67                  STAA    $01,X
E812  86 5D           68                  LDAA    #$5D
E814  A7 02           69                  STAA    $02,X
E816  B6 7E 00        70                  LDAA    SWBNK1
E819  84 02           71                  ANDA    #$02
E81B  27 0A           72                  BEQ     LE827
E81D  86 20           73                  LDAA    #$20
E81F  A7 04           74                  STAA    $04,X
E821  86 14           75                  LDAA    #$14
E823  A7 05           76                  STAA    $05,X
E825  20 08           77                  BRA     LE82F
E827  86 40           78  LE827           LDAA    #$40
E829  A7 04           79                  STAA    $04,X
E82B  86 35           80                  LDAA    #$35
E82D  A7 05           81                  STAA    $05,X
E82F  86 17           82  LE82F           LDAA    #$17
E831  A7 03           83                  STAA    $03,X
E833  A7 06           84                  STAA    $06,X
```

```
E835  97 15           85                STAA    FIRSTLN
E837  4F              86                CLRA
E838  A7 0D           87                STAA    $0D,X
E83A  A7 0C           88                STAA    $0C,X
E83C  A7 0E           89                STAA    $0E,X
E83E  8E 03 FF        90                LDS     #$03FF
E841  7F 8F 80        91                CLR     $8F80
E844  7F 8F 81        92                CLR     $8F81
E847  CE 03 FF        93                LDX     #$03FF
E84A  C6 AA           94                LDAB    #$AA
E84C  E7 00           95  LE84C         STAB    $00,X
E84E  09              96                DEX
E84F  26 FB           97                BNE     LE84C
E851  CE 03 FF        98                LDX     #$03FF
E854  C6 AA           99  LE854         LDAB    #$AA
E856  E1 00          100                CMPB    $00,X
E858  26 10          101                BNE     LE86A
E85A  53             102                COMB
E85B  E7 00          103                STAB    $00,X
E85D  E1 00          104                CMPB    $00,X
E85F  26 09          105                BNE     LE86A
E861  6F 00          106  LE861         CLR     $00,X
E863  09             107                DEX
E864  26 EE          108                BNE     LE854
E866  6F 00          109                CLR     $00,X
E868  20 0A          110                BRA     LE874
E86A  A6 00          111  LE86A         LDAA    $00,X
E86C  B7 8F 81       112                STAA    $8F81
E86F  F7 8F 80       113                STAB    $8F80
E872  20 ED          114                BRA     LE861
E874  B6 8F 81       115  LE874         LDAA    $8F81
E877  36             116                PSHA
E878  B6 8F 80       117                LDAA    $8F80
E87B  36             118                PSHA
E87C  CE 8F FF       119                LDX     #$8FFF
E87F  C6 AA          120                LDAB    #$AA
E881  E7 00          121  LE881         STAB    $00,X
E883  09             122                DEX
E884  8C 80 00       123                CPX     #$8000
E887  26 F8          124                BNE     LE881
E889  CE 8F FF       125                LDX     #$8FFF
E88C  C6 AA          126  LE88C         LDAB    #$AA
E88E  E1 00          127                CMPB    $00,X
E890  26 11          128                BNE     LE8A3
E892  53             129                COMB
E893  E7 00          130                STAB    $00,X
E895  E1 00          131                CMPB    $00,X
E897  26 0A          132                BNE     LE8A3
E899  6F 00          133  LE899         CLR     $00,X
E89B  09             134                DEX
E89C  8C 80 00       135                CPX     #$8000
E89F  26 EB          136                BNE     LE88C
E8A1  20 08          137                BRA     LE8AB
E8A3  A6 00          138  LE8A3         LDAA    $00,X
E8A5  97 BA          139                STAA    $BA
E8A7  D7 B9          140                STAB    $B9
E8A9  20 EE          141                BRA     LE899
```

```
E8AB  96 BA          142  LE8AB           LDAA    $BA
E8AD  36             143                  PSHA
E8AE  96 B9          144                  LDAA    $B9
E8B0  36             145                  PSHA
E8B1  4F             146                  CLRA
E8B2  CE E8 00       147                  LDX     #RESET
E8B5  A8 00          148  ROMSUM          EORA    $00,X
E8B7  08             149                  INX
E8B8  26 FB          150                  BNE     ROMSUM
E8BA  36             151                  PSHA
E8BB                 152  ; Plug-in probe: is there a ROM in socket A that
E8BB                 153  ; starts "TJ"?
E8BB  FE E0 00       154  PLUGPROB        LDX     $E000
E8BE  8C 54 4A       155                  CPX     #$544A
E8C1  26 0D          156                  BNE     NOPLUG
E8C3  B6 7D 00       157                  LDAA    SWBNK3
E8C6  2B 08          158                  BMI     NOPLUG
E8C8  7C 00 00       159                  FCB     $7C,$00,$00 ; INC $0000
E8CB  BD E0 7D       160                  JSR     $E07D
E8CE  20 04          161                  BRA     PIAINIT
E8D0                 162  ; No usable plug-in: continue the normal boot.
E8D0  86 1D          163  NOPLUG          LDAA    #$1D
E8D2  97 DB          164                  STAA    $DB
E8D4                 165  ; Initialise PIA 1 (keyboard, display page, bell).
E8D4  CE 78 00       166  PIAINIT         LDX     #$7800
E8D7  6F 01          167                  CLR     $01,X
E8D9  6F 03          168                  CLR     $03,X
E8DB  86 7F          169                  LDAA    #$7F
E8DD  A7 00          170                  STAA    $00,X
E8DF  86 1D          171                  LDAA    #$1D
E8E1  A7 02          172                  STAA    $02,X
E8E3  86 3C          173                  LDAA    #$3C
E8E5  A7 01          174                  STAA    $01,X
E8E7  A7 03          175                  STAA    $03,X
E8E9  86 01          176                  LDAA    #$01
E8EB  A7 00          177                  STAA    $00,X
E8ED  A7 02          178                  STAA    $02,X
E8EF  86 43          179                  LDAA    #$43
E8F1  B7 7A 00       180                  STAA    ACIA
E8F4  B7 7C 00       181                  STAA    PACIA
E8F7  B6 7E 00       182                  LDAA    SWBNK1
E8FA  16             183                  TAB
E8FB  84 1C          184                  ANDA    #$1C
E8FD  8A C1          185                  ORAA    #$C1
E8FF  97 BF          186                  STAA    ACIACTL
E901  B7 7A 00       187                  STAA    ACIA
E904  53             188                  COMB
E905  C4 20          189                  ANDB    #$20
E907  58             190                  ASLB
E908  58             191                  ASLB
E909  D7 CE          192                  STAB    $CE
E90B  B6 79 00       193                  LDAA    SWBNK2
E90E  84 1C          194                  ANDA    #$1C
E910  8A 41          195                  ORAA    #$41
E912  97 C0          196                  STAA    $C0
E914  B7 7C 00       197                  STAA    PACIA
E917  86 17          198                  LDAA    #$17
```

```
E919  CE 80 00       199                LDX     #$8000
E91C  DF 11          200                STX     CRTBASE
E91E  DF 13          201                STX     CURADR
E920  97 15          202                STAA    FIRSTLN
E922  FF 87 D0       203                STX     $87D0
E925  FF 87 D2       204                STX     $87D2
E928  B7 87 D4       205                STAA    $87D4
E92B  CE 88 00       206                LDX     #$8800
E92E  FF 8F D0       207                STX     $8FD0
E931  FF 8F D2       208                STX     $8FD2
E934  B7 8F D4       209                STAA    $8FD4
E937  BD ED 7B       210                JSR     LED7B
E93A  BD FD 6A       211                JSR     LFD6A
E93D  CE 00 45       212                LDX     #$0045
E940  DF B1          213                STX     $B1
E942  FE 7B 1E       214                LDX     $7B1E
E945  8C 30 46       215                CPX     #$3046
E948  26 05          216                BNE     LE94F
E94A  CE 7B 00       217                LDX     #$7B00
E94D  20 0C          218                BRA     LE95B
E94F  96 00          219  LE94F         LDAA    PLUGFLG
E951  26 05          220                BNE     LE958
E953  CE E9 F8       221                LDX     #INITTAB
E956  20 03          222                BRA     LE95B
E958  CE E5 30       223  LE958         LDX     #$E530
E95B  DF 43          224  LE95B         STX     $43
E95D  DF B3          225                STX     $B3
E95F  C6 1E          226                LDAB    #$1E
E961  BD F6 BB       227                JSR     BLITCPY
E964  B6 00 4D       228                FCB     $B6,$00,$4D ; LDAA $004D
E967  F6 00 55       229                FCB     $F6,$00,$55 ; LDAB $0055
E96A  C5 40          230                BITB    #$40
E96C  26 02          231                BNE     LE970
E96E  86 20          232                LDAA    #$20
E970  36             233  LE970         PSHA
E971  BD F6 5E       234                JSR     LF65E
E974  BD F7 90       235                JSR     LF790
E977  B6 00 52       236                FCB     $B6,$00,$52 ; LDAA $0052
E97A  BD F5 56       237                JSR     LF556
E97D  BD F6 5E       238                JSR     LF65E
E980  BD F7 90       239                JSR     LF790
E983  B6 00 52       240                FCB     $B6,$00,$52 ; LDAA $0052
E986  BD F5 56       241                JSR     LF556
E989  32             242                PULA
E98A  30             243                TSX
E98B  A6 01          244                LDAA    $01,X
E98D  AA 03          245                ORAA    $03,X
E98F  AA 00          246                ORAA    $00,X
E991  27 42          247                BEQ     LE9D5
E993  CE E9 BD       248                LDX     #ERRMSG
E996  8D 32          249                BSR     LE9CA
E998  BD F3 13       250                JSR     LF313
E99B  30             251                TSX
E99C  C6 05          252                LDAB    #$05
E99E  BD F1 D0       253                JSR     LF1D0
E9A1  32             254                PULA
E9A2  32             255                PULA
```

```
E9A3  32             256                 PULA
E9A4  32             257                 PULA
E9A5  32             258                 PULA
E9A6  BD EF 8B       259  LE9A6          JSR     LEF8B
E9A9  7D 00 AE       260                 FCB     $7D,$00,$AE ; TST $00AE
E9AC  2A F8          261                 BPL     LE9A6
E9AE  20 2A          262                 BRA     LE9DA
E9B0                 263  ; Self-test messages (13 characters each).
E9B0  54 65 73 74    264  TSTMSG         FCB     "Test Complete"
E9BD  4D 45 4D 4F    265  ERRMSG         FCB     "MEMORY ERROR "
E9CA  DF B3          266  LE9CA          STX     $B3
E9CC  C6 0D          267                 LDAB    #$0D
E9CE  DE 13          268                 LDX     CURADR
E9D0  DF B1          269                 STX     $B1
E9D2  7E F6 BB       270                 JMP     BLITCPY
E9D5  CE E9 B0       271  LE9D5          LDX     #TSTMSG
E9D8  8D F0          272                 BSR     LE9CA
E9DA  B6 00 56       273  LE9DA          FCB     $B6,$00,$56 ; LDAA $0056
E9DD  16             274                 TAB
E9DE  84 01          275                 ANDA    #$01
E9E0  B7 78 02       276                 STAA    PIA1B
E9E3  C5 08          277                 BITB    #$08
E9E5  27 03          278                 BEQ     LE9EA
E9E7  BD F0 B3       279                 JSR     LF0B3
E9EA  B6 7E 00       280  LE9EA          LDAA    SWBNK1
E9ED  84 C0          281                 ANDA    #$C0
E9EF  97 41          282                 STAA    $41
E9F1  BD F1 43       283                 JSR     LF143
E9F4  7E EB F8       284                 JMP     LEBF8
E9F7  03             285                 FCB     $03
E9F8  1B 0D 00 00    286  INITTAB        FCB     $1B,$0D,$00,$00,$00,$00
E9FE  01 00 20 00    287                 FCB     $01,$00,$20,$00,$00,$00
EA04  00 00 02 F0    288                 FCB     $00,$00,$02,$F0,$03,$01
EA0A  00 00 00 00    289                 FCB     $00,$00,$00,$00,$00,$00
EA10  00 00 00 00    290                 FCB     $00,$00,$00,$00,$00,$00
EA16  00 00          291                 FCB     $00,$00
EA18  03 80          292  LEA18          FCB     $03,$80
EA1A                 293  ; Keyboard table: 88 scan codes x {unshifted,
EA1A                 294  ; shifted}.
EA1A  D1 D1 D7 D7    295  KBDTAB         FCB     $D1,$D1,$D7,$D7,$C5,$C5
EA20  D2 D2 D4 F4    296                 FCB     $D2,$D2,$D4,$F4,$D9,$F9
EA26  32 22 1B 1B    297                 FCB     $32,$22,$1B,$1B,$7F,$7F
EA2C  9E 9F D0 F0    298                 FCB     $9E,$9F,$D0,$F0,$B4,$B6
EA32  92 93 31 21    299                 FCB     $92,$93,$31,$21,$71,$51
EA38  98 98 33 23    300                 FCB     $98,$98,$33,$23,$34,$24
EA3E  35 25 36 26    301                 FCB     $35,$25,$36,$26,$37,$27
EA44  38 28 39 29    302                 FCB     $38,$28,$39,$29,$30,$5F
EA4A  2D 3D 5E 7E    303                 FCB     $2D,$3D,$5E,$7E,$5C,$7C
EA50  09 C9 87 87    304                 FCB     $09,$C9,$87,$87,$88,$88
EA56  89 89 2D 2D    305                 FCB     $89,$89,$2D,$2D,$77,$57
EA5C  65 45 72 52    306                 FCB     $65,$45,$72,$52,$74,$54
EA62  79 59 75 55    307                 FCB     $79,$59,$75,$55,$69,$49
EA68  6F 4F 70 50    308                 FCB     $6F,$4F,$70,$50,$5B,$7B
EA6E  5D 7D 0D 0D    309                 FCB     $5D,$7D,$0D,$0D,$84,$84
EA74  85 85 86 86    310                 FCB     $85,$85,$86,$86,$09,$09
EA7A  61 41 73 53    311                 FCB     $61,$41,$73,$53,$64,$44
EA80  66 46 67 47    312                 FCB     $66,$46,$67,$47,$68,$48
```

```
EA86  6A 4A 6B 4B    313                FCB     $6A,$4A,$6B,$4B,$6C,$4C
EA8C  3B 2B 3A 2A    314                FCB     $3B,$2B,$3A,$2A,$40,$60
EA92  0A 0A 82 82    315                FCB     $0A,$0A,$82,$82,$83,$83
EA98  81 81 90 91    316                FCB     $81,$81,$90,$91,$7A,$5A
EA9E  78 58 63 43    317                FCB     $78,$58,$63,$43,$76,$56
EAA4  62 42 6E 4E    318                FCB     $62,$42,$6E,$4E,$6D,$4D
EAAA  2C 3C 2E 3E    319                FCB     $2C,$3C,$2E,$3E,$2F,$3F
EAB0  20 20 1F CA    320                FCB     $20,$20,$1F,$CA,$30,$30
EAB6  2E 2E 0D 0D    321                FCB     $2E,$2E,$0D,$0D,$9D,$94
EABC  1E 1E 0A 0A    322                FCB     $1E,$1E,$0A,$0A,$0B,$0B
EAC2  08 08 0C 0C    323                FCB     $08,$08,$0C,$0C
EAC6  E3 00 2C 2C    324  LEAC6           FCB     $E3,$00,$2C,$2C
EACA                 325  ; ESC table: one handler per character $10-$7F that
EACA                 326  ; follows ESC.
EACA  EB EA EF EA    327  ESCTAB          FDB     ESCNOP,LEFEA,LEF4A,LEF4E
EAD2  EF C6 EB EA    328                FDB     LEFC6,ESCNOP,ESCNOP,ESCNOP
EADA  EF 61 EB EA    329                FDB     LEF61,ESCNOP,ESCNOP,ESCNOP
EAE2  EB EA EF C6    330                FDB     ESCNOP,LEFC6,SENDBRK,LEFA5
EAEA  EB EA EB EA    331                FDB     ESCNOP,ESCNOP,LF0AF,LF0AA
EAF2  EB EA EB EA    332                FDB     ESCNOP,ESCNOP,LF060,LF070
EAFA  F0 79 F0 74    333                FDB     LF079,LF074,LF767,CLRSCR
EB02  F7 63 F5 67    334                FDB     LF763,LF567,LF0F3,LF9D1
EB0A  F5 01 F5 0F    335                FDB     LF501,LF50F,LF51B,LF553
EB12  F9 B5 F9 BD    336                FDB     LF9B5,LF9BD,LF9B9,LF9C1
EB1A  F2 37 F2 3C    337                FDB     LF237,LF23C,LF731,LF73B
EB22  EB EA F5 AA    338                FDB     ESCNOP,CURPOS,ESCNOP,LF9CD
EB2A  F2 15 F2 21    339                FDB     LF215,LF221,LF07D,LF083
EB32  F0 8A F8 74    340                FDB     LF08A,INSLINE,ESCNOP,LF0E8
EB3A  EB EA F4 58    341                FDB     ESCNOP,LF458,LF631,LF631
EB42  F2 11 EB EA    342                FDB     LF211,ESCNOP,ESCNOP,ESCNOP
EB4A  F2 0D F8 24    343                FDB     LF20D,LF824,DELLINE,LF9C5
EB52  F7 D5 F0 A1    344                FDB     CLREOL,LF0A1,ESCNOP,LF84B
EB5A  F0 A6 F6 D5    345                FDB     LF0A6,LF6D5,ESCNOP,ESCNOP
EB62  EB EA EB EA    346                FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EB6A  F2 2C EB EA    347                FDB     ESCGRAVE,ESCNOP,ESCNOP,ESCNOP
EB72  EB EA EB EA    348                FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EB7A  EB EA F3 AA    349                FDB     ESCNOP,LF3AA,ESCNOP,ESCNOP
EB82  F2 21 F2 40    350                FDB     LF221,LF240,ESCNOP,LF14E
EB8A  F2 11 F0 D1    351                FDB     LF211,LF0D1,LF0D6,LF9C9
EB92  F8 1D F0 A6    352                FDB     LF81D,LF0A6,LF0B3,LF0CD
EB9A  EB EA F6 E1    353                FDB     ESCNOP,LF6E1,ESCNOP,ESCNOP
EBA2  EB EA EB EA    354                FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EBAA                 355  ; Control-code table: one handler per code $00-$1F.
EBAA  EB EA EB EA    356  CTLTAB          FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EBB2  EB EA F9 D5    357                FDB     ESCNOP,LF9D5,ESCNOP,BELL
EBBA  F2 8C F3 AA    358                FDB     LF28C,LF3AA,LF27D,LF293
EBC2  F2 72 F2 A1    359                FDB     LF272,LF2A1,ESCNOP,ESCNOP
EBCA  EB EA EB EA    360                FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EBD2  EB EA EB EA    361                FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EBDA  EB EA EB EA    362                FDB     ESCNOP,ESCNOP,ESCNOP,ESCNOP
EBE2  EB EA EB EA    363                FDB     ESCNOP,ESCNOP,LF259,LF29A
EBEA  39             364  ESCNOP          RTS
EBEB  5D 00 00 00    365                FCB     $5D,$00,$00,$00,$00,$00
EBF1  00 00 00 00    366                FCB     $00,$00,$00,$00,$00,$00
EBF7  00             367                FCB     $00
EBF8  CE 00 85       368  LEBF8           LDX     #$0085
EBFB  86 80          369                LDAA    #$80
```

```
EBFD  C6 08          370                LDAB    #$08
EBFF  8D 4B          371                BSR     LEC4C
EC01  CE 00 8D       372                LDX     #$008D
EC04  C6 08          373                LDAB    #$08
EC06  4F             374                CLRA
EC07  8D 43          375                BSR     LEC4C
EC09  CE EE 06       376                LDX     #KBSCAN
EC0C  DF 65          377                STX     $65
EC0E  CE FD 27       378                LDX     #TXDELAY
EC11  DF 73          379                STX     $73
EC13  96 00          380                LDAA    PLUGFLG
EC15  26 1B          381                BNE     LEC32
EC17  CE EE A1       382                LDX     #KEYHAND
EC1A  DF 67          383                STX     $67
EC1C  CE ED 05       384                LDX     #LED05
EC1F  DF 69          385                STX     $69
EC21  CE ED 58       386                LDX     #LED58
EC24  DF 6B          387                STX     $6B
EC26  CE F9 F5       388                LDX     #LF9F5
EC29  DF 6D          389                STX     $6D
EC2B  CE FE 03       390                LDX     #LFE03
EC2E  DF 6F          391                STX     $6F
EC30  20 03          392                BRA     LEC35
EC32  BD E0 CA       393  LEC32           JSR     $E0CA
EC35  CE 00 00       394  LEC35           LDX     #$0000
EC38  DF 9D          395                STX     CURTASK
EC3A  DF 9F          396                STX     $9F
EC3C  CE 00 00       397                LDX     #$0000
EC3F  BD EC BA       398                JSR     WAKE
EC42  CE 00 02       399                LDX     #$0002
EC45  BD EC BA       400                JSR     WAKE
EC48  0E             401                CLI
EC49  7E EC 53       402                JMP     SCHED
EC4C  A7 00          403  LEC4C           STAA    $00,X
EC4E  08             404                INX
EC4F  5A             405                DECB
EC50  2E FA          406                BGT     LEC4C
EC52  39             407                RTS
EC53                 408  ; Cooperative scheduler: tasks wait with bit 7 of
EC53                 409  ; their status set.
EC53  DE 9D          410  SCHED           LDX     CURTASK
EC55  A6 85          411  LEC55           LDAA    $85,X
EC57  2A 33          412                BPL     LEC8C
EC59  08             413  LEC59           INX
EC5A  8C 00 08       414                CPX     #$0008
EC5D  26 F6          415                BNE     LEC55
EC5F  7F 00 9E       416                FCB     $7F,$00,$9E ; CLR $009E
EC62  F6 78 02       417                LDAB    PIA1B
EC65  C5 02          418                BITB    #$02
EC67  26 EA          419                BNE     SCHED
EC69  C6 34          420                LDAB    #$34
EC6B  F7 78 01       421                STAB    PIA1CRA
EC6E  C6 3C          422                LDAB    #$3C
EC70  F7 78 01       423                STAB    PIA1CRA
EC73  CE 00 00       424                LDX     #$0000
EC76  A6 8D          425  LEC76           LDAA    $8D,X
EC78  27 0A          426                BEQ     LEC84
```

```
EC7A  6A 8D          427                 DEC     $8D,X
EC7C  26 06          428                 BNE     LEC84
EC7E  A6 85          429                 LDAA    $85,X
EC80  84 7F          430                 ANDA    #$7F
EC82  A7 85          431                 STAA    $85,X
EC84  08             432  LEC84          INX
EC85  8C 00 08       433                 CPX     #$0008
EC88  26 EC          434                 BNE     LEC76
EC8A  20 C7          435                 BRA     SCHED
EC8C  DF 9D          436  LEC8C          STX     CURTASK
EC8E  DF 9F          437                 STX     $9F
EC90  16             438                 TAB
EC91  84 7C          439                 ANDA    #$7C
EC93  A7 85          440                 STAA    $85,X
EC95  A6 95          441                 LDAA    $95,X
EC97  78 00 A0       442                 FCB     $78,$00,$A0 ; ASL $00A0
EC9A  DE 9F          443                 LDX     $9F
EC9C  C5 01          444                 BITB    #$01
EC9E  27 0E          445                 BEQ     LECAE
ECA0  C5 02          446                 BITB    #$02
ECA2  27 06          447                 BEQ     LECAA
ECA4  36             448                 PSHA
ECA5  8D 1A          449                 BSR     BLOCKME
ECA7  32             450                 PULA
ECA8  DE 9F          451                 LDX     $9F
ECAA  EE 75          452  LECAA          LDX     $75,X
ECAC  20 02          453                 BRA     LECB0
ECAE  EE 65          454  LECAE          LDX     $65,X
ECB0  C4 30          455  LECB0          ANDB    #$30
ECB2  AD 00          456                 JSR     $00,X
ECB4  DE 9D          457                 LDX     CURTASK
ECB6  20 A1          458                 BRA     LEC59
ECB8  A7 95          459  LECB8          STAA    $95,X
ECBA                 460  ; WAKE: clear bit 7 of status byte X (task X may
ECBA                 461  ; run).
ECBA  A6 85          462  WAKE           LDAA    $85,X
ECBC  84 7F          463                 ANDA    #$7F
ECBE  A7 85          464                 STAA    $85,X
ECC0  39             465                 RTS
ECC1                 466  ; BLOCKME: the running task blocks itself.
ECC1  DE 9D          467  BLOCKME        LDX     CURTASK
ECC3                 468  ; BLOCKX: block task X.
ECC3  A6 85          469  BLOCKX         LDAA    $85,X
ECC5  8A 80          470                 ORAA    #$80
ECC7  A7 85          471                 STAA    $85,X
ECC9  6F 8D          472                 CLR     $8D,X
ECCB  39             473                 RTS
ECCC                 474  ; SLEEP: block the caller for A ticks (yield).
ECCC  DE 9D          475  SLEEP          LDX     CURTASK
ECCE  E6 85          476                 LDAB    $85,X
ECD0  CA 81          477                 ORAB    #$81
ECD2  A7 8D          478                 STAA    $8D,X
ECD4  26 02          479                 BNE     LECD8
ECD6  C4 7F          480                 ANDB    #$7F
ECD8  E7 85          481  LECD8          STAB    $85,X
ECDA  32             482                 PULA
ECDB  33             483                 PULB
```

```
ECDC  DF 9F          484                 STX     $9F
ECDE  78 00 A0       485                 FCB     $78,$00,$A0 ; ASL $00A0
ECE1  DE 9F          486                 LDX     $9F
ECE3  A7 75          487                 STAA    $75,X
ECE5  E7 76          488                 STAB    $76,X
ECE7  39             489                 RTS
ECE8                 490  ; GETRX: fetch the next received character (blocks
ECE8                 491  ; if none).
ECE8  DE 9D          492  GETRX          LDX     CURTASK
ECEA  E6 85          493                 LDAB    $85,X
ECEC  CA 83          494                 ORAB    #$83
ECEE  20 E8          495                 BRA     LECD8
ECF0  DE 9D          496  LECF0          LDX     CURTASK
ECF2  36             497                 PSHA
ECF3  A6 85          498                 LDAA    $85,X
ECF5  84 CF          499                 ANDA    #$CF
ECF7  C4 30          500                 ANDB    #$30
ECF9  1B             501                 ABA
ECFA  A7 85          502                 STAA    $85,X
ECFC  32             503                 PULA
ECFD  39             504                 RTS
ECFE  DE 9D          505  LECFE          LDX     CURTASK
ED00  E6 85          506                 LDAB    $85,X
ED02  C4 30          507                 ANDB    #$30
ED04  39             508                 RTS
ED05  DE A1          509  LED05          LDX     $A1
ED07  9C A3          510                 CPX     $A3
ED09  27 4C          511                 BEQ     LED57
ED0B  8D 25          512                 BSR     LED32
ED0D  7D 00 D2       513                 FCB     $7D,$00,$D2 ; TST $00D2
ED10  2F 09          514                 BLE     LED1B
ED12  BD FF 82       515                 JSR     LFF82
ED15  D6 D2          516                 LDAB    $D2
ED17  C5 02          517                 BITB    #$02
ED19  27 3C          518                 BEQ     LED57
ED1B  CE 00 03       519  LED1B          LDX     #$0003
ED1E  4D             520                 TSTA
ED1F  2A 0E          521                 BPL     LED2F
ED21  D6 4C          522                 LDAB    $4C
ED23  27 08          523                 BEQ     LED2D
ED25  E6 85          524                 LDAB    $85,X
ED27  C4 FE          525                 ANDB    #$FE
ED29  E7 85          526                 STAB    $85,X
ED2B  20 02          527                 BRA     LED2F
ED2D  84 7F          528  LED2D          ANDA    #$7F
ED2F  7E EC B8       529  LED2F          JMP     LECB8
ED32  A6 00          530  LED32          LDAA    $00,X
ED34  16             531                 TAB
ED35  D8 0E          532                 EORB    $0E
ED37  D7 0E          533                 STAB    $0E
ED39  08             534                 INX
ED3A  8C 03 80       535                 CPX     #$0380
ED3D  26 03          536                 BNE     LED42
ED3F  CE 01 10       537                 LDX     #$0110
ED42  4D             538  LED42          TSTA
ED43  2A 0D          539                 BPL     LED52
ED45  E6 00          540                 LDAB    $00,X
```

```
ED47  D7 A5          541                 STAB    $A5
ED49  08             542                 INX
ED4A  8C 03 80       543                 CPX     #$0380
ED4D  26 03          544                 BNE     LED52
ED4F  CE 01 10       545                 LDX     #$0110
ED52  DF A1          546  LED52          STX     $A1
ED54  7F 00 A6       547                 FCB     $7F,$00,$A6 ; CLR $00A6
ED57  39             548  LED57          RTS
ED58  36             549  LED58          PSHA
ED59  BD EC C1       550                 JSR     BLOCKME
ED5C  32             551                 PULA
ED5D  4D             552                 TSTA
ED5E  2A 18          553                 BPL     LED78
ED60  84 7F          554                 ANDA    #$7F
ED62  D6 A5          555                 LDAB    $A5
ED64  C5 10          556                 BITB    #$10
ED66  26 EF          557                 BNE     LED57
ED68  C5 60          558                 BITB    #$60
ED6A  27 0C          559                 BEQ     LED78
ED6C  D6 4C          560                 LDAB    $4C
ED6E  27 08          561                 BEQ     LED78
ED70  37             562                 PSHB
ED71  BD F0 DA       563                 JSR     BELL
ED74  32             564                 PULA
ED75  7E F0 3A       565                 JMP     LF03A
ED78  7E EF F6       566  LED78          JMP     LEFF6
ED7B  CE 01 10       567  LED7B          LDX     #$0110
ED7E  DF A3          568                 STX     $A3
ED80  DF A1          569                 STX     $A1
ED82  39             570                 RTS
ED83                 571  ; IRQ service: ACIA first, then the printer ACIA.
ED83  F6 7A 00       572  IRQ            LDAB    ACIA
ED86  2A 02          573                 BPL     LED8A
ED88  8D 09          574                 BSR     ACIAIRQ
ED8A  F6 7C 00       575  LED8A          LDAB    PACIA
ED8D  2A 03          576                 BPL     LED92
ED8F  BD FF AB       577                 JSR     LFFAB
ED92  3B             578  LED92          RTI
ED93  C5 01          579  ACIAIRQ        BITB    #$01
ED95  27 58          580                 BEQ     LEDEF
ED97  B6 7A 01       581                 LDAA    ACIAD
ED9A  84 7F          582                 ANDA    #$7F
ED9C  C4 70          583                 ANDB    #$70
ED9E  27 09          584                 BEQ     LEDA9
EDA0  C5 10          585                 BITB    #$10
EDA2  27 03          586                 BEQ     LEDA7
EDA4  D7 A8          587                 STAB    $A8
EDA6  39             588                 RTS
EDA7  8A 80          589  LEDA7          ORAA    #$80
EDA9  7D 00 A8       590  LEDA9          FCB     $7D,$00,$A8 ; TST $00A8
EDAC  27 04          591                 BEQ     LEDB2
EDAE  7F 00 A8       592                 FCB     $7F,$00,$A8 ; CLR $00A8
EDB1  39             593                 RTS
EDB2  7D 00 A6       594  LEDB2          FCB     $7D,$00,$A6 ; TST $00A6
EDB5  2B 31          595                 BMI     LEDE8
EDB7  DE A3          596                 LDX     $A3
EDB9  A7 00          597                 STAA    $00,X
```

```
EDBB  08          598                 INX
EDBC  8C 03 80    599                 CPX     #$0380
EDBF  26 03       600                 BNE     LEDC4
EDC1  CE 01 10    601                 LDX     #$0110
EDC4  DF A3       602  LEDC4          STX     $A3
EDC6  4D          603                 TSTA
EDC7  2A 0D       604                 BPL     LEDD6
EDC9  E7 00       605                 STAB    $00,X
EDCB  08          606                 INX
EDCC  8C 03 80    607                 CPX     #$0380
EDCF  26 03       608                 BNE     LEDD4
EDD1  CE 01 10    609                 LDX     #$0110
EDD4  DF A3       610  LEDD4          STX     $A3
EDD6  08          611  LEDD6          INX
EDD7  8C 03 80    612                 CPX     #$0380
EDDA  26 03       613                 BNE     LEDDF
EDDC  CE 01 10    614                 LDX     #$0110
EDDF  9C A1       615  LEDDF          CPX     $A1
EDE1  26 04       616                 BNE     LEDE7
EDE3  86 80       617                 LDAA    #$80
EDE5  97 A6       618                 STAA    $A6
EDE7  39          619  LEDE7          RTS
EDE8  86 80       620  LEDE8          LDAA    #$80
EDEA  97 A7       621                 STAA    $A7
EDEC  7E F0 DA    622                 JMP     BELL
EDEF  C5 02       623  LEDEF          BITB    #$02
EDF1  27 0B       624                 BEQ     LEDFE
EDF3  96 BF       625                 LDAA    ACIACTL
EDF5  84 60       626                 ANDA    #$60
EDF7  81 20       627                 CMPA    #$20
EDF9  26 03       628                 BNE     LEDFE
EDFB  7E FD 85    629                 JMP     LFD85
EDFE  C5 04       630  LEDFE          BITB    #$04
EE00  27 03       631                 BEQ     LEE05
EE02  B6 7A 01    632                 LDAA    ACIAD
EE05  39          633  LEE05          RTS
EE06              634  ; Keyboard scan task.
EE06  C6 04       635  KBSCAN         LDAB    #$04
EE08  96 A9       636  LEE08          LDAA    $A9
EE0A  B7 78 00    637                 STAA    PIA1A
EE0D  B6 78 00    638                 LDAA    PIA1A
EE10  2B 0B       639                 BMI     LEE1D
EE12  26 02       640                 BNE     LEE16
EE14  86 58       641                 LDAA    #$58
EE16  4A          642  LEE16          DECA
EE17  97 A9       643                 STAA    $A9
EE19  5A          644                 DECB
EE1A  26 EC       645                 BNE     LEE08
EE1C  39          646                 RTS
EE1D  F6 78 02    647  LEE1D          LDAB    PIA1B
EE20  D7 AB       648                 STAB    $AB
EE22  58          649                 ASLB
EE23  49          650                 ROLA
EE24  CE EA 1A    651                 LDX     #KBDTAB
EE27  BD EF AD    652                 JSR     XPLUSA
EE2A  C6 0F       653                 LDAB    #$0F
EE2C  D7 AD       654                 STAB    $AD
```

```
EE2E  A6 00          655                 LDAA    $00,X
EE30  2B 1E          656                 BMI     LEE50
EE32  D6 AB          657                 LDAB    $AB
EE34  C5 01          658                 BITB    #$01
EE36  27 0A          659                 BEQ     LEE42
EE38  81 61          660                 CMPA    #$61
EE3A  2D 06          661                 BLT     LEE42
EE3C  81 7A          662                 CMPA    #$7A
EE3E  2E 02          663                 BGT     LEE42
EE40  80 20          664                 SUBA    #$20
EE42  C5 40          665  LEE42          BITB    #$40
EE44  27 0A          666                 BEQ     LEE50
EE46  81 40          667                 CMPA    #$40
EE48  2D 06          668                 BLT     LEE50
EE4A  81 7F          669                 CMPA    #$7F
EE4C  27 02          670                 BEQ     LEE50
EE4E  84 1F          671                 ANDA    #$1F
EE50  4D             672  LEE50          TSTA
EE51  2A 0D          673                 BPL     LEE60
EE53  81 90          674                 CMPA    #$90
EE55  2C 09          675                 BGE     LEE60
EE57  7D 00 57       676                 FCB     $7D,$00,$57 ; TST $0057
EE5A  2B 04          677                 BMI     LEE60
EE5C  84 7F          678                 ANDA    #$7F
EE5E  8B 30          679                 ADDA    #$30
EE60  97 AA          680  LEE60          STAA    $AA
EE62  86 7D          681                 LDAA    #$7D
EE64  97 AC          682                 STAA    $AC
EE66  CE 00 01       683  LEE66          LDX     #$0001
EE69  96 AA          684                 LDAA    $AA
EE6B  BD EC B8       685                 JSR     LECB8
EE6E  86 0A          686                 LDAA    #$0A
EE70  BD EC CC       687                 JSR     SLEEP
EE73  96 AC          688                 LDAA    $AC
EE75  27 1C          689                 BEQ     LEE93
EE77  86 02          690  LEE77          LDAA    #$02
EE79  BD EC CC       691                 JSR     SLEEP
EE7C  B6 78 00       692                 LDAA    PIA1A
EE7F  2A 1C          693                 BPL     LEE9D
EE81  7A 00 AC       694                 FCB     $7A,$00,$AC ; DEC $00AC
EE84  26 F1          695                 BNE     LEE77
EE86  96 AD          696                 LDAA    $AD
EE88  81 03          697                 CMPA    #$03
EE8A  2F 03          698                 BLE     LEE8F
EE8C  7A 00 AD       699                 FCB     $7A,$00,$AD ; DEC $00AD
EE8F  97 AC          700  LEE8F          STAA    $AC
EE91  20 D3          701                 BRA     LEE66
EE93  86 04          702  LEE93          LDAA    #$04
EE95  BD EC CC       703                 JSR     SLEEP
EE98  B6 78 00       704                 LDAA    PIA1A
EE9B  2B F6          705                 BMI     LEE93
EE9D  7F 00 AC       706  LEE9D          FCB     $7F,$00,$AC ; CLR $00AC
EEA0  39             707                 RTS
EEA1  36             708  KEYHAND        PSHA
EEA2  BD EC C1       709                 JSR     BLOCKME
EEA5  32             710                 PULA
EEA6  16             711                 TAB
```

```
EEA7  C4 F8          712                ANDB    #$F8
EEA9  C1 98          713                CMPB    #$98
EEAB  27 0C          714                BEQ     LEEB9
EEAD  D6 1D          715                LDAB    KBLOCK
EEAF  27 08          716                BEQ     LEEB9
EEB1  81 90          717                CMPA    #$90
EEB3  26 03          718                BNE     LEEB8
EEB5  7F 00 1D       719                FCB     $7F,$00,$1D ; CLR $001D
EEB8  39             720  LEEB8         RTS
EEB9  4D             721  LEEB9         TSTA
EEBA  2A 1C          722                BPL     LEED8
EEBC  5F             723                CLRB
EEBD  BD EC F0       724                JSR     LECF0
EEC0  84 7F          725                ANDA    #$7F
EEC2  85 60          726                BITA    #$60
EEC4  27 6B          727                BEQ     ESCDISP
EEC6  D6 41          728                LDAB    $41
EEC8  C4 C0          729                ANDB    #$C0
EECA  C1 40          730                CMPB    #$40
EECC  26 63          731                BNE     ESCDISP
EECE  36             732                PSHA
EECF  96 45          733                LDAA    ESCCHR
EED1  BD FC F5       734                JSR     LFCF5
EED4  32             735                PULA
EED5  7E FC F5       736                JMP     LFCF5
EED8  81 1B          737  LEED8         CMPA    #$1B
EEDA  26 06          738                BNE     LEEE2
EEDC  D6 AB          739                LDAB    $AB
EEDE  2A 02          740                BPL     LEEE2
EEE0  96 45          741                LDAA    ESCCHR
EEE2  D6 41          742  LEEE2         LDAB    $41
EEE4  C5 40          743                BITB    #$40
EEE6  27 20          744                BEQ     LEF08
EEE8  BD FC F5       745                JSR     LFCF5
EEEB  D6 41          746                LDAB    $41
EEED  2B 01          747                BMI     LEEF0
EEEF  39             748                RTS
EEF0  85 60          749  LEEF0         BITA    #$60
EEF2  26 14          750                BNE     LEF08
EEF4  81 0D          751                CMPA    #$0D
EEF6  27 08          752                BEQ     LEF00
EEF8  81 04          753                CMPA    #$04
EEFA  27 04          754                BEQ     LEF00
EEFC  81 03          755                CMPA    #$03
EEFE  26 08          756                BNE     LEF08
EF00  36             757  LEF00         PSHA
EF01  CE 00 07       758                LDX     #$0007
EF04  BD EC BA       759                JSR     WAKE
EF07  32             760                PULA
EF08  BD EF F6       761  LEF08         JSR     LEFF6
EF0B  DE 9D          762  LEF0B         LDX     CURTASK
EF0D  E6 85          763                LDAB    $85,X
EF0F  C5 01          764                BITB    #$01
EF11  26 01          765                BNE     LEF14
EF13  39             766                RTS
EF14  DF 9F          767  LEF14         STX     $9F
EF16  78 00 A0       768                FCB     $78,$00,$A0 ; ASL $00A0
```

```
EF19  DE 9F          769                 LDX     $9F
EF1B  EE 75          770                 LDX     $75,X
EF1D  DF AF          771                 STX     $AF
EF1F  BD EC E8       772                 JSR     GETRX
EF22  D6 41          773                 LDAB    $41
EF24  C1 C0          774                 CMPB    #$C0
EF26  26 03          775                 BNE     LEF2B
EF28  BD FC F5       776                 JSR     LFCF5
EF2B  DE AF          777  LEF2B          LDX     $AF
EF2D  AD 00          778                 JSR     $00,X
EF2F  20 DA          779                 BRA     LEF0B
EF31                 780  ; ESC dispatch: code - $10, doubled, indexes ESCTAB.
EF31  84 7F          781  ESCDISP         ANDA    #$7F
EF33  80 10          782                 SUBA    #$10
EF35  2B 0B          783                 BMI     LEF42
EF37  CE EA CA       784                 LDX     #ESCTAB
EF3A  48             785  LEF3A          ASLA
EF3B  BD EF AD       786                 JSR     XPLUSA
EF3E  EE 00          787                 LDX     $00,X
EF40  6E 00          788                 JMP     $00,X
EF42  8B 4F          789  LEF42          ADDA    #$4F
EF44  7F 00 AC       790                 FCB     $7F,$00,$AC ; CLR $00AC
EF47  7E F9 D9       791                 JMP     LF9D9
EF4A  96 57          792  LEF4A          LDAA    $57
EF4C  20 03          793                 BRA     LEF51
EF4E  96 57          794  LEF4E          LDAA    $57
EF50  43             795                 COMA
EF51  85 02          796  LEF51          BITA    #$02
EF53  27 04          797                 BEQ     LEF59
EF55  86 D3          798                 LDAA    #$D3
EF57  20 02          799                 BRA     LEF5B
EF59  86 B5          800  LEF59          LDAA    #$B5
EF5B  CE 00 01       801  LEF5B          LDX     #$0001
EF5E  7E EC B8       802                 JMP     LECB8
EF61  7F 00 AC       803  LEF61          FCB     $7F,$00,$AC ; CLR $00AC
EF64  B6 78 02       804                 LDAA    PIA1B
EF67  88 01          805                 EORA    #$01
EF69  B7 78 02       806                 STAA    PIA1B
EF6C  39             807  LEF6C          RTS
EF6D  7F 00 AC       808  SENDBRK         FCB     $7F,$00,$AC ; CLR $00AC
EF70  B6 7E 00       809                 LDAA    SWBNK1
EF73  84 01          810                 ANDA    #$01
EF75  26 F5          811                 BNE     LEF6C
EF77  96 BF          812                 LDAA    ACIACTL
EF79  8A 60          813                 ORAA    #$60
EF7B  B7 7A 00       814                 STAA    ACIA
EF7E  86 C8          815                 LDAA    #$C8
EF80  BD EC CC       816                 JSR     SLEEP
EF83  96 BF          817                 LDAA    ACIACTL
EF85  B7 7A 00       818                 STAA    ACIA
EF88  7E EC C1       819                 JMP     BLOCKME
EF8B  F6 78 00       820  LEF8B          LDAB    PIA1A
EF8E  37             821                 PSHB
EF8F  C6 09          822                 LDAB    #$09
EF91  F7 78 00       823                 STAB    PIA1A
EF94  F6 78 00       824                 LDAB    PIA1A
EF97  2A 07          825                 BPL     LEFA0
```

```
EF99  F6 78 02       826                 LDAB    PIA1B
EF9C  2A 02          827                 BPL     LEFA0
EF9E  8D 05          828                 BSR     LEFA5
EFA0  33             829  LEFA0          PULB
EFA1  F7 78 00       830                 STAB    PIA1A
EFA4  39             831                 RTS
EFA5  7F 00 AC       832  LEFA5          FCB     $7F,$00,$AC ; CLR $00AC
EFA8  C6 80          833                 LDAB    #$80
EFAA  D7 AE          834                 STAB    $AE
EFAC  39             835                 RTS
EFAD                 836  ; XPLUSA: X = X + A.
EFAD  DF 9F          837  XPLUSA         STX     $9F
EFAF  9B A0          838                 ADDA    $A0
EFB1  97 A0          839                 STAA    $A0
EFB3  96 9F          840                 LDAA    $9F
EFB5  89 00          841                 ADCA    #$00
EFB7  97 9F          842                 STAA    $9F
EFB9  DE 9F          843                 LDX     $9F
EFBB  39             844                 RTS
EFBC  D6 9E          845  LEFBC          LDAB    $9E
EFBE  C1 01          846                 CMPB    #$01
EFC0  26 03          847                 BNE     LEFC5
EFC2  7F 00 AC       848                 FCB     $7F,$00,$AC ; CLR $00AC
EFC5  39             849  LEFC5          RTS
EFC6  BD F0 DA       850  LEFC6          JSR     BELL
EFC9  7F 00 AC       851                 FCB     $7F,$00,$AC ; CLR $00AC
EFCC  D6 AB          852                 LDAB    $AB
EFCE  2A 0A          853                 BPL     LEFDA
EFD0  BD EC E8       854                 JSR     GETRX
EFD3  7F 00 AC       855                 FCB     $7F,$00,$AC ; CLR $00AC
EFD6  4D             856                 TSTA
EFD7  7E EF 31       857                 JMP     ESCDISP
EFDA  BD EC E8       858  LEFDA          JSR     GETRX
EFDD  7F 00 AC       859                 FCB     $7F,$00,$AC ; CLR $00AC
EFE0  4D             860                 TSTA
EFE1  2B E2          861                 BMI     LEFC5
EFE3  85 60          862                 BITA    #$60
EFE5  27 DE          863                 BEQ     LEFC5
EFE7  7E F9 D9       864                 JMP     LF9D9
EFEA  96 AB          865  LEFEA          LDAA    $AB
EFEC  85 40          866                 BITA    #$40
EFEE  26 03          867                 BNE     LEFF3
EFF0  7E F7 3B       868                 JMP     LF73B
EFF3  7E F7 67       869  LEFF3          JMP     LF767
EFF6  BD EC FE       870  LEFF6          JSR     LECFE
EFF9  C5 20          871                 BITB    #$20
EFFB  26 09          872                 BNE     LF006
EFFD  91 45          873                 CMPA    ESCCHR
EFFF  26 20          874                 BNE     LF021
F001  C6 20          875                 LDAB    #$20
F003  7E EC F0       876                 JMP     LECF0
F006  5F             877  LF006          CLRB
F007  BD EC F0       878                 JSR     LECF0
F00A  81 58          879                 CMPA    #$58
F00C  27 04          880                 BEQ     LF012
F00E  81 75          881                 CMPA    #$75
F010  26 03          882                 BNE     LF015
```

```
F012  7E F0 A6       883  LF012          JMP     LF0A6
F015  85 60          884  LF015          BITA    #$60
F017  27 21          885                 BEQ     LF03A
F019  F6 00 1A       886                 FCB     $F6,$00,$1A ; LDAB $001A
F01C  26 33          887                 BNE     LF051
F01E  7E EF 31       888                 JMP     ESCDISP
F021  85 60          889  LF021          BITA    #$60
F023  27 0A          890                 BEQ     LF02F
F025  81 7F          891                 CMPA    #$7F
F027  26 11          892                 BNE     LF03A
F029  F6 00 1A       893                 FCB     $F6,$00,$1A ; LDAB $001A
F02C  26 0C          894                 BNE     LF03A
F02E  39             895                 RTS
F02F  F6 00 1A       896  LF02F          FCB     $F6,$00,$1A ; LDAB $001A
F032  26 06          897                 BNE     LF03A
F034  CE EB AA       898                 LDX     #CTLTAB
F037  7E EF 3A       899                 JMP     LEF3A
F03A  7D 00 1E       900  LF03A          FCB     $7D,$00,$1E ; TST $001E
F03D  27 05          901                 BEQ     LF044
F03F  36             902                 PSHA
F040  BD F8 27       903                 JSR     LF827
F043  32             904                 PULA
F044  9A 19          905  LF044          ORAA    ATTRMSK
F046  DE 13          906                 LDX     CURADR
F048  B7 70 00       907                 STAA    HALT
F04B  01             908                 NOP
F04C  A7 00          909                 STAA    $00,X
F04E  7E F2 72       910                 JMP     LF272
F051  36             911  LF051          PSHA
F052  86 1B          912                 LDAA    #$1B
F054  8D E4          913                 BSR     LF03A
F056  32             914                 PULA
F057  20 E1          915                 BRA     LF03A
F059  00 00 00 00    916                 FCB     $00,$00,$00,$00,$00,$00
F05F  BF             917                 FCB     $BF
F060  86 80          918  LF060          LDAA    #$80
F062  97 18          919                 STAA    PROTECT
F064  96 54          920                 LDAA    $54
F066  85 10          921                 BITA    #$10
F068  27 03          922                 BEQ     LF06D
F06A  7E F2 59       923                 JMP     LF259
F06D  7E F2 79       924  LF06D          JMP     LF279
F070  7F 00 18       925  LF070          FCB     $7F,$00,$18 ; CLR $0018
F073  39             926                 RTS
F074  86 80          927  LF074          LDAA    #$80
F076  97 19          928                 STAA    ATTRMSK
F078  39             929                 RTS
F079  7F 00 19       930  LF079          FCB     $7F,$00,$19 ; CLR $0019
F07C  39             931                 RTS
F07D  96 41          932  LF07D          LDAA    $41
F07F  84 BF          933                 ANDA    #$BF
F081  20 04          934                 BRA     LF087
F083  96 41          935  LF083          LDAA    $41
F085  8A 40          936                 ORAA    #$40
F087  97 41          937  LF087          STAA    $41
F089  39             938  LF089          RTS
F08A  BD EC E8       939  LF08A          JSR     GETRX
```

```
F08D  81 46          940                 CMPA    #$46
F08F  26 06          941                 BNE     LF097
F091  96 41          942                 LDAA    $41
F093  84 7F          943                 ANDA    #$7F
F095  20 F0          944                 BRA     LF087
F097  81 48          945  LF097         CMPA    #$48
F099  26 EE          946                 BNE     LF089
F09B  96 41          947                 LDAA    $41
F09D  8A 80          948                 ORAA    #$80
F09F  20 E6          949                 BRA     LF087
F0A1  86 80          950  LF0A1         LDAA    #$80
F0A3  97 1A          951                 STAA    MONMODE
F0A5  39             952                 RTS
F0A6  7F 00 1A       953  LF0A6         FCB     $7F,$00,$1A ; CLR $001A
F0A9  39             954                 RTS
F0AA  86 80          955  LF0AA         LDAA    #$80
F0AC  97 1D          956                 STAA    KBLOCK
F0AE  39             957                 RTS
F0AF  7F 00 1D       958  LF0AF         FCB     $7F,$00,$1D ; CLR $001D
F0B2  39             959                 RTS
F0B3  7D 00 42       960  LF0B3         FCB     $7D,$00,$42 ; TST $0042
F0B6  26 14          961                 BNE     LF0CC
F0B8  7D 00 D2       962                 FCB     $7D,$00,$D2 ; TST $00D2
F0BB  26 0F          963                 BNE     LF0CC
F0BD  96 50          964                 LDAA    $50
F0BF  85 20          965                 BITA    #$20
F0C1  26 09          966                 BNE     LF0CC
F0C3  B6 78 02       967                 LDAA    PIA1B
F0C6  84 04          968                 ANDA    #$04
F0C8  8A 80          969                 ORAA    #$80
F0CA  97 42          970                 STAA    SPLIT
F0CC  39             971  LF0CC         RTS
F0CD  7F 00 42       972  LF0CD         FCB     $7F,$00,$42 ; CLR $0042
F0D0  39             973                 RTS
F0D1  86 02          974  LF0D1         LDAA    #$02
F0D3  97 1E          975                 STAA    $1E
F0D5  39             976                 RTS
F0D6  7F 00 1E       977  LF0D6         FCB     $7F,$00,$1E ; CLR $001E
F0D9  39             978                 RTS
F0DA                 979  ; BELL: pulse bit 3 of PIA1CRB.
F0DA  B6 78 03       980  BELL           LDAA    PIA1CRB
F0DD  84 F7          981                 ANDA    #$F7
F0DF  B7 78 03       982                 STAA    PIA1CRB
F0E2  8A 08          983                 ORAA    #$08
F0E4  B7 78 03       984                 STAA    PIA1CRB
F0E7  39             985                 RTS
F0E8  BD EC E8       986  LF0E8         JSR     GETRX
F0EB  BD F6 07       987                 JSR     HEX2NIB
F0EE  8A 90          988                 ORAA    #$90
F0F0  7E F0 3A       989                 JMP     LF03A
F0F3  D6 9E          990  LF0F3         LDAB    $9E
F0F5  C1 01          991                 CMPB    #$01
F0F7  26 04          992                 BNE     LF0FD
F0F9  D6 53          993                 LDAB    $53
F0FB  2B 4D          994                 BMI     LF14A
F0FD  BD EC E8       995  LF0FD         JSR     GETRX
F100  7F 00 64       996                 FCB     $7F,$00,$64 ; CLR $0064
```

```
F103  81 2E          997                CMPA    #$2E
F105  26 07          998                BNE     LF10E
F107  86 10          999                LDAA    #$10
F109  97 64         1000                STAA    $64
F10B  BD EC E8      1001                JSR     GETRX
F10E  BD F6 07      1002  LF10E         JSR     HEX2NIB
F111  25 38         1003                BCS     LF14B
F113  9B 64         1004                ADDA    $64
F115  97 64         1005                STAA    $64
F117  BD EC E8      1006                JSR     GETRX
F11A  BD F6 07      1007                JSR     HEX2NIB
F11D  48            1008                ASLA
F11E  48            1009                ASLA
F11F  48            1010                ASLA
F120  48            1011                ASLA
F121  97 63         1012                STAA    $63
F123  BD EC E8      1013                JSR     GETRX
F126  BD F6 07      1014                JSR     HEX2NIB
F129  25 20         1015                BCS     LF14B
F12B  CE 00 45      1016                LDX     #$0045
F12E  DF 9F         1017                STX     $9F
F130  D6 64         1018                LDAB    $64
F132  DB A0         1019                ADDB    $A0
F134  D7 A0         1020                STAB    $A0
F136  D6 9F         1021                LDAB    $9F
F138  C9 00         1022                ADCB    #$00
F13A  D7 9F         1023                STAB    $9F
F13C  DE 9F         1024                LDX     $9F
F13E  D6 63         1025                LDAB    $63
F140  1B            1026                ABA
F141  A7 00         1027                STAA    $00,X
F143  D6 00         1028  LF143         LDAB    PLUGFLG
F145  27 03         1029                BEQ     LF14A
F147  BD E0 8A      1030                JSR     $E08A
F14A  39            1031  LF14A         RTS
F14B  7E F0 DA      1032  LF14B         JMP     BELL
F14E  BD EC E8      1033  LF14E         JSR     GETRX
F151  36            1034                PSHA
F152  BD F7 67      1035                JSR     LF767
F155  32            1036                PULA
F156  81 21         1037                CMPA    #$21
F158  26 03         1038                BNE     LF15D
F15A  7E E8 00      1039                JMP     RESET
F15D  81 30         1040  LF15D         CMPA    #$30
F15F  2D 61         1041                BLT     LF1C2
F161  26 02         1042                BNE     LF165
F163  8D 0D         1043                BSR     LF172
F165  81 39         1044  LF165         CMPA    #$39
F167  26 02         1045                BNE     LF16B
F169  8D 2E         1046                BSR     LF199
F16B  81 38         1047  LF16B         CMPA    #$38
F16D  26 02         1048                BNE     LF171
F16F  8D 3B         1049                BSR     LF1AC
F171  39            1050  LF171         RTS
F172  DE 43         1051  LF172         LDX     $43
F174  C6 10         1052                LDAB    #$10
F176  8D 58         1053                BSR     LF1D0
```

```
F178  DF B7         1054                STX     $B7
F17A  8D 51         1055                BSR     LF1CD
F17C  CE 00 45      1056                LDX     #$0045
F17F  C6 10         1057                LDAB    #$10
F181  8D 4D         1058                BSR     LF1D0
F183  8D 48         1059                BSR     LF1CD
F185  8D 46         1060                BSR     LF1CD
F187  DE B7         1061                LDX     $B7
F189  C6 0E         1062                LDAB    #$0E
F18B  8D 43         1063                BSR     LF1D0
F18D  8D 3E         1064                BSR     LF1CD
F18F  CE 00 55      1065                LDX     #$0055
F192  C6 0E         1066                LDAB    #$0E
F194  8D 3A         1067                BSR     LF1D0
F196  8D 35         1068                BSR     LF1CD
F198  39            1069                RTS
F199  CE EA C6      1070  LF199           LDX     #LEAC6
F19C  C6 02         1071                LDAB    #$02
F19E  8D 30         1072                BSR     LF1D0
F1A0  8D 2B         1073                BSR     LF1CD
F1A2  CE EA 18      1074                LDX     #LEA18
F1A5  C6 02         1075                LDAB    #$02
F1A7  8D 27         1076                BSR     LF1D0
F1A9  8D 22         1077                BSR     LF1CD
F1AB  39            1078                RTS
F1AC  B6 7E 00      1079  LF1AC           LDAA    SWBNK1
F1AF  8D 3F         1080                BSR     LF1F0
F1B1  8D 1A         1081                BSR     LF1CD
F1B3  B6 79 00      1082                LDAA    SWBNK2
F1B6  8D 38         1083                BSR     LF1F0
F1B8  8D 13         1084                BSR     LF1CD
F1BA  B6 7D 00      1085                LDAA    SWBNK3
F1BD  8D 31         1086                BSR     LF1F0
F1BF  8D 0C         1087                BSR     LF1CD
F1C1  39            1088                RTS
F1C2  8D AE         1089  LF1C2           BSR     LF172
F1C4  8D 07         1090                BSR     LF1CD
F1C6  8D D1         1091                BSR     LF199
F1C8  8D 03         1092                BSR     LF1CD
F1CA  8D E0         1093                BSR     LF1AC
F1CC  39            1094                RTS
F1CD  7E F3 13      1095  LF1CD           JMP     LF313
F1D0  DF 9F         1096  LF1D0           STX     $9F
F1D2  37            1097                PSHB
F1D3  A6 00         1098                LDAA    $00,X
F1D5  36            1099                PSHA
F1D6  BD F6 22      1100                JSR     NIB2HEX
F1D9  BD F0 3A      1101                JSR     LF03A
F1DC  32            1102                PULA
F1DD  BD F6 26      1103                JSR     LF626
F1E0  BD F0 3A      1104                JSR     LF03A
F1E3  BD F2 F0      1105                JSR     LF2F0
F1E6  33            1106                PULB
F1E7  DE 9F         1107                LDX     $9F
F1E9  08            1108                INX
F1EA  DF 9F         1109                STX     $9F
F1EC  5A            1110                DECB
```

```
F1ED  26 E1         1111                BNE     LF1D0
F1EF  39            1112                RTS
F1F0  97 B7         1113  LF1F0         STAA    $B7
F1F2  C6 08         1114                LDAB    #$08
F1F4  D7 B8         1115                STAB    $B8
F1F6  BD F2 F0      1116  LF1F6         JSR     LF2F0
F1F9  74 00 B7      1117                FCB     $74,$00,$B7 ; LSR $00B7
F1FC  24 04         1118                BCC     LF202
F1FE  86 46         1119                LDAA    #$46
F200  20 02         1120                BRA     LF204
F202  86 4E         1121  LF202         LDAA    #$4E
F204  BD F0 3A      1122  LF204         JSR     LF03A
F207  7A 00 B8      1123                FCB     $7A,$00,$B8 ; DEC $00B8
F20A  26 EA         1124                BNE     LF1F6
F20C  39            1125                RTS
F20D  86 01         1126  LF20D         LDAA    #$01
F20F  20 06         1127                BRA     LF217
F211  86 80         1128  LF211         LDAA    #$80
F213  20 02         1129                BRA     LF217
F215  86 11         1130  LF215         LDAA    #$11
F217  F6 79 00      1131  LF217         LDAB    SWBNK2
F21A  C5 80         1132                BITB    #$80
F21C  26 0D         1133                BNE     LF22B
F21E  7E FD AF      1134                JMP     LFDAF
F221  F6 79 00      1135  LF221         LDAB    SWBNK2
F224  C5 80         1136                BITB    #$80
F226  26 03         1137                BNE     LF22B
F228  7E FF 40      1138                JMP     LFF40
F22B  39            1139  LF22B         RTS
F22C                1140  ; ESC ` : if socket A holds "TJ" jump to its start
F22C                1141  ; ($E03C).
F22C  FE E0 00      1142  ESCGRAVE       LDX     $E000
F22F  8C 54 4A      1143                CPX     #$544A
F232  26 F7         1144                BNE     LF22B
F234  7E E0 3C      1145                JMP     $E03C
F237  86 02         1146  LF237         LDAA    #$02
F239  7E F0 3A      1147  LF239         JMP     LF03A
F23C  86 03         1148  LF23C         LDAA    #$03
F23E  20 F9         1149                BRA     LF239
F240  BD EC E8      1150  LF240         JSR     GETRX
F243  80 20         1151                SUBA    #$20
F245  97 63         1152                STAA    $63
F247  BD EC E8      1153                JSR     GETRX
F24A  91 45         1154                CMPA    ESCCHR
F24C  27 0A         1155                BEQ     LF258
F24E  36            1156  LF24E         PSHA
F24F  BD F0 21      1157                JSR     LF021
F252  32            1158                PULA
F253  7A 00 63      1159                FCB     $7A,$00,$63 ; DEC $0063
F256  2E F6         1160                BGT     LF24E
F258  39            1161  LF258         RTS
F259  96 42         1162  LF259         LDAA    SPLIT
F25B  2A 0D         1163                BPL     LF26A
F25D  84 04         1164                ANDA    #$04
F25F  F6 78 02      1165                LDAB    PIA1B
F262  C4 04         1166                ANDB    #$04
F264  11            1167                CBA
```

```
F265  27 03        1168                BEQ     LF26A
F267  BD F6 5E     1169                JSR     LF65E
F26A  BD F2 CF     1170  LF26A         JSR     LF2CF
F26D  8D 51        1171                BSR     LF2C0
F26F  7E F3 FA     1172  LF26F         JMP     LF3FA
F272  BD F2 F0     1173  LF272         JSR     LF2F0
F275  8D 49        1174  LF275         BSR     LF2C0
F277  20 F6        1175                BRA     LF26F
F279  DE 13        1176  LF279         LDX     CURADR
F27B  20 F8        1177                BRA     LF275
F27D  D6 18        1178  LF27D         LDAB    PROTECT
F27F  27 04        1179                BEQ     LF285
F281  D6 54        1180                LDAB    $54
F283  2B 15        1181                BMI     LF29A
F285  BD F3 17     1182  LF285         JSR     LF317
F288  8D 36        1183                BSR     LF2C0
F28A  20 E3        1184                BRA     LF26F
F28C  BD F3 6F     1185  LF28C         JSR     LF36F
F28F  8D 2F        1186  LF28F         BSR     LF2C0
F291  20 F9        1187                BRA     LF28C
F293  BD F3 88     1188  LF293         JSR     LF388
F296  8D 28        1189                BSR     LF2C0
F298  20 F2        1190                BRA     LF28C
F29A  BD F3 13     1191  LF29A         JSR     LF313
F29D  8D 21        1192                BSR     LF2C0
F29F  20 CE        1193                BRA     LF26F
F2A1  D6 18        1194  LF2A1         LDAB    PROTECT
F2A3  DA B5        1195                ORAB    $B5
F2A5  26 09        1196                BNE     LF2B0
F2A7  D6 53        1197                LDAB    $53
F2A9  C5 40        1198                BITB    #$40
F2AB  27 03        1199                BEQ     LF2B0
F2AD  BD F8 20     1200                JSR     LF820
F2B0  BD F2 E5     1201  LF2B0         JSR     LF2E5
F2B3  D6 53        1202                LDAB    $53
F2B5  C5 20        1203                BITB    #$20
F2B7  27 03        1204                BEQ     LF2BC
F2B9  BD F3 17     1205                JSR     LF317
F2BC  8D 02        1206  LF2BC         BSR     LF2C0
F2BE  20 AF        1207                BRA     LF26F
F2C0  96 18        1208  LF2C0         LDAA    PROTECT
F2C2  2A 08        1209                BPL     LF2CC
F2C4  B7 70 00     1210                STAA    HALT
F2C7  01           1211                NOP
F2C8  A6 00        1212                LDAA    $00,X
F2CA  2B 02        1213                BMI     LF2CE
F2CC  31           1214  LF2CC         INS
F2CD  31           1215                INS
F2CE  39           1216  LF2CE         RTS
F2CF  5F           1217  LF2CF         CLRB
F2D0  F7 7F 0C     1218                STAB    VTACC
F2D3  D7 17        1219                STAB    COL
F2D5  96 15        1220                LDAA    FIRSTLN
F2D7  80 17        1221  LF2D7         SUBA    #$17
F2D9  2C 02        1222                BGE     LF2DD
F2DB  8B 18        1223                ADDA    #$18
F2DD  B7 7F 0D     1224  LF2DD         STAA    VTACD
```

```
F2E0  97 16         1225                STAA    ROW
F2E2  7E F5 F8      1226                JMP     CURADDR
F2E5  5F            1227  LF2E5         CLRB
F2E6  F7 7F 0C      1228                STAB    VTACC
F2E9  D7 17         1229                STAB    COL
F2EB  D7 B5         1230                STAB    $B5
F2ED  7E F5 F8      1231                JMP     CURADDR
F2F0  96 17         1232  LF2F0         LDAA    COL
F2F2  81 4F         1233                CMPA    #$4F
F2F4  27 0F         1234                BEQ     LF305
F2F6  7F 00 B5      1235                FCB     $7F,$00,$B5 ; CLR $00B5
F2F9  4C            1236                INCA
F2FA  B7 7F 0C      1237                STAA    VTACC
F2FD  97 17         1238                STAA    COL
F2FF  DE 13         1239                LDX     CURADR
F301  08            1240                INX
F302  DF 13         1241                STX     CURADR
F304  39            1242                RTS
F305  D6 18         1243  LF305         LDAB    PROTECT
F307  2B 0A         1244                BMI     LF313
F309  D6 53         1245                LDAB    $53
F30B  C5 10         1246                BITB    #$10
F30D  27 04         1247                BEQ     LF313
F30F  7C 00 B5      1248                FCB     $7C,$00,$B5 ; INC $00B5
F312  39            1249                RTS
F313  8D D0         1250  LF313         BSR     LF2E5
F315  20 00         1251                BRA     LF317
F317  96 16         1252  LF317         LDAA    ROW
F319  91 15         1253                CMPA    FIRSTLN
F31B  27 0A         1254                BEQ     LF327
F31D  81 17         1255                CMPA    #$17
F31F  26 02         1256                BNE     LF323
F321  86 FF         1257                LDAA    #$FF
F323  4C            1258  LF323         INCA
F324  7E F2 DD      1259  LF324         JMP     LF2DD
F327  7D 00 42      1260  LF327         FCB     $7D,$00,$42 ; TST $0042
F32A  26 36         1261                BNE     LF362
F32C  7D 00 18      1262                FCB     $7D,$00,$18 ; TST $0018
F32F  27 03         1263                BEQ     LF334
F331  7E F2 D7      1264  LF331         JMP     LF2D7
F334  D6 55         1265  LF334         LDAB    $55
F336  C5 08         1266                BITB    #$08
F338  26 F7         1267                BNE     LF331
F33A  4C            1268                INCA
F33B  81 18         1269                CMPA    #$18
F33D  2D 01         1270                BLT     LF340
F33F  4F            1271                CLRA
F340  97 15         1272  LF340         STAA    FIRSTLN
F342  B7 7F 06      1273                STAA    VTAC6
F345  BD F2 DD      1274                JSR     LF2DD
F348  90 17         1275                SUBA    COL
F34A  C2 00         1276                SBCB    #$00
F34C  D7 B7         1277                STAB    $B7
F34E  97 B8         1278                STAA    $B8
F350  DE B7         1279                LDX     $B7
F352  C6 50         1280                LDAB    #$50
F354  4F            1281                CLRA
```

```
F355  B7 70 00      1282  LF355           STAA    HALT
F358  01            1283                  NOP
F359  A7 00         1284                  STAA    $00,X
F35B  08            1285                  INX
F35C  5A            1286                  DECB
F35D  26 F6         1287                  BNE     LF355
F35F  DE 13         1288                  LDX     CURADR
F361  39            1289                  RTS
F362  BD F6 5E      1290  LF362           JSR     LF65E
F365  96 15         1291                  LDAA    FIRSTLN
F367  80 17         1292                  SUBA    #$17
F369  2A 02         1293                  BPL     LF36D
F36B  8B 18         1294                  ADDA    #$18
F36D  20 B5         1295  LF36D           BRA     LF324
F36F  96 17         1296  LF36F           LDAA    COL
F371  27 0C         1297                  BEQ     LF37F
F373  4A            1298                  DECA
F374  B7 7F 0C      1299                  STAA    VTACC
F377  97 17         1300                  STAA    COL
F379  DE 13         1301                  LDX     CURADR
F37B  09            1302                  DEX
F37C  DF 13         1303                  STX     CURADR
F37E  39            1304                  RTS
F37F  86 4F         1305  LF37F           LDAA    #$4F
F381  B7 7F 0C      1306                  STAA    VTACC
F384  97 17         1307                  STAA    COL
F386  20 00         1308                  BRA     LF388
F388  96 16         1309  LF388           LDAA    ROW
F38A  26 02         1310                  BNE     LF38E
F38C  86 18         1311                  LDAA    #$18
F38E  4A            1312  LF38E           DECA
F38F  B7 7F 0D      1313                  STAA    VTACD
F392  97 16         1314                  STAA    ROW
F394  91 15         1315                  CMPA    FIRSTLN
F396  26 0F         1316                  BNE     LF3A7
F398  7D 00 42      1317                  FCB     $7D,$00,$42 ; TST $0042
F39B  27 0A         1318                  BEQ     LF3A7
F39D  BD F6 5E      1319                  JSR     LF65E
F3A0  96 15         1320                  LDAA    FIRSTLN
F3A2  B7 7F 0D      1321                  STAA    VTACD
F3A5  97 16         1322                  STAA    ROW
F3A7  7E F5 F8      1323  LF3A7           JMP     CURADDR
F3AA  7D 00 18      1324  LF3AA           FCB     $7D,$00,$18 ; TST $0018
F3AD  27 5B         1325                  BEQ     LF40A
F3AF  BD F4 9B      1326                  JSR     LF49B
F3B2  96 17         1327                  LDAA    COL
F3B4  DE 13         1328                  LDX     CURADR
F3B6  DF B7         1329                  STX     $B7
F3B8  20 04         1330                  BRA     LF3BE
F3BA  9C BB         1331  LF3BA           CPX     $BB
F3BC  27 26         1332                  BEQ     LF3E4
F3BE  81 4F         1333  LF3BE           CMPA    #$4F
F3C0  27 2A         1334                  BEQ     LF3EC
F3C2  08            1335                  INX
F3C3  4C            1336                  INCA
F3C4  9C B7         1337  LF3C4           CPX     $B7
F3C6  27 1C         1338                  BEQ     LF3E4
```

```
F3C8  B7 70 00      1339                STAA    HALT
F3CB  01            1340                NOP
F3CC  E6 00         1341                LDAB    $00,X
F3CE  2A EA         1342                BPL     LF3BA
F3D0  81 4F         1343  LF3D0         CMPA    #$4F
F3D2  27 1F         1344                BEQ     LF3F3
F3D4  08            1345                INX
F3D5  4C            1346                INCA
F3D6  B7 70 00      1347  LF3D6         STAA    HALT
F3D9  01            1348                NOP
F3DA  E6 00         1349                LDAB    $00,X
F3DC  2A 06         1350                BPL     LF3E4
F3DE  9C B7         1351                CPX     $B7
F3E0  27 1E         1352                BEQ     LF400
F3E2  20 EC         1353                BRA     LF3D0
F3E4  B7 7F 0C      1354  LF3E4         STAA    VTACC
F3E7  97 17         1355                STAA    COL
F3E9  DF 13         1356                STX     CURADR
F3EB  39            1357                RTS
F3EC  BD F3 13      1358  LF3EC         JSR     LF313
F3EF  96 17         1359                LDAA    COL
F3F1  20 D1         1360                BRA     LF3C4
F3F3  BD F3 13      1361  LF3F3         JSR     LF313
F3F6  96 17         1362                LDAA    COL
F3F8  20 DC         1363                BRA     LF3D6
F3FA  96 17         1364  LF3FA         LDAA    COL
F3FC  DF B7         1365                STX     $B7
F3FE  20 D0         1366                BRA     LF3D0
F400  BD F2 CF      1367  LF400         JSR     LF2CF
F403  BD F3 6F      1368                JSR     LF36F
F406  4F            1369                CLRA
F407  A7 00         1370                STAA    $00,X
F409  39            1371                RTS
F40A  7D 00 1F      1372  LF40A         FCB     $7D,$00,$1F ; TST $001F
F40D  27 2D         1373                BEQ     LF43C
F40F  2B 2C         1374                BMI     LF43D
F411  4F            1375                CLRA
F412  D6 17         1376                LDAB    COL
F414  9B 1F         1377  LF414         ADDA    $1F
F416  81 4F         1378                CMPA    #$4F
F418  22 05         1379                BHI     LF41F
F41A  11            1380                CBA
F41B  2E 05         1381                BGT     LF422
F41D  20 F5         1382                BRA     LF414
F41F  7E F3 13      1383  LF41F         JMP     LF313
F422  36            1384  LF422         PSHA
F423  D6 13         1385                LDAB    CURADR
F425  96 14         1386                LDAA    $14
F427  90 17         1387                SUBA    COL
F429  C2 00         1388                SBCB    #$00
F42B  30            1389                TSX
F42C  AB 00         1390                ADDA    $00,X
F42E  C9 00         1391                ADCB    #$00
F430  D7 13         1392                STAB    CURADR
F432  97 14         1393                STAA    $14
F434  32            1394                PULA
F435  B7 7F 0C      1395                STAA    VTACC
```

```
F438  97 17         1396                STAA    COL
F43A  DE 13         1397                LDX     CURADR
F43C  39            1398  LF43C         RTS
F43D  96 17         1399  LF43D         LDAA    COL
F43F  81 4F         1400                CMPA    #$4F
F441  27 12         1401                BEQ     LF455
F443  7C 00 17      1402                FCB     $7C,$00,$17 ; INC $0017
F446  BD F5 3B      1403                JSR     LF53B
F449  E5 00         1404                BITB    $00,X
F44B  27 F0         1405                BEQ     LF43D
F44D  96 17         1406                LDAA    COL
F44F  B7 7F 0C      1407                STAA    VTACC
F452  7E F5 F8      1408                JMP     CURADDR
F455  7E F3 13      1409  LF455         JMP     LF313
F458  7D 00 18      1410  LF458         FCB     $7D,$00,$18 ; TST $0018
F45B  27 7A         1411                BEQ     LF4D7
F45D  8D 3C         1412                BSR     LF49B
F45F  96 17         1413                LDAA    COL
F461  DE 13         1414                LDX     CURADR
F463  DF B7         1415                STX     $B7
F465  8D 1F         1416                BSR     LF486
F467  2B 13         1417                BMI     LF47C
F469  9C B7         1418  LF469         CPX     $B7
F46B  27 16         1419                BEQ     LF483
F46D  9C BB         1420                CPX     $BB
F46F  27 12         1421                BEQ     LF483
F471  8D 13         1422                BSR     LF486
F473  2A F4         1423                BPL     LF469
F475  DF 13         1424                STX     CURADR
F477  97 17         1425                STAA    COL
F479  7E F2 F0      1426                JMP     LF2F0
F47C  BD F4 86      1427  LF47C         JSR     LF486
F47F  2B FB         1428                BMI     LF47C
F481  20 E6         1429                BRA     LF469
F483  7E F3 E4      1430  LF483         JMP     LF3E4
F486  4D            1431  LF486         TSTA
F487  27 09         1432                BEQ     LF492
F489  4A            1433                DECA
F48A  09            1434                DEX
F48B  B7 70 00      1435  LF48B         STAA    HALT
F48E  01            1436                NOP
F48F  E6 00         1437                LDAB    $00,X
F491  39            1438                RTS
F492  DF 13         1439  LF492         STX     CURADR
F494  BD F3 7F      1440                JSR     LF37F
F497  96 17         1441                LDAA    COL
F499  20 F0         1442                BRA     LF48B
F49B  96 15         1443  LF49B         LDAA    FIRSTLN
F49D  97 BC         1444                STAA    $BC
F49F  4F            1445                CLRA
F4A0  D6 42         1446                LDAB    SPLIT
F4A2  27 1D         1447                BEQ     LF4C1
F4A4  C4 04         1448                ANDB    #$04
F4A6  F8 78 02      1449                EORB    PIA1B
F4A9  C5 04         1450                BITB    #$04
F4AB  27 14         1451                BEQ     LF4C1
F4AD  86 08         1452                LDAA    #$08
```

```
F4AF  98 11         1453                EORA    CRTBASE
F4B1  8B 07         1454                ADDA    #$07
F4B3  97 B9         1455                STAA    $B9
F4B5  C6 D4         1456                LDAB    #$D4
F4B7  D7 BA         1457                STAB    $BA
F4B9  DE B9         1458                LDX     $B9
F4BB  A6 00         1459                LDAA    $00,X
F4BD  97 BC         1460                STAA    $BC
F4BF  86 08         1461                LDAA    #$08
F4C1  98 11         1462  LF4C1          EORA    CRTBASE
F4C3  97 BB         1463                STAA    $BB
F4C5  96 BC         1464                LDAA    $BC
F4C7  80 17         1465                SUBA    #$17
F4C9  2C 02         1466                BGE     LF4CD
F4CB  8B 18         1467                ADDA    #$18
F4CD  BD F5 EA      1468  LF4CD          JSR     ROWADDR
F4D0  DB BB         1469                ADDB    $BB
F4D2  D7 BB         1470                STAB    $BB
F4D4  97 BC         1471                STAA    $BC
F4D6  39            1472                RTS
F4D7  7D 00 1F      1473  LF4D7          FCB     $7D,$00,$1F ; TST $001F
F4DA  27 24         1474                BEQ     LF500
F4DC  2B 14         1475                BMI     LF4F2
F4DE  D6 17         1476                LDAB    COL
F4E0  26 05         1477                BNE     LF4E7
F4E2  BD F3 6F      1478                JSR     LF36F
F4E5  D6 17         1479                LDAB    COL
F4E7  4F            1480  LF4E7          CLRA
F4E8  9B 1F         1481  LF4E8          ADDA    $1F
F4EA  11            1482                CBA
F4EB  2B FB         1483                BMI     LF4E8
F4ED  90 1F         1484                SUBA    $1F
F4EF  7E F4 22      1485                JMP     LF422
F4F2  BD F3 6F      1486  LF4F2          JSR     LF36F
F4F5  96 17         1487                LDAA    COL
F4F7  27 07         1488                BEQ     LF500
F4F9  BD F5 3B      1489                JSR     LF53B
F4FC  E5 00         1490                BITB    $00,X
F4FE  27 F2         1491                BEQ     LF4F2
F500  39            1492  LF500          RTS
F501  CE 00 20      1493  LF501          LDX     #$0020
F504  C6 0A         1494                LDAB    #$0A
F506  4F            1495                CLRA
F507  A7 00         1496  LF507          STAA    $00,X
F509  08            1497                INX
F50A  5A            1498                DECB
F50B  26 FA         1499                BNE     LF507
F50D  20 21         1500                BRA     LF530
F50F  8D 2A         1501  LF50F          BSR     LF53B
F511  EA 00         1502                ORAB    $00,X
F513  E7 00         1503                STAB    $00,X
F515  D6 1F         1504                LDAB    $1F
F517  CA 80         1505                ORAB    #$80
F519  20 1D         1506                BRA     LF538
F51B  8D 1E         1507  LF51B          BSR     LF53B
F51D  53            1508                COMB
F51E  E4 00         1509                ANDB    $00,X
```

```
F520  E7 00         1510                STAB    $00,X
F522  CE 00 20      1511                LDX     #$0020
F525  C6 0A         1512                LDAB    #$0A
F527  4F            1513                CLRA
F528  A1 00         1514  LF528         CMPA    $00,X
F52A  26 0E         1515                BNE     LF53A
F52C  08            1516                INX
F52D  5A            1517                DECB
F52E  26 F8         1518                BNE     LF528
F530  D6 1F         1519  LF530         LDAB    $1F
F532  C1 D0         1520                CMPB    #$D0
F534  27 04         1521                BEQ     LF53A
F536  C4 7F         1522                ANDB    #$7F
F538  D7 1F         1523  LF538         STAB    $1F
F53A  39            1524  LF53A         RTS
F53B  CE 00 20      1525  LF53B         LDX     #$0020
F53E  96 17         1526                LDAA    COL
F540  44            1527                LSRA
F541  44            1528                LSRA
F542  44            1529                LSRA
F543  BD EF AD      1530                JSR     XPLUSA
F546  C6 01         1531                LDAB    #$01
F548  96 17         1532                LDAA    COL
F54A  84 07         1533                ANDA    #$07
F54C  27 04         1534                BEQ     LF552
F54E  58            1535  LF54E         ASLB
F54F  4A            1536                DECA
F550  26 FC         1537                BNE     LF54E
F552  39            1538  LF552         RTS
F553  BD EC E8      1539  LF553         JSR     GETRX
F556  84 7F         1540  LF556         ANDA    #$7F
F558  80 20         1541                SUBA    #$20
F55A  2B 0A         1542                BMI     LF566
F55C  81 50         1543                CMPA    #$50
F55E  2D 04         1544                BLT     LF564
F560  2E 04         1545                BGT     LF566
F562  86 D0         1546                LDAA    #$D0
F564  97 1F         1547  LF564         STAA    $1F
F566  39            1548  LF566         RTS
F567  BD EC E8      1549  LF567         JSR     GETRX
F56A  D6 54         1550                LDAB    $54
F56C  C5 40         1551                BITB    #$40
F56E  26 01         1552                BNE     LF571
F570  4A            1553                DECA
F571  81 30         1554  LF571         CMPA    #$30
F573  2C 02         1555                BGE     LF577
F575  86 30         1556                LDAA    #$30
F577  81 31         1557  LF577         CMPA    #$31
F579  2F 02         1558                BLE     LF57D
F57B  86 31         1559                LDAA    #$31
F57D  84 01         1560  LF57D         ANDA    #$01
F57F  D6 42         1561                LDAB    SPLIT
F581  26 0F         1562                BNE     LF592
F583  48            1563                ASLA
F584  48            1564                ASLA
F585  F6 78 02      1565                LDAB    PIA1B
F588  C4 04         1566                ANDB    #$04
```

```
F58A  11           1567                  CBA
F58B  27 1D        1568                  BEQ     CURPOS
F58D  BD F6 31     1569                  JSR     LF631
F590  20 18        1570                  BRA     CURPOS
F592  36           1571  LF592           PSHA
F593  C4 04        1572                  ANDB    #$04
F595  B6 78 02     1573                  LDAA    PIA1B
F598  84 04        1574                  ANDA    #$04
F59A  11           1575                  CBA
F59B  26 06        1576                  BNE     LF5A3
F59D  33           1577                  PULB
F59E  5D           1578                  TSTB
F59F  27 09        1579                  BEQ     CURPOS
F5A1  20 04        1580                  BRA     LF5A7
F5A3  33           1581  LF5A3           PULB
F5A4  5D           1582                  TSTB
F5A5  26 03        1583                  BNE     CURPOS
F5A7  BD F6 31     1584  LF5A7           JSR     LF631
F5AA               1585  ; ESC = row col : cursor address.
F5AA  BD EC E8     1586  CURPOS          JSR     GETRX
F5AD  8D 33        1587                  BSR     LF5E2
F5AF  81 18        1588                  CMPA    #$18
F5B1  2D 02        1589                  BLT     LF5B5
F5B3  86 17        1590                  LDAA    #$17
F5B5  9B 15        1591  LF5B5           ADDA    FIRSTLN
F5B7  4C           1592                  INCA
F5B8  81 18        1593                  CMPA    #$18
F5BA  2D 02        1594                  BLT     LF5BE
F5BC  80 18        1595                  SUBA    #$18
F5BE  B7 7F 0D     1596  LF5BE           STAA    VTACD
F5C1  97 16        1597                  STAA    ROW
F5C3  BD EC E8     1598                  JSR     GETRX
F5C6  8D 1A        1599                  BSR     LF5E2
F5C8  81 50        1600                  CMPA    #$50
F5CA  2D 02        1601                  BLT     LF5CE
F5CC  86 4F        1602                  LDAA    #$4F
F5CE  B7 7F 0C     1603  LF5CE           STAA    VTACC
F5D1  97 17        1604                  STAA    COL
F5D3  BD F5 F8     1605                  JSR     CURADDR
F5D6  96 53        1606                  LDAA    $53
F5D8  85 02        1607                  BITA    #$02
F5DA  27 0D        1608                  BEQ     LF5E9
F5DC  BD F2 C0     1609                  JSR     LF2C0
F5DF  7E F3 FA     1610                  JMP     LF3FA
F5E2  84 7F        1611  LF5E2           ANDA    #$7F
F5E4  80 20        1612                  SUBA    #$20
F5E6  2C 01        1613                  BGE     LF5E9
F5E8  4F           1614                  CLRA
F5E9  39           1615  LF5E9           RTS
F5EA               1616  ; Row to screen address (row * 80 by shifts).
F5EA  16           1617  ROWADDR         TAB
F5EB  48           1618                  ASLA
F5EC  48           1619                  ASLA
F5ED  1B           1620                  ABA
F5EE  5F           1621                  CLRB
F5EF  48           1622                  ASLA
F5F0  59           1623                  ROLB
```

```
F5F1  48           1624                ASLA
F5F2  59           1625                ROLB
F5F3  48           1626                ASLA
F5F4  59           1627                ROLB
F5F5  48           1628                ASLA
F5F6  59           1629                ROLB
F5F7  39           1630                RTS
F5F8               1631  ; Cursor address from ROW and COL.
F5F8  96 16        1632  CURADDR       LDAA    ROW
F5FA  8D EE        1633                BSR     ROWADDR
F5FC  9B 17        1634                ADDA    COL
F5FE  D9 11        1635                ADCB    CRTBASE
F600  D7 13        1636                STAB    CURADR
F602  97 14        1637                STAA    $14
F604  DE 13        1638                LDX     CURADR
F606  39           1639                RTS
F607               1640  ; HEX2NIB: ASCII hex digit to 4-bit value.
F607  81 30        1641  HEX2NIB       CMPA    #$30
F609  2B 14        1642                BMI     LF61F
F60B  81 39        1643                CMPA    #$39
F60D  2F 0C        1644                BLE     LF61B
F60F  84 DF        1645                ANDA    #$DF
F611  81 41        1646                CMPA    #$41
F613  2B 0A        1647                BMI     LF61F
F615  81 46        1648                CMPA    #$46
F617  2E 06        1649                BGT     LF61F
F619  80 07        1650                SUBA    #$07
F61B  84 0F        1651  LF61B         ANDA    #$0F
F61D  0C           1652                CLC
F61E  39           1653                RTS
F61F  4F           1654  LF61F         CLRA
F620  0D           1655                SEC
F621  39           1656                RTS
F622               1657  ; NIB2HEX: 4-bit value to ASCII hex digit.
F622  44           1658  NIB2HEX       LSRA
F623  44           1659                LSRA
F624  44           1660                LSRA
F625  44           1661                LSRA
F626  84 0F        1662  LF626         ANDA    #$0F
F628  81 0A        1663                CMPA    #$0A
F62A  2D 02        1664                BLT     LF62E
F62C  8B 07        1665                ADDA    #$07
F62E  8B 30        1666  LF62E         ADDA    #$30
F630  39           1667                RTS
F631  7D 00 D2     1668  LF631         FCB     $7D,$00,$D2 ; TST $00D2
F634  26 3B        1669                BNE     LF671
F636  96 50        1670                LDAA    $50
F638  85 20        1671                BITA    #$20
F63A  26 35        1672                BNE     LF671
F63C  C6 07        1673                LDAB    #$07
F63E  8D 20        1674                BSR     LF660
F640  B7 70 00     1675                STAA    HALT
F643  01           1676                NOP
F644  A6 00        1677                LDAA    $00,X
F646  95 18        1678                BITA    PROTECT
F648  27 27        1679                BEQ     LF671
F64A  96 42        1680                LDAA    SPLIT
```

```
F64C  27 0D         1681                 BEQ     LF65B
F64E  84 04         1682                 ANDA    #$04
F650  F6 78 02      1683                 LDAB    PIA1B
F653  C4 04         1684                 ANDB    #$04
F655  11            1685                 CBA
F656  27 03         1686                 BEQ     LF65B
F658  7E F2 8C      1687                 JMP     LF28C
F65B  7E F3 FA      1688  LF65B          JMP     LF3FA
F65E  C6 06         1689  LF65E          LDAB    #$06
F660  D7 B6         1690  LF660          STAB    $B6
F662  BD F6 72      1691                 JSR     LF672
F665  B6 78 02      1692                 LDAA    PIA1B
F668  88 04         1693                 EORA    #$04
F66A  B7 78 02      1694                 STAA    PIA1B
F66D  8D 16         1695                 BSR     LF685
F66F  DE 13         1696                 LDX     CURADR
F671  39            1697  LF671          RTS
F672  96 11         1698  LF672          LDAA    CRTBASE
F674  8B 07         1699                 ADDA    #$07
F676  97 B1         1700                 STAA    $B1
F678  86 D0         1701                 LDAA    #$D0
F67A  97 B2         1702                 STAA    $B2
F67C  CE 00 11      1703                 LDX     #$0011
F67F  DF B3         1704                 STX     $B3
F681  C6 30         1705                 LDAB    #$30
F683  20 36         1706                 BRA     BLITCPY
F685  CE 00 11      1707  LF685          LDX     #$0011
F688  DF B1         1708                 STX     $B1
F68A  B6 78 02      1709                 LDAA    PIA1B
F68D  84 04         1710                 ANDA    #$04
F68F  48            1711                 ASLA
F690  8B 80         1712                 ADDA    #$80
F692  97 11         1713                 STAA    CRTBASE
F694  8B 07         1714                 ADDA    #$07
F696  97 B3         1715                 STAA    $B3
F698  86 D0         1716                 LDAA    #$D0
F69A  97 B4         1717                 STAA    $B4
F69C  7D 00 42      1718                 FCB     $7D,$00,$42 ; TST $0042
F69F  27 04         1719                 BEQ     LF6A5
F6A1  D6 B6         1720                 LDAB    $B6
F6A3  20 02         1721                 BRA     LF6A7
F6A5  C6 30         1722  LF6A5          LDAB    #$30
F6A7  8D 12         1723  LF6A7          BSR     BLITCPY
F6A9  96 15         1724                 LDAA    FIRSTLN
F6AB  B7 7F 06      1725                 STAA    VTAC6
F6AE  96 16         1726                 LDAA    ROW
F6B0  B7 7F 0D      1727                 STAA    VTACD
F6B3  96 17         1728                 LDAA    COL
F6B5  B7 7F 0C      1729                 STAA    VTACC
F6B8  7E F5 F8      1730                 JMP     CURADDR
F6BB                1731  ; BLITCPY: copy B bytes from ($B3) to ($B1) with
F6BB                1732  ; HALT strobe.
F6BB  DE B3         1733  BLITCPY         LDX     $B3
F6BD  B7 70 00      1734                 STAA    HALT
F6C0  01            1735                 NOP
F6C1  A6 00         1736                 LDAA    $00,X
F6C3  08            1737                 INX
```

```
F6C4  DF B3         1738                STX     $B3
F6C6  DE B1         1739                LDX     $B1
F6C8  B7 70 00      1740                STAA    HALT
F6CB  01            1741                NOP
F6CC  A7 00         1742                STAA    $00,X
F6CE  08            1743                INX
F6CF  DF B1         1744                STX     $B1
F6D1  5A            1745                DECB
F6D2  26 E7         1746                BNE     BLITCPY
F6D4  39            1747                RTS
F6D5  86 20         1748  LF6D5         LDAA    #$20
F6D7  D6 55         1749                LDAB    $55
F6D9  C5 80         1750                BITB    #$80
F6DB  27 05         1751                BEQ     LF6E2
F6DD  96 4D         1752                LDAA    BLANK
F6DF  20 01         1753                BRA     LF6E2
F6E1  4F            1754  LF6E1         CLRA
F6E2  D6 55         1755  LF6E2         LDAB    $55
F6E4  C5 02         1756                BITB    #$02
F6E6  27 03         1757                BEQ     LF6EB
F6E8  7F 00 19      1758                FCB     $7F,$00,$19 ; CLR $0019
F6EB  9A 19         1759  LF6EB         ORAA    ATTRMSK
F6ED  36            1760                PSHA
F6EE  BD EF BC      1761                JSR     LEFBC
F6F1  96 42         1762                LDAA    SPLIT
F6F3  2A 35         1763                BPL     LF72A
F6F5  84 04         1764                ANDA    #$04
F6F7  F6 78 02      1765                LDAB    PIA1B
F6FA  C4 04         1766                ANDB    #$04
F6FC  11            1767                CBA
F6FD  26 2B         1768                BNE     LF72A
F6FF  32            1769                PULA
F700  D6 13         1770                LDAB    CURADR
F702  37            1771                PSHB
F703  D6 14         1772                LDAB    $14
F705  37            1773                PSHB
F706  D6 16         1774                LDAB    ROW
F708  37            1775                PSHB
F709  D6 17         1776                LDAB    COL
F70B  37            1777                PSHB
F70C  36            1778                PSHA
F70D  BD F6 5E      1779                JSR     LF65E
F710  BD F7 90      1780                JSR     LF790
F713  BD F6 5E      1781                JSR     LF65E
F716  32            1782                PULA
F717  33            1783                PULB
F718  F7 7F 0C      1784                STAB    VTACC
F71B  D7 17         1785                STAB    COL
F71D  33            1786                PULB
F71E  F7 7F 0D      1787                STAB    VTACD
F721  D7 16         1788                STAB    ROW
F723  33            1789                PULB
F724  D7 14         1790                STAB    $14
F726  33            1791                PULB
F727  D7 13         1792                STAB    CURADR
F729  36            1793                PSHA
F72A  BD F7 93      1794  LF72A         JSR     LF793
```

```
F72D  31           1795                 INS
F72E  7E F2 79     1796                 JMP     LF279
F731  D6 55        1797  LF731          LDAB    $55
F733  C5 04        1798                 BITB    #$04
F735  27 30        1799                 BEQ     LF767
F737  4F           1800                 CLRA
F738  36           1801                 PSHA
F739  20 33        1802                 BRA     LF76E
F73B  8D 0F        1803  LF73B          BSR     LF74C
F73D  36           1804                 PSHA
F73E  20 2E        1805                 BRA     LF76E
F740               1806  ; Clear the screen.
F740  D6 55        1807  CLRSCR         LDAB    $55
F742  C5 04        1808                 BITB    #$04
F744  27 F5        1809                 BEQ     LF73B
F746  8D 0F        1810                 BSR     LF757
F748  8D 02        1811                 BSR     LF74C
F74A  20 1E        1812                 BRA     LF76A
F74C  86 20        1813  LF74C          LDAA    #$20
F74E  D6 55        1814                 LDAB    $55
F750  C5 40        1815                 BITB    #$40
F752  27 02        1816                 BEQ     LF756
F754  96 4D        1817                 LDAA    BLANK
F756  39           1818  LF756          RTS
F757  96 1F        1819  LF757          LDAA    $1F
F759  2A 07        1820                 BPL     LF762
F75B  96 52        1821                 LDAA    $52
F75D  2A 03        1822                 BPL     LF762
F75F  BD F5 01     1823                 JSR     LF501
F762  39           1824  LF762          RTS
F763  86 A0        1825  LF763          LDAA    #$A0
F765  20 03        1826                 BRA     LF76A
F767  8D EE        1827  LF767          BSR     LF757
F769  4F           1828                 CLRA
F76A  36           1829  LF76A          PSHA
F76B  7F 00 18     1830                 FCB     $7F,$00,$18 ; CLR $0018
F76E  7F 00 19     1831  LF76E          FCB     $7F,$00,$19 ; CLR $0019
F771  BD EF BC     1832                 JSR     LEFBC
F774  96 42        1833                 LDAA    SPLIT
F776  27 12        1834                 BEQ     LF78A
F778  84 04        1835                 ANDA    #$04
F77A  F6 78 02     1836                 LDAB    PIA1B
F77D  C4 04        1837                 ANDB    #$04
F77F  11           1838                 CBA
F780  26 03        1839                 BNE     LF785
F782  BD F6 5E     1840                 JSR     LF65E
F785  8D 09        1841  LF785          BSR     LF790
F787  BD F6 5E     1842                 JSR     LF65E
F78A  8D 04        1843  LF78A          BSR     LF790
F78C  31           1844                 INS
F78D  7E F2 79     1845                 JMP     LF279
F790  BD F2 CF     1846  LF790          JSR     LF2CF
F793  D6 11        1847  LF793          LDAB    CRTBASE
F795  CB 07        1848                 ADDB    #$07
F797  86 7F        1849                 LDAA    #$7F
F799  D7 B7        1850                 STAB    $B7
F79B  97 B8        1851                 STAA    $B8
```

```
F79D  96 15         1852                LDAA    FIRSTLN
F79F  BD F5 EA      1853                JSR     ROWADDR
F7A2  8B 4F         1854                ADDA    #$4F
F7A4  D9 11         1855                ADCB    CRTBASE
F7A6  D7 B9         1856                STAB    $B9
F7A8  97 BA         1857                STAA    $BA
F7AA  30            1858                TSX
F7AB  E6 02         1859                LDAB    $02,X
F7AD  DE 13         1860                LDX     CURADR
F7AF  96 18         1861  LF7AF         LDAA    PROTECT
F7B1  2A 08         1862                BPL     LF7BB
F7B3  B7 70 00      1863                STAA    HALT
F7B6  01            1864                NOP
F7B7  A5 00         1865                BITA    $00,X
F7B9  26 06         1866                BNE     LF7C1
F7BB  B7 70 00      1867  LF7BB         STAA    HALT
F7BE  01            1868                NOP
F7BF  E7 00         1869                STAB    $00,X
F7C1  9C B9         1870  LF7C1         CPX     $B9
F7C3  27 0D         1871                BEQ     LF7D2
F7C5  9C B7         1872                CPX     $B7
F7C7  27 03         1873                BEQ     LF7CC
F7C9  08            1874                INX
F7CA  20 E3         1875                BRA     LF7AF
F7CC  DE 11         1876  LF7CC         LDX     CRTBASE
F7CE  DF 13         1877                STX     CURADR
F7D0  20 DD         1878                BRA     LF7AF
F7D2  7E F5 F8      1879  LF7D2         JMP     CURADDR
F7D5                1880  ; Clear to end of line.
F7D5  86 20         1881  CLREOL        LDAA    #$20
F7D7  D6 55         1882                LDAB    $55
F7D9  C5 80         1883                BITB    #$80
F7DB  27 02         1884                BEQ     LF7DF
F7DD  96 4D         1885                LDAA    BLANK
F7DF  BD EF BC      1886  LF7DF         JSR     LEFBC
F7E2  D6 55         1887  LF7E2         LDAB    $55
F7E4  C5 01         1888                BITB    #$01
F7E6  27 03         1889                BEQ     LF7EB
F7E8  7F 00 19      1890                FCB     $7F,$00,$19 ; CLR $0019
F7EB  9A 19         1891  LF7EB         ORAA    ATTRMSK
F7ED  C6 50         1892                LDAB    #$50
F7EF  D0 17         1893                SUBB    COL
F7F1  D7 BA         1894                STAB    $BA
F7F3  DE 13         1895                LDX     CURADR
F7F5  D6 18         1896  LF7F5         LDAB    PROTECT
F7F7  2A 08         1897                BPL     LF801
F7F9  B7 70 00      1898                STAA    HALT
F7FC  01            1899                NOP
F7FD  E6 00         1900                LDAB    $00,X
F7FF  2B 0C         1901                BMI     LF80D
F801  B7 70 00      1902  LF801         STAA    HALT
F804  01            1903                NOP
F805  A7 00         1904                STAA    $00,X
F807  08            1905                INX
F808  7A 00 BA      1906                FCB     $7A,$00,$BA ; DEC $00BA
F80B  26 E8         1907                BNE     LF7F5
F80D  DE 13         1908  LF80D         LDX     CURADR
```

```
F80F  B7 70 00      1909                STAA    HALT
F812  01            1910                NOP
F813  A6 00         1911                LDAA    $00,X
F815  95 18         1912                BITA    PROTECT
F817  27 03         1913                BEQ     LF81C
F819  BD F3 FA      1914                JSR     LF3FA
F81C  39            1915  LF81C         RTS
F81D  4F            1916  LF81D         CLRA
F81E  20 BF         1917                BRA     LF7DF
F820  96 4D         1918  LF820         LDAA    BLANK
F822  20 BE         1919                BRA     LF7E2
F824  BD F0 79      1920  LF824         JSR     LF079
F827  DE 13         1921  LF827         LDX     CURADR
F829  86 50         1922                LDAA    #$50
F82B  90 17         1923                SUBA    COL
F82D  97 BA         1924                STAA    $BA
F82F  C6 20         1925                LDAB    #$20
F831  B7 70 00      1926  LF831         STAA    HALT
F834  01            1927                NOP
F835  A6 00         1928                LDAA    $00,X
F837  95 18         1929                BITA    PROTECT
F839  26 0D         1930                BNE     LF848
F83B  B7 70 00      1931                STAA    HALT
F83E  01            1932                NOP
F83F  E7 00         1933                STAB    $00,X
F841  16            1934                TAB
F842  08            1935                INX
F843  7A 00 BA      1936                FCB     $7A,$00,$BA ; DEC $00BA
F846  26 E9         1937                BNE     LF831
F848  DE 13         1938  LF848         LDX     CURADR
F84A  39            1939                RTS
F84B  BD F0 79      1940  LF84B         JSR     LF079
F84E  DE 13         1941                LDX     CURADR
F850  C6 4F         1942                LDAB    #$4F
F852  F0 7F 09      1943                SUBB    VTAC9
F855  27 14         1944  LF855         BEQ     LF86B
F857  B7 70 00      1945                STAA    HALT
F85A  01            1946                NOP
F85B  A6 01         1947                LDAA    $01,X
F85D  95 18         1948                BITA    PROTECT
F85F  26 0A         1949                BNE     LF86B
F861  B7 70 00      1950                STAA    HALT
F864  01            1951                NOP
F865  A7 00         1952                STAA    $00,X
F867  08            1953                INX
F868  5A            1954                DECB
F869  20 EA         1955                BRA     LF855
F86B  96 4D         1956  LF86B         LDAA    BLANK
F86D  B7 70 00      1957                STAA    HALT
F870  01            1958                NOP
F871  A7 00         1959                STAA    $00,X
F873  39            1960                RTS
F874                1961  ; ESC E : insert line.
F874  7D 00 18      1962  INSLINE       FCB     $7D,$00,$18 ; TST $0018
F877  27 01         1963                BEQ     LF87A
F879  39            1964                RTS
F87A  7F 00 19      1965  LF87A         FCB     $7F,$00,$19 ; CLR $0019
```

```
F87D  BD F2 E5      1966                JSR     LF2E5
F880  DE 13         1967                LDX     CURADR
F882  DF BB         1968                STX     $BB
F884  96 42         1969                LDAA    SPLIT
F886  2A 3D         1970                BPL     LF8C5
F888  F6 78 02      1971                LDAB    PIA1B
F88B  84 04         1972                ANDA    #$04
F88D  C4 04         1973                ANDB    #$04
F88F  11            1974                CBA
F890  26 33         1975                BNE     LF8C5
F892  C6 87         1976                LDAB    #$87
F894  4D            1977                TSTA
F895  26 02         1978                BNE     LF899
F897  CB 08         1979                ADDB    #$08
F899  86 D0         1980  LF899          LDAA    #$D0
F89B  D7 BB         1981                STAB    $BB
F89D  97 BC         1982                STAA    $BC
F89F  DE BB         1983                LDX     $BB
F8A1  A6 04         1984                LDAA    $04,X
F8A3  36            1985                PSHA
F8A4  4A            1986                DECA
F8A5  2A 02         1987                BPL     LF8A9
F8A7  86 17         1988                LDAA    #$17
F8A9  A7 04         1989  LF8A9          STAA    $04,X
F8AB  32            1990                PULA
F8AC  BD F5 EA      1991                JSR     ROWADDR
F8AF  EB 00         1992                ADDB    $00,X
F8B1  D7 B1         1993                STAB    $B1
F8B3  97 B2         1994                STAA    $B2
F8B5  96 15         1995                LDAA    FIRSTLN
F8B7  BD F5 EA      1996                JSR     ROWADDR
F8BA  DB 11         1997                ADDB    CRTBASE
F8BC  D7 B3         1998                STAB    $B3
F8BE  97 B4         1999                STAA    $B4
F8C0  C6 50         2000                LDAB    #$50
F8C2  BD F6 BB      2001                JSR     BLITCPY
F8C5  96 15         2002  LF8C5          LDAA    FIRSTLN
F8C7  91 16         2003  LF8C7          CMPA    ROW
F8C9  27 42         2004                BEQ     LF90D
F8CB  4D            2005                TSTA
F8CC  27 24         2006                BEQ     LF8F2
F8CE  4A            2007                DECA
F8CF  97 B9         2008                STAA    $B9
F8D1  BD F5 EA      2009                JSR     ROWADDR
F8D4  DB 11         2010                ADDB    CRTBASE
F8D6  D7 B7         2011                STAB    $B7
F8D8  97 B8         2012                STAA    $B8
F8DA  DE B7         2013                LDX     $B7
F8DC  C6 50         2014                LDAB    #$50
F8DE  B7 70 00      2015  LF8DE          STAA    HALT
F8E1  01            2016                NOP
F8E2  A6 00         2017                LDAA    $00,X
F8E4  B7 70 00      2018                STAA    HALT
F8E7  01            2019                NOP
F8E8  A7 50         2020                STAA    $50,X
F8EA  08            2021                INX
F8EB  5A            2022                DECB
```

```
F8EC  26 F0         2023                  BNE     LF8DE
F8EE  96 B9         2024                  LDAA    $B9
F8F0  20 D5         2025                  BRA     LF8C7
F8F2  DE 11         2026  LF8F2           LDX     CRTBASE
F8F4  DF B1         2027                  STX     $B1
F8F6  96 B2         2028                  LDAA    $B2
F8F8  8B 30         2029                  ADDA    #$30
F8FA  97 B4         2030                  STAA    $B4
F8FC  96 B1         2031                  LDAA    $B1
F8FE  89 07         2032                  ADCA    #$07
F900  97 B3         2033                  STAA    $B3
F902  C6 50         2034                  LDAB    #$50
F904  BD F6 BB      2035                  JSR     BLITCPY
F907  86 17         2036                  LDAA    #$17
F909  97 B9         2037                  STAA    $B9
F90B  20 BA         2038                  BRA     LF8C7
F90D  96 4D         2039  LF90D           LDAA    BLANK
F90F  7E F7 E2      2040                  JMP     LF7E2
F912                2041  ; ESC R : delete line.
F912  7D 00 18      2042  DELLINE         FCB     $7D,$00,$18 ; TST $0018
F915  27 01         2043                  BEQ     LF918
F917  39            2044                  RTS
F918  7F 00 19      2045  LF918           FCB     $7F,$00,$19 ; CLR $0019
F91B  BD F2 E5      2046                  JSR     LF2E5
F91E  96 16         2047                  LDAA    ROW
F920  91 15         2048  LF920           CMPA    FIRSTLN
F922  27 3C         2049                  BEQ     LF960
F924  81 17         2050                  CMPA    #$17
F926  26 16         2051                  BNE     LF93E
F928  D6 11         2052                  LDAB    CRTBASE
F92A  CB 07         2053                  ADDB    #$07
F92C  86 30         2054                  LDAA    #$30
F92E  D7 B1         2055                  STAB    $B1
F930  97 B2         2056                  STAA    $B2
F932  DE 11         2057                  LDX     CRTBASE
F934  DF B3         2058                  STX     $B3
F936  C6 50         2059                  LDAB    #$50
F938  BD F6 BB      2060                  JSR     BLITCPY
F93B  4F            2061                  CLRA
F93C  20 E2         2062                  BRA     LF920
F93E  36            2063  LF93E           PSHA
F93F  BD F5 EA      2064                  JSR     ROWADDR
F942  DB 11         2065                  ADDB    CRTBASE
F944  D7 B7         2066                  STAB    $B7
F946  97 B8         2067                  STAA    $B8
F948  DE B7         2068                  LDX     $B7
F94A  C6 50         2069                  LDAB    #$50
F94C  B7 70 00      2070  LF94C           STAA    HALT
F94F  01            2071                  NOP
F950  A6 50         2072                  LDAA    $50,X
F952  B7 70 00      2073                  STAA    HALT
F955  01            2074                  NOP
F956  A7 00         2075                  STAA    $00,X
F958  08            2076                  INX
F959  5A            2077                  DECB
F95A  26 F0         2078                  BNE     LF94C
F95C  32            2079                  PULA
```

```
F95D  4C           2080                INCA
F95E  20 C0        2081                BRA     LF920
F960  BD F5 EA     2082  LF960        JSR     ROWADDR
F963  DB 11        2083                ADDB    CRTBASE
F965  D7 B1        2084                STAB    $B1
F967  D7 B7        2085                STAB    $B7
F969  97 B2        2086                STAA    $B2
F96B  97 B8        2087                STAA    $B8
F96D  96 42        2088                LDAA    SPLIT
F96F  27 33        2089                BEQ     LF9A4
F971  F6 78 02     2090                LDAB    PIA1B
F974  C4 04        2091                ANDB    #$04
F976  84 04        2092                ANDA    #$04
F978  11           2093                CBA
F979  26 29        2094                BNE     LF9A4
F97B  88 04        2095                EORA    #$04
F97D  48           2096                ASLA
F97E  8B 87        2097                ADDA    #$87
F980  C6 D0        2098                LDAB    #$D0
F982  97 B3        2099                STAA    $B3
F984  D7 B4        2100                STAB    $B4
F986  DE B3        2101                LDX     $B3
F988  A6 04        2102                LDAA    $04,X
F98A  4C           2103                INCA
F98B  81 17        2104                CMPA    #$17
F98D  2F 01        2105                BLE     LF990
F98F  4F           2106                CLRA
F990  A7 04        2107  LF990        STAA    $04,X
F992  BD F5 EA     2108                JSR     ROWADDR
F995  EB 00        2109                ADDB    $00,X
F997  D7 B3        2110                STAB    $B3
F999  97 B4        2111                STAA    $B4
F99B  DE B3        2112                LDX     $B3
F99D  DF B7        2113                STX     $B7
F99F  C6 50        2114                LDAB    #$50
F9A1  BD F6 BB     2115                JSR     BLITCPY
F9A4  DE B7        2116  LF9A4        LDX     $B7
F9A6  C6 50        2117                LDAB    #$50
F9A8  96 4D        2118                LDAA    BLANK
F9AA  B7 70 00     2119  LF9AA        STAA    HALT
F9AD  01           2120                NOP
F9AE  A7 00        2121                STAA    $00,X
F9B0  08           2122                INX
F9B1  5A           2123                DECB
F9B2  26 F6        2124                BNE     LF9AA
F9B4  39           2125                RTS
F9B5  86 02        2126  LF9B5        LDAA    #$02
F9B7  20 21        2127                BRA     LF9DA
F9B9  86 03        2128  LF9B9        LDAA    #$03
F9BB  20 1D        2129                BRA     LF9DA
F9BD  86 04        2130  LF9BD        LDAA    #$04
F9BF  20 19        2131                BRA     LF9DA
F9C1  86 05        2132  LF9C1        LDAA    #$05
F9C3  20 15        2133                BRA     LF9DA
F9C5  86 06        2134  LF9C5        LDAA    #$06
F9C7  20 11        2135                BRA     LF9DA
F9C9  86 07        2136  LF9C9        LDAA    #$07
```

```
F9CB  20 0D         2137                BRA     LF9DA
F9CD  86 08         2138  LF9CD         LDAA    #$08
F9CF  20 09         2139                BRA     LF9DA
F9D1  86 09         2140  LF9D1         LDAA    #$09
F9D3  20 05         2141                BRA     LF9DA
F9D5  86 0A         2142  LF9D5         LDAA    #$0A
F9D7  20 01         2143                BRA     LF9DA
F9D9  43            2144  LF9D9         COMA
F9DA  97 C5         2145  LF9DA         STAA    $C5
F9DC  BD EF BC      2146                JSR     LEFBC
F9DF  7F 00 AE      2147                FCB     $7F,$00,$AE ; CLR $00AE
F9E2  4F            2148                CLRA
F9E3  CE 00 04      2149                LDX     #$0004
F9E6  BD EC B8      2150                JSR     LECB8
F9E9  CE 00 00      2151                LDX     #$0000
F9EC  BD EC C3      2152                JSR     BLOCKX
F9EF  CE 00 02      2153                LDX     #$0002
F9F2  7E EC C3      2154                JMP     BLOCKX
F9F5  BD FC DD      2155  LF9F5         JSR     LFCDD
F9F8  96 C5         2156                LDAA    $C5
F9FA  2A 05         2157                BPL     LFA01
F9FC  CE FA 77      2158                LDX     #LFA77
F9FF  20 0F         2159                BRA     LFA10
FA01  D6 55         2160  LFA01         LDAB    $55
FA03  C4 20         2161                ANDB    #$20
FA05  D7 CD         2162                STAB    $CD
FA07  CE FA 32      2163                LDX     #JMPTAB
FA0A  48            2164                ASLA
FA0B  BD EF AD      2165                JSR     XPLUSA
FA0E  EE 00         2166                LDX     $00,X
FA10  AD 00         2167  LFA10         JSR     $00,X
FA12  96 C5         2168                LDAA    $C5
FA14  81 0A         2169                CMPA    #$0A
FA16  27 0E         2170                BEQ     LFA26
FA18  96 46         2171                LDAA    $46
FA1A  27 0A         2172                BEQ     LFA26
FA1C  BD FC F5      2173                JSR     LFCF5
FA1F  96 47         2174                LDAA    $47
FA21  27 03         2175                BEQ     LFA26
FA23  BD FC F5      2176                JSR     LFCF5
FA26  BD FB 2F      2177  LFA26         JSR     LFB2F
FA29  CE 00 07      2178                LDX     #$0007
FA2C  BD EC BA      2179                JSR     WAKE
FA2F  7E EC C1      2180                JMP     BLOCKME
FA32  FA 48 FA 48   2181  JMPTAB        FCB     $FA,$48,$FA,$48,$FA,$A5
FA38  FA A5 FA AD   2182                FCB     $FA,$A5,$FA,$AD,$FA,$AD
FA3E  FC 0B FC 05   2183                FCB     $FC,$0B,$FC,$05,$FA,$62
FA44  FA 49 FA 82   2184                FCB     $FA,$49,$FA,$82
FA48  39            2185  LFA48         RTS
FA49  B6 78 02      2186  LFA49         LDAA    PIA1B
FA4C  D6 42         2187                LDAB    SPLIT
FA4E  27 02         2188                BEQ     LFA52
FA50  98 42         2189                EORA    SPLIT
FA52  84 04         2190  LFA52         ANDA    #$04
FA54  44            2191                LSRA
FA55  44            2192                LSRA
FA56  8A 30         2193                ORAA    #$30
```

```
FA58  D6 54         2194                LDAB    $54
FA5A  C5 40         2195                BITB    #$40
FA5C  26 01         2196                BNE     LFA5F
FA5E  4C            2197                INCA
FA5F  BD FC F5      2198  LFA5F         JSR     LFCF5
FA62  96 16         2199  LFA62         LDAA    ROW
FA64  4A            2200                DECA
FA65  90 15         2201                SUBA    FIRSTLN
FA67  2C 02         2202                BGE     LFA6B
FA69  8B 18         2203                ADDA    #$18
FA6B  8B 20         2204  LFA6B         ADDA    #$20
FA6D  BD FC F5      2205                JSR     LFCF5
FA70  96 17         2206                LDAA    COL
FA72  8B 20         2207                ADDA    #$20
FA74  7E FC F5      2208                JMP     LFCF5
FA77  96 4B         2209  LFA77         LDAA    $4B
FA79  BD FC F5      2210                JSR     LFCF5
FA7C  96 C5         2211                LDAA    $C5
FA7E  43            2212                COMA
FA7F  7E FC F5      2213                JMP     LFCF5
FA82  CE 7B 80      2214  LFA82         LDX     #$7B80
FA85  DF B7         2215                STX     $B7
FA87  A6 1E         2216                LDAA    $1E,X
FA89  81 0D         2217                CMPA    #$0D
FA8B  26 17         2218                BNE     LFAA4
FA8D  A6 1F         2219                LDAA    $1F,X
FA8F  81 8A         2220                CMPA    #$8A
FA91  26 11         2221                BNE     LFAA4
FA93  84 7F         2222                ANDA    #$7F
FA95  97 C5         2223                STAA    $C5
FA97  DE B7         2224  LFA97         LDX     $B7
FA99  A6 00         2225                LDAA    $00,X
FA9B  08            2226                INX
FA9C  DF B7         2227                STX     $B7
FA9E  BD FC F5      2228                JSR     LFCF5
FAA1  4D            2229                TSTA
FAA2  2A F3         2230                BPL     LFA97
FAA4  39            2231  LFAA4         RTS
FAA5  BD FB 26      2232  LFAA5         JSR     LFB26
FAA8  BD F2 E5      2233                JSR     LF2E5
FAAB  20 17         2234                BRA     LFAC4
FAAD  BD FB 26      2235  LFAAD         JSR     LFB26
FAB0  96 42         2236                LDAA    SPLIT
FAB2  27 0D         2237                BEQ     LFAC1
FAB4  F6 78 02      2238                LDAB    PIA1B
FAB7  84 04         2239                ANDA    #$04
FAB9  C4 04         2240                ANDB    #$04
FABB  11            2241                CBA
FABC  27 03         2242                BEQ     LFAC1
FABE  BD F6 5E      2243                JSR     LF65E
FAC1  BD F2 CF      2244  LFAC1         JSR     LF2CF
FAC4  4F            2245  LFAC4         CLRA
FAC5  B7 00 1E      2246                FCB     $B7,$00,$1E ; STAA $001E
FAC8  97 C6         2247                STAA    $C6
FACA  BD EF 8B      2248  LFACA         JSR     LEF8B
FACD  7D 00 AE      2249                FCB     $7D,$00,$AE ; TST $00AE
FAD0  2B 2D         2250                BMI     LFAFF
```

```
FAD2  BD FB 57      2251                JSR     LFB57
FAD5  D6 50         2252                LDAB    $50
FAD7  2A 04         2253                BPL     LFADD
FAD9  81 03         2254                CMPA    #$03
FADB  27 22         2255                BEQ     LFAFF
FADD  C5 10         2256  LFADD         BITB    #$10
FADF  27 06         2257                BEQ     LFAE7
FAE1  DE 13         2258                LDX     CURADR
FAE3  9C C1         2259                CPX     $C1
FAE5  27 18         2260                BEQ     LFAFF
FAE7  4D            2261  LFAE7         TSTA
FAE8  27 03         2262                BEQ     LFAED
FAEA  BD FC F5      2263                JSR     LFCF5
FAED  DE 13         2264  LFAED         LDX     CURADR
FAEF  9C C1         2265                CPX     $C1
FAF1  27 0C         2266                BEQ     LFAFF
FAF3  BD FB 4C      2267                JSR     LFB4C
FAF6  7D 00 17      2268                FCB     $7D,$00,$17 ; TST $0017
FAF9  26 CF         2269                BNE     LFACA
FAFB  8D 03         2270                BSR     LFB00
FAFD  20 CB         2271                BRA     LFACA
FAFF  39            2272  LFAFF         RTS
FB00  96 55         2273  LFB00         LDAA    $55
FB02  85 10         2274                BITA    #$10
FB04  27 04         2275                BEQ     LFB0A
FB06  96 18         2276                LDAA    PROTECT
FB08  2B 1B         2277                BMI     LFB25
FB0A  96 48         2278  LFB0A         LDAA    $48
FB0C  27 17         2279                BEQ     LFB25
FB0E  BD FC F5      2280                JSR     LFCF5
FB11  96 49         2281                LDAA    $49
FB13  27 10         2282                BEQ     LFB25
FB15  7E FC F5      2283                JMP     LFCF5
FB18  36            2284  LFB18         PSHA
FB19  96 CD         2285                LDAA    $CD
FB1B  26 07         2286                BNE     LFB24
FB1D  96 4A         2287                LDAA    $4A
FB1F  27 03         2288                BEQ     LFB24
FB21  BD FC F5      2289                JSR     LFCF5
FB24  32            2290  LFB24         PULA
FB25  39            2291  LFB25         RTS
FB26  DE 16         2292  LFB26         LDX     ROW
FB28  DF C3         2293                STX     $C3
FB2A  DE 13         2294                LDX     CURADR
FB2C  DF C1         2295                STX     $C1
FB2E  39            2296                RTS
FB2F  96 18         2297  LFB2F         LDAA    PROTECT
FB31  26 15         2298                BNE     LFB48
FB33  96 C5         2299                LDAA    $C5
FB35  81 02         2300                CMPA    #$02
FB37  2D 0F         2301                BLT     LFB48
FB39  81 06         2302                CMPA    #$06
FB3B  2E 0B         2303                BGT     LFB48
FB3D  2D 06         2304                BLT     LFB45
FB3F  D6 54         2305                LDAB    $54
FB41  C5 20         2306                BITB    #$20
FB43  27 03         2307                BEQ     LFB48
```

```
FB45  BD F2 9A      2308  LFB45           JSR     LF29A
FB48  7F 00 C5      2309  LFB48           FCB     $7F,$00,$C5 ; CLR $00C5
FB4B  39            2310                  RTS
FB4C  BD F2 F0      2311  LFB4C           JSR     LF2F0
FB4F  7D 00 B5      2312                  FCB     $7D,$00,$B5 ; TST $00B5
FB52  27 1E         2313                  BEQ     LFB72
FB54  7E F3 13      2314                  JMP     LF313
FB57  DE 13         2315  LFB57           LDX     CURADR
FB59  B7 70 00      2316                  STAA    HALT
FB5C  01            2317                  NOP
FB5D  A6 00         2318                  LDAA    $00,X
FB5F  7D 00 18      2319                  FCB     $7D,$00,$18 ; TST $0018
FB62  26 0F         2320                  BNE     LFB73
FB64  16            2321                  TAB
FB65  C4 F0         2322                  ANDB    #$F0
FB67  C1 90         2323                  CMPB    #$90
FB69  26 02         2324                  BNE     LFB6D
FB6B  86 20         2325                  LDAA    #$20
FB6D  84 7F         2326  LFB6D           ANDA    #$7F
FB6F  7F 00 CD      2327                  FCB     $7F,$00,$CD ; CLR $00CD
FB72  39            2328  LFB72           RTS
FB73  D6 C5         2329  LFB73           LDAB    $C5
FB75  C5 01         2330                  BITB    #$01
FB77  26 3D         2331                  BNE     LFBB6
FB79  D6 C6         2332                  LDAB    $C6
FB7B  4D            2333                  TSTA
FB7C  2B 0B         2334                  BMI     LFB89
FB7E  C1 01         2335                  CMPB    #$01
FB80  26 F0         2336                  BNE     LFB72
FB82  7F 00 C6      2337                  FCB     $7F,$00,$C6 ; CLR $00C6
FB85  8D 91         2338                  BSR     LFB18
FB87  20 E4         2339                  BRA     LFB6D
FB89  81 E0         2340  LFB89           CMPA    #$E0
FB8B  27 18         2341                  BEQ     LFBA5
FB8D  C1 02         2342                  CMPB    #$02
FB8F  27 DC         2343                  BEQ     LFB6D
FB91  C6 01         2344  LFB91           LDAB    #$01
FB93  D7 C6         2345  LFB93           STAB    $C6
FB95  9C C1         2346                  CPX     $C1
FB97  27 1B         2347                  BEQ     LFBB4
FB99  BD FB 4C      2348  LFB99           JSR     LFB4C
FB9C  D6 17         2349                  LDAB    COL
FB9E  26 B7         2350                  BNE     LFB57
FBA0  BD FB 00      2351                  JSR     LFB00
FBA3  20 B2         2352                  BRA     LFB57
FBA5  C1 02         2353  LFBA5           CMPB    #$02
FBA7  27 E8         2354                  BEQ     LFB91
FBA9  C1 01         2355                  CMPB    #$01
FBAB  26 03         2356                  BNE     LFBB0
FBAD  BD FB 18      2357                  JSR     LFB18
FBB0  C6 02         2358  LFBB0           LDAB    #$02
FBB2  20 DF         2359                  BRA     LFB93
FBB4  4F            2360  LFBB4           CLRA
FBB5  39            2361                  RTS
FBB6  4D            2362  LFBB6           TSTA
FBB7  2B 16         2363                  BMI     LFBCF
FBB9  7D 00 C6      2364                  FCB     $7D,$00,$C6 ; TST $00C6
```

```
FBBC  27 AF        2365                BEQ     LFB6D
FBBE  36           2366                PSHA
FBBF  96 45        2367                LDAA    ESCCHR
FBC1  BD FC F5     2368                JSR     LFCF5
FBC4  86 28        2369                LDAA    #$28
FBC6  BD FC F5     2370                JSR     LFCF5
FBC9  7F 00 C6     2371                FCB     $7F,$00,$C6 ; CLR $00C6
FBCC  32           2372                PULA
FBCD  20 9E        2373                BRA     LFB6D
FBCF  16           2374  LFBCF         TAB
FBD0  C4 F0        2375                ANDB    #$F0
FBD2  C1 90        2376                CMPB    #$90
FBD4  26 17        2377                BNE     LFBED
FBD6  36           2378                PSHA
FBD7  96 45        2379                LDAA    ESCCHR
FBD9  BD FC F5     2380                JSR     LFCF5
FBDC  86 47        2381                LDAA    #$47
FBDE  BD FC F5     2382                JSR     LFCF5
FBE1  32           2383                PULA
FBE2  84 0F        2384                ANDA    #$0F
FBE4  BD F6 26     2385                JSR     LF626
FBE7  BD FC F5     2386                JSR     LFCF5
FBEA  7E FB 99     2387                JMP     LFB99
FBED  7D 00 C6     2388  LFBED         FCB     $7D,$00,$C6 ; TST $00C6
FBF0  26 10        2389                BNE     LFC02
FBF2  36           2390                PSHA
FBF3  96 45        2391                LDAA    ESCCHR
FBF5  BD FC F5     2392                JSR     LFCF5
FBF8  86 29        2393                LDAA    #$29
FBFA  BD FC F5     2394                JSR     LFCF5
FBFD  86 01        2395                LDAA    #$01
FBFF  97 C6        2396                STAA    $C6
FC01  32           2397                PULA
FC02  7E FB 6D     2398  LFC02         JMP     LFB6D
FC05  96 54        2399  LFC05         LDAA    $54
FC07  85 20        2400                BITA    #$20
FC09  26 F7        2401                BNE     LFC02
FC0B  BD FB 26     2402  LFC0B         JSR     LFB26
FC0E  7F 00 BB     2403                FCB     $7F,$00,$BB ; CLR $00BB
FC11  7F 00 BC     2404                FCB     $7F,$00,$BC ; CLR $00BC
FC14  F6 78 02     2405                LDAB    PIA1B
FC17  C4 04        2406                ANDB    #$04
FC19  D7 BD        2407                STAB    $BD
FC1B  96 54        2408                LDAA    $54
FC1D  85 20        2409                BITA    #$20
FC1F  27 0E        2410                BEQ     LFC2F
FC21  86 1C        2411                LDAA    #$1C
FC23  97 B9        2412                STAA    $B9
FC25  97 BA        2413                STAA    $BA
FC27  B7 70 00     2414                STAA    HALT
FC2A  01           2415                NOP
FC2B  A7 00        2416                STAA    $00,X
FC2D  20 23        2417                BRA     LFC52
FC2F  86 02        2418  LFC2F         LDAA    #$02
FC31  97 B9        2419                STAA    $B9
FC33  86 03        2420                LDAA    #$03
FC35  97 BA        2421                STAA    $BA
```

```
FC37  96 42         2422                LDAA    SPLIT
FC39  27 08         2423                BEQ     LFC43
FC3B  84 04         2424                ANDA    #$04
FC3D  11            2425                CBA
FC3E  26 03         2426                BNE     LFC43
FC40  BD F6 5E      2427                JSR     LF65E
FC43  96 15         2428  LFC43         LDAA    FIRSTLN
FC45  BD F5 EA      2429                JSR     ROWADDR
FC48  DB 11         2430                ADDB    CRTBASE
FC4A  8B 4F         2431                ADDA    #$4F
FC4C  C9 00         2432                ADCB    #$00
FC4E  D7 BB         2433                STAB    $BB
FC50  97 BC         2434                STAA    $BC
FC52  96 42         2435  LFC52         LDAA    SPLIT
FC54  27 0D         2436                BEQ     LFC63
FC56  84 04         2437                ANDA    #$04
FC58  F6 78 02      2438                LDAB    PIA1B
FC5B  C4 04         2439                ANDB    #$04
FC5D  11            2440                CBA
FC5E  27 03         2441                BEQ     LFC63
FC60  BD F6 5E      2442                JSR     LF65E
FC63  96 15         2443  LFC63         LDAA    FIRSTLN
FC65  80 17         2444                SUBA    #$17
FC67  2C 02         2445                BGE     LFC6B
FC69  8B 18         2446                ADDA    #$18
FC6B  BD F5 EA      2447  LFC6B         JSR     ROWADDR
FC6E  DB 11         2448                ADDB    CRTBASE
FC70  D7 B7         2449                STAB    $B7
FC72  97 B8         2450                STAA    $B8
FC74  F6 78 02      2451                LDAB    PIA1B
FC77  C4 04         2452                ANDB    #$04
FC79  D1 BD         2453                CMPB    $BD
FC7B  27 03         2454                BEQ     LFC80
FC7D  BD F6 5E      2455                JSR     LF65E
FC80  DE 13         2456  LFC80         LDX     CURADR
FC82  9C B7         2457                CPX     $B7
FC84  27 27         2458                BEQ     LFCAD
FC86  BD F3 6F      2459  LFC86         JSR     LF36F
FC89  B7 70 00      2460                STAA    HALT
FC8C  01            2461                NOP
FC8D  A6 00         2462                LDAA    $00,X
FC8F  91 B9         2463                CMPA    $B9
FC91  27 06         2464                BEQ     LFC99
FC93  9C B7         2465                CPX     $B7
FC95  26 EF         2466                BNE     LFC86
FC97  20 14         2467                BRA     LFCAD
FC99  D6 54         2468  LFC99         LDAB    $54
FC9B  C5 20         2469                BITB    #$20
FC9D  27 06         2470                BEQ     LFCA5
FC9F  D6 16         2471                LDAB    ROW
FCA1  D1 C3         2472                CMPB    $C3
FCA3  26 05         2473                BNE     LFCAA
FCA5  BD FB 4C      2474  LFCA5         JSR     LFB4C
FCA8  20 03         2475                BRA     LFCAD
FCAA  BD F3 13      2476  LFCAA         JSR     LF313
FCAD  DE BB         2477  LFCAD         LDX     $BB
FCAF  DF C1         2478                STX     $C1
```

```
FCB1  BD EF 8B      2479  LFCB1           JSR     LEF8B
FCB4  7D 00 AE      2480                  FCB     $7D,$00,$AE ; TST $00AE
FCB7  2B 20         2481                  BMI     LFCD9
FCB9  BD FB 57      2482                  JSR     LFB57
FCBC  4D            2483                  TSTA
FCBD  27 07         2484                  BEQ     LFCC6
FCBF  91 BA         2485                  CMPA    $BA
FCC1  27 16         2486                  BEQ     LFCD9
FCC3  BD FC F5      2487                  JSR     LFCF5
FCC6  DE 13         2488  LFCC6           LDX     CURADR
FCC8  9C C1         2489                  CPX     $C1
FCCA  27 0E         2490                  BEQ     LFCDA
FCCC  BD FB 4C      2491                  JSR     LFB4C
FCCF  7D 00 17      2492                  FCB     $7D,$00,$17 ; TST $0017
FCD2  26 DD         2493                  BNE     LFCB1
FCD4  BD FB 00      2494                  JSR     LFB00
FCD7  20 D8         2495                  BRA     LFCB1
FCD9  39            2496  LFCD9           RTS
FCDA  7E F2 8F      2497  LFCDA           JMP     LF28F
FCDD  D6 CC         2498  LFCDD           LDAB    $CC
FCDF  CA 01         2499                  ORAB    #$01
FCE1  D7 CC         2500                  STAB    $CC
FCE3  36            2501                  PSHA
FCE4  F6 7E 00      2502                  LDAB    SWBNK1
FCE7  C4 1C         2503                  ANDB    #$1C
FCE9  96 BF         2504                  LDAA    ACIACTL
FCEB  84 83         2505                  ANDA    #$83
FCED  1B            2506                  ABA
FCEE  97 BF         2507                  STAA    ACIACTL
FCF0  B7 7A 00      2508                  STAA    ACIA
FCF3  32            2509                  PULA
FCF4  39            2510                  RTS
FCF5  36            2511  LFCF5           PSHA
FCF6  7D 00 CC      2512                  FCB     $7D,$00,$CC ; TST $00CC
FCF9  26 02         2513                  BNE     LFCFD
FCFB  8D E0         2514                  BSR     LFCDD
FCFD  D6 CB         2515  LFCFD           LDAB    TXCOUNT
FCFF  C1 0C         2516                  CMPB    #$0C
FD01  27 FA         2517                  BEQ     LFCFD
FD03  DE C9         2518                  LDX     $C9
FD05  32            2519                  PULA
FD06  A7 00         2520                  STAA    $00,X
FD08  16            2521                  TAB
FD09  D8 0E         2522                  EORB    $0E
FD0B  D7 0E         2523                  STAB    $0E
FD0D  7C 00 CB      2524                  FCB     $7C,$00,$CB ; INC $00CB
FD10  08            2525                  INX
FD11  8C 01 0C      2526                  CPX     #$010C
FD14  26 03         2527                  BNE     LFD19
FD16  CE 01 00      2528                  LDX     #$0100
FD19  DF C9         2529  LFD19           STX     $C9
FD1B  D6 BF         2530                  LDAB    ACIACTL
FD1D  C4 9F         2531                  ANDB    #$9F
FD1F  CA 20         2532                  ORAB    #$20
FD21  D7 BF         2533                  STAB    ACIACTL
FD23  F7 7A 00      2534                  STAB    ACIA
FD26  39            2535                  RTS
```

```
FD27                 2536  ; TXDELAY: wait for the TX queue and ACIA to empty,
FD27                 2537  ; delay, then rewrite ACIA control (bits 6-5 = 10),
FD27                 2538  ; wake tasks 0 and 2.
FD27  7F 00 CC       2539  TXDELAY         FCB     $7F,$00,$CC ; CLR $00CC
FD2A  96 CB          2540  LFD2A           LDAA    TXCOUNT
FD2C  27 07          2541                  BEQ     LFD35
FD2E  86 01          2542                  LDAA    #$01
FD30  BD EC CC       2543                  JSR     SLEEP
FD33  20 F5          2544                  BRA     LFD2A
FD35  B6 7A 00       2545  LFD35           LDAA    ACIA
FD38  85 02          2546                  BITA    #$02
FD3A  27 EE          2547                  BEQ     LFD2A
FD3C  96 4F          2548                  LDAA    $4F
FD3E  26 0D          2549                  BNE     LFD4D
FD40  CE FD 75       2550                  LDX     #BAUDTAB
FD43  B6 7E 80       2551                  LDAA    BAUDSW
FD46  84 0F          2552                  ANDA    #$0F
FD48  BD EF AD       2553                  JSR     XPLUSA
FD4B  A6 00          2554                  LDAA    $00,X
FD4D  BD EC CC       2555  LFD4D           JSR     SLEEP
FD50  96 BF          2556                  LDAA    ACIACTL
FD52  84 9F          2557                  ANDA    #$9F
FD54  8A 40          2558                  ORAA    #$40
FD56  97 BF          2559                  STAA    ACIACTL
FD58  B7 7A 00       2560                  STAA    ACIA
FD5B  CE 00 00       2561                  LDX     #$0000
FD5E  BD EC BA       2562                  JSR     WAKE
FD61  CE 00 02       2563                  LDX     #$0002
FD64  BD EC BA       2564                  JSR     WAKE
FD67  7E EC C1       2565                  JMP     BLOCKME
FD6A  CE 01 00       2566  LFD6A           LDX     #$0100
FD6D  DF C9          2567                  STX     $C9
FD6F  DF C7          2568                  STX     $C7
FD71  7F 00 CB       2569                  FCB     $7F,$00,$CB ; CLR $00CB
FD74  39             2570                  RTS
FD75                 2571  ; Delay counts (ticks) indexed by the baud switch:
FD75                 2572  ; low nibble of $7E80.
FD75  AC 73 58 40    2573  BAUDTAB         FCB     $AC,$73,$58,$40,$3A,$1D
FD7B  0F 08 05 05    2574                  FCB     $0F,$08,$05,$05,$04,$03
FD81  02 02 02 01    2575                  FCB     $02,$02,$02,$01
FD85  7D 00 CB       2576  LFD85           FCB     $7D,$00,$CB ; TST $00CB
FD88  27 1A          2577                  BEQ     LFDA4
FD8A  DE C7          2578                  LDX     $C7
FD8C  A6 00          2579                  LDAA    $00,X
FD8E  84 7F          2580                  ANDA    #$7F
FD90  9A CE          2581                  ORAA    $CE
FD92  B7 7A 01       2582                  STAA    ACIAD
FD95  7A 00 CB       2583                  FCB     $7A,$00,$CB ; DEC $00CB
FD98  08             2584                  INX
FD99  8C 01 0C       2585                  CPX     #$010C
FD9C  26 03          2586                  BNE     LFDA1
FD9E  CE 01 00       2587                  LDX     #$0100
FDA1  DF C7          2588  LFDA1           STX     $C7
FDA3  39             2589                  RTS
FDA4  96 BF          2590  LFDA4           LDAA    ACIACTL
FDA6  84 9F          2591                  ANDA    #$9F
FDA8  97 BF          2592                  STAA    ACIACTL
```

```
FDAA  B7 7A 00      2593                STAA    ACIA
FDAD  39            2594                RTS
FDAE  AA            2595                FCB     $AA
FDAF  D6 53         2596  LFDAF         LDAB    $53
FDB1  C5 04         2597                BITB    #$04
FDB3  26 4B         2598                BNE     LFE00
FDB5  D6 D1         2599                LDAB    $D1
FDB7  26 47         2600                BNE     LFE00
FDB9  CA 80         2601                ORAB    #$80
FDBB  D7 D1         2602                STAB    $D1
FDBD  97 D0         2603                STAA    $D0
FDBF  BD EF BC      2604                JSR     LEFBC
FDC2  7F 00 AE      2605                FCB     $7F,$00,$AE ; CLR $00AE
FDC5  96 D0         2606                LDAA    $D0
FDC7  2B 10         2607                BMI     LFDD9
FDC9  DE 13         2608                LDX     CURADR
FDCB  96 DB         2609                LDAA    $DB
FDCD  B7 70 00      2610                STAA    HALT
FDD0  01            2611                NOP
FDD1  A7 00         2612                STAA    $00,X
FDD3  BD F2 59      2613                JSR     LF259
FDD6  BD F2 CF      2614                JSR     LF2CF
FDD9  96 9E         2615  LFDD9         LDAA    $9E
FDDB  97 CF         2616                STAA    $CF
FDDD  CE 00 00      2617                LDX     #$0000
FDE0  DF B7         2618                STX     $B7
FDE2  B6 79 00      2619  LFDE2         LDAA    SWBNK2
FDE5  84 1C         2620                ANDA    #$1C
FDE7  8A 01         2621                ORAA    #$01
FDE9  97 C0         2622                STAA    $C0
FDEB  B7 7C 00      2623                STAA    PACIA
FDEE  CE 00 05      2624                LDX     #$0005
FDF1  BD EC BA      2625                JSR     WAKE
FDF4  CE 00 00      2626                LDX     #$0000
FDF7  BD EC C3      2627                JSR     BLOCKX
FDFA  CE 00 02      2628                LDX     #$0002
FDFD  7E EC C3      2629                JMP     BLOCKX
FE00  7E F0 DA      2630  LFE00         JMP     BELL
FE03  8D 40         2631  LFE03         BSR     LFE45
FE05  7D 00 D0      2632                FCB     $7D,$00,$D0 ; TST $00D0
FE08  27 01         2633                BEQ     LFE0B
FE0A  39            2634                RTS
FE0B  96 CF         2635  LFE0B         LDAA    $CF
FE0D  81 03         2636                CMPA    #$03
FE0F  26 25         2637                BNE     LFE36
FE11  86 01         2638                LDAA    #$01
FE13  BD F9 DA      2639                JSR     LF9DA
FE16  7E EC C1      2640                JMP     BLOCKME
FE19  CE 00 02      2641  LFE19         LDX     #$0002
FE1C  BD EC BA      2642                JSR     WAKE
FE1F  B6 00 01      2643  LFE1F         FCB     $B6,$00,$01 ; LDAA $0001
FE22  81 02         2644                CMPA    #$02
FE24  27 02         2645                BEQ     LFE28
FE26  8D 1D         2646                BSR     LFE45
FE28  7D 00 D0      2647  LFE28         FCB     $7D,$00,$D0 ; TST $00D0
FE2B  27 09         2648                BEQ     LFE36
FE2D  86 01         2649                LDAA    #$01
```

```
FE2F  BD EC CC      2650                JSR     SLEEP
FE32  20 EB         2651                BRA     LFE1F
FE34  27 56         2652  LFE34         BEQ     LFE8C
FE36  CE 00 02      2653  LFE36         LDX     #$0002
FE39  BD EC BA      2654                JSR     WAKE
FE3C  CE 00 00      2655                LDX     #$0000
FE3F  BD EC BA      2656                JSR     WAKE
FE42  7E EC C1      2657                JMP     BLOCKME
FE45  BD EF 8B      2658  LFE45         JSR     LEF8B
FE48  F6 7C 00      2659                LDAB    PACIA
FE4B  C4 0A         2660                ANDB    #$0A
FE4D  C1 02         2661                CMPB    #$02
FE4F  27 06         2662                BEQ     LFE57
FE51  7D 00 AE      2663                FCB     $7D,$00,$AE ; TST $00AE
FE54  2B 78         2664                BMI     LFECE
FE56  39            2665                RTS
FE57  D6 D0         2666  LFE57         LDAB    $D0
FE59  2B 73         2667                BMI     LFECE
FE5B  C4 0F         2668                ANDB    #$0F
FE5D  5A            2669                DECB
FE5E  27 1F         2670                BEQ     LFE7F
FE60  5A            2671                DECB
FE61  27 29         2672                BEQ     LFE8C
FE63  5A            2673                DECB
FE64  27 22         2674                BEQ     LFE88
FE66  5A            2675                DECB
FE67  27 3B         2676                BEQ     LFEA4
FE69  5A            2677                DECB
FE6A  27 62         2678                BEQ     LFECE
FE6C  5A            2679                DECB
FE6D  27 10         2680                BEQ     LFE7F
FE6F  5A            2681                DECB
FE70  27 1A         2682                BEQ     LFE8C
FE72  5A            2683                DECB
FE73  27 13         2684                BEQ     LFE88
FE75  5A            2685                DECB
FE76  27 14         2686                BEQ     LFE8C
FE78  5A            2687                DECB
FE79  27 14         2688                BEQ     LFE8F
FE7B  5F            2689                CLRB
FE7C  D7 D0         2690                STAB    $D0
FE7E  39            2691                RTS
FE7F  86 0D         2692  LFE7F         LDAA    #$0D
FE81  7C 00 D0      2693  LFE81         FCB     $7C,$00,$D0 ; INC $00D0
FE84  B7 7C 01      2694                STAA    PACIAD
FE87  39            2695                RTS
FE88  86 0A         2696  LFE88         LDAA    #$0A
FE8A  20 F5         2697                BRA     LFE81
FE8C  4F            2698  LFE8C         CLRA
FE8D  20 F2         2699                BRA     LFE81
FE8F  7F 00 D0      2700  LFE8F         FCB     $7F,$00,$D0 ; CLR $00D0
FE92  96 D1         2701                LDAA    $D1
FE94  84 7F         2702                ANDA    #$7F
FE96  97 D1         2703                STAA    $D1
FE98  96 C0         2704                LDAA    $C0
FE9A  84 9F         2705                ANDA    #$9F
FE9C  8A 40         2706                ORAA    #$40
```

```
FE9E  97 C0         2707                STAA    $C0
FEA0  B7 7C 00      2708                STAA    PACIA
FEA3  39            2709                RTS
FEA4  8D E6         2710  LFEA4         BSR     LFE8C
FEA6  DE 13         2711                LDX     CURADR
FEA8  DF B7         2712                STX     $B7
FEAA  96 B8         2713                LDAA    $B8
FEAC  8B 50         2714                ADDA    #$50
FEAE  97 B8         2715                STAA    $B8
FEB0  96 B7         2716                LDAA    $B7
FEB2  89 00         2717                ADCA    #$00
FEB4  97 B7         2718                STAA    $B7
FEB6  DE B7         2719                LDX     $B7
FEB8  09            2720  LFEB8         DEX
FEB9  9C 13         2721                CPX     CURADR
FEBB  27 0E         2722                BEQ     LFECB
FEBD  B7 70 00      2723                STAA    HALT
FEC0  01            2724                NOP
FEC1  A6 00         2725                LDAA    $00,X
FEC3  84 7F         2726                ANDA    #$7F
FEC5  27 F1         2727                BEQ     LFEB8
FEC7  81 20         2728                CMPA    #$20
FEC9  27 ED         2729                BEQ     LFEB8
FECB  DF B7         2730  LFECB         STX     $B7
FECD  39            2731                RTS
FECE  DE 13         2732  LFECE         LDX     CURADR
FED0  B7 70 00      2733                STAA    HALT
FED3  01            2734                NOP
FED4  A6 00         2735                LDAA    $00,X
FED6  16            2736                TAB
FED7  D1 DB         2737                CMPB    $DB
FED9  27 4A         2738                BEQ     LFF25
FEDB  7D 00 AE      2739                FCB     $7D,$00,$AE ; TST $00AE
FEDE  2B 1E         2740                BMI     LFEFE
FEE0  7D 00 D0      2741                FCB     $7D,$00,$D0 ; TST $00D0
FEE3  2B 12         2742                BMI     LFEF7
FEE5  D5 18         2743                BITB    PROTECT
FEE7  27 08         2744                BEQ     LFEF1
FEE9  D6 D0         2745                LDAB    $D0
FEEB  C5 10         2746                BITB    #$10
FEED  27 02         2747                BEQ     LFEF1
FEEF  86 20         2748                LDAA    #$20
FEF1  84 7F         2749  LFEF1         ANDA    #$7F
FEF3  26 02         2750                BNE     LFEF7
FEF5  86 20         2751                LDAA    #$20
FEF7  B7 7C 01      2752  LFEF7         STAA    PACIAD
FEFA  9C B7         2753                CPX     $B7
FEFC  27 1D         2754                BEQ     LFF1B
FEFE  96 17         2755  LFEFE         LDAA    COL
FF00  81 4F         2756                CMPA    #$4F
FF02  26 14         2757                BNE     LFF18
FF04  96 16         2758                LDAA    ROW
FF06  91 15         2759                CMPA    FIRSTLN
FF08  26 0E         2760                BNE     LFF18
FF0A  96 42         2761                LDAA    SPLIT
FF0C  27 81         2762                BEQ     LFE8F
FF0E  F6 78 02      2763                LDAB    PIA1B
```

```
FF11  84 02        2764                ANDA    #$02
FF13  C4 04        2765                ANDB    #$04
FF15  11           2766                CBA
FF16  26 1E        2767                BNE     LFF36
FF18  7E F2 F0     2768  LFF18         JMP     LF2F0
FF1B  96 D0        2769  LFF1B         LDAA    $D0
FF1D  84 F0        2770                ANDA    #$F0
FF1F  4C           2771                INCA
FF20  97 D0        2772                STAA    $D0
FF22  7E F3 13     2773                JMP     LF313
FF25  86 20        2774  LFF25         LDAA    #$20
FF27  B7 70 00     2775                STAA    HALT
FF2A  01           2776                NOP
FF2B  A7 00        2777                STAA    $00,X
FF2D  7D 00 AE     2778                FCB     $7D,$00,$AE ; TST $00AE
FF30  2B 04        2779                BMI     LFF36
FF32  96 D0        2780                LDAA    $D0
FF34  2A 03        2781                BPL     LFF39
FF36  7E FE 8F     2782  LFF36         JMP     LFE8F
FF39  84 F0        2783  LFF39         ANDA    #$F0
FF3B  8A 06        2784                ORAA    #$06
FF3D  97 D0        2785                STAA    $D0
FF3F  39           2786                RTS
FF40  BD EC E8     2787  LFF40         JSR     GETRX
FF43  84 03        2788                ANDA    #$03
FF45  27 33        2789                BEQ     LFF7A
FF47  81 03        2790                CMPA    #$03
FF49  27 36        2791                BEQ     LFF81
FF4B  0E           2792                CLI
FF4C  7D 00 D2     2793                FCB     $7D,$00,$D2 ; TST $00D2
FF4F  26 26        2794                BNE     LFF77
FF51  97 D2        2795                STAA    $D2
FF53  7F 00 42     2796                FCB     $7F,$00,$42 ; CLR $0042
FF56  96 11        2797                LDAA    CRTBASE
FF58  85 08        2798                BITA    #$08
FF5A  27 05        2799                BEQ     LFF61
FF5C  CE 80 00     2800                LDX     #$8000
FF5F  20 03        2801                BRA     LFF64
FF61  CE 88 00     2802  LFF61         LDX     #$8800
FF64  DF D3        2803  LFF64         STX     $D3
FF66  DF D7        2804                STX     $D7
FF68  DF D9        2805                STX     $D9
FF6A  96 D3        2806                LDAA    $D3
FF6C  8B 07        2807                ADDA    #$07
FF6E  97 D5        2808                STAA    $D5
FF70  86 CF        2809                LDAA    #$CF
FF72  97 D6        2810                STAA    $D6
FF74  7E FD E2     2811                JMP     LFDE2
FF77  97 D2        2812  LFF77         STAA    $D2
FF79  39           2813                RTS
FF7A  7D 00 D2     2814  LFF7A         FCB     $7D,$00,$D2 ; TST $00D2
FF7D  27 02        2815                BEQ     LFF81
FF7F  20 07        2816                BRA     LFF88
FF81  39           2817  LFF81         RTS
FF82  84 7F        2818  LFF82         ANDA    #$7F
FF84  81 14        2819                CMPA    #$14
FF86  26 06        2820                BNE     LFF8E
```

```
FF88  86 80         2821  LFF88           LDAA    #$80
FF8A  97 D2         2822                  STAA    $D2
FF8C  20 11         2823                  BRA     LFF9F
FF8E  DE D7         2824  LFF8E           LDX     $D7
FF90  B7 70 00      2825                  STAA    HALT
FF93  01            2826                  NOP
FF94  A7 00         2827                  STAA    $00,X
FF96  08            2828                  INX
FF97  9C D5         2829                  CPX     $D5
FF99  26 02         2830                  BNE     LFF9D
FF9B  DE D3         2831                  LDX     $D3
FF9D  DF D7         2832  LFF9D           STX     $D7
FF9F  D6 C0         2833  LFF9F           LDAB    $C0
FFA1  C4 9F         2834                  ANDB    #$9F
FFA3  CA 20         2835                  ORAB    #$20
FFA5  D7 C0         2836                  STAB    $C0
FFA7  F7 7C 00      2837                  STAB    PACIA
FFAA  39            2838                  RTS
FFAB  DE D9         2839  LFFAB           LDX     $D9
FFAD  9C D7         2840                  CPX     $D7
FFAF  27 13         2841                  BEQ     LFFC4
FFB1  B7 70 00      2842                  STAA    HALT
FFB4  01            2843                  NOP
FFB5  A6 00         2844                  LDAA    $00,X
FFB7  B7 7C 01      2845                  STAA    PACIAD
FFBA  08            2846                  INX
FFBB  9C D5         2847                  CPX     $D5
FFBD  26 02         2848                  BNE     LFFC1
FFBF  DE D3         2849                  LDX     $D3
FFC1  DF D9         2850  LFFC1           STX     $D9
FFC3  39            2851                  RTS
FFC4  96 C0         2852  LFFC4           LDAA    $C0
FFC6  84 9F         2853                  ANDA    #$9F
FFC8  D6 D1         2854                  LDAB    $D1
FFCA  C1 01         2855                  CMPB    #$01
FFCC  27 04         2856                  BEQ     LFFD2
FFCE  D6 D2         2857                  LDAB    $D2
FFD0  2A 0E         2858                  BPL     LFFE0
FFD2  D6 D2         2859  LFFD2           LDAB    $D2
FFD4  C5 08         2860                  BITB    #$08
FFD6  27 0E         2861                  BEQ     LFFE6
FFD8  7F 00 D2      2862                  FCB     $7F,$00,$D2 ; CLR $00D2
FFDB  8A 40         2863                  ORAA    #$40
FFDD  7F 00 D1      2864                  FCB     $7F,$00,$D1 ; CLR $00D1
FFE0  97 C0         2865  LFFE0           STAA    $C0
FFE2  B7 7C 00      2866                  STAA    PACIA
FFE5  39            2867                  RTS
FFE6  4F            2868  LFFE6           CLRA
FFE7  B7 7C 01      2869                  STAA    PACIAD
FFEA  86 88         2870                  LDAA    #$88
FFEC  97 D2         2871                  STAA    $D2
FFEE  39            2872                  RTS
FFEF  00 00 00 00   2873                  FCB     $00,$00,$00,$00,$00,$00
FFF5  00 00 72      2874                  FCB     $00,$00,$72
FFF8  ED 83 E8 00   2875                  FDB     IRQ,RESET,RESET,RESET
0000                2876
0000                2877                  END
```

```
SYMBOL TABLE
  HALT     7000  PIA1A    7800  PIA1CRA  7801  PIA1B    7802  PIA1CRB  7803
  SWBNK2   7900  ACIA     7A00  ACIAD    7A01  PERROM   7B00  PACIA    7C00
  PACIAD   7C01  SWBNK3   7D00  SWBNK1   7E00  BAUDSW   7E80  VTAC0    7F00
  VTAC1    7F01  VTAC2    7F02  VTAC3    7F03  VTAC4    7F04  VTAC5    7F05
  VTAC6    7F06  VTAC7    7F07  VTAC8    7F08  VTAC9    7F09  VTACA    7F0A
  VTACB    7F0B  VTACC    7F0C  VTACD    7F0D  VTACE    7F0E  PLUGFLG  0000
  CRTBASE  0011  CURADR   0013  FIRSTLN  0015  ROW      0016  COL      0017
  PROTECT  0018  ATTRMSK  0019  MONMODE  001A  KBLOCK   001D  SPLIT    0042
  ESCCHR   0045  BLANK    004D  CURTASK  009D  ACIACTL  00BF  TXCOUNT  00CB
  RESET    E800  LE827    E827  LE82F    E82F  LE84C    E84C  LE854    E854
  LE861    E861  LE86A    E86A  LE874    E874  LE881    E881  LE88C    E88C
  LE899    E899  LE8A3    E8A3  LE8AB    E8AB  ROMSUM   E8B5  PLUGPROB E8BB
  NOPLUG   E8D0  PIAINIT  E8D4  LE94F    E94F  LE958    E958  LE95B    E95B
  LE970    E970  LE9A6    E9A6  TSTMSG   E9B0  ERRMSG   E9BD  LE9CA    E9CA
  LE9D5    E9D5  LE9DA    E9DA  LE9EA    E9EA  INITTAB  E9F8  LEA18    EA18
  KBDTAB   EA1A  LEAC6    EAC6  ESCTAB   EACA  CTLTAB   EBAA  ESCNOP   EBEA
  LEBF8    EBF8  LEC32    EC32  LEC35    EC35  LEC4C    EC4C  SCHED    EC53
  LEC55    EC55  LEC59    EC59  LEC76    EC76  LEC84    EC84  LEC8C    EC8C
  LECAA    ECAA  LECAE    ECAE  LECB0    ECB0  LECB8    ECB8  WAKE     ECBA
  BLOCKME  ECC1  BLOCKX   ECC3  SLEEP    ECCC  LECD8    ECD8  GETRX    ECE8
  LECF0    ECF0  LECFE    ECFE  LED05    ED05  LED1B    ED1B  LED2D    ED2D
  LED2F    ED2F  LED32    ED32  LED42    ED42  LED52    ED52  LED57    ED57
  LED58    ED58  LED78    ED78  LED7B    ED7B  IRQ      ED83  LED8A    ED8A
  LED92    ED92  ACIAIRQ  ED93  LEDA7    EDA7  LEDA9    EDA9  LEDB2    EDB2
  LEDC4    EDC4  LEDD4    EDD4  LEDD6    EDD6  LEDDF    EDDF  LEDE7    EDE7
  LEDE8    EDE8  LEDEF    EDEF  LEDFE    EDFE  LEE05    EE05  KBSCAN   EE06
  LEE08    EE08  LEE16    EE16  LEE1D    EE1D  LEE42    EE42  LEE50    EE50
  LEE60    EE60  LEE66    EE66  LEE77    EE77  LEE8F    EE8F  LEE93    EE93
  LEE9D    EE9D  KEYHAND  EEA1  LEEB8    EEB8  LEEB9    EEB9  LEED8    EED8
  LEEE2    EEE2  LEEF0    EEF0  LEF00    EF00  LEF08    EF08  LEF0B    EF0B
  LEF14    EF14  LEF2B    EF2B  ESCDISP  EF31  LEF3A    EF3A  LEF42    EF42
  LEF4A    EF4A  LEF4E    EF4E  LEF51    EF51  LEF59    EF59  LEF5B    EF5B
  LEF61    EF61  LEF6C    EF6C  SENDBRK  EF6D  LEF8B    EF8B  LEFA0    EFA0
  LEFA5    EFA5  XPLUSA   EFAD  LEFBC    EFBC  LEFC5    EFC5  LEFC6    EFC6
  LEFDA    EFDA  LEFEA    EFEA  LEFF3    EFF3  LEFF6    EFF6  LF006    F006
  LF012    F012  LF015    F015  LF021    F021  LF02F    F02F  LF03A    F03A
  LF044    F044  LF051    F051  LF060    F060  LF06D    F06D  LF070    F070
  LF074    F074  LF079    F079  LF07D    F07D  LF083    F083  LF087    F087
  LF089    F089  LF08A    F08A  LF097    F097  LF0A1    F0A1  LF0A6    F0A6
  LF0AA    F0AA  LF0AF    F0AF  LF0B3    F0B3  LF0CC    F0CC  LF0CD    F0CD
  LF0D1    F0D1  LF0D6    F0D6  BELL     F0DA  LF0E8    F0E8  LF0F3    F0F3
  LF0FD    F0FD  LF10E    F10E  LF143    F143  LF14A    F14A  LF14B    F14B
  LF14E    F14E  LF15D    F15D  LF165    F165  LF16B    F16B  LF171    F171
  LF172    F172  LF199    F199  LF1AC    F1AC  LF1C2    F1C2  LF1CD    F1CD
  LF1D0    F1D0  LF1F0    F1F0  LF1F6    F1F6  LF202    F202  LF204    F204
  LF20D    F20D  LF211    F211  LF215    F215  LF217    F217  LF221    F221
  LF22B    F22B  ESCGRAVE F22C  LF237    F237  LF239    F239  LF23C    F23C
  LF240    F240  LF24E    F24E  LF258    F258  LF259    F259  LF26A    F26A
  LF26F    F26F  LF272    F272  LF275    F275  LF279    F279  LF27D    F27D
  LF285    F285  LF28C    F28C  LF28F    F28F  LF293    F293  LF29A    F29A
  LF2A1    F2A1  LF2B0    F2B0  LF2BC    F2BC  LF2C0    F2C0  LF2CC    F2CC
  LF2CE    F2CE  LF2CF    F2CF  LF2D7    F2D7  LF2DD    F2DD  LF2E5    F2E5
  LF2F0    F2F0  LF305    F305  LF313    F313  LF317    F317  LF323    F323
  LF324    F324  LF327    F327  LF331    F331  LF334    F334  LF340    F340
  LF355    F355  LF362    F362  LF36D    F36D  LF36F    F36F  LF37F    F37F
```

```
LF388    F388  LF38E    F38E  LF3A7    F3A7  LF3AA    F3AA  LF3BA    F3BA
LF3BE    F3BE  LF3C4    F3C4  LF3D0    F3D0  LF3D6    F3D6  LF3E4    F3E4
LF3EC    F3EC  LF3F3    F3F3  LF3FA    F3FA  LF400    F400  LF40A    F40A
LF414    F414  LF41F    F41F  LF422    F422  LF43C    F43C  LF43D    F43D
LF455    F455  LF458    F458  LF469    F469  LF47C    F47C  LF483    F483
LF486    F486  LF48B    F48B  LF492    F492  LF49B    F49B  LF4C1    F4C1
LF4CD    F4CD  LF4D7    F4D7  LF4E7    F4E7  LF4E8    F4E8  LF4F2    F4F2
LF500    F500  LF501    F501  LF507    F507  LF50F    F50F  LF51B    F51B
LF528    F528  LF530    F530  LF538    F538  LF53A    F53A  LF53B    F53B
LF54E    F54E  LF552    F552  LF553    F553  LF556    F556  LF564    F564
LF566    F566  LF567    F567  LF571    F571  LF577    F577  LF57D    F57D
LF592    F592  LF5A3    F5A3  LF5A7    F5A7  CURPOS   F5AA  LF5B5    F5B5
LF5BE    F5BE  LF5CE    F5CE  LF5E2    F5E2  LF5E9    F5E9  ROWADDR  F5EA
CURADDR  F5F8  HEX2NIB  F607  LF61B    F61B  LF61F    F61F  NIB2HEX  F622
LF626    F626  LF62E    F62E  LF631    F631  LF65B    F65B  LF65E    F65E
LF660    F660  LF671    F671  LF672    F672  LF685    F685  LF6A5    F6A5
LF6A7    F6A7  BLITCPY  F6BB  LF6D5    F6D5  LF6E1    F6E1  LF6E2    F6E2
LF6EB    F6EB  LF72A    F72A  LF731    F731  LF73B    F73B  CLRSCR   F740
LF74C    F74C  LF756    F756  LF757    F757  LF762    F762  LF763    F763
LF767    F767  LF76A    F76A  LF76E    F76E  LF785    F785  LF78A    F78A
LF790    F790  LF793    F793  LF7AF    F7AF  LF7BB    F7BB  LF7C1    F7C1
LF7CC    F7CC  LF7D2    F7D2  CLREOL   F7D5  LF7DF    F7DF  LF7E2    F7E2
LF7EB    F7EB  LF7F5    F7F5  LF801    F801  LF80D    F80D  LF81C    F81C
LF81D    F81D  LF820    F820  LF824    F824  LF827    F827  LF831    F831
LF848    F848  LF84B    F84B  LF855    F855  LF86B    F86B  INSLINE  F874
LF87A    F87A  LF899    F899  LF8A9    F8A9  LF8C5    F8C5  LF8C7    F8C7
LF8DE    F8DE  LF8F2    F8F2  LF90D    F90D  DELLINE  F912  LF918    F918
LF920    F920  LF93E    F93E  LF94C    F94C  LF960    F960  LF990    F990
LF9A4    F9A4  LF9AA    F9AA  LF9B5    F9B5  LF9B9    F9B9  LF9BD    F9BD
LF9C1    F9C1  LF9C5    F9C5  LF9C9    F9C9  LF9CD    F9CD  LF9D1    F9D1
LF9D5    F9D5  LF9D9    F9D9  LF9DA    F9DA  LF9F5    F9F5  LFA01    FA01
LFA10    FA10  LFA26    FA26  JMPTAB   FA32  LFA48    FA48  LFA49    FA49
LFA52    FA52  LFA5F    FA5F  LFA62    FA62  LFA6B    FA6B  LFA77    FA77
LFA82    FA82  LFA97    FA97  LFAA4    FAA4  LFAA5    FAA5  LFAAD    FAAD
LFAC1    FAC1  LFAC4    FAC4  LFACA    FACA  LFADD    FADD  LFAE7    FAE7
LFAED    FAED  LFAFF    FAFF  LFB00    FB00  LFB0A    FB0A  LFB18    FB18
LFB24    FB24  LFB25    FB25  LFB26    FB26  LFB2F    FB2F  LFB45    FB45
LFB48    FB48  LFB4C    FB4C  LFB57    FB57  LFB6D    FB6D  LFB72    FB72
LFB73    FB73  LFB89    FB89  LFB91    FB91  LFB93    FB93  LFB99    FB99
LFBA5    FBA5  LFBB0    FBB0  LFBB4    FBB4  LFBB6    FBB6  LFBCF    FBCF
LFBED    FBED  LFC02    FC02  LFC05    FC05  LFC0B    FC0B  LFC2F    FC2F
LFC43    FC43  LFC52    FC52  LFC63    FC63  LFC6B    FC6B  LFC80    FC80
LFC86    FC86  LFC99    FC99  LFCA5    FCA5  LFCAA    FCAA  LFCAD    FCAD
LFCB1    FCB1  LFCC6    FCC6  LFCD9    FCD9  LFCDA    FCDA  LFCDD    FCDD
LFCF5    FCF5  LFCFD    FCFD  LFD19    FD19  TXDELAY  FD27  LFD2A    FD2A
LFD35    FD35  LFD4D    FD4D  LFD6A    FD6A  BAUDTAB  FD75  LFD85    FD85
LFDA1    FDA1  LFDA4    FDA4  LFDAF    FDAF  LFDD9    FDD9  LFDE2    FDE2
LFE00    FE00  LFE03    FE03  LFE0B    FE0B  LFE19    FE19  LFE1F    FE1F
LFE28    FE28  LFE34    FE34  LFE36    FE36  LFE45    FE45  LFE57    FE57
LFE7F    FE7F  LFE81    FE81  LFE88    FE88  LFE8C    FE8C  LFE8F    FE8F
LFEA4    FEA4  LFEB8    FEB8  LFECB    FECB  LFECE    FECE  LFEF1    FEF1
LFEF7    FEF7  LFEFE    FEFE  LFF18    FF18  LFF1B    FF1B  LFF25    FF25
LFF36    FF36  LFF39    FF39  LFF40    FF40  LFF61    FF61  LFF64    FF64
LFF77    FF77  LFF7A    FF7A  LFF81    FF81  LFF82    FF82  LFF88    FF88
LFF8E    FF8E  LFF9D    FF9D  LFF9F    FF9F  LFFAB    FFAB  LFFC1    FFC1
LFFC4    FFC4  LFFD2    FFD2  LFFE0    FFE0  LFFE6    FFE6
```

```
CROSS-REFERENCE TABLE
SYMBOL          TYP  VAL   DEF     CNT  REFERENCES
--------------  ---  ----  -----  ----  ------------------------------------
ACIA            EQU  7A00  L15      10  L180  L187  L572  L814  L818  L2508
                                        L2534 L2545 L2560 L2593
ACIACTL         EQU  00BF  L53      12  L186  L625  L812  L817  L2504 L2507
                                        L2530 L2533 L2556 L2559 L2590 L2592
ACIAD           EQU  7A01  L16       3  L581  L632  L2582
ACIAIRQ         LAB  ED93  L579      1  L574
ATTRMSK         EQU  0019  L46       4  L905  L928  L1759 L1891
BAUDSW          EQU  7E80  L22       1  L2551
BAUDTAB         LAB  FD75  L2573     1  L2550
BELL            LAB  F0DA  L980      6  L357  L563  L622  L850  L1032 L2630
BLANK           EQU  004D  L51       7  L1752 L1817 L1885 L1918 L1956 L2039
                                        L2118
BLITCPY         LAB  F6BB  L1733     9  L227  L270  L1706 L1723 L1746 L2001
                                        L2035 L2060 L2115
BLOCKME         LAB  ECC1  L467      8  L449  L550  L709  L819  L2180 L2565
                                        L2640 L2657
BLOCKX          LAB  ECC3  L469      4  L2152 L2154 L2627 L2629
CLREOL          LAB  F7D5  L1881     1  L344
CLRSCR          LAB  F740  L1807     1  L333
COL             EQU  0017  L44      36  L1219 L1229 L1232 L1238 L1275 L1296
                                        L1300 L1307 L1327 L1355 L1359 L1362
                                        L1364 L1376 L1387 L1396 L1399 L1406
                                        L1413 L1425 L1441 L1476 L1479 L1487
                                        L1526 L1532 L1604 L1634 L1728 L1776
                                        L1785 L1893 L1923 L2206 L2349 L2755
CRTBASE         EQU  0011  L40      19  L200  L1453 L1462 L1635 L1698 L1713
                                        L1847 L1855 L1876 L1997 L2010 L2026
                                        L2052 L2057 L2065 L2083 L2430 L2448
                                        L2797
CTLTAB          LAB  EBAA  L356      1  L898
CURADDR         LAB  F5F8  L1632     7  L1226 L1231 L1323 L1408 L1605 L1730
                                        L1879
CURADR          EQU  0013  L41      40  L201  L268  L906  L1176 L1239 L1241
                                        L1288 L1301 L1303 L1328 L1356 L1385
                                        L1392 L1397 L1414 L1424 L1439 L1636
                                        L1638 L1696 L1770 L1792 L1860 L1877
                                        L1895 L1908 L1921 L1938 L1941 L1967
                                        L2258 L2264 L2294 L2315 L2456 L2488
                                        L2608 L2711 L2721 L2732
CURPOS          LAB  F5AA  L1586     5  L338  L1568 L1570 L1579 L1583
CURTASK         EQU  009D  L52      10  L395  L410  L436  L457  L467  L475
                                        L492  L496  L505  L762
DELLINE         LAB  F912  L2042     1  L343
ERRMSG          LAB  E9BD  L265      1  L248
ESCCHR          EQU  0045  L50       7  L733  L741  L873  L1154 L2367 L2379
                                        L2391
ESCDISP         LAB  EF31  L781      4  L727  L731  L857  L888
ESCGRAVE        LAB  F22C  L1142     1  L347
ESCNOP          LAB  EBEA  L364     26  L327  L328  L329  L330  L331  L332
                                        L338  L340  L341  L342  L344  L345
                                        L346  L347  L348  L349  L350  L353
                                        L354  L356  L357  L359  L360  L361
                                        L362  L363
ESCTAB          LAB  EACA  L327      1  L784
```

```
FIRSTLN         EQU  0015  L42      19  L85   L202  L1220 L1253 L1272 L1291
                                        L1315 L1320 L1443 L1591 L1724 L1852
                                        L1995 L2002 L2048 L2201 L2428 L2443
                                        L2759
GETRX           LAB  ECE8  L492     17  L772  L854  L858  L939  L986  L995
                                        L1001 L1006 L1013 L1033 L1150 L1153
                                        L1539 L1549 L1586 L1598 L2787
HALT            EQU  7000  L9       33  L907  L1210 L1282 L1339 L1347 L1435
                                        L1675 L1734 L1740 L1863 L1867 L1898
                                        L1902 L1909 L1926 L1931 L1945 L1950
                                        L1957 L2015 L2018 L2070 L2073 L2119
                                        L2316 L2414 L2460 L2610 L2723 L2733
                                        L2775 L2825 L2842
HEX2NIB         LAB  F607  L1641     4  L987  L1002 L1007 L1014
INITTAB         LAB  E9F8  L286      1  L221
INSLINE         LAB  F874  L1962     1  L340
IRQ             LAB  ED83  L572      1  L2875
JMPTAB          LAB  FA32  L2181     1  L2163
KBDTAB          LAB  EA1A  L295      1  L651
KBLOCK          EQU  001D  L48       2  L715  L956
KBSCAN          LAB  EE06  L635      1  L376
KEYHAND         LAB  EEA1  L708      1  L382
LE827           LAB  E827  L78       1  L72
LE82F           LAB  E82F  L82       1  L77
LE84C           LAB  E84C  L95       1  L97
LE854           LAB  E854  L99       1  L108
LE861           LAB  E861  L106      1  L114
LE86A           LAB  E86A  L111      2  L101  L105
LE874           LAB  E874  L115      1  L110
LE881           LAB  E881  L121      1  L124
LE88C           LAB  E88C  L126      1  L136
LE899           LAB  E899  L133      1  L141
LE8A3           LAB  E8A3  L138      2  L128  L132
LE8AB           LAB  E8AB  L142      1  L137
LE94F           LAB  E94F  L219      1  L216
LE958           LAB  E958  L223      1  L220
LE95B           LAB  E95B  L224      2  L218  L222
LE970           LAB  E970  L233      1  L231
LE9A6           LAB  E9A6  L259      1  L261
LE9CA           LAB  E9CA  L266      2  L249  L272
LE9D5           LAB  E9D5  L271      1  L247
LE9DA           LAB  E9DA  L273      1  L262
LE9EA           LAB  E9EA  L280      1  L278
LEA18           LAB  EA18  L292      1  L1074
LEAC6           LAB  EAC6  L324      1  L1070
LEBF8           LAB  EBF8  L368      1  L284
LEC32           LAB  EC32  L393      1  L381
LEC35           LAB  EC35  L394      1  L392
LEC4C           LAB  EC4C  L403      3  L371  L375  L406
LEC55           LAB  EC55  L411      1  L415
LEC59           LAB  EC59  L413      1  L458
LEC76           LAB  EC76  L425      1  L434
LEC84           LAB  EC84  L432      2  L426  L428
LEC8C           LAB  EC8C  L436      1  L412
LECAA           LAB  ECAA  L452      1  L447
LECAE           LAB  ECAE  L454      1  L445
LECB0           LAB  ECB0  L455      1  L453
```

```
LECB8           LAB  ECB8  L459      4  L529  L685  L802  L2150
LECD8           LAB  ECD8  L481      2  L479  L495
LECF0           LAB  ECF0  L496      3  L724  L876  L878
LECFE           LAB  ECFE  L505      1  L870
LED05           LAB  ED05  L509      1  L384
LED1B           LAB  ED1B  L519      1  L514
LED2D           LAB  ED2D  L528      1  L523
LED2F           LAB  ED2F  L529      2  L521  L527
LED32           LAB  ED32  L530      1  L512
LED42           LAB  ED42  L538      1  L536
LED52           LAB  ED52  L546      2  L539  L544
LED57           LAB  ED57  L548      3  L511  L518  L557
LED58           LAB  ED58  L549      1  L386
LED78           LAB  ED78  L566      3  L553  L559  L561
LED7B           LAB  ED7B  L567      1  L210
LED8A           LAB  ED8A  L575      1  L573
LED92           LAB  ED92  L578      1  L576
LEDA7           LAB  EDA7  L589      1  L586
LEDA9           LAB  EDA9  L590      1  L584
LEDB2           LAB  EDB2  L594      1  L591
LEDC4           LAB  EDC4  L602      1  L600
LEDD4           LAB  EDD4  L610      1  L608
LEDD6           LAB  EDD6  L611      1  L604
LEDDF           LAB  EDDF  L615      1  L613
LEDE7           LAB  EDE7  L619      1  L616
LEDE8           LAB  EDE8  L620      1  L595
LEDEF           LAB  EDEF  L623      1  L580
LEDFE           LAB  EDFE  L630      2  L624  L628
LEE05           LAB  EE05  L633      1  L631
LEE08           LAB  EE08  L636      1  L645
LEE16           LAB  EE16  L642      1  L640
LEE1D           LAB  EE1D  L647      1  L639
LEE42           LAB  EE42  L665      3  L659  L661  L663
LEE50           LAB  EE50  L672      4  L656  L666  L668  L670
LEE60           LAB  EE60  L680      3  L673  L675  L677
LEE66           LAB  EE66  L683      1  L701
LEE77           LAB  EE77  L690      1  L695
LEE8F           LAB  EE8F  L700      1  L698
LEE93           LAB  EE93  L702      2  L689  L705
LEE9D           LAB  EE9D  L706      1  L693
LEEB8           LAB  EEB8  L720      1  L718
LEEB9           LAB  EEB9  L721      2  L714  L716
LEED8           LAB  EED8  L737      1  L722
LEEE2           LAB  EEE2  L742      2  L738  L740
LEEF0           LAB  EEF0  L749      1  L747
LEF00           LAB  EF00  L757      2  L752  L754
LEF08           LAB  EF08  L761      3  L744  L750  L756
LEF0B           LAB  EF0B  L762      1  L779
LEF14           LAB  EF14  L767      1  L765
LEF2B           LAB  EF2B  L777      1  L775
LEF3A           LAB  EF3A  L785      1  L899
LEF42           LAB  EF42  L789      1  L783
LEF4A           LAB  EF4A  L792      1  L327
LEF4E           LAB  EF4E  L794      1  L327
LEF51           LAB  EF51  L796      1  L793
LEF59           LAB  EF59  L800      1  L797
LEF5B           LAB  EF5B  L801      1  L799
```

```
LEF61           LAB  EF61  L803      1  L329
LEF6C           LAB  EF6C  L807      1  L811
LEF8B           LAB  EF8B  L820      4  L259  L2248 L2479 L2658
LEFA0           LAB  EFA0  L829      2  L825  L827
LEFA5           LAB  EFA5  L832      2  L330  L828
LEFBC           LAB  EFBC  L845      5  L1761 L1832 L1886 L2146 L2604
LEFC5           LAB  EFC5  L849      3  L847  L861  L863
LEFC6           LAB  EFC6  L850      2  L328  L330
LEFDA           LAB  EFDA  L858      1  L853
LEFEA           LAB  EFEA  L865      1  L327
LEFF3           LAB  EFF3  L869      1  L867
LEFF6           LAB  EFF6  L870      2  L566  L761
LF006           LAB  F006  L877      1  L872
LF012           LAB  F012  L883      1  L880
LF015           LAB  F015  L884      1  L882
LF021           LAB  F021  L889      2  L874  L1157
LF02F           LAB  F02F  L896      1  L890
LF03A           LAB  F03A  L900     12  L565  L885  L892  L894  L897  L913
                                        L915  L989  L1101 L1104 L1122 L1147
LF044           LAB  F044  L905      1  L901
LF051           LAB  F051  L911      1  L887
LF060           LAB  F060  L918      1  L332
LF06D           LAB  F06D  L924      1  L922
LF070           LAB  F070  L925      1  L332
LF074           LAB  F074  L927      1  L333
LF079           LAB  F079  L930      3  L333  L1920 L1940
LF07D           LAB  F07D  L932      1  L339
LF083           LAB  F083  L935      1  L339
LF087           LAB  F087  L937      3  L934  L944  L949
LF089           LAB  F089  L938      1  L946
LF08A           LAB  F08A  L939      1  L340
LF097           LAB  F097  L945      1  L941
LF0A1           LAB  F0A1  L950      1  L344
LF0A6           LAB  F0A6  L953      3  L345  L352  L883
LF0AA           LAB  F0AA  L955      1  L331
LF0AF           LAB  F0AF  L958      1  L331
LF0B3           LAB  F0B3  L960      2  L279  L352
LF0CC           LAB  F0CC  L971      3  L961  L963  L966
LF0CD           LAB  F0CD  L972      1  L352
LF0D1           LAB  F0D1  L974      1  L351
LF0D6           LAB  F0D6  L977      1  L351
LF0E8           LAB  F0E8  L986      1  L340
LF0F3           LAB  F0F3  L990      1  L334
LF0FD           LAB  F0FD  L995      1  L992
LF10E           LAB  F10E  L1002     1  L998
LF143           LAB  F143  L1028     1  L283
LF14A           LAB  F14A  L1031     2  L994  L1029
LF14B           LAB  F14B  L1032     2  L1003 L1015
LF14E           LAB  F14E  L1033     1  L350
LF15D           LAB  F15D  L1040     1  L1038
LF165           LAB  F165  L1044     1  L1042
LF16B           LAB  F16B  L1047     1  L1045
LF171           LAB  F171  L1050     1  L1048
LF172           LAB  F172  L1051     2  L1043 L1089
LF199           LAB  F199  L1070     2  L1046 L1091
LF1AC           LAB  F1AC  L1079     2  L1049 L1093
LF1C2           LAB  F1C2  L1089     1  L1041
```

```
LF1CD           LAB  F1CD  L1095    12  L1055 L1059 L1060 L1064 L1068 L1073
                                        L1077 L1081 L1084 L1087 L1090 L1092
LF1D0           LAB  F1D0  L1096     8  L253  L1053 L1058 L1063 L1067 L1072
                                        L1076 L1111
LF1F0           LAB  F1F0  L1113     3  L1080 L1083 L1086
LF1F6           LAB  F1F6  L1116     1  L1124
LF202           LAB  F202  L1121     1  L1118
LF204           LAB  F204  L1122     1  L1120
LF20D           LAB  F20D  L1126     1  L343
LF211           LAB  F211  L1128     2  L342  L351
LF215           LAB  F215  L1130     1  L339
LF217           LAB  F217  L1131     2  L1127 L1129
LF221           LAB  F221  L1135     2  L339  L350
LF22B           LAB  F22B  L1139     3  L1133 L1137 L1144
LF237           LAB  F237  L1146     1  L337
LF239           LAB  F239  L1147     1  L1149
LF23C           LAB  F23C  L1148     1  L337
LF240           LAB  F240  L1150     1  L350
LF24E           LAB  F24E  L1156     1  L1160
LF258           LAB  F258  L1161     1  L1155
LF259           LAB  F259  L1162     3  L363  L923  L2613
LF26A           LAB  F26A  L1170     2  L1163 L1168
LF26F           LAB  F26F  L1172     4  L1175 L1184 L1193 L1207
LF272           LAB  F272  L1173     2  L359  L910
LF275           LAB  F275  L1174     1  L1177
LF279           LAB  F279  L1176     3  L924  L1796 L1845
LF27D           LAB  F27D  L1178     1  L358
LF285           LAB  F285  L1182     1  L1179
LF28C           LAB  F28C  L1185     4  L358  L1187 L1190 L1687
LF28F           LAB  F28F  L1186     1  L2497
LF293           LAB  F293  L1188     1  L358
LF29A           LAB  F29A  L1191     3  L363  L1181 L2308
LF2A1           LAB  F2A1  L1194     1  L359
LF2B0           LAB  F2B0  L1201     2  L1196 L1199
LF2BC           LAB  F2BC  L1206     1  L1204
LF2C0           LAB  F2C0  L1208     8  L1171 L1174 L1183 L1186 L1189 L1192
                                        L1206 L1609
LF2CC           LAB  F2CC  L1214     1  L1209
LF2CE           LAB  F2CE  L1216     1  L1213
LF2CF           LAB  F2CF  L1217     5  L1170 L1367 L1846 L2244 L2614
LF2D7           LAB  F2D7  L1221     1  L1264
LF2DD           LAB  F2DD  L1224     3  L1222 L1259 L1274
LF2E5           LAB  F2E5  L1227     5  L1201 L1250 L1966 L2046 L2233
LF2F0           LAB  F2F0  L1232     6  L1105 L1116 L1173 L1426 L2311 L2768
LF305           LAB  F305  L1243     1  L1234
LF313           LAB  F313  L1250    12  L250  L1095 L1191 L1244 L1247 L1358
                                        L1361 L1383 L1409 L2314 L2476 L2773
LF317           LAB  F317  L1252     3  L1182 L1205 L1251
LF323           LAB  F323  L1258     1  L1256
LF324           LAB  F324  L1259     1  L1295
LF327           LAB  F327  L1260     1  L1254
LF331           LAB  F331  L1264     1  L1267
LF334           LAB  F334  L1265     1  L1263
LF340           LAB  F340  L1272     1  L1270
LF355           LAB  F355  L1282     1  L1287
LF362           LAB  F362  L1290     1  L1261
LF36D           LAB  F36D  L1295     1  L1293
```

```
LF36F           LAB  F36F  L1296     5  L1185 L1368 L1478 L1486 L2459
LF37F           LAB  F37F  L1305     2  L1297 L1440
LF388           LAB  F388  L1309     2  L1188 L1308
LF38E           LAB  F38E  L1312     1  L1310
LF3A7           LAB  F3A7  L1323     2  L1316 L1318
LF3AA           LAB  F3AA  L1324     2  L349  L358
LF3BA           LAB  F3BA  L1331     1  L1342
LF3BE           LAB  F3BE  L1333     1  L1330
LF3C4           LAB  F3C4  L1337     1  L1360
LF3D0           LAB  F3D0  L1343     2  L1353 L1366
LF3D6           LAB  F3D6  L1347     1  L1363
LF3E4           LAB  F3E4  L1354     4  L1332 L1338 L1350 L1430
LF3EC           LAB  F3EC  L1358     1  L1334
LF3F3           LAB  F3F3  L1361     1  L1344
LF3FA           LAB  F3FA  L1364     4  L1172 L1610 L1688 L1914
LF400           LAB  F400  L1367     1  L1352
LF40A           LAB  F40A  L1372     1  L1325
LF414           LAB  F414  L1377     1  L1382
LF41F           LAB  F41F  L1383     1  L1379
LF422           LAB  F422  L1384     2  L1381 L1485
LF43C           LAB  F43C  L1398     1  L1373
LF43D           LAB  F43D  L1399     2  L1374 L1405
LF455           LAB  F455  L1409     1  L1401
LF458           LAB  F458  L1410     1  L341
LF469           LAB  F469  L1418     2  L1423 L1429
LF47C           LAB  F47C  L1427     2  L1417 L1428
LF483           LAB  F483  L1430     2  L1419 L1421
LF486           LAB  F486  L1431     3  L1416 L1422 L1427
LF48B           LAB  F48B  L1435     1  L1442
LF492           LAB  F492  L1439     1  L1432
LF49B           LAB  F49B  L1443     2  L1326 L1412
LF4C1           LAB  F4C1  L1462     2  L1447 L1451
LF4CD           LAB  F4CD  L1468     1  L1466
LF4D7           LAB  F4D7  L1473     1  L1411
LF4E7           LAB  F4E7  L1480     1  L1477
LF4E8           LAB  F4E8  L1481     1  L1483
LF4F2           LAB  F4F2  L1486     2  L1475 L1491
LF500           LAB  F500  L1492     2  L1474 L1488
LF501           LAB  F501  L1493     2  L335  L1823
LF507           LAB  F507  L1496     1  L1499
LF50F           LAB  F50F  L1501     1  L335
LF51B           LAB  F51B  L1507     1  L335
LF528           LAB  F528  L1514     1  L1518
LF530           LAB  F530  L1519     1  L1500
LF538           LAB  F538  L1523     1  L1506
LF53A           LAB  F53A  L1524     2  L1515 L1521
LF53B           LAB  F53B  L1525     4  L1403 L1489 L1501 L1507
LF54E           LAB  F54E  L1535     1  L1537
LF552           LAB  F552  L1538     1  L1534
LF553           LAB  F553  L1539     1  L335
LF556           LAB  F556  L1540     2  L237  L241
LF564           LAB  F564  L1547     1  L1544
LF566           LAB  F566  L1548     2  L1542 L1545
LF567           LAB  F567  L1549     1  L334
LF571           LAB  F571  L1554     1  L1552
LF577           LAB  F577  L1557     1  L1555
LF57D           LAB  F57D  L1560     1  L1558
```

```
LF592           LAB  F592  L1571     1  L1562
LF5A3           LAB  F5A3  L1581     1  L1576
LF5A7           LAB  F5A7  L1584     1  L1580
LF5B5           LAB  F5B5  L1591     1  L1589
LF5BE           LAB  F5BE  L1596     1  L1594
LF5CE           LAB  F5CE  L1603     1  L1601
LF5E2           LAB  F5E2  L1611     2  L1587 L1599
LF5E9           LAB  F5E9  L1615     2  L1608 L1613
LF61B           LAB  F61B  L1651     1  L1644
LF61F           LAB  F61F  L1654     3  L1642 L1647 L1649
LF626           LAB  F626  L1662     2  L1103 L2385
LF62E           LAB  F62E  L1666     1  L1664
LF631           LAB  F631  L1668     3  L341  L1569 L1584
LF65B           LAB  F65B  L1688     2  L1681 L1686
LF65E           LAB  F65E  L1689    13  L234  L238  L1169 L1290 L1319 L1779
                                         L1781 L1840 L1842 L2243 L2427 L2442
                                         L2455
LF660           LAB  F660  L1690     1  L1674
LF671           LAB  F671  L1697     3  L1669 L1672 L1679
LF672           LAB  F672  L1698     1  L1691
LF685           LAB  F685  L1707     1  L1695
LF6A5           LAB  F6A5  L1722     1  L1719
LF6A7           LAB  F6A7  L1723     1  L1721
LF6D5           LAB  F6D5  L1748     1  L345
LF6E1           LAB  F6E1  L1754     1  L353
LF6E2           LAB  F6E2  L1755     2  L1751 L1753
LF6EB           LAB  F6EB  L1759     1  L1757
LF72A           LAB  F72A  L1794     2  L1763 L1768
LF731           LAB  F731  L1797     1  L337
LF73B           LAB  F73B  L1803     3  L337  L868  L1809
LF74C           LAB  F74C  L1813     2  L1803 L1811
LF756           LAB  F756  L1818     1  L1816
LF757           LAB  F757  L1819     2  L1810 L1827
LF762           LAB  F762  L1824     2  L1820 L1822
LF763           LAB  F763  L1825     1  L334
LF767           LAB  F767  L1827     4  L333  L869  L1035 L1799
LF76A           LAB  F76A  L1829     2  L1812 L1826
LF76E           LAB  F76E  L1831     2  L1802 L1805
LF785           LAB  F785  L1841     1  L1839
LF78A           LAB  F78A  L1843     1  L1834
LF790           LAB  F790  L1846     5  L235  L239  L1780 L1841 L1843
LF793           LAB  F793  L1847     1  L1794
LF7AF           LAB  F7AF  L1861     2  L1875 L1878
LF7BB           LAB  F7BB  L1867     1  L1862
LF7C1           LAB  F7C1  L1870     1  L1866
LF7CC           LAB  F7CC  L1876     1  L1873
LF7D2           LAB  F7D2  L1879     1  L1871
LF7DF           LAB  F7DF  L1886     2  L1884 L1917
LF7E2           LAB  F7E2  L1887     2  L1919 L2040
LF7EB           LAB  F7EB  L1891     1  L1889
LF7F5           LAB  F7F5  L1896     1  L1907
LF801           LAB  F801  L1902     1  L1897
LF80D           LAB  F80D  L1908     1  L1901
LF81C           LAB  F81C  L1915     1  L1913
LF81D           LAB  F81D  L1916     1  L352
LF820           LAB  F820  L1918     1  L1200
LF824           LAB  F824  L1920     1  L343
```

```
LF827           LAB  F827  L1921     1  L903
LF831           LAB  F831  L1926     1  L1937
LF848           LAB  F848  L1938     1  L1930
LF84B           LAB  F84B  L1940     1  L344
LF855           LAB  F855  L1944     1  L1955
LF86B           LAB  F86B  L1956     2  L1944 L1949
LF87A           LAB  F87A  L1965     1  L1963
LF899           LAB  F899  L1980     1  L1978
LF8A9           LAB  F8A9  L1989     1  L1987
LF8C5           LAB  F8C5  L2002     2  L1970 L1975
LF8C7           LAB  F8C7  L2003     2  L2025 L2038
LF8DE           LAB  F8DE  L2015     1  L2023
LF8F2           LAB  F8F2  L2026     1  L2006
LF90D           LAB  F90D  L2039     1  L2004
LF918           LAB  F918  L2045     1  L2043
LF920           LAB  F920  L2048     2  L2062 L2081
LF93E           LAB  F93E  L2063     1  L2051
LF94C           LAB  F94C  L2070     1  L2078
LF960           LAB  F960  L2082     1  L2049
LF990           LAB  F990  L2107     1  L2105
LF9A4           LAB  F9A4  L2116     2  L2089 L2094
LF9AA           LAB  F9AA  L2119     1  L2124
LF9B5           LAB  F9B5  L2126     1  L336
LF9B9           LAB  F9B9  L2128     1  L336
LF9BD           LAB  F9BD  L2130     1  L336
LF9C1           LAB  F9C1  L2132     1  L336
LF9C5           LAB  F9C5  L2134     1  L343
LF9C9           LAB  F9C9  L2136     1  L351
LF9CD           LAB  F9CD  L2138     1  L338
LF9D1           LAB  F9D1  L2140     1  L334
LF9D5           LAB  F9D5  L2142     1  L357
LF9D9           LAB  F9D9  L2144     2  L791  L864
LF9DA           LAB  F9DA  L2145    10  L2127 L2129 L2131 L2133 L2135 L2137
                                         L2139 L2141 L2143 L2639
LF9F5           LAB  F9F5  L2155     1  L388
LFA01           LAB  FA01  L2160     1  L2157
LFA10           LAB  FA10  L2167     1  L2159
LFA26           LAB  FA26  L2177     3  L2170 L2172 L2175
LFA48           LAB  FA48  L2185     0
LFA49           LAB  FA49  L2186     0
LFA52           LAB  FA52  L2190     1  L2188
LFA5F           LAB  FA5F  L2198     1  L2196
LFA62           LAB  FA62  L2199     0
LFA6B           LAB  FA6B  L2204     1  L2202
LFA77           LAB  FA77  L2209     1  L2158
LFA82           LAB  FA82  L2214     0
LFA97           LAB  FA97  L2224     1  L2230
LFAA4           LAB  FAA4  L2231     2  L2218 L2221
LFAA5           LAB  FAA5  L2232     0
LFAAD           LAB  FAAD  L2235     0
LFAC1           LAB  FAC1  L2244     2  L2237 L2242
LFAC4           LAB  FAC4  L2245     1  L2234
LFACA           LAB  FACA  L2248     2  L2269 L2271
LFADD           LAB  FADD  L2256     1  L2253
LFAE7           LAB  FAE7  L2261     1  L2257
LFAED           LAB  FAED  L2264     1  L2262
LFAFF           LAB  FAFF  L2272     4  L2250 L2255 L2260 L2266
```

```
LFB00           LAB  FB00  L2273     3  L2270 L2351 L2494
LFB0A           LAB  FB0A  L2278     1  L2275
LFB18           LAB  FB18  L2284     2  L2338 L2357
LFB24           LAB  FB24  L2290     2  L2286 L2288
LFB25           LAB  FB25  L2291     3  L2277 L2279 L2282
LFB26           LAB  FB26  L2292     3  L2232 L2235 L2402
LFB2F           LAB  FB2F  L2297     1  L2177
LFB45           LAB  FB45  L2308     1  L2304
LFB48           LAB  FB48  L2309     4  L2298 L2301 L2303 L2307
LFB4C           LAB  FB4C  L2311     4  L2267 L2348 L2474 L2491
LFB57           LAB  FB57  L2315     4  L2251 L2350 L2352 L2482
LFB6D           LAB  FB6D  L2326     6  L2324 L2339 L2343 L2365 L2373 L2398
LFB72           LAB  FB72  L2328     2  L2313 L2336
LFB73           LAB  FB73  L2329     1  L2320
LFB89           LAB  FB89  L2340     1  L2334
LFB91           LAB  FB91  L2344     1  L2354
LFB93           LAB  FB93  L2345     1  L2359
LFB99           LAB  FB99  L2348     1  L2387
LFBA5           LAB  FBA5  L2353     1  L2341
LFBB0           LAB  FBB0  L2358     1  L2356
LFBB4           LAB  FBB4  L2360     1  L2347
LFBB6           LAB  FBB6  L2362     1  L2331
LFBCF           LAB  FBCF  L2374     1  L2363
LFBED           LAB  FBED  L2388     1  L2377
LFC02           LAB  FC02  L2398     2  L2389 L2401
LFC05           LAB  FC05  L2399     0
LFC0B           LAB  FC0B  L2402     0
LFC2F           LAB  FC2F  L2418     1  L2410
LFC43           LAB  FC43  L2428     2  L2423 L2426
LFC52           LAB  FC52  L2435     1  L2417
LFC63           LAB  FC63  L2443     2  L2436 L2441
LFC6B           LAB  FC6B  L2447     1  L2445
LFC80           LAB  FC80  L2456     1  L2454
LFC86           LAB  FC86  L2459     1  L2466
LFC99           LAB  FC99  L2468     1  L2464
LFCA5           LAB  FCA5  L2474     1  L2470
LFCAA           LAB  FCAA  L2476     1  L2473
LFCAD           LAB  FCAD  L2477     3  L2458 L2467 L2475
LFCB1           LAB  FCB1  L2479     2  L2493 L2495
LFCC6           LAB  FCC6  L2488     1  L2484
LFCD9           LAB  FCD9  L2496     2  L2481 L2486
LFCDA           LAB  FCDA  L2497     1  L2490
LFCDD           LAB  FCDD  L2498     2  L2155 L2514
LFCF5           LAB  FCF5  L2511    24  L734  L736  L745  L776  L2173 L2176
                                         L2198 L2205 L2208 L2210 L2213 L2228
                                         L2263 L2280 L2283 L2289 L2368 L2370
                                         L2380 L2382 L2386 L2392 L2394 L2487
LFCFD           LAB  FCFD  L2515     2  L2513 L2517
LFD19           LAB  FD19  L2529     1  L2527
LFD2A           LAB  FD2A  L2540     2  L2544 L2547
LFD35           LAB  FD35  L2545     1  L2541
LFD4D           LAB  FD4D  L2555     1  L2549
LFD6A           LAB  FD6A  L2566     1  L211
LFD85           LAB  FD85  L2576     1  L629
LFDA1           LAB  FDA1  L2588     1  L2586
LFDA4           LAB  FDA4  L2590     1  L2577
LFDAF           LAB  FDAF  L2596     1  L1134
```

```
LFDD9           LAB  FDD9  L2615     1  L2607
LFDE2           LAB  FDE2  L2619     1  L2811
LFE00           LAB  FE00  L2630     2  L2598 L2600
LFE03           LAB  FE03  L2631     1  L390
LFE0B           LAB  FE0B  L2635     1  L2633
LFE19           LAB  FE19  L2641     0
LFE1F           LAB  FE1F  L2643     1  L2651
LFE28           LAB  FE28  L2647     1  L2645
LFE34           LAB  FE34  L2652     0
LFE36           LAB  FE36  L2653     2  L2637 L2648
LFE45           LAB  FE45  L2658     2  L2631 L2646
LFE57           LAB  FE57  L2666     1  L2662
LFE7F           LAB  FE7F  L2692     2  L2670 L2680
LFE81           LAB  FE81  L2693     2  L2697 L2699
LFE88           LAB  FE88  L2696     2  L2674 L2684
LFE8C           LAB  FE8C  L2698     5  L2652 L2672 L2682 L2686 L2710
LFE8F           LAB  FE8F  L2700     3  L2688 L2762 L2782
LFEA4           LAB  FEA4  L2710     1  L2676
LFEB8           LAB  FEB8  L2720     2  L2727 L2729
LFECB           LAB  FECB  L2730     1  L2722
LFECE           LAB  FECE  L2732     3  L2664 L2667 L2678
LFEF1           LAB  FEF1  L2749     2  L2744 L2747
LFEF7           LAB  FEF7  L2752     2  L2742 L2750
LFEFE           LAB  FEFE  L2755     1  L2740
LFF18           LAB  FF18  L2768     2  L2757 L2760
LFF1B           LAB  FF1B  L2769     1  L2754
LFF25           LAB  FF25  L2774     1  L2738
LFF36           LAB  FF36  L2782     2  L2767 L2779
LFF39           LAB  FF39  L2783     1  L2781
LFF40           LAB  FF40  L2787     1  L1138
LFF61           LAB  FF61  L2802     1  L2799
LFF64           LAB  FF64  L2803     1  L2801
LFF77           LAB  FF77  L2812     1  L2794
LFF7A           LAB  FF7A  L2814     1  L2789
LFF81           LAB  FF81  L2817     2  L2791 L2815
LFF82           LAB  FF82  L2818     1  L515
LFF88           LAB  FF88  L2821     1  L2816
LFF8E           LAB  FF8E  L2824     1  L2820
LFF9D           LAB  FF9D  L2832     1  L2830
LFF9F           LAB  FF9F  L2833     1  L2823
LFFAB           LAB  FFAB  L2839     1  L577
LFFC1           LAB  FFC1  L2850     1  L2848
LFFC4           LAB  FFC4  L2852     1  L2841
LFFD2           LAB  FFD2  L2859     1  L2856
LFFE0           LAB  FFE0  L2865     1  L2858
LFFE6           LAB  FFE6  L2868     1  L2861
MONMODE         EQU  001A  L47       1  L951
NIB2HEX         LAB  F622  L1658     1  L1100
NOPLUG          LAB  E8D0  L163      2  L156  L158
PACIA           EQU  7C00  L18       8  L181  L197  L575  L2623 L2659 L2708
                                        L2837 L2866
PACIAD          EQU  7C01  L19       4  L2694 L2752 L2845 L2869
PERROM          EQU  7B00  L17       0
PIA1A           EQU  7800  L10       8  L637  L638  L692  L704  L820  L823
                                        L824  L830
PIA1B           EQU  7802  L12      25  L276  L417  L647  L804  L806  L826
                                        L967  L1165 L1449 L1565 L1573 L1683
```

```
                                        L1692 L1694 L1709 L1765 L1836 L1971
                                        L2090 L2186 L2238 L2405 L2438 L2451
                                        L2763
PIA1CRA         EQU  7801  L11       2  L421  L423
PIA1CRB         EQU  7803  L13       3  L980  L982  L984
PIAINIT         LAB  E8D4  L166      1  L161
PLUGFLG         EQU  0000  L39       3  L219  L380  L1028
PLUGPROB        LAB  E8BB  L154      0
PROTECT         EQU  0018  L45      14  L919  L1178 L1194 L1208 L1243 L1678
                                        L1861 L1896 L1912 L1929 L1948 L2276
                                        L2297 L2743
RESET           LAB  E800  L60       3  L147  L1039 L2875
ROMSUM          LAB  E8B5  L148      1  L150
ROW             EQU  0016  L43      16  L1225 L1252 L1309 L1314 L1322 L1597
                                        L1632 L1726 L1774 L1788 L2003 L2047
                                        L2199 L2292 L2471 L2758
ROWADDR         LAB  F5EA  L1617    11  L1468 L1633 L1853 L1991 L1996 L2009
                                        L2064 L2082 L2108 L2429 L2447
SCHED           LAB  EC53  L410      3  L402  L419  L435
SENDBRK         LAB  EF6D  L808      1  L330
SLEEP           LAB  ECCC  L475      7  L687  L691  L703  L816  L2543 L2555
                                        L2650
SPLIT           EQU  0042  L49      15  L970  L1162 L1446 L1561 L1680 L1762
                                        L1833 L1969 L2088 L2187 L2189 L2236
                                        L2422 L2435 L2761
SWBNK1          EQU  7E00  L21       6  L70   L182  L280  L809  L1079 L2502
SWBNK2          EQU  7900  L14       5  L193  L1082 L1131 L1135 L2619
SWBNK3          EQU  7D00  L20       2  L157  L1085
TSTMSG          LAB  E9B0  L264      1  L271
TXCOUNT         EQU  00CB  L54       2  L2515 L2540
TXDELAY         LAB  FD27  L2539     1  L378
VTAC0           EQU  7F00  L23       0
VTAC1           EQU  7F01  L24       0
VTAC2           EQU  7F02  L25       0
VTAC3           EQU  7F03  L26       0
VTAC4           EQU  7F04  L27       0
VTAC5           EQU  7F05  L28       0
VTAC6           EQU  7F06  L29       2  L1273 L1725
VTAC7           EQU  7F07  L30       0
VTAC8           EQU  7F08  L31       0
VTAC9           EQU  7F09  L32       1  L1943
VTACA           EQU  7F0A  L33       1  L62
VTACB           EQU  7F0B  L34       0
VTACC           EQU  7F0C  L35      11  L1218 L1228 L1237 L1299 L1306 L1354
                                        L1395 L1407 L1603 L1729 L1784
VTACD           EQU  7F0D  L36       6  L1224 L1313 L1321 L1596 L1727 L1787
VTACE           EQU  7F0E  L37       1  L61
WAKE            LAB  ECBA  L462     10  L398  L400  L759  L2179 L2562 L2564
                                        L2625 L2642 L2654 L2656
XPLUSA          LAB  EFAD  L837      5  L652  L786  L1530 L2165 L2553

================================================================
ASSEMBLY SUMMARY
================================================================
TOTAL ERRORS: 0
STATUS: ASSEMBLY SUCCESSFUL - NO ERRORS
================================================================
```

Appendix G

Two plug-ins by other programmers

These are listings of two plug-in ROMs that run on real ADM-31 terminals: **Rubout** and **Shuttle Attack**. Neither was written by the author of this book (he did write Outbreak, a variant of Rubout, which is not reproduced here). He disassembled the ROM images and reassembled them byte for byte with the AcPx assembler, and the listings are printed as that assembler produced them. They are the evidence behind every [PGM] tag in this book. Where the book says that a program "known to run" uses an address, it is these listings (and those of Space Invaders, Cosmic Intruders and Outbreak, which are not reproduced here) that were read. Chapter 24 explains how to read them.

Rubout

```
0000                     2  ; rubout.bin -- "Rubout" v1.0, a Breakout-style game>
0000                     3  ; ADM-31 plugin ROM ($E000-$E7FF), MD5 d3a4f2b754242>
0000                     4  ;
0000                     5  ; VERIFIED BYTE-EXACT: reassembled through acp6800, >
0000                     6  ; original (0 errors, matching MD5).
0000                     7  ;
0000                     8  ; Embedded strings confirm the game: paddle controll>
0000                     9  ; right, 9 balls (lives), bricks worth 7/5/3 points >
0000                    10  ; '='/'%'/'#'), new walls generated when few bricks >
0000                    11  ;
0000                    12  ; ============================ COLLISION SYSTEM ===>
0000                    13  ; This is the part relevant for reuse -- the ball/pa>
0000                    14  ; logic, in BALLMOV:
0000                    15  ;
0000                    16  ;   BALLCOL   - advances the ball's column ($17), bo>
0000                    17  ;               right screen edge (bounds-check agai>
0000                    18  ;   BALLROW   - advances the ball's row (a 0-26 coun>
0000                    19  ;   CALCADR   - converts (row, col) into a CRTRAM sc>
0000                    20  ;               width + col, via a shift/add sequenc>
0000                    21  ;   BALLMOV   - the actual collision check: computes>
0000                    22  ;               screen address, HALT-strobe reads wh>
0000                    23  ;               already there, and branches on its v>
0000                    24  ;                 - blank ($00)        -> nothing th>
0000                    25  ;                 - '-' ($2D)          -> paddle hit>
0000                    26  ;                                         row-direct>
0000                    27  ;                                         with a sec>
0000                    28  ;                                         (column di>
```

```
0000                    29  ;                                          which way >
0000                    30  ;                  - anything else      -> BALLCHK, t>
0000                    31  ;                                          brick hit:>
0000                    32  ;                                          old positi>
0000                    33  ;                                          new one (">
0000                    34  ;   BALLCHK   - checks the row counter against 22; p>
0000                    35  ;               has reached the bottom (a miss) -- I>
0000                    36  ;               return address on the stack (unwinds>
0000                    37  ;               per-frame call chain) before looping>
0000                    38  ;   BEEP      - PIA1 CRB bit-3 toggle, the collision>
0000                    39  ;
0000                    40  ; Ball position state: row/col counters at $16/$17, >
0000                    41  ; $F4 (row)/$F5 (col), current/previous screen-addre>
0000                    42  ; $F0/$F2 (used to erase the ball's previous positio>
0000                    43  ;
0000                    44  ; NOTE ON TRACING: BALLMOV and its neighbors are rea>
0000                    45  ; confirmed by direct byte-pattern search for JSR/JM>
0000                    46  ; regions the control-flow tracer didn't initially r>
0000                    47  ; ROM doesn't call them via the fixed $E03C/$E002 en>
0000                    48  ; other ADM-31 game ROMs in this project use) -- but>
0000                    49  ; down the exact external trigger that first invokes>
0000                    50  ; here is still byte-exact regardless: untraced-vs-t>
0000                    51  ; whether a region got mnemonics or raw FCB bytes, n>
0000                    52  ;
0000                    53  ; ================================================>
0000                    54
E000                    55          ORG     $E000
E000  54 4A             56          FCB     $54,$4A
E002                    57  BALLROW
E002  7D 00 F4          58          FCB     $7D,$00,$F4
E005  2B 02             59          BMI     BALLROW_007
E007  20 0C             60          BRA     BALLROW_013
E009                    61  BALLROW_007
E009  96 16             62          LDAA    $16
E00B  4C                63          INCA
E00C  97 16             64          STAA    $16
E00E  81 1A             65          CMPA    #$1A
E010  2F 02             66          BLE     BALLROW_012
E012  20 6C             67          BRA     RESET_044
E014                    68  BALLROW_012
E014  39                69          RTS
E015                    70  BALLROW_013
E015  96 16             71          LDAA    $16
E017  4A                72          DECA
E018  97 16             73          STAA    $16
E01A  81 00             74          CMPA    #$00
E01C  2E 06             75          BGT     BALLROW_022
E01E  7C 00 FD 73       76          FCB     $7C,$00,$FD,$73,$00,$F4
E024                    77  BALLROW_022
E024  39                78          RTS
E025                    79  DELAY1
E025  97 FC             80          STAA    $FC
E027  43                81          COMA
E028                    82  DELAY1_003
E028  4A                83          DECA
E029  91 FC             84          CMPA    $FC
E02B  26 FB             85          BNE     DELAY1_003
```

```
E02D  39             86          RTS
E02E                 87  DELAY2
E02E  97 FC          88          STAA    $FC
E030                 89  DELAY2_002
E030  4A             90          DECA
E031  91 FC          91          CMPA    $FC
E033  26 FB          92          BNE     DELAY2_002
E035  39             93          RTS
E036                 94  DELAY2_008
E036  DF F0          95          STX     $F0
E038  7F 00 00       96          FCB     $7F,$00,$00
E03B  39             97          RTS
E03C                 98  RESET
E03C  0F             99          SEI
E03D  CE 8F FF      100          LDX     #$8FFF
E040                101  RESET_004
E040  6F 00         102          CLR     $00,X
E042  09            103          DEX
E043  8C 7F FF      104          CPX     #$7FFF
E046  26 F8         105          BNE     RESET_004
E048  8D 03         106          BSR     RESET_011
E04A  7E E7 F2      107          JMP     BALLCOL_03C
E04D                108  RESET_011
E04D  86 17         109          LDAA    #$17
E04F  B7 7F 06      110          STAA    $7F06
E052  43            111          COMA
E053  B7 7F 0C      112          STAA    $7F0C
E056  B6 78 02      113          LDAA    $7802
E059  84 FB         114          ANDA    #$FB
E05B  B7 78 02      115          STAA    $7802
E05E  CE 00 00      116          LDX     #$0000
E061  86 80         117          LDAA    #$80
E063  A7 11         118          STAA    $11,X
E065  A7 F4         119          STAA    $F4,X
E067  A7 F5         120          STAA    $F5,X
E069  86 09         121          LDAA    #$09
E06B  A7 FE         122          STAA    $FE,X
E06D  6F 17         123          CLR     $17,X
E06F  6F FD         124          CLR     $FD,X
E071  86 08         125          LDAA    #$08
E073  A7 16         126          STAA    $16,X
E075  CE 86 E0      127          LDX     #$86E0
E078  DF F6         128          STX     $F6
E07A  CE 82 80      129          LDX     #$8280
E07D  20 B7         130          BRA     DELAY2_008
E07F  39            131          RTS
E080                132  RESET_044
E080  86 08         133          LDAA    #$08
E082  97 16         134          STAA    $16
E084  7A 00 FE      135          FCB     $7A,$00,$FE
E087  96 FE         136          LDAA    $FE
E089  4D            137          TSTA
E08A  2A 03         138          BPL     RESET_053
E08C  7E E2 6F      139          JMP     BALLCHK_069
E08F                140  RESET_053
E08F  8A 30         141          ORAA    #$30
E091  B7 87 7A      142          STAA    $877A
```

```
E094                 143  RESET_058
E094  86 4B          144          LDAA    #$4B
E096  B7 78 00       145          STAA    $7800
E099  B6 78 00       146          LDAA    $7800
E09C  2A 05          147          BPL     RESET_067
E09E  8D 0D          148          BSR     RESET_071
E0A0  8D 0B          149          BSR     RESET_071
E0A2  39             150          RTS
E0A3                 151  RESET_067
E0A3  8D 08          152          BSR     RESET_071
E0A5  8D 11          153          BSR     RESET_07C
E0A7  8D 04          154          BSR     RESET_071
E0A9  8D 3F          155          BSR     RESET_0AE
E0AB  20 E7          156          BRA     RESET_058
E0AD                 157  RESET_071
E0AD  DF FA          158          STX     $FA
E0AF  CE 13 00       159          LDX     #$1300
E0B2                 160  RESET_076
E0B2  09             161          DEX
E0B3  26 FD          162          BNE     RESET_076
E0B5  DE FA          163          LDX     $FA
E0B7  39             164          RTS
E0B8                 165  RESET_07C
E0B8  86 03          166          LDAA    #$03
E0BA  97 F8          167          STAA    $F8
E0BC                 168  RESET_080
E0BC  86 55          169          LDAA    #$55
E0BE  B7 78 00       170          STAA    $7800
E0C1  B6 78 00       171          LDAA    $7800
E0C4  2A 1E          172          BPL     RESET_0A8
E0C6  DE F6          173          LDX     $F6
E0C8  8C 87 2B       174          CPX     #$872B
E0CB  27 17          175          BEQ     RESET_0A8
E0CD  BD E0 25       176          JSR     DELAY1
E0D0  B7 70 00       177          STAA    $7000
E0D3  01             178          NOP
E0D4  6F 00          179          CLR     $00,X
E0D6  08             180          INX
E0D7  C6 2D          181          LDAB    #$2D
E0D9  BD E0 25       182          JSR     DELAY1
E0DC  B7 70 00       183          STAA    $7000
E0DF  01             184          NOP
E0E0  E7 04          185          STAB    $04,X
E0E2  DF F6          186          STX     $F6
E0E4                 187  RESET_0A8
E0E4  7A 00 F8       188          FCB     $7A,$00,$F8
E0E7  26 D3          189          BNE     RESET_080
E0E9  39             190          RTS
E0EA                 191  RESET_0AE
E0EA  86 03          192          LDAA    #$03
E0EC  97 F8          193          STAA    $F8
E0EE                 194  RESET_0B2
E0EE  86 54          195          LDAA    #$54
E0F0  B7 78 00       196          STAA    $7800
E0F3  B6 78 00       197          LDAA    $7800
E0F6  2A 1E          198          BPL     RESET_0DA
E0F8  DE F6          199          LDX     $F6
```

```
E0FA  8C 86 E0       200          CPX     #$86E0
E0FD  27 17          201          BEQ     RESET_0DA
E0FF  BD E0 25       202          JSR     DELAY1
E102  B7 70 00       203          STAA    $7000
E105  01             204          NOP
E106  6F 04          205          CLR     $04,X
E108  09             206          DEX
E109  C6 2D          207          LDAB    #$2D
E10B  BD E0 25       208          JSR     DELAY1
E10E  B7 70 00       209          STAA    $7000
E111  01             210          NOP
E112  E7 00          211          STAB    $00,X
E114  DF F6          212          STX     $F6
E116                 213  RESET_0DA
E116  7A 00 F8       214          FCB     $7A,$00,$F8
E119  26 D3          215          BNE     RESET_0B2
E11B  39             216          RTS
E11C                 217  INITWAL
E11C  CE 80 A0       218          LDX     #$80A0
E11F  86 FF          219          LDAA    #$FF
E121                 220  INITWAL_005
E121  A7 00          221          STAA    $00,X
E123  08             222          INX
E124  8C 81 40       223          CPX     #$8140
E127  26 F8          224          BNE     INITWAL_005
E129  86 25          225          LDAA    #$25
E12B                 226  INITWAL_00F
E12B  A7 00          227          STAA    $00,X
E12D  08             228          INX
E12E  8C 81 E0       229          CPX     #$81E0
E131  26 F8          230          BNE     INITWAL_00F
E133  86 23          231          LDAA    #$23
E135                 232  INITWAL_019
E135  A7 00          233          STAA    $00,X
E137  08             234          INX
E138  8C 82 80       235          CPX     #$8280
E13B  26 F8          236          BNE     INITWAL_019
E13D  CE 4C 45       237          LDX     #$4C45
E140  FF 87 7C       238          STX     $877C
E143  CE 46 54       239          LDX     #$4654
E146  FF 87 7E       240          STX     $877E
E149  96 FE          241          LDAA    $FE
E14B  8A 30          242          ORAA    #$30
E14D  B7 87 7A       243          STAA    $877A
E150  86 F0          244          LDAA    #$F0
E152  97 FF          245          STAA    $FF
E154  39             246          RTS
E155                 247  INITWAL_039
E155  CE 30 30       248          LDX     #$3030
E158  FF 87 30       249          STX     $8730
E15B  FF 87 32       250          STX     $8732
E15E  FF 87 33       251          STX     $8733
E161  CE 2D 2D       252          LDX     #$2D2D
E164  FF 86 E0       253          STX     $86E0
E167  FF 86 E2       254          STX     $86E2
E16A  CE 86 E0       255          LDX     #$86E0
E16D  DF F6          256          STX     $F6
```

```
E16F  39             257          RTS
E170                 258  INITWAL_054
E170  CE 8F FF       259          LDX     #$8FFF
E173                 260  INITWAL_057
E173  6F 00          261          CLR     $00,X
E175  09             262          DEX
E176  8C 7F FF       263          CPX     #$7FFF
E179  26 F8          264          BNE     INITWAL_057
E17B  86 09          265          LDAA    #$09
E17D  97 FE          266          STAA    $FE
E17F  8D 9B          267          BSR     INITWAL
E181  8D D2          268          BSR     INITWAL_039
E183  86 90          269          LDAA    #$90
E185  36             270          PSHA
E186  86 E1          271          LDAA    #$E1
E188  36             272          PSHA
E189  7E E0 94       273          JMP     RESET_058
E18C  FF FF FF FF    274          FCB     $FF,$FF,$FF,$FF
E190                 275  BALLINI
E190  DE F0          276          LDX     $F0
E192  DF F2          277          STX     $F2
E194                 278  BALLMOV
E194  BD E7 B6       279          JSR     BALLCOL
E197                 280  BALLMOV_003
E197  BD E0 02       281          JSR     BALLROW
E19A  BD E4 20       282          JSR     CALCADR
E19D  DF F0          283          STX     $F0
E19F  B7 70 00       284          STAA    $7000
E1A2  01             285          NOP
E1A3  E6 00          286          LDAB    $00,X
E1A5  5D             287          TSTB
E1A6  27 2C          288          BEQ     BALLMOV_040
E1A8  C1 2D          289          CMPB    #$2D
E1AA  26 25          290          BNE     BALLMOV_03D
E1AC  73 00 F4 7D    291          FCB     $73,$00,$F4,$7D,$00,$F5
E1B2  2B 10          292          BMI     BALLMOV_030
E1B4  BD E0 2E       293          JSR     DELAY2
E1B7  B7 70 00       294          STAA    $7000
E1BA  01             295          NOP
E1BB  6D 02          296          TST     $02,X
E1BD  27 03          297          BEQ     BALLMOV_02E
E1BF                 298  BALLMOV_02B
E1BF  73 00 F5       299          FCB     $73,$00,$F5
E1C2                 300  BALLMOV_02E
E1C2  20 D3          301          BRA     BALLMOV_003
E1C4                 302  BALLMOV_030
E1C4  BD E0 2E       303          JSR     DELAY2
E1C7  B7 70 00       304          STAA    $7000
E1CA  01             305          NOP
E1CB  6D 02          306          TST     $02,X
E1CD  26 F0          307          BNE     BALLMOV_02B
E1CF  20 C6          308          BRA     BALLMOV_003
E1D1                 309  BALLMOV_03D
E1D1  BD E2 06       310          JSR     BALLCHK
E1D4                 311  BALLMOV_040
E1D4  86 6F          312          LDAA    #$6F
E1D6  BD E0 2E       313          JSR     DELAY2
```

```
E1D9  B7 70 00       314          STAA    $7000
E1DC  01             315          NOP
E1DD  A7 00          316          STAA    $00,X
E1DF  DE F2          317          LDX     $F2
E1E1  86 6F          318          LDAA    #$6F
E1E3  BD E0 2E       319          JSR     DELAY2
E1E6  B7 70 00       320          STAA    $7000
E1E9  01             321          NOP
E1EA  A1 00          322          CMPA    $00,X
E1EC  26 09          323          BNE     BALLMOV_063
E1EE  BD E0 2E       324          JSR     DELAY2
E1F1  B7 70 00       325          STAA    $7000
E1F4  01             326          NOP
E1F5  6F 00          327          CLR     $00,X
E1F7                 328  BALLMOV_063
E1F7  BD E0 AD       329          JSR     RESET_071
E1FA  BD E0 B8       330          JSR     RESET_07C
E1FD  BD E0 AD       331          JSR     RESET_071
E200  BD E0 EA       332          JSR     RESET_0AE
E203  7E E3 26       333          JMP     BALLCHK_120
E206                 334  BALLCHK
E206  96 16          335          LDAA    $16
E208  81 16          336          CMPA    #$16
E20A  2F 05          337          BLE     BALLCHK_00B
E20C                 338  BALLCHK_006
E20C  31             339          INS
E20D  31             340          INS
E20E  7E E1 94       341          JMP     BALLMOV
E211                 342  BALLCHK_00B
E211  7D 00 F4       343          FCB     $7D,$00,$F4
E214  2A 0A          344          BPL     BALLCHK_01A
E216  7D 00 FD       345          FCB     $7D,$00,$FD
E219  26 02          346          BNE     BALLCHK_017
E21B  20 EF          347          BRA     BALLCHK_006
E21D                 348  BALLCHK_017
E21D  7F 00 FD       349          FCB     $7F,$00,$FD
E220                 350  BALLCHK_01A
E220  C4 07          351          ANDB    #$07
E222  5A             352          DECB
E223  DF FA          353          STX     $FA
E225  CE 87 34       354          LDX     #$8734
E228                 355  BALLCHK_022
E228  BD E0 2E       356          JSR     DELAY2
E22B  B7 70 00       357          STAA    $7000
E22E  01             358          NOP
E22F  A6 00          359          LDAA    $00,X
E231  1B             360          ABA
E232  5F             361          CLRB
E233  4C             362          INCA
E234  81 39          363          CMPA    #$39
E236  2F 13          364          BLE     BALLCHK_045
E238  80 0A          365          SUBA    #$0A
E23A  BD E0 2E       366          JSR     DELAY2
E23D  B7 70 00       367          STAA    $7000
E240  01             368          NOP
E241  A7 00          369          STAA    $00,X
E243  09             370          DEX
```

```
E244  8C 87 2F       371          CPX     #$872F
E247  26 DF          372          BNE     BALLCHK_022
E249  20 09          373          BRA     BALLCHK_04E
E24B                 374  BALLCHK_045
E24B  BD E0 2E       375          JSR     DELAY2
E24E  B7 70 00       376          STAA    $7000
E251  01             377          NOP
E252  A7 00          378          STAA    $00,X
E254                 379  BALLCHK_04E
E254  BD E4 80       380          JSR     BEEP
E257  D6 FF          381          LDAB    $FF
E259  5A             382          DECB
E25A  C1 0A          383          CMPB    #$0A
E25C  26 03          384          BNE     BALLCHK_05B
E25E  BD E1 1C       385          JSR     INITWAL
E261                 386  BALLCHK_05B
E261  D7 FF          387          STAB    $FF
E263  DE FA          388          LDX     $FA
E265  BD E0 2E       389          JSR     DELAY2
E268  B7 70 00       390          STAA    $7000
E26B  01             391          NOP
E26C  20 20          392          BRA     BALLCHK_088
E26E  39             393          RTS
E26F                 394  BALLCHK_069
E26F  8E 03 FF       395          LDS     #$03FF
E272  CE E7 D7       396          LDX     #$E7D7
E275  DF B3          397          STX     $B3
E277  CE 82 A0       398          LDX     #$82A0
E27A  DF B1          399          STX     $B1
E27C  C6 09          400          LDAB    #$09
E27E  BD F6 BB       401          JSR     $F6BB
E281  CE 82 EC       402          LDX     #$82EC
E284  DF B1          403          STX     $B1
E286  C6 12          404          LDAB    #$12
E288  BD F6 BB       405          JSR     $F6BB
E28B  01             406          NOP
E28C  20 0D          407          BRA     BALLCHK_095
E28E                 408  BALLCHK_088
E28E  BD E0 2E       409          JSR     DELAY2
E291  B7 70 00       410          STAA    $7000
E294  01             411          NOP
E295  6F 01          412          CLR     $01,X
E297  73 00 F4       413          FCB     $73,$00,$F4
E29A  39             414          RTS
E29B                 415  BALLCHK_095
E29B  CE 87 30       416          LDX     #$8730
E29E                 417  BALLCHK_098
E29E  7D 70 00       418          TST     $7000
E2A1  01             419          NOP
E2A2  A6 00          420          LDAA    $00,X
E2A4  7D 70 00       421          TST     $7000
E2A7  01             422          NOP
E2A8  A1 07          423          CMPA    $07,X
E2AA  22 0A          424          BHI     BALLCHK_0B0
E2AC  26 06          425          BNE     BALLCHK_0AE
E2AE  08             426          INX
E2AF  8C 87 35       427          CPX     #$8735
```

```
E2B2  26 EA          428          BNE     BALLCHK_098
E2B4                 429  BALLCHK_0AE
E2B4  20 19          430          BRA     BALLCHK_0C9
E2B6                 431  BALLCHK_0B0
E2B6  CE 87 30       432          LDX     #$8730
E2B9                 433  BALLCHK_0B3
E2B9  7D 70 00       434          TST     $7000
E2BC  01             435          NOP
E2BD  A6 00          436          LDAA    $00,X
E2BF  7D 70 00       437          TST     $7000
E2C2  01             438          NOP
E2C3  A7 07          439          STAA    $07,X
E2C5  08             440          INX
E2C6  8C 87 35       441          CPX     #$8735
E2C9  26 EE          442          BNE     BALLCHK_0B3
E2CB  86 92          443          LDAA    #$92
E2CD  A7 00          444          STAA    $00,X
E2CF                 445  BALLCHK_0C9
E2CF  86 40          446          LDAA    #$40
E2D1  B7 78 00       447          STAA    $7800
E2D4                 448  BALLCHK_0CE
E2D4  7D 78 00       449          TST     $7800
E2D7  2A FB          450          BPL     BALLCHK_0CE
E2D9  8E 03 FF       451          LDS     #$03FF
E2DC  CE 87 30       452          LDX     #$8730
E2DF  86 30          453          LDAA    #$30
E2E1                 454  BALLCHK_0DB
E2E1  7D 70 00       455          TST     $7000
E2E4  01             456          NOP
E2E5  A7 00          457          STAA    $00,X
E2E7  08             458          INX
E2E8  8C 87 35       459          CPX     #$8735
E2EB  26 F4          460          BNE     BALLCHK_0DB
E2ED  86 90          461          LDAA    #$90
E2EF  A7 00          462          STAA    $00,X
E2F1  CE 82 80       463          LDX     #$8280
E2F4                 464  BALLCHK_0EE
E2F4  6F 00          465          CLR     $00,X
E2F6  08             466          INX
E2F7  8C 83 2F       467          CPX     #$832F
E2FA  26 F8          468          BNE     BALLCHK_0EE
E2FC  36             469          PSHA
E2FD  86 E1          470          LDAA    #$E1
E2FF  36             471          PSHA
E300  86 09          472          LDAA    #$09
E302  97 FE          473          STAA    $FE
E304  BD E1 1C       474          JSR     INITWAL
E307  7E E0 94       475          JMP     RESET_058
E30A  FF FF FF FF    476          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E312  FF FF FF FF    477          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E31A  FF FF FF FF    478          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E322  FF FF FF FF    479          FCB     $FF,$FF,$FF,$FF
E326                 480  BALLCHK_120
E326  86 09          481          LDAA    #$09
E328  B7 78 00       482          STAA    $7800
E32B  7D 78 00       483          TST     $7800
E32E  2A 01          484          BPL     BALLCHK_12B
```

```
E330  3F             485          SWI
E331                 486  BALLCHK_12B
E331  7E E1 90       487          JMP     BALLINI
E334  FF FF FF FF    488          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E33C  FF FF FF FF    489          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E344  FF FF FF FF    490          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E34C  FF FF FF FF    491          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E354  FF FF FF FF    492          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E35C  FF FF FF FF    493          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E364  FF FF FF FF    494          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E36C  FF FF FF FF    495          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E374  FF FF FF FF    496          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E37C  FF FF FF FF    497          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E384  FF FF FF FF    498          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E38C  FF FF FF FF    499          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E394  FF FF FF FF    500          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E39C  FF FF FF FF    501          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3A4  FF FF FF FF    502          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3AC  FF FF FF FF    503          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3B4  FF FF FF FF    504          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3BC  FF FF FF FF    505          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3C4  FF FF FF FF    506          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3CC  FF FF FF FF    507          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3D4  FF FF FF FF    508          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3DC  FF FF FF FF    509          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3E4  FF FF FF FF    510          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3EC  FF FF FF FF    511          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3F4  FF FF FF FF    512          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E3FC  FF FF FF FF    513          FCB     $FF,$FF,$FF,$FF
E400                 514  SCRNOUT
E400  DE B3          515          LDX     $B3
E402  B7 70 00       516          STAA    $7000
E405  01             517          NOP
E406  A6 00          518          LDAA    $00,X
E408  08             519          INX
E409  DF B3          520          STX     $B3
E40B  DE B1          521          LDX     $B1
E40D  B7 70 00       522          STAA    $7000
E410  01             523          NOP
E411  A7 00          524          STAA    $00,X
E413  08             525          INX
E414  DF B1          526          STX     $B1
E416  5A             527          DECB
E417  26 E7          528          BNE     SCRNOUT
E419  39             529          RTS
E41A  FF FF FF FF    530          FCB     $FF,$FF,$FF,$FF,$FF,$FF
E420                 531  CALCADR
E420  96 16          532          LDAA    $16
E422  16             533          TAB
E423  48             534          ASLA
E424  48             535          ASLA
E425  1B             536          ABA
E426  5F             537          CLRB
E427  48             538          ASLA
E428  59             539          ROLB
E429  48             540          ASLA
E42A  59             541          ROLB
```

```
E42B  48             542          ASLA
E42C  59             543          ROLB
E42D  48             544          ASLA
E42E  59             545          ROLB
E42F  9B 17          546          ADDA    $17
E431  D9 11          547          ADCB    $11
E433  D7 13          548          STAB    $13
E435  97 14          549          STAA    $14
E437  DE 13          550          LDX     $13
E439  39             551          RTS
E43A  FF FF FF FF    552          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E442  FF FF FF FF    553          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E44A  FF FF FF FF    554          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E452  FF FF FF FF    555          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E45A  FF FF FF FF    556          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E462  FF FF FF FF    557          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E46A  FF FF FF FF    558          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E472  FF FF FF FF    559          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E47A  FF FF FF FF    560          FCB     $FF,$FF,$FF,$FF,$FF,$FF
E480                 561  BEEP
E480  B6 78 03       562          LDAA    $7803
E483  84 F7          563          ANDA    #$F7
E485  B7 78 03       564          STAA    $7803
E488  8A 08          565          ORAA    #$08
E48A  B7 78 03       566          STAA    $7803
E48D  39             567          RTS
E48E  FF FF FF FF    568          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E496  FF FF FF FF    569          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E49E  FF FF FF FF    570          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4A6  FF FF FF FF    571          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4AE  FF FF FF FF    572          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4B6  FF FF FF FF    573          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4BE  FF FF FF FF    574          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4C6  FF FF FF FF    575          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4CE  FF FF FF FF    576          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4D6  FF FF FF FF    577          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4DE  FF FF FF FF    578          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4E6  FF FF FF FF    579          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4EE  FF FF FF FF    580          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4F6  FF FF FF FF    581          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E4FE  FF FF FF FF    582          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E506  FF FF FF FF    583          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E50E  FF FF FF FF    584          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E516  FF FF FF FF    585          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E51E  FF FF FF FF    586          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E526  FF FF FF FF    587          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E52E  FF FF FF FF    588          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E536  FF FF FF FF    589          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E53E  FF FF FF FF    590          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E546  FF FF FF FF    591          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E54E  FF FF FF FF    592          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E556  FF FF FF FF    593          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E55E  FF FF FF FF    594          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E566  FF FF FF FF    595          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E56E  FF FF FF FF    596          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E576  FF FF FF FF    597          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E57E  FF FF FF FF    598          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
```

```
E586  FF FF FF FF    599          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E58E  FF FF FF FF    600          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E596  FF FF FF FF    601          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E59E  FF FF FF FF    602          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5A6  FF FF FF FF    603          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5AE  FF FF FF FF    604          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5B6  FF FF FF FF    605          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5BE  FF FF FF FF    606          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5C6  FF FF FF FF    607          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5CE  FF FF FF FF    608          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5D6  FF FF FF FF    609          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5DE  FF FF FF FF    610          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5E6  FF FF FF FF    611          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5EE  FF FF FF FF    612          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5F6  FF FF FF FF    613          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E5FE  FF FF 52 75    614          FCB     $FF,$FF,$52,$75,$62,$6F,$75,$74
E606  20 56 65 72    615          FCB     $20,$56,$65,$72,$73,$69,$6F,$6E
E60E  20 31 2E 30    616          FCB     $20,$31,$2E,$30,$46,$53,$6F,$66
E616  74 77 61 72    617          FCB     $74,$77,$61,$72,$65,$20,$63,$72
E61E  65 61 74 65    618          FCB     $65,$61,$74,$65,$64,$20,$31,$39
E626  38 31 62 79    619          FCB     $38,$31,$62,$79,$20,$44,$61,$76
E62E  69 64 20 4D    620          FCB     $69,$64,$20,$4D,$65,$6E,$61,$72
E636  64 50 72 65    621          FCB     $64,$50,$72,$65,$73,$73,$20,$63
E63E  6C 65 61 72    622          FCB     $6C,$65,$61,$72,$20,$66,$6F,$72
E646  20 67 61 6D    623          FCB     $20,$67,$61,$6D,$65,$6F,$72,$20
E64E  73 70 61 63    624          FCB     $73,$70,$61,$63,$65,$20,$66,$6F
E656  72 20 67 61    625          FCB     $72,$20,$67,$61,$6D,$65,$20,$6C
E65E  69 73 74 FF    626          FCB     $69,$73,$74,$FF,$3D,$37,$20,$70
E666  74 73 2E 25    627          FCB     $74,$73,$2E,$25,$3D,$35,$20,$70
E66E  74 73 2E 23    628          FCB     $74,$73,$2E,$23,$3D,$33,$20,$70
E676  74 73 2E 54    629          FCB     $74,$73,$2E,$54,$68,$65,$72,$65
E67E  20 61 72 65    630          FCB     $20,$61,$72,$65,$20,$39,$20,$62
E686  61 6C 6C 73    631          FCB     $61,$6C,$6C,$73,$2C,$20,$65,$61
E68E  63 68 20 70    632          FCB     $63,$68,$20,$70,$69,$74,$63,$68
E696  65 64 20 61    633          FCB     $65,$64,$20,$61,$73,$20,$73,$70
E69E  61 63 65 20    634          FCB     $61,$63,$65,$20,$69,$73,$20,$73
E6A6  74 72 75 63    635          FCB     $74,$72,$75,$63,$6B,$63,$75,$72
E6AE  73 6F 72 20    636          FCB     $73,$6F,$72,$20,$6C,$65,$66,$74
E6B6  20 26 20 72    637          FCB     $20,$26,$20,$72,$69,$67,$68,$74
E6BE  20 6B 65 79    638          FCB     $20,$6B,$65,$79,$73,$20,$63,$6F
E6C6  6E 74 72 6F    639          FCB     $6E,$74,$72,$6F,$6C,$20,$74,$68
E6CE  65 20 70 61    640          FCB     $65,$20,$70,$61,$64,$64,$6C,$65
E6D6  69 66 20 6C    641          FCB     $69,$66,$20,$6C,$65,$73,$73,$20
E6DE  74 68 61 6E    642          FCB     $74,$68,$61,$6E,$20,$31,$30,$20
E6E6  63 68 61 72    643          FCB     $63,$68,$61,$72,$61,$63,$74,$65
E6EE  72 73 20 72    644          FCB     $72,$73,$20,$72,$65,$6D,$61,$69
E6F6  6E 20 6E 65    645          FCB     $6E,$20,$6E,$65,$77,$20,$77,$61
E6FE  6C 6C 73 20    646          FCB     $6C,$6C,$73,$20,$61,$72,$65,$20
E706  73 65 74       647          FCB     $73,$65,$74
E709                 648  BEEP_289
E709  CE E6 00       649          LDX     #$E600
E70C  DF B3          650          STX     $B3
E70E  CE 81 AC       651          LDX     #$81AC
E711  DF B1          652          STX     $B1
E713  C6 13          653          LDAB    #$13
E715  BD E4 00       654          JSR     SCRNOUT
E718  CE 81 FB       655          LDX     #$81FB
```

```
E71B  DF B1          656          STX     $B1
E71D  C6 15          657          LDAB    #$15
E71F  BD E4 00       658          JSR     SCRNOUT
E722  CE 82 4E       659          LDX     #$824E
E725  DF B1          660          STX     $B1
E727  C6 0F          661          LDAB    #$0F
E729  BD E4 00       662          JSR     SCRNOUT
E72C  CE 84 7C       663          LDX     #$847C
E72F  DF B1          664          STX     $B1
E731  C6 14          665          LDAB    #$14
E733  BD E4 00       666          JSR     SCRNOUT
E736  CE 84 CB       667          LDX     #$84CB
E739  C6 16          668          LDAB    #$16
E73B  DF B1          669          STX     $B1
E73D  BD E4 00       670          JSR     SCRNOUT
E740                 671  BEEP_2C0
E740  86 40          672          LDAA    #$40
E742  B7 78 00       673          STAA    $7800
E745  B6 78 00       674          LDAA    $7800
E748  2A 03          675          BPL     BEEP_2CD
E74A  7E E1 70       676          JMP     INITWAL_054
E74D                 677  BEEP_2CD
E74D  86 4B          678          LDAA    #$4B
E74F  B7 78 00       679          STAA    $7800
E752  B6 78 00       680          LDAA    $7800
E755  2B 02          681          BMI     BEEP_2D9
E757  20 E7          682          BRA     BEEP_2C0
E759                 683  BEEP_2D9
E759  CE 80 00       684          LDX     #$8000
E75C                 685  BEEP_2DC
E75C  6F 00          686          CLR     $00,X
E75E  08             687          INX
E75F  8C 8F FF       688          CPX     #$8FFF
E762  26 F8          689          BNE     BEEP_2DC
E764  CE 82 53       690          LDX     #$8253
E767  C6 08          691          LDAB    #$08
E769  DF B1          692          STX     $B1
E76B  CE E6 61       693          LDX     #$E661
E76E  DF B3          694          STX     $B3
E770  BD E4 00       695          JSR     SCRNOUT
E773  CE 82 A3       696          LDX     #$82A3
E776  DF B1          697          STX     $B1
E778  C6 08          698          LDAB    #$08
E77A  BD E4 00       699          JSR     SCRNOUT
E77D  CE 82 F3       700          LDX     #$82F3
E780  DF B1          701          STX     $B1
E782  C6 08          702          LDAB    #$08
E784  BD E4 00       703          JSR     SCRNOUT
E787  CE 83 2E       704          LDX     #$832E
E78A  DF B1          705          STX     $B1
E78C  C6 32          706          LDAB    #$32
E78E  BD E4 00       707          JSR     SCRNOUT
E791  CE 83 D2       708          LDX     #$83D2
E794  DF B1          709          STX     $B1
E796  C6 2B          710          LDAB    #$2B
E798  BD E4 00       711          JSR     SCRNOUT
E79B  CE 84 6D       712          LDX     #$846D
```

```
E79E  DF B1          713          STX     $B1
E7A0  C6 33          714          LDAB    #$33
E7A2  BD E4 00       715          JSR     SCRNOUT
E7A5  CE 85 1E       716          LDX     #$851E
E7A8  DF B1          717          STX     $B1
E7AA  C6 14          718          LDAB    #$14
E7AC  CE E6 37       719          LDX     #$E637
E7AF  DF B3          720          STX     $B3
E7B1  BD E4 00       721          JSR     SCRNOUT
E7B4  20 8A          722          BRA     BEEP_2C0
E7B6                 723  BALLCOL
E7B6  7D 00 F5       724          FCB     $7D,$00,$F5
E7B9  2B 02          725          BMI     BALLCOL_007
E7BB  20 0D          726          BRA     BALLCOL_014
E7BD                 727  BALLCOL_007
E7BD  96 17          728          LDAA    $17
E7BF  4C             729          INCA
E7C0  97 17          730          STAA    $17
E7C2  81 4F          731          CMPA    #$4F
E7C4  2D 03          732          BLT     BALLCOL_013
E7C6  73 00 F5       733          FCB     $73,$00,$F5
E7C9                 734  BALLCOL_013
E7C9  39             735          RTS
E7CA                 736  BALLCOL_014
E7CA  96 17          737          LDAA    $17
E7CC  4A             738          DECA
E7CD  97 17          739          STAA    $17
E7CF  81 00          740          CMPA    #$00
E7D1  2E 03          741          BGT     BALLCOL_020
E7D3  73 00 F5       742          FCB     $73,$00,$F5
E7D6                 743  BALLCOL_020
E7D6  39             744          RTS
E7D7  47 41 4D 45    745          FCB     $47,$41,$4D,$45,$20,$4F,$56,$45
E7DF  52 48 49 54    746          FCB     $52,$48,$49,$54,$20,$43,$4C,$45
E7E7  41 52 20 54    747          FCB     $41,$52,$20,$54,$4F,$20,$52,$45
E7EF  53 45 54       748          FCB     $53,$45,$54
E7F2                 749  BALLCOL_03C
E7F2  86 03          750          LDAA    #$03
E7F4  B7 7A 00       751          STAA    $7A00
E7F7  B7 7C 00       752          STAA    $7C00
E7FA  7E E7 09       753          JMP     BEEP_289
E7FD  FF FF FF       754          FCB     $FF,$FF,$FF
E800                 755
E800                 756          END
```

Shuttle Attack

```
0000                   2  ; shuttleattack.bin -- "SHUTTLE ATTACK V.2", COPYRIG>
0000                   3  ; ADM-31 plugin ROM ($E000-$E7FF), MD5 a3d616ea819c7>
0000                   4  ;
0000                   5  ; VERIFIED BYTE-EXACT: reassembled through acp6800, >
0000                   6  ; original (confirmed 0 errors, matching MD5).
0000                   7  ;
0000                   8  ; Standard plugin-ROM header: "TJ" signature at $E00>
0000                   9  ; $E03C (3 NOPs then JMP to the real init routine at>
0000                  10  ; variant of the usual bare "JMP" directly at $E03C >
```

```
0000                    11  ; this project). $E002 is unused padding -- this gam>
0000                    12  ; the KEY_ENTRY ($E002) callback convention seen in >
0000                    13  ; game ROMs (Cosmic Intruders, Space Invaders); it p>
0000                    14  ; directly instead.
0000                    15  ;
0000                    16  ; Embedded UI text confirms this is a Space-Invaders>
0000                    17  ; "GAME OVER", "SCORE", "HIGH SCORE", "3 SHIPS LEFT",
0000                    18  ; "PRESS FUNCTION TO START".
0000                    19  ;
0000                    20  ; LABELS: routine entry points (JSR/BSR-independent >
0000                    21  ; evidence-based descriptive names; internal branch >
0000                    22  ; <ROUTINE>_<offset>. Base names capped at 7 charact>
0000                    23  ; of branch targets that land in the middle of anoth>
0000                    24  ; operand bytes (a real overlapping-code case, not a>
0000                    25  ; ENTRY's neighborhood) can't be given a symbolic la>
0000                    26  ; linear listing can't represent one byte range as t>
0000                    27  ; those stay as raw hex targets, still byte-exact.
0000                    28  ;
0000                    29  ; Control-flow-traced coverage: ~66.7% of bytes; the>
0000                    30  ; (sprite/UI-text tables, an object table, etc.), no>
0000                    31  ; ================================================>
0000                    32
E000                    33          ORG     $E000
E000  54 4A FF FF       34          FCB     $54,$4A,$FF,$FF,$FF,$FF,$FF,$FF
E008  FF FF FF FF       35          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E010  FF FF FF FF       36          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E018  FF FF FF FF       37          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E020  FF FF FF FF       38          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E028  FF FF FF FF       39          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E030  FF FF FF FF       40          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E038  FF FF FF FF       41          FCB     $FF,$FF,$FF,$FF
E03C                    42  ENTRY
E03C  01                43          NOP
E03D  01                44          NOP
E03E  01                45          NOP
E03F  7E E2 CF          46          JMP     RESET
E042  FF FF FF FF       47          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E04A  FF FF FF FF       48          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E052  FF FF FF FF       49          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E05A  FF FF FF FF       50          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E062  FF FF FF FF       51          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E06A  FF FF FF FF       52          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E072  FF FF FF FF       53          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E07A  FF FF FF 7F       54          FCB     $FF,$FF,$FF,$7F,$00,$00,$39,$FF
E082  FF FF FF FF       55          FCB     $FF,$FF,$FF,$FF,$FF,$20,$F4,$FF
E08A  FF FF FF FF       56          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E092  FF FF FF FF       57          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E09A  FF FF FF FF       58          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0A2  FF FF FF FF       59          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0AA  FF FF FF FF       60          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0B2  FF FF FF FF       61          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0BA  FF FF FF FF       62          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0C2  FF FF FF FF       63          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0CA  FF FF FF FF       64          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0D2  FF FF FF FF       65          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0DA  FF FF FF FF       66          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0E2  FF FF FF FF       67          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
```

```
E0EA  FF FF FF FF     68          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0F2  FF FF FF FF     69          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E0FA  FF FF FF FF     70          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E102  FF FF FF FF     71          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E10A  FF FF FF FF     72          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E112  FF FF FF FF     73          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E11A  FF FF FF FF     74          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E122  FF FF FF FF     75          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E12A  FF FF FF FF     76          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E132  FF FF FF FF     77          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E13A  FF FF FF FF     78          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E142  FF FF FF FF     79          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E14A  FF FF FF FF     80          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E152  FF FF FF FF     81          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E15A  FF FF FF FF     82          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E162  FF FF FF FF     83          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E16A  FF FF FF FF     84          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E172  FF FF FF FF     85          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E17A  FF FF FF FF     86          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E182  FF FF FF FF     87          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E18A  FF FF FF FF     88          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E192  FF FF FF FF     89          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E19A  FF FF FF FF     90          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1A2  FF FF FF FF     91          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1AA  FF FF FF FF     92          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1B2  FF FF FF FF     93          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1BA  FF FF FF FF     94          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1C2  FF FF FF FF     95          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1CA  FF FF FF FF     96          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1D2  FF FF FF FF     97          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1DA  FF FF FF FF     98          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1E2  FF FF FF FF     99          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1EA  FF FF FF FF    100          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1F2  FF FF FF FF    101          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E1FA  FF FF FF FF    102          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E202  FF FF FF FF    103          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E20A  FF FF FF FF    104          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
E212  FF FF FF FF    105          FCB     $FF,$FF,$FF,$FF,$FF,$FF,$47,$41
E21A  4D 45 20 4F    106          FCB     $4D,$45,$20,$4F,$56,$45,$52,$53
E222  43 4F 52 45    107          FCB     $43,$4F,$52,$45,$90,$30,$30,$30
E22A  30 30 90 48    108          FCB     $30,$30,$90,$48,$49,$47,$48,$20
E232  53 43 4F 52    109          FCB     $53,$43,$4F,$52,$45,$90,$30,$30
E23A  30 30 30 90    110          FCB     $30,$30,$30,$90,$33,$20,$53,$48
E242  49 50 53 20    111          FCB     $49,$50,$53,$20,$4C,$45,$46,$54
E24A  50 52 45 53    112          FCB     $50,$52,$45,$53,$53,$20,$46,$55
E252  4E 43 54 49    113          FCB     $4E,$43,$54,$49,$4F,$4E,$20,$54
E25A  4F 20 53 54    114          FCB     $4F,$20,$53,$54,$41,$52,$54
E261                 115  ENTRY_225
E261  CE 80 17       116          LDX     #$8017
E264                 117  ENTRY_228
E264  7D 70 00       118          TST     $7000
E267  01             119          NOP
E268  A6 00          120          LDAA    $00,X
E26A  7D 70 00       121          TST     $7000
E26D  01             122          NOP
E26E  A1 11          123          CMPA    $11,X
E270  22 0B          124          BHI     ENTRY_241
```

```
E272  26 06          125          BNE     ENTRY_23E
E274  08             126          INX
E275  8C 80 1C       127          CPX     #$801C
E278  26 EA          128          BNE     ENTRY_228
E27A                 129  ENTRY_23E
E27A  7E E3 55       130          JMP     RESET_086
E27D                 131  ENTRY_241
E27D  CE 80 17       132          LDX     #$8017
E280                 133  ENTRY_244
E280  7D 70 00       134          TST     $7000
E283  01             135          NOP
E284  A6 00          136          LDAA    $00,X
E286  7D 70 00       137          TST     $7000
E289  01             138          NOP
E28A  A7 11          139          STAA    $11,X
E28C  08             140          INX
E28D  8C 80 1C       141          CPX     #$801C
E290  26 EE          142          BNE     ENTRY_244
E292  86 92          143          LDAA    #$92
E294  7D 70 00       144          TST     $7000
E297  01             145          NOP
E298  B7 80 27       146          STAA    $8027
E29B  20 DD          147          BRA     ENTRY_23E
E29D                 148  ENTRY_261
E29D  CE E2 18       149          LDX     #$E218
E2A0  DF F2          150          STX     $F2
E2A2  CE 83 91       151          LDX     #$8391
E2A5  C6 09          152          LDAB    #$09
E2A7  8D 73          153          BSR     RESET_04D
E2A9  20 B6          154          BRA     ENTRY_225
E2AB                 155  ENTRY_26F
E2AB  7D 70 00       156          TST     $7000
E2AE  01             157          NOP
E2AF  B6 80 2F       158          LDAA    $802F
E2B2  4A             159          DECA
E2B3  81 30          160          CMPA    #$30
E2B5  27 E6          161          BEQ     ENTRY_261
E2B7  7D 70 00       162          TST     $7000
E2BA  01             163          NOP
E2BB  B7 80 2F       164          STAA    $802F
E2BE  7E E3 80       165          JMP     RESET_0B1
E2C1                 166  BEEP
E2C1  B6 78 03       167          LDAA    $7803
E2C4  88 08          168          EORA    #$08
E2C6  B7 78 03       169          STAA    $7803
E2C9  88 08          170          EORA    #$08
E2CB  B7 78 03       171          STAA    $7803
E2CE  39             172          RTS
E2CF                 173  RESET
E2CF  CE 80 00       174          LDX     #$8000
E2D2                 175  RESET_003
E2D2  7D 70 00       176          TST     $7000
E2D5  01             177          NOP
E2D6  6F 00          178          CLR     $00,X
E2D8  08             179          INX
E2D9  8C 87 80       180          CPX     #$8780
E2DC  26 F4          181          BNE     RESET_003
```

```
E2DE  86 03          182          LDAA    #$03
E2E0  B7 7A 00       183          STAA    $7A00
E2E3  B7 7C 00       184          STAA    $7C00
E2E6  86 17          185          LDAA    #$17
E2E8  B7 7F 06       186          STAA    $7F06
E2EB  86 51          187          LDAA    #$51
E2ED  B7 7F 0C       188          STAA    $7F0C
E2F0  B6 78 02       189          LDAA    $7802
E2F3  84 FB          190          ANDA    #$FB
E2F5  B7 78 02       191          STAA    $7802
E2F8  86 98          192          LDAA    #$98
E2FA  B7 87 30       193          STAA    $8730
E2FD  8E 00 3F       194          LDS     #$003F
E300  CE E7 B7       195          LDX     #$E7B7
E303  DF F2          196          STX     $F2
E305  C6 12          197          LDAB    #$12
E307  CE 83 3D       198          LDX     #$833D
E30A  8D 10          199          BSR     RESET_04D
E30C  C6 12          200          LDAB    #$12
E30E  CE 83 DD       201          LDX     #$83DD
E311  8D 09          202          BSR     RESET_04D
E313  C6 25          203          LDAB    #$25
E315  CE 84 74       204          LDX     #$8474
E318  8D 02          205          BSR     RESET_04D
E31A  20 16          206          BRA     RESET_063
E31C                 207  RESET_04D
E31C  DF F0          208          STX     $F0
E31E  DE F2          209          LDX     $F2
E320  A6 00          210          LDAA    $00,X
E322  08             211          INX
E323  DF F2          212          STX     $F2
E325  DE F0          213          LDX     $F0
E327  7D 70 00       214          TST     $7000
E32A  01             215          NOP
E32B  A7 00          216          STAA    $00,X
E32D  08             217          INX
E32E  5A             218          DECB
E32F  26 EB          219          BNE     RESET_04D
E331  39             220          RTS
E332                 221  RESET_063
E332  86 10          222          LDAA    #$10
E334                 223  RESET_065
E334  09             224          DEX
E335  26 FD          225          BNE     RESET_065
E337  4A             226          DECA
E338  26 FA          227          BNE     RESET_065
E33A  CE E2 21       228          LDX     #$E221
E33D  DF F2          229          STX     $F2
E33F  C6 1D          230          LDAB    #$1D
E341  CE 80 11       231          LDX     #$8011
E344  8D D6          232          BSR     RESET_04D
E346  CE 83 20       233          LDX     #$8320
E349                 234  RESET_07A
E349  7D 70 00       235          TST     $7000
E34C  01             236          NOP
E34D  6F 00          237          CLR     $00,X
E34F  08             238          INX
```

```
E350  8C 85 50       239          CPX     #$8550
E353  26 F4          240          BNE     RESET_07A
E355                 241  RESET_086
E355  CE E2 3E       242          LDX     #$E23E
E358  DF F2          243          STX     $F2
E35A  C6 0C          244          LDAB    #$0C
E35C  CE 80 2F       245          LDX     #$802F
E35F  8D BB          246          BSR     RESET_04D
E361  CE 84 2A       247          LDX     #$842A
E364  C6 17          248          LDAB    #$17
E366  8D B4          249          BSR     RESET_04D
E368  86 50          250          LDAA    #$50
E36A  B7 78 00       251          STAA    $7800
E36D                 252  RESET_09E
E36D  7D 78 00       253          TST     $7800
E370  2A FB          254          BPL     RESET_09E
E372  CE E2 21       255          LDX     #$E221
E375  DF F2          256          STX     $F2
E377  C6 0C          257          LDAB    #$0C
E379  CE 80 11       258          LDX     #$8011
E37C  8D 9E          259          BSR     RESET_04D
E37E  01             260          NOP
E37F  01             261          NOP
E380                 262  RESET_0B1
E380  CE 83 20       263          LDX     #$8320
E383  86 98          264          LDAA    #$98
E385                 265  RESET_0B6
E385  C6 50          266          LDAB    #$50
E387  7D 70 00       267          TST     $7000
E38A  01             268          NOP
E38B  A7 00          269          STAA    $00,X
E38D                 270  RESET_0BE
E38D  7D 70 00       271          TST     $7000
E390  01             272          NOP
E391  6F 01          273          CLR     $01,X
E393  08             274          INX
E394  5A             275          DECB
E395  26 F6          276          BNE     RESET_0BE
E397  8C 85 50       277          CPX     #$8550
E39A  26 E9          278          BNE     RESET_0B6
E39C  CE 00 60       279          LDX     #$0060
E39F                 280  RESET_0D0
E39F  6F 00          281          CLR     $00,X
E3A1  08             282          INX
E3A2  8C 00 D9       283          CPX     #$00D9
E3A5  26 F8          284          BNE     RESET_0D0
E3A7  86 0D          285          LDAA    #$0D
E3A9  97 C8          286          STAA    $C8
E3AB  86 20          287          LDAA    #$20
E3AD  97 C9          288          STAA    $C9
E3AF  86 90          289          LDAA    #$90
E3B1  7D 70 00       290          TST     $7000
E3B4  01             291          NOP
E3B5  B7 80 27       292          STAA    $8027
E3B8  7E E4 87       293          JMP     RESET_1B8
E3BB                 294  RESET_0EC
E3BB  CE 80 1B       295          LDX     #$801B
```

```
E3BE                  296  RESET_0EF
E3BE  7D 70 00        297          TST     $7000
E3C1  01              298          NOP
E3C2  A6 00           299          LDAA    $00,X
E3C4  4C              300          INCA
E3C5  81 39           301          CMPA    #$39
E3C7  22 08           302          BHI     RESET_102
E3C9  7D 70 00        303          TST     $7000
E3CC  01              304          NOP
E3CD  A7 00           305          STAA    $00,X
E3CF  20 0E           306          BRA     RESET_110
E3D1                  307  RESET_102
E3D1  86 30           308          LDAA    #$30
E3D3  7D 70 00        309          TST     $7000
E3D6  01              310          NOP
E3D7  A7 00           311          STAA    $00,X
E3D9  09              312          DEX
E3DA  8C 80 16        313          CPX     #$8016
E3DD  26 DF           314          BNE     RESET_0EF
E3DF                  315  RESET_110
E3DF  5A              316          DECB
E3E0  26 D9           317          BNE     RESET_0EC
E3E2  39              318          RTS
E3E3                  319  RESET_114
E3E3  08              320          INX
E3E4                  321  RESET_115
E3E4  8E 00 3F        322          LDS     #$003F
E3E7  7D 70 00        323          TST     $7000
E3EA  01              324          NOP
E3EB  A7 00           325          STAA    $00,X
E3ED  86 10           326          LDAA    #$10
E3EF                  327  RESET_120
E3EF  36              328          PSHA
E3F0  BD E2 C1        329          JSR     BEEP
E3F3  32              330          PULA
E3F4  CE 0D 00        331          LDX     #$0D00
E3F7                  332  RESET_128
E3F7  09              333          DEX
E3F8  26 FD           334          BNE     RESET_128
E3FA  4A              335          DECA
E3FB  26 F2           336          BNE     RESET_120
E3FD  7E E2 AB        337          JMP     ENTRY_26F
E400                  338  RESET_131
E400  86 69           339          LDAA    #$69
E402  7D 70 00        340          TST     $7000
E405  01              341          NOP
E406  A7 00           342          STAA    $00,X
E408  DF F4           343          STX     $F4
E40A  09              344          DEX
E40B  9C CA           345          CPX     $CA
E40D  27 D5           346          BEQ     RESET_115
E40F  08              347          INX
E410  9C CA           348          CPX     $CA
E412  27 CF           349          BEQ     RESET_114
E414  9C 82           350          CPX     $82
E416  27 5B           351          BEQ     RESET_1A4
E418  9C 8A           352          CPX     $8A
```

```
E41A  27 5B          353          BEQ     RESET_1A8
E41C  9C 92          354          CPX     $92
E41E  27 5B          355          BEQ     RESET_1AC
E420                 356  RESET_151
E420  DE F0          357          LDX     $F0
E422  6F 06          358          CLR     $06,X
E424  6F 07          359          CLR     $07,X
E426  6F 02          360          CLR     $02,X
E428  6F 03          361          CLR     $03,X
E42A  CE 00 60       362          LDX     #$0060
E42D                 363  RESET_15E
E42D  A6 02          364          LDAA    $02,X
E42F  91 F4          365          CMPA    $F4
E431  26 06          366          BNE     RESET_16A
E433  A6 03          367          LDAA    $03,X
E435  91 F5          368          CMPA    $F5
E437  27 0D          369          BEQ     RESET_177
E439                 370  RESET_16A
E439  C6 08          371          LDAB    #$08
E43B                 372  RESET_16C
E43B  08             373          INX
E43C  5A             374          DECB
E43D  26 FC          375          BNE     RESET_16C
E43F  8C 00 C8       376          CPX     #$00C8
E442  26 E9          377          BNE     RESET_15E
E444  20 24          378          BRA     RESET_19B
E446                 379  RESET_177
E446  6F 06          380          CLR     $06,X
E448  6F 07          381          CLR     $07,X
E44A  CE 00 D0       382          LDX     #$00D0
E44D                 383  RESET_17E
E44D  6D 02          384          TST     $02,X
E44F  26 0C          385          BNE     RESET_18E
E451  96 F4          386          LDAA    $F4
E453  A7 00          387          STAA    $00,X
E455  96 F5          388          LDAA    $F5
E457  A7 01          389          STAA    $01,X
E459  6C 02          390          INC     $02,X
E45B  20 0A          391          BRA     RESET_198
E45D                 392  RESET_18E
E45D  08             393          INX
E45E  08             394          INX
E45F  08             395          INX
E460  8C 00 D9       396          CPX     #$00D9
E463  26 E8          397          BNE     RESET_17E
E465  20 03          398          BRA     RESET_19B
E467                 399  RESET_198
E467  7E E2 C1       400          JMP     BEEP
E46A                 401  RESET_19B
E46A  DE F4          402          LDX     $F4
E46C  7D 70 00       403          TST     $7000
E46F  01             404          NOP
E470  6F 00          405          CLR     $00,X
E472  39             406          RTS
E473                 407  RESET_1A4
E473  C6 0F          408          LDAB    #$0F
E475  20 06          409          BRA     RESET_1AE
```

```
E477                    410  RESET_1A8
E477  C6 19             411          LDAB    #$19
E479  20 02             412          BRA     RESET_1AE
E47B                    413  RESET_1AC
E47B  C6 23             414          LDAB    #$23
E47D                    415  RESET_1AE
E47D  37                416          PSHB
E47E  8D A0             417          BSR     RESET_151
E480  33                418          PULB
E481  7E E3 BB          419          JMP     RESET_0EC
E484  43 19 82          420          FCB     $43,$19,$82
E487                    421  RESET_1B8
E487  86 01             422          LDAA    #$01
E489  95 CF             423          BITA    $CF
E48B  26 03             424          BNE     RESET_1C1
E48D  BD E6 AD          425          JSR     SCRDRAW
E490                    426  RESET_1C1
E490  CE 00 D0          427          LDX     #$00D0
E493                    428  RESET_1C4
E493  DF F0             429          STX     $F0
E495  6D 02             430          TST     $02,X
E497  27 14             431          BEQ     RESET_1DE
E499  86 02             432          LDAA    #$02
E49B  A5 02             433          BITA    $02,X
E49D  26 0C             434          BNE     RESET_1DC
E49F  6F 02             435          CLR     $02,X
E4A1  EE 00             436          LDX     $00,X
E4A3  7D 70 00          437          TST     $7000
E4A6  01                438          NOP
E4A7  6F 00             439          CLR     $00,X
E4A9  20 02             440          BRA     RESET_1DE
E4AB                    441  RESET_1DC
E4AB  6C 02             442          INC     $02,X
E4AD                    443  RESET_1DE
E4AD  DE F0             444          LDX     $F0
E4AF  08                445          INX
E4B0  08                446          INX
E4B1  08                447          INX
E4B2  8C 00 D9          448          CPX     #$00D9
E4B5  26 DC             449          BNE     RESET_1C4
E4B7  7C 00 CF          450          FCB     $7C,$00,$CF
E4BA  86 01             451          LDAA    #$01
E4BC  95 CF             452          BITA    $CF
E4BE  26 03             453          BNE     RESET_1F4
E4C0  7E E5 F7          454          JMP     RESET_328
E4C3                    455  RESET_1F4
E4C3  DE C8             456          LDX     $C8
E4C5  DF F0             457          STX     $F0
E4C7  86 01             458          LDAA    #$01
E4C9  95 CE             459          BITA    $CE
E4CB  26 2C             460          BNE     RESET_22A
E4CD  86 52             461          LDAA    #$52
E4CF  D6 C8             462          LDAB    $C8
E4D1  C4 7F             463          ANDB    #$7F
E4D3  B7 78 00          464          STAA    $7800
E4D6  7D 78 00          465          TST     $7800
E4D9  2A 07             466          BPL     RESET_213
```

```
E4DB  C1 10          467          CMPB    #$10
E4DD  27 01          468          BEQ     RESET_211
E4DF  5C             469          INCB
E4E0                 470  RESET_211
E4E0  20 0E          471          BRA     RESET_221
E4E2                 472  RESET_213
E4E2  4C             473          INCA
E4E3  B7 78 00       474          STAA    $7800
E4E6  7D 78 00       475          TST     $7800
E4E9  2A 0E          476          BPL     RESET_22A
E4EB  C1 0A          477          CMPB    #$0A
E4ED  27 01          478          BEQ     RESET_221
E4EF  5A             479          DECB
E4F0                 480  RESET_221
E4F0  7D 00 C8       481          FCB     $7D,$00,$C8
E4F3  2A 02          482          BPL     RESET_228
E4F5  CA 80          483          ORAB    #$80
E4F7                 484  RESET_228
E4F7  D7 C8          485          STAB    $C8
E4F9                 486  RESET_22A
E4F9  7C 00 CE       487          FCB     $7C,$00,$CE
E4FC  86 01          488          LDAA    #$01
E4FE  95 CE          489          BITA    $CE
E500  26 3F          490          BNE     RESET_272
E502  86 55          491          LDAA    #$55
E504  B7 78 00       492          STAA    $7800
E507  7D 78 00       493          TST     $7800
E50A  2A 15          494          BPL     RESET_252
E50C  D6 C8          495          LDAB    $C8
E50E  C4 7F          496          ANDB    #$7F
E510  D7 C8          497          STAB    $C8
E512  C6 01          498          LDAB    #$01
E514  D5 CE          499          BITB    $CE
E516  26 46          500          BNE     RESET_28F
E518  D6 C9          501          LDAB    $C9
E51A  C1 28          502          CMPB    #$28
E51C  24 01          503          BCC     RESET_250
E51E  5C             504          INCB
E51F                 505  RESET_250
E51F  20 1C          506          BRA     RESET_26E
E521                 507  RESET_252
E521  4A             508          DECA
E522  B7 78 00       509          STAA    $7800
E525  7D 78 00       510          TST     $7800
E528  2A 17          511          BPL     RESET_272
E52A  D6 C8          512          LDAB    $C8
E52C  CA 80          513          ORAB    #$80
E52E  D7 C8          514          STAB    $C8
E530  C6 01          515          LDAB    #$01
E532  D5 CE          516          BITB    $CE
E534  26 28          517          BNE     RESET_28F
E536  D6 C9          518          LDAB    $C9
E538  C1 28          519          CMPB    #$28
E53A  25 01          520          BCS     RESET_26E
E53C  5A             521          DECB
E53D                 522  RESET_26E
E53D  D7 C9          523          STAB    $C9
```

```
E53F  20 1D          524          BRA     RESET_28F
E541                 525  RESET_272
E541  C6 03          526          LDAB    #$03
E543  D5 CE          527          BITB    $CE
E545  26 1A          528          BNE     RESET_292
E547  D6 C9          529          LDAB    $C9
E549  7D 00 C8       530          FCB     $7D,$00,$C8
E54C  2B 07          531          BMI     RESET_286
E54E  C1 10          532          CMPB    #$10
E550  27 01          533          BEQ     RESET_284
E552  5A             534          DECB
E553                 535  RESET_284
E553  20 05          536          BRA     RESET_28B
E555                 537  RESET_286
E555  C1 3A          538          CMPB    #$3A
E557  27 01          539          BEQ     RESET_28B
E559  5C             540          INCB
E55A                 541  RESET_28B
E55A  D7 C9          542          STAB    $C9
E55C  20 03          543          BRA     RESET_292
E55E                 544  RESET_28F
E55E  78 00 CF       545          FCB     $78,$00,$CF
E561                 546  RESET_292
E561  DE C8          547          LDX     $C8
E563  9C F0          548          CPX     $F0
E565  27 3A          549          BEQ     RESET_2D2
E567  DE CA          550          LDX     $CA
E569  DF CC          551          STX     $CC
E56B  CE 00 C8       552          LDX     #$00C8
E56E  BD E6 93       553          JSR     POSADR
E571  EE 04          554          LDX     $04,X
E573  7D 70 00       555          TST     $7000
E576  01             556          NOP
E577  6F 00          557          CLR     $00,X
E579  7D 70 00       558          TST     $7000
E57C  01             559          NOP
E57D  6F 01          560          CLR     $01,X
E57F  DE CA          561          LDX     $CA
E581  86 2D          562          LDAA    #$2D
E583  7D 00 C8       563          FCB     $7D,$00,$C8
E586  2B 0A          564          BMI     RESET_2C3
E588  7D 70 00       565          TST     $7000
E58B  01             566          NOP
E58C  A7 01          567          STAA    $01,X
E58E  86 29          568          LDAA    #$29
E590  20 09          569          BRA     RESET_2CC
E592                 570  RESET_2C3
E592  7D 70 00       571          TST     $7000
E595  01             572          NOP
E596  A7 00          573          STAA    $00,X
E598  08             574          INX
E599  86 28          575          LDAA    #$28
E59B                 576  RESET_2CC
E59B  7D 70 00       577          TST     $7000
E59E  01             578          NOP
E59F  A7 00          579          STAA    $00,X
E5A1                 580  RESET_2D2
```

```
E5A1  86 AF          581          LDAA    #$AF
E5A3  95 CE          582          BITA    $CE
E5A5  26 0D          583          BNE     RESET_2E5
E5A7  CE 00 98       584          LDX     #$0098
E5AA  DF F2          585          STX     $F2
E5AC  CE 00 80       586          LDX     #$0080
E5AF  DF F0          587          STX     $F0
E5B1  BD E7 2E       588          JSR     FINDOBJ
E5B4                 589  RESET_2E5
E5B4  CE 00 C8       590          LDX     #$00C8
E5B7  DF F2          591          STX     $F2
E5B9  CE 00 60       592          LDX     #$0060
E5BC  DF F0          593          STX     $F0
E5BE  8D 40          594          BSR     RESET_331
E5C0  86 01          595          LDAA    #$01
E5C2  95 CE          596          BITA    $CE
E5C4  27 0D          597          BEQ     RESET_304
E5C6  CE 00 98       598          LDX     #$0098
E5C9  DF F2          599          STX     $F2
E5CB  CE 00 80       600          LDX     #$0080
E5CE  DF F0          601          STX     $F0
E5D0  BD E7 6B       602          JSR     PROCOBJ
E5D3                 603  RESET_304
E5D3  86 09          604          LDAA    #$09
E5D5  B7 78 00       605          STAA    $7800
E5D8  7D 78 00       606          TST     $7800
E5DB  2A 01          607          BPL     RESET_30F
E5DD  3F             608          SWI
E5DE                 609  RESET_30F
E5DE  86 40          610          LDAA    #$40
E5E0  B7 78 00       611          STAA    $7800
E5E3  7D 78 00       612          TST     $7800
E5E6  2A 0F          613          BPL     RESET_328
E5E8  CE 00 80       614          LDX     #$0080
E5EB  DF F2          615          STX     $F2
E5ED  CE 00 60       616          LDX     #$0060
E5F0  DF F0          617          STX     $F0
E5F2  86 2D          618          LDAA    #$2D
E5F4  BD E7 2E       619          JSR     FINDOBJ
E5F7                 620  RESET_328
E5F7  CE 12 00       621          LDX     #$1200
E5FA                 622  RESET_32B
E5FA  09             623          DEX
E5FB  26 FD          624          BNE     RESET_32B
E5FD  7E E4 87       625          JMP     RESET_1B8
E600                 626  RESET_331
E600  A6 06          627          LDAA    $06,X
E602  27 02          628          BEQ     RESET_337
E604  8D 08          629          BSR     RESET_33F
E606                 630  RESET_337
E606  BD E7 21       631          JSR     NEXTOBJ
E609  9C F2          632          CPX     $F2
E60B  26 F1          633          BNE     $E5FE
E60D  39             634          RTS
E60E                 635  RESET_33F
E60E  81 2D          636          CMPA    #$2D
E610  27 75          637          BEQ     RESET_3B8
```

```
E612  C6 01          638          LDAB    #$01
E614  D5 CE          639          BITB    $CE
E616  26 30          640          BNE     RESET_379
E618  E6 00          641          LDAB    $00,X
E61A  5D             642          TSTB
E61B  2B 16          643          BMI     RESET_364
E61D  84 07          644          ANDA    #$07
E61F  48             645          ASLA
E620  48             646          ASLA
E621  95 CE          647          BITA    $CE
E623  26 07          648          BNE     RESET_35D
E625  C1 0A          649          CMPB    #$0A
E627  27 01          650          BEQ     RESET_35B
E629  5A             651          DECB
E62A                 652  RESET_35B
E62A  20 1A          653          BRA     RESET_377
E62C                 654  RESET_35D
E62C  C1 10          655          CMPB    #$10
E62E  27 01          656          BEQ     RESET_362
E630  5C             657          INCB
E631                 658  RESET_362
E631  20 13          659          BRA     RESET_377
E633                 660  RESET_364
E633  7D 00 C8       661          FCB     $7D,$00,$C8
E636  2B 02          662          BMI     RESET_36B
E638  C4 7F          663          ANDB    #$7F
E63A                 664  RESET_36B
E63A  D1 C8          665          CMPB    $C8
E63C  27 07          666          BEQ     $E645
E63E  25 03          667          BCS     RESET_374
E640  5A             668          DECB
E641  20 01          669          BRA     RESET_375
E643                 670  RESET_374
E643  5C             671          INCB
E644                 672  RESET_375
E644  CA 80          673          ORAB    #$80
E646                 674  RESET_377
E646  E7 00          675          STAB    $00,X
E648                 676  RESET_379
E648  6D 07          677          TST     $07,X
E64A  27 0C          678          BEQ     RESET_389
E64C  EE 04          679          LDX     $04,X
E64E  7D 70 00       680          TST     $7000
E651  01             681          NOP
E652  6F 00          682          CLR     $00,X
E654  DE F0          683          LDX     $F0
E656  20 03          684          BRA     RESET_38C
E658                 685  RESET_389
E658  6C 07          686          INC     $07,X
E65A  39             687          RTS
E65B                 688  RESET_38C
E65B  A6 06          689          LDAA    $06,X
E65D  E6 01          690          LDAB    $01,X
E65F  5D             691          TSTB
E660  26 09          692          BNE     RESET_39C
E662                 693  RESET_393
E662  81 2D          694          CMPA    #$2D
```

```
E664  26 02          695          BNE     RESET_399
E666  6F 06          696          CLR     $06,X
E668                 697  RESET_399
E668  6F 07          698          CLR     $07,X
E66A  39             699          RTS
E66B                 700  RESET_39C
E66B  C1 4F          701          CMPB    #$4F
E66D  24 F3          702          BCC     RESET_393
E66F  8D 22          703          BSR     POSADR
E671  A6 06          704          LDAA    $06,X
E673  EE 02          705          LDX     $02,X
E675  7D 70 00       706          TST     $7000
E678  01             707          NOP
E679  6D 00          708          TST     $00,X
E67B  27 03          709          BEQ     RESET_3B1
E67D  7E E4 00       710          JMP     RESET_131
E680                 711  RESET_3B1
E680  7D 70 00       712          TST     $7000
E683  01             713          NOP
E684  A7 00          714          STAA    $00,X
E686  39             715          RTS
E687                 716  RESET_3B8
E687  6D 00          717          TST     $00,X
E689  2B 04          718          BMI     RESET_3C0
E68B  6C 01          719          INC     $01,X
E68D  20 B9          720          BRA     RESET_379
E68F                 721  RESET_3C0
E68F  6A 01          722          DEC     $01,X
E691  20 B5          723          BRA     RESET_379
E693                 724  POSADR
E693  A6 00          725          LDAA    $00,X
E695  84 7F          726          ANDA    #$7F
E697  16             727          TAB
E698  48             728          ASLA
E699  48             729          ASLA
E69A  1B             730          ABA
E69B  5F             731          CLRB
E69C  48             732          ASLA
E69D  59             733          ROLB
E69E  48             734          ASLA
E69F  59             735          ROLB
E6A0  48             736          ASLA
E6A1  59             737          ROLB
E6A2  48             738          ASLA
E6A3  59             739          ROLB
E6A4  AB 01          740          ADDA    $01,X
E6A6  C9 80          741          ADCB    #$80
E6A8  A7 03          742          STAA    $03,X
E6AA  E7 02          743          STAB    $02,X
E6AC  39             744          RTS
E6AD                 745  SCRDRAW
E6AD  7D 00 C8       746          FCB     $7D,$00,$C8
E6B0  2B 26          747          BMI     SCRDRAW_02B
E6B2  CE 87 31       748          LDX     #$8731
E6B5                 749  SCRDRAW_008
E6B5  7D 70 00       750          TST     $7000
E6B8  01             751          NOP
```

```
E6B9  A6 01          752          LDAA    $01,X
E6BB  7D 70 00       753          TST     $7000
E6BE  01             754          NOP
E6BF  A7 00          755          STAA    $00,X
E6C1  08             756          INX
E6C2  8C 87 7F       757          CPX     #$877F
E6C5  26 EE          758          BNE     SCRDRAW_008
E6C7  C6 01          759          LDAB    #$01
E6C9  D5 CE          760          BITB    $CE
E6CB  26 09          761          BNE     SCRDRAW_029
E6CD  7A 00 81 7A    762          FCB     $7A,$00,$81,$7A,$00,$89,$7A,$00
E6D5  91             763          FCB     $91
E6D6                 764  SCRDRAW_029
E6D6  20 25          765          BRA     SCRDRAW_050
E6D8                 766  SCRDRAW_02B
E6D8  CE 87 7E       767          LDX     #$877E
E6DB                 768  SCRDRAW_02E
E6DB  7D 70 00       769          TST     $7000
E6DE  01             770          NOP
E6DF  A6 00          771          LDAA    $00,X
E6E1  7D 70 00       772          TST     $7000
E6E4  01             773          NOP
E6E5  A7 01          774          STAA    $01,X
E6E7  09             775          DEX
E6E8  8C 87 30       776          CPX     #$8730
E6EB  26 EE          777          BNE     SCRDRAW_02E
E6ED  08             778          INX
E6EE  C6 01          779          LDAB    #$01
E6F0  D5 CE          780          BITB    $CE
E6F2  26 09          781          BNE     SCRDRAW_050
E6F4  7C 00 81 7C    782          FCB     $7C,$00,$81,$7C,$00,$89,$7C,$00
E6FC  91             783          FCB     $91
E6FD                 784  SCRDRAW_050
E6FD  C6 5F          785          LDAB    #$5F
E6FF  86 0A          786          LDAA    #$0A
E701  95 CE          787          BITA    $CE
E703  27 01          788          BEQ     SCRDRAW_059
E705  5A             789          DECB
E706                 790  SCRDRAW_059
E706  7D 70 00       791          TST     $7000
E709  01             792          NOP
E70A  E7 00          793          STAB    $00,X
E70C  CE 00 60       794          LDX     #$0060
E70F  DF F0          795          STX     $F0
E711                 796  SCRDRAW_064
E711  A6 02          797          LDAA    $02,X
E713  A7 04          798          STAA    $04,X
E715  A6 03          799          LDAA    $03,X
E717  A7 05          800          STAA    $05,X
E719  8D 06          801          BSR     NEXTOBJ
E71B  8C 00 C8       802          CPX     #$00C8
E71E  26 F1          803          BNE     SCRDRAW_064
E720  39             804          RTS
E721                 805  NEXTOBJ
E721  DE F0          806          LDX     $F0
E723  C6 08          807          LDAB    #$08
E725                 808  NEXTOBJ_004
```

```
E725  08            809          INX
E726  5A            810          DECB
E727  26 FC         811          BNE     NEXTOBJ_004
E729  DF F0         812          STX     $F0
E72B  39            813          RTS
E72C  DF F0         814          FCB     $DF,$F0
E72E                815  FINDOBJ
E72E  6D 06         816          TST     $06,X
E730  27 07         817          BEQ     FINDOBJ_00B
E732  8D ED         818          BSR     NEXTOBJ
E734  9C F2         819          CPX     $F2
E736  26 F6         820          BNE     FINDOBJ
E738  39            821          RTS
E739                822  FINDOBJ_00B
E739  A7 06         823          STAA    $06,X
E73B  81 2D         824          CMPA    #$2D
E73D  27 1E         825          BEQ     FINDOBJ_02F
E73F  96 F1         826          LDAA    $F1
E741  44            827          LSRA
E742  44            828          LSRA
E743  8A 40         829          ORAA    #$40
E745  A7 06         830          STAA    $06,X
E747  C6 0D         831          LDAB    #$0D
E749  81 60         832          CMPA    #$60
E74B  26 02         833          BNE     FINDOBJ_021
E74D  CA 80         834          ORAB    #$80
E74F                835  FINDOBJ_021
E74F  E7 00         836          STAB    $00,X
E751  86 68         837          LDAA    #$68
E753  7D 00 C8      838          FCB     $7D,$00,$C8
E756  2A 02         839          BPL     FINDOBJ_02C
E758  8A 80         840          ORAA    #$80
E75A                841  FINDOBJ_02C
E75A  A7 01         842          STAA    $01,X
E75C  39            843          RTS
E75D                844  FINDOBJ_02F
E75D  96 C8         845          LDAA    $C8
E75F  A7 00         846          STAA    $00,X
E761  96 C9         847          LDAA    $C9
E763  6D 00         848          TST     $00,X
E765  2B 01         849          BMI     FINDOBJ_03A
E767  4C            850          INCA
E768                851  FINDOBJ_03A
E768  A7 01         852          STAA    $01,X
E76A  39            853          RTS
E76B                854  PROCOBJ
E76B  A6 06         855          LDAA    $06,X
E76D  27 02         856          BEQ     PROCOBJ_006
E76F  8D 07         857          BSR     PROCOBJ_00D
E771                858  PROCOBJ_006
E771  8D AE         859          BSR     NEXTOBJ
E773  9C F2         860          CPX     $F2
E775  26 F4         861          BNE     PROCOBJ
E777  39            862          RTS
E778                863  PROCOBJ_00D
E778  E6 01         864          LDAB    $01,X
E77A  5D            865          TSTB
```

```
E77B  26 01          866          BNE     PROCOBJ_013
E77D                 867  PROCOBJ_012
E77D  39             868          RTS
E77E                 869  PROCOBJ_013
E77E  C1 4F          870          CMPB    #$4F
E780  24 FB          871          BCC     PROCOBJ_012
E782  86 2D          872          LDAA    #$2D
E784  C5 07          873          BITB    #$07
E786  26 F5          874          BNE     PROCOBJ_012
E788  6D 1E          875          TST     $1E,X
E78A  27 05          876          BEQ     PROCOBJ_026
E78C  6D 36          877          TST     $36,X
E78E  27 14          878          BEQ     PROCOBJ_039
E790  39             879          RTS
E791                 880  PROCOBJ_026
E791  A7 1E          881          STAA    $1E,X
E793  A6 00          882          LDAA    $00,X
E795  84 7F          883          ANDA    #$7F
E797  E6 01          884          LDAB    $01,X
E799  E7 19          885          STAB    $19,X
E79B  D1 C9          886          CMPB    $C9
E79D  25 02          887          BCS     PROCOBJ_036
E79F  8A 80          888          ORAA    #$80
E7A1                 889  PROCOBJ_036
E7A1  A7 18          890          STAA    $18,X
E7A3  39             891          RTS
E7A4                 892  PROCOBJ_039
E7A4  A7 36          893          STAA    $36,X
E7A6  A6 00          894          LDAA    $00,X
E7A8  84 7F          895          ANDA    #$7F
E7AA  E6 01          896          LDAB    $01,X
E7AC  E7 31          897          STAB    $31,X
E7AE  D1 C9          898          CMPB    $C9
E7B0  25 02          899          BCS     PROCOBJ_049
E7B2  8A 80          900          ORAA    #$80
E7B4                 901  PROCOBJ_049
E7B4  A7 30          902          STAA    $30,X
E7B6  39             903          RTS
E7B7  53 48 55 54    904          FCB     $53,$48,$55,$54,$54,$4C,$45,$20
E7BF  41 54 54 41    905          FCB     $41,$54,$54,$41,$43,$4B,$20,$56
E7C7  2E 32 43 4F    906          FCB     $2E,$32,$43,$4F,$50,$59,$52,$49
E7CF  47 48 54 20    907          FCB     $47,$48,$54,$20,$28,$43,$29,$20
E7D7  31 39 38 32    908          FCB     $31,$39,$38,$32,$55,$4E,$41,$55
E7DF  54 48 4F 52    909          FCB     $54,$48,$4F,$52,$49,$5A,$45,$44
E7E7  20 52 45 50    910          FCB     $20,$52,$45,$50,$52,$4F,$44,$55
E7EF  43 54 49 4F    911          FCB     $43,$54,$49,$4F,$4E,$20,$50,$52
E7F7  4F 48 49 42    912          FCB     $4F,$48,$49,$42,$49,$54,$45,$44
E7FF  2E             913          FCB     $2E
E800                 914
E800                 915          END
```

Appendix H

The two firmware sets compared

Two firmware sets were available for this book. **129867** is the one the book reads and lists (Appendix F). **129827** is an earlier set: the same design, at other addresses. Every row below that has a source in the text names its chapter. A cell that says “not checked” means exactly that.

Fixed points

What	129867	129827	Chapter
ROMs, in address order	`-67`, `-66`, `-65` at `$E800`, `$F000`, `$F800`	files `129827-149`, `-148`, `-147`	6
Size	6,144 bytes	6,144 bytes	26
IRQ vector (`$FFF8`)	`$ED83`	`$FA87`	4
SWI vector (`$FFFA`)	`$E800`	`$E900`	4
NMI vector (`$FFFC`)	`$E800`	`$FA87`	4
RESET vector (`$FFFE`)	`$E800`	`$E900`	4, 25
Exclusive-OR of all 6,144 bytes	`$00`	`$00`	26
Exclusive-OR from `$F000` to `$FFFF`	`$00`	`$59`	26

A program that has to restart the firmware reads the RESET vector (`LDX $FFFE`, `JMP 0,X`) and never hard-codes `$E800`.

Plug-in hooks

What	129867	129827	Chapter
Boot hook (`JSR` after `INC $0000`)	`$E07D`	`$E087`	5, 25
Stock boot continuation, no plug-in	`$E8D0`	`$E9D0`	5, 25
Configuration table, when the flag is set	`$E530`	`$E545`	25
Task-table hook, when the flag is set	`$E0CA`	`$E0C3`	25
Post-boot hook, when the flag is set	`$E08A`	`$E094`	25

What	129867	129827	Chapter
ESC ` entry	$E03C	$E03E	5, 28

The frame of Chapter 5 supplies all of them: three NOPs before the launch JMP, and an RTS at each hook.

Routines and tables

What	129867	129827	Chapter
XPLUSA	$EFAD, scratch $9F-$A0	$EE9C, scratch $CD-$CE	23
BELL	$F0DA	$FB8B	23
BLITCPY	$F6BB, source $B3, destination $B1	$F412, source $7F, destination $7D	23
KBDTAB (176 bytes)	$EA1A	$EB1B	9, 28
ESCTAB	$EACA	$EBCB (the same 176 bytes after KBDTAB; its + entry is $FD83)	28

Only the three routines in the first rows were located by searching the 129827 images for the 129867 instruction sequences; the rest of the 129827 was not disassembled for this book. In particular, the addresses of the display routines of Chapter 29 and of the serial routines of Chapter 30 in the 129827 are **not** listed here because they were not needed: the experiments of those chapters give the same screens on both sets by reading the VTAC registers and screen memory, which are the same on both (Chapter 29).

Zero page

The two sets keep the display state in different zero-page bytes. Chapter 29's table (CRTBASE at $11, FIRSTLN at $15, ...) is the 129867's. A program that must run on both sets reads the hardware (VTAC register 6 for the scroll position, for example), as FW31 and the tests of Chapters 28 to 31 do, or detects the set first (Chapter 23's T14AUTO tells them apart by the first byte of BELL: $B6 on the 129867).

What the experiments show

The keyboard, display and serial experiments of Chapters 28 and 29 and the differential tests of FW31 (Chapter 31) were run on the real ROM images of **both** sets, in the emulator, and gave the same screens. That is the evidence that the two sets behave alike for those functions. It is [EMU] evidence: it does not say how either set behaves on a terminal.

Appendix I

How each program was verified

This appendix is the book's own audit. For each program: what was checked, how, and what was not. "Emulator" means the emulator described at the end of this appendix, running the real firmware ROM images unless stated. Nothing in this appendix is a claim about a real terminal unless it says so.

The matrix

Program	Assembled	Ran in emulator	Both firmware sets	On a real ADM-31
T01 to T21 (tutorials)	yes, 0 errors; the binary equals that of a second, independent assembler	yes; every tutorial's screen or RAM is checked by `tests.py`	the launch is checked on both; T14FWCALL is 129867 only (Chapter 23)	not tried
PONG	yes, 0 errors (MD5 below)	yes: a deterministic run of twelve million instructions, sampled	yes: title screen on both	**yes: runs and plays** (the author's terminal)
FW31 (Chapter 31)	yes, 2,048 bytes	yes: differential tests against the real firmware, and plug-in launches	compared with both sets	**not tried**
Keyboard, display and serial experiments (Chapters 28 to 30)	scripted keystrokes	yes	yes, the same screens	not tried

Program	Assembled	Ran in emulator	Both firmware sets	On a real ADM-31
Rubout, Shuttle Attack	yes: the disassembly sources made for this book reassemble to the ROM files, byte for byte (0 errors)	not run for this book	not checked	**yes: runs** (programs by others)
Space Invaders, Cosmic Intruders, Outbreak	not rebuilt in this book's pipeline	not run for this book	not checked	**yes: runs** (Space Invaders and Cosmic Intruders by others; Outbreak is the author's); known here from images and scans only (Chapter 24)
Firmware 129867 (Appendix F)	yes: the reassembly equals the ROM, byte for byte	boots in the emulator	n/a	it is the firmware of real terminals

PONG.BIN, 2,048 bytes, MD5 `04dc28290ac9edd938f9382e00991dc0`.

Two details of the table. The Rubout and Shuttle Attack rows rest on the assembler and the listing format; all five plug-in ROMs have been verified to run on a real ADM-31, but this book's emulator ran none of them, and the book rebuilt only these two. The firmware row says that the *listing* is exact; it does not say that every comment in it is right.

What was measured, and what was not

- **The tick.** A "tick" in PONG is a count of loop passes. The processor clock was not measured, and the book never states a speed in seconds. (If the clock is 1 MHz, a PONG tick at the starting delay is a little over 51 ms; that is an *if*.)
- **The vertical-blank tick.** The firmware's scheduler has a tick bit in `PIA1B` (bit 1, read in Chapter 27). Whether it is the display's vertical sync is not verified.
- **Key auto-repeat.** The firmware's repeat intervals are 260 ticks for the first, 40 for the second, then 38, 36 and down to 16 (Chapter 28). Their duration in time depends on the unmeasured tick rate. The emulator holds a key for a fixed 20,000 instructions, so auto-repeat could not be demonstrated.
- **Switch banks.** The emulator reads zero from every switch bank (`$7900`, `$7D00`, `$7E00`, `$7E80`). Behaviour that depends on a switch setting, such as bit 7 of the serial data bytes (`$CE`, Chapter 30), is therefore the *zero-switch* behaviour, and is labelled so where it is described.
- **Output readback.** The emulator's `PIA1B` read does not return the output bits. The alpha-lock toggle and the display-page switch (`LF65E`) therefore could not be run, only read.
- **Board designators.** See Appendix B.

The corrections to older notes

Three facts differ from some older notes about this keyboard, and the text follows the firmware and the emulator runs:

1. **SHIFT, CTRL and alpha lock.** `SHIFT` is `PIA1B` bit 7, `CTRL` bit 6, and the alpha-lock latch (an output) bit 0.
2. **The launch key.** The firmware's `ESC` followed by code `$27` (the apostrophe) clears `PROTECT` (`CLR $0018`). The launch is `ESC` followed by `` ` `` (`$60`), the shifted form of the key that types `@` (scan `$3B`). The apostrophe is the shifted `7` (scan `$14`).
3. **Socket labels.** See Appendix B.

The emulator

The emulator is a C program (`bookemu`) derived from the emulator the author supplied. It models the 6800, RAM, ROM, the PIA, the CRT controller's registers, the ACIAs (as registers with a status flag), the keyboard matrix and a text rendering of the screen. It runs without a display, from a command line, and can dump RAM and the screen.

Three things were added for Chapters 28 to 31:

- an interrupt flag on the host ACIA, so that the receive and transmit interrupts of Chapter 30 run;
- `--txlog`, which writes every byte sent to the host ACIA to a file;
- `--rx`, which feeds bytes to the host ACIA's receive register.

Its limits, in one place: the PIA output bits cannot be read back; the switch banks read zero; the vertical-tick bit is never set, so every pass of the scheduler counts as a tick; a key is held for 20,000 instructions and then released; its ASCII key table covers 79 of the scan codes, so `BS`, `VT`, `FF` and `RS` must be pressed by their scan codes (`$54`, `$53`, `$55`, `$51`); it has no clock. The printer interface was read in the firmware and not run.

Troubleshooting a first run on hardware

If a plug-in does not start from `ESC` `` ` ``:

1. Is the keystroke right? The grave accent is the shifted `@` key. The apostrophe is a different key and does something else (clears protect).
2. Does `$E000` hold `T` and `J` (`$54`, `$4A`)? The launch routine tests nothing else.
3. Is the ROM in socket A, at `$E000`? Check the address decoding on your own board rather than the silk-screen label.
4. Does the program set the stack (`LDS #$03FF`) and mask interrupts (`SEI`) before using either? The firmware's scheduler keeps running unless the program stops it, and the firmware's interrupt handler uses the stack.
5. Does it touch `HALT` (`$7000`) before each screen access, as every working program does?

If the terminal does not boot with the ROM fitted: the boot hook (`$E07D` or `$E087`) must decline, clear `$0000`, drop the return address and jump to the stock boot (`$E8D0` or `$E9D0`). A hook that returns without doing so leaves the scheduler half configured (Chapter 25).

Appendix J

Glossary

ACIA. Asynchronous Communications Interface Adapter: the serial chip. The ADM-31 has one for the host (`$7A00`) and one for the printer (`$7C00`).

Accumulator. A or B: the 6800's two 8-bit working registers.

Alpha lock. A latch on the keyboard that makes letters capital without shifting the other keys; the output bit is `PIA1B` bit 0 (Chapter 28).

Assembler. A program that turns source text into machine code. This book uses the AcPx/acp6800 dialect.

Branch. A conditional jump of up to -128 to +127 bytes. For farther targets use `JMP`.

CRT RAM. The screen memory at `$8000`-`$8FFF`: one byte per character cell, 80 x 24 cells.

Direct mode. An addressing mode whose operand is an address below `$100`, one byte long. Not accepted for extended-mode instructions by this assembler; see Chapter 3.

ESC sequence. The character `ESC` (`$1B`) followed by one or more characters that name a function. Table in Appendix C.

Extended mode. An addressing mode with a 16-bit operand.

FIRSTLN. The firmware's variable that records which memory row is at the bottom of the screen; mirrored in VTAC register 6 (Chapter 29).

Frame. The standard 2 KB plug-in header of Chapter 5: signature `TJ`, launch entry, hooks.

HALT. The address `$7000`. A program touches it before every access to screen memory (Chapter 6).

Hook. An address in the plug-in at which the firmware calls the plug-in during boot (Chapter 25).

Index register. X, the 16-bit register used for tables and pointers.

Interrupt. An event that makes the processor save its state and run a handler. `SEI` masks IRQ.

KBDTAB. The firmware's translation table from scan codes to characters (Appendix D).

Listing. The assembler's printout: address, object bytes, line number, source (Appendix E and following).

PIA. Peripheral Interface Adapter: four registers at `$7800`-`$7803` that carry the keyboard scan, the modifier keys and the bell.

Plug-in. A 2 KB ROM in socket A, at `$E000`, that the firmware can start.

Protected cell. A screen cell whose bit 7 is set (Chapter 29); the clear routines skip it.

Scan code. The number of a key on the matrix. A program writes it to `PIA1` and reads back bit 7: set means the key is down.

Scheduler. The firmware's cooperative multitasking kernel (Chapter 27).

Stack. RAM used by `JSR`, `RTS`, `PSH` and `PUL`. The plug-ins in this book put it at `$03FF`.

Tick. One pass of PONG's main loop, or one pass of the firmware's scheduler. Its duration in time was not measured.

VTAC. The video timing and address controller; registers at `$7F00`-`$7F0E`.

Appendix K

The assembler: acpx.c

Every listing in this book was printed by one program, the author's ACPX assembler, and this appendix prints its complete C source: 1,777 lines of portable C. The file's own header comment calls it `acp6800.c` and describes it as an ADM-style program development system retargeted to the Motorola 6800. It is made of three parts, as its header says: **ACP**, the top-level menu shell; **ASSEDIT**, a line-oriented source editor; and **ASM**, the two-pass assembler whose listing, label list and error messages are the ones described in Chapter 3. Only the instruction set was replaced: the shell, editor and assembler follow the workflow of the ADM-2 development environment, and the target is the 6800.

Getting the source

Typing 1,777 lines by hand is possible, but it is not recommended. The same file is published on GitHub:

https://github.com/MickeyLawless88/ADM-ACP-6800/blob/main/acpx.c

Download it from there (use the *Raw* button, or the download icon) rather than copying from this page, where long lines have been set in a small type and a PDF reader may add line breaks or soft hyphens when text is selected.

Compiling it

The header of the source gives the build lines. It is plain ANSI C (C89), uses only standard input and output, and builds with GCC or Turbo C:

```
cc -o acp6800 acp6800.c          (GCC: add -ansi -pedantic)
tcc acp6800.c                    (Turbo C, large model suggested)
```

This book's listings were produced with a build of the same source under GCC on Linux, named `acpx`.

Using it for this book's programs

The program is driven from its menu. For every program in this book the sequence was the same: choose the assemble option (2), give the source file name, answer the *print label list* question, answer **Y** to *continue with assembly*, and at the last prompt give the size to which the binary is to be padded (2048 for a 2K ROM image). The assembler writes three files, named

after the first eight characters of the source name: a listing (`.LST`), a Motorola S-record file (`.S19`) and a binary image (`.BIN`). The binary runs from the lowest to the highest address the source defined; bytes in gaps inside that range are $00, and if the padding size asked for is larger the end of the file is filled with $FF.

Three properties of the assembler, all visible in the source below, explain the rules of Chapter 3: the expression evaluator works strictly left to right with no precedence; a comment-only line must start with a semicolon in column 1; and the listing's source field is 53 characters wide, so a longer line is cut and ends in >.

The source

The listing is printed in a small type so that its longest lines (117 characters) fit on the page without being broken. It is the file exactly as published, with tab characters expanded to the next multiple of 8.

```
/* ========================================================================
 * acp6800.c  -  ADM-style program development system, retargeted to the
 *               Motorola 6800.
 *
 * A faithful recreation of the Lear Siegler ADM-2 MICOS development
 * environment described in Section 5 of the ADM-2 microprogramming manual:
 *
 *     ACP      - Assembly Control Program (the top-level menu shell)
 *     ASSEDIT  - the line-oriented source file editor
 *     ASM      - the two-pass assembler
 *
 * The ADM-2 microcode instruction set has been REPLACED by the standard
 * Motorola 6800 assembly language.  The shell, editor and two-pass
 * assembler workflow / listing / label-list / error-message behaviour are
 * preserved from the manual; only the target ISA differs.
 *
 * Portable ANSI C (C89).  Builds unchanged under Turbo C (DOS) and GCC.
 * Pure stdio: runs on a console or piped over a tty / serial line.
 *
 *     cc -o acp6800 acp6800.c              (GCC: add -ansi -pedantic)
 *     tcc acp6800.c                        (Turbo C, large model suggested)
 * ======================================================================== */
#include <time.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>

/* getcwd lives in different headers per toolchain; used only to tell the
 * user where output files were written */
#if defined(__TURBOC__) || defined(__BORLANDC__)
#  include <dir.h>
#  define HAVE_GETCWD 1
#elif defined(__unix__) || defined(__APPLE__) || defined(__linux__)
#  include <unistd.h>
#  define HAVE_GETCWD 1
#endif

/* ---- portable types ------------------------------------------------- */
typedef unsigned char  BYTE;
typedef unsigned int   WORD;     /* 16 bits is enough; masked to 0xFFFF   */

/* ---- limits --------------------------------------------------------- */
#define ACP_VERSION "1.3"        /* bump on every change so stale binaries  */
                                 /* are identifiable from the banner        */

#define MAXLINES  3000           /* records per source file               */
#define LINELEN   128            /* characters per source line            */
#define NAMELEN   16
#define MAXSYM    2000
#define MAXREF    12000          /* total symbol references tracked for XREF */
#define OBJMAX    64             /* max object bytes a single line can emit */
```

```
/* Listing geometry.  LST_LINEMAX is only the DEFAULT width; the effective
 * width is the runtime `list_width`, adjustable from the ACP shell.  A width
 * of 0 means "unlimited" - no source clipping and no XREF reference wrap.
 * The fixed columns are ADDR(4) + 2 + CODE(12) + 2 + LINE(4) + 2 = 26.     */
#define LST_LINEMAX  79
#define LST_PREFIX   26
#define LST_SRCMAX   (LST_LINEMAX - LST_PREFIX)   /* default source columns  */

static int list_width = LST_LINEMAX;   /* effective width; 0 => unlimited    */

/* One format string drives every data row so the columns can never drift.
 * Field widths: ADDR 4, CODE 12 (fixed by the .12 precision so over-long
 * code can't push later columns), LINE 4.  Header/rule below must match.  */
#define LST_ROW_FMT  "%04X  %-12.12s  %4d  %s\n"

/* Header and rule share the same field widths as the data row (4/12/4 with
 * two-space gaps) so the labels sit exactly over their columns.           */
#define LST_HDR_FMT  "%-4s  %-12s  %-4s  %s\n"

/* The large tables are heap-allocated at start-up rather than declared as
 * static arrays.  Turbo C (16-bit) forbids any single static/auto array
 * larger than 64 KB; allocating keeps each request comfortably under that
 * limit and behaves identically under GCC. */

/* ======================================================================== */
/*  small portable helpers                                                  */
/* ======================================================================== */

static char *xstrdup(const char *s)
{
    char *p = (char *)malloc(strlen(s) + 1);
    if (p) strcpy(p, s);
    return p;
}

static void rstrip(char *s)
{
    int n = (int)strlen(s);
    while (n > 0 && (s[n-1] == '\n' || s[n-1] == '\r' ||
                     s[n-1] == ' '  || s[n-1] == '\t')) {
        s[--n] = '\0';
    }
}

static void upcase(char *s)
{
    for (; *s; s++) *s = (char)toupper((unsigned char)*s);
}

/* read one line from the terminal; returns 0 on EOF */
static int getline_p(const char *prompt, char *buf, int size)
{
    if (prompt) { fputs(prompt, stdout); fflush(stdout); }
    if (fgets(buf, size, stdin) == NULL) { buf[0] = '\0'; return 0; }
    rstrip(buf);
    return 1;
}

/* ======================================================================== */
/*  source file buffer (shared by editor and assembler)                     */
/*                                                                          */
/*  Record 1 is reserved for the system and is never displayed, exactly     */
/*  as in the manual.  User text begins at line 2.  Internally lines[0]     */
/*  holds the reserved record; lines[1] displays as line 2, and so on.      */
/* ======================================================================== */

static char **gl_lines;          /* malloc'd: MAXLINES char* slots          */
static int   gl_count;           /* number of slots used incl. reserved     */

static void buf_clear(void)
{
    int i;
    for (i = 0; i < gl_count; i++) {
        if (gl_lines[i]) { free(gl_lines[i]); gl_lines[i] = NULL; }
    }
    gl_count = 1;                /* reserve record 1                        */
    gl_lines[0] = xstrdup("");
```

```
}

/* display-line number n (>=2) -> internal index, or -1 if out of range   */
static int idx_of(int dispno)
{
    int i = dispno - 1;          /* line 2 -> index 1                      */
    if (i < 1 || i >= gl_count) return -1;
    return i;
}

static int last_dispno(void) { return gl_count; } /* highest display no.  */

/* load file <name> into the buffer; returns record count (user lines) or
 * -1 if the file does not exist                                          */
static int buf_load(const char *name)
{
    FILE *f = fopen(name, "r");
    char tmp[LINELEN];
    buf_clear();
    if (!f) return -1;
    while (fgets(tmp, sizeof(tmp), f)) {
        rstrip(tmp);
        if (gl_count >= MAXLINES) break;
        gl_lines[gl_count++] = xstrdup(tmp);
    }
    fclose(f);
    return gl_count - 1;
}

static int buf_save(const char *name)
{
    FILE *f = fopen(name, "w");
    int i;
    if (!f) return -1;
    for (i = 1; i < gl_count; i++) {
        fputs(gl_lines[i] ? gl_lines[i] : "", f);
        fputc('\n', f);
    }
    fclose(f);
    return 0;
}

/* append text as a new last line */
static int buf_append(const char *text)
{
    if (gl_count >= MAXLINES) return -1;
    gl_lines[gl_count++] = xstrdup(text);
    return 0;
}

/* replace the text of display line dispno */
static int buf_replace(int dispno, const char *text)
{
    int i = idx_of(dispno);
    if (i < 0) return -1;
    free(gl_lines[i]);
    gl_lines[i] = xstrdup(text);
    return 0;
}

/* insert one line before display line dispno (existing line slides down) */
static int buf_insert(int dispno, const char *text)
{
    int i = dispno - 1;
    int k;
    if (gl_count >= MAXLINES) return -1;
    if (i < 1) i = 1;
    if (i > gl_count) i = gl_count;
    for (k = gl_count; k > i; k--) gl_lines[k] = gl_lines[k-1];
    gl_lines[i] = xstrdup(text);
    gl_count++;
    return 0;
}

/* delete display lines first..last inclusive; returns count deleted */
static int buf_delete(int first, int last)
{
```

```
    int fi = first - 1, li = last - 1;
    int n, k;
    if (fi < 1) fi = 1;
    if (li >= gl_count) li = gl_count - 1;
    if (li < fi) return 0;
    n = li - fi + 1;
    for (k = fi; k <= li; k++) if (gl_lines[k]) free(gl_lines[k]);
    for (k = li + 1; k < gl_count; k++) gl_lines[k - n] = gl_lines[k];
    gl_count -= n;
    return n;
}

/* ====================================================================== */
/*  ASSEDIT - the source file editor                                      */
/* ====================================================================== */

static int valid_filename(const char *s)
{
    const char *base, *ext, *p;
    int baselen, extlen, i;

    if (!s || !*s) return 0;

    /* Strip off any drive or directory prefix; validate only the final component. */
    base = s;
    for (p = s + strlen(s) - 1; p >= s; p--) {
        if (*p == ':' || *p == '\\' || *p == '/') {
            base = p + 1;
            break;
        }
    }

    /* Find an optional extension separator in the final component. */
    ext = NULL;
    for (p = base; *p; p++) {
        if (*p == '.') {
            if (ext) return 0;           /* more than one dot in final component */
            ext = p + 1;
        }
    }
    if (ext && *ext == '\0') return 0;  /* trailing dot not allowed */

    /* Validate the base name part (1-8 chars, starts with letter, alphanumeric). */
    for (p = base; *p && *p != '.'; p++) { }
    baselen = (int)(p - base);
    if (baselen < 1 || baselen > 8) return 0;
    if (!isalpha((unsigned char)base[0])) return 0;
    for (i = 0; i < baselen; i++)
        if (!isalnum((unsigned char)base[i])) return 0;

    /* Validate the optional extension (1-3 alphanumeric chars). */
    if (ext) {
        extlen = 0;
        for (p = ext; *p; p++) {
            if (!isalnum((unsigned char)*p)) return 0;
            extlen++;
        }
        if (extlen < 1 || extlen > 3) return 0;
    }

    return 1;
}

/* enter source lines starting at display line `start`, terminated by "//" */
static void enter_lines(int start)
{
    char in[LINELEN];
    int  no = start;
    char prompt[16];
    for (;;) {
        sprintf(prompt, "%5d   ", no);
        if (!getline_p(prompt, in, sizeof(in))) break;
        if (strncmp(in, "//", 2) == 0) break;
        if (buf_append(in) < 0) {
            printf("FILE FULL\n");
            break;
        }
```

```
        no++;
    }
}

static void edit_list(int from, int to)
{
    int i;
    if (to > last_dispno()) to = last_dispno();
    for (i = from; i <= to; i++) {
        int x = idx_of(i);
        if (x < 0) continue;
        printf("%5d   %s\n", i, gl_lines[x]);
    }
    if (to >= last_dispno()) printf("END OF FILE\n");
}

static void edit_change(int dispno)
{
    char in[LINELEN], prompt[16];
    int x = idx_of(dispno);
    if (x < 0) { printf("NO SUCH LINE\n"); return; }
    printf("%5d   %s\n", dispno, gl_lines[x]);
    sprintf(prompt, "%5d   ", dispno);
    if (!getline_p(prompt, in, sizeof(in))) return;
    if (strncmp(in, "//", 2) == 0) return;
    buf_replace(dispno, in);
}

static void edit_insert(int nlines, int before)
{
    char in[LINELEN], prompt[16];
    int  at = before, made = 0;
    printf("OPENING THE FILE FOR INSERTION OF %d LINES\n", nlines);
    printf("READY FOR INSERTION\n");
    while (made < nlines) {
        sprintf(prompt, "%5d   ", at);
        if (!getline_p(prompt, in, sizeof(in))) break;
        if (strncmp(in, "//", 2) == 0) break;
        buf_insert(at, in);
        at++; made++;
    }
}

static void edit_delete(int first, int last)
{
    char in[LINELEN];
    int  i, n;
    printf("ARE THESE THE CORRECT LINE(S)   (Y/N)?\n");
    for (i = first; i <= last; i++) {
        int x = idx_of(i);
        if (x >= 0) printf("%5d   %s\n", i, gl_lines[x]);
    }
    if (!getline_p("...", in, sizeof(in))) return;
    upcase(in);
    if (in[0] != 'Y') { printf("DELETE IGNORED\n"); return; }
    n = buf_delete(first, last);
    printf("DELETING %d LINES\n", n);
    printf("DELETE COMPLETE\n");
}

static void edit_search(const char *str)
{
    int i, hits = 0;
    for (i = 2; i <= last_dispno(); i++) {
        int x = idx_of(i);
        if (x >= 0 && strstr(gl_lines[x], str)) {
            printf("%5d   %s\n", i, gl_lines[x]);
            hits++;
        }
    }
    if (!hits) printf("STRING NOT FOUND\n");
}

static void edit_menu(void)
{
    printf("\n           READY FOR EDITING\n");
    printf("A    APPEND TO FILE\n");
```

```
    printf("C    CHANGE LINE\n");
    printf("D    DELETE LINE\n");
    printf("I    INSERT LINE BEFORE\n");
    printf("L    DISPLAY LINE(S)\n");
    printf("S    SEARCH FOR STRING BETWEEN LINES\n");
    printf("T    DISPLAY FILE\n");
    printf("X    EXIT FROM EDITOR\n");
}

/* edit-mode command loop on the file already loaded as `name` */
static void edit_mode(const char *name)
{
    char in[LINELEN], arg[LINELEN];
    int  a, b, n;
    edit_menu();
    for (;;) {
        if (!getline_p("...", in, sizeof(in))) break;
        if (in[0] == '\0') continue;
        switch (toupper((unsigned char)in[0])) {
        case 'A':
            printf("%s READY FOR INPUT\n", name);
            enter_lines(last_dispno() + 1);
            break;
        case 'C':
            { char *p = strchr(in, '#');
              if (p) a = atoi(p + 1);
              else { getline_p("# LINE-NO. ", arg, sizeof(arg)); a = atoi(arg); }
              edit_change(a); }
            break;
        case 'D':
            getline_p("FROM LINE, TO LINE ", arg, sizeof(arg));
            if (sscanf(arg, "%d ,%d", &a, &b) < 2 &&
                sscanf(arg, "%d,%d", &a, &b) < 2) { a = b = atoi(arg); }
            edit_delete(a, b);
            break;
        case 'I':
            getline_p("# OF LINES TO INSERT BEFORE LINE # n,line ", arg, sizeof(arg));
            if (sscanf(arg, "%d,%d", &n, &a) < 2 &&
                sscanf(arg, "%d ,%d", &n, &a) < 2) { n = 1; a = atoi(arg); }
            edit_insert(n, a);
            break;
        case 'L':
            getline_p("FROM LINE, TO LINE ", arg, sizeof(arg));
            if (sscanf(arg, "%d,%d", &a, &b) < 2 &&
                sscanf(arg, "%d ,%d", &a, &b) < 2) { a = atoi(arg); b = a; }
            edit_list(a, b);
            break;
        case 'S':
            getline_p("STR ", arg, sizeof(arg));
            edit_search(arg);
            break;
        case 'T':
            edit_list(2, last_dispno());
            break;
        case 'X':
            buf_save(name);
            printf("%s SAVED\n", name);
            return;
        case '?':
            edit_menu();
            break;
        default:
            printf("CMD ERR -- TYPE ? FOR LIST\n");
            break;
        }
    }
    buf_save(name);
}

/* ASSEDIT command-mode menu */
static void assedit_menu(void)
{
    printf("\n'ASSEDIT' HERE -- WHAT DO YOU WANT TO DO?\n");
    printf("C    CREATE A NEW SOURCE FILE.\n");
    printf("A    APPEND TO AN EXISTING SOURCE FILE.\n");
    printf("E    EDIT AN EXISTING SOURCE FILE.\n");
    printf("D    DUPLICATE A SOURCE FILE.\n");
```

```
    printf("S    SHRINK/EXPAND A SOURCE FILE\n");
    printf("M    MERGE TWO FILES\n");
    printf("(CR) EXIT FROM EDITOR.\n");
}

static void cmd_create(void)
{
    char name[LINELEN];
    printf("NAME THE NEW ADM SOURCE FILE\n");
    if (!getline_p("", name, sizeof(name))) return;
    upcase(name);
    if (!valid_filename(name)) {
        printf("NAME MUST BE 1-8 ALPHANUMERIC, BEGIN WITH A LETTER\n");
        return;
    }
    buf_clear();
    printf("%s READY FOR INPUT\n", name);
    enter_lines(2);
    buf_save(name);
    printf("%s SAVED\n", name);
}

static void cmd_append(void)
{
    char name[LINELEN];
    printf("WHAT IS THE NAME OF THE FILE TO WHICH YOU WISH TO APPEND?\n");
    if (!getline_p("", name, sizeof(name))) return;
    upcase(name);
    if (buf_load(name) < 0) { printf("FILE NOT FOUND\n"); return; }
    printf("%s READY FOR INPUT\n", name);
    enter_lines(last_dispno() + 1);
    buf_save(name);
    printf("%s SAVED\n", name);
}

static void cmd_edit(void)
{
    char name[LINELEN];
    int  recs;
    if (!getline_p("EDIT WHAT FILE ? ", name, sizeof(name))) return;
    upcase(name);
    recs = buf_load(name);
    if (recs < 0) { printf("FILE NOT FOUND\n"); return; }
    printf("%d RECORDS ON FILE %s\n", recs, name);
    edit_mode(name);
}

static void cmd_duplicate(void)
{
    char nw[LINELEN], old[LINELEN];
    int  recs;
    getline_p("NEW FILE NAME  ? ", nw, sizeof(nw));   upcase(nw);
    getline_p("OLD FILE NAME  ? ", old, sizeof(old)); upcase(old);
    recs = buf_load(old);
    if (recs < 0) { printf("FILE NOT FOUND\n"); return; }
    buf_save(nw);
    printf("DUPLICATING %s TO %s   RECORD COUNT = %d\n", old, nw, recs);
}

static void cmd_shrink(void)
{
    char name[LINELEN];
    if (!getline_p("FILE NAME ? ", name, sizeof(name))) return;
    upcase(name);
    if (buf_load(name) < 0) { printf("FILE NOT FOUND\n"); return; }
    printf("THERE ARE %d RECORDS ON FILE %s\n", last_dispno() - 1, name);
    printf("FILE %s HAS BEEN COMPACTED\n", name);
    buf_save(name);
}

/* merge lines L1..L2 of file1 into file2 after a given line */
static void cmd_merge(void)
{
    char file1[LINELEN], file2[LINELEN], arg[LINELEN];
    char **seg;
    int  l1, l2, after, i, segn = 0, recs2, segcap;
```

```
    getline_p("SOURCE VOLUME 'XXXX' ? ", file1, sizeof(file1)); upcase(file1);
    if (buf_load(file1) < 0) { printf("FILE NOT FOUND\n"); return; }
    getline_p("TAKE DATA BETWEEN LINES 'L1' & 'L2' (L1,L2) ? ", arg, sizeof(arg));
    if (sscanf(arg, "%d,%d", &l1, &l2) < 2) { printf("BAD RANGE\n"); return; }
    segcap = (l2 >= l1) ? (l2 - l1 + 1) : 1;
    seg = (char **)malloc((unsigned)segcap * sizeof(char *));
    if (!seg) { printf("OUT OF MEMORY\n"); return; }
    for (i = l1; i <= l2; i++) {
        int x = idx_of(i);
        if (x >= 0) seg[segn++] = xstrdup(gl_lines[x]);
    }

    getline_p("DESTINATION FILE ? ", file2, sizeof(file2)); upcase(file2);
    recs2 = buf_load(file2);
    if (recs2 < 0) { printf("FILE NOT FOUND\n"); free(seg); return; }
    getline_p("MERGE AFTER LINE 'L' ? ", arg, sizeof(arg));
    after = atoi(arg);
    printf("FILE %s CONTAINS %d RECORDS\n", file2, recs2);
    printf("WRITING DATA INTO FILE %s\n", file2);
    for (i = 0; i < segn; i++) {
        buf_insert(after + 1 + i, seg[i]);
        free(seg[i]);
    }
    free(seg);
    buf_save(file2);
    printf("MERGE COMPLETE\n");
    printf("FILE %s NOW CONTAINS %d RECORDS\n", file2, last_dispno() - 1);
}

static void assedit(void)
{
    char in[LINELEN];
    assedit_menu();
    for (;;) {
        if (!getline_p("? ", in, sizeof(in))) return;
        if (in[0] == '\0') return;                /* CR -> exit           */
        switch (toupper((unsigned char)in[0])) {
        case 'C': cmd_create();    break;
        case 'A': cmd_append();    break;
        case 'E': cmd_edit();      break;
        case 'D': cmd_duplicate(); break;
        case 'S': cmd_shrink();    break;
        case 'M': cmd_merge();     break;
        default:  printf("CMD ERR -- TYPE A LISTED COMMAND\n"); break;
        }
        assedit_menu();
    }
}

/* ====================================================================== */
/*  6800 ASSEMBLER                                                        */
/* ====================================================================== */

#define M_NONE 0
#define M_INH  1                /* inherent / accumulator, 1 byte         */
#define M_IMM  2                /* immediate                              */
#define M_DIR  3                /* direct (zero page)                     */
#define M_IND  4                /* indexed  n,X                           */
#define M_EXT  5                /* extended                               */
#define M_REL  6                /* relative branch                        */

#define NOOP   (-1)
#define F_IMM16 1               /* immediate operand is 16 bits           */

typedef struct {
    char  mnem[6];
    short inh, imm, dir, ind, ext, rel;
    short flags;
} OPC;

/* Standard MC6800 opcode table (opcodes in hex).  -1 = mode unavailable. */
static OPC optab[] = {
 /* mnem   inh   imm   dir   ind   ext   rel  flags */
 {"ABA", 0x1B, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"ADCA",NOOP, 0x89, 0x99, 0xA9, 0xB9, NOOP, 0},
 {"ADCB",NOOP, 0xC9, 0xD9, 0xE9, 0xF9, NOOP, 0},
 {"ADDA",NOOP, 0x8B, 0x9B, 0xAB, 0xBB, NOOP, 0},
```

```
{"ADDB",NOOP, 0xCB, 0xDB, 0xEB, 0xFB, NOOP, 0},
{"ANDA",NOOP, 0x84, 0x94, 0xA4, 0xB4, NOOP, 0},
{"ANDB",NOOP, 0xC4, 0xD4, 0xE4, 0xF4, NOOP, 0},
{"ASL", NOOP, NOOP, NOOP, 0x68, 0x78, NOOP, 0},
{"ASLA",0x48, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"ASLB",0x58, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"ASR", NOOP, NOOP, NOOP, 0x67, 0x77, NOOP, 0},
{"ASRA",0x47, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"ASRB",0x57, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"BCC", NOOP, NOOP, NOOP, NOOP, NOOP, 0x24, 0},
{"BCS", NOOP, NOOP, NOOP, NOOP, NOOP, 0x25, 0},
{"BEQ", NOOP, NOOP, NOOP, NOOP, NOOP, 0x27, 0},
{"BGE", NOOP, NOOP, NOOP, NOOP, NOOP, 0x2C, 0},
{"BGT", NOOP, NOOP, NOOP, NOOP, NOOP, 0x2E, 0},
{"BHI", NOOP, NOOP, NOOP, NOOP, NOOP, 0x22, 0},
{"BITA",NOOP, 0x85, 0x95, 0xA5, 0xB5, NOOP, 0},
{"BITB",NOOP, 0xC5, 0xD5, 0xE5, 0xF5, NOOP, 0},
{"BLE", NOOP, NOOP, NOOP, NOOP, NOOP, 0x2F, 0},
{"BLS", NOOP, NOOP, NOOP, NOOP, NOOP, 0x23, 0},
{"BLT", NOOP, NOOP, NOOP, NOOP, NOOP, 0x2D, 0},
{"BMI", NOOP, NOOP, NOOP, NOOP, NOOP, 0x2B, 0},
{"BNE", NOOP, NOOP, NOOP, NOOP, NOOP, 0x26, 0},
{"BPL", NOOP, NOOP, NOOP, NOOP, NOOP, 0x2A, 0},
{"BRA", NOOP, NOOP, NOOP, NOOP, NOOP, 0x20, 0},
{"BSR", NOOP, NOOP, NOOP, NOOP, NOOP, 0x8D, 0},
{"BVC", NOOP, NOOP, NOOP, NOOP, NOOP, 0x28, 0},
{"BVS", NOOP, NOOP, NOOP, NOOP, NOOP, 0x29, 0},
{"CBA", 0x11, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"CLC", 0x0C, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"CLI", 0x0E, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"CLR", NOOP, NOOP, NOOP, 0x6F, 0x7F, NOOP, 0},
{"CLRA",0x4F, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"CLRB",0x5F, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"CLV", 0x0A, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"CMPA",NOOP, 0x81, 0x91, 0xA1, 0xB1, NOOP, 0},
{"CMPB",NOOP, 0xC1, 0xD1, 0xE1, 0xF1, NOOP, 0},
{"COM", NOOP, NOOP, NOOP, 0x63, 0x73, NOOP, 0},
{"COMA",0x43, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"COMB",0x53, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"CPX", NOOP, 0x8C, 0x9C, 0xAC, 0xBC, NOOP, F_IMM16},
{"DAA", 0x19, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"DEC", NOOP, NOOP, NOOP, 0x6A, 0x7A, NOOP, 0},
{"DECA",0x4A, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"DECB",0x5A, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"DES", 0x34, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"DEX", 0x09, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"EORA",NOOP, 0x88, 0x98, 0xA8, 0xB8, NOOP, 0},
{"EORB",NOOP, 0xC8, 0xD8, 0xE8, 0xF8, NOOP, 0},
{"INC", NOOP, NOOP, NOOP, 0x6C, 0x7C, NOOP, 0},
{"INCA",0x4C, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"INCB",0x5C, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"INS", 0x31, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"INX", 0x08, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"JMP", NOOP, NOOP, NOOP, 0x6E, 0x7E, NOOP, 0},
{"JSR", NOOP, NOOP, NOOP, 0xAD, 0xBD, NOOP, 0},
{"LDAA",NOOP, 0x86, 0x96, 0xA6, 0xB6, NOOP, 0},
{"LDAB",NOOP, 0xC6, 0xD6, 0xE6, 0xF6, NOOP, 0},
{"LDS", NOOP, 0x8E, 0x9E, 0xAE, 0xBE, NOOP, F_IMM16},
{"LDX", NOOP, 0xCE, 0xDE, 0xEE, 0xFE, NOOP, F_IMM16},
{"LSR", NOOP, NOOP, NOOP, 0x64, 0x74, NOOP, 0},
{"LSRA",0x44, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"LSRB",0x54, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
/* LSL / LSLA / LSLB are synonyms for ASL / ASLA / ASLB */
{"LSL", NOOP, NOOP, NOOP, 0x68, 0x78, NOOP, 0},
{"LSLA",0x48, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"LSLB",0x58, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"NEG", NOOP, NOOP, NOOP, 0x60, 0x70, NOOP, 0},
{"NEGA",0x40, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"NEGB",0x50, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"NOP", 0x01, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"ORAA",NOOP, 0x8A, 0x9A, 0xAA, 0xBA, NOOP, 0},
{"ORAB",NOOP, 0xCA, 0xDA, 0xEA, 0xFA, NOOP, 0},
{"PSHA",0x36, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"PSHB",0x37, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"PULA",0x32, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"PULB",0x33, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
{"ROL", NOOP, NOOP, NOOP, 0x69, 0x79, NOOP, 0},
```

```
 {"ROLA",0x49, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"ROLB",0x59, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"ROR", NOOP, NOOP, NOOP, 0x66, 0x76, NOOP, 0},
 {"RORA",0x46, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"RORB",0x56, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"RTI", 0x3B, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"RTS", 0x39, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"SBA", 0x10, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"SBCA",NOOP, 0x82, 0x92, 0xA2, 0xB2, NOOP, 0},
 {"SBCB",NOOP, 0xC2, 0xD2, 0xE2, 0xF2, NOOP, 0},
 {"SEC", 0x0D, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"SEI", 0x0F, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"SEV", 0x0B, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"STAA",NOOP, NOOP, 0x97, 0xA7, 0xB7, NOOP, 0},
 {"STAB",NOOP, NOOP, 0xD7, 0xE7, 0xF7, NOOP, 0},
 {"STS", NOOP, NOOP, 0x9F, 0xAF, 0xBF, NOOP, 0},
 {"STX", NOOP, NOOP, 0xDF, 0xEF, 0xFF, NOOP, 0},
 {"SUBA",NOOP, 0x80, 0x90, 0xA0, 0xB0, NOOP, 0},
 {"SUBB",NOOP, 0xC0, 0xD0, 0xE0, 0xF0, NOOP, 0},
 {"SWI", 0x3F, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"TAB", 0x16, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"TAP", 0x06, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"TBA", 0x17, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"TPA", 0x07, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"TST", NOOP, NOOP, NOOP, 0x6D, 0x7D, NOOP, 0},
 {"TSTA",0x4D, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"TSTB",0x5D, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"TSX", 0x30, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"TXS", 0x35, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"WAI", 0x3E, NOOP, NOOP, NOOP, NOOP, NOOP, 0},
 {"",    NOOP, NOOP, NOOP, NOOP, NOOP, NOOP, 0}
};

static OPC *find_opc(const char *m)
{
    int i;
    for (i = 0; optab[i].mnem[0]; i++)
        if (strcmp(optab[i].mnem, m) == 0) return &optab[i];
    return NULL;
}

/* ---- symbol table ------------------------------------------------- */
#define ST_LAB 0                /* address label (code or initialised data)*/
#define ST_EQU 1                /* EQU constant (absolute value)           */
#define ST_MEM 2                /* RMB reserved storage (a variable)       */
typedef struct { char name[NAMELEN]; long val; int defined; int defline;
                 int type; } SYM;
static SYM *symtab;             /* malloc'd: MAXSYM entries               */
static int nsym;

/* ---- cross-reference store ---------------------------------------- *
 * One (symbol,line) pair per operand use.  Filled only during pass 2 so a
 * symbol is not counted twice (pass 1 also evaluates expressions).  The
 * pairs are sorted and de-duplicated when the XREF table is printed.       */
typedef struct { int sym; int line; } XREF;
static XREF *xref;              /* malloc'd: MAXREF entries               */
static int  nxref;
static int  xref_on;            /* 1 => eval() records references          */
static int  xref_curline;       /* display line eval() is working on       */
static int  g_defline;          /* display line sym_define() is defining on  */
static int  g_symtype;          /* kind (ST_*) of the symbol being defined   */

static SYM *sym_find(const char *name)
{
    int i;
    for (i = 0; i < nsym; i++)
        if (strcmp(symtab[i].name, name) == 0) return &symtab[i];
    return NULL;
}

static int sym_define(const char *name, long val)
{
    SYM *s = sym_find(name);
    if (s) {
        if (s->defined) return -1;     /* already exists */
        s->val = val; s->defined = 1;
        return 0;
```

```
    }
    if (nsym >= MAXSYM) return -2;
    strncpy(symtab[nsym].name, name, NAMELEN - 1);
    symtab[nsym].name[NAMELEN - 1] = '\0';
    symtab[nsym].val = val;
    symtab[nsym].defined = 1;
    symtab[nsym].defline = g_defline;
    symtab[nsym].type    = g_symtype;
    nsym++;
    return 0;
}

static const char *sym_type_name(int t)
{
    if (t == ST_EQU) return "EQU";
    if (t == ST_MEM) return "MEM";
    return "LAB";
}

/* record one operand reference to sym index while assembling xref_curline */
static void xref_note(int symidx)
{
    if (!xref_on || nxref >= MAXREF) return;
    xref[nxref].sym  = symidx;
    xref[nxref].line = xref_curline;
    nxref++;
}

/* ---- expression evaluator ---------------------------------------- *
 * Supports: $hex  %binary  'c char  decimal  symbol  *(=loc counter)
 *           binary + and - between terms, left to right, optional sign.   *
 * Returns 1 if fully resolved, 0 if an undefined symbol was referenced.   */
static int eval(const char *s, WORD loc, long *out)
{
    long acc = 0, term;
    int  op = '+', resolved = 1, any = 0;
    char tok[NAMELEN];
    const char *p = s;

    while (*p) {
        while (*p == ' ' || *p == '\t') p++;
        if (*p == '+' || *p == '-') { op = *p++; continue; }
        if (*p == '\0') break;
        term = 0;
        any = 1;
        if (*p == '*') {
            term = (long)loc; p++;
        } else if (*p == '$') {
            p++;
            while (isxdigit((unsigned char)*p)) {
                int d = isdigit((unsigned char)*p) ? *p - '0'
                        : (toupper((unsigned char)*p) - 'A' + 10);
                term = term * 16 + d; p++;
            }
        } else if (*p == '%') {
            p++;
            while (*p == '0' || *p == '1') { term = term * 2 + (*p - '0'); p++; }
        } else if (*p == '\'') {
            p++;
            if (*p) { term = (unsigned char)*p; p++; }
        } else if (isdigit((unsigned char)*p)) {
            while (isdigit((unsigned char)*p)) { term = term * 10 + (*p - '0'); p++; }
        } else if (isalpha((unsigned char)*p) || *p == '_' || *p == '.') {
            int n = 0;
            while ((isalnum((unsigned char)*p) || *p == '_' || *p == '.') &&
                   n < NAMELEN - 1) tok[n++] = (char)toupper((unsigned char)*p++);
            tok[n] = '\0';
            { SYM *sy = sym_find(tok);
              if (sy) xref_note((int)(sy - symtab));
              if (sy && sy->defined) term = sy->val;
              else resolved = 0; }
        } else {
            p++;             /* skip stray character */
            continue;
        }
        if (op == '+') acc += term; else acc -= term;
    }
```

```
    if (!any) resolved = 0;
    *out = acc & 0xFFFFL;
    return resolved;
}

/* ---- parse a source line into label / mnemonic / operand ------------- *
 * Column 1 non-blank, non-'*' => label.  '*' in column 1 => comment.      */
static void parse_line(const char *src, char *lab, char *mn, char *opnd)
{
    const char *p = src;
    int n;
    lab[0] = mn[0] = opnd[0] = '\0';
    if (*p == '*' || *p == ';') return;            /* full-line comment    */

    if (*p && *p != ' ' && *p != '\t') {           /* label in column 1    */
        n = 0;
        while (*p && *p != ' ' && *p != '\t' && n < NAMELEN - 1)
            lab[n++] = (char)toupper((unsigned char)*p++);
        lab[n] = '\0';
        if (n > 0 && lab[n-1] == ':') lab[n-1] = '\0';  /* allow trailing : */
    }
    while (*p == ' ' || *p == '\t') p++;
    n = 0;                                          /* mnemonic             */
    while (*p && *p != ' ' && *p != '\t' && n < NAMELEN - 1)
        mn[n++] = (char)toupper((unsigned char)*p++);
    mn[n] = '\0';
    while (*p == ' ' || *p == '\t') p++;
    n = 0;                                          /* operand field        */
    /* Data directives (FCC and the byte/word lists FCB/DB, FDB/DW) may
     * contain quoted strings with embedded spaces, so for these we keep
     * the rest of the line verbatim instead of stopping at the first space.
     * A trailing comment introduced by an unquoted ';' is dropped. */
    if (strcmp(mn, "FCC") == 0 || strcmp(mn, "FCB") == 0 ||
        strcmp(mn, "DB")  == 0 || strcmp(mn, "FDB") == 0 ||
        strcmp(mn, "DW")  == 0) {
        int in_quote = 0; char q = 0;
        while (*p && n < LINELEN - 1) {
            if (in_quote) {
                if (*p == q) in_quote = 0;
            } else {
                if (*p == '"' || *p == '\'') { in_quote = 1; q = *p; }
                else if (*p == ';') break;          /* start of comment    */
            }
            opnd[n++] = *p++;
        }
        /* strip trailing whitespace left before a dropped comment */
        while (n > 0 && (opnd[n-1] == ' ' || opnd[n-1] == '\t')) n--;
    } else {
        while (*p && *p != ' ' && *p != '\t' && n < LINELEN - 1)
            opnd[n++] = *p++;
    }
    opnd[n] = '\0';
}

/* Count the bytes an FCB/DB (or, *2, FDB/DW) operand list will emit.
 * This MUST agree byte-for-byte with the pass-2 emitter below, including
 * backslash escape collapsing inside quoted strings, or every following
 * address will be wrong.  Items are comma-separated; a quoted "..." or
 * '...' contributes one byte per (post-escape) character, a bare term
 * contributes one byte. */
static int count_items(const char *s)
{
    int  n = 0;
    const char *p = s;

    while (*p == ' ' || *p == '\t') p++;
    if (*p == '\0') return 0;

    for (;;) {
        while (*p == ' ' || *p == '\t') p++;
        if (*p == '"' || *p == '\'') {        /* quoted string item        */
            char delim = *p++;
            while (*p && *p != delim) {
                if (*p == '\\' && *(p+1)) p++; /* escape -> one byte        */
                n++; p++;
            }
            if (*p == delim) p++;
```

```
            while (*p && *p != ',') p++;        /* skip to separator         */
        } else {                                /* bare numeric/label item   */
            while (*p && *p != ',') p++;
            n++;
        }
        if (*p == ',') { p++; continue; }
        break;
    }
    return n;
}

/* split an indexed operand "expr,X" -> returns 1 and copies expr; else 0 */
static int is_indexed(const char *opnd, char *expr)
{
    int n = (int)strlen(opnd);
    if (n >= 2 && (opnd[n-1] == 'X' || opnd[n-1] == 'x') && opnd[n-2] == ',') {
        strncpy(expr, opnd, n - 2);
        expr[n - 2] = '\0';
        return 1;
    }
    return 0;
}

/* assembled-line record produced by pass 1 (kept small: no text copies,
 * pass 2 re-parses the source line from the buffer) */
typedef struct {
    int           dispno;
    WORD          addr;
    unsigned char mode;          /* M_* */
    unsigned char is_dir;        /* DIR_* */
    short         size;          /* object bytes */
    OPC          *op;
} ALINE;

#define DIR_NONE 0
#define DIR_ORG  1
#define DIR_EQU  2
#define DIR_FCB  3
#define DIR_FDB  4
#define DIR_FCC  5
#define DIR_RMB  6
#define DIR_END  7

static int directive_of(const char *mn)
{
    if (!strcmp(mn, "ORG")) return DIR_ORG;
    if (!strcmp(mn, "EQU")) return DIR_EQU;
    if (!strcmp(mn, "FCB")) return DIR_FCB;
    if (!strcmp(mn, "DB"))  return DIR_FCB;   /* DB = define byte  (FCB)  */
    if (!strcmp(mn, "FDB")) return DIR_FDB;
    if (!strcmp(mn, "DW"))  return DIR_FDB;   /* DW = define word  (FDB)  */
    if (!strcmp(mn, "FCC")) return DIR_FCC;
    if (!strcmp(mn, "RMB")) return DIR_RMB;
    if (!strcmp(mn, "END")) return DIR_END;
    return DIR_NONE;
}

static int fcc_len(const char *opnd)
{
    char delim;
    const char *p = opnd;
    int n = 0;
    if (!*p) return 0;
    delim = *p++;
    while (*p && *p != delim) { n++; p++; }
    return n;
}

/* ---- the two passes ------------------------------------------------- */

static ALINE *prog;              /* malloc'd: MAXLINES entries             */
static int   nprog;
static int   err_count;
static WORD  start_addr;
static int   have_start;

static void emit_err(FILE *lst, int dispno, const char *msg, const char *what)
```

```
{
    char line[160];
    int x = idx_of(dispno);
    const char *src = (x >= 0) ? gl_lines[x] : "";

    if (what && *what) sprintf(line, "*** ERROR (LINE %d): %s - %s", dispno, msg, what);
    else               sprintf(line, "*** ERROR (LINE %d): %s", dispno, msg);

    printf("%s\n", line);
    if (lst) {
        fprintf(lst, "%s\n", line);
        fprintf(lst, "    SOURCE: %s\n", src);
        fprintf(lst, "\n");
    }
    err_count++;
}

/* PASS 1: build symbol table and size every line */
static void pass1(void)
{
    char lab[NAMELEN], mn[NAMELEN], opnd[LINELEN], ex[LINELEN];
    int  i, dir;
    long v;
    WORD loc = 0;
    OPC *op;

    nprog = 0; nsym = 0; err_count = 0; have_start = 0;
    nxref = 0; xref_on = 0;             /* references are collected in pass 2 */

    for (i = 2; i <= last_dispno(); i++) {
        int x = idx_of(i);
        ALINE *a;
        const char *src = (x >= 0) ? gl_lines[x] : "";

        if (nprog >= MAXLINES) break;
        a = &prog[nprog++];
        a->dispno = i; a->addr = loc; a->mode = M_NONE; a->size = 0;
        a->op = NULL; a->is_dir = DIR_NONE;

        parse_line(src, lab, mn, opnd);
        g_defline = i;                 /* line any label here is defined on   */
        g_symtype = ST_LAB;            /* default; EQU and RMB override below  */

        if (mn[0] == '\0') {                /* blank / comment / label only */
            if (lab[0]) {
                if (sym_define(lab, (long)loc) == -1)
                    emit_err(NULL, i, "LABEL ALREADY EXISTS", lab);
            }
            continue;
        }

        dir = directive_of(mn);

        /* EQU defines its label from the operand, not from loc */
        if (dir == DIR_EQU) {
            a->is_dir = DIR_EQU;
            g_symtype = ST_EQU;
            if (!eval(opnd, loc, &v))
                emit_err(NULL, i, "OPERAND NOT FOUND", opnd);
            if (lab[0] && sym_define(lab, v) == -1)
                emit_err(NULL, i, "LABEL ALREADY EXISTS", lab);
            continue;
        }

        /* every other line: a leading label takes the current address */
        if (lab[0]) {
            if (dir == DIR_RMB) g_symtype = ST_MEM;   /* reserved storage    */
            if (sym_define(lab, (long)loc) == -1)
                emit_err(NULL, i, "LABEL ALREADY EXISTS", lab);
        }

        if (dir != DIR_NONE) {
            a->is_dir = dir;
            switch (dir) {
            case DIR_ORG:
                if (!eval(opnd, loc, &v))
                    emit_err(NULL, i, "OPERAND NOT FOUND", opnd);
```

```
                loc = (WORD)(v & 0xFFFF);
                a->addr = loc;
                if (!have_start) { start_addr = loc; have_start = 1; }
                break;
            case DIR_FCB: a->size = count_items(opnd);       break;
            case DIR_FDB: a->size = count_items(opnd) * 2;   break;
            case DIR_FCC: a->size = fcc_len(opnd);           break;
            case DIR_RMB:
                if (!eval(opnd, loc, &v))
                    emit_err(NULL, i, "OPERAND NOT FOUND", opnd);
                a->size = (int)v;
                break;
            case DIR_END: a->size = 0;                       break;
            default: break;
            }
            loc = (WORD)((loc + a->size) & 0xFFFF);
            continue;
        }

        /* a machine instruction */
        op = find_opc(mn);
        if (!op) { emit_err(NULL, i, "INSTRUCTION NOT FOUND", mn); a->size = 1; loc++; continue; }
        a->op = op;

        if (op->rel != NOOP) {                   /* branch               */
            a->mode = M_REL; a->size = 2;
        } else if (op->inh != NOOP) {            /* inherent: ignore any   */
            a->mode = M_INH; a->size = 1;        /* trailing comment text  */
        } else if (opnd[0] == '\0') {            /* needs an operand        */
            emit_err(NULL, i, "OPERAND MISSING IN SOURCE STATEMENT", mn);
            a->mode = M_INH; a->size = 1;
        } else if (opnd[0] == '#') {             /* immediate            */
            a->mode = M_IMM; a->size = (op->flags & F_IMM16) ? 3 : 2;
        } else if (is_indexed(opnd, ex)) {       /* indexed n,X          */
            a->mode = M_IND; a->size = 2;
        } else {                                 /* direct or extended    */
            int resolved = eval(opnd, loc, &v);
            if (op->dir != NOOP && resolved && v >= 0 && v <= 0xFF) {
                a->mode = M_DIR; a->size = 2;
            } else {
                a->mode = M_EXT; a->size = 3;    /* fwd ref defaults EXT  */
            }
        }
        if (!have_start) { start_addr = loc; have_start = 1; }
        loc = (WORD)((loc + a->size) & 0xFFFF);
    }
}

/* S-record output state */
static FILE *srec;
static BYTE  srbuf[32];
static int   srn;
static WORD  sraddr;

/* Binary output state.
 * The buffer is indexed by OFFSET from the program origin, not by absolute
 * address.  This keeps the buffer small enough for Turbo C (no >64K static
 * array) yet still captures high ROM addresses such as the ADM-31 plugin
 * slot at $E000-$E7FF.  A 32K window covers any single 6800 ROM image. */
static FILE *binfile;
static BYTE  binbuf[32768];     /* 32K window, indexed by (addr - bin_base) */
static long  bin_base;          /* absolute address that maps to binbuf[0] */
static long  bin_min_addr;      /* long: addresses exceed 16-bit signed int */
static long  bin_max_addr;

static void srec_flush(void)
{
    int i, sum, cnt;
    if (!srec || srn == 0) return;
    cnt = srn + 3;                      /* addr(2) + data + checksum(1)   */
    fprintf(srec, "S1%02X%04X", cnt, sraddr & 0xFFFF);
    sum = cnt + ((sraddr >> 8) & 0xFF) + (sraddr & 0xFF);
    for (i = 0; i < srn; i++) { fprintf(srec, "%02X", srbuf[i]); sum += srbuf[i]; }
    fprintf(srec, "%02X\n", (~sum) & 0xFF);
    srn = 0;
}
```

```
static void srec_byte(WORD addr, BYTE b)
{
    if (!srec) return;
    if (srn > 0 && (sraddr + srn) != addr) srec_flush();
    if (srn == 0) sraddr = addr;
    if (srn >= 16) { srec_flush(); sraddr = addr; }
    srbuf[srn++] = b;
}

static void srec_end(WORD start)
{
    int sum;
    srec_flush();
    if (!srec) return;
    sum = 3 + ((start >> 8) & 0xFF) + (start & 0xFF);
    fprintf(srec, "S903%04X%02X\n", start & 0xFFFF, (~sum) & 0xFF);
}

/* Binary output: collect bytes during assembly.
 * Stored at binbuf[addr - bin_base], so absolute addresses anywhere in the
 * 64K space are captured as long as the whole image fits in a 32K window. */
static void bin_byte(WORD addr, BYTE b)
{
    long off;
    if (!binfile) return;
    off = (long)addr - bin_base;
    if (off < 0 || off >= 32768L) return;   /* outside the 32K window */
    binbuf[off] = b;
    if (bin_min_addr > (long)addr) bin_min_addr = (long)addr;
    if (bin_max_addr < (long)addr) bin_max_addr = (long)addr;
}

/* Binary output: write collected bytes to file */
static void bin_flush(void)
{
    long i;
    if (!binfile || bin_min_addr > bin_max_addr) return;

    /* Write from min to max address, indexed by offset from base */
    for (i = bin_min_addr; i <= bin_max_addr; i++) {
        fputc(binbuf[i - bin_base], binfile);
    }
}

/* format up to 4 object bytes for the listing CODE column */
static void code_str(char *dst, BYTE *b, int n)
{
    int i, k = 0;
    dst[0] = '\0';
    for (i = 0; i < n && i < 4; i++) { sprintf(dst + k, "%02X ", b[i]); k += 3; }
    while (k < 12) dst[k++] = ' ';
    dst[k] = '\0';
}

/* Copy src into dst (buffer >= LINELEN+2), clipped so the whole listing line
 * stays within the effective list_width.  A shortened line ends in '>' to
 * flag the truncation.  When list_width is 0 (unlimited) the source is
 * copied in full and the row may run past 80 columns by design.            */
static const char *fit_src(char *dst, const char *src)
{
    int budget = (list_width > 0) ? list_width - LST_PREFIX : -1;
    int n = (int)strlen(src);
    if (budget < 0 || n <= budget) {        /* unlimited, or it already fits */
        strcpy(dst, src);
    } else if (budget <= 0) {               /* no room for any source at all */
        dst[0] = '\0';
    } else {
        memcpy(dst, src, (size_t)(budget - 1));
        dst[budget - 1] = '>';
        dst[budget]     = '\0';
    }
    return dst;
}

/* PASS 2: generate object code and the assembly listing */
static void pass2(FILE *lst)
{
```

```
int  i, j;
long v;
char code[16];
BYTE ob[OBJMAX];
int  nb;

xref_on = 1;                /* eval() now records symbol references     */

for (i = 0; i < nprog; i++) {
    ALINE *a = &prog[i];
    OPC *op = a->op;
    WORD here = a->addr;
    char ex[LINELEN];
    int  x = idx_of(a->dispno);
    const char *src = (x >= 0) ? gl_lines[x] : "";
    char lab[NAMELEN], mn[NAMELEN], opbuf[LINELEN];
    char lsrc[LINELEN + 2];      /* clipped (or full) src for the listing */
    const char *opnd;

    xref_curline = a->dispno;    /* references on this line credit here  */
    parse_line(src, lab, mn, opbuf);   /* parses the full, unclipped src */
    fit_src(lsrc, src);                /* clipped copy for the listing   */
    opnd = opbuf;
    nb = 0;

    if (a->is_dir == DIR_EQU || a->is_dir == DIR_ORG ||
        a->is_dir == DIR_END || (a->op == NULL && a->is_dir == DIR_NONE)) {
        /* no object bytes (comment, label-only, EQU, ORG, END) */
        code_str(code, ob, 0);
        if (a->is_dir == DIR_EQU) {
            eval(opnd, here, &v);
            sprintf(code, "=%04lX", v & 0xFFFF);
        }
        printf(LST_ROW_FMT, here, code, a->dispno, lsrc);
        if (lst) fprintf(lst, LST_ROW_FMT, here, code, a->dispno, lsrc);
        if (a->is_dir == DIR_END) break;
        continue;
    }

    if (a->is_dir == DIR_FCB) {
        char tmp[LINELEN]; const char *p = opnd; int k = 0;
        while (*p && nb < OBJMAX) {
            char *q = tmp; k = 0;

            /* Check for quoted string */
            if (*p == '"' || *p == '\'') {
                char delim = *p++;
                while (*p && *p != delim && nb < OBJMAX) {
                    if (*p == '\\' && *(p+1)) {
                        p++;
                        switch (*p) {
                        case 'n': ob[nb++] = 0x0A; break;
                        case 'r': ob[nb++] = 0x0D; break;
                        case 't': ob[nb++] = 0x09; break;
                        case '\\': ob[nb++] = 0x5C; break;
                        case '"': ob[nb++] = 0x22; break;
                        case '\'': ob[nb++] = 0x27; break;
                        default: ob[nb++] = (BYTE)*p; break;
                        }
                        p++;
                    } else {
                        ob[nb++] = (BYTE)*p++;
                    }
                }
                if (*p == delim) p++;
                /* Skip to next comma or end */
                while (*p && *p != ',') p++;
                if (*p == ',') p++;
            } else {
                /* Regular number/label operand */
                while (*p && *p != ',') { tmp[k++] = *p++; }
                tmp[k] = '\0'; if (*p == ',') p++;
                if (!eval(tmp, here, &v)) emit_err(lst, a->dispno, "OPERAND NOT FOUND", tmp);
                ob[nb++] = (BYTE)(v & 0xFF);
            }
            (void)q;
        }
```

```
        } else if (a->is_dir == DIR_FDB) {
            char tmp[LINELEN]; const char *p = opnd; int k;
            while (*p && nb < OBJMAX) {
                k = 0;
                while (*p && *p != ',') { tmp[k++] = *p++; }
                tmp[k] = '\0'; if (*p == ',') p++;
                if (!eval(tmp, here, &v)) emit_err(lst, a->dispno, "OPERAND NOT FOUND", tmp);
                ob[nb++] = (BYTE)((v >> 8) & 0xFF);
                ob[nb++] = (BYTE)(v & 0xFF);
            }
        } else if (a->is_dir == DIR_FCC) {
            const char *p = opnd; char delim;
            if (*p) { delim = *p++; while (*p && *p != delim && nb < OBJMAX) ob[nb++] = (BYTE)*p++; }
        } else if (a->is_dir == DIR_RMB) {
            /* reserve: emit no object, just advance the listing address */
            code_str(code, ob, 0);
            printf(LST_ROW_FMT, here, code, a->dispno, lsrc);
            if (lst) fprintf(lst, LST_ROW_FMT, here, code, a->dispno, lsrc);
            continue;
        } else if (op) {
            /* machine instruction */
            switch (a->mode) {
            case M_INH:
                ob[nb++] = (BYTE)op->inh;
                break;
            case M_IMM:
                ob[nb++] = (BYTE)op->imm;
                if (!eval(opnd + 1, here, &v)) emit_err(lst, a->dispno, "OPERAND NOT FOUND", opnd+1);
                if (op->flags & F_IMM16) { ob[nb++] = (BYTE)((v>>8)&0xFF); ob[nb++] = (BYTE)(v&0xFF); }
                else { if (v < 0 || v > 0xFF) emit_err(lst, a->dispno, "VALUE OUT OF BOUNDS", opnd+1);
                       ob[nb++] = (BYTE)(v & 0xFF); }
                break;
            case M_DIR:
                ob[nb++] = (BYTE)op->dir;
                if (!eval(opnd, here, &v)) emit_err(lst, a->dispno, "OPERAND NOT FOUND", opnd);
                ob[nb++] = (BYTE)(v & 0xFF);
                break;
            case M_IND:
                ob[nb++] = (BYTE)op->ind;
                is_indexed(opnd, ex);
                if (ex[0] == '\0') v = 0;
                else if (!eval(ex, here, &v)) emit_err(lst, a->dispno, "OPERAND NOT FOUND", ex);
                ob[nb++] = (BYTE)(v & 0xFF);
                break;
            case M_EXT:
                if (op->ext == NOOP) { emit_err(lst, a->dispno, "OPERAND MISSING IN SOURCE STATEMENT", ""); break; }
                ob[nb++] = (BYTE)op->ext;
                if (!eval(opnd, here, &v)) emit_err(lst, a->dispno, "OPERAND NOT FOUND", opnd);
                ob[nb++] = (BYTE)((v >> 8) & 0xFF);
                ob[nb++] = (BYTE)(v & 0xFF);
                break;
            case M_REL: {
                long target, off;
                ob[nb++] = (BYTE)op->rel;
                if (opnd[0] == '\0') { emit_err(lst, a->dispno, "OPERAND MISSING IN SOURCE STATEMENT", ""); break; }
                if (!eval(opnd, here, &target)) emit_err(lst, a->dispno, "OPERAND NOT FOUND", opnd);
                off = target - ((long)here + 2);
                if (off < -128 || off > 127)
                    emit_err(lst, a->dispno, "BRANCH OUT OF RANGE", opnd);
                ob[nb++] = (BYTE)(off & 0xFF);
                break; }
            default: break;
            }
        }

        for (j = 0; j < nb; j++) {
            srec_byte((WORD)(here + j), ob[j]);
            bin_byte((WORD)(here + j), ob[j]);
        }
        code_str(code, ob, nb);
        printf(LST_ROW_FMT, here, code, a->dispno, lsrc);
        if (lst) fprintf(lst, LST_ROW_FMT, here, code, a->dispno, lsrc);
    }
}

/* write s to the terminal and, when open, to the listing file */
static void xout(FILE *lst, const char *s)
```

```
{
    fputs(s, stdout);
    if (lst) fputs(s, lst);
}

/* Symbol table, up to five entries across.  Each cell is NAME(8) + value and
 * occupies a 15-column stride (with the 2-space indent), so the number that
 * fits is list_width/15, capped at the requested 5 (5 => 75 cols).          */
static void print_labels(FILE *lst)
{
    int  i, col = 0;
    int  percol = (list_width > 0) ? list_width / 15 : 5;
    char line[128], cell[32];

    if (percol > 5) percol = 5;
    if (percol < 1) percol = 1;

    xout(lst, "\nSYMBOL TABLE\n");
    strcpy(line, "  ");                     /* leading indent               */
    for (i = 0; i < nsym; i++) {
        sprintf(cell, "%-8.8s %04lX", symtab[i].name, symtab[i].val & 0xFFFF);
        strcat(line, cell);
        if (++col == percol) {              /* row full -> flush            */
            strcat(line, "\n"); xout(lst, line);
            strcpy(line, "  "); col = 0;
        } else {
            strcat(line, "  ");             /* gap before next cell         */
        }
    }
    if (col > 0) { strcat(line, "\n"); xout(lst, line); }
}

/* ---- cross-reference table ------------------------------------------- *
 * Alphabetical list of every symbol with its kind, value, defining line,
 * reference count, and every line that references it.  Reference lists wrap
 * under the REFERENCES column to honour the effective list_width (0 = no
 * wrap, subject only to the output-buffer cap).                            *
 * Columns: SYMBOL 14, TYPE 3, VAL 4, DEF 5, CNT 4, then refs; 2-space gaps. */
#define XREF_HDR_FMT  "%-14s  %-3s  %-4s  %-5s  %4s  %s\n"
#define XREF_PREFIX   40           /* chars before the REFERENCES field     */
#define XREF_TOKW     6            /* width of one "Lnnnn " reference token  */
#define XREF_LINEBUF  512          /* build buffer for one physical row     */

/* references per physical line for the current width (>= 1, buffer-capped) */
static int xref_perline(void)
{
    int cap = (XREF_LINEBUF - XREF_PREFIX - 2) / XREF_TOKW;   /* buffer limit */
    int n;
    if (list_width <= 0) return cap;                          /* unlimited    */
    n = (list_width - XREF_PREFIX) / XREF_TOKW;
    if (n < 1)   n = 1;
    if (n > cap) n = cap;
    return n;
}

static int xref_cmp(const void *a, const void *b)
{
    const XREF *x = (const XREF *)a, *y = (const XREF *)b;
    if (x->sym != y->sym) return x->sym - y->sym;
    return x->line - y->line;
}

static int symname_cmp(const void *a, const void *b)
{
    return strcmp(symtab[*(const int *)a].name, symtab[*(const int *)b].name);
}

static int xref_lower(int symidx)   /* first xref[] slot with sym >= symidx  */
{
    int lo = 0, hi = nxref;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        if (xref[mid].sym < symidx) lo = mid + 1; else hi = mid;
    }
    return lo;
}
```

```
static void print_xref(FILE *lst)
{
    int  *order;
    int   k, j;
    int   perline = xref_perline();
    char  line[XREF_LINEBUF], tok[16], defstr[16], refrule[64];

    if (nsym == 0) return;

    /* group references by symbol, ascending line within each symbol */
    if (nxref > 1) qsort(xref, (size_t)nxref, sizeof(XREF), xref_cmp);

    /* print symbols alphabetically without disturbing the symbol table */
    order = (int *)malloc((unsigned)nsym * sizeof(int));
    if (!order) return;
    for (k = 0; k < nsym; k++) order[k] = k;
    qsort(order, (size_t)nsym, sizeof(int), symname_cmp);

    xout(lst, "\nCROSS-REFERENCE TABLE\n");
    sprintf(line, XREF_HDR_FMT, "SYMBOL", "TYP", "VAL", "DEF", "CNT",
            "REFERENCES");
    xout(lst, line);
    {
        int nd = perline * XREF_TOKW;        /* underline the usable ref area */
        if (nd < 4)  nd = 4;
        if (nd > 48) nd = 48;
        memset(refrule, '-', (size_t)nd); refrule[nd] = '\0';
    }
    sprintf(line, XREF_HDR_FMT, "--------------", "---", "----", "-----",
            "----", refrule);
    xout(lst, line);

    for (k = 0; k < nsym; k++) {
        SYM *s = &symtab[order[k]];
        int  count = 0, emitted = 0, prev = -1;
        int  refs = 0;

        if (s->defline > 0) sprintf(defstr, "L%d", s->defline);
        else                strcpy(defstr, "-");

        /* first walk: how many distinct lines reference this symbol */
        for (j = xref_lower(order[k]); j < nxref && xref[j].sym == order[k]; j++) {
            if (xref[j].line == prev) continue;
            prev = xref[j].line; refs++;
        }
        prev = -1;

        /* first physical line carries SYMBOL / TYP / VAL / DEF / CNT */
        sprintf(line, "%-14.14s  %-3s  %04lX  %-5s  %4d  ",
                s->name, sym_type_name(s->type), s->val & 0xFFFF, defstr, refs);

        /* second walk: the reference tokens, wrapped under REFERENCES */
        for (j = xref_lower(order[k]); j < nxref && xref[j].sym == order[k]; j++) {
            if (xref[j].line == prev) continue;       /* one hit per line     */
            prev = xref[j].line;
            if (count == 0 && emitted) {              /* continuation indent  */
                memset(line, ' ', XREF_PREFIX);
                line[XREF_PREFIX] = '\0';
            }
            sprintf(tok, "L%d", xref[j].line);
            sprintf(line + strlen(line), "%-*s", XREF_TOKW, tok);
            if (++count == perline) {
                strcat(line, "\n"); xout(lst, line);
                emitted = 1; count = 0;
            }
        }
        if (count > 0) { strcat(line, "\n"); xout(lst, line); emitted = 1; }
        if (!emitted)  { strcat(line, "\n"); xout(lst, line); }
    }

    free(order);
}

static char last_listing[LINELEN] = "";

/* ACP option 2: assemble a program (TURBO C FIX) */
static void do_assemble(void)
```

```
{
    char name[LINELEN], base[12], lst_name[LINELEN], obj_name[LINELEN], bin_name[LINELEN], ans[128];
    FILE *lst;

    if (!getline_p("SOURCE FILE NAME ? ", name, sizeof(name))) return;
    upcase(name);
    if (buf_load(name) < 0) { printf("FILE NOT FOUND\n"); return; }

    /* Derive an 8.3-safe output base from the source name: drop any drive
     * or directory prefix and any extension, keep at most 8 chars.  This
     * keeps the output as e.g. SINE.LST / SINE.S19 even when the source was
     * entered as SINE.ASM or C:\SRC\SINE.ASM, so the DOS file create works. */
    {
        int s, k = 0;
        for (s = 0; name[s]; s++)
            if (name[s] == ':' || name[s] == '\\' || name[s] == '/') k = s + 1;
        s = k; k = 0;
        while (name[s] && name[s] != '.' && k < 8) base[k++] = name[s++];
        base[k] = '\0';
    }
    if (base[0] == '\0') strcpy(base, "OUT");
    sprintf(lst_name, "%s.LST", base);
    sprintf(obj_name, "%s.S19", base);
    sprintf(bin_name, "%s.BIN", base);

    printf("\nASSEMBLY PASS 1\n");
    pass1();
    printf("PASS 1 COMPLETE\n");
    printf("LAST ADDRESS USED IS %04X\n",
           nprog ? (WORD)(prog[nprog-1].addr + prog[nprog-1].size) : 0);
    printf("ERROR COUNT = %d\n", err_count);

    getline_p("PRINT LABEL LIST (Y) ? ", ans, 128);
    upcase(ans);

    printf("\nCONTINUE WITH ASSEMBLY (Y) ? ");
    if (!getline_p("", ans, 128)) return;
    upcase(ans);
    if (ans[0] == 'N') { printf("ASSEMBLY ABORTED\n"); return; }

    lst  = fopen(lst_name, "w");
    if (!lst)  printf("*** CANNOT CREATE LISTING FILE %s (%s)\n",
                      lst_name, strerror(errno));
    srec = fopen(obj_name, "w");
    if (!srec) printf("*** CANNOT CREATE OBJECT FILE %s (%s)\n",
                      obj_name, strerror(errno));
    binfile = fopen(bin_name, "wb");
    if (!binfile) printf("*** CANNOT CREATE BINARY FILE %s (%s)\n",
                      bin_name, strerror(errno));
    srn = 0;
    bin_min_addr = 0xFFFFL;
    bin_max_addr = 0L;
    bin_base = have_start ? (long)start_addr : 0L;  /* binbuf[0] = origin */

    printf("\nINSTRUCTION CODE GENERATOR FOR 6800 ASSEMBLER\n\n");
    if (lst) {
        fprintf(lst, "ASSEMBLY LISTING - SOURCE FILE %s\n\n", name);
        fprintf(lst, LST_HDR_FMT, "ADDR", "CODE", "LINE", "SOURCE STATEMENT");
        fprintf(lst, LST_HDR_FMT, "----", "------------", "----", "----------------");
    }
    printf(LST_HDR_FMT, "ADDR", "CODE", "LINE", "SOURCE STATEMENT");
    printf(LST_HDR_FMT, "----", "------------", "----", "----------------");

    pass2(lst);
    print_labels(lst);
    print_xref(lst);

    printf("\n****** END OF ASSEMBLY ******\n");
    printf("ERROR COUNT = %d\n", err_count);

    if (lst) {
        fprintf(lst, "\n");
        fprintf(lst, "=============================================================================\n");
        fprintf(lst, "ASSEMBLY SUMMARY\n");
        fprintf(lst, "=============================================================================\n");
        fprintf(lst, "TOTAL ERRORS: %d\n", err_count);
        if (err_count == 0) {
```

```
            fprintf(lst, "STATUS: ASSEMBLY SUCCESSFUL - NO ERRORS\n");
        } else {
            fprintf(lst, "STATUS: ASSEMBLY FAILED - %d ERROR(S) FOUND\n", err_count);
            fprintf(lst, "\nNote: Review error messages above (marked with ***).\n");
            fprintf(lst, "Each error shows:\n");
            fprintf(lst, "  - Error type and details\n");
            fprintf(lst, "  - Line number\n");
            fprintf(lst, "  - Source statement that caused the error\n");
        }
        fprintf(lst, "================================================================================\n");
        fclose(lst);
        strcpy(last_listing, lst_name);
        printf("LISTING WRITTEN TO %s\n", lst_name);
    } else {
        printf("LISTING NOT WRITTEN\n");
    }
    if (srec) {
        srec_end(have_start ? start_addr : 0);
        fclose(srec); srec = NULL;
        printf("OBJECT (S-RECORD) WRITTEN TO %s\n", obj_name);
    } else {
        printf("OBJECT NOT WRITTEN\n");
    }
    if (binfile) {
        bin_flush();
        fclose(binfile); binfile = NULL;
        if (bin_min_addr <= bin_max_addr) {
            int actual_size = (int)(bin_max_addr - bin_min_addr + 1);
            printf("BINARY %04lX-%04lX WRITTEN TO %s (%d bytes)\n",
                   bin_min_addr & 0xFFFFL, bin_max_addr & 0xFFFFL,
                   bin_name, actual_size);

            /* Ask for padding (TURBO C FIX) */
            printf("\nPAD BINARY TO SIZE (0=none, 2048=2K ROM, other): ");
            if (getline_p("", ans, 128)) {
                int pad_size = atoi(ans);
                if (pad_size > 0 && pad_size > actual_size) {
                    FILE *padfile = fopen(bin_name, "wb");
                    if (padfile) {
                        long a;
                        int j;
                        /* Write original code (offset-indexed) */
                        for (a = bin_min_addr; a <= bin_max_addr; a++) {
                            fputc(binbuf[a - bin_base], padfile);
                        }
                        /* Pad with 0xFF */
                        for (j = actual_size; j < pad_size; j++) {
                            fputc(0xFF, padfile);
                        }
                        fclose(padfile);
                        printf("PADDED BINARY WRITTEN TO %s (%d bytes, padded to %d)\n",
                               bin_name, actual_size, pad_size);
                    }
                }
            }
        } else {
            printf("BINARY NOT WRITTEN (no code)\n");
        }
    } else {
        printf("BINARY NOT WRITTEN\n");
    }
#ifdef HAVE_GETCWD
    {
        char cwd[96];
        if (getcwd(cwd, sizeof(cwd)) != NULL)
            printf("(IN DIRECTORY %s)\n", cwd);
    }
#endif
}

/* ACP option 3: show the last assembler output file on the terminal */
static void show_listing(void)
{
    char name[LINELEN], line[LINELEN];
    FILE *f;
    if (last_listing[0]) strcpy(name, last_listing);
    else { if (!getline_p("LISTING FILE ? ", name, sizeof(name))) return; upcase(name); }
```

```
    f = fopen(name, "r");
    if (!f) { printf("FILE NOT FOUND\n"); return; }
    while (fgets(line, sizeof(line), f)) fputs(line, stdout);
    fclose(f);
}

/* ACP option 4: set the maximum listing width (0 = unlimited) */
static void set_list_width(void)
{
    char ans[LINELEN];
    int  w, floor = XREF_PREFIX + 10;   /* room for the XREF header label     */

    if (list_width > 0) printf("\nLISTING WIDTH IS %d COLUMNS.\n", list_width);
    else                printf("\nLISTING WIDTH IS UNLIMITED.\n");
    printf("(72=TTY  80=CRT  132=PRINTER  0=UNLIMITED, RETURN=KEEP)\n");
    if (!getline_p("...WIDTH ? ", ans, sizeof ans)) return;
    rstrip(ans);
    if (ans[0] == '\0') { printf("WIDTH UNCHANGED.\n"); return; }

    w = atoi(ans);
    if (w < 0) w = 0;
    if (w > 0 && w < floor) {
        printf("TOO NARROW - MINIMUM IS %d COLUMNS (OR 0 FOR UNLIMITED).\n",
               floor);
        return;
    }
    list_width = w;
    if (list_width > 0) printf("LISTING WIDTH SET TO %d COLUMNS.\n", list_width);
    else                printf("LISTING WIDTH SET TO UNLIMITED.\n");
}

/* ======================================================================== */
/*  ACP - Assembly Control Program (top-level shell)                        */
/* ======================================================================== */

static void acp_menu(void)
{
   char w[24];
   if (list_width > 0) sprintf(w, "%d", list_width); else strcpy(w, "OFF");
   printf("\n'ACP' AT YOUR SERVICE - CHOOSE YOUR OPTION.\n");
   printf("============================================================\n");
   printf("1    SOURCE FILE EDITOR.\n");
   printf("2    ASSEMBLE PROGRAM.\n");
   printf("3    SHOW THE ASSEMBLER OUTPUT FILE ON THE TERMINAL\n");
   printf("4    SET LISTING WIDTH   (NOW %s)\n", w);
   printf("5    END\n");
}

int main(void)
{
    char in[LINELEN];

    gl_lines = (char **)malloc((unsigned)MAXLINES * sizeof(char *));
    prog     = (ALINE *)malloc((unsigned)MAXLINES * sizeof(ALINE));
    symtab   = (SYM  *)malloc((unsigned)MAXSYM   * sizeof(SYM));
    xref     = (XREF *)malloc((unsigned)MAXREF   * sizeof(XREF));
    if (!gl_lines || !prog || !symtab || !xref) {
        printf("OUT OF MEMORY - CANNOT START\n");
        return 1;
    }

    gl_count = 1;
    gl_lines[0] = xstrdup("");

{
        time_t      now = time(NULL);
        struct tm  *lt  = localtime(&now);
        char        datebuf[32];

        strftime(datebuf, sizeof datebuf, "%m/%d/%y", lt);   /* e.g. 06/20/26 */

        printf("\n");
        printf("\n");
        printf("ACCOUNT-ID: LSI01\n");
        printf("PASSWORD: XXXXX  %s \n", datebuf);
        printf("       GOOD DAY USER 4100  PORT 7\n");
        printf("           WELCOME TO THE WONDERFUL WORLD OF 'MICOS'\n");
```

```
        printf("************************************************************\n");
        printf("BELIEVE NOTHING YOU HEAR, AND ONLY ONE HALF THAT YOU SEE\n");
        printf("************************************************************\n");
        printf("##\n");
        printf("RUN'ACP'\n");
        printf("(ADM-ACP / ASSEDIT, MOTOROLA 6800 ASM)  V%s\n", ACP_VERSION);
        printf("============================================================\n");
        printf("ENTER \"BYE\" AT THE ACP PROMPT TO DISCONNECT.\n");
    }

    for (;;) {
        acp_menu();
        if (!getline_p("...? ", in, sizeof(in))) break;
        rstrip(in);
        if (in[0] == '\0') continue;

        { char up[LINELEN]; strcpy(up, in); upcase(up);
          if (strcmp(up, "BYE") == 0) { printf("BYE: IDLE\n"); break; } }

        switch (in[0]) {
        case '1': assedit();      break;
        case '2': do_assemble();  break;
        case '3': show_listing(); break;
        case '4': set_list_width(); break;
        case '5': printf("##\n"); printf("SIGNED OFF.\n"); return 0;
        default:  printf("INVALID OPTION - ENTER 1, 2, 3, 4 OR 5\n"); break;
        }
    }
    return 0;
}
```

Appendix L

The emulator: adm31emu.c

The programs in this book can be tried without a terminal on the desk. `adm31emu.c` is the author's ADM-31 emulator: a *functional* emulator (it is not cycle-exact) that runs 6800 plug-in ROMs, and the stock firmware around them, on a PC. This appendix prints its complete source (1,114 lines of C).

Getting the source

The same file is published on GitHub:

https://github.com/MickeyLawless88/ADM-ACP-6800/blob/main/adm31emu.c

Download it from there rather than typing it in. (The assembler, Appendix K, is in the same repository.)

What it emulates

As its header says: a complete MC6800 CPU core; the 1,024 bytes of RAM at `$0000-$03FF`; the I/O page `$7000-$7FFF` (the HALT/VSYNC strobe, PIA1 with the keyboard matrix, the ACIA, the CRT controller registers); the character RAM at `$8000-$88FF` as an 80 x 24 screen; and the four ROM sockets, `$E000` (the plug-in socket, ROM A) to `$FFFF`. A periodic interrupt stands in for the vertical retrace. What it does *not* reproduce is the hardware's video timing, the exact interrupt rate and cycle counts; none of these matter for a game that polls the keyboard and writes the screen, but a program that depends on exact timing must be checked on real hardware.

The source is written for Turbo C++ 1.0 under MS-DOS: the two 64 KB address spaces are allocated on the far heap, and the screen is the PC's text-mode video memory, written directly, one cell at a time, only where the character changed. The first part of the file (the CPU, memory and keyboard model, up to `reset_cpu`) is ordinary C and does not depend on the DOS code that follows it.

Using it

The command line (from the source):

option	meaning
`--plugin file.bin` or `--romA file.bin`	load the plug-in ROM at `$E000`
`--romB`, `--romC`, `--romD file.bin`	load a ROM at `$E800`, `$F000`, `$F800`
`--rom file@addr`	load a file at the hexadecimal address `addr`
`--stock`	load the fixed 129867 firmware from `ROM67.BIN`, `ROM66.BIN` and `ROM65.BIN` in the current directory, at `$E800`, `$F000` and `$F800`
`--boot plugin`	start the CPU at `$E000`: the plug-in alone, no firmware
`--boot game`	run the firmware's own start-up for 400,000 instructions, then jump to the plug-in at `$E03C`
`--boot firmware`	start through the reset vector, as the real machine does

PONG, with the stock firmware, is started with

```
ADM31EMU --stock --romA PONG.BIN --boot game
```

and, alone, with `ADM31EMU --plugin PONG.BIN --boot plugin`. A board screen shows the four sockets and what is in them; ENTER boots. The host keys are mapped onto the terminal's keyboard matrix: the arrow keys press the ADM-31 cursor keys (scan codes `$53` up, `$52` down, `$54` left, `$55` right), the space bar presses the key at scan code `$40` (which PONG accepts as a serve key), RETURN presses the numeric-pad ENTER key, and **ESC then Q** quits. The bottom line of the screen shows the CPU registers. When the CPU meets an opcode the emulator does not implement, it stops and prints it, for example `CPU HALTED: illegal/unimplemented opcode ($FD at $0073)`. An address in the zero page there means that the program ran into RAM; with the stock firmware a plug-in that does not clear the plug-in flag at `$0000` in its boot hook (Chapter 5) is the first thing to suspect.

What was checked for this book

The CPU, memory and keyboard code of this file was compiled unchanged on a Linux host, with only the DOS screen code left out, and `PONG.BIN` (MD5 `04dc28290ac9edd938f9382e00991dc0`) was run through both the `plugin` and the `game` boot paths with the stock firmware images. In both, the court and the serve prompt were drawn, SPACE served, the ball moved, and the arrow keys moved the bat to its limits at rows 2 and 17. The DOS executable itself was not run for this check.

The source

The listing is printed in a small type, so that its longest line (138 characters) fits the page without being broken. It is the file exactly as published, with tab characters (there are none) expanded to the next multiple of 8.

```
/* ========================================================================
 * adm31emu.c  -  Lear Siegler ADM-31 functional emulator (Motorola 6800)
 *
 * A functional (not cycle-exact) emulator for the ADM-31 terminal, intended
 * for running and testing 6800 plugin-ROM programs assembled with acp6800.
 *
 * It emulates:
 *   - a full MC6800 CPU core
 *   - 1K RAM            $0000-$03FF
 *   - I/O page          $7000-$7FFF  (HALT/VSYNC, PIA1, ACIA, VTAC, banks)
 *   - CRTRAM            $8000-$88FF  (80x24 character display)
```

```
 *   - ROM A (plugin)    $E000-$E7FF
 *   - ROM B / 67        $E800-$EFFF
 *   - ROM C / 65        $F000-$F7FF
 *   - ROM D / 66        $F800-$FFFF
 *
 * The host keyboard is fed to the ACIA receive register (the path RCCHR at
 * $F740 reads), and the CRTRAM is rendered with curses, so the firmware's
 * own SNDCHR/RCCHR routines drive real screen output and input.
 *
 * NOT emulated to hardware fidelity: VTAC video timing, exact IRQ cadence,
 * cycle counts.  These are not needed to run/test plugin programs.
 *
 * Build (interactive, curses TUI):
 *  DOS build (Turbo C++ 1.0):  tcc adm31dos.c
 * Build (headless self-test, no curses):
 *     cc -O2 -DHEADLESS -o adm31test adm31emu.c
 *
 * Usage:
 *     adm31emu --romA cint.bin --romB rom67.bin \
 *              --romC rom65.bin --romD rom66.bin [--boot firmware|plugin]
 *     adm31emu --plugin cint.bin --boot plugin        (run plugin alone)
 *     adm31emu --rom file@E000                         (load file at $E000)
 * ================================================================== */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* Turbo C++ 1.0 has no <stdint.h>; define the fixed-width types we use. */
typedef unsigned char  uint8_t;
typedef unsigned short uint16_t;
typedef unsigned long  uint32_t;
typedef signed   char  int8_t;
typedef signed   short int16_t;

/* (DOS build: no curses; the DOS front-end below includes <conio.h>/<dos.h>) */

/* ---- machine state ------------------------------------------------- */
/* DOS real mode forbids a single >=64K object.  Allocate the two 64K spaces
 * on the far heap and normalise each base to a paragraph boundary (offset 0)
 * so plain 'far' indexing [0..0xFFFF] stays inside one 64K segment -- this is
 * far faster than 'huge' (which re-normalises the pointer on every access). */
static uint8_t far *mem;            /* full 64K address space             */
static uint8_t far *is_rom;         /* 1 => writes ignored (ROM region)   */
static void far *mem_raw;           /* original blocks, kept for farfree  */
static void far *isrom_raw;

static uint8_t  A, B, CC;
static uint16_t IX, SP, PC;
static int      halted;
static unsigned long cycles;

/* VSYNC / IRQ generation.  A real ADM-31 drives screen refresh, cursor
 * blink and keyboard scan from a periodic vertical-retrace interrupt.
 * Terminal firmware typically does nothing visible until it receives one,
 * so we generate a periodic IRQ (gated by the I flag) and also expose a
 * toggling retrace bit on the HALT/VSYNC strobe for poll-style waits. */
static int           irq_enabled = 0;
static unsigned long irq_period  = 20000; /* instrs between VSYNC IRQs    */
static unsigned long irq_accum   = 0;
static uint8_t       halt_op;             /* last bad opcode (for display)*/
static uint16_t      halt_pc;             /* address of last bad opcode   */

/* condition-code bits */
#define FLAG_C 0x01
#define FLAG_V 0x02
#define FLAG_Z 0x04
#define FLAG_N 0x08
#define FLAG_I 0x10
#define FLAG_H 0x20
#define CC_HI  0xC0                 /* bits 6,7 always read as 1          */

/* ---- I/O addresses ------------------------------------------------- */
#define IO_HALT  0x7000
#define IO_PIA1  0x7800
#define IO_ACIA  0x7A00             /* $7A00 status/ctrl, $7A01 data      */
#define IO_VTAC  0x7F00

/* ---- CRTRAM display geometry --------------------------------------- */
#define CRTRAM_BASE 0x8000
#define SCR_COLS    80
#define SCR_ROWS    24

/* ---- ACIA receive queue (host keyboard -> 6800) -------------------- */
static unsigned char rxq[256];
static int rxq_head, rxq_tail;
static int rx_empty(void) { return rxq_head == rxq_tail; }
static void rx_push(unsigned char c)
{
    int nt = (rxq_tail + 1) & 0xFF;
```

```
    if (nt != rxq_head) { rxq[rxq_tail] = c; rxq_tail = nt; }
}
static unsigned char rx_pop(void)
{
    unsigned char c;
    if (rx_empty()) return 0;
    c = rxq[rxq_head];
    rxq_head = (rxq_head + 1) & 0xFF;
    return c;
}

/* tx capture (6800 -> host) - logged, not otherwise used here */
static FILE *txlog;

/* ---- PIA1 keyboard matrix ---------------------------------------------
 * Derived from the ADM-31 manual (sec 4.7) and verified against ROM67:
 *   - MPU writes a binary scan count to PIA Per.Reg.A ($7800).
 *     low 4 bits = column (4-to-16 decoder), high 3 bits = row (8-to-1 mux).
 *   - On a key at the selected intersection, STROBE feeds back as $7800 bit7.
 *   - SHIFT / CONTROL are read on Per.Reg.B ($7802): bit0 = SHIFT, bit6 = CTRL.
 * We hold one "pressed" key as a scan index (0..KBD_NSCAN-1) plus modifiers,
 * for a short number of scan cycles so the firmware's debounce accepts it. */
#define KBD_NSCAN 79
static const unsigned char kbd_tbl[KBD_NSCAN][2] = {
  {0xD1,0xD1},{0xD7,0xD7},{0xC5,0xC5},{0xD2,0xD2},
  {0xD4,0xF4},{0xD9,0xF9},{0x32,0x22},{0x1B,0x1B},
  {0x7F,0x7F},{0x9E,0x9F},{0xD0,0xF0},{0xB4,0xB6},
  {0x92,0x93},{0x31,0x21},{0x71,0x51},{0x98,0x98},
  {0x33,0x23},{0x34,0x24},{0x35,0x25},{0x36,0x26},
  {0x37,0x27},{0x38,0x28},{0x39,0x29},{0x30,0x5F},
  {0x2D,0x3D},{0x5E,0x7E},{0x5C,0x7C},{0x09,0xC9},
  {0x87,0x87},{0x88,0x88},{0x89,0x89},{0x2D,0x2D},
  {0x77,0x57},{0x65,0x45},{0x72,0x52},{0x74,0x54},
  {0x79,0x59},{0x75,0x55},{0x69,0x49},{0x6F,0x4F},
  {0x70,0x50},{0x5B,0x7B},{0x5D,0x7D},{0x0D,0x0D},
  {0x84,0x84},{0x85,0x85},{0x86,0x86},{0x09,0x09},
  {0x61,0x41},{0x73,0x53},{0x64,0x44},{0x66,0x46},
  {0x67,0x47},{0x68,0x48},{0x6A,0x4A},{0x6B,0x4B},
  {0x6C,0x4C},{0x3B,0x2B},{0x3A,0x2A},{0x40,0x60},
  {0x0A,0x0A},{0x82,0x82},{0x83,0x83},{0x81,0x81},
  {0x90,0x91},{0x7A,0x5A},{0x78,0x58},{0x63,0x43},
  {0x76,0x56},{0x62,0x42},{0x6E,0x4E},{0x6D,0x4D},
  {0x2C,0x3C},{0x2E,0x3E},{0x2F,0x3F},{0x20,0x20},
  {0x1F,0xCA},{0x30,0x30},{0x2E,0x2E},
};

static int     kbd_scan_idx = -1;      /* pressed key's scan index, -1=none  */
static int     kbd_shift = 0;          /* SHIFT held ($7802 bit0)            */
static int     kbd_ctrl  = 0;          /* CTRL  held ($7802 bit6)            */
static int     kbd_hold  = 0;          /* scan cycles remaining to hold key  */
static uint8_t pia_a_scan = 0;         /* last value MPU wrote to $7800      */

/* press an ASCII char: find its scan index + needed modifiers from the table */
/* press the numpad ENTER key (scan $4F) - distinct from main RETURN (scan 43).
 * Both decode to CR, but some programs (e.g. CINT) poll the numpad-ENTER line
 * directly, so the two are NOT interchangeable. */
static void kbd_press_numpad_enter(void)
{
    kbd_scan_idx = 0x4F; kbd_shift = 0; kbd_ctrl = 0; kbd_hold = 20000;
}

/* Press a key by its matrix SCAN index directly (not via ASCII lookup).
   Needed for the ADM-31 arrow-key cluster: several arrow scans share ASCII
   codes with control chars ($08 BS, $0A LF, $0B VT, $0C FF), so an ASCII
   lookup would land on the wrong matrix line. Games poll the matrix by scan,
   so we drive the scan index the game actually tests. */
static void kbd_press_scan(int scan)
{
    kbd_scan_idx = scan; kbd_shift = 0; kbd_ctrl = 0; kbd_hold = 20000;
}

/* ADM-31 cursor/arrow key scan codes (function-key cluster):
     scan $54 = cursor LEFT  (sends $08)
     scan $52 = cursor DOWN  (sends $0A)
     scan $53 = cursor UP    (sends $0B)
     scan $55 = cursor RIGHT (sends $0C) */
#define ADM_SCAN_LEFT   0x54
#define ADM_SCAN_DOWN   0x52
#define ADM_SCAN_UP     0x53
#define ADM_SCAN_RIGHT  0x55

/* FIRE: Shuttle Attack polls scan $40 (an ADM-31 function key, code $90) to
   launch the player's missile. There is no such key on a Mac keyboard, so map
   the host spacebar to that matrix line. */
#define ADM_SCAN_FIRE   0x40

static void kbd_press_ascii(unsigned char ch)
{
    int i;
```

```
    for (i = 0; i < KBD_NSCAN; i++) {
        if (kbd_tbl[i][0] == ch) { kbd_scan_idx = i; kbd_shift = 0; kbd_ctrl = 0; kbd_hold = 20000; return; }
    }
    for (i = 0; i < KBD_NSCAN; i++) {
        if (kbd_tbl[i][1] == ch) { kbd_scan_idx = i; kbd_shift = 1; kbd_ctrl = 0; kbd_hold = 20000; return; }
    }
    /* control chars $00-$1F: press the base key with CTRL held */
    if (ch < 0x20) {
        unsigned char base = (unsigned char)(ch | 0x40);   /* ^A -> 'A' etc */
        for (i = 0; i < KBD_NSCAN; i++) {
            if (kbd_tbl[i][0] == (base|0x20) || kbd_tbl[i][0] == base) {
                kbd_scan_idx = i; kbd_shift = 0; kbd_ctrl = 1; kbd_hold = 20000; return;
            }
        }
    }
}

/* ======================================================================== */
/*  memory access with I/O side effects                                     */
/* ======================================================================== */

static uint8_t rd8(uint16_t a)
{
    /* I/O page */
    if (a >= 0x7000 && a < 0x8000) {
        if (a == IO_ACIA) {                /* ACIA status                */
            uint8_t s = 0x02;              /* TDRE: always ready to send */
            if (!rx_empty()) s |= 0x01;    /* RDRF: receive data ready   */
            return s;
        }
        if (a == IO_ACIA + 1) {            /* ACIA data (read clears RDRF)*/
            return rx_pop();
        }
        if (a == IO_HALT) {                /* VSYNC strobe: toggling bit  */
            return ((cycles >> 13) & 1) ? 0x80 : 0x00;
        }
        if (a == 0x7800) {                 /* PIA1 Per.Reg.A: scan readback */
            uint8_t base = pia_a_scan & 0x7F;
            /* STROBE (bit7) high when the held key's scan index is selected */
            if (kbd_scan_idx >= 0 && kbd_hold > 0 &&
                base == (uint8_t)kbd_scan_idx) {
                return (uint8_t)(0x80 | base);
            }
            return base;
        }
        if (a == 0x7802) {                 /* PIA1 Per.Reg.B: SHIFT/CTRL    */
            uint8_t v = 0;
            if (kbd_scan_idx >= 0 && kbd_hold > 0) {
                if (kbd_shift) v |= 0x80;  /* SHIFT on bit7 (selects shifted */
                                           /* table variant via ASLB/ROLA)  */
                if (kbd_ctrl)  v |= 0x40;  /* CTRL  on bit6                 */
            }
            return v;
        }
        /* PIA1, VTAC, bank regs: return last value written (in mem[])     */
        return mem[a];
    }
    return mem[a];
}

static void wr8(uint16_t a, uint8_t v)
{
    if (a >= 0x7000 && a < 0x8000) {
        if (a == IO_ACIA + 1) {            /* ACIA data write -> tx       */
            if (txlog) { fputc(v, txlog); fflush(txlog); }
            return;
        }
        if (a == 0x7800) {                 /* MPU writes scan count to PIA-A */
            pia_a_scan = v;
        }
        mem[a] = v;                        /* latch control/bank/VTAC     */
        return;
    }
    if (is_rom[a]) return;                  /* writes to ROM ignored       */
    mem[a] = v;
}

static uint16_t rd16(uint16_t a)
{
    return (uint16_t)((rd8(a) << 8) | rd8((uint16_t)(a + 1)));
}

/* ---- fetch / stack ------------------------------------------------------ */
static uint8_t  fetch8(void)  { return rd8(PC++); }
static uint16_t fetch16(void) { uint16_t v = rd16(PC); PC += 2; return v; }

static void push8(uint8_t v)  { wr8(SP, v); SP = (uint16_t)(SP - 1); }
static uint8_t pull8(void)    { SP = (uint16_t)(SP + 1); return rd8(SP); }
static void push16(uint16_t v){ push8((uint8_t)(v & 0xFF)); push8((uint8_t)(v >> 8)); }
```

```
static uint16_t pull16(void)  { uint8_t hi = pull8(); uint8_t lo = pull8();
                                return (uint16_t)((hi << 8) | lo); }

/* ================================================================== */
/*  flag helpers                                                      */
/* ================================================================== */

static void setNZ8(uint8_t v)
{
    CC &= ~(FLAG_N | FLAG_Z);
    if (v & 0x80) CC |= FLAG_N;
    if (v == 0)   CC |= FLAG_Z;
}
static void setNZ16(uint16_t v)
{
    CC &= ~(FLAG_N | FLAG_Z);
    if (v & 0x8000) CC |= FLAG_N;
    if (v == 0)     CC |= FLAG_Z;
}

/* 8-bit add with optional carry-in; sets H,N,Z,V,C */
static uint8_t do_add(uint8_t a, uint8_t m, int carry)
{
    unsigned r = (unsigned)a + m + carry;
    unsigned h = (unsigned)(a & 0x0F) + (m & 0x0F) + carry;
    CC &= ~(FLAG_H | FLAG_N | FLAG_Z | FLAG_V | FLAG_C);
    if (h & 0x10) CC |= FLAG_H;
    if (r & 0x100) CC |= FLAG_C;
    if ((~(a ^ m) & (a ^ r) & 0x80)) CC |= FLAG_V;
    setNZ8((uint8_t)r);
    return (uint8_t)r;
}

/* 8-bit subtract with optional borrow-in; sets N,Z,V,C (H unaffected) */
static uint8_t do_sub(uint8_t a, uint8_t m, int borrow)
{
    unsigned r = (unsigned)a - m - borrow;
    CC &= ~(FLAG_N | FLAG_Z | FLAG_V | FLAG_C);
    if (r & 0x100) CC |= FLAG_C;                /* borrow                */
    if (((a ^ m) & (a ^ r) & 0x80)) CC |= FLAG_V;
    setNZ8((uint8_t)r);
    return (uint8_t)r;
}

/* ================================================================== */
/*  effective-address helpers                                         */
/* ================================================================== */
static uint16_t ea_dir(void) { return fetch8(); }
static uint16_t ea_ext(void) { return fetch16(); }
static uint16_t ea_idx(void) { return (uint16_t)(IX + fetch8()); }

/* read-modify-write helpers for shifts/rotates/inc/dec on memory or acc */
static uint8_t op_neg(uint8_t m)
{
    uint8_t r = (uint8_t)(0 - m);
    CC &= ~(FLAG_N | FLAG_Z | FLAG_V | FLAG_C);
    if (r != 0) CC |= FLAG_C;
    if (m == 0x80) CC |= FLAG_V;
    setNZ8(r);
    return r;
}
static uint8_t op_com(uint8_t m)
{
    uint8_t r = (uint8_t)~m;
    CC &= ~(FLAG_N | FLAG_Z | FLAG_V);
    CC |= FLAG_C;                               /* COM sets C=1          */
    setNZ8(r);
    return r;
}
static uint8_t op_inc(uint8_t m)
{
    uint8_t r = (uint8_t)(m + 1);
    CC &= ~(FLAG_N | FLAG_Z | FLAG_V);
    if (m == 0x7F) CC |= FLAG_V;
    setNZ8(r);
    return r;
}
static uint8_t op_dec(uint8_t m)
{
    uint8_t r = (uint8_t)(m - 1);
    CC &= ~(FLAG_N | FLAG_Z | FLAG_V);
    if (m == 0x80) CC |= FLAG_V;
    setNZ8(r);
    return r;
}
static void op_tst(uint8_t m)
{
    CC &= ~(FLAG_N | FLAG_Z | FLAG_V | FLAG_C);
    setNZ8(m);
```

```
}
static uint8_t op_clr(void)
{
    CC &= ~(FLAG_N | FLAG_V | FLAG_C);
    CC |= FLAG_Z;
    return 0;
}
static void setV_shift(uint8_t r)               /* V = N ^ C             */
{
    int n = (r & 0x80) ? 1 : 0;
    int c = (CC & FLAG_C) ? 1 : 0;
    if (n ^ c) CC |= FLAG_V; else CC &= ~FLAG_V;
}
static uint8_t op_asl(uint8_t m)
{
    uint8_t r = (uint8_t)(m << 1);
    CC &= ~(FLAG_C);
    if (m & 0x80) CC |= FLAG_C;
    setNZ8(r); setV_shift(r);
    return r;
}
static uint8_t op_asr(uint8_t m)
{
    uint8_t r = (uint8_t)((m >> 1) | (m & 0x80));
    CC &= ~(FLAG_C);
    if (m & 0x01) CC |= FLAG_C;
    setNZ8(r); setV_shift(r);
    return r;
}
static uint8_t op_lsr(uint8_t m)
{
    uint8_t r = (uint8_t)(m >> 1);
    CC &= ~(FLAG_C);
    if (m & 0x01) CC |= FLAG_C;
    setNZ8(r); setV_shift(r);
    return r;
}
static uint8_t op_rol(uint8_t m)
{
    int cin = (CC & FLAG_C) ? 1 : 0;
    uint8_t r = (uint8_t)((m << 1) | cin);
    CC &= ~(FLAG_C);
    if (m & 0x80) CC |= FLAG_C;
    setNZ8(r); setV_shift(r);
    return r;
}
static uint8_t op_ror(uint8_t m)
{
    int cin = (CC & FLAG_C) ? 0x80 : 0;
    uint8_t r = (uint8_t)((m >> 1) | cin);
    CC &= ~(FLAG_C);
    if (m & 0x01) CC |= FLAG_C;
    setNZ8(r); setV_shift(r);
    return r;
}
static void op_daa(void)
{
    uint8_t corr = 0;
    int setc = (CC & FLAG_C) ? 1 : 0;
    if ((A & 0x0F) > 9 || (CC & FLAG_H)) corr |= 0x06;
    if (A > 0x99 || (CC & FLAG_C))       { corr |= 0x60; setc = 1; }
    A = (uint8_t)(A + corr);
    CC &= ~(FLAG_C);
    if (setc) CC |= FLAG_C;
    setNZ8(A);
}

/* generic RMW dispatch on an effective address */
typedef uint8_t (*rmw_fn)(uint8_t);
static void rmw_mem(uint16_t ea, rmw_fn f) { wr8(ea, f(rd8(ea))); }

/* 16-bit compare (CPX): functional full-width form; sets N,Z,V (C left) */
static void op_cpx(uint16_t m)
{
    uint16_t x = IX;
    unsigned r = (unsigned)x - m;
    CC &= ~(FLAG_N | FLAG_Z | FLAG_V);
    if (r & 0x8000) CC |= FLAG_N;
    if ((r & 0xFFFF) == 0) CC |= FLAG_Z;
    if (((x ^ m) & (x ^ r) & 0x8000)) CC |= FLAG_V;
}

/* ---- interrupt entry (IRQ/SWI/NMI) ---------------------------------- */
static void do_interrupt(uint16_t vector)
{
    push16(PC);
    push16(IX);
    push8(A);
    push8(B);
```

```
    push8((uint8_t)(CC | CC_HI));
    CC |= FLAG_I;
    PC = rd16(vector);
}

/* periodic VSYNC IRQ: fire if enabled, interval elapsed, and I-flag clear */
static void maybe_irq(void)
{
    if (kbd_hold > 0) kbd_hold--;        /* time-based key auto-release      */
    if (!irq_enabled) return;
    if (++irq_accum >= irq_period) {
        irq_accum = 0;
        if (!(CC & FLAG_I)) do_interrupt(0xFFF8);   /* IRQ vector $FFF8    */
    }
}

/* ================================================================== */
/*  one instruction                                                   */
/* ================================================================== */
static const char *last_err = NULL;

static void step(void)
{
    uint8_t op = fetch8();
    uint16_t ea;
    uint8_t  m;

    switch (op) {

    /* ---- inherent ----------------------------------------------- */
    case 0x01: break;                                   /* NOP           */
    case 0x06: CC = (uint8_t)(A | CC_HI); break;        /* TAP           */
    case 0x07: A = (uint8_t)(CC | CC_HI); break;        /* TPA           */
    case 0x08: IX = (uint16_t)(IX + 1);                 /* INX           */
               CC &= ~FLAG_Z; if (IX == 0) CC |= FLAG_Z; break;
    case 0x09: IX = (uint16_t)(IX - 1);                 /* DEX           */
               CC &= ~FLAG_Z; if (IX == 0) CC |= FLAG_Z; break;
    case 0x0A: CC &= ~FLAG_V; break;                    /* CLV           */
    case 0x0B: CC |= FLAG_V; break;                     /* SEV           */
    case 0x0C: CC &= ~FLAG_C; break;                    /* CLC           */
    case 0x0D: CC |= FLAG_C; break;                     /* SEC           */
    case 0x0E: CC &= ~FLAG_I; break;                    /* CLI           */
    case 0x0F: CC |= FLAG_I; break;                     /* SEI           */
    case 0x10: A = do_sub(A, B, 0); break;              /* SBA           */
    case 0x11: do_sub(A, B, 0); break;                  /* CBA (no store)*/
    case 0x16: B = A; CC &= ~FLAG_V; setNZ8(B); break;  /* TAB           */
    case 0x17: A = B; CC &= ~FLAG_V; setNZ8(A); break;  /* TBA           */
    case 0x19: op_daa(); break;                         /* DAA           */
    case 0x1B: A = do_add(A, B, 0); break;              /* ABA           */

    case 0x30: IX = (uint16_t)(SP + 1); break;          /* TSX           */
    case 0x31: SP = (uint16_t)(SP + 1); break;          /* INS           */
    case 0x32: A = pull8(); break;                      /* PULA          */
    case 0x33: B = pull8(); break;                      /* PULB          */
    case 0x34: SP = (uint16_t)(SP - 1); break;          /* DES           */
    case 0x35: SP = (uint16_t)(IX - 1); break;          /* TXS           */
    case 0x36: push8(A); break;                         /* PSHA          */
    case 0x37: push8(B); break;                         /* PSHB          */

    case 0x39: PC = pull16(); break;                    /* RTS           */
    case 0x3B:                                           /* RTI           */
        CC = (uint8_t)(pull8() | CC_HI);
        B  = pull8();
        A  = pull8();
        IX = pull16();
        PC = pull16();
        break;
    case 0x3E: halted = 1; last_err = "WAI (waiting for interrupt)"; break; /* WAI */
    case 0x3F: do_interrupt(0xFFFA); break;             /* SWI           */

    /* accumulator A inherent ops */
    case 0x40: A = op_neg(A); break;                    /* NEGA          */
    case 0x43: A = op_com(A); break;                    /* COMA          */
    case 0x44: A = op_lsr(A); break;                    /* LSRA          */
    case 0x46: A = op_ror(A); break;                    /* RORA          */
    case 0x47: A = op_asr(A); break;                    /* ASRA          */
    case 0x48: A = op_asl(A); break;                    /* ASLA/LSLA     */
    case 0x49: A = op_rol(A); break;                    /* ROLA          */
    case 0x4A: A = op_dec(A); break;                    /* DECA          */
    case 0x4C: A = op_inc(A); break;                    /* INCA          */
    case 0x4D: op_tst(A); break;                        /* TSTA          */
    case 0x4F: A = op_clr(); break;                     /* CLRA          */

    /* accumulator B inherent ops */
    case 0x50: B = op_neg(B); break;                    /* NEGB          */
    case 0x53: B = op_com(B); break;                    /* COMB          */
    case 0x54: B = op_lsr(B); break;                    /* LSRB          */
    case 0x56: B = op_ror(B); break;                    /* RORB          */
    case 0x57: B = op_asr(B); break;                    /* ASRB          */
```

```
    case 0x58: B = op_asl(B); break;                    /* ASLB/LSLB     */
    case 0x59: B = op_rol(B); break;                    /* ROLB          */
    case 0x5A: B = op_dec(B); break;                    /* DECB          */
    case 0x5C: B = op_inc(B); break;                    /* INCB          */
    case 0x5D: op_tst(B); break;                        /* TSTB          */
    case 0x5F: B = op_clr(); break;                     /* CLRB          */

    /* indexed RMW */
    case 0x60: ea = ea_idx(); rmw_mem(ea, op_neg); break; /* NEG ,X      */
    case 0x63: ea = ea_idx(); rmw_mem(ea, op_com); break; /* COM ,X      */
    case 0x64: ea = ea_idx(); rmw_mem(ea, op_lsr); break; /* LSR ,X      */
    case 0x66: ea = ea_idx(); rmw_mem(ea, op_ror); break; /* ROR ,X      */
    case 0x67: ea = ea_idx(); rmw_mem(ea, op_asr); break; /* ASR ,X      */
    case 0x68: ea = ea_idx(); rmw_mem(ea, op_asl); break; /* ASL ,X      */
    case 0x69: ea = ea_idx(); rmw_mem(ea, op_rol); break; /* ROL ,X      */
    case 0x6A: ea = ea_idx(); rmw_mem(ea, op_dec); break; /* DEC ,X      */
    case 0x6C: ea = ea_idx(); rmw_mem(ea, op_inc); break; /* INC ,X      */
    case 0x6D: ea = ea_idx(); op_tst(rd8(ea)); break;     /* TST ,X      */
    case 0x6E: ea = ea_idx(); PC = ea; break;             /* JMP ,X      */
    case 0x6F: ea = ea_idx(); wr8(ea, op_clr()); break;   /* CLR ,X      */

    /* extended RMW */
    case 0x70: ea = ea_ext(); rmw_mem(ea, op_neg); break; /* NEG ext     */
    case 0x73: ea = ea_ext(); rmw_mem(ea, op_com); break; /* COM ext     */
    case 0x74: ea = ea_ext(); rmw_mem(ea, op_lsr); break; /* LSR ext     */
    case 0x76: ea = ea_ext(); rmw_mem(ea, op_ror); break; /* ROR ext     */
    case 0x77: ea = ea_ext(); rmw_mem(ea, op_asr); break; /* ASR ext     */
    case 0x78: ea = ea_ext(); rmw_mem(ea, op_asl); break; /* ASL ext     */
    case 0x79: ea = ea_ext(); rmw_mem(ea, op_rol); break; /* ROL ext     */
    case 0x7A: ea = ea_ext(); rmw_mem(ea, op_dec); break; /* DEC ext     */
    case 0x7C: ea = ea_ext(); rmw_mem(ea, op_inc); break; /* INC ext     */
    case 0x7D: ea = ea_ext(); op_tst(rd8(ea)); break;     /* TST ext     */
    case 0x7E: ea = ea_ext(); PC = ea; break;             /* JMP ext     */
    case 0x7F: ea = ea_ext(); wr8(ea, op_clr()); break;   /* CLR ext     */

    /* ---- branches (relative) ------------------------------------ */
#define BRANCH(cond) do { int8_t off = (int8_t)fetch8(); \
                          if (cond) PC = (uint16_t)(PC + off); } while (0)
    case 0x20: BRANCH(1); break;                        /* BRA           */
    case 0x22: BRANCH(!((CC&FLAG_C)||(CC&FLAG_Z))); break; /* BHI       */
    case 0x23: BRANCH(((CC&FLAG_C)||(CC&FLAG_Z))); break;  /* BLS       */
    case 0x24: BRANCH(!(CC&FLAG_C)); break;             /* BCC           */
    case 0x25: BRANCH((CC&FLAG_C)); break;              /* BCS           */
    case 0x26: BRANCH(!(CC&FLAG_Z)); break;             /* BNE           */
    case 0x27: BRANCH((CC&FLAG_Z)); break;              /* BEQ           */
    case 0x28: BRANCH(!(CC&FLAG_V)); break;             /* BVC           */
    case 0x29: BRANCH((CC&FLAG_V)); break;              /* BVS           */
    case 0x2A: BRANCH(!(CC&FLAG_N)); break;             /* BPL           */
    case 0x2B: BRANCH((CC&FLAG_N)); break;              /* BMI           */
    case 0x2C: { int n=(CC&FLAG_N)?1:0,v=(CC&FLAG_V)?1:0;
                 BRANCH(!(n^v)); } break;               /* BGE           */
    case 0x2D: { int n=(CC&FLAG_N)?1:0,v=(CC&FLAG_V)?1:0;
                 BRANCH((n^v)); } break;                /* BLT           */
    case 0x2E: { int n=(CC&FLAG_N)?1:0,v=(CC&FLAG_V)?1:0,z=(CC&FLAG_Z)?1:0;
                 BRANCH(!z && !(n^v)); } break;         /* BGT           */
    case 0x2F: { int n=(CC&FLAG_N)?1:0,v=(CC&FLAG_V)?1:0,z=(CC&FLAG_Z)?1:0;
                 BRANCH(z || (n^v)); } break;           /* BLE           */
    case 0x8D: { int8_t off = (int8_t)fetch8();         /* BSR           */
                 push16(PC); PC = (uint16_t)(PC + off); } break;

    /* ---- immediate / direct / indexed / extended data ops --------- */
    /* SUBA */
    case 0x80: A = do_sub(A, fetch8(), 0); break;
    case 0x90: A = do_sub(A, rd8(ea_dir()), 0); break;
    case 0xA0: A = do_sub(A, rd8(ea_idx()), 0); break;
    case 0xB0: A = do_sub(A, rd8(ea_ext()), 0); break;
    /* CMPA */
    case 0x81: do_sub(A, fetch8(), 0); break;
    case 0x91: do_sub(A, rd8(ea_dir()), 0); break;
    case 0xA1: do_sub(A, rd8(ea_idx()), 0); break;
    case 0xB1: do_sub(A, rd8(ea_ext()), 0); break;
    /* SBCA */
    case 0x82: A = do_sub(A, fetch8(), (CC&FLAG_C)?1:0); break;
    case 0x92: A = do_sub(A, rd8(ea_dir()), (CC&FLAG_C)?1:0); break;
    case 0xA2: A = do_sub(A, rd8(ea_idx()), (CC&FLAG_C)?1:0); break;
    case 0xB2: A = do_sub(A, rd8(ea_ext()), (CC&FLAG_C)?1:0); break;
    /* ANDA */
    case 0x84: A &= fetch8();        CC&=~FLAG_V; setNZ8(A); break;
    case 0x94: A &= rd8(ea_dir());   CC&=~FLAG_V; setNZ8(A); break;
    case 0xA4: A &= rd8(ea_idx());   CC&=~FLAG_V; setNZ8(A); break;
    case 0xB4: A &= rd8(ea_ext());   CC&=~FLAG_V; setNZ8(A); break;
    /* BITA */
    case 0x85: { uint8_t r=A&fetch8();       CC&=~FLAG_V; setNZ8(r);} break;
    case 0x95: { uint8_t r=A&rd8(ea_dir());  CC&=~FLAG_V; setNZ8(r);} break;
    case 0xA5: { uint8_t r=A&rd8(ea_idx());  CC&=~FLAG_V; setNZ8(r);} break;
    case 0xB5: { uint8_t r=A&rd8(ea_ext());  CC&=~FLAG_V; setNZ8(r);} break;
    /* LDAA */
    case 0x86: A = fetch8();        CC&=~FLAG_V; setNZ8(A); break;
    case 0x96: A = rd8(ea_dir());   CC&=~FLAG_V; setNZ8(A); break;
```

```
case 0xA6: A = rd8(ea_idx());   CC&=~FLAG_V; setNZ8(A); break;
case 0xB6: A = rd8(ea_ext());   CC&=~FLAG_V; setNZ8(A); break;
/* STAA */
case 0x97: ea=ea_dir(); wr8(ea,A); CC&=~FLAG_V; setNZ8(A); break;
case 0xA7: ea=ea_idx(); wr8(ea,A); CC&=~FLAG_V; setNZ8(A); break;
case 0xB7: ea=ea_ext(); wr8(ea,A); CC&=~FLAG_V; setNZ8(A); break;
/* EORA */
case 0x88: A ^= fetch8();        CC&=~FLAG_V; setNZ8(A); break;
case 0x98: A ^= rd8(ea_dir());   CC&=~FLAG_V; setNZ8(A); break;
case 0xA8: A ^= rd8(ea_idx());   CC&=~FLAG_V; setNZ8(A); break;
case 0xB8: A ^= rd8(ea_ext());   CC&=~FLAG_V; setNZ8(A); break;
/* ADCA */
case 0x89: A = do_add(A, fetch8(), (CC&FLAG_C)?1:0); break;
case 0x99: A = do_add(A, rd8(ea_dir()), (CC&FLAG_C)?1:0); break;
case 0xA9: A = do_add(A, rd8(ea_idx()), (CC&FLAG_C)?1:0); break;
case 0xB9: A = do_add(A, rd8(ea_ext()), (CC&FLAG_C)?1:0); break;
/* ORAA */
case 0x8A: A |= fetch8();        CC&=~FLAG_V; setNZ8(A); break;
case 0x9A: A |= rd8(ea_dir());   CC&=~FLAG_V; setNZ8(A); break;
case 0xAA: A |= rd8(ea_idx());   CC&=~FLAG_V; setNZ8(A); break;
case 0xBA: A |= rd8(ea_ext());   CC&=~FLAG_V; setNZ8(A); break;
/* ADDA */
case 0x8B: A = do_add(A, fetch8(), 0); break;
case 0x9B: A = do_add(A, rd8(ea_dir()), 0); break;
case 0xAB: A = do_add(A, rd8(ea_idx()), 0); break;
case 0xBB: A = do_add(A, rd8(ea_ext()), 0); break;
/* CPX (16-bit) */
case 0x8C: op_cpx(fetch16()); break;
case 0x9C: op_cpx(rd16(ea_dir())); break;
case 0xAC: op_cpx(rd16(ea_idx())); break;
case 0xBC: op_cpx(rd16(ea_ext())); break;
/* JSR */
case 0xAD: ea=ea_idx(); push16(PC); PC=ea; break;
case 0xBD: ea=ea_ext(); push16(PC); PC=ea; break;
/* LDS (16-bit) */
case 0x8E: SP = fetch16();       CC&=~FLAG_V; setNZ16(SP); break;
case 0x9E: SP = rd16(ea_dir());  CC&=~FLAG_V; setNZ16(SP); break;
case 0xAE: SP = rd16(ea_idx());  CC&=~FLAG_V; setNZ16(SP); break;
case 0xBE: SP = rd16(ea_ext());  CC&=~FLAG_V; setNZ16(SP); break;
/* STS (16-bit) */
case 0x9F: ea=ea_dir(); wr8(ea,(uint8_t)(SP>>8)); wr8((uint16_t)(ea+1),(uint8_t)SP);
           CC&=~FLAG_V; setNZ16(SP); break;
case 0xAF: ea=ea_idx(); wr8(ea,(uint8_t)(SP>>8)); wr8((uint16_t)(ea+1),(uint8_t)SP);
           CC&=~FLAG_V; setNZ16(SP); break;
case 0xBF: ea=ea_ext(); wr8(ea,(uint8_t)(SP>>8)); wr8((uint16_t)(ea+1),(uint8_t)SP);
           CC&=~FLAG_V; setNZ16(SP); break;

/* SUBB */
case 0xC0: B = do_sub(B, fetch8(), 0); break;
case 0xD0: B = do_sub(B, rd8(ea_dir()), 0); break;
case 0xE0: B = do_sub(B, rd8(ea_idx()), 0); break;
case 0xF0: B = do_sub(B, rd8(ea_ext()), 0); break;
/* CMPB */
case 0xC1: do_sub(B, fetch8(), 0); break;
case 0xD1: do_sub(B, rd8(ea_dir()), 0); break;
case 0xE1: do_sub(B, rd8(ea_idx()), 0); break;
case 0xF1: do_sub(B, rd8(ea_ext()), 0); break;
/* SBCB */
case 0xC2: B = do_sub(B, fetch8(), (CC&FLAG_C)?1:0); break;
case 0xD2: B = do_sub(B, rd8(ea_dir()), (CC&FLAG_C)?1:0); break;
case 0xE2: B = do_sub(B, rd8(ea_idx()), (CC&FLAG_C)?1:0); break;
case 0xF2: B = do_sub(B, rd8(ea_ext()), (CC&FLAG_C)?1:0); break;
/* ANDB */
case 0xC4: B &= fetch8();        CC&=~FLAG_V; setNZ8(B); break;
case 0xD4: B &= rd8(ea_dir());   CC&=~FLAG_V; setNZ8(B); break;
case 0xE4: B &= rd8(ea_idx());   CC&=~FLAG_V; setNZ8(B); break;
case 0xF4: B &= rd8(ea_ext());   CC&=~FLAG_V; setNZ8(B); break;
/* BITB */
case 0xC5: { uint8_t r=B&fetch8();       CC&=~FLAG_V; setNZ8(r);} break;
case 0xD5: { uint8_t r=B&rd8(ea_dir());  CC&=~FLAG_V; setNZ8(r);} break;
case 0xE5: { uint8_t r=B&rd8(ea_idx());  CC&=~FLAG_V; setNZ8(r);} break;
case 0xF5: { uint8_t r=B&rd8(ea_ext());  CC&=~FLAG_V; setNZ8(r);} break;
/* LDAB */
case 0xC6: B = fetch8();        CC&=~FLAG_V; setNZ8(B); break;
case 0xD6: B = rd8(ea_dir());   CC&=~FLAG_V; setNZ8(B); break;
case 0xE6: B = rd8(ea_idx());   CC&=~FLAG_V; setNZ8(B); break;
case 0xF6: B = rd8(ea_ext());   CC&=~FLAG_V; setNZ8(B); break;
/* STAB */
case 0xD7: ea=ea_dir(); wr8(ea,B); CC&=~FLAG_V; setNZ8(B); break;
case 0xE7: ea=ea_idx(); wr8(ea,B); CC&=~FLAG_V; setNZ8(B); break;
case 0xF7: ea=ea_ext(); wr8(ea,B); CC&=~FLAG_V; setNZ8(B); break;
/* EORB */
case 0xC8: B ^= fetch8();        CC&=~FLAG_V; setNZ8(B); break;
case 0xD8: B ^= rd8(ea_dir());   CC&=~FLAG_V; setNZ8(B); break;
case 0xE8: B ^= rd8(ea_idx());   CC&=~FLAG_V; setNZ8(B); break;
case 0xF8: B ^= rd8(ea_ext());   CC&=~FLAG_V; setNZ8(B); break;
/* ADCB */
case 0xC9: B = do_add(B, fetch8(), (CC&FLAG_C)?1:0); break;
case 0xD9: B = do_add(B, rd8(ea_dir()), (CC&FLAG_C)?1:0); break;
```

```
    case 0xE9: B = do_add(B, rd8(ea_idx()), (CC&FLAG_C)?1:0); break;
    case 0xF9: B = do_add(B, rd8(ea_ext()), (CC&FLAG_C)?1:0); break;
    /* ORAB */
    case 0xCA: B |= fetch8();        CC&=~FLAG_V; setNZ8(B); break;
    case 0xDA: B |= rd8(ea_dir());   CC&=~FLAG_V; setNZ8(B); break;
    case 0xEA: B |= rd8(ea_idx());   CC&=~FLAG_V; setNZ8(B); break;
    case 0xFA: B |= rd8(ea_ext());   CC&=~FLAG_V; setNZ8(B); break;
    /* ADDB */
    case 0xCB: B = do_add(B, fetch8(), 0); break;
    case 0xDB: B = do_add(B, rd8(ea_dir()), 0); break;
    case 0xEB: B = do_add(B, rd8(ea_idx()), 0); break;
    case 0xFB: B = do_add(B, rd8(ea_ext()), 0); break;
    /* LDX (16-bit) */
    case 0xCE: IX = fetch16();        CC&=~FLAG_V; setNZ16(IX); break;
    case 0xDE: IX = rd16(ea_dir());   CC&=~FLAG_V; setNZ16(IX); break;
    case 0xEE: IX = rd16(ea_idx());   CC&=~FLAG_V; setNZ16(IX); break;
    case 0xFE: IX = rd16(ea_ext());   CC&=~FLAG_V; setNZ16(IX); break;
    /* STX (16-bit) */
    case 0xDF: ea=ea_dir(); wr8(ea,(uint8_t)(IX>>8)); wr8((uint16_t)(ea+1),(uint8_t)IX);
               CC&=~FLAG_V; setNZ16(IX); break;
    case 0xEF: ea=ea_idx(); wr8(ea,(uint8_t)(IX>>8)); wr8((uint16_t)(ea+1),(uint8_t)IX);
               CC&=~FLAG_V; setNZ16(IX); break;
    case 0xFF: ea=ea_ext(); wr8(ea,(uint8_t)(IX>>8)); wr8((uint16_t)(ea+1),(uint8_t)IX);
               CC&=~FLAG_V; setNZ16(IX); break;

    /* --- undocumented MC6800 opcode $00 ---
       Real NMOS 6800 silicon executes $00 as a 1-byte no-op (it is NOT one of the
       lock-up/"HCF" codes). Shuttle Attack deliberately uses $00 as a 1-byte
       filler between SUBA #$E7 and TST $07,X at $E645-$E648, reached via the
       BEQ at $E63C. Treat it as NOP rather than halting. Other illegal opcodes
       still halt so they can be examined individually before being trusted. */
    case 0x00:
        break;

    default:
        halted = 1;
        last_err = "illegal/unimplemented opcode";
        halt_op = op;
        halt_pc = (uint16_t)(PC - 1);
        break;
    }

    (void)m;
    cycles++;
}

/* ========================================================================= */
/*  ROM / image loading                                                      */
/* ========================================================================= */
/* ---- ROM socket bookkeeping (for the board-layout selection screen) ---- */
#define NSOCKETS 4
static struct { const char *label; uint16_t base; const char *file; long size; }
sockets[NSOCKETS] = {
    { "U62", 0xE000, NULL, 0 },   /* ROM A  - plugin slot           */
    { "U63", 0xE800, NULL, 0 },   /* ROM B                          */
    { "U64", 0xF000, NULL, 0 },   /* ROM C                          */
    { "U65", 0xF800, NULL, 0 },   /* ROM D                          */
};
static void socket_record(uint16_t base, const char *file, long size)
{
    int i;
    for (i = 0; i < NSOCKETS; i++)
        if (sockets[i].base == base) { sockets[i].file = file; sockets[i].size = size; }
}
/* strip directory part of a path for compact display */
static const char *base_name(const char *p)
{
    const char *s = p, *r = p;
    if (!p) return "EMPTY";
    for (; *s; s++) if (*s == '/' || *s == '\\') r = s + 1;
    return r;
}

static int load_at(const char *path, uint16_t base, int mark_rom)
{
    FILE *f = fopen(path, "rb");
    long n;
    size_t got;
    if (!f) { fprintf(stderr, "cannot open %s\n", path); return -1; }
    fseek(f, 0, SEEK_END); n = ftell(f); fseek(f, 0, SEEK_SET);
    if (n <= 0 || base + n > 0x10000) { fprintf(stderr, "bad size %s\n", path); fclose(f); return -1; }
    {   /* fread() targets the near data segment, so stage through a small near
         * buffer and copy each chunk into the huge image one byte at a time. */
        static char buf[512];
        long off = 0; size_t r, k;
        while (off < n && (r = fread(buf, 1, sizeof(buf), f)) > 0) {
            for (k = 0; k < r; k++) mem[base + off + k] = (uint8_t)buf[k];
            off += (long)r;
        }
```

```
        got = (size_t)off;
    }
    fclose(f);
    if (mark_rom) { long i; for (i = 0; i < n; i++) is_rom[base + i] = 1; }
    socket_record(base, path, n);
    fprintf(stderr, "loaded %s -> $%04X (%ld bytes%s)\n",
            path, base, n, mark_rom ? ", ROM" : "");
    return (int)got;
}

static void reset_cpu(uint16_t pc, uint16_t sp)
{
    A = B = 0; IX = 0; CC = CC_HI | FLAG_I;
    SP = sp; PC = pc; halted = 0; cycles = 0;
}

/* ====================================================================== */
/*  headless self-test (no curses): used to verify the CPU core           */
/* ====================================================================== */


/* ====================================================================== */
/*  DOS front-end  (Turbo C++ 1.0)  --  direct video-RAM, conio keyboard   */
/* ====================================================================== */
#include <conio.h>
#include <dos.h>
#include <alloc.h>                      /* farmalloc / farfree              */

/* --- direct text-mode video memory ---------------------------------- */
static unsigned VSEG = 0xB800;          /* colour text page; 0xB000 if MDA */
static unsigned char SHADOW[25*80];     /* last glyph drawn per cell       */

static void vid_detect(void)
{
    /* BIOS video mode byte at 0040:0049; mode 7 == monochrome adapter */
    unsigned char far *modep = (unsigned char far *)MK_FP(0x0040, 0x0049);
    VSEG = (*modep == 7) ? 0xB000 : 0xB800;
}

/* put one cell straight into video RAM (no BIOS, no scroll = no flicker) */
static void vid_cell(int row, int col, unsigned char ch, unsigned char attr)
{
    unsigned char far *v = (unsigned char far *)MK_FP(VSEG, (row*80 + col) * 2);
    v[0] = ch;
    v[1] = attr;
}

static void vid_puts(int row, int col, unsigned char attr, const char *s)
{
    while (*s && col < 80) vid_cell(row, col++, (unsigned char)*s++, attr);
}

static void vid_clear(unsigned char attr)
{
    int i;
    unsigned char far *v = (unsigned char far *)MK_FP(VSEG, 0);
    for (i = 0; i < 25*80; i++) { v[i*2] = ' '; v[i*2+1] = attr; }
    for (i = 0; i < 25*80; i++) SHADOW[i] = 0xFF;   /* force full repaint   */
}

static void hide_cursor(void)
{
    union REGS r;
    r.h.ah = 0x01; r.h.ch = 0x20; r.h.cl = 0x00;    /* bit5 set = cursor off */
    int86(0x10, &r, &r);
}

/* --- CRTRAM -> screen, writing only the cells that changed ---------- */
#define ATTR_NORM  0x07                 /* light grey on black            */
#define ATTR_HI    0x0F                 /* bright (CRTRAM intensity bit)  */

static void render_crtram(void)
{
    int row, col, idx;
    for (row = 0; row < SCR_ROWS; row++) {
        for (col = 0; col < SCR_COLS; col++) {
            unsigned char raw = mem[0x8000 + row*80 + col];
            unsigned char g   = raw & 0x7F;
            if (g < 32 || g >= 127) g = ' ';
            idx = row*80 + col;
            if (SHADOW[idx] != g) {
                SHADOW[idx] = g;
                vid_cell(row, col, g, (raw & 0x80) ? ATTR_HI : ATTR_NORM);
            }
        }
    }
}

static void status_line(void)
```

```
{
    char buf[81];
    sprintf(buf, " PC=%04X A=%02X B=%02X X=%04X SP=%04X CC=%02X  [arrows move  ESC-q quit] ",
            PC, A, B, IX, SP, CC);
    vid_puts(SCR_ROWS, 0, 0x70, "                                                                                ");
    vid_puts(SCR_ROWS, 0, 0x70, buf);
}

/* ================================================================== */
/*  ROM board layout / selection screen                               */
/* ================================================================== */
static void draw_dip(int top, int left, int sel,
                     const char *label, uint16_t base, uint16_t end,
                     const char *file, long size)
{
    const char *name = base_name(file);
    int loaded = (file != 0);
    unsigned char a = sel ? ATTR_HI : ATTR_NORM;
    char nm[10], line[16];
    int i;

    if (loaded) { for (i=0;i<9 && name[i] && name[i]!='.';i++) nm[i]=name[i]; nm[i]=0; }
    else        { strcpy(nm, "(empty)"); }

    vid_puts(top+0,  left, a, "  ___||___  ");
    vid_puts(top+1,  left, a, " |   ..   | ");
    vid_puts(top+2,  left, a, "-|        |-");
    sprintf(line, "-| %-6.6s |-", loaded ? nm : "      "); vid_puts(top+3, left, a, line);
    vid_puts(top+4,  left, a, "-|        |-");
    sprintf(line, "-| %04X   |-", base); vid_puts(top+5, left, a, line);
    sprintf(line, "-| %04X   |-", end ); vid_puts(top+6, left, a, line);
    vid_puts(top+7,  left, a, "-|        |-");
    if (loaded) { sprintf(line, "-| %4ldB  |-", size); vid_puts(top+8, left, a, line); }
    else        { vid_puts(top+8, left, a, "-|(empty) |-"); }
    vid_puts(top+9,  left, a, "-|        |-");
    vid_puts(top+10, left, a, " '--------' ");
    sprintf(line, "    %-4s", label); vid_puts(top+11, left, a, line);
    if (sel) vid_puts(top+12, left, a, "   >PLUGIN<");
}

static void rom_board_screen(const char *boot)
{
    int left0 = 4, top = 2, gap = 16, i, ch;
    static const char *rolelbl[NSOCKETS] = {
        "ROM A (plugin)", "ROM B", "ROM C", "ROM D"
    };
    static uint16_t ends[NSOCKETS] = { 0xE7FF, 0xEFFF, 0xF7FF, 0xFFFF };
    char line[81];

    vid_clear(ATTR_NORM);
    vid_puts(0, 4, ATTR_HI, "LEAR SIEGLER ADM-31   ::   ROM BOARD CONFIGURATION");

    for (i = 0; i < NSOCKETS; i++)
        draw_dip(top, left0 + i*gap, (i == 0),
                 sockets[i].label, sockets[i].base, ends[i],
                 sockets[i].file, sockets[i].size);

    for (i = 0; i < NSOCKETS; i++) {
        sprintf(line, "%-4s  $%04X-$%04X  %-14s  %s",
                sockets[i].label, sockets[i].base, ends[i], rolelbl[i],
                sockets[i].file ? base_name(sockets[i].file) : "-- empty --");
        vid_puts(top+14+i, 4, ATTR_NORM, line);
    }

    sprintf(line, "boot mode: %s", boot);
    vid_puts(top+19, 4, ATTR_NORM, line);
    vid_puts(top+21, 4, 0x70, " ENTER = boot terminal    Q = quit ");

    for (;;) {
        ch = getch();
        if (ch == '\r' || ch == '\n') break;
        if (ch == 'q' || ch == 'Q') { hide_cursor(); textmode(C80); exit(0); }
    }
    vid_clear(ATTR_NORM);
}

static void show_cursor(void)
{
    union REGS r;
    r.h.ah = 0x01; r.h.ch = 0x06; r.h.cl = 0x07;    /* restore normal underline cursor */
    int86(0x10, &r, &r);
}

/* ================================================================== */
/*  main  (DOS)                                                       */
/* ================================================================== */
int main(int argc, char **argv)
{
    int i, n, ch, sc, c2;
```

```
const char *boot = "firmware";
uint16_t plugin_org = 0xE000;
int have_any = 0;

/* allocate 64K + a paragraph of slack, then align base to offset 0 */
mem_raw   = farmalloc(0x10000UL + 16UL);
isrom_raw = farmalloc(0x10000UL + 16UL);
if (!mem_raw || !isrom_raw) { printf("out of memory\n"); return 1; }
mem    = (uint8_t far *)MK_FP(FP_SEG(mem_raw)   + ((FP_OFF(mem_raw)   + 15) >> 4), 0);
is_rom = (uint8_t far *)MK_FP(FP_SEG(isrom_raw) + ((FP_OFF(isrom_raw) + 15) >> 4), 0);
{ long k; for (k = 0; k < 0x10000L; k++) { mem[k] = 0; is_rom[k] = 0; } }
txlog = fopen("adm31tx.log", "wb");

for (i = 1; i < argc; i++) {
    if      (!strcmp(argv[i],"--romA")  && i+1<argc) { load_at(argv[++i],0xE000,1); have_any=1; }
    else if (!strcmp(argv[i],"--plugin")&& i+1<argc) { load_at(argv[++i],0xE000,1); have_any=1; }
    else if (!strcmp(argv[i],"--romB")  && i+1<argc) { load_at(argv[++i],0xE800,1); have_any=1; }
    else if (!strcmp(argv[i],"--romC")  && i+1<argc) { load_at(argv[++i],0xF000,1); have_any=1; }
    else if (!strcmp(argv[i],"--romD")  && i+1<argc) { load_at(argv[++i],0xF800,1); have_any=1; }
    else if (!strcmp(argv[i],"--boot")  && i+1<argc) { boot = argv[++i]; }
    else if (!strcmp(argv[i],"--stock")) {
        /* the fixed 129867 firmware set (ROM67/66/65.bin -> ROM B/C/D)
           loaded from files of these exact names in the current
           directory - so only --romA and --boot need specifying. */
        load_at("ROM67.BIN", 0xE800, 1);
        load_at("ROM66.BIN", 0xF000, 1);
        load_at("ROM65.BIN", 0xF800, 1);
        have_any = 1;
    }
    else if (!strcmp(argv[i],"--rom")   && i+1<argc) {
        char *spec = argv[++i]; char *at = strchr(spec,'@');
        if (at) { *at = 0; load_at(spec,(uint16_t)strtol(at+1,0,16),1); have_any=1; }
    }
}
if (!have_any) {
    printf("usage: %s --romA plugin.bin --romB rom67.bin --romC rom65.bin --romD rom66.bin [--boot firmware|plugin|game]\n", argv[0]);
    printf("   or: %s --stock --romA plugin.bin --boot game   (loads ROM67/66/65.BIN automatically)\n", argv[0]);
    return 2;
}

if (!strcmp(boot,"plugin")) reset_cpu(plugin_org, 0x003F);
else { reset_cpu(rd16(0xFFFE), 0x003F); irq_enabled = 1; }

vid_detect();
textmode(C80);
hide_cursor();
vid_clear(ATTR_NORM);

rom_board_screen(boot);                    /* config graphic FIRST: confirm */

if (!strcmp(boot,"game")) {                 /* now do the slow firmware boot */
    unsigned long b;
    vid_puts(12, 30, ATTR_HI, " BOOTING FIRMWARE... ");
    for (b = 0; b < 400000UL && !halted; b++) { maybe_irq(); step(); }
    SP = 0x003F;
    mem[SP--] = 0xD0; mem[SP--] = 0xE8;    /* fake return -> firmware $E8D0 */
    PC = 0xE03C;                           /* generic plugin launch vector  */
    vid_clear(ATTR_NORM);
}
render_crtram();

for (;;) {
    for (n = 0; n < 5000 && !halted; n++) { maybe_irq(); step(); }

    while (kbhit()) {
        ch = getch();
        if (ch == 0 || ch == 0xE0) {       /* extended key: read scan code  */
            sc = getch();
            switch (sc) {
                case 72: kbd_press_scan(ADM_SCAN_UP);    break;  /* up    */
                case 80: kbd_press_scan(ADM_SCAN_DOWN);  break;  /* down  */
                case 75: kbd_press_scan(ADM_SCAN_LEFT);  break;  /* left  */
                case 77: kbd_press_scan(ADM_SCAN_RIGHT); break;  /* right */
                default: break;
            }
        } else if (ch == 27) {             /* ESC: ESC-q quits, else matrix */
            if (kbhit()) {
                c2 = getch();
                if (c2 == 'q' || c2 == 'Q') goto done;
                kbd_press_ascii(27);
                render_crtram(); delay(30);
                if (c2) kbd_press_ascii((unsigned char)c2);
            } else {
                kbd_press_ascii(27);
            }
        } else if (ch == '\r' || ch == '\n') {
            kbd_press_numpad_enter();      /* Return -> numpad ENTER ($4F)  */
        } else if (ch == 8 || ch == 127) {
            kbd_press_ascii(0x08);         /* Backspace                     */
```

```
            } else if (ch == ' ') {
                kbd_press_scan(ADM_SCAN_FIRE);  /* Space -> fire (scan $40)      */
            } else {
                kbd_press_ascii((unsigned char)ch);
            }
        }

        render_crtram();
        status_line();

        if (halted) {
            char msg[81];
            sprintf(msg, " CPU HALTED: %s ($%02X at $%04X)   press ESC-q ",
                    last_err ? last_err : "?", halt_op, halt_pc);
            vid_puts(SCR_ROWS, 0, 0x70, "                                                                                ");
            vid_puts(SCR_ROWS, 0, 0x4F, msg);
            for (;;) {                          /* wait for ESC-q               */
                ch = getch();
                if (ch == 27) {                 /* ESC, then q to quit           */
                    c2 = getch();
                    if (c2 == 'q' || c2 == 'Q') break;
                }
            }
            break;
        }
    }

done:
    textmode(C80);
    show_cursor();
    clrscr();
    if (mem_raw)   farfree(mem_raw);
    if (isrom_raw) farfree(isrom_raw);
    return 0;
}
```


EVERY BYTE OF THE ADM-31, FROM RESET TO THE LAST ESCAPE SEQUENCE, EXPLAINED FROM THE FIRST LDAA TO A COMPLETE GAME OF PONG.
REAL ASSEMBLER LISTINGS, THE FULL FIRMWARE, AND THE ASSEMBLER ITSELF.
WRITTEN BY
MICKEY W. LAWLESS_